

aper = 0.9; sweep = 0.7; dihamp = 0.02

Second Edition

ENGINEERING COMPUTATION

with **MATLAB®**

```
DMS
ymax = 10;
th = linspace(0, 2*pi, 40);
yl = linspace(1, ymax, 40);
r = 1.5; g = 1;
taper = 0.9; sweep = 0.7; dihamp = 0.02; center = complex( sqrt(r^2-g^2)
- 1, g ); [yy th] = meshgrid( y, th); z = +.* complex( cos(th),
sin(th) ) + center; ww = z + 1./z; xx = real(ww) .* (ymax - taper*yy) /
ymax - sweep * yy; alf = 0; bet = 0; lim = zeros(3,2); figure('Color'
[0, 0.5, 0.8]) while alf < 2*pi
    tw = yy * bet / (4 * ymax);
    ct = cos(tw); st = sin(tw);
    dihed = dihamp * sin(alf);
    zz = imag(ww) .* (ymax - taper*yy) / ymax + dihed * yy.^2;
    xxa = xx .* ct - zz .* st;
    zza = xx .* st + zz .* ct;
    surf( xxa, yy, 2*zza, zz )
    axis([-7.2037 1.1440 0 10.0000 -1 4 ]); axis off
    alpha(0.05); shading interp; hold on
    lightangle( 45, 45); view(0, 15)
    pause( 0.03 )
    alf = alf + .02; bet = alf;
    if bet > pi
        bet = bet - 2*pi;
    elseif bet > pi/2
        bet = pi - bet;
    end
    if bet < -pi/2
        bet = -pi - bet;
    end
end
```

DAVID M. SMITH

ENGINEERING COMPUTATION

with ***MATLAB***[®]

Second Edition

ENGINEERING COMPUTATION

with ***MATLAB***[®]

Second Edition

DAVID M. SMITH
Georgia Institute of Technology

Addison-Wesley

Boston San Francisco New York
London Toronto Sydney Tokyo Singapore Madrid
Mexico City Munich Paris Cape Town Hong Kong Montreal

Executive Editor
Editorial Assistant
Managing Editor
Production Supervisor
Composition
Art Director
Cover Designer
Online Product Manager
Marketing Manager
Marketing Coordinator
Senior Manufacturing Buyer
Illustrations
Production Coordination
Proofreading, Copyediting
Indexing
Text Design

Michael Hirsch
Stephanie Sellinger
Jeffrey Holcomb
Heather McNally
Aptara[®], Inc.
Linda Knowles
Suzanne Harbison
Bethany Tidd
Erin Davis
Kathryn Ferranti
Carol Melville
Craig Cantwell
Dennis Free, Aptara[®], Inc.
Write With, Inc.
WordCo Indexing Services, Inc.
Susan Carsten Raymond

Cover Image: The image on the cover was created from a three-dimensional model of a typical swept wing. The image was then subjected to bending forces, and the plots of resulting shapes overlaid with a very low transparency. The ribbed effect inside the wing's leading edge is an interesting artifact of the overlaying process.

Cover Art: © David M. Smith

Photo Credits: Figure 1.2 *The Colossus computer* on page 4 is provided courtesy of the United Kingdom Government. Figure 2.10 *Space Ship One* on page 39 is © AP Wideworld Photos. Figure 3.5 *Conveyer moving dirt* on page 74 is provided courtesy of Hartsfield-Jackson International Airport. The photograph of a nautilus shell in Figure 9.3 *Fibonacci in nature* on page 208 is © PhotoDisc. Figure 14.13 *Oil platform* on page 372 is provided courtesy of NASA, U.S. Federal Government. Figure 14.14 *Model diagram* and Figure 14.15 *Analysis results* on page 373 are provided courtesy of Aerospace Report ATR-82(2830)-IND written in 1981 for the U.S. Department of the Interior, Conservation Division, Geological Survey, Reston, VA 22086. Figure 17.4 *Map of the London Underground* on page 17-9 is provided courtesy of Transport for London (<http://www.tfl.gov.uk/tube/>).

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and Addison-Wesley was aware of a trademark claim, the designations have been printed in initial caps or all caps.

This interior of this book was composed in QuarkXpress 6.5.

Library of Congress Cataloging-in-Publication Data

Smith, David M., 1942-

Engineering computation with MATLAB / David M. Smith. -- 2nd ed.
p. cm.

Includes index.

ISBN-13: 978-0-13-608063-3

ISBN-10: 0-13-608063-4

1. Engineering mathematics--Data processing. 2. MATLAB. I. Title.

TA345.S585 2008b

620.001'51--dc22

2008048810

Copyright ©2010 Pearson Education, Inc. All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher. Printed in the United States of America. For information on obtaining permission for use of material in this work, please submit a written request to Pearson Education, Inc., Rights and Contracts Department, 501 Boylston Street, Suite 900 Boston, MA 02116, fax your request to (617)671-3447, or e-mail at <http://www.pearsoned.com/legal/permissions.htm>.

Addison-Wesley
is an imprint of



www.pearsonhighered.com

ISBN-13: 978-0-13-608063-3

ISBN-10: 0-13-608063-4

1 2 3 4 5 6 7 8 9 10—DOH—13 12 11 10 09

*This book is dedicated to the
glory of Almighty God*

~David M. Smith



Contents

Chapter 1	Introduction to Computers and Programming	I
1.1	Background	2
1.2	History of Computer Architectures	3
1.3	Computing Systems Today	5
1.4	Executing a MATLAB Program	13
1.5	Problem Solving	13
Chapter 2	Getting Started with MATLAB	17
2.1	Programming Language Background	19
2.2	Basic Data Manipulation	21
2.3	The MATLAB User Interface	25
2.4	Scripts	35
2.5	Engineering Example—Spacecraft Launch	39
Chapter 3	Vectors and Arrays	47
3.1	Concept: Using Built-in Functions	48
3.2	Concept: Data Collections	48
3.3	MATLAB Vectors	48
3.4	Engineering Example—Forces and Moments	61
3.5	MATLAB Arrays	62
3.6	Engineering Example—Computing Soil Volume	74
Chapter 4	Execution Control	85
4.1	Concept: Code Blocks	86
4.2	Conditional Execution in General	86
4.3	if Statements	87
4.4	switch Statements	92
4.5	Iteration in General	94
4.6	for Loops	95
4.7	while Loops	98
4.8	Engineering Example—Computing Liquid Levels	101
Chapter 5	Functions	111
5.1	Concepts: Abstraction and Encapsulation	112
5.2	Black Box View of a Function	112
5.3	MATLAB Implementation	113
5.4	Engineering Example—Measuring a Solid Object	119
Chapter 6	Character Strings	127
6.1	Character String Concepts: Mapping Casting, Tokens and Delimiting	128
6.2	MATLAB Implementation	129

6.3	Format Conversion Functions	131
6.4	Character String Operations	134
6.5	Arrays of Strings	137
6.6	Engineering Example—Encryption	138
Chapter 7	Cell Arrays and Structures	147
7.1	Concept: Collecting Dissimilar Objects	148
7.2	Cell Arrays	148
7.3	MATLAB Structures	153
7.4	Structure Arrays	156
7.5	Engineering Example—Assembling a Structure	162
Chapter 8	File Input and Output	173
8.1	Concept: Serial Input and Output (I/O)	174
8.2	MATLAB Workspace I/O	174
8.3	High-Level I/O Functions	175
8.4	Lower-Level File I/O	180
8.5	Engineering Example—Spreadsheet Data	184
Chapter 9	Recursion	191
9.1	Concept: The Activation Stack	192
9.2	Recursion Defined	193
9.3	Implementing a Recursive Function in MATLAB	194
9.4	Exceptions	196
9.5	Wrapper Functions	200
9.6	Tail Recursion	203
9.7	Mutual Recursion	205
9.8	Generative Recursion	205
9.9	Examples of Recursion	205
9.10	Engineering Example—Robot Arm Motion	211
Chapter 10	Principles of Problem Solving	219
10.1	Solving Simple Problems	220
10.2	Assembling Solution Steps	220
10.3	Summary of Operations	220
10.4	Solving Larger Problems	236
10.5	Engineering Example—Processing Geopolitical Data	238
Chapter 11	Plotting	247
11.1	Plotting in General	248
11.2	2-D Plotting	252
11.3	3-D Plotting	258
11.4	Surface Plots	261
11.5	Interacting with Plotted Data	279
11.6	Engineering Example—Visualizing Geographic Data	283
Chapter 12	Matrices	293
12.1	Concept: Behavioral Abstraction	294
12.2	Matrix Operations	294
12.3	MATLAB Implementation	297
12.4	Rotating Coordinates	300

12.5	Solving Simultaneous Linear Equations	307
12.6	Engineering Examples	311
Chapter 13	Images	319
13.1	Nature of an Image	320
13.2	Image Types	321
13.3	Reading, Displaying, and Writing Images	323
13.4	Operating on Images	323
13.5	Engineering Example—Detecting Edges	339
Chapter 14	Processing Sound	347
14.1	The Physics of Sound	348
14.2	Recording and Playback	348
14.3	MATLAB Implementation	349
14.4	Time Domain Operations	350
14.5	The Fast Fourier Transform	359
14.6	Frequency Domain Operations	364
14.7	Engineering Example—Oil Rig Structural Integrity	371
Chapter 15	Numerical Methods	379
15.1	Interpolation	380
15.2	Curve Fitting	384
15.3	Numerical Integration	390
15.4	Numerical Differentiation	394
15.5	Engineering Example—Analyzing Rocket Data	397
15.6	Engineering Example—Cleaning up Images	402
Chapter 16	Sorting	411
16.1	Measuring Algorithm Cost	412
16.2	Algorithms for Sorting Data	415
16.3	Performance Analysis	425
16.4	Applications of Sorting Algorithms	426
16.5	Engineering Example—A Selection of Countries	430
Chapter 17	Searching Graphs	(online)
Chapter 18	Object-Oriented Programming	(online)
Chapter 19	Linked Lists	(online)
Chapter 20	Binary Trees	(online)
Chapter 21	N-ary Trees and Graphs	(online)
Chapter 22	The Cost of Computing	(online)
Appendices		
Appendix A	MATLAB Special Characters, Reserved Words, and Symbols	A–I
Appendix B	The ASCII Character Set	B–I
Appendix C	Internal Number Representation	C–I
Appendix D	Web Reference Materials	(online)
Appendix E	Answers to True or False and Fill in the Blanks	(online)
Index	I–I	

About the Author

David Smith has been teaching introductory computer science classes for engineers at the Georgia Institute of Technology since 1997 when he retired from industry. Previously, he worked 31 years for Lockheed-Martin at their Marietta, Georgia facility as a systems and software specialist with a focus on intelligent systems. He was active in designing and developing software for the C-130J, C-27J, F-22, and C-5 aircraft and was the technical leader of the Pilot's Associate program, a \$42 million research project sponsored by the Defense Advanced Research Projects Agency.

Mr. Smith has a bachelor's degree in aeronautical engineering from Southampton University and a master's degree in control systems from Imperial College, London.

The top of the page features a background image. On the left, there is a snippet of MATLAB code in a light gray font. On the right, the MATLAB logo is partially visible, showing the characteristic 'M' shape. The title 'Preface' is centered in a large, bold, black font.

Preface

“That of all the several ways of beginning a book which are now in practice throughout the known world, I am confident my own way of doing it is the best—I’m sure it is the most religious—for I begin with writing the first sentence, and trusting to Almighty God for the second.”

Laurence Sterne (1713–1768), British author, clergyman

This book introduces the power, satisfaction, and joy of computing to beginning engineering students who have little or no previous computing experience. It began as a snapshot of the content of a Georgia Tech course that introduces engineers to computing. However, it has been extensively enhanced to meet the needs of a wider audience of students and educators who want to understand programming for other reasons. In this book, to understand computing, we use MATLAB, a user-friendly language that is emerging as one of the most popular computing languages in engineering.

Since the 1st edition of this book, the release of MATLAB 7.6 (R2008a) has significant changes in the use of the product. The following changes have been incorporated into the new release:

- You can now customize the tool bars on the desktop and editor windows to reorganize your tools and de-clutter the windows by removing unused links.
- In the Editor Window, many code blocks can be collapsed to allow easier visibility into the overall structure of scripts and functions.
- The implementation of object-oriented programming (OOP) has been completely revised to be more compatible with conventional OOP languages.
- Two tools have been added to plots to enable interaction with the underlying data: Brushing allows you to select specific values on a plot and manipulate those values to change the behavior of the plot; linking allows you to link the data presented in a plot to the underlying data so that when you make changes to a plot, the underlying data are updated.
- A Variable Editor has been added to replace the old Array Editor, expanding the editing ability from arrays to include structures and objects.

The text of this book has been updated as appropriate to reflect these and other detailed changes. The examples and end-of-chapter programming projects have also been extensively revised and improved.

Pedagogical Style

Computing is not a spectator sport. Students learn computing by computing. This text not only presents computing concepts and their MATLAB implementation, but also offers students extensive hands-on exercises. The text illustrates the ideas with examples from the world of engineering, provides style points, and presents sample problems that students might encounter.

Each chapter includes topics that go a step beyond the basic content of an introductory class. This gives professors the choice to progress slowly, and more thoroughly, through the material in two semesters. It also offers advanced students enrichment materials for their personal study.

The overall philosophy of this text approaches programming tools in the following manner:

1. Explain a computing concept in general
2. Discuss its implementation in MATLAB
3. Provide exercises to master the concept

To help facilitate students' understanding of the concept and its implementation, the text uses two features: general templates and MATLAB listings. The general templates provide a foundation for students to understand concepts in general and can be applied to any language. The MATLAB listings show students how to implement concepts in MATLAB and are followed by detailed explanations of the code.

Features of the Text

- **Exercises:** Allow students a “Do It Yourself” approach to master concepts by trying what they just learned. Exercises follow each new topic.
- **Style Points:** Advise students about writing quality code that is easy to understand, debug, and reuse.
- **Hints:** Enrich students' understanding of a topic. Hints are interspersed through the book at points where students may benefit from a little extra “aside.”
- **Engineering Examples:** Provide robust models and apply to real-world issues that will motivate students. Examples from different engineering disciplines are presented at the end of each chapter.
- **Special Characters, Reserved Words, and Functions:** Provides a quick reference for the key MATLAB principles discussed in each chapter.

- **Self Test:** Helps students to check their understanding of the material in each chapter.
- **Programming Projects:** Offer a variety of large-scale projects that students can work on to solidify their skills.

Chapter Overview

Chapter 1: *Introduction to Computers and Programming* discusses the history of computer architectures as they apply to computing systems today. The chapter provides an overview of computer hardware and software and how programs execute.

Chapter 2: *Getting Started with MATLAB* discusses some basic concepts of computing and then introduces the basic operation of the MATLAB user interface. The chapter also describes how to capture simple MATLAB programs in the form of a script.

Chapter 3: *Vectors and Arrays* introduces the fundamental machinery that sets MATLAB apart from other languages—its ability to perform mathematical and logical operations on homogeneous collections of numbers.

Chapter 4: *Execution Control* describes the common techniques used to control the execution of code blocks—conditional operation and iteration.

Chapter 5: *Functions* describes how to implement procedural abstraction by defining reusable code blocks.

Chapter 6: *Character Strings* discusses how MATLAB operates on variables containing text.

Chapter 7: *Cell Arrays and Structures* discusses two kinds of heterogeneous data collections accessed by index and by name.

Chapter 8: *File Input and Output* describes three levels of ability provided in MATLAB for transferring data to and from data files—saving workspaces, specific tools that read and write specific data files, and general-purpose tools for processing any kind of file.

Chapter 9: *Recursion* discusses and illustrates a widely used alternative approach to repetitive code block execution.

Chapter 10: *Principles of Problem Solving* introduces ideas that help students design solutions to new problems and avoid the “blank sheet of paper” problem—how to start.

Chapter 11: *Plotting* takes the student from basic plotting in two dimensions to the advanced tools that draw representations of three-dimensional objects with smooth shading and even multiple light effects.

Chapter 12: *Matrices* describes specific MATLAB capabilities that implement matrix algebra.

Chapter 13: *Images* discusses how to use vector and array algebra to manipulate color pictures.

Chapter 14: *Processing Sound* shows how to analyze, synthesize, and operate on sound files.

Chapter 15: *Numerical Methods* introduces numerical techniques that commonly occur in engineering: interpolation, curve fitting, integration, and differentiation.

Chapter 16: *Sorting* presents five algorithms for ordering data, each of which has applicability under certain circumstances—Insertion Sort, Bubble Sort, Quick Sort, Merge Sort, and Bucket Sort—and then compares their performance on large quantities of data.

Appendices provide a summary of the MATLAB special characters, reserved words, and functions used throughout the text, the ASCII character set, the internal number representation inside the computer, Web reference materials, and answers to the True or False and Fill in the Blanks questions.

Paths through the Book

Not all courses that cover programming and MATLAB follow the same syllabus. *Engineering Computation with MATLAB* is designed to facilitate teaching the material with different styles and at different speeds. For example, Chapters 3, 4, and 5 cover MATLAB array manipulation, iteration, and writing your own functions. There are three schools of thought about the appropriate way to introduce these concepts. One would introduce array constructs first and follow up with the more “traditional” concept of iteration; another would teach iteration first and deal with the MATLAB-specific array operations later; and the third would treat functions first. I chose to order the book according to the arrays-first approach, to suit a particular teaching style. However, should you prefer iteration or functions first, Chapters 3, 4, and 5 can be used in any order you wish.

I have attempted numerous approaches to the sequence of lessons covering Chapters 1–9. The order in which these chapters appear has been the most intuitive for me as an instructor and, by and large, for the students. Chapters 10–16 are virtually independent and can be taught in any order.

Supplements

Various supplemental materials for this text are available at the book’s Companion Website: www.aw.com/smith_mat. The following are accessible to all readers (register using the code in the front of this book):

- Solutions to selected Programming Projects
- Selected full-color figures
- Source code for all MATLAB listings

- Bonus chapters including: Searching Graphs, Object-Oriented Programming, Linked Lists, Binary Trees, N-ary Trees and Graphs, and the Cost of Computing
- Bonus appendices including: Web Reference Materials and Answers to True or False and Fill in the Blanks

In addition, the following supplements are available to qualified instructors at Addison-Wesley's Instructor Resource Center. Please visit www.pearsonhighered.com/irc, or send e-mail to computing@aw.com.

- Source Code
- Solutions to all of the Programming Projects
- PowerPoint lecture slides

Acknowledgments

The underlying philosophy of this book and the material that forms its skeleton originated in the work of Professor Russell Shackelford around 1996.

Dr. Melody Moore, currently an assistant professor in the Computer Information Systems department of the College of Business Administration at Georgia State University, was instrumental in creating many of the teaching materials (then as overhead transparencies) from which this class was first taught.

I am deeply indebted to Professor James Craig from the Aerospace Engineering department at Georgia Tech, who joined me in co-teaching the first engineering version of CS1, taught me much about MATLAB, and pioneered this class from the original 35 students to its current size of over 1,000 engineering students per semester. This engineering class became a vessel for introducing the students to the MATLAB language.

I would like to thank the following reviewers for their insight and wisdom during the process of manuscript development:

Kenneth Rouse, *Auburn University*

Suparna Datta, *Northeastern University*

Gerardine G. Botte, *Ohio University*

Mica Grujicic, *Clemson University*

Kuldip S. Rattan, *Wright State University*

Y.J. Lin, *The University of Akron*

Mark Nagurka, *Marquette University*

Michael Peshkin, *Northwestern University*

Howard Silver, *Fairleigh Dickinson University*

Steve Swinnea, *The University of Texas at Austin*

The material has benefited from the efforts of every Georgia Tech teaching assistant (TA), graduate student, instructor, and professor who has taught

CS1, a list too long to enumerate. In particular, those wonderfully creative TAs who developed the ideas for examples used in this text have enriched it immeasurably. I wish to credit Professor Aaron Bobick with an important contribution made in the course of one short conversation. That conversation was responsible for pulling the class back from the brink of being merely a MATLAB programming class to one with roots in CS concepts. Professor Bobick taught CS1 with me in the fall of 2004. Early in the semester he made a very simple request: He said it would be easier for him to teach the class if we explicitly expressed the computing concepts inherent in each lesson, rather than leaving him—and the students—to tease the concepts out of the teaching materials.

I cannot adequately express my appreciation for the team at Addison-Wesley that helped bring this book to fruition. Many of them have done, and I am sure continue to do, their work “behind the scenes”: Michael Hirsch, Stephanie Sellinger, Erin Davis, Heather McNally, and Jeff Holcomb. I would like to thank Greg Cantwell for creating the illustrations. I have developed a personal appreciation for three people on this team: Kathy Cantwell, my copyeditor for the first edition, who has been so talented, patient, and ready to work with material that must seem to her to be in a foreign language; Linda Knowles, who did an incredible job transforming my vague idea into a stunning cover design; and Marika Sharpe, Addison-Wesley’s representative at Georgia Tech, who finally convinced me to write this book and continues to support and encourage me. I also really appreciate the work of Dennis Free and his team at Aptara[®], Inc. for the care with which they designed the second edition and the work of Suzanne Harbison for the second edition cover.

Most importantly, I would like to acknowledge the personal contributions of those people without whom this book would not exist. My wife and best friend, Julie, has been an unwavering source of strength and encouragement during the process of writing this text. Bill Leahy was a student in the first CS1 class I taught in the spring of 1998. In spite of this beginning, he continued to a master’s in computer science from Tech and is now an instructor in the College of Computing. Beyond his uncountable technical contributions to the material in this book, I want to acknowledge his friendship, encouragement, and wise judgment, all of which have been an inspiration to me during the process of developing this text. Most of all, if there is any enduring wisdom in this book, I give all the credit to Almighty God and His Son, Jesus.

ENGINEERING COMPUTATION

with ***MATLAB***[®]

Second Edition

Introduction to Computers and Programming

Chapter Objectives

This chapter presents an overview of the historical background of computing and the computer hardware and software concepts that build the foundation for the rest of this book:

- Hardware architectures
- Software categories
- Programming languages
- Problem solving

- I.1 Background
- I.2 History of Computer Architectures
 - I.2.1 Babbage's Difference Engine
 - I.2.2 Colossus
 - I.2.3 The von Neumann Architecture
- I.3 Computing Systems Today
 - I.3.1 Computer Hardware
 - I.3.2 Computer Memory
 - I.3.3 Computer Configurations
 - I.3.4 Computer Software
 - I.3.5 Executing a Computer Program
- I.4 Executing a MATLAB Program
- I.5 Problem Solving



1.1 Background

Advances in technology are achieved in two steps as follows:

- A visionary conceives an idea that has never been tried before
- Engineers find or invent tools that will bring that vision to reality

The search for new software tools is therefore an inescapable part of an engineer's life. The process of creating these tools frequently spawns sub-problems, which themselves require creative solutions.

The pace of change in our world is increasing, and nowhere is this phenomenon more dramatically obvious than computer science. In the span of just a few generations, computers have invaded every conceivable aspect of our lives, and there is no indication that this trend is slowing.

This book will help you become familiar with one specific programming tool: MATLAB. It is intended to bring you to a basic proficiency level so that you can confidently proceed on your own to learn the features of other programming languages that are useful to your interests.

A word of caution: Learning a programming language is very much like learning to speak a foreign language. In order to find something to eat in Munich, you must be able to express yourself in terms a German can understand. This involves knowing not only some vocabulary words, but also the grammatical rules that make those words comprehensible—in German, for example, this means putting the verbs at the ends of phrases.

If languages were a strictly theoretical exercise, you could make up your own vocabulary and grammar, and it would undoubtedly be an improvement over existing languages—especially English, with its incredibly complex spelling and pronunciation rules. However, language is not a theoretical exercise; it is a practical tool for communication, so we can't make up our own rules, but are constrained to the vocabulary and grammar expected by the people with whom we want to converse.

Similarly, this book is not an abstract text about the nature of computer languages. It is a practical guide to creating solutions to problems. Accomplishing this involves expressing your solutions in such a form that the computer can “understand” your solutions; therefore, it requires that you use the vocabulary (that is, the appropriate key words) and grammar (the syntax) of the language.

To become proficient in this, as in any other language, it is not enough to merely know the grammar and vocabulary. You have to practice your language skills by communicating. For foreign languages, this means traveling to the country, immersing yourself in the culture, and talking with people. For computer languages, this means actually writing programs,

seeing what they do, and determining how to use their capabilities to solve your engineering problems.



1.2 History of Computer Architectures

Computing concepts developed as tools to solve previously intractable problems. This section will trace the growth of computing architectures, review the basic organization of computer hardware components, and emphasize the implementation of the data storage and processing capabilities by highlighting three milestones on the road to today's computers: Babbage's difference engine, Colossus, and the von Neumann architecture.

1.2.1 Babbage's Difference Engine

Charles Babbage (1791–1871) is generally recognized as the earliest pioneer of the modern computer. Babbage's **difference engine**—a relatively simple device that can subtract adjacent values in a column of numbers—is a good example of a computing device designed to improve the speed and repeatability of mathematical operations. Babbage was concerned about the process engineers used to develop the tables of logarithms and trigonometric functions. In his day, the only way to develop these tables was for mathematicians to calculate the values in the tables by hand. While the algorithms were simple—combining tables of the differences between adjacent values—the opportunity for human error was unacceptably high. In 1854 Babbage designed a difference engine that could automate the process of generating tables of mathematical functions. Since the objective was to create numerical tables, the output device was to be a set of copper plates ready for a printing press. The memory devices for storing numerical values were wheels arranged in vertical columns. The arithmetic operations were accomplished by ratchet devices cranked by hand.

Sadly, the manufacturing tools and materials available then prevented him from actually building his machine. However, in 1991 the Science Museum in London built a machine to his specifications, as shown in Figure 1.1. With only minor changes to the design, they were able to make the machine work. Although limited in its flexibility, the machine was able to compute difference equations up to the seventh order with up to 13 significant digits.

1.2.2 Colossus

Colossus was a computing machine developed to solve large, complex problems quickly. Early in the Second World War, Britain was losing the Battle of the Atlantic—German U-boats were sinking an enormous number of cargo ships that were resupplying the Allied war effort. The Government Code and Cypher School was established at Bletchley Hall in Britain with the goal of

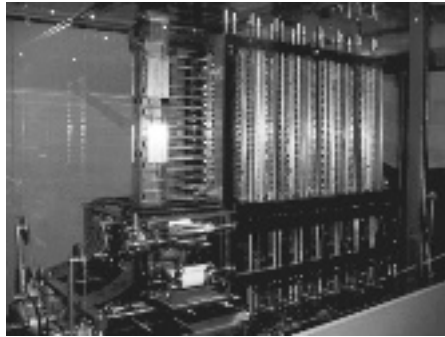


Figure 1.1 *The Babbage difference engine*

breaking the German codes used to communicate with their U-boats in the North Atlantic. They were using Enigma machines, relatively simple devices that encrypted messages by shifting characters in the alphabet. However, to crack the code they needed to exhaustively evaluate text shifted by arbitrary amounts. Although the algorithm was known, the manual solution took too long, and it was often too late to make use of the information. A computer later named Colossus (see Figure 1.2) was designed by Max Newman and was custom built for this purpose. While not a general-purpose processor, Colossus was fast enough to crack all but the most sophisticated Enigma codes. Sadly, due to security concerns, the machine was destroyed when the war ended. However, the dawn of ubiquitous computing was breaking, and general-purpose computers were soon to be available.

1.2.3 The von Neumann Architecture

These and other contemporary achievements demonstrated the ability of special-purpose machines to solve specific problems. However, the creativity of John von Neumann ushered in the current era of general-purpose computing in which computers are flexible enough to solve an astonishing array of different problems. Dr. von Neumann proposed a computer architecture that separated the Central Processing Unit (CPU) from the computer memory and the Input/Output (I/O) devices (see Figure 1.3).



Figure 1.2 *The Colossus computer*

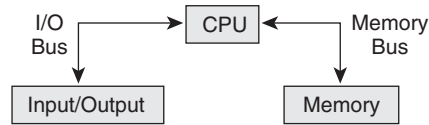


Figure 1.3 *The von Neumann architecture*

Together with binary encoding for storing numerical values, this was the genesis of general-purpose computing as we know it today. Although the implementation of each component has improved beyond recognition, the fundamental processing architecture remains unchanged today.



1.3 Computing Systems Today

Today's computing systems—the combination of hardware and software that collectively solve problems—retain many of the key characteristics of these inventions: they process more data than is humanly possible, quickly enough for the results to be useful, and they basically follow the von Neumann architecture. Computer **hardware** refers to the physical equipment: the keyboard, mouse, monitor, hard disk, and printer. The **software** refers to the programs that describe the steps we want the computer to perform.

1.3.1 Computer Hardware

All computers have a similar internal organization, as shown in Figure 1.4, that is closely related to the von Neumann architecture. The CPU is usually separated into two parts: the Control Unit, which manages the flow of data between the other modules, and the Arithmetic and Logic Unit (ALU). The ALU performs all the arithmetic and logical operations required by the software.

The individual logic devices that comprise the electronic components of the computer operate in binary mode, which is represented electrically by the presence or absence of voltage at a connection. These states, called **bits**, have

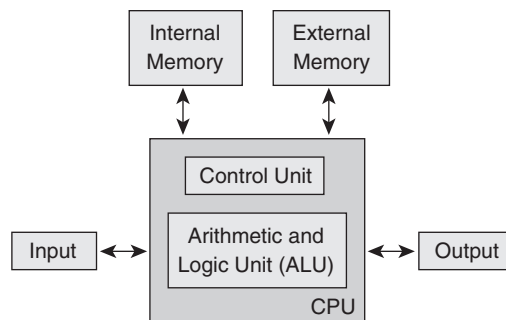


Figure 1.4 *Internal organization of a computer*

the value of 1 (present) or 0 (absent). Most computer operations assemble these bits into larger collections—a **byte** being 8 bits, and **words** consisting of 16, 32, 64, or more bits. We refer to the data items coming into the computer as the **input**, and the results coming from the computations as the **output**.

Input and output (I/O) is accomplished by moving data between the memory (CPU) and external equipment designed to communicate with users or other computers. In the early days all devices had to be individually installed in the computer with dedicated wiring—a process called **hard-wiring**. In contrast, today this is usually accomplished merely by plugging devices into one of many **data buses** (see Figure 1.5). A data bus is an electronic “pathway” for transporting data between devices. Since most devices expect to be able to send data on the bus as well as receive data from it, data bus design always involves a protocol which ensures that only one device is writing to the bus at any given time.

1.3.2 Computer Memory

Memory comes in many forms. Not long ago it could be nicely divided into two categories—solid state and mechanical. Solid-state memory modules were directly connected to the processor and used digital addresses to save and restore data. Mechanical memory relied upon devices that moved rewriteable storage media past sensors that converted the impressions on the storage media to digital form. Tape drives, floppy disks, hard drives, and optical disks (CDs and DVDs) share this architecture, and they are usually externally connected to the input/output system. Recently, however, these

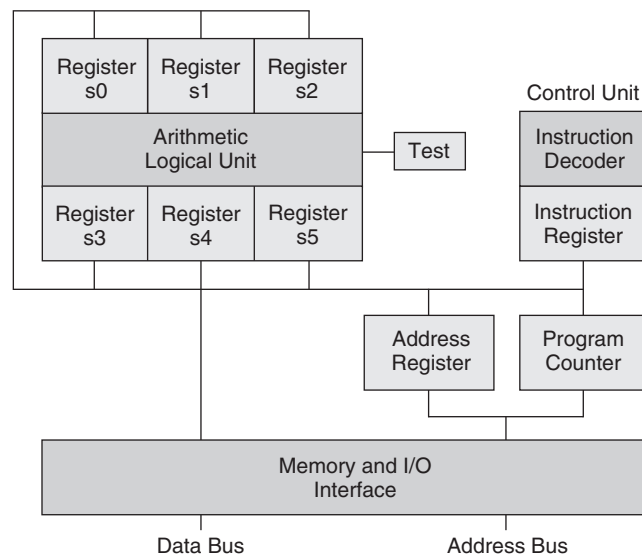


Figure 1.5 Internal computer details

distinctions have been blurred by the arrival of devices like **flash cards**, which are solid-state memory devices but attach to the computer's I/O ports and behave as if they were mechanical memory.

Today CPUs use many forms of solid-state memory. The first instructions executed when power is turned on are usually stored in **Read-Only Memory (ROM)**, sometimes referred to as the **Basic Input/Output System (BIOS)**. These instructions are just enough to wake up the keyboard and screen in basic mode and look around for a memory device containing the real programs. These real programs are transferred from the memory device, frequently referred to as “mass memory,” to **Random-Access Memory (RAM)**—large amounts of high-speed, solid-state memory used to hold all of the programs and data users need immediately.

Most processors achieve significant performance improvement by using smaller amounts of even higher speed memory as **cache**. Cache memory processors are smart devices that “guess” what instructions and data the computer needs next, and preload those guesses into cache memory where the CPU can reach them quickly. These guesses are based on the likelihood that the program will continue linearly through the program as opposed to branching to go somewhere else for the next instruction. A significant amount of today's computer architecture design effort focuses on the effective use of cache memory to improve performance.

As programs become larger and process more data, and the systems allow more than one program to run simultaneously, RAM occasionally fills up. Most operating systems today use **virtual memory**—a data file usually on the hard drive that contains an image of everything you would like to have in RAM divided into **pages**. When the CPU requires access to a page that is not actually in RAM, it has to take the time to find a special area in RAM referred to as a “page buffer” that it can safely use, write its contents back to virtual memory, and read in the page needed. No matter how smart this process might be about looking ahead and predicting required pages, there is always a huge performance loss when a computer begins using virtual memory.

Figure 1.6 illustrates some aspects of how computer memory is managed. The operating system (UNIX, Windows, Mac OS X, or whatever) consumes some memory and determines from the I/O devices available what internal

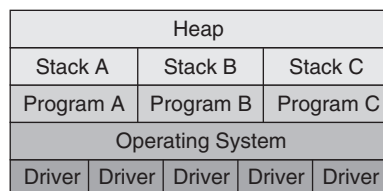


Figure 1.6 *Typical memory layout*

software (**drivers**) must be present to enable the application programs to communicate with the outside world. As mentioned earlier, many programs are loaded automatically when the operating system starts, and others are loaded upon user request. In addition to the memory needed to store the instructions, each program is allocated some **stack** space for storing local static data. The remaining memory, the **heap**, is accessible to all programs upon request to the operating system. The heap is typically used to store most of the data being manipulated by the programs. When a program finishes with a block from the heap, it is usually released by that program for other programs to use as necessary.

1.3.3 Computer Configurations

Depending on the application, there is a broad range of computer configurations, the most common of which are discussed in this section. As these components become cheaper to produce and as they operate at comparable reliabilities, computers are used more and more frequently to replace mechanical and electrical devices. These are called **embedded computers**—computers that are built into other equipment rather than standing alone. Because they usually run a fixed set of programs, embedded computers are known as **special-purpose processors**. The programs are usually in ROM, start running as soon as power is applied, and continually read data from multiple sensors and send out signals to control the equipment for which they are responsible. Modern automobiles use embedded computers to implement the complex algorithms that improve the efficiency of the engines and reduce emissions, and many of the I/O devices connected to computers are controlled by embedded processors. **Personal computers (PCs)** are small, inexpensive, **general-purpose computers** that can run a wide variety of programs; they are commonly used in offices, homes, and laboratories. **Mainframes** are even more powerful computers that are often used in businesses and research laboratories. **Supercomputers**, the fastest of all computers, can process billions of instructions per second. Because of their speed, supercomputers are capable of solving very complex problems that cannot feasibly be solved on other computers. Mainframes and supercomputers require special facilities and a specialized staff to maintain them.

1.3.4 Computer Software

Computer software contains the instructions that the CPU uses to run programs. There are several important categories of software, including operating systems, software applications, and language compilers. Not all processors need all these facilities. Figure 1.7 illustrates the interactions among these categories of software and the computer hardware, and the following sections describe each in more detail.

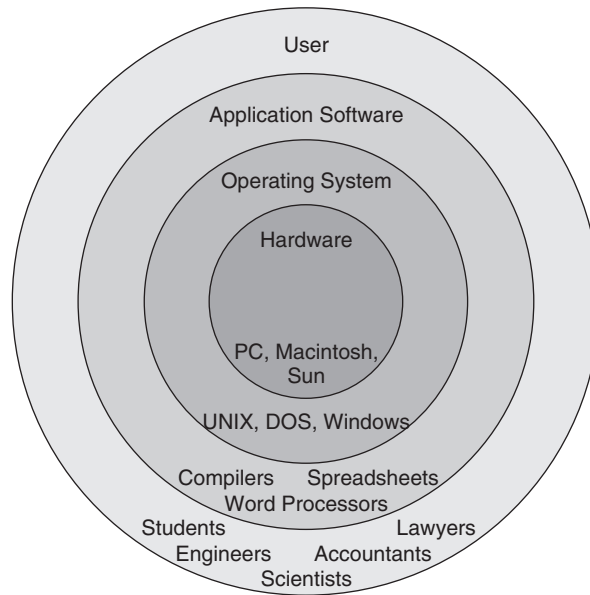


Figure 1.7 *Interactions between hardware and software*

Operating Systems The **operating system (OS)** serves as the manager of the computer system as a whole. It controls access to the processor by users and networked devices, and it organizes the hardware and software according to the users' specifications. The OS is the first major software component fetched by the BIOS from mass storage, and it automatically loads and starts the myriad programs that make computers "user friendly." It also provides the tools for making the computer's peripheral devices—such as printers, scanners, and DVD drives—available to other software. Common modern operating systems are Microsoft Windows, Linux, UNIX, and Apple Mac OS.

Operating systems also contain a group of programs called **utilities** that allow you to perform functions such as printing files, copying files from one disk to another, and listing the files that you have saved on a disk. Although these utilities are common to most operating systems, the commands themselves vary from operating system to operating system. While computer systems give the appearance of stability, like automobiles, they require periodic maintenance to maintain peak performance.

- You should protect your computer by installing and configuring utilities that protect it from viruses, intrusive advertising, and external influences that make illegal use of the processor or its data. Refer to the documentation for your specific operating system.

- Over time most disk drives become fragmented—the available space gets chopped up into smaller and smaller pieces—and the performance of your system begins to suffer. Defragmentation of a large disk drive may be an overnight effort, but should be done periodically.
- While very reliable, computers are not indestructible. You should establish a regular policy of backing up your personal files onto removable media. Most operating systems provide such utilities. You do not need to back up commercial software that can be reloaded from the manufacturer's installation disks.

Software Tools Software tools are commercial programs that have been written to solve specific problems. They are highly sophisticated, complex applications that use the facilities provided by the operating system to enable you to create, save, recall, manipulate, and present ideas in the form of data files on your computer. The specific nature of those files depends on the nature of the problem.

If you need a well-formatted document or report, **word processors**, such as Microsoft Word, are programs that enable you to enter and format text and graphics. They allow you to develop documents in outline form; move words, sentences, and paragraphs; and check your spelling and grammar. **Desktop publishing** combines a very powerful word processor with a high-quality printer to produce professional-grade documents.

If you need relatively unsophisticated results from tabular data, **spreadsheets** let you work easily with data that can be displayed in a grid of rows and columns. Most spreadsheet packages include plotting capabilities to create charts and graphs, so they can be especially useful in analyzing and displaying information. Microsoft Excel and Lotus 1-2-3 are popular spreadsheet programs.

If you need to store, quickly retrieve, and format large amounts of data, **database management** programs such as Microsoft Access, MySQL, and Oracle are useful tools. They are used by large organizations, such as banks, hospitals, universities, hotels, and airlines, to store and organize crucial information; they are also used to analyze large amounts of scientific data. Meteorology and oceanography are examples of scientific fields that commonly require large databases for storage and analysis of data.

Computer-aided design (CAD) packages such as AutoCAD let you define computer models of real-world objects, assemble groups of such models, and then manipulate them graphically. CAD packages are frequently used in engineering applications, and the designs of most automobiles and aircraft (e.g., the Lockheed Martin F-22 and F-35) are now “paperless”—the essential information is in a CAD database rather than on paper.

Programming Languages All programming languages are merely tools a programmer uses to express the logic for a computer to implement. Like any spoken language, a computer language is defined by its grammar (**syntax**) and its vocabulary. There are three necessary attributes of a computer language: the scope of the logic expressed in each line of code (the **power** of the language), the clarity of each line of code from the human viewpoint, and its **portability** between different types of processor. Computer languages are frequently described in terms of **generations** that reflect the development of language power, clarity and portability.

First generation, or machine languages, are the most primitive languages, usually tied closely to the nature of the computer hardware. Since the basic logic of the CPU is binary, the syntax of machine language is expressed as sequences of 0s and 1s. This maximizes the control over the processor, but results in programs that are completely incomprehensible to anyone including the original programmer, and are absolutely not portable.

A **second generation** language, frequently called **assembly language**, is a means of expressing machine language in symbolic form where each line of code usually produces a single machine instruction. While programming in assembly language is easier than machine language, it is still a tedious process that requires each detailed instruction to be specified, and like the first generation languages, is completely tied to the nature of the CPU.

Third generation languages, such as C, FORTRAN, and BASIC have commands and instructions that are more similar to spoken languages. One line of code of these languages creates many machine level instructions. Consequently, they are much clearer expressions of the logic of a program, and the power of each instruction is significantly increased. The resulting programs are to some degree portable between processor types. Third generation languages and beyond are referred to as **high-level languages**.

The **fourth generation** of languages that includes Ada and Java take this trend to the next level. They are completely portable between supported processor types and each line of code creates a significant amount of machine instructions. MATLAB and its close competitors, Mathematica, Mathcad, and Maple, are very powerful fourth generation languages that combine mathematical functions and commands with extensive capabilities for presenting results in graphical form. This combination of computation and visualization power makes them particularly useful tools for engineers. The current language development trend is to allow the programmer express the overall program logic in graphical form, and have the programming tools automatically convert the diagrams to working programs. Programmers involved with these implementations still need language skills to complete the implementation of the algorithms.

The goal of **fifth generation** of languages is to allow a programmer to use **natural language**. Programmers in this generation would program in the syntax of natural speech. Implementation of a fifth generation language will require the achievement of one of the grand challenges of computer science: computerized speech understanding.

1.3.5 Executing a Computer Program

Programs written in most high-level languages, such as C, need to be compiled (that is, translated into machine language) before the instructions can be executed by the computer. A special program called a **compiler** performs this translation. Thus, in order to write and execute C programs on a computer, the computer's software must include a C compiler.

If any errors (often called **bugs**, a reference to an unidentified insect that caused a short in one of the early digital computers) are detected by the compiler during compilation, the compiler generates corresponding error messages. Programmers must correct the program statements and then perform the compilation step again. The errors identified during this stage are called **compile errors** or **compile-time errors**. For example, if you want to divide the value stored in a variable called `sum` by 3, the correct expression in C is `sum/3`. If you incorrectly write the expression using the backslash, as in `sum\3`, you will get a compiler error. The process of compiling, correcting statements (or **debugging**), and recompiling often must be repeated several times before the program compiles without compiler errors. When there are no compiler errors the compiler generates a program in machine language that performs the steps specified by the original C program. The original C program is referred to as the **source code**, and the machine-language version is called the **object code**. Thus, the source code and the object code specify the same steps, but the source code is specified in a high-level language and the object code is specified in machine language.

Once the program has compiled correctly, additional steps are necessary to prepare the object code for **execution**. A **linker** will search libraries of built-in capabilities required by this program and collect them in a single executable file stored on the hard drive. A **loader** is then used to copy the executable program into memory where its instructions can be executed by the computer. New errors, synonymously called **execution errors**, **runtime errors**, **logic errors**, or **program bugs**, may be identified in this stage. Execution errors often cause the termination of a program. For example, the program statements may attempt to perform a division by zero, which generates an execution error.

Some execution errors do not stop the program from executing, but they cause incorrect results to be computed. These types of errors can be caused by programmer errors in determining the correct steps in the solutions and

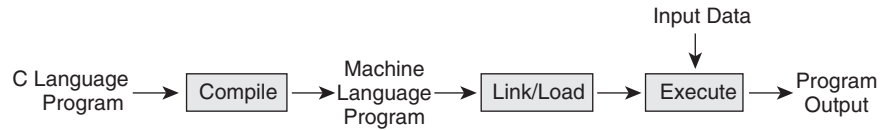


Figure 1.8 Program compilation, linking/loading, and execution

by errors in the data processed by the program. When execution errors occur because of errors in the program statements, you must correct the errors in the source program and then begin again with the compilation step. Even when a program appears to execute properly, you must check the results carefully to be sure that they are correct. The computer will perform the steps precisely as you specify them. If you specify the wrong steps, the computer will execute these wrong (but syntactically legal) steps and present you with an answer that is incorrect.

The process of converting an assembly language program to binary language is performed by an **assembler**, and the corresponding processes are called assembly, linking/loading, and execution. These processes are outlined in Figure 1.8.



1.4 Executing a MATLAB Program

MATLAB is officially classed as an interpreted language—one that does not require compilation. It does have a compilation step, but it is hidden from users. In the MATLAB environment you can develop and execute programs (**scripts**) that contain MATLAB commands. You can also execute a MATLAB command, observe the results, and then execute another MATLAB command that interacts with the information in memory, observe its results, and so on. This **interactive environment** does not require the formal compilation, linking/loading, and execution process described earlier for high-level computer languages. However, errors in the syntax of a MATLAB command are detected when the MATLAB environment attempts to translate the command, and logic errors can cause execution errors when the MATLAB environment attempts to execute the command.



1.5 Problem Solving

If we think about a computer program as a logical component that consumes data in one form and produces data in another, we can think about problem solving as the process of designing a collection of solutions to subproblems. The term **state** refers to the collection of data handed off from one subproblem solution to another.

In general terms, solutions to nontrivial problems are found by the two-pronged approach shown in Figure 1.9. We can consider the original

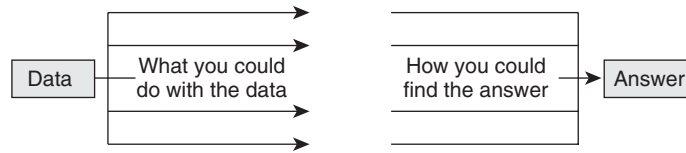


Figure 1.9 *Generalized problem solving*

information and ask ourselves what could be done with that information using existing tools, and we can also consider the objective and the different ways in which that objective might be achieved. The process of creative problem solving then becomes a search for a match between states that can be achieved from the given data and states from which the answer can be achieved.

For example, say you have a big collection of baseball cards and you want to find the names of the 10 “qualified” players with the highest lifetime batting averages. To qualify, the players must have been in the league at least 5 years, had at least 100 plate appearances per year, and made fewer than 10 errors per year.

The cards contain all the relevant information for each player. You just have to organize the cards to solve the problem. Clearly there are a number of steps between the stack of cards and the solution. In no particular order these are:

- a. Write down the names of the players from some cards
- b. Sort the stack of cards by the lifetime batting average
- c. Select all players from the stack with 5 years or more in the league
- d. Select all players from the stack with fewer than 10 errors per year
- e. Select all players from the stack with over 100 plate appearances per year
- f. Keep the first 10 players from the stack

When you think about it from right to left as shown in Figure 1.9, step a is probably the last step and step f is probably the step before that. The hard work starts when you think about it from left to right. Intuitively, when you think about sorting the stack of cards, this seems like a lengthy process. Since the sorting should probably be done on a small number of cards, you should do all the selecting before the sorting. Continuing that line of reasoning, you would reduce the total effort if the first selection pass was the criterion that eliminated most cards. You might even consider combining all three selection steps into one.

One logical way to find the players’ names that you need would be to perform the steps in this order: c, d, and e in any order, followed by b, f, and then a.



Chapter Summary

This chapter presented an overview of the historical background of computing and the computer hardware and software concepts that build the foundation for the rest of this book:

- How the basic von Neumann architecture has been adopted for the full range of computers, from small, embedded processors to massive mainframes
- The spectrum of software products, ranging from operating systems to the many flavors of specific programming tools
- The rich variety of programming languages currently in use, and the place of MATLAB in that spectrum as a legitimate fourth-generation language
- The basics of problem solving as a search for a path from the data provided to the answers required



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. Computers were originally conceived as tools for solving specific problems.
2. Bill Gates designed the first working computer.
3. Programs cannot interact with the world outside the computer without an operating system.
4. Programs cannot interact with the world outside the computer without drivers.
5. Programs cannot interact with the world outside the computer without hardware interfaces.
6. Application programs have access to shared memory.
7. An algorithm bridges the gap between the available data and the result to be achieved.

Fill in the Blanks

1. A computer language is not a(n) _____ exercise; it is a _____ tool for communication and problem solving.
2. Together with the use of binary encoding for storing numerical values, _____ was the genesis of general-purpose computing as we know it today.
3. Most operating systems today use _____, which is actually a data file containing an image of everything you would like to have in RAM.
4. Operating systems contain a group of programs called _____ that allow you to perform functions such as printing, copying files, and listing the file names.
5. Many _____ are loaded automatically when the operating system starts, and others are loaded upon user request.
6. Even when a program appears to execute properly, you must check the results carefully to find _____ errors.
7. Problem solving is the process of designing a collection of _____.
8. The process of problem solving is a search for a match between _____ one can achieve from the given data and _____ from which the answer can be achieved.

Getting Started with MATLAB

Chapter Objectives

This chapter introduces you to some of the fundamentals of computing that apply to all programming languages, and specifically to the programming environment used for MATLAB program development.

The fundamentals of programming include:

- How to use abstraction to think in a general way about a collection of data and procedural steps
- How to describe the solution of a problem as an algorithm
- The three paradigms of computing and the position of MATLAB in that spectrum
- Three aspects of the apparently simple task of assigning a value to a variable

As you study the MATLAB user interface, you will understand:

- How to use the Command window to explore single commands interactively and how to recall earlier commands to be repeated or changed
- Where to examine the variables and files created in MATLAB
- How to view and edit data tables created in MATLAB
- How MATLAB presents graphical data in separate windows
- How to create scripts to solve simple arithmetic problems

Introduction

The name MATLAB is a contraction of **Matrix Laboratory**. It was developed for engineers to create, manipulate, and visualize matrices—rectangular arrays of numerical values. As you will see, at its most basic level MATLAB can perform the same functions as your scientific calculator, but MATLAB has expanded far beyond

- 2.1 Programming Language Background**
 - 2.1.1 Abstraction
 - 2.1.2 Algorithms
 - 2.1.3 Programming Paradigms
- 2.2 Basic Data Manipulation**
 - 2.2.1 Starting and Stopping MATLAB
 - 2.2.2 Assigning Values to Variables
 - 2.2.3 Data Typing
 - 2.2.4 Classes and Objects
- 2.3 The MATLAB User Interface**
 - 2.3.1 Command Window
 - 2.3.2 Command History
 - 2.3.3 Workspace Window
 - 2.3.4 Current Directory Window
 - 2.3.5 Document Window
 - 2.3.6 Graphics Window
 - 2.3.7 Editor Window
 - 2.3.8 Start Button
- 2.4 Scripts**
 - 2.4.1 Text Files
 - 2.4.2 Creating Scripts
 - 2.4.3 The Current Directory
 - 2.4.4 Running Scripts
 - 2.4.5 Punctuating Scripts
 - 2.4.6 Debugging Scripts
- 2.5 Engineering Example—Spacecraft Launch**

its original capabilities and now provides an interactive system and programming language for many applications, including financial analysis as well as general scientific and technical computation.

The following are the fundamental components of MATLAB:

- A computing system that accepts one instruction at a time in text form and implements the logic of that instruction. Instructions must conform to a specific syntax and vocabulary, which will be the topic of Chapters 3–9.
- A large library of modules that provide high-level capabilities for processing data. These modules will be the major topic of Chapters 10–17.
- A large collection of toolboxes, which are separate application programs that provide graphical capabilities for computing in a number of engineering and scientific disciplines.
- A graphical user interface (GUI) that lets users assemble and implement programs that solve specific problems. The rest of this chapter will describe the basic behavior of these windows.

MATLAB offers a number of advantages to users over conventional, compiled languages like C++, Java, or FORTRAN:

- Because MATLAB programs are interpreted rather than compiled, the process of producing a working solution can be much quicker than with compiled languages.
- MATLAB excels at numerical calculations, especially matrix calculations.
- MATLAB includes a wide spectrum of toolboxes that implement fourth-generation language concepts for graphical problem solving.
- One of the MATLAB tools, the Graphical User Interface Development Environment (GUIDE), enables you to build your own GUIs.
- MATLAB has built-in graphics capabilities that produce professional-looking images for reports.

However, because MATLAB is not a compiled language, there are problems you should not attempt to solve using MATLAB. For example:

- MATLAB does not work well for large computing projects where a number of developers share coding responsibilities.
- Professional GUIs and windowing applications (like MATLAB itself) are best written in a compiled language.



2.1 Programming Language Background

Before learning about concepts in computing, you need to understand the background of programming languages. This section discusses the following aspects of programming languages: abstraction, algorithms, programming paradigms, and three fundamental concepts of programming—assigning values to variables, data typing, and the difference between classes and objects.

2.1.1 Abstraction

For the purpose of this text, we will define **abstraction** as “expressing a quality apart from a particular implementation.” We use the concept of abstraction in everyday conversation without thinking about it:

“To convert from degrees Celsius to Kelvin, you add 273 to the temperature.”

“He drove home from the office.”

The first is an example of **data abstraction**. “The temperature” could mean a single reading from the thermometer hanging outside the window or a table of temperature readings for the month of August. The specifics are unimportant; the phrase captures all you need to know.

The second example is actually much more complex—an example of multiple levels of **procedural abstraction**. To a businessperson taking the same route home every night, “drive home” is all that is required to understand the idea. To a competent driver unfamiliar with the route, the next level of abstraction might be necessary—turn right out of the parking lot, left onto Main Street, and so on. For instructions to guide a future robotic commuter vehicle, an incredibly fine-grained level of abstraction will be required. Everything taken for granted in these abstractions will need to be meticulously spelled out for the robotic vehicle—start the engine, accelerate the vehicle, look out for traffic, keep in the lane, find the turn, steer the vehicle, control the speed, observe and obey all signs, and so on.

2.1.2 Algorithms

Chapter 1 defined problem solving as the ability to isolate subproblems that seem simple and appropriate to solve, and then assemble the solutions to these subproblems. The solutions to each of these subproblems would be expressed as an **algorithm**, which is a sequence of instructions for solving a problem. The process of solving each subproblem and assembling the solutions to form the solution to the whole problem would also be expressed as an algorithm at a higher level of abstraction.

The level of abstraction needed to describe an algorithm varies greatly with the mechanism available. For example, describing the algorithm (recipe) for baking cookies might take the following forms:

- To your grandmother, who has been baking cookies for the last 50 years, it might be “Bake some cookies.”
- To others it might be “Buy a cookie mix and follow the directions.”
- To a young person learning to cook from scratch, the algorithm might include an intricate series of instructions for measuring, sifting, and combining ingredients; setting the oven temperature and preheating the oven; forming the cookies and putting them on the cookie sheet; and so on.

In programming terms, algorithms are frequently expressed first conceptually at a high level of abstraction, as demonstrated in Section 1.5. The solutions to each subproblem would then be expressed at lower and lower levels of abstraction until the description is sufficient to write programs that solve each subproblem, thereby contributing the pieces that, when assembled, solve the whole problem.

2.1.3 Programming Paradigms

From the Greek word *paradeigma*—“to show alongside”—the *American Heritage Dictionary* defines a paradigm as “a set of assumptions, concepts, values, and practices that constitutes a way of viewing reality for the community that shares them, especially in an intellectual discipline.” So a programming paradigm becomes a codified set of practices allowing the community of computing professionals to frame their ideas. This section considers three radically different paradigms: functional programming, procedural programming, and object-oriented programming.

Functional programming is typically associated with languages like Lisp and Forth, in which every programming operation is actually implemented as a function call with no side effects (changes of state of the program surroundings) permitted or implemented in the language. Without side effects, a programming solution can be mathematically proven to be correct—an enormous advantage. Except for the discussion of recursion, this paradigm will not be mentioned again.

Procedural programming is typical of languages like FORTRAN, C, and MATLAB, where the basic programs or subprograms are sequences of operations on data items that are generally accessible to all programs. Although side effects from subprograms—such as changing the values of variables outside that subprogram—are considered poor practice, they are not prohibited by the language.

Object-oriented programming (OOP), typical of languages like C++, Ada, and Java, is a relatively new addition to the world of programming paradigms. It is characterized by the concept of encapsulating, or packaging, data items together with the methods or functions that manipulate those data items. In this paradigm, side effects are explicitly managed by controlling access to the data and methods in a particular grouping. The major theme in true OOP is that “everything is an object.” You will see MATLAB exhibit many traits of OOP as you work through this book, but you will not need to use this programming paradigm.



2.2 Basic Data Manipulation

In order to use MATLAB to demonstrate basic data manipulation, we begin with an exercise in starting and stopping the MATLAB system.

2.2.1 Starting and Stopping MATLAB

MATLAB is available in both a professional version and a student version. Student versions for the Windows, Macintosh, and Linux operating systems include the most commonly used components of the professional version. See the documentation for your particular installation for details.

The only obvious difference between the student and the professional versions is that the command prompt in the professional version is `>>`, whereas the student version's prompt is `EDU>>`. Exercise 2.1 shows you how to start and stop the MATLAB user interface.

We will soon see the details of all the program's windows. For the moment, however, we will interact with MATLAB by typing instructions in the large Command window that occupies the right side of your screen.



Exercise 2.1 Starting and stopping MATLAB

Do It Yourself

If you have not installed MATLAB on your computer yet, follow the manufacturer's directions for performing the installation.

To start MATLAB on a PC or Macintosh, double-click on the MATLAB icon (which should be located on the desktop) or use the Start menu. If you are using a UNIX operating system, type `MATLAB` at the shell prompt. In the Command window you should see the MATLAB prompt (`>>` or `EDU>>`), which tells you that MATLAB is waiting for you to enter a command.

To exit MATLAB, type `quit` or `exit` at the MATLAB prompt, choose the menu option `File > Exit MATLAB`, or click the close icon (x) in the upper-right corner of the screen.

2.2.2 Assigning Values to Variables

The concept of assigning values to variables is the first challenge facing novice programmers. The difficulty arises because many programming languages (including MATLAB) present this simple concept in a syntax that is very similar to conventional algebra, but with significantly different meaning. Consider, for example, the following algebraic expression:

$$z = x + y$$

In normal algebra, this is a two-way relationship that is an identity for the duration of the problem. If you knew the values of z and x , you could derive the value of y with no further analysis. To a programmer, however, this statement has a different meaning. It means that you want to sum the values given to the variables x and y , and store the result in a variable called z . If either x or y are unknown at the time of executing this statement, an error ensues. In particular, this relationship is true *only for this statement*. The relationship can be revoked in the next instruction, which might be:

$$z = 4*x - y$$

In algebra, this pair of statements collectively constrains the values of x , y , and z . In programming, the only significance is that the programmer decided to calculate the current value of z differently. A few languages are sensitive to this dilemma and use a different symbol for assigning values to a variable. For example, in Pascal or Ada $z = x + y$ would be written as follows:

$$z := x + y$$

This clearly indicates that this is an assignment statement, not an algebraic identity.

Style Points

1. Some early versions of the FORTRAN and Basic languages severely restricted the number of characters you could use for variable names. It is no longer necessary to program as if you were still in the “bad old days.” Choose names for variables that describe their content. For example, a variable used to store the velocity of an object should be named `velocity_in_feet_per_second` rather than `v`.
2. Since the space character is not permitted in variable names, there are two conventions for joining multiple words together to make a single variable name. One uses the underscore character to separate the words (`file_size`), and the other capitalizes the first letter of additional words (`fileSize`). You should choose one convention and be consistent with it. You cannot use a hyphen to concatenate words—MATLAB treats the name `file-size` as the arithmetic operation subtracting the value of the variable `size` from the value of the variable `file`.

In general, variable names may contain any combination of uppercase and lowercase alphabetic letters, numbers, and the special characters `_` (underscore) and `$` (dollar sign). The underscore character is frequently used to represent a space in a variable name because spaces are not allowed. However, variable names may not begin with a numeric character, and even though the names may be hundreds of characters long, the first 64 characters must be unique.

Exercise 2.2 demonstrates the assignment of values to variables.



Exercise 2.2 Assigning variables

Do It Yourself

Start MATLAB according to the directions.

When MATLAB finishes initialization, you will see three windows on the screen (see Figure 2.1, page 26). For now just focus on the Command window, the large panel to the right in the default screen configuration.

You should see a prompt in the Command window—it will be `>>` or `EDU>>` depending on your MATLAB license. This is your invitation to type something. Text that you should type will be shown like this throughout this book:

```
>> radius = 49
```

Note that all entries in the MATLAB Command window terminate with the **Enter** key. The MATLAB system response will be shown like this:

```
radius =  
    49  
>>
```

This response indicates that MATLAB has stored the value 49 in a variable named `radius`. To retrieve the value of `radius`, you just type its name and press **Enter**:

```
>> radius  
ans =  
    49
```

This response shows that MATLAB retrieved the value 49. Since you didn't specify where to put this result, MATLAB stored the result in a default variable named `ans`.

2.2.3 Data Typing

It is important to understand how MATLAB treats the data stored in a variable. Different languages take varying approaches to this, and languages in general fall into two broad categories: untyped and typed. In general, interpreted languages like Lisp, Forth, Python, and MATLAB determine the type of data contained by a variable based on the type of data being stored there. Such languages are referred to as **untyped languages**. Each assignment statement is presumed correct. If the variable already exists, both its type and value are reassigned; if it did not exist before, a new variable is created. Exercise 2.3 illustrates the effect of performing simple mathematical operations in MATLAB.

By putting 49 into the variable `radius` in Exercise 2.3, you established its type as numeric and enabled it to be used in normal arithmetic operations. Character strings are specified by including arbitrary characters between single quote marks. These have the type `char`, and must be handled differently, as discussed fully in Chapter 6. When you stored a character string in the variable `radius`, adding 1 to it did not cause an error in MATLAB as it would in some other languages, because addition is actually defined



Exercise 2.3 Performing basic mathematical operations

Do It Yourself

Make the following entries in the MATLAB Command window. You should see the responses as shown here:

```
>> radius = 49
radius =
    49
>> radius + 1
ans =
    50
>> radius = 'radius of a circle'
radius =
    radius of a circle
>> radius + 1
ans =
   115  98 101 106 118 116 33 112 103 ...
```

for character strings It just did something radically different—it actually converted the individual characters to numbers and then added 1.

While this ability to assign data types dynamically is good for interpreted languages, it has two undesirable consequences that are really hard to unravel as the program runs:

- Typographical errors that misspell variable names in assignment statements cause new variables to be declared unintentionally and without the user noticing the error
- Logical errors that assign incompatible data to the same variable can cause obscure runtime errors

Typed languages require that programmers specify both the name and type of a variable before a value can be assigned to it. With this information a compiler can then do a better job of ensuring that the programmer is not using a variable in an unintended way. Typed languages fall into two categories: weak typing and strong typing.

If programmers decide to use only the normal data types, such as `double` and `char` as we saw above, this is known as **weak typing**, and is the usual approach to typing.

In some extreme circumstances programmers may choose to be more restrictive and define specific data types with a limited set of permitted interactions. This is called **strong typing**. For example, programmers might define the following data types, all of which are actually of type `double`: `Meters`, `Seconds`, and `MetersPerSecond`. The compiler would then be provided with a set of rules specifying the legal relationships between these types. For

example, assignments can only be made to a variable of type `MetersPerSecond` from another variable of the same type, or by dividing a variable of type `Meters` by a variable of type `Seconds`.

Before rushing to judge on the pickiness of this approach, note that this would have avoided the loss in 1999 of the Mars Climate Orbiter, which crashed into Mars because one group of programmers used English units while another used metric.

2.2.4 Classes and Objects

This section discusses two different attributes of a variable: its type and its value. In Section 2.2.2 you saw that a variable is a container for data, whose **value** is determined by what is assigned to the variable. In Section 2.2.3 you saw that by making that assignment to a variable, MATLAB also infers the **type** of data stored in that variable. You will see that while MATLAB is an untyped language, the programs you write will behave differently if applied to data of different types. For example, the type `double` specifies the form and expected behavior of number. Adding 1 to a variable of class `double` containing 4 will, as expected, produce the result 5. Similarly, the type `char` is intended to hold a single character. Adding 1 to a `char` variable containing the value `'d'` will produce the numerical equivalent of the character `'e'`.

MATLAB refers to the type of data in a variable as its **class**, and the value contained in the variable at any time as an **object**, an instance of that class. So in the operation:

```
thisNumber = 42.0
```

the variable `thisNumber` would be defined (if it didn't already exist); its class would be set to `double`, the inherent type of a floating point number; and its value to `42.0`. So the word `double` corresponds to a type definition or **class**, while the variable `thisNumber` is a variable of that type, which is an instance of that class or, in programming terms, an **object**.



2.3 The MATLAB User Interface

MATLAB uses several display windows (see Figure 2.1). The default view includes a large Command window on the right, and stacked on the left are the Current Directory, Workspace, and Command History windows.

Hint

You can customize how your initial MATLAB window will display. If you make a mistake and close an essential window, you can always restore the default configuration by choosing Desktop > Desktop Layout > Default.

The tabs near the middle of the windows on the left indicate which views are layered in that particular window. Selecting a tab will bring that view to the top. Older versions of MATLAB included a Launch Pad window, which has been replaced by the Start button in

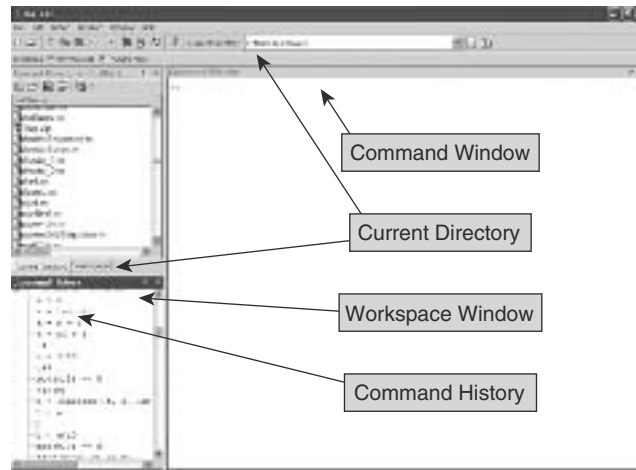


Figure 2.1 *The MATLAB default window configuration*

the lower left corner. Other windows, such as the Document window, Graphics window, and Editor window, will automatically open when needed.

2.3.1 Command Window

You can use MATLAB in two modes: Command mode, which is useful if you need instant responses to specific MATLAB commands, and Edit mode, in which practical solutions are developed. When working in Command mode, we use the Command window, which offers an environment similar to a scientific calculator. This window lets you save any values you calculate, but you cannot permanently save the commands used to generate those values. You will see in the next section how to use the Editor window to create and execute a text file of commands as the first step to unleashing the full programming capability of the language.

Hint

When you make a mistake, you cannot easily correct it as you would in a word processor. The Command window really is functioning like a calculator, performing one instruction at a time exactly as you specify them. When you enter the command, it is immediately executed, regardless of whether that is what you intended. MATLAB offers several ways to correct erroneous commands. One way is to use the arrow keys, usually located on the right side of your keyboard. The up and down arrows let you move through the list of commands you have executed. Once you find the appropriate command, you can edit it and then press **Enter** to execute your new version.

The Command window is very useful for performing quick experiments to discover the effects of different commands in MATLAB before embedding them in a larger program. You can perform calculations in the Command window much like doing calculations on a scientific calculator. Most of the syntax is even the same. Exercise 2.4 shows how you might use the Command window to test two simple calculations.

**Exercise 2.4** Using the Command window**Do It Yourself**

To compute the value of 5^2 , type this command:

```
>> 5^2
```

The following output will be displayed:

```
ans =  
    25
```

To find the cosine of π , type:

```
>> cos(pi)
```

which results in the following output:

```
ans =  
   -1
```

Notice that in both of the examples in Exercise 2.4, MATLAB echoes the result as if it were saved in a variable called `ans`. This is the default variable used to save the result of any calculation you perform in the Command window that is not specifically assigned to another variable.

Notice also the use of one of MATLAB's many built-in functions, `cos(...)`, that computes the cosine of an angle in radians, and of the built-in constant `pi`.

2.3.2 Command History

The Command History window records the commands you issued in the Command window in chronological sequence. When you exit MATLAB or when you issue the `clc` (Clear Commands) instruction, the commands listed in the Command window are cleared. However, the Command History window retains a list of all the commands you issued. You can clear the Command History using the Edit menu if you need to by selecting Edit and then Clear Command History.

Hint

As a security precaution, when you use MATLAB on a public computer, you can set MATLAB's defaults to clear the Command History window when you exit MATLAB or when you log off the computer.

If you entered the sample commands in Exercise 2.4, notice that they are repeated in the Command History window. This window lets you review previous MATLAB sessions, and you can transfer the commands to the Command window.

Exercise 2.5 demonstrates the use of the Command History window. You will find the Command History window useful as you perform more and more complicated calculations in the Command window.



Exercise 2.5 Using the Command History window

Do It Yourself

In the Command window, type:

```
>> clc
```

This should clear the Command window but leave the data in the Command History window intact. You can transfer any command from the Command History window to the Command window by double-clicking it (which also executes the command) or by clicking and dragging the line of code into the Command window. Try double-clicking:

```
cos(pi)
```

This should result in the following display in the Command window:

```
ans =  
    -1
```

Now click and drag 5^2 from the Command History window into the Command window. The command won't execute until you press **Enter**, and then you'll get the following result:

```
ans =  
    25
```

2.3.3 Workspace Window

The Workspace window keeps track of the variables you have defined as you execute commands in the Command window. As you have seen in the exercises so far, because you have not created other variables yet, the Workspace window should just show one variable, `ans`. The columns in the window display the name of the variable, its current value, and two indications of its data type—an icon to the left and an entry in the class column (see Figure 2.2). In this case the variable `ans` has a value of 25 and is a `double` array. Actually, even a single number you would usually consider a scalar is a 1×1 array to MATLAB. Exercise 2.6 shows how to obtain more information about a particular variable. Figure 2.2 shows the normal workspace display for the variable `ans`.

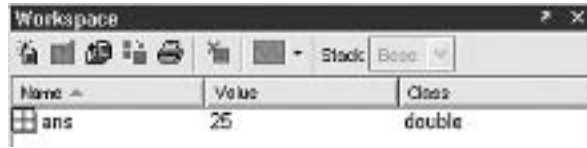
Figure 2.3 shows that the variable `ans` is a 1×1 array, uses 8 bytes of memory, and is an object of class `double`.



Exercise 2.6 Showing more details in the Workspace window

Do It Yourself

Set the Workspace window to show more about the variable `ans` by right-clicking on the bar with the column labels. On the drop-down menu, check the boxes next to **Size** and **Bytes**, so that these will display in addition to **Name**, **Value**, and **Class**. Your Workspace window should now look like Figure 2.3.

Figure 2.2 *The Workspace window*Figure 2.3 *Additional information in the Workspace window*

In Exercise 2.7, variable `a` has been added to the Workspace window, which lists variables in alphabetical order. Variables beginning with capital letters are listed first, followed by variables starting with lowercase letters, as shown in Figure 2.4.

Exercise 2.8 added the variable `b` to the Workspace window, and in Figure 2.5 you can see that its size is a 1×4 array.



Exercise 2.7 Defining other variables

Do It Yourself

You can define additional variables in the Command window and they will be listed in the Workspace window. For example, type:

```
>> A = 5
```

This returns:

```
A =
    5
```



Exercise 2.8 Creating a vector

Do It Yourself

Entering matrices into MATLAB is not discussed in detail in this section. However, you can enter a simple one-dimensional matrix by typing:

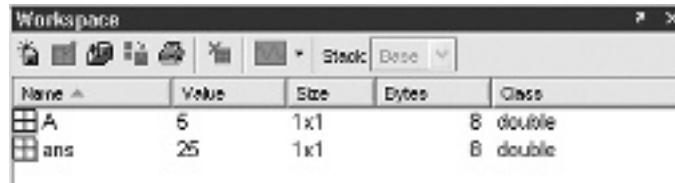
```
>> B = [1, 2, 3, 4]
```

This returns:

```
B =
    1     2     3     4
```

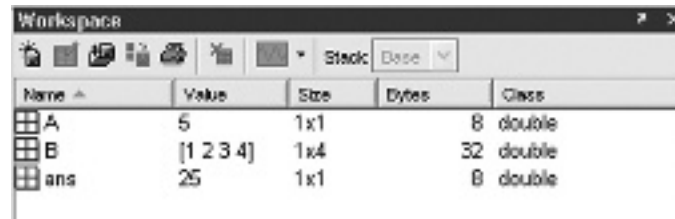
The commas are optional. You would see the same result from:

```
>> B = [1 2 3 4]
```



Name	Value	Size	Bytes	Class
A	5	1x1	8	double
ans	25	1x1	8	double

Figure 2.4 Additional variables



Name	Value	Size	Bytes	Class
A	5	1x1	8	double
B	[1 2 3 4]	1x4	32	double
ans	25	1x1	8	double

Figure 2.5 Vector added in the Workspace window

You define two-dimensional arrays in a similar fashion. Semicolons are used to separate rows, as illustrated in Exercise 2.9. As you can see in Figure 2.6, variable *c* appears in the Workspace window as a 3×4 array. Vectors and arrays are discussed fully in Chapter 3.

Style Points

MATLAB presents numerical results in the following default format: If the value is an integer, there are no decimal places presented, but if there is a fractional part, four decimal places appear. You can change this by using the `format` command. See MATLAB help for details.

You can recall the values for any variable by just typing in the variable name, as shown in Exercise 2.10.

If you prefer to have a less cluttered desktop, you can close any of the windows (except the Command window) by clicking the *x* in the upper-right corner of each window. You can also personalize which windows you prefer to keep open by selecting *View* from the menu bar and checking the appropriate windows. If you suppress the Workspace window, you can still find out what variables have been defined by using the commands `who` or `whos`. The command `who` lists the



Exercise 2.9 Creating a 3×4 matrix

Do It Yourself

For example:

```
>> C = [ 1 2 3 4; 10 20 30 40; 5 10 15 20]
```

returns:

```
C =
     1     2     3     4
    10    20    30    40
     5    10    15    20
```



Exercise 2.10 Recalling values for variables

Do It Yourself

For example, entering:

```
>> A
```

returns:

```
A =
    5
```

Name	Value	Size	Bytes	Class
A	5	1x1	8	double
B	[1 2 3 4]	1x4	32	double
C	<3x4 double>	3x4	96	double
ans	25	1x1	8	double

Figure 2.6 Array added in the Workspace window

variable names and `whos` lists the variable names together with their size and class. Exercise 2.11 illustrates this capability.

2.3.4 Current Directory Window

When MATLAB accesses files from and saves information to your hard drive, it uses the current directory. The default for the current directory depends on your version of the software and how it was installed. The current directory is listed at the top of the main window (see Figure 2.7). This can be changed by selecting another directory from the drop-down list to the right of the current directory name, or by browsing through your computer files using the browse button located to the right of the drop-down list (circled in Figure 2.7).



Exercise 2.11 Using the whos command

Do It Yourself

In the Command window, type:

```
>> whos
```

You should see the following display in the Command window:

```
Name  Size  Bytes  Class
A      1x1    8     double array
B      1x4   32     double array
C      3x4   96     double array
ans    1x1    8     double array
Grand total is 18 elements using 144 bytes
```

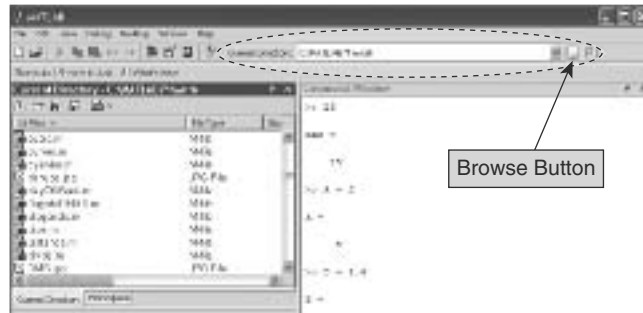


Figure 2.7 The current directory

2.3.5 Variable Editor

Double-clicking on any variable listed in the Workspace window automatically launches a Variable Editor window* (see Figure 2.8). Values stored in the variable are displayed in a spreadsheet-like format. You can change values in the Variable editor, or you can add new values (try Exercise 2.12).

Notice that placing a semicolon at the end of the command (see Exercise 2.12) suppresses the display of that assignment so that the variable value is not repeated in the Command window.

The Variable editor can also be used in conjunction with the Workspace window to create entirely new arrays. Run your mouse slowly over the icons in the shortcut bar at the top of the Workspace window (see Figure 2.2). The function of each icon should appear if you are patient. The icon to create new variables looks like a page with a large asterisk behind it. Click this icon and a new variable called unnamed should appear in the variable list. You can change its name by right-clicking the variable and selecting Rename from the pop-up menu. To add values to this new variable, double-click it and add your data from the Command window.

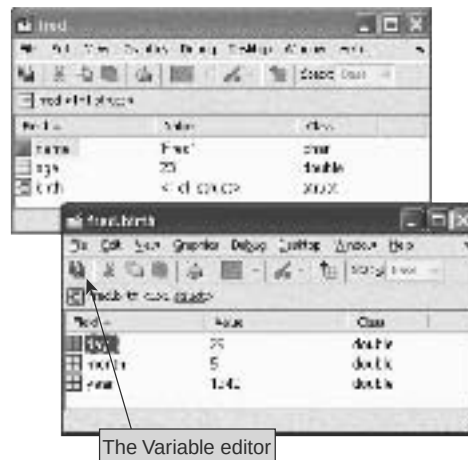


Figure 2.8 The Variable Editor

*Earlier MATLAB versions have a more restricted editing window referred to as an Array Editor

**Exercise 2.12** Creating a two-dimensional matrix***Do It Yourself***

If you haven't already entered the two-dimensional matrix *c*, enter the following in the Command window:

```
>> c = [ 1 2 3 4; 10 20 30 40; 5 10 15 20];
```

c should now be listed in the Workspace window. Double-click it. You will see a Variable Editor window appear. Select any entry and type a new value. You can also click in an empty space, enter a value and thereby increase the size of the original array. Typing *C* in the Command Window will show you the updated values in the array *C*.

2.3.6 Graphics Window

The Graphics window is created automatically when a MATLAB command requests a graph. Exercise 2.13 guides you through creating a graph.

The Graphics window opens automatically (see Figure 2.9). Any additional graphs you create will overwrite the plot in the current Graphics window unless you specifically command MATLAB to open a new Graphics window. The first time you open a graphics window using MATLAB 7.6 (R2008a) a pop-up window appears with links to information about brushing and linking. You can permanently hide this window by clicking the *x* at its right side. If you ever need information about brushing or linking, use the help menu.

MATLAB makes it easy to modify graphs by adding titles, *x* and *y* labels, multiple lines, and more with MATLAB built-in commands. Details of these commands will be presented in Chapter 11.

2.3.7 Editor Window

MATLAB provides a text editor, enabling you to create or modify text files that run in the Editor Window. The Editor Window is opened by choosing

**Exercise 2.13** Creating a graph***Do It Yourself***

To create a simple graph, first create an array of *x* values:

```
>> x = [ 1 2 3 4 5];
```

A new variable, *x*, appears in the Workspace window. Now create a list of *y* values:

```
>> y = [10 20 30 40 50];
```

To create a graph, use the `plot` command:

```
>> plot(x,y)
```

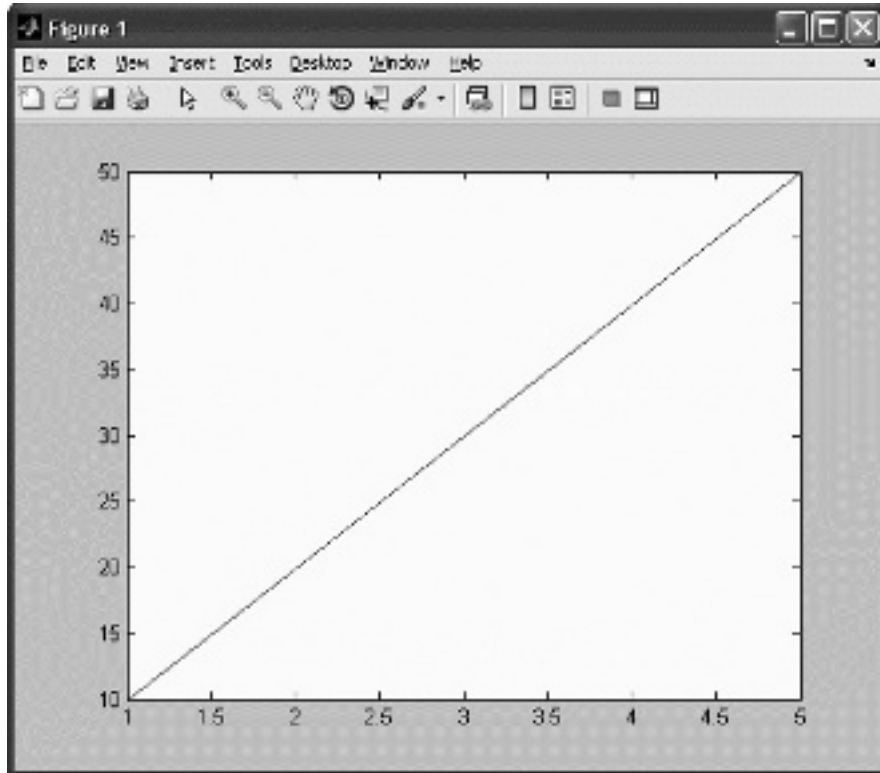



Figure 2.9 *A Graphics window*

File > New > M-File. This window lets you type and save a series of commands without executing them. You can also open the Editor window by double-clicking a file name in the Current Directory window or by typing:

```
>> edit <file_name>
```

in the Command window, where <file_name> is the name of the file you want to open. You can open multiple files at the same time, using the tabbed overlays to identify the files. An asterisk appears on the tab with the file name to indicate that the file has been modified since it was saved. Options under the Window menu let you organize the multiple files in various ways that make more than one file visible at once. When closing the Editor window, MATLAB displays a dialog asking if you need to save any changed files.

2.3.8 Start Button

The Start button is located in the lower-left corner of the MATLAB window. It offers access to the many features of MATLAB that are beyond the scope of this book. The Start button is new to MATLAB 7 and replaces the Launch Pad window used in MATLAB 6.



2.4 Scripts

This section describes the basic mechanism for creating, saving, and executing scripts as m-files. Building script files lets you save and reuse program statements without retyping them in the Command window.

2.4.1 Text Files

MATLAB uses text files as a permanent means of saving scripts (sets of instructions) rather than just entering commands in the Command window. As you will see in Chapter 8, text files are streams of characters stored sequentially with “markers” that indicate the end of each line of text. For now, think of a script much like writing an e-mail message—a number of lines of text written in a “smart” editor. The MATLAB Editor uses various techniques to help you format commands in these files.

2.4.2 Creating Scripts

A MATLAB script consists of a combination of executable instructions that MATLAB interprets and comment statements that help readers understand the script. You create **comments** by putting a percent sign (%) in the text file. MATLAB will ignore all text from that mark to the end of the current line. The MATLAB editor colors all such comments green to distinguish them from the executable instructions.

Most application programs that use text files specify a particular file name extension (the characters after the period in the file name) to identify how the text files will be used. MATLAB uses the extension `.m`, and its script files are often referred to as **m-files**.

In MATLAB you create a new script file either by choosing File > New > M-File or by clicking the new file icon on the far left of the tool bar. The MATLAB Editor will then open a blank file in which you can enter the commands and comments of your script. Try creating the script described in Exercise 2.14 and shown in Listing 2.1.



Exercise 2.14 Creating a script

Do It Yourself

Create a script derived from the Pythagorean theorem to compute the hypotenuse of a right triangle:

$$H^2 = A^2 + B^2$$

where `A` and `B` are the sides adjacent to the right angle, and `H` is the hypotenuse opposite.

Open a new script file and type the commands shown in Listing 2.1 (don't type the accompanying line numbers—MATLAB automatically displays these).

Listing 2.1 Script to solve for the hypotenuse

```
1. clear
2. clc
3.
4. A = 3;    % the first side of a triangle
5. B = 4;    % the second side of a triangle
6. hypSq = A^2 + B^2;    % the square of the
7.                    % hypotenuse
8. H = sqrt(hypSq)      % the answer
```

In Listing 2.1:

Line 1: Instructs MATLAB to delete all variables in your working directory.

Line 2: Instructs MATLAB to clear the Command window. Any text that now appears in the Command window will be the result of running this script, not the result of previous activities.

Line 3: Blank lines help improve the readability of a script.

Lines 4 and 5: Assign values to A and B. The semicolon prevents MATLAB from displaying the result in the Command window; the percent sign begins the legible comment.

Line 6: Intermediate results with suitable names sometimes improve the legibility of the algorithm.

Line 7: Lines may contain nothing but a comment.

Line 8: Invokes the built-in library function `sqrt(...)` to compute the final result.

2.4.3 The Current Directory

After you have entered a script, you must name it and save it in a directory. MATLAB will need to find that directory—its working directory—in order to run the script. By default, MATLAB expects scripts to be stored in the work directory, with a path like `c:\MATLABxxx\work`. The specific path will vary with your version of MATLAB. However, the Current Directory window circled in Figure 2.7 always shows the default location when MATLAB starts. If you decide to store your scripts elsewhere, you will need to redirect MATLAB to that directory by typing it into the Current Directory window or using the browse feature.

Once script files are saved in your working directory, you can edit them again by selecting and opening them with the MATLAB editor. To open them, either use the File > Open menu command or double-click the file name in the current directory window. Do Exercise 2.15 to understand how to save your script.

**Exercise 2.15** Saving a script**Do It Yourself**

Save the file you just created with the name *hypotenuse.m*.

2.4.4 Running Scripts

After you have built and saved a script, you can run it using any of the following methods:

- Type the name of the script in the Command window.
- Choose the Debug > Run menu item in the MATLAB Editor window.
- Press the F5 key when the script is visible in the editor. Doing this saves the script automatically before executing it.

After you execute the script, the trace output is written to the Command window as if you had typed the script instructions there one at a time. Do Exercise 2.16 to run your script.

2.4.5 Punctuating Scripts

Many programming languages put a semicolon (;) at the end of a line—whether in the Command window or a text file—to indicate the end of a command. Since MATLAB uses the end of a line to indicate the end of a command, it does not require an end-of-command character. If a long command needs to be extended to the next line for convenience in viewing the program, three periods, frequently referred to as **ellipses**, must be placed at the end of the line to continue the script.

MATLAB uses the semicolon for a different purpose. By default, all assignment commands display their results in the Command window in text form. For more complex programs, the volume of this output can become large. Whenever you really don't want to see all that output, putting a semicolon character at the end of a command line will prevent the results of that assignment from displaying in the Command window.

**Exercise 2.16** Running a script**Do It Yourself**

Run the hypotenuse script by one of the methods just discussed in Section 2.4.4. You should see the following in the Command window:

```
H =  
    5
```

Now run the script using the other two methods.

2.4.6 Debugging Scripts

MATLAB provides extensive debugging capabilities based on the use of **break points**, which are places in your program where you want to verify that the code is doing what you expect. You insert break points as you edit a code segment by clicking the small dash between the line number and the start of the text. If the program is ready to run, a red dot appears in place of

the dash where you clicked. If the file has been changed and hasn't been saved, the dot will be gray, in which case you should save the file and then enter the break point. You can set any number of break points throughout your code.

After you start running a program, when MATLAB reaches a break point, execution stops, an arrow overwrites

the break point symbol, and you can examine the contents of the variables either in the Workspace window or by passing the mouse slowly over the variable in the Editor window. A Debugging tool bar is then available with icons that let you:

- Continue executing the logic from this point (other break points may come into effect)
- Step over the logic in this line to the next line in this code block
- Step into any modules referenced by this line of code
- Step out of this current code block

Do Exercise 2.17 to practice inserting break points.

Style Points

1. When writing scripts, you should invest some time to add comments. Comments make the scripts easier to understand as you are developing them, and make it more likely that you will be able to reuse the script later.
2. Scripts should be written incrementally—build a little, test a little—rather than writing a whole script and then trying to find out where in that pile of code you made the mistake(s).

Common Pitfalls

You will quickly become accustomed to understanding the general flow of your script by observing the assignment statements reported in the Command window. However, especially if you have programmed in a language that requires semicolons at the end of a command, you may inadvertently put semicolons in your script. These will suppress the presentation of results and could mislead you into believing that a specific set of instructions haven't been executed.



Exercise 2.17 Debugging a script

Do It Yourself

Put a break point at Line 5 of the script in Listing 2.1 and run the script again. Observe that the code does not complete; it stops with a yellow arrow at the break point. Examine the Workspace window for the values of the variables and step through the rest of the script one line at a time, noting the display of new variables in the Workspace window and their updated values.

2.5 Engineering Example—Spacecraft Launch

In 1996, the Ansari X Prize was offered for the first time for a private venture: a reusable spacecraft. The requirements were for the same vehicle to carry three people into outer space twice in a two-week time period. The competition was won in 2004 by Tier 1, a company led by Burt Rutan. Their concept was to have a mother ship take off and land on a conventional runway carrying Space Ship One (see Figure 2.10). The spacecraft would be launched at 25,000 feet altitude and would reach outer space (an altitude of 100 km), then glide back and land on the same runway. They repeated this within a week, and they won the prize.



Figure 2.10 Space Ship One

Problem:

Assuming that the spacecraft uses all its fuel to achieve a vertical velocity u at 25,000 feet, what is the value of u for the spacecraft to reach outer space?

Solution:

There are two parts to this problem: Converting units to the metric system, and choosing and solving an equation for motion under constant acceleration (the rocket motor is no longer burning).

1. **Convert the launch altitude from feet to meters.** I like to remember as few numbers as possible. I do remember that 1 inch = 2.54 cm., so we will use this in a MATLAB script to find the conversion from feet to meters. The appropriate chain of calculations is this:

$$\frac{\text{meters}}{\text{foot}} \times \frac{\text{meters}}{\text{cm}} \times \frac{\text{cm}}{\text{inch}} \times \frac{\text{inch}}{\text{foot}}$$

Style Points

Notice that when presented in this manner, the “inner values” like *cm* and *inch* cancel to ensure that the conversions are consistent.

Listing 2.2 shows the beginning of the script to solve this problem.

In Listing 2.2:

Lines 1 and 2: Should be at the beginning of any script.

Listing 2.2 Script to compute the spacecraft's velocity (Part 1)

```

1. clear
2. clc

3. cmPerInch = 2.54;      % general knowledge
4. inchesPerFt = 12;      % general knowledge
5. metersPerCm = 1/100;   % general knowledge
6. MetersPerFt = metersPerCm * cmPerInch * inchesPerFt;

7. startFt = 25000;      % ft - given
8. startM = startFt * MetersPerFt;

```

Lines 3–5: Define general knowledge with meaningful variable names to enable subsequent use of these values without ambiguity.

Line 6: The conversion factor needed. Notice that because the variable names are consistent with the logic, they help to avoid errors.

2. ***Find and solve the equation.*** Given the following:

- Initial and final altitudes from which you can compute the distance traveled: s
- The motion is under constant acceleration, the force of gravity: g
- To just reach outer space, the final velocity, v , is 0
- The initial velocity, u , is needed

So after some diligent head scratching, we remember the equation of motion under constant acceleration connecting u , v , s , and a is:

$$v^2 = u^2 + 2as$$

However, this is not yet in a useful form. For computers to be able to solve an equation, you need the unknown quantity on the left of the equation and everything known on the right. Since u is the unknown, we move this to the left side of the assignment, and organize the known quantities to the right. These are the final velocity, v (that is, 0) the given distance, s , and the acceleration, a . Since the positive direction for u and s is upwards, but gravity is downwards, we must use $a = -g$, and the equation can be transformed to:

$$u = \sqrt{2gs}$$

With this information you can now solve this problem. Listing 2.3 shows the rest of Listing 2.2 to complete this calculation.

Listing 2.3 Script to complete the computation of the spacecraft's velocity

```
9. g = 9.81; % m/sec^2
10. top = 100; % km - given

11. s = (top*1000) - startM; % m

12. initial_v = (2*g*s)^0.5 % the final answer
```

In Listing 2.3:

Line 9: The standard value for the acceleration due to gravity.

Line 10: The altitude of outer space is given in the problem statement.

Line 11: Compute the distance traveled, including the unit conversion from kilometers to meters. Note the optional, and in this case unnecessary, use of parentheses to define the order of operations.

Line 12: The final computation. The operator $\wedge$ is the MATLAB expression for exponentiation; x^y in MATLAB results in computing x^y . Notice that the parentheses are required here to force the multiplication to happen before the exponentiation.

Although most modern computing environments, including MATLAB, have tools that actually solve symbolic equations, these tools are not appropriate for an introduction to programming and will not be discussed in this book.



Chapter Summary

This chapter presented some fundamental notions of computing and introduced you to the nature of MATLAB, its user interface, and the fundamental tools for making programs work.

- Abstraction lets you refer to collections of data or instructions as a whole.
- An algorithm is a set of instructions at an appropriate level of abstraction for solving a specific problem.
- A data class describes the type of data and the nature of operations that can be performed on that data.
- An object is a specific instance of a class with specific values that can be assigned to a variable.
- The Command window lets you experiment with ideas by entering commands line-by-line and seeing immediate results.
- The Command History window lets you review and recall previous commands.

- The Workspace window lists the names, values, and class of your local variables.
- The Current Directory window lists the current files in the directory to which MATLAB is currently pointed.
- A Document window opens when a variable in the Workspace window is selected, to let you view and edit data items.
- The Graphics window presents data and/or images when invoked by programs.
- The Editor window lets you view and modify text files.
- Scripts provide the basic mechanism for implementing solutions to problems.



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
'abc'	Single quotes enclose a literal character string	2.2.3
%	A percent sign indicates a comment in an M-file	2.4.2
;	A semicolon suppresses output from assignment statements	2.4.5
...	Ellipses continue a MATLAB command to the next line	2.4.5
=	The assignment operator assigns a value to a memory location; this isn't the same as an equality test	2.2.2
ans	The default variable name for results of MATLAB calculations	2.3.1
clc	Clears the Command window	2.3.2
clear	Clears the Workspace window	2.4.2
sqrt(x)	Calculates the square root of x	2.4.2



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. A bag of groceries is an example of abstraction.
2. An algorithm is a series of logical steps that solves one specific problem.

3. It is impossible to write a complete, practical program in any paradigm other than procedural.
4. To be useful to an algorithm, the result of every computation must be assigned to a variable.
5. In programming, if you know the values of z and x in the expression $z = x + y$, you can derive the value of y .
6. Untyped languages are free to ignore the nature of the data in variables.
7. Anything assigned to be the value of a variable is an object.
8. Class is a concept restricted to object-oriented programming.
9. You can permanently save the commands entered in the Command window.
10. Double-clicking an entry in the Command History window lets you rerun that command.
11. You can manually change the values of variables displayed in the Workspace window.
12. You double-click a file name in the Current Directory window to run that script.
13. A Document window lets you view and edit data items.
14. MATLAB permits multiple Graphics windows to be open simultaneously.
15. An asterisk on the File Name tab in the Editor window indicates that this is a script that can be executed.
16. MATLAB echoes comments entered in a script in the Command window.
17. When the name of script is typed in the Command window, it will be saved if necessary before it is executed.

Fill in the Blanks

1. _____ means expressing a quality apart from a particular implementation.
2. _____ is a sequence of instructions for solving a problem.

3. Without _____, a programming solution can be mathematically proven to be correct.
4. Variable names must not begin with _____.
5. Armed with both the _____ and _____ of a variable, a compiler can do a better job of ensuring that the programmer isn't misinterpreting data.
6. An instance of a _____ is _____ that is usually stored in a variable of that _____.
7. You can _____ in the Command window in a manner similar to the way you _____ on a scientific calculator.
8. You _____ an entry in the Command History window to _____ that command.
9. The columns in the Workspace window show the _____ of the variable, its _____, and two indications of its _____.
10. You _____ the name of a file in the Current Directory window to edit that file.
11. A Document window opens automatically when you _____ a _____ in the Workspace window.
12. Graphics windows are created _____ when a _____ requests a graph.
13. You create comments by putting a _____ in the text file.
14. MATLAB will _____ all text from the comment mark to _____.



Programming Projects

1. You are given two sides of a triangle, $a = 4.5$ and $b = 6$. The angle between them is 35 degrees. Write a script to find the length of the third side and the area of the triangle.
2. In the bottom of the 9th inning, the bases are loaded and the Braves are down by three runs. Chipper Jones steps to the plate. Twice he swings and misses. The crowd heads for the exits. The next pitch is a fast ball down the middle. He swings and makes perfect contact with the ball, sending it up at a 45 degree towards the fence 400 ft away.
 - a. Write a script to determine how fast he must hit the ball to land at the base of the fence, neglecting the air resistance.
 - b. Perform a brief experiment to determine whether there was a better angle at which to hit the ball so that it could clear a 12 ft fence.
3. If an ice cream cone is 6 inches tall and its rim has a diameter of 2 inches, write a script to determine the weight of ice cream that can fit in the cone assuming that the ice cream above the cone is a perfect hemisphere. You may neglect the thickness of the cone material. Assume that a gallon of ice cream weighs 8 lb, and occupies 7.5 cubic feet.
4. Write a script that validates the relationship between $\sin \theta$, $\cos \theta$ and $\tan \theta$ by evaluating these functions at suitably chosen values of θ .
5. I like my shower to remain hot for hours at 100 deg F, but am too cheap to buy one of those on-demand hot water systems. I don't care how slowly the water runs. The water supply is at 50 deg F and the water heater is rated at 50,000 BTU/hour. Write a script to compute the maximum flow rate of my shower (in cubic feet per minute) that keeps the water temperature above 100 deg F.
6. It takes an average of 45 horse power to run an electric car at an average speed of 35 mph. Write a script to compute the electrical storage capacity of the battery system that would make this car practical for a 25 mile commute, recharging the batteries only at home at night when the electricity is cheap. How many D cell alkaline batteries would be needed for this?
7. You want to buy a \$300,000 home with 20% down payment. The current compound interest rate is 4.5%.
 - a. Write a script to determine:
 - the monthly payments for a 30 year loan,
 - the equivalent simple interest rate,
 - the total interest paid over the life of the loan.
 - b. Now, repeat the computation for a 15 year loan at 5%. Is this a better deal?

8. The distance from my house to my office is 1.5 miles. Every morning, I have to decide whether to take the bus that averages (once it arrives) 25 mph, or to walk. I can walk at 4 mph. Write a script to determine how frequently the buses should run to give them a 50% chance of getting me to the office faster than walking.
9. A glass has the shape of a truncated cone of height 5 inches. Its top diameter is 3.5 inches and its base diameter is 2 inches. If water is poured into the glass at 2 gallons per minute, write a script to calculate how long it takes to fill the glass to the brim. One gallon is 7.5 cubic feet.
10. You can calculate the aerodynamic drag on an object by the formula:

$$\text{Drag} = \frac{1}{2} \rho V^2 C_d S$$

The air density, ρ , is 1.3 kg/m^3 and the value of the drag area, $C_d S$, is a measure of the resistance of the object as it moves through the air. An object falling through air reaches terminal velocity when the aerodynamic drag equals the object's weight.

A sky diver weighing 80 kg has a $C_d S$ value of 0.7 when horizontal with arms and legs extended, and 0.15 when head down with arms and legs in line. One diver jumps from a plane at an altitude of 5,000 m in the horizontal position. 20 sec later, another diver jumps. Write a script to determine how much time the second diver must spend head down in order to catch up to the first diver. Also compute the height above the ground where they first meet. For simplicity, you may assume that the sky divers immediately reach their terminal velocity when jumping.

11. You are given a circle with radius 5 centered at $x = 1$, $y = 2$. You want to calculate the intersection of some lines with that circle. Write a MATLAB script to find the x and y coordinates of both points of intersection. You should test this code at least with these lines:

$$\begin{aligned} y &= 2x - 1 \\ y &= -2x - 10 \\ y &= x + 5.9054 \end{aligned}$$

Hint

1. The equation of a circle is $(x - a)^2 + (y - b)^2 = r^2$ where (a, b) is the center of the circle, and r is the radius.
2. The equation of a line is $y = m x + c$ where m is the slope of the line, and c is the intercept on the y axis.

Vectors and Arrays

Chapter Objectives

This chapter discusses the basic calculations involving rectangular collections of numbers in the form of vectors and arrays. For each of these collections, you will learn how to:

- Create them
- Manipulate them
- Access their elements
- Perform mathematical and logical operations on them

This study of arrays will introduce the first of many language characteristics that sets MATLAB apart from other languages: its ability to perform arithmetic and logical operations on collections of numbers as a whole. You need to understand how to create these collections, access the data in them, and manipulate the values in the collections with mathematical and logical operators. First, however, we need to understand the idea of functions built into the language.

- 3.1** Concept: Using Built-in Functions
- 3.2** Concept: Data Collections
 - 3.2.1 Data Abstraction
 - 3.2.2 Homogeneous Collection
- 3.3** MATLAB Vectors
 - 3.3.1 Creating a Vector
 - 3.3.2 Size of a Vector
 - 3.3.3 Indexing a Vector
 - 3.3.4 Shortening a Vector
 - 3.3.5 Operating on Vectors
- 3.4** Engineering Example—Forces and Moments
- 3.5** MATLAB Arrays
 - 3.5.1 Properties of an Array
 - 3.5.2 Creating an Array
 - 3.5.3 Accessing Elements of an Array
 - 3.5.4 Removing Elements of an Array
 - 3.5.5 Operating on Arrays
- 3.6** Engineering Example—Computing Soil Volume



3.1 Concept: Using Built-in Functions

We are familiar with the use of a trigonometric function like $\cos(\theta)$ that consumes an angle in radians and produces the cosine of that angle. In general, a function is a named collection of instructions that operates on the data provided to produce a result according to the specifications of that function. In Chapter 5 we will see how to write our own functions. In this chapter we will see the use of some of the functions built into MATLAB. At the end of each chapter that uses built-in functions, you will find a summary table listing the function specifications. For help on a specific function, you can type the following in the Command window:

```
>> help <function name>
```

where `<function name>` is the name of a MATLAB function. This will produce a detailed discussion of the capabilities of that function.



3.2 Concept: Data Collections

Chapter 2 showed how to perform mathematical operations on single data items. This section considers the concept of grouping data items in general, and then specifically considers two very common ways to group data: in arrays and in vectors, which are a powerful subset of arrays.

3.2.1 Data Abstraction

It is frequently convenient to refer to groups of data collectively, for example, “all the temperature readings for May” or “all the purchases from Wal-Mart.” This allows us not only to move these items around as a group, but also to consider mathematical or logical operations on these groups.

For example, we could discuss the average, maximum, or minimum temperatures for a month, or that the cost of the Wal-Mart purchases had gone up 3 percent.

3.2.2 Homogeneous Collection

In Chapter 7 we will encounter more general collection implementations that allow items in a collection to be of different data types. These first collection types, however, will be constrained to accept only items of the same data type. The collections with this constraint are called **homogeneous collections**.



3.3 MATLAB Vectors

A vector is the simplest means of grouping a collection of like data items. Initially we will consider vectors of numbers or logical values. Some languages

refer to vectors as *linear arrays* or *linear matrices*. As these names suggest, a vector is a one-dimensional grouping of data, as shown in Figure 3.1.

Individual items in a vector are usually referred to as its **elements**. Vector elements have two separate and distinct attributes that make them unique in a specific vector: their *numerical value* and their *position* in that vector. For example, the individual number 66 is in the third position in the vector in Figure 3.1. Its value is 66 and its index is 3. There may be other items in the vector with the value of 66, but no other item will be located in this vector at position 3.

3.3.1 Creating a Vector

There are two ways to create vectors that are directly analogous to the techniques for creating individual data items:

- Creating vectors as a series of constant values
- Producing new vectors by operating on existing vectors

The following shows how you can create vectors from constant values:

- Entering the values directly, for example, `A = [2, 5, 7, 1, 3]` (the commas are optional and are frequently omitted)
- Entering the values as a range of numbers using the colon operator, for example, `B = 1:3:20`, where the first number is the starting value, the second number is the increment, and the third number is the ending value (you may omit the increment if the increment you need is 1)
- Using the `linspace(...)` function to create a fixed number of values between two limits, for example, `C = linspace(0, 20, 11)`, where the first parameter is the lower limit, the second parameter is the upper limit, and the third parameter is the number of values in the vector
- Using the functions `zeros(1,n)`, `ones(1,n)`, `rand(1,n)` (uniformly distributed random numbers), and `randn(1,n)` (random numbers with normal distribution) to create vectors filled with 0, 1, or random values between 0 and 1

Try working with vectors in Exercise 3.1.

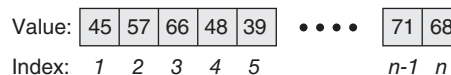


Figure 3.1 A vector



Exercise 3.1 Working with vectors

Do It Yourself

In the Command window, enter the following:

```
>> A = [2 5 7 1 3]
A =
```

continued on next page


```

    2   5   7   1   3
>> B = 1:3:20
B =
    1    4    7   10   13   16   19
>> C = linspace(0, 20, 11)
C =
    0    2    4    6    8   10   12   14   16   18   20
>> D = [4]
D =
    4
>> E = zeros(1,4)
E =
    0    0    0    0

```

Now, open the Workspace tab in the upper-left MATLAB window and study the contents.

The Workspace window gives you three pieces of information about each of the variables you created: the name, the value, and the “class,” which for now you can equate to “data type.”

Notice that if the size of the vector is small enough, the value field shows its actual contents; otherwise, you see a description of its attributes, like `1 x 11 double`.

Exercise 3.1 deliberately created the vector `D` with only one element, and perhaps the result surprised you. `D` was presented in both the Command window and the Workspace window as if it were a scalar quantity. This is generally true in MATLAB—all scalar quantities are considered vectors of unit length.

3.3.2 Size of a Vector

A vector also has a specific attribute: its length (n in Figure 3.1). In most implementations this length is fixed when the vector is created. However, as you will see shortly, MATLAB provides the ability to increase or decrease the size of a vector by inserting or deleting elements. MATLAB provides two functions to determine the size of arrays in general, and of vectors in particular. The function `size(v)` when applied to the vector `v` returns another vector containing two quantities: the number of rows in the vector (always 1) and the number of columns (the length of the vector). The function `length(v)` returns the maximum value in the size of an array—for a vector, this is a number indicating its length.

3.3.3 Indexing a Vector

As mentioned earlier, each element in a vector has two attributes: its value and its position in the vector. You can access the elements in a vector in either of two ways: by indexing with a numerical vector or indexing with a logical vector.

Numerical Indexing The elements of a vector can be accessed individually or in groups by enclosing the index of the one or more required elements in parentheses. Continuing Exercise 3.1, `A(3)` would return the third element, 7. If you attempt to read beyond the length of the vector or below index 1, an error will result.

You can also change the values of a vector element by using an assignment statement where the left-hand side indexes that specific element (try Exercise 3.2).

A feature unique to MATLAB is its behavior when attempting to write beyond the bounds of a vector. While it is still illegal to write below the index 1, MATLAB will automatically extend the vector if you write beyond its current end. If there are missing elements between the current vector elements and the index at which you attempt to store a new value, MATLAB will zero-fill the missing elements. Try Exercise 3.3 to see how this works.

In Exercise 3.3 we asked to store a value in the eighth element of a vector with length 5. Rather than complaining, MATLAB was able to complete the instruction by doing two things automatically: it extended the length to 8 and stored the value 0 in the as yet unassigned elements.

This process is referred to as **indexing a vector**. In these simple examples, we used a single number as the index. However, in general, we can use a vector of index values to index another vector. Furthermore, the size of the index vector does not need to match the size of the vector being indexed—it can be either shorter or longer. However, all values in an index vector must



Exercise 3.2 Changing elements of a vector

Do It Yourself

Extending the exercise above:

```
>> A(5) = 42
A =
    2     5     7     1    42
```



Exercise 3.3 Extending a vector

Do It Yourself

Again extending the exercises above:

```
>> A(8) = 3
A =
    2     5     7     1    42     0     0     3
```

be positive, and if they are being used to extract values from a vector, the index values must not exceed the length of that vector.

Logical Indexing So far, the only type of data we have used has been numerical values of type `double`. The result of a logical operation, however, is data of a different type, with values either `true` or `false`. Such data are called **Boolean** or **logical** values. Like numbers, logical values can be assembled into arrays by specifying true or false values. For example, we might specify the variable `mask` as follows:

```
>> mask = [true false false true]
mask =
     1     0     0     1
```

Notice that the Command window echoes the values of logical variables as if 1 represented true and 0 represented false.

We can index any vector with a logical vector as follows:

```
>> A = [2 4 6 8 10];
>> A(mask)
ans =
     2     8
```

When indexing with a logical vector, the result will contain the elements of the original vector corresponding in position to the `true` values in the logical index vector. The logical index vector can be shorter than the source vector, but it cannot be longer.

3.3.4 Shortening a Vector

There are times when we need to remove elements from a vector. For example, if we had a vector of measurements from an instrument, and it was known that the setup for the third reading was incorrect, we would want to remove that erroneous reading before processing the data. To accomplish this we make a rather strange use of the empty vector, `[]`. The empty vector, as its name and symbol suggest, is a vector with no elements in it. When you assign the empty vector to an element in another vector—say, `A`—that element is removed from `A`, and `A` is shortened by one element. Try shortening the vector in Exercise 3.4.



Exercise 3.4 Shortening a vector

Do It Yourself

Using the vector `A` from Exercise 3.3:

```
>> A(4) = []
A =
     2     5     7    42     0     0     3
```

Common Pitfalls

Shortening a vector is very rarely the right solution to a problem, and can lead to logical difficulties. Wherever possible, use indexing to copy the elements you want to keep.

As you can see, we asked for the fourth element to be removed from a vector initially with eight elements. The resulting vector has only seven elements, and the fourth element, originally with value 1, has been removed.

3.3.5 Operating on Vectors

The essential core of the MATLAB language is a rich collection of tools for manipulating vectors and arrays. This section first shows how these tools operate on vectors, and then generalizes to how they apply to arrays (multi-dimensional vectors) and, later, matrices.

Three techniques extend directly from operations on scalar values:

- Arithmetic operations
- Logical operations
- Applying library functions

Two techniques are unique to arrays in general, and to vectors in particular:

- Concatenation
- Slicing (generalized indexing)

Arithmetic Operations Arithmetic operations can be performed collectively on the individual components of two vectors as long as both vectors are the same length, or one of the vectors is a scalar (i.e., a vector of length 1). Addition and subtraction have exactly the syntax you would expect, as illustrated in Exercise 3.5. Multiplication, division, and exponentiation, however, have a small syntactic idiosyncrasy related to the fact that these are element-by-element, not matrix operations. We will discuss matrix operations in Chapter 12. When the MATLAB language was designed, the ordinary symbols were allocated to the matrix operations. However, since element-by-element multiplicative operations are fundamentally different from matrix operations, a new set of symbols is required for them. The symbols `.*`, `./`, and `.^` (the dots are part of the symbols, but the commas are not) are used respectively for element-by-element multiplication, division, and exponentiation. Because matrix and element-by-element addition and subtraction are identical, no special operation symbols are required for `+` and `-`.

**Exercise 3.5** Using vector mathematics**Do It Yourself**

In the Command window, enter the following:

```
>> A = [2 5 7 1 3];
>> A + 5
```

Continued on next page

```

ans =
    7    10    12     6     8
>> A .* 2
ans =
    4    10    14     2     6
>> B = -1:1:3
B =
   -1     0     1     2     3
>> A .* B % element-by-element multiplication
ans =
   -2     0     7     2     9
>> A * B % matrix multiplication!!
??? Error using ==> mtimes
Inner matrix dimensions must agree.
>> C = [1 2 3]
C =
     1     2     3
>> A .* C % A and C must have the same length
??? Error using ==> times
Matrix dimensions must agree.

```

Here, we first see the addition and multiplication of a vector by a scalar quantity, and then element-by-element multiplication of A and B. The first error is generated because we omitted the '.' on the multiply symbol, thereby invoking matrix multiplication, which is improper with the vector A and B. The second error occurs because two vectors involved in arithmetic operations must have the same size. Notice, incidentally, the use of the % sign indicating that the rest of the line is a comment.

You can change the signs of all the values of a vector with the unary minus (-) operator.

Logical Operations In the earlier discussion about logical indexing, you might have wondered why you would ever use that. In this section, we will see that logical operations on vectors produce vectors of logical results. We can then use these logical result vectors to index vectors in a style that makes the logic of complex expressions very clear.

As with arithmetic operations, logical operations can be performed element-by-element on two vectors as long as both vectors are the same length, or if one of the vectors is a scalar (i.e., a vector of length 1). The result will be a vector of logical values with the same length as the original vector(s). Try Exercise 3.6 to see how vector logical expressions work.

First we built the vectors A and B, and then we performed two legal logical operations: finding where A is not less than 5, and where each element of A is not less than the corresponding element of B. As with arithmetic operations, an error occurs if you attempt a logical operation with vectors of different sizes (neither size being 1).



Exercise 3.6 Working with vector logical expressions

Do It Yourself

In the Command window, enter the following:

```
>> A = [2 5 7 1 3];
>> B = [0 6 5 3 2];
>> A >= 5
ans =
     0     1     1     0     0
>> A >= B
ans =
     1     0     1     0     1
>> C = [1 2 3]
>> A > C
??? Error using ==> gt
Matrix dimensions must agree.
```

Logical operators can be assembled into more complex operations using logical and (&) and or (|) operators. These operators actually come in two flavors: &/| and && / ||. The single operators & and | operate on logical arrays of matching size to perform element-wise matches of the individual logical values. The doubled operators && and || combine individual logical results, and are usually associated with conditional statements (see Chapter 4). Try Exercise 3.7 to see how logical operators work.

In Exercise 3.7 we combine two logical vectors of the same length successfully, but fail, as with arithmetic operations, to combine vectors of different lengths.

If you need the indices in a vector where the elements of a logical vector are true, the function `find(...)` accomplishes this by consuming an array of logical values and producing a vector of the positions of the true elements. Try Exercise 3.8 to see how this function works.



Exercise 3.7 Working with logical vectors

Do It Yourself

In the Command window, enter the following:

```
>> A = [true true false false];
>> B = [true false true false];
>> A & B
ans =
     1     0     0     0
>> A | B
ans =
     1     1     1     0
>> C = [1 0 0]
>> A & C
??? Error using ==> and
Matrix dimensions must agree.
```



Exercise 3.8 Using the `find(...)` function

Do It Yourself

In the Command window, enter the following:

```
>> A = [2 5 7 1 3];
>> A > 4
ans =
    0 1 1 0 0
>> find(A > 4)
ans =
    2 3
```

You can negate the values of all elements of a logical vector (changing true to false and false to true) using the unary not operator, `~`. For example:

```
>> na = ~[true true false true]
na = 0 0 1 0
```

As you can see, each element of `na` is the logical inverse of the corresponding original element. As is usual with arithmetic and logical operations, the precedence of operators governs the order in which operations are performed. Table 3.1 shows the operator precedence in MATLAB. Operations listed on the same row of the table are performed from left to right. The normal precedence of operators can be overruled by enclosing preferred operations in parentheses: `(...)`.

Table 3.1 Operator precedence

Operators	Description
<code>.' , .^</code>	Scalar transpose and power
<code>' , ^</code>	Matrix transpose and power
<code>+, -, ~</code>	Unary operators
<code>.*, ./, .\, *, /, \</code>	Multiplication, division, left division
<code>+, -</code>	Addition and subtraction
<code>:</code>	Colon operator
<code><, <=, >=, >, ==, ~=</code>	Comparison
<code>&</code>	Element-wise AND
<code> </code>	Element-wise OR
<code>&&</code>	Logical AND
<code> </code>	Logical OR

Applying Library Functions MATLAB supplies the usual rich collection of mathematical functions that cover mathematical, trigonometric, and statistics capabilities. A partial list is provided in Appendix A. For a complete list, refer to the Help menu option in the MATLAB tool bar. All MATLAB functions accept vectors of numbers rather than single values and return a vector of the same length. The following functions deserve special mention because they provide specific capabilities that are frequently useful:

- `sum(v)` and `mean(v)` consume a vector and return the sum and mean of all the elements of the vector respectively.
- `min(v)` and `max(v)` return two quantities: the minimum or maximum value in a vector, as well as the position in that vector where that value occurred. For example:

```
>> [value where] = max([2 7 42 9 -4])
value = 42
where = 3
```

indicates that the largest value is 42, and it occurs in the third element of the vector. You will see in Chapter 5 how to implement returning multiple results from a function.

- `round(v)`, `ceil(v)`, `floor(v)`, and `fix(v)` remove the fractional part of the numbers in a vector by conventional rounding, rounding up, rounding down, and rounding toward zero, respectively.

Concatenation In Section 3.3.1 we saw the technique for creating a vector by assembling numbers between square brackets:

```
A = [2 5 7 1 3]
```

This is in fact a special case of concatenation. MATLAB lets you construct a new vector by concatenating other vectors:

```
A = [B C D ... X Y Z]
```

where the individual items in the brackets may be any vector defined as a constant or variable, and the length of `A` will be the sum of the lengths of the individual vectors. The simple vector constructor in Section 3.3.1 is a special case of this rule because each number is implicitly a 1×1 vector. The result is therefore a $1 \times N$ vector, where N is the number of values in the brackets. Try concatenating the vectors in Exercise 3.9.



Exercise 3.9 Concatenating vectors

Do It Yourself

In the Command window, enter the following:

```
>> A = [2 5 7];
>> B = [1 3];
>> [A B]
ans =
2 5 7 1 3
```


Notice that the resulting vector is not nested like `[[2 5 7], [1 3]]`, but is completely flat.

Slicing As mentioned earlier, the basic operation of extracting and replacing the elements of a vector is called indexing. However, indexing is not confined to single elements in a vector; you can also use **vectors of indices**. These index vectors either can be the values of previously named variables, or they can be created anonymously as they are needed. When you index a single element in a vector—for example, `A(4)`—you are actually creating an anonymous 1×1 index vector, 4, and then using it to extract the specified element(s) from the array `A`.

Creating anonymous index vectors as needed makes some additional features of the colon operator available. The general form for generating a vector of numbers is:

```
<start> : <increment> : <end>
```

We already know that by omitting the `<increment>` portion, the default increment is 1. When used anonymously while indexing a vector, the following features are also available:

- The key word `end` is defined as the length of the vector
- The operator `:` by itself is short for `1:end`

Finally, as you saw earlier, it is legal in MATLAB to index with a vector of logical values of the same length as, or shorter than, the vector being indexed. For example, if `A` is defined as:

```
A = [2 5 7 1 3];
```

then `A([false true false true])` returns:

```
ans =
     5     1
```

yielding a new vector containing only those values of the original vector where the corresponding logical index is `true`. This is extremely useful, as you will see later in this chapter, for indexing items in a vector that match a specific test.

The general form of statements for slicing vectors (moving sections of one vector into sections of another) is:

```
B(<rangeB>) = A(<rangeA>)
```

where `<rangeA>` and `<rangeB>` are both index vectors, `A` is an existing array, and `B` can be an existing array, a new array, or absent altogether (giving `B` the name `ans`). The values in `B` at the indices in `rangeB` are assigned the values of `A` from `rangeA`. The rules for use of this template are as follows:

- Either the size of `rangeB` must be equal to the size of `rangeA` or `rangeA` must be of size 1
- If `B` did not exist before this statement was implemented, it is zero filled where assignments were not explicitly made
- If `B` did exist before this statement, the values not directly assigned in `rangeB` remain unchanged

Study the comments in Listing 3.1 and do Exercise 3.10.

Listing 3.1 Vector indexing script

```

1. clear
2. clc
3. A = [2 5 7 1 3 4];
4. odds = 1:2:length(A);
5.
6. disp('odd values of A using predefined indices')
7. A(odds)
8. disp('odd values of A using anonymous indices')
9. A(1:2:end)
10. disp('put evens into odd values in a new array')
11. B(odds) = A(2:2:end)
12. disp('set the even values in B to 99')
13. B(2:2:end) = 99
14. disp('find the small values in A')
15. small = A < 4
16. disp('add 10 to the small values')
17. A(small) = A(small) + 10
18. disp('this can be done in one ugly operation')
19. A(A < 4) = A(A < 4) + 10

```



Exercise 3.10 Running the vector indexing script

Do It Yourself

Execute the script in Listing 3.1. You should see the following output in the Command window:

```

odd values of A using predefined indices
ans =
     2     7     3
odd values of A using anonymous indices
ans =
     2     7     3
put even values into odd values in a new array
B =
     5     0     1     0     4
set the even values in B to 99
B =
     5    99     1    99     4

```

continued on next page

```

find the small values in A
small =
     1     0     0     1     1     0
add 10 to the small values
A =
    12     5     7    11    13     4
this can be done in one ugly operation
A =
    12     5     7    11    13     4
>>

```

In Listing 3.1:

Lines 1 and 2: `clear` and `clc` should be the first two commands in a script.

Line 3: Creates a vector `A` with five elements.

Line 4: When predefining an index vector, if you want to refer to the size of a vector, you must use either the `length(...)` function or the `size` function.

Line 5: Blank to improve readability.

Line 6: The `disp(...)` function shows the contents of its parameter in the Command window, in this case: 'odd values of `A` using predefined indices'. We use `disp(...)` rather than comments because comments are visible only in the script itself, not in the program output, which we need here.

Line 7: Using a predefined index vector to access elements in vector `A`. Since no assignment is made, the variable `ans` takes on the value of a three-element vector containing the odd-numbered elements of `A`. Notice that these are the odd-numbered elements, not the elements with odd values.

Line 8: Again, the `disp(...)` function shows the contents of its parameter in the Command window. In this case, 'odd values of `A` using anonymous indices'.

Line 9: The anonymous version of the command given in Line 7. Notice that you can use the word `end` within the vector.

Line 10: Displays 'put evens into odd values in a new array'.

Line 11: Since `B` did not previously exist (a good reason to run the `clear` command at the beginning of a script is to be sure this is true), a new vector is created with five elements (the largest index assigned in `B`). Elements in `B` with fewer than five indices that were not assigned are zero filled.

Line 12: Displays 'set the even values in `B` to 99'.

Line 13: If you assign a scalar quantity to a range of indices in a vector, all values at those indices are assigned the scalar value.

Line 14: Displays 'find the small values in A'.

Line 15: Logical operations on a vector produce a vector of Boolean results. This is not the same as the line `small = [1 0 0 1 1 0]`. If you want to create a logical vector you must use true and false, for example:

```
small = [true false false true true false]
```

Line 16: Displays 'add 10 to the small values'.

Line 17: This is actually performing the scalar arithmetic operation + 10 on an anonymous vector of three elements, and then assigning those values to the range of elements in A.

Line 18: Displays 'this can be done in one ugly operation'.

Line 19: Not only is this unnecessarily complex, but it is also less efficient because it is applying the logical operator to A twice. It is better to use the form in Line 17.

3.4 Engineering Example—Forces and Moments

MATLAB vectors are ideal representations of the concept of a vector used in physics. Consider two forces acting on an object at a point P, as shown in Figure 3.2. Calculate the resultant force at P, the unit vector in the direction of that resultant, and the moment of that force about the point M. We can represent each of the vectors in this problem as a MATLAB vector with three components: the x, y, and z values of the vector. The solution to this problem for specific vectors is shown in Listing 3.2.

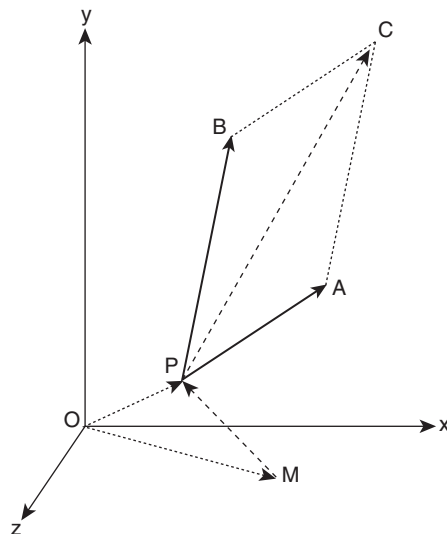


Figure 3.2 Vector analysis problem

Listing 3.2 Script to solve vector problems

```

1. clear
2. clc

3. PA = [0 1 1]
4. PB = [1 1 0]
5. P = [2 1 1]
6. M = [4 0 1]
7. % find the resultant of PA and PB
8. PC = PA + PB
9. % find the unit vector in the direction of PC
10. mag = sqrt(sum(PC.^2))
11. unit_vector = PC/mag
12. % find the moment of the force PC about M
13. % this is the cross product of MP and PC
14. MP = P - M
15. moment = cross( MP, PC )

```

In Listing 3.2:

Lines 1 and 2: The usual script beginning, followed by a blank line to improve readability.

Common Pitfalls

After any nontrivial computation, a good engineer will always perform a sanity check on the answers. When you run the code for this problem, the answers returned are:

```

PC = [ 1  2  1]
unit_vector = [0.4082  0.8165  0.4082]
moment = [ 1      2      -5]

```

To check the moment result, visualize the rotation of $\mathbf{PC}$ about $\mathbf{M}$ and apply the right-hand rule to find the axis of rotation of the moment. Roughly speaking, the right-hand rule states that the direction of the moment is the direction in which a normal, right-handed screw at point $\mathbf{M}$ would turn under the influence of this force. Without being too accurate, we can conclude that the axis of the moment is approximately along the negative z -axis, an estimate confirmed by the result shown.

Lines 3–6: Typical initial values for the problem.

Line 8: $\mathbf{PC}$ is the sum of the vectors $\mathbf{PA}$ and $\mathbf{PB}$.

Lines 10 and 11: The unit vector along $\mathbf{PC}$ is $\mathbf{PC}$ divided by its magnitude. The magnitude is the square root of the sum of the squares of the individual components.

Line 14: The vector $\mathbf{PM}$ is the vector difference between $\mathbf{P}$ and $\mathbf{M}$.

Line 15: MATLAB has a built-in function, `cross`, to compute the cross product of two vectors.

**3.5 MATLAB Arrays**

In Section 3.2 we saw that a vector is the simplest way to group a collection of similar data items. We will now extend these ideas to include arrays of multiple dimensions, initially confined to two dimensions. Each row will have the same number of columns, and each column will have the same number of rows.

At this point, we will refer to these collections as *arrays* to distinguish them from the *matrices* discussed in Chapter 12. While arrays and matrices are stored in the same way, they differ in their multiplication, division, and exponentiation operations. Figure 3.3 illustrates a typical two-dimensional array **A** with *m* rows and *n* columns, commonly referred to as an $m \times n$ array.

3.5.1 Properties of an Array

As with vectors, individual items in an array are referred to as its *elements*. These elements also have the unique attributes combining their value and their position. In a two-dimensional array, the position will be the row and column (in that order) of the element. In general, in an *n*-dimensional array, the element position will be a vector of *n* index values.

When applied to an array **A** with *n* dimensions, the function `size` will return the information in one of two forms.

- If called with a single return value like `sz = size(A)`, it will return a vector of length *n* containing the size of each dimension of the array.
- If called with multiple return values like `[rows, cols] = size(A)`, it returns the individual array dimension up to the number of values requested. To avoid erroneous results, you should always provide as many variables as there are dimensions of the array.

The `length(...)` function returns the maximum dimension of the array. So if we created an array **A** dimensioned $2 \times 8 \times 3$, `size(A)` would return `[2 8 3]` and `length(A)` would return 8.

The transpose of an $m \times n$ array, indicated by the apostrophe character (`'`) placed after the array identifier, returns an $n \times m$ array with the values in the rows and columns interchanged. Figure 3.4 shows a transposed array.

$$A_{(m \times n)} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Figure 3.3 An array

$$A'_{(n \times m)} = \begin{bmatrix} a_{11} & a_{21} & \dots & a_{m1} \\ a_{12} & a_{22} & \dots & a_{m2} \\ \vdots & & \ddots & \\ a_{1n} & a_{2n} & \dots & a_{mn} \end{bmatrix}$$

Figure 3.4 Transpose of an array

A number of special cases arise that are worthy of note:

- When a 2-D matrix has the same number of rows and columns, it is called **square**.
- When the only nonzero values in an array occur when the row and column indices are the same, the array is called **diagonal**.
- When there is only one row, the array is a **row vector**, or just a *vector* as you saw earlier.
- When there is only one column, the array is a **column vector**, the transpose of a row vector.

3.5.2 Creating an Array

Arrays can be created either by entering values directly, or by using one of a number of built-in MATLAB functions that create arrays with specific characteristics.

- As with vectors, you can directly enter the values in an array using either a semicolon (;) or a new line to indicate the end of a row, for example: `A = [2, 5, 7; 1, 3, 42]`.
- The functions `zeros(m, n)` and `ones(m, n)` create arrays with `m` rows and `n` columns filled with zeros and ones respectively.
- The functions `rand(m, n)` and `randn(m, n)` fill an array with random numbers in the range 0 .. 1.
- The function `diag(...)` takes several forms, the most useful of which are `diag(A)`, where `A` is an array, that returns its diagonal as a vector, and `diag(v)`, where `v` is a vector, that returns a square matrix with that diagonal.
- MATLAB also provides the function `magic(m)`, which fills a square matrix with the numbers 1 to m^2 organized in such a way that its rows, columns, and diagonals all add up to the same value. Try Exercise 3.11 to practice working with arrays.



Exercise 3.11 Creating arrays

Do It Yourself

Enter the following in the Command window and observe the results:

```
>> A = [2, 5, 7; 1, 3, 42]
A =
     2     5     7
     1     3    42
>> z = zeros(3,2)
z =
     0     0
     0     0
     0     0
>> [z ones(3, 4)] % concatenating arrays
ans =
     0     0     1     1     1     1
     0     0     1     1     1     1
     0     0     1     1     1     1
>> rand(3,4)
ans =
    0.9501    0.4860    0.4565    0.4447
    0.2311    0.8913    0.0185    0.6154
    0.6068    0.7621    0.8214    0.7919
>> rand(size(A))
ans =
    0.9218    0.1763    0.9355
    0.7382    0.4057    0.9169
>> diag(A)
ans =
     2
     3
>> diag(diag(A))
ans =
     2     0
     0     3
>> magic(4)
ans =
    16     2     3    13
     5    11    10     8
     9     7     6    12
     4    14    15     1
>>
```

3.5.3 Accessing Elements of an Array

The elements of an array may be addressed by enclosing the indices of the required element in parentheses, with the first index being the row index and the second index the column index. Considering the values produced by Example 3.11, `A(2, 3)` would return the element in the second row, third column: 42. If you were to attempt to read outside the length of the rows or columns, an error would result.

We can also store values that are elements of an array. For example, continuing Example 3.11, $A(2, 3) = 0$ would result in this answer:

```
A =
     2     5     7
     1     3     0
```

As with vectors, MATLAB will automatically extend the array if you write beyond its boundaries. If there are missing elements between the current array elements and the index at which you attempt to store a new value, MATLAB will zero fill the missing elements. For example, again continuing Example 3.11, $A(4, 1) = 3$ would result in this answer:

```
A =
     2     5     7
     1     3     0
     0     0     0
     3     0     0
```

3.5.4 Removing Elements of an Array

You can remove elements from arrays in the same way that you remove elements from a vector. However, since the arrays must remain rectangular, elements have to be removed as complete rows or columns. For example, for the array *A* in the previous section, entering $A(3, :) = []$ would remove all elements from the third row, and the result would be:

```
A =
     2     5     7
     1     3     0
     3     0     0
```

Similarly, if $A(:, 3) = []$ was then entered, the result would be:

```
A =
     2     5
     1     3
     3     0
```

Common Pitfalls

Removing rows or columns from an array is very rarely the right solution to a problem and can lead to logical difficulties. Wherever possible, use indexing to copy the rows and columns you want to keep.

3.5.5 Operating on Arrays

This section discusses how array operations extend directly from vector operations: arithmetic and logical operations, the application of functions, concatenation, and slicing. This section will also discuss two topics peculiar to arrays: reshaping and linearizing arrays.

Array Arithmetic Operations Arithmetic operations can be performed collectively on the individual components of two arrays as long as both arrays have the same dimensions or one of them is a scalar (i.e., has a vector of

Common Pitfalls

Performing array multiplication, division, or exponentiation without appending a dot operator requests one of the specialized matrix operations that will be covered in Chapter 12. The error message that displays when this occurs is quite obscure if you are not expecting it:

```
??? Error using ==> mtimes
Inner matrix dimensions must agree.
```

Even more obscure is the case where the dimensions of the arrays happen to be consistent (when multiplying square arrays), but the results are not scalar multiples.

length 1). Addition and subtraction have exactly the syntax you would expect, as shown in Exercise 3.12. Multiplication, division, and exponentiation, however, *must* use the “dot operator” symbols: `.*`, `./`, and `.^` (the dot is part of the symbol, but the commas are not) for scalar multiplication, division, and exponentiation.

Array Logical Operations As with vectors, logical array operations can be performed collectively on the individual components of two arrays as long as both arrays have the same dimensions or one of the arrays is a scalar (i.e., has a vector of length 1). The result will be an array of Boolean values with the same size as the original array(s). Do Exercise 3.13 to see how array logical operations work.

Here, we successfully compare the array *A* to a scalar value, and to the array *B* that has the same dimensions as *A*. However, comparing to the array *C* that has the same number of elements but the wrong shape produces an error.



Exercise 3.12 Working with array mathematics

Do It Yourself

Enter the following in the Command window and observe the results.

```
>> A = [2 5 7
        1 3 2]
A =
     2     5     7
     1     3     2
>> A + 5
ans =
     7    10    12
     6     8     7
B = ones(2, 3)
B =
     1     1     1
     1     1     1
>> B = B * 2
B =
     2     2     2
     2     2     2
>> A.*B % scalar multiplication
ans =
     4    10    14
     2     6     4
>> A*B % matrix multiplication does not work here
??? Error using ==> mtimes
Inner matrix dimensions must agree.
```



Exercise 3.13 Working with array logical operations

Do It Yourself

In the Command window, enter the following:

```
>> A = [2 5; 1 3]
A =
     2     5
     1     3
>> B = [0 6; 3 2];
>> A >= 4
ans =
     0     1
     0     0
>> A >= B
ans =
     1     0
     0     1
>> C = [1 2 3 4]
>> A > C
??? Error using ==> gt
Matrix dimensions must agree.
```

Applying Library Functions In addition to being able to consume vectors, most MATLAB mathematical functions can consume an array of numbers and return an array of the same shape. The following functions deserve special mention because they are exceptions to this rule and provide specific capabilities that are frequently useful:

- `sum(v)` and `mean(v)` when applied to a 2-D array return a row vector containing the sum and mean of each column of the array, respectively. If you want the sum of the whole array, use `sum(sum(v))`.
- `min(v)` and `max(v)` return two row vectors: the minimum or maximum value in each column and also the row in that column where that value occurred. For example:

```
>> [values rows] = max([2 7 42;
                        9 14 8;
                        10 12 -6])
values = [10 14 42]
rows = [3 2 1]
```

This indicates that the maximum values in each column are 10, 14, and 42 respectively, and they occur in rows 3, 2, and 1. If you really need the row and column containing, say, the maximum value of the whole array, continue the preceding example with the following lines:

```
>> [value col] = max(values)
value = 42
col = 3
```

This finds the maximum value in the whole array and determines that it occurs in column 3. So to determine the row in which that maximum occurred, we index the vector of row maximum locations, `rows`, with the column in which the maximum occurred.

```
>> row = rows(col)
row = 1
```

Therefore we correctly conclude that the maximum number in this array is 42, and it occurs at row 1, column 3.

Array Concatenation MATLAB permits programmers to construct a new array by concatenating other arrays in the following ways:

- Horizontally, as long as each component has the same number of rows:
`A = [B C D ... X Y Z]`
- Vertically, as long as each has the same number of columns:
`A = [B; C; D; ... X; Y; Z]`

The result will be an array with that number of rows and a number of columns equaling the sum of the number of columns in each individual item. Exercise 3.14 gives you the opportunity to concatenate an array.

Slicing Arrays The general form of statements for moving sections of one array into sections of another is as follows:

```
B(<rangeBR>, <rangeBC>) = A(<rangeAR>, <rangeAC>)
```

where each `<range..>` is an index vector, `A` is an existing array, and `B` can be an existing array, a new array, or absent altogether (giving `B` the name `ans`). The values in `B` at the specified indices are all assigned the corresponding values copied from `A`. The rules for using this template are as follows:

- Either each dimension of each sliced array must be equal, or the size of the slice from `A` must be 1×1 .
- If `B` did not exist before this statement was implemented, it is zero filled where assignments were not explicitly made.
- If `B` did exist before this statement, the values not directly assigned remain unchanged.



Exercise 3.14 Concatenating an array

Do It Yourself

In the Command window, enter the following:

```
>> A = [2 5; 1 7];
>> B = [1 3]';    % makes a column vector
>> [A B]
ans =
 2 5 1
 1 7 3
```

Reshaping Arrays Occasionally it is useful to take an array with one set of dimensions and reshape it to another set. The function `reshape( ... )` accomplishes this. The command:

```
reshape(A, rows, cols, ...)
```

will take the array `A`, whatever its dimensions, and reform it into an array sized:

```
(rows × cols × ...)
```

out to as many dimensions as desired. However, `reshape(...)` does not pad the data to fill any empty space. The product of all the original dimensions of `A` must equal the product of the new dimensions. Try Exercise 3.15 to see how to reshape an array.

Here, we first take a 1×10 array, `A`, and attempt to reshape it to 4×3 . Since the element count does not match, an error results. When we concatenate two zeros to the array `A`, it has the right element count and the reshape succeeds.

Linearized Arrays A discussion of arrays would not be complete without revealing an infamous MATLAB secret: multi-dimensional arrays are not stored in some nice, rectangular chunk of memory. Like all other blocks of memory, the block allocated for an array is sequential, and the array is stored in that space in column order. Normally, if MATLAB behaved as we “have a right to expect,” we would not care how an array is stored. However, there are circumstances under which the authors of MATLAB needed to expose this secret.

The primary situation in which array linearization becomes evident is the mechanization of the `find(...)` function. If we perform a logical operation



Exercise 3.15 Reshaping an array

Do It Yourself

In the Command window, enter the following:

```
>> A = 1:10
A =
     1     2     3     4     5     6     7     8     9    10
>> reshape(A, 4, 3)
??? Error using ==> reshape
To RESHAPE the number of elements must not change.
>> reshape([A 0 0], 4, 3)
ans =
     1     5     9
     2     6    10
     3     7     0
     4     8     0
```

on an array, the result is an array of logical values of the same size as the original array. In general, the `true` values would be scattered randomly about that result array. If we wanted to convert this to a collection of indices, what would we expect to see? An array of [row column] index pairs would be really awkward to process. Consequently, the developers of MATLAB decided to return a vector of the linear positions of the `true` values. Indexing with this result exposes the linearized nature of MATLAB arrays. The way this facet manifests itself is shown in Exercise 3.16.

Style Points

1. It is best not to expose the detailed steps of finding logical results in arrays, but to use an integrated approach:

```
5) = A(A>5) + 3
```

This produces the expected answers without exposing the nasty secrets underneath.

2. Never use an array linearization as part of your program logic. It makes the code hideous to look at and/or understand, and it is never the “only way to do” anything.

Here, we build a 4×3 array `A` and calculate the logical array where `A` is greater than 5. When we save the result of finding these locations in the variable `ix`, we see that this is a vector of values. If we count down the columns from the top left, we see that the second, seventh, eighth, and eleventh values in the linearized version of `A` are indeed `true`. We also see that it is legal to use this linearized index vector to access the values in the original array—in this case, to add 3 to each one. Finally, we would expect a loud complaint from

Exercise 3.16 Linearizing an array

Do It Yourself

In the Command window, enter the following:

```
>> A = [ 2 5 7 3
        8 0 9 42
        1 3 4 2 ]
A =
     2     5     7     3
     8     0     9    42
     1     3     4     2
>> A > 5
ans =
     0     0     1     0
     1     0     1     1
     0     0     0     0
>> ix = find(A > 5)
ix =
     2     7     8    11
>> A(ix) = A(ix) + 3
A =
     2     5    10     3
    11     0    12    45
     1     3     4     2
>> A(11)
ans =
    42    % (sigh!)
```

MATLAB when trying to reference the eleventh element of an array with only three rows. In fact MATLAB “unwinds” the storage of the array, counts down to the eleventh entry—3 for column 1, 3 for column 2, and 3 for column 3—and then extracts the second element of column 4.

To understand all these array manipulation ideas fully, you should work carefully through the script in Listing 3.3, study the explanatory notes that follow, and do Exercise 3.17.

Listing 3.3 Array manipulation script

```

1. A = [2 5 7 3
2.     1 3 4 2];
3. [rows, cols] = size(A);
4. odds = 1:2:cols;
5. disp('odd columns of A using predefined indices')
6. A(:, odds)
7. disp('odd columns of A using anonymous indices')
8. A(end, 1:2:end)
9. disp('put evens into odd values in a new array')
10. B(:, odds) = A(:, 2:2:end)
11. disp('set the even values in B to 99')
12. B(1, 2:2:end) = 99
13. disp('find the small values in A')
14. small = A < 4
15. disp('add 10 to the small values')
16. A(small) = A(small) + 10
17. disp('this can be done in one ugly operation')
18. A(A < 4) = A(A < 4) + 10
19. small_index = find(small)
20. A(small_index) = A(small_index) + 100

```



Exercise 3.17 Running the array manipulation script

Do It Yourself

Execute the script in Listing 3.3 and observe the results:

```

odds =
     1     3
odd columns of A using predefined indices
ans =
     2     7
     1     4
odd columns of A using anonymous indices
ans =
     1     4
put evens into odd values in a new array
B =
     5     0     3
     3     0     2
set the even values in B to 99

```

continued on next page

```

B =
     5     99     3
     3      0     2
find the small values in A
small =
     1      0      0      1
     1      1      0      1
add 10 to the small values
A =
    12      5      7     13
    11     13      4     12
this can be done in one ugly operation
A =
    12      5      7     13
    11     13      4     12
do the same thing with indices
small_index =
     1
     2
     4
     7
     8
A =
    112      5      7    113
    111     13      4    112

```

In Listing 3.3:

Lines 1 and 2: Create a 2×4 array **a**.

Line 3: Determines the number of rows and columns.

Line 4: Builds a vector **odds** containing the indices of the odd-numbered columns.

Line 6: Uses **odds** to access the columns in **a**. The **:** specifies that this is using all the rows.

Line 8: The anonymous version of the command in Line 7. Notice that you can use the word **end** in any dimension of the array to mean the end of that dimension.

Line 10: Because **b** did not previously exist (a good reason to have **clear** at the beginning of the script to be sure this is true), a new array is created. Elements in **b** that were not assigned are zero filled.

Line 12: Puts only 99 into selected locations in **b**.

Style Points

1. Do not forget to begin all scripts with the two commands **clear** and **clc**.

a. **clear** empties the current Workspace window of all variables and prevents the values of old variables from causing strange behavior in this script.

b. **clc** clears the Command window to prevent confusion about whether an output line was caused by this script or some earlier activity.

2. It is better to enter a few lines at a time and run each version of the script incrementally, rather than editing one huge script and running the whole thing for the first time. When you have added only a few lines to a previously working script, it is easy to locate the source of logic problems that arise.

3. It is very tempting to build large, complex vector operation expressions that solve messy problems “in one line of code.” While this might be an interesting mental exercise, the code is much more maintainable if the solution is expressed one step at a time using intermediate variables.

Line 14: Logical operations on arrays produce an array of logical results.

Line 16: Adds 10 to the values in `a` that are small.

Line 18: Not only is this unnecessarily complex, but it is also less efficient because it is applying the logical operator to `a` twice.

Line 19: The function `find(...)` actually returns a column vector of the index values in the linearized version of the original array, as shown in Exercise 3.16

Line 20: As illustrated above, it is not necessary to use `find(...)` before indexing an array. However, this does still work.

Notice that all the results are consistent with our expectations.

3.6 Engineering Example—Computing Soil Volume

Building a highway across rugged terrain requires careful planning to minimize the amount of dirt to be moved for cutting through hills and filling valleys. Significant amounts of money can be saved if the quantity of soil removed when cutting into a hillside can be used to fill in a nearby valley without transporting the soil too far.

However, this is not always possible. The City of Atlanta recently paid almost \$300 million for fill dirt in the construction of the fifth runway at Hartsfield-Jackson Airport because they had to make the level of the runway match the terrain. In other words, the runway had to be at the same ground level as the other runways and terminal areas so airplanes could taxi to and from the new runway. Figure 3.5 shows part of the conveyor that moved 27 million cubic yards of fill dirt over five miles from a quarry to the site of the new runway. At 2,600 cubic yards an hour, 7,000 tons a day, if the conveyor had run continuously night and day, it would have taken nearly a year and a half to move the dirt.



Figure 3.5 *Conveyer moving dirt*

Here is a slightly simpler problem: estimating the amount of soil that must be removed to prepare the foundations of a building. The first step is to survey the land on which the building is to be built, which results in a rectangular grid defining the altitude of each grid point as shown in Figure 3.6.

The next step is to consider an architectural drawing of the basement of the building as shown in Figure 3.7. The shaded areas indicate those places where the soil really must be removed to make the building foundation. We can estimate from this figure the fraction of each surveyed square (for our purposes, a number between 0 and 1) where the soil must actually be removed.

The total amount of soil to move is then the sum of the individual square depths multiplied by the area in each square to be removed. The code in Listing 3.4 solves this problem.

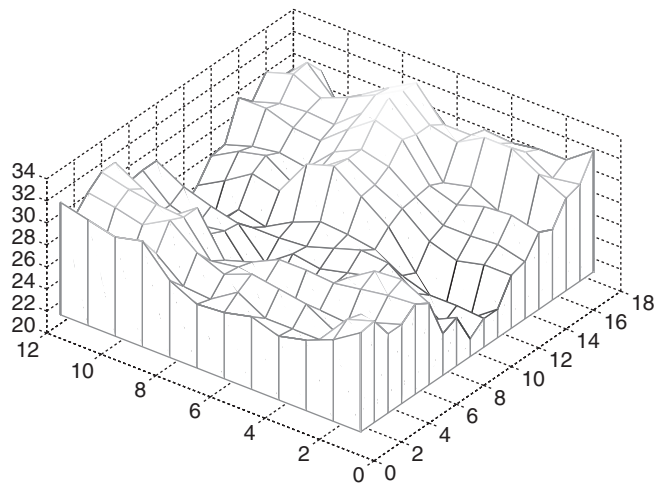


Figure 3.6 *Landscape survey*

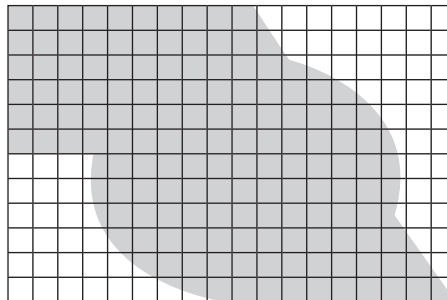


Figure 3.7 *Calculating soil volume*

Listing 3.4 Script to compute total soil

```

clear
clc
% soil depth data for each square produced by the survey
dpth = [8 8 9 8 8 8 8 8 7 8 7 7 7 7 8 8 8 8 7
        8 8 8 8 8 8 8 7 7 7 7 7 8 7 8 8 8 8 7
        8 8 8 8 7 7 8 7 8 8 8 8 8 7 8 8 8 8 8
        7 7 7 8 7 8 8 8 8 8 8 8 7 6 7 7 7 7 7
        8 8 8 8 8 8 8 8 7 7 7 7 7 6 6 7 7 8
        8 7 7 8 7 7 8 7 7 7 7 7 7 7 7 7 7 8
        9 8 8 9 8 7 8 7 7 7 7 7 6 7 6 7 7 8
        8 8 8 9 9 8 8 8 7 6 6 6 6 7 7 8 7 8
        9 8 8 7 7 7 7 7 7 6 6 7 7 7 8 8 7 8
        9 8 8 7 7 7 6 7 7 6 6 8 8 8 9 9 7 8
        9 9 8 8 8 8 7 7 7 7 7 8 8 9 9 9 8 8
        9 8 8 7 7 8 7 7 7 7 8 8 9 9 9 8 7 8];

% estimated proportion of each square that should be excavated
area = [1 1 1 1 1 1 1 1 1 1 .3 0 0 0 0 0 0 0
        1 1 1 1 1 1 1 1 1 1 .7 0 0 0 0 0 0 0
        1 1 1 1 1 1 1 1 1 1 1 .8 .4 0 0 0 0 0
        1 1 1 1 1 1 1 1 1 1 1 1 1 .8 .3 0 0 0
        1 1 1 1 1 1 1 1 1 1 1 1 1 1 .7 .2 0 0
        1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 .6 0 0
        0 0 0 .7 1 1 1 1 1 1 1 1 1 1 1 .8 0 0
        0 0 0 .7 1 1 1 1 1 1 1 1 1 1 1 .7 0 0
        0 0 0 .4 1 1 1 1 1 1 1 1 1 1 1 .6 0 0
        0 0 0 .1 .8 1 1 1 1 1 1 1 1 1 1 .4 0
        0 0 0 0 .2 .7 1 1 1 1 1 1 1 1 1 .9 .1
        0 0 0 0 0 0 .4 .8 .9 1 1 1 1 1 1 1 .6];

square_volume = dpth .* area;
total_soil = sum(sum(square_volume))

```

When you run this script, it produces the answer: 1,117.5 cubic units.

Common Pitfalls

The code in Listing 3.4 produces an answer around 1,120, and we should ask whether this is reasonable. There are 12×18 squares, each with area 1 unit, about 80 percent of which are to be excavated, giving a surface area of about 180 square units. The average depth of soil is about 7 units, so the answer ought to be about $180 \times 7 \cong 1,300$ cubic units. This is reasonably close to the computed result.



Chapter Summary

This chapter introduced you to vectors and arrays. For each collection, you saw how to:

- Create them by concatenation and a variety of special-purpose functions
- Access and remove elements, rows, or columns
- Perform mathematical and logical operations on them
- Apply library functions, including those that summarize whole columns or rows
- Move arbitrary selected rows and columns from one array to another
- Reshape and linearize arrays



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
[]	The empty vector	3.3.4
[...]	Concatenates data, vectors, and arrays	3.2.1
:	Specifies a vector as <code>from:incr:to</code>	3.2.1
:	Used in slicing vectors and arrays	3.3.5
()	Used with an array name to identify specific elements	3.3.3
'	Transposes an array	3.5.1
;	Separates rows in an array definition	3.5.2
+	Scalar and array addition	3.3.5
-	Scalar and array subtraction	3.3.5
-	Unary negation	3.3.5
.*	Array multiplication	3.3.5
./	Array division	3.3.5
.^	Array exponentiation	3.3.5
<	Less than	3.3.5
<=	Less than or equal to	3.3.5
>	Greater than	3.3.5
>=	Greater than or equal to	3.3.5
==	Equal to	3.3.5
~=	Not equal to	3.3.5

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>&</code>	Element-wise logical AND (vectors)	3.3.5
<code>&&</code>	Short-circuit logical AND (scalar)	3.3.5
<code> </code>	Element-wise logical OR (vectors)	3.3.5
<code> </code>	Short-circuit logical OR (scalar)	3.3.5
<code>~</code>	Unary not	3.3.5
<code>false</code>	Logical false	3.2.3
<code>help</code>	Invokes help utility	3.3.5
<code>true</code>	Logical true	3.2.3
<code>sum(a)</code>	Totals the values in <code>a</code>	3.3.5
<code>cross(a, b)</code>	Vector cross product	3.3
<code>disp(value)</code>	Displays an array or text	3.3.5
<code>find()</code>	Computes a vector of the locations of the true values in a logical array	3.3.5, 3.5.5
<code>linspace(fr,to,n)</code>	Defines a linearly spaced vector	3.2.1
<code>rand(r, c)</code>	Calculates an $r \times c$ array of evenly distributed random numbers in the range 0...1	3.2.1
<code>randn(r, c)</code>	Calculates an $r \times c$ array of normally distributed random numbers in the range 0...1	3.2.1
<code>[v,in] = max(a)</code>	Finds the maximum value and its position in <code>a</code>	3.3.5
<code>mean(a)</code>	Computes the average of the elements in <code>a</code>	3.3.5
<code>[v,in] = min(a)</code>	Finds the minimum value and its position in <code>a</code>	3.3.5
<code>diag(a)</code>	Extracts the diagonal from an array or, if provided with a vector, constructs an array with the given diagonal	3.5.2
<code>length(a)</code>	Determines the largest dimension of an array	3.2.2, 3.5.1
<code>magic(n)</code>	Generates a magic square	3.5.2
<code>ones(r, c)</code>	Generates an array filled with the value 1	3.2.1
<code>size(a)</code>	Determines the dimensions of an array	3.2.2, 3.5.1
<code>zeros(r, c)</code>	Builds an array filled with the value 0	3.2.1
<code>ceil(x)</code>	Rounds <code>x</code> to the nearest integer toward positive infinity	3.3.5
<code>fix(x)</code>	Rounds <code>x</code> to the nearest integer toward zero	3.3.5
<code>floor(x)</code>	Rounds <code>x</code> to the nearest integer toward minus infinity	3.3.5
<code>round(x)</code>	Rounds <code>x</code> to the nearest integer	3.3.5
<code>end</code>	Last element in a vector	3.3.5



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. A homogeneous collection must consist entirely of numbers.
2. The function `linspace(...)` can create only vectors, whereas the functions `zeros(...)`, `ones(...)`, and `rand(...)` produce either vectors or arrays of any dimension.
3. The `length(...)` function applied to a column vector gives you the number of rows.
4. You can access any element(s) of an array of any dimension using a single index vector.
5. Mathematical or logical operators are allowed only between two arrays of the same shape (rows and columns).
6. You can access data in a vector `A` with an index vector that is longer than `A`.
7. You can access data in a vector `A` with a logical vector that is longer than `A`.
8. When moving a block of data in the form of specified rows and columns from array `A` to array `B`, the shape of the block in `A` must match the shape of the block in `B`.

Fill in the Blanks

1. Vector elements have two attributes that make them unique: their _____ and their _____.
2. Vectors can be created using the colon operator, for example, `B = 1:3:20`, where the first number is the _____, the second number is the _____, and the third number is the _____.
3. When indexing a source vector with a logical vector, the result will contain the _____ of the source vector corresponding in position to the _____ in the logical vector.

4. The normal precedence of operators can be overruled by the use of _____.
5. Arithmetic operations can be performed collectively on the individual components of two arrays as long as both arrays _____ or one of them is _____.
6. To remove elements from arrays, you write _____ in _____.
7. Removing rows or columns from an array is _____, and can lead to _____. Wherever possible, use _____ to _____.



Programming Projects

1. For these exercises, do not use the direct entry method to construct the vectors. Write a script that does the following:
 - a. Construct a vector containing all of the even numbers between 6 and 33, inclusive of the end points. Store your answer in the variable `evens`.
(*Note:* 33 is not an even number)
 - b. Construct a vector, `threes`, containing every third number starting with 8 and ending at 38.
 - c. Construct a vector, `reverse`, containing numbers starting at 20 and counting backwards by ones to 10.
 - d. Construct a vector, `theta`, containing 100 evenly spaced values between-pi.
 - e. Construct a vector, `myZeros`, containing 15 elements, all of which are zeros.
 - f. Construct a vector, `random`, containing 15 randomly generated numbers between 1 and 2.

2. Write a script that performs the following exercises on vectors:

- a. You are given a vector `vec`, defined as:

```
vec = [45 8 2 6 98 55 45 -48 75];
```

You decide that you don't want the numbers with even values. Write the MATLAB commands to remove all of the even numbers (i.e., 8, 2, 6, 98, & -48) from `vec`. You should alter the vector `vec` rather than storing your answer in a new variable. Since your commands should work for any vector of any length, you should not use direct entry.

- b. Create a variable called `vLength` that holds the length of the vector `vec` modified in part a. You should use a built-in MATLAB function to calculate the value based on the vector itself.
 - c. Create a variable called `vSum` that holds the sum of the elements in vector `vec`. Do not just enter the value. You should use a built-in MATLAB function to calculate the value based on the vector itself.
 - d. Calculate the average of the values in the vector `vec` two ways. First, use a built-in MATLAB function to find the average of `vec`, then, use the results from parts b and c to calculate the average of `vec`.
 - e. Create a variable called `vProd` that holds the product of the elements in vector `vec`. You should use a built-in MATLAB function to calculate the value based on the vector itself.
3. Write a script to solve the following problems using only vector operations:

- a. Assume you have two vectors named `A1` and `B1` of equal length and create a vector `c1` that combines `A1` and `B1` such that

$$c1 = [A1(1) \ B1(1) \ A1(2) \ B1(2) \ . \ . \ . \ A1(end) \ B1(end)].$$

For example, if

`A1 = [2, 4, 8]` and `B1 = [3, 9, 27]`, `c1` should contain
`[2, 3, 4, 9, 8, 27]`

- b. Assume that you have two vectors named `A2` and `B2` of different lengths. Create a vector `c2` that combines `A2` and `B2` in a manner similar to part a. However, if you run out of elements in one of the vectors, `c2` also contains the elements remaining from the longer vector.

For example:

if `A2 = [1, 2, 3, 4, 5, 6]` and `B2 = [10, 20, 30]`, then `c2 =`
`[1, 10, 2, 20, 3, 30, 4, 5, 6]`

if `A2 = [1, 2, 3]` and `B2 = [10, 20, 30, 40, 50]`, then `c2 =`
`[1, 10, 2, 20, 3, 30, 40, 50]`

4. Write a MATLAB script that, when given a vector of numbers, `nums`, creates a vector `newNums` containing every other element of the original vector, starting with the first element. For example,

$$nums = [6 \ 3 \ 56 \ 7 \ 8 \ 9 \ 445 \ 6 \ 7 \ 437 \ 357 \ 5 \ 4 \ 3]$$
`newNums` should be `[6 56 8 445 7 357 4]`

Note: You must not simply hard-code the numbers into your answer; your script should work with any vector of numbers.

5. You are given a vector of test scores and wish to normalize these scores by computing a new vector, `normTests`, which will contain the test scores on linear scale from 0 to 100. A zero still corresponds to a zero, and the highest test score will correspond to 100.

For example,

```
tests = [90 45 76 21 85 97 91 84 79 67 76 72 89 95 55];
```

Hint

See the built-in MATLAB function `max()`

`normTests` should be

```
[92.78 46.39 78.35 21.65 87.63 100 93.81 86.6 ...  
81.44 69.07 78.35 74.23 91.75 97.94 56.7];
```

6. Write a script that takes a vector of numbers, `A`, and return a new vector `B`, containing the cubes of the positive numbers in `A`. If a particular entry is negative, replace its cube with 0.

Hint

It might be helpful as an intermediate step to create a logical vector indicating whether each element in `A` is negative.

For example:

If `A = [1 2 -1 5 6 7 -4 3 -2 0]`

`B` should be `[1 8 0 125 216 343 0 27 0 0]`

7. Great news! You have just been selected to appear on Jeopardy this fall. You decide that it might be to your advantage to use MATLAB to generate an array representing the values of the questions on the board.
 - a. Write a script to generate the matrix `jeopardy` that consists of 6 columns and 5 rows. The columns are all identical but the values of the rows range from 200 to 1000 in equal increments.
 - b. Next, generate the matrix `doubleJeopardy`, which has the same dimensions as `jeopardy` but whose values range from 400 to 2000.
 - c. You've decided to go even one step further and practice for a round that doesn't even exist yet. Generate the matrix `squaredJeopardy` that contains each entry of the original `jeopardy` matrix squared.

And now all that's left to do is practice your buzzer thumb.
8. Write a script named `arrayCollide` that will combine two arrays, sort them, and then return a new array of a specified size. Your script should process the following data:
 - A: a 2D array of any size
 - B: another 2D array that may be a different size from A

- N**: a number specifying the number of rows for the new array
- M**: a number specifying the number of columns for the new array.

Your script should produce an array, *res*, of size $N \times M$ that contains the first $N \times M$ elements of *A* and *B* and is sorted columnwise. If $N \times M$ is larger than the total number of elements in *A* and *B*, you should fill empty spots with 0.

Test this script by writing another script that repeatedly sets the values of *A*, *B*, *M* and *N* and then invokes your `arrayCollide` script. You can then create as many test cases as you wish.

Hint

Look at the `sort()` function.

For example:

```
If  A = [1 2 3; 5 4 6];
     B = [7 8; 9 10; 12 11];
     N = 3; M = 4;
```

```
res will be  [1  4  7  10
              2  5  8  11
              3  6  9  12]
```

Change *N* to 4, and *res* will be

```
[1  5  9  0
 2  6 10  0
 3  7 11  0
 4  8 12  0]
```


Execution Control

Chapter Objectives

This chapter discusses techniques for changing the flow of control of a program, which may be necessary for two reasons:

- You may want to execute some parts of the code under certain circumstances only
- You may want to repeat a section of code a certain number of times

In Chapter 3 we used the array notation to gather numbers into a form where they could be processed collectively rather than individually. This chapter deals with collections of lines of code (**code blocks**) that solve a particular segment of a problem in the same way. We will see how to define a code block, how to decide to execute a code block under certain conditions only, and how to repeat execution of a code block.

- 4.1 Concept: Code Blocks
- 4.2 Conditional Execution in General
- 4.3 `if` Statements
 - 4.3.1 General Template
 - 4.3.2 MATLAB Implementation
 - 4.3.3 Important Ideas
- 4.4 `switch` Statements
 - 4.4.1 General Template
 - 4.4.2 MATLAB Implementation
- 4.5 Iteration in General
- 4.6 `for` Loops
 - 4.6.1 General `for` Loop Template
 - 4.6.2 MATLAB Implementation
 - 4.6.3 Indexing Implementation
 - 4.6.4 Breaking a `for` Loop
- 4.7 `while` Loops
 - 4.7.1 General `while` Template
 - 4.7.2 MATLAB `while` Loop Implementation
 - 4.7.3 MATLAB Loop-and-a-Half Implementation
 - 4.7.4 Breaking a `while` Loop
- 4.8 Engineering Example—Computing Liquid Levels

4.1 Concept: Code Blocks

Some languages identify code blocks by enclosing them in braces (`{ . . . }`); others identify them by the level of indentation of the text. MATLAB uses the occurrence of key command words in the text to define the extent of code blocks. Keywords like `if`, `switch`, `while`, `for`, `case`, `otherwise`, `else`, `elseif`, and `end` are identified with blue coloring by the MATLAB text editor. They are not part of the code block, but they serve as instructions on what to do with the code block and as delimiters that define the extent of the code block.

4.2 Conditional Execution in General

To this point, the statements written in our scripts have been executed in sequence from the instruction at the top to the instruction at the bottom. However, it is frequently necessary to make choices about how to process a set of data based on some characteristic of that data. We have seen logical expressions that result in a Boolean result—`true` or `false`. This section discusses the code that implements the idea shown in Figure 4.1.

In the flowchart shown in Figure 4.1 a set of statements (the code block to be executed) is shown as a rectangle, a decision point is shown as a diamond, and the flow of program control is indicated by arrows. When decision points are drawn, there will be at least two arrows leaving that symbol, each labeled with the reason one would take that path.

This concept makes the execution of a code block conditional upon some test. If the result of the test is `true`, the code block is executed. Otherwise, the code block is omitted and the instruction(s) after the end of that code block are executed next.

An important generalization of this concept is shown in Figure 4.2. Here the solution is generalized to permit the first code block to be implemented under the first condition as before. Now, however, if that first logical test

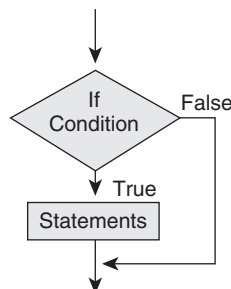


Figure 4.1 A simple conditional statement

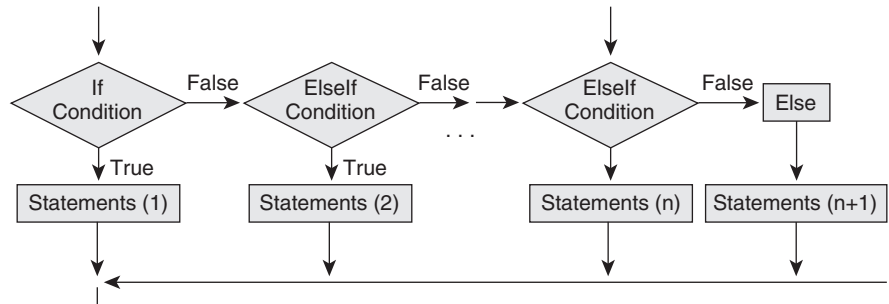


Figure 4.2 *A compound conditional statement*

returns `false`, a second test is performed to determine whether the second code block should be executed. If that test returns `false`, as many further tests as necessary may be performed, each with the appropriate code block to be implemented when the result is `true`. Finally, if none of these tests returns `true`, the last code block, usually identified by the `else` keyword— $(n + 1)$ in the figure—is executed. As the flowchart shows, as soon as one of the code blocks is executed, the next instruction to execute is the one that follows the conditional code after the `end` statement.

In particular, if there is no `else` clause, it is possible that no code at all is executed in this conditional statement.

There are two common styles in which to implement this conditional behavior. First we will discuss the most general form, the `if` statement, and then we will discuss the more restrictive, but tidier, `switch` statement. Both implementations are found in most modern languages, albeit with slightly different syntax. In each case, the code block to be implemented is all the statements between the introductory test statement and the next test statement or the end of the conditional.

4.3 if Statements

Here we introduce the concept of a programming template. Many programming texts still use the idea of **flowcharts**, such as those illustrated in Figures 4.1 and 4.2, to describe the design of a solution in a manner independent of the code implementation. However, since this graphical form cannot be maintained with a text editor, if the design of the solution changes during coding, it is difficult to maintain any design description that is separate from the code itself.

Throughout the remainder of this text, we will describe the overall design of a code module using a **design template**. Design templates are a textual form of flowchart consisting of the key words that control program flow and placeholders that identify the code blocks and expressions that are

necessary to implement the solution logic. Design templates are powerful tools for the novice programmer to overcome the “blank sheet of paper” problem—“how do I start solving this problem?” All programmers need to do is recognize the nature of the solution and write down the appropriate template. Then solving a particular problem becomes the relatively simple task of defining the code blocks identified by the template.

To discuss the `if` statement, first we consider its general, language-independent template and then its MATLAB implementation.

4.3.1 General Template

Template 4.1 shows the general template for the `if` statement.

Note the following:

- The only essential ingredients are the first `if` statement, one code block, and the `end` statement. All other features may be added as the logic requires.
- The code blocks may contain any sequence of legal MATLAB statements, including other `if` statements (nested `ifs`), `switch` statements, or iterations (see Section 4.5).
- Nested `if` statements with a code block are an alternative implementation of a logical `AND` statement.
- Recall that logical operations can be applied to a vector, resulting in a vector of Boolean values. This vector may be used as a logical expression. The `if` statement will accept this expression as `true` if all of the elements are `true`.

4.3.2 MATLAB Implementation

Listing 4.1 shows the MATLAB solution to a typical logical problem: determining whether a day is a weekday or a weekend day. It is assumed that the variable `day` is a number containing integer values from 1 to 7.

Template 4.1 General template for the `if` statement

```
if <logical expression 1>
    <code block 1>
elseif <logical expression 2>
    <code block 2>
    .
    .
    .
elseif <logical expression n>
    <code block n>
else
    <default code block>
end
```

Listing 4.1 if statement example

```
1. if day == 7           % Saturday
2.   state = 'weekend'
3. elseif day == 1      % Sunday
4.   state = 'weekend'
5. else
6.   state = 'weekday'
7. end
```

In Listing 4.1:

Line 1: The first logical expression determines whether `day` is 7.

Line 2: The corresponding code block sets the value of the variable `state` to the string `'weekend'`. In general, there can be as many statements within a code block as necessary.

Line 3: The second logical expression determines whether `day` is 6.

Line 4: The corresponding code block also sets the value of the variable `state` to the string `'weekend'`.

Line 5: The key word `else` introduces the default code block executed when none of the previous tests pass.

Line 6: The default code block sets the value of the variable `state` to the string `'weekday'`.

Exercise 4.1 gives you the opportunity to practice using `if` statements, and Listing 4.2 shows a script that will satisfy Exercise 4.1.

**Exercise 4.1** Using `if` statements***Do It Yourself***

Write a script that uses `input(...)` to request a numerical grade in percentage and uses `if` statements to convert that grade to a letter grade according to the following table:

90% and better: A

80%–90%: B

70%–80%: C

60%–70%: D

Below 60%: F

Test your script by running it repeatedly for legal and illegal values of the grade percentage.

Check your work against the script shown in Listing 4.2.

Listing 4.2 Script with `if` statements

```
1. grade = input('what grade? ');
2. if grade >= 90
3.     letter = 'A'
4. elseif grade >= 80
5.     letter = 'B'
6. elseif grade >= 70
7.     letter = 'C'
8. elseif grade >= 60
9.     letter = 'D'
10. else
11.     letter = 'F'
12. end
```

In Listing 4.2:

Line 1: Requests a grade value from the user with the `input(...)` function. The prompt appears in the Command window, and the system waits for a line of text from the user and converts that line as it would any other Command window line, returning the result to the variable `grade`.

Line 2: The first logical expression looks for the grade that earns an A.

Line 3: The corresponding code block sets the value of the variable `letter` to 'A'.

Lines 4–9: The corresponding logic for letter grades B, C, and D.

Lines 10–12: The default logic setting the variable `letter` to 'F'.

4.3.3 Important Ideas

There are two important ideas that are necessary for the successful implementation of `if` statements: the general form of the logical expressions and short-circuit analysis.

Logical Expressions The `if` statement requires a logical expression for its condition. A logical expression is any collection of constants, variables, and operators whose result is a Boolean `true` or `false` value. Unfortunately, the MATLAB Command window echoes Boolean results as 1 (`true`) or 0 (`false`). In fact, in many (but not all) cases MATLAB will treat an array of logical results as if they were numerical values.

Logical expressions can be created in the following ways:

- The value of a Boolean constant (e.g., `true` or `false`)
- The value of a variable containing a Boolean result (e.g., `found`)
- The result of a logical operation on two scalar quantities (e.g., `A > 5`)

- The result of logically negating a Boolean quantity using the unary negation operator (e.g., `~found`)
- The result of combining multiple scalar logical expressions with the operators `&&` or `||` (e.g., `A && B` or `A || B`)
- The results of the MATLAB functions that are the logical equivalent of the `&&`, `||`, and `~` operators: `and(A, B)` or `(A, B)` and `not(A)`
- The results of other MATLAB functions that operate on Boolean vectors: `any(...)` and `all(...)`

The result from `any(...)` will be `true` if any logical value in the vector is `true`. The result from `all(...)` will be `true` only if all logical values in the vector are `true`. The function `all(...)` is implicitly called by MATLAB if you supply a vector of logical values to the `if` statement, as shown in Listing 4.3.

In Listing 4.3:

Line 1: Makes the variable `A` a logical vector.

Line 2: Using this as a logical expression internally converts this expression to `all(A)`.

Line 3: All the values of `A` are not true; therefore, this code body does not execute.

Line 4: The end of the first code body and of the first `if` statement.

Line 5: Now, all the elements of `A` are true.

Lines 6–8: If we repeat the test, the code body will now execute.

Short-Circuit Evaluation When evaluating a sequence of logical `&&` or `||`, MATLAB will stop processing when it finds the first result that makes all subsequent processing irrelevant. This concept is best illustrated by an example. Assume that `A` and `B` are logical results and you want to evaluate `A && B`. Since the result of this is `true` only if both `A` and `B` are `true`, if you evaluate `A` and the result is `false`, no value of `B` can change the outcome `A && B`. Therefore there is no reason to evaluate any more components of a logical and expression once a `false` result has been found. Similarly, if you want `A || B`, if `A` is found to be `true`, you do not need to evaluate `B`.

Listing 4.3 The `if` statement with a logical vector

```
1. A = [true true false]
2. if A
3.     % will not execute
4. end
5. A(3) = true;
6. if A
7.     % will execute
8. end
```

For example, suppose you want to test the n th element of a vector v using a variable n , and you are concerned that n might not be a legal index value. The following code could be used:

```
if (n <= length(v)) && (v(n) > 0)
    % success!
end
```

If n were not a legal index, the indexed accessor $v(n)$ would cause an error for attempting to reach beyond the end of the vector. However, by putting the test of n first, the short-circuit logic would not process the second part of the expression if the test of n failed.



4.4 switch Statements

`switch` statements implement the logic shown in Figure 4.2 in a different programming style by allowing the programmer to consider a number of different cases for the value of one variable. First we consider the general, language-independent template for `switch` statements, and then its MATLAB implementation.

4.4.1 General Template

Template 4.2 shows the general template for the `switch` statement.

Note the following:

- All tests refer to the value of the same parameter
- `case` specifications may be either a single value or a set of parameters enclosed in braces { ... }
- `otherwise` specifies the code block to be executed when none of the case values apply
- The code blocks may contain any sequence of legal MATLAB statements, including other `if` statements (nested `ifs`), `switch` statements, or iterations

Template 4.2 General template for the `switch` statement

```
switch <parameter>
    case <case specification 1>
        <code block 1>
    case <case specification 2>
        <code block 2>
    .
    .
    case <case specification n>
        <code block n>
    otherwise
        <default code block>
end
```

4.4.2 MATLAB Implementation

Listing 4.4 shows the MATLAB implementation of a typical logical problem: determining the number of days in a month. It assumes that the value of month is 1 ... 12, and `leapYear` is a Boolean variable identifying the current year as a leap year.

Hint

The second parameter to the `input(...)` statement prevents MATLAB from attempting to parse the data provided, returning a string instead. Without that activity suppressed, if you enter the string 'yes', MATLAB will rush off looking for a variable by that name.

Style Points

1. The use of indentation is not required in MATLAB, and it has no significance with regard to syntax. However, the appropriate use of indentation greatly improves the legibility of code and you should use it. You have probably already noted that in addition to colorizing control statements, the MATLAB text editor automatically places the control statements in the indented positions illustrated in Listings 4.3 and 4.4.

2. It is good practice to include `otherwise` in a `switch` statement and `else` in an `if` statement to trap illegal values. The `otherwise` and `else` clauses should not be used in the main thread of your program unless you are confident that it is safe to use for all incoming values.

In Listing 4.4:

Line 1: All tests refer to the value of the variable `month`.

Line 2: This case specification is a cell array containing the numbers of the months with 30 days.

Line 4: The code blocks extend from the case statement to the next control statement (`case`, `otherwise`, or `end`).

Line 6: This code block contains an `if` statement to deal with the February case. It presumes that a Boolean variable `leapYear` has been created to indicate whether this month is in a leap year.

Line 11: The remaining months. The usual description of the logic suggests that this could be the `otherwise` clause. However, that would prevent you from being able to detect bad month number values, as this code does.

Listing 4.4 Example of a switch statement

```

1. switch month
2.     case {9, 4, 6, 11}
3.         % Sept, Apr, June, Nov
4.         days = 30;
5.     case 2 % Feb
6.         if leapYear
7.             days = 29;
8.         else
9.             days = 28;
10.        end
11.    case {1, 3, 5, 7, 8, 10, 12}
12.        % other months

```

continued on next page

```

13.             days = 31;
14.     otherwise
15.             error('bad month index')
16. end

```



Exercise 4.2 Using the switch statement

Do It Yourself

Write and test the script in Listing 4.4 using `input(...)` to request a numerical month value.

You will need to preset a value for `leapYear`.

Test your script by running it repeatedly for legal and illegal values of the month.

Modify your script to ask whether the current year is a leap year. (It's best to ask only for February.) You could use code like the following:

```

ans = input('leap year (yes/no)', 's');
leapYear = (ans(1) == 'y');

```

Test this new script thoroughly.

Try this script without the second parameter to `input(...)`. Can you explain what is happening?

Modify the script again to accept the year rather than yes/no, and implement the logic to determine whether that year is a leap year.

Line 15: A built-in MATLAB function that announces the error and terminates the script.

Try using the `switch` statement in Exercise 4.2.



4.5 Iteration in General

Iteration allows controlled repetition of a code block. Control statements at the beginning of the code block specify the manner and extent of the repetition:

- The `for` loop is designed to repeat its code block a fixed number of times and largely automates the process of managing the iteration.
- The `while` loop is more flexible in character. In contrast to the fixed repetition of the `for` loop, its code block can be repeated a variable number of times, depending on the values of data being processed. It is much more of a “do-it-yourself” iteration kit.

The `if` and `switch` statements allow us to decide to skip code blocks based on conditions in the data. The `for` and `while` constructs allow us to repeat code blocks. Note, however, that the MATLAB language is designed to avoid iteration. Under most circumstances of processing numbers, the array

processing operations built into the language make do-it-yourself loop constructs unnecessary.



4.6 for Loops

Figure 4.3 shows a simple `for` loop. The hexagonal shape illustrates the control of repetition. The repeated execution of the code block is performed under the control of a loop-control variable. It is first set to an initial value that is tested against a terminating condition. If the terminating test succeeds, the program leaves the `for` loop. Otherwise, the computations in the code block are performed using the current value of that variable. When one pass through the code block is finished, the variable is updated to its next value, and control returns to the termination test.

4.6.1 General for Loop Template

The general template for implementing `for` loops is shown in Template 4.3. All of the mechanics of iteration control are handled automatically in the variable specification section. In some languages—especially those with their origins in C—the variable specification is a formidable collection of statements that provide great generality of loop management. The designers of MATLAB, with its origins in matrix processing, chose a much simpler approach for specifying the variable range, as shown in the general template.

The repetition of the code block is managed completely by the specification of the loop control variable.

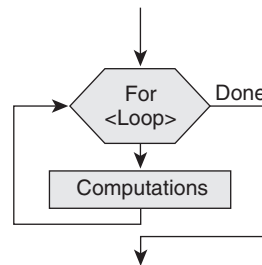


Figure 4.3 Structure of a `for` loop

Template 4.3 General template for the `for` statement

```

for <variable specification>
    <code block>
end
  
```

4.6.2 MATLAB Implementation

The core concept in the MATLAB `for` loop implementation is in the style of the variable specification, which is accomplished as follows:

`<variable specification>: <variable> = <vector>`

where `<variable>` is the name of the loop control variable and `<vector>` is any vector that can be created by the techniques discussed in Chapter 3. If we were to use the variable specification `x = A`, MATLAB would proceed as follows:

1. Set the value of `x` to `A(1)`
2. Evaluate the code block with that value of `x`
3. Advance `x` to the next value in `A`
4. Repeat Steps 2 and 3 until `x` exceeds the range of `A`

For a simple example of `for` loops, the code shown in Listing 4.5 solves a problem that should be done in a single MATLAB instruction: `max(A)` where `A` is a vector of integers. However, by expanding this into a `for` loop, we see the basic structure of the `for` loop at work.

In Listing 4.5:

Line 1: Creates a vector `A` with six elements.

Line 2: The tidiest way to find limits of a collection of numbers is to seed the result, `theMax`, with the first number, which avoids the problem of seeding the result with a value that is already outside the range of the vector. For example, we might think that `theMax = 0`; would be a satisfactory seed. However, this would not do well if all the elements of `A` were negative.

Line 3: The loop-control mechanism as described above.

Lines 4–6: The code block extends from the `for` statement to the associated `end` statement. The code will be executed the same number of times as the length of `A` *even if you change the value of `x` within the code block*. At each iteration the value of `x` will be set to the next element from the array `A`.

Listing 4.5 Example of a `for` statement

```

1. A = [6 12 6 91 13 6] % initial vector
2. theMax = A(1);      % set initial max value
3. for x = A            % iterate through A
4.     if x > theMax    % test each element
5.         theMax = x;
6.     end
7. end
8. fprintf('max(A) is %d\n', theMax);

```

Line 8: The `fprintf(...)` function is a very flexible means of formatting output to the Command window. See the discussion in Chapter 8, or enter the following in the Command window:

```
> help fprintf
```

4.6.3 Indexing Implementation

The above `for` loop implementation may seem very strange to those with a C-based language background, in which the loop-control variable is usually an index into the array being traversed rather than an element from that array. In order to illustrate the difference, we will adapt the code from Listing 4.5 to solve a slightly different problem that approximates the behavior of `max(A)`. This time we need to know not only the maximum value in the array, but also its index. This requires that we resort to indexing the array in a more conventional style, as shown in Listing 4.6.

In Listing 4.6:

Line 1: Generalizes the creation of the vector `A` using the `rand(...)` function to create a vector with 10 elements each between zero and 100.

Lines 2 and 3: Initialize `theMax` and `theIndex`.

Line 4: Creates an anonymous vector of indices from 1 to the length of `A` and uses it to define the loop-control variable, `index`.

Line 5: Extracts the appropriate element from `A` to operate with as before.

Lines 6 and 7: The same comparison logic as shown in Listing 4.5.

Line 8: In addition to saving the new `max` value, we save the index where it occurs.

Line 11: This is the first occurrence of an example where a logical line of code extends beyond the physical limitations of a single line.

Listing 4.6 `for` statement using indexing

```
1. A = floor(rand(1,10)*100)
2. theMax = A(1);
3. theIndex = 1;
4. for index = 1:length(A)
5.     x = A(index);
6.     if x > theMax
7.         theMax = x;
8.         theIndex = index;
9.     end
10. end
11. fprintf('the max value in A is %d at %d\n', ...
12.         theMax, theIndex);
```



Exercise 4.3 Producing for statement results

Do It Yourself

Enter and run the scripts in Listings 4.5 and 4.6. They should each produce the following results:

```
A =
    6 12 6 91 13 61 26 22 71 54
the max value in A is 91 at 4
>>
```

Since MATLAB normally uses the end of the line to indicate the end of an operation, we use the ellipses (. . .) to specify that the logic is continued onto the next line.

Style Points

We wrote the `for` loop examples in two styles: the direct access style and the indexing style. Many people code in the indexing style even when the index value is not explicitly required. This is slightly tacky and demonstrates some ignorance of the full power of the MATLAB language.

You can enter and run these scripts in Exercise 4.3.

4.6.4 Breaking a for Loop

If you are in a `for` loop and find a circumstance where you really do not want to continue iterating, the `break` statement will skip immediately out of the innermost containing `for` loop. If you want to continue iterating but omit all further steps of this iteration, you can use the `continue` statement.



4.7 while Loops

We use `while` loops in general for more control over the number of times the iteration is repeated. Figure 4.4 illustrates the control flow for a `while` loop. Since the termination test is performed before the loop is entered, the loop control expression must be initialized to a state that will normally permit loop entry. It is possible that the code block is not executed at all—for example—if there is no data to process.

4.7.1 General while Template

Template 4.4 shows the general template for implementing `while` loops. The logical expression controlling the iteration is testing some state of the workspace; therefore, two things that were automatic in the `for` loop must be

Template 4.4 General template for the while statement

```
<initialization>
while <logical expression>
    <code block> % must make some changes
                % to enable the loop to terminate
end
```

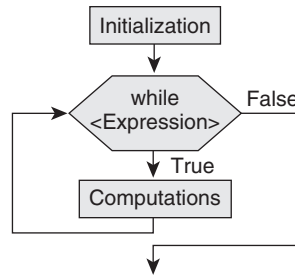


Figure 4.4 Structure of a while Loop

manually accomplished with the `while` loop: initializing the test and updating the workspace in the code block so that the test will eventually fail and the iteration will stop.

4.7.2 MATLAB `while` Loop Implementation

For the sake of consistency, Listing 4.7 shows you how to solve the same problem using the `while` syntax.

In Listing 4.7:

Lines 1–3: Creates a test vector and initializes the answers as before.

Line 4: Initializes the index value since this is manually updated.

Line 5: This test will fail immediately if the vector `A` is empty.

Line 6: Extracts the item `x` from the array (good practice in general to clarify your code).

Lines 7–9: The same test as before to update the maximum value.

Line 11: “Manually” updates the index to move the loop closer to finishing.

Enter and run the script as described in Exercise 4.4.

Listing 4.7 `while` statement example

```

1. A = floor(rand(1,10)*100)
2. theMax = A(1);
3. theIndex = 1;
4. index = 1;
5. while index <= length(A)
6.     x = A(index);
7.     if x > theMax
8.         theMax = x;
9.         theIndex = index;
10.    end
11.    index = index + 1;
12. end
13. fprintf('the max value in A is %d at %d\n', ...
14.         theMax, theIndex);
  
```



Exercise 4.4 Producing while statement results

Do It Yourself

Enter and run the script in Listing 4.7. It should produce the following results:

```
A =
    6 12 6 91 13 61 26 22 71 54
the max value in A is 91 at 4
>>
```

4.7.3 MATLAB Loop-and-a-Half Implementation

Listing 4.8 illustrates the implementation of the loop-and-a-half iteration style, in which we must enter the loop and perform some computation before realizing that we do not need to continue. Here we continually ask the user for the radius of a circle until an illegal radius is entered, which is our cue to terminate the iteration. For each radius entered, we want to display the area and circumference of the circle with that radius.

Style Points

The use of `break` and `continue` statements is frowned upon in programming circles for the same reason that the `goto` statement has fallen into disrepute—they make it more difficult to understand the flow of control through a complex program. It is preferable to express the logic for remaining in a `while` loop explicitly in its controlling Boolean expression, combined with `if` statements inside the loop to skip blocks of code. However, sometimes this latter approach causes code to be more complex than would be the case with judicious use of `break` or `continue`.

In Listing 4.8:

Line 1: Initializes the radius value to allow the loop to be entered the first time.

Line 2: We will remain in this loop until the user enters an illegal radius.

Line 3: The `input(...)` function shows the user the text string, translates what is typed, and stores the result in the variable provided. This is described fully in Chapter 8.

Line 4: We want to present the area and circumference only if the radius has a legal value. Since this test occurs in the middle of the `while` loop, we call this “loop-and-a-half” processing.

Try this script in Exercise 4.5.

Listing 4.8 Loop-and-a-half example

```
1. R = 1;
2. while R > 0
3.     R = input('Enter a radius: ');
4.     if R > 0
5.         area = pi * R^2;
6.         circum = 2 * pi * R;
7.         fprintf('area = %f; circum = %f\n', ...
8.             area, circum);
9.     end
10. end
```



Exercise 4.5 Producing loop-and-a-half test results

Do It Yourself

Enter and run the script in Listing 4.8. It should produce the following results:

```
Enter a radius: 4
area = 50.265482; circum = 25.132741
Enter a radius: 3
area = 28.274334; circum = 18.849556
Enter a radius: 100
area = 31415.926536; circum = 628.318531
Enter a radius: 0
>>
```

4.7.4 Breaking a while loop

As with the `for` loop, `break` will exit the innermost `while` loop, and `continue` will skip to the end of the loop but remain within it.



4.8 Engineering Example—Computing Liquid Levels

Figure 4.5 shows a cylindrical tank of height H and radius r with a spherical cap on each end (also of radius, r). If the height of the liquid is h , what is the volume of liquid in the tank?

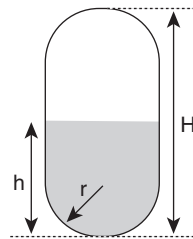


Figure 4.5 A tank containing liquid

Clearly, the calculation of the volume of liquid in the tank depends upon the relationship between h , H , and r :

- If h is less than r , we need the volume, v , of a partially filled sphere given by:

$$v = \frac{1}{3}\pi h^2(3r - h)$$

- If h is greater than r but less than $H - r$, we need the volume of a fully filled hemisphere plus the volume of a cylinder of height $h - r$:

$$v = \frac{2}{3}\pi r^3 + \pi r^2(h - r)$$

- If h is greater than $H - r$, we need the volume of a fully filled sphere plus the volume of a cylinder of height $H - 2r$ minus the partially empty upper hemisphere of height $H - h$:

$$v = \frac{4}{3}\pi r^3 + \pi r^2(H - 2r) - \frac{1}{3}\pi(H - h)^2(3r - H + h)$$

The script to perform this calculation is shown in Listing 4.9. Rather than performing the computations for one liquid level only, we should write the script so that we continue to consider tanks of different dimensions and different liquid heights for each tank until the user indicates that he needs no more results.

In Listing 4.9:

Line 1: Initializes the value to keep it in the first `while` loop.

Lines 3 and 4: Get the tank sizes.

Line 5: Initializes the value to keep it in the inner `while` loop.

Line 7: Gets the liquid height.

Lines 8–14: Calculations for legal values of h . Notice that no dot operators are required here, because these conditional computations will not work correctly with vectors of H , r , or h .

Lines 15 and 16: Illegal h values end up here.

Line 17: Goes to the end of the loop, skipping the printout.

Lines 19–21: Print the result.

Line 22: More heights when “y” is entered.

Line 24: Another tank when “y” is entered.

Listing 4.10 shows some typical results.

Listing 4.9 Script to compute liquid levels

```
1. another_tank = true;
2. while another_tank
3.     H = input('Overall tank height: ');
4.     r = input('tank radius: ');
5.     more_heights = true;
6.     while more_heights
7.         h = input('liquid height: ');
8.         if h < r
9.             v = (1/3)*pi*h.^2.*(3*r-h);
```

continued on next page

```

10.         elseif h < H-r
11.             v = (2/3)*pi*r^3 + pi*r^2*(h-r);
12.         elseif h <= H
13.             v = (4/3)*pi*r^3 + pi*r^2*(H-2*r) ...
14.                 - (1/3)*pi*(H-h)^2*(3*r-H+h);
15.         else
16.             disp('liquid level too high')
17.             continue
18.         end
19.         fprintf( ...
20.             'rad %0.2f ht %0.2f level %0.2f vol %0.2f\n', ...
21.             r,             H,             h,             v);
22.         more_heights = input('more levels? (y/n)','s')== 'y';
23.     end
24.     another_tank = input('another tank? (y/n)','s')== 'y';
25. end

```

Listing 4.10 Results for liquid levels

```

Overall tank height: 10
tank radius: 2
liquid height: 1
radius 2.00 height 10.00 level 1.00 vol 5.24
more levels? (y/n)y
liquid height: 2
radius 2.00 height 10.00 level 2.00 vol 16.76
more levels? (y/n)y
liquid height: 4
radius 2.00 height 10.00 level 4.00 vol 41.89
more levels? (y/n)y
liquid height: 8
radius 2.00 height 10.00 level 8.00 vol 92.15
more levels? (y/n)y
liquid height: 9
radius 2.00 height 10.00 level 9.00 vol 103.67
more levels? (y/n)y
liquid height: 10
radius 2.00 height 10.00 level 10.00 vol 108.91
more levels? (y/n)y
liquid height: 11
liquid level too high
liquid height: 10
radius 2.00 height 10.00 level 10.00 vol 108.91
more levels? (y/n)n
another tank? (y/n)y
Overall tank height: 8
tank radius: 4
liquid height: 4
radius 4.00 height 8.00 level 4.00 vol 134.04
more levels? (y/n)y
liquid height: 8
radius 4.00 height 8.00 level 8.00 vol 268.08
more levels? (y/n)
another tank? (y/n)

```



Chapter Summary

This chapter presented techniques for changing the flow of control of a program for condition execution and repetitive execution:

- The most general conditional form is the `if` statement, with or without the accompanying `elseif` and `else` statements
- The `switch` statement considers different cases of the values of a countable variable
- A `for` loop in its most basic form executes a code block for each of the elements of a vector
- A `while` loop repeats a code block a variable number of times, as long as the conditions specified for continuing the repetition remain true



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>false</code>	Logical false	4.2
<code>true</code>	Logical true	4.2
<code>break</code>	A command within a loop module that forces control to the statement following the innermost loop	4.6.4, 4.7.4
<code>case</code>	A specific value within a <code>switch</code> statement	4.1, 4.4.1
<code>continue</code>	Skips to the end of the innermost loop, but remains inside it	4.6.4, 4.7.4
<code>else</code>	Within an <code>if</code> statement, begins the code block executed when the condition is false	4.1, 4.3.2
<code>elseif</code>	Within an <code>if</code> statement, begins a second test when the first condition is false	4.1, 4.3.2
<code>end</code>	Terminates an <code>if</code> , <code>switch</code> , <code>for</code> or <code>while</code> module	4.1, 4.3.2, 4.4.1, 4.6
<code>for var = v</code>	A code module repeats as many times as there are elements in the vector <code>v</code>	4.1, 4.6
<code>if <exp></code>	Begins a conditional module; the following code block is executed if the logical expression <code><exp></code> is true	4.1, 4.3.2
<code>input(str)</code>	Requests input from the user	4.3.2
<code>otherwise</code>	Catch-all code block at the end of a <code>switch</code> statement	4.1, 4.4.1
<code>switch(variable)</code>	Begins a code module selecting specific values of the <code>variable</code> (must be countable)	4.1, 4.4.1

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>while <exp></code>	A code module repeats as long as the logical expression <code><exp></code> is true	4.1
<code>all(a)</code>	True if all the values in <code>a</code> , a logical vector, are true	4.3.3
<code>and(a, b)</code>	True if both <code>a</code> and <code>b</code> are true	4.3.3
<code>any(a)</code>	True if any of the values in <code>a</code> , a logical vector, is true	4.3.3
<code>not(a)</code>	True if <code>a</code> is false; false if <code>a</code> is true	4.3.3
<code>or(a, b)</code>	True if either <code>a</code> or <code>b</code> is true	4.3.3



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. MATLAB keywords are colored green by the editor.
2. Indentation is required in MATLAB to define code blocks.
3. It is possible that no code at all is executed by `if` or `switch` constructs.
4. The word `true` is a valid logical expression.
5. When evaluating a sequence of logical `&&` expressions, MATLAB will stop processing when it finds the first `true` result.
6. The `for` loop repeats the enclosed code block a fixed number of times even if you modify the index variable within the code block.
7. Using a `break` statement is illegal in a `while` loop.
8. The logical expression used in a `while` loop specifies the conditions for exiting the loop.

Fill in the Blanks

1. MATLAB uses _____ in the text to define the extent of code blocks.
2. The function _____ is implicitly called by MATLAB if you supply a vector of logical values to the `if` statement.

3. It is good practice to include _____ in a `switch` statement to trap illegal values entering the `switch`.
4. There is no reason to evaluate any more components of a logical or expression once a _____ result has been found.
5. A `while` loop can be repeated a _____ number of times, depending on the _____ being processed.
6. If you are in a _____ loop, you can use the `break` statement to skip immediately out of the _____ loop.



Programming Projects

1. Write a script to solve this problem. Assume you have a vector named `D`. Using iteration (`for` and/or `while`) and conditionals (`if` and/or `switch`), separate vector `D` into four vectors `posEven`, `negEven`, `posOdd` and `negOdd`.
 - `posEven` contains all of the positive even numbers in `D`.
 - `negEven` contains all of the negative even numbers in `D`.
 - `posOdd` contains all of the positive odd numbers in `D`.
 - `negOdd` contains all of the negative odd numbers in `D`.

For example:

```
if D = [-4,-3,-2,-1,0,1,2,3,4],
posEven=[2,4], negEven=[-4,-2],
posOdd=[1,3] and negOdd=[-3,-1]
```

2. You must use either `for` or `while` to solve the following problems.
 - a. Iterate through a vector, `A`, using a `for` loop and create a new vector, `B`, containing logical values. The new vector should contain `true` for positive values and `false` for all other values. For example, if


```
A = [-300 2 5 -63 4 0 -46] the result should be
B = [false true true false true true false]
```
 - b. Iterate through the vector, `A`, using a `while` loop and return a new vector, `B`, containing `true` for positive values and `false` for all other values.
 - c. Iterate through a logical array, `N`, using a `for` loop and return a new vector, `M`, containing the value 2 wherever an element of `N` is true and the value -1 (not a logical value) wherever `N` is false. For example, if

`N = [true false false true true false true]` the result should be `M = [2 -1 -1 2 2 -1 2]`

- d. Iterate through an array, `Z`, using a `while` loop. Replace every element with the number 3 until you reach a number larger than 50. Leave the rest unchanged. For example, if

`Z = [4 3 2 5 7 9 0 64 34 43]`, after running your script,

`Z = [3 3 3 3 3 3 3 3 34 43]`

3. You are hiring grad students from Tech to work for your company, which you have recently started. The Human Resources department has asked you to write a script that will help them determine the chances of an individual applicant getting a job after interviewing. The following table outlines the rules for determining the chances for the applicant to get a job:

GPA Value	Chance of Being Hired
$\text{GPA} \geq 3.5$	90%
$3.0 \leq \text{GPA} < 3.5$	80%
$2.5 \leq \text{GPA} < 3.0$	70%
$2.0 \leq \text{GPA} < 2.5$	60%
$1.5 \leq \text{GPA} < 2.0$	40%
$\text{GPA} < 1.5$	30%

Your script should repeatedly ask the user for a GPA value and compute the student's chances of being hired. It should continue asking for GPA values until a negative number is entered. For example:

- GPA input: 4 should give the answer 0.9
- GPA input: 3.5 should give the answer 0.9
- GPA input: 3.4 should give the answer 0.8

4. You were just hired for a summer internship with one Atlanta's best software companies; however, on your first day of work you learn that for the next 3 months, the only job you will have is to convert binary (base 2) numbers into decimal numbers (base 10). You decide to write a script that will repetitively ask the user for a binary number and return its decimal equivalent until an illegal number (one containing digits other than 0 or 1) is entered. The number entered should contain only the digits 0 and 1. The rightmost digit has the value 2^0 and the digit N places to the left of that has the value 2^N . For example:

entering 110101 returns $53 = 2^5 + 2^4 + 2^2 + 2^0$

You must use iteration to solve this problem. Note: The `input (...)` function prompts the user for a value and returns the numerical result.

5. You have a friend who has too many clothes to store in her/his tiny wardrobe provided by Georgia Tech. Being a good friend, you offer to help to decide whether each piece of clothing is worth saving. You decide to write a script in MATLAB that will compute the value of each piece of clothing. A piece of clothing has five attributes that can be used to determine its value. The attributes are:
 condition, color, price, number of matches, and comfort.
 Each attribute will be rated on a scale of 1-5. Write a script called `clothes` in MATLAB that will ask the user for the ratings for each attribute and store the result in a vector. The order of attributes in the vector are as follows:

```
[condition color price matches comfort]
```

The script should compute a value between 0 and 100; 100 represents a good piece of clothing while 0 represents a bad piece of clothing. The points that should be given for each attribute are shown below:

Condition: 1=>0; 2=>5; 3=>10; 4=>15; 5=>20

Color: 1 => blue => 12;
 2 => red (UGA Colors) => 2;
 3 => pink => 15;
 4 => yellow (GT Colors) => 20;
 5 => white => 12

Price: 1 => 8, 2-3 => 16, 4-5 => 20

Matches: 1-2 => 8, 3-5 => 19

Comfort: 1 => 6, 2-3 => 13, 4-5 => 18

Note: If a number other than 1-5 is assigned for one of the attributes, no points should be given.

6. A “yard” is a traditional English container. It is 36 inches long, and can be approximated by a 4-inch diameter glass sphere attached to a conical section whose narrow end is 1 inch in diameter, and whose wide end is 6 inches in diameter. Write a script to do the following:
 - a. ask the user for the height of the liquid in the yard, and
 - b. calculate the volume of liquid needed to fill the yard to that level.
7. Now that you’re comfortable with iteration, you’re going to have to solve an interesting problem. It seems that the Math department at UGA once again dropped the ball, and forgot the value of `pi`. You are to write a function called `mypi` which consumes a number that specifies the required accuracy and approximates the value of `pi` to that accuracy. You are going to use the following algorithm based on geometric probability.

Think about a quarter circle inside of a unit square (the quarter circle has area $\pi/4$). You pick a random point inside the square. If it is in the quarter circle, you get a “hit” and if not, you get a “miss”. The approximate area of the quarter circle will be given by the number of hits divided by the number of points you chose.

Hint

Use the function `rand (...)` in this problem.

Your function should repeat the process of counting hits and misses until at least 10,000 tries have been made, and the successive estimates of `pi` are within the prescribed accuracy. It should return the estimated value of `pi`.

Functions

Chapter Objectives

This chapter discusses the nature, implementation, and behavior of user-defined functions in MATLAB:

- How to define a function
- How data are passed into a function
- How to return data including multiple results
- How to include other functions not needed except as helpers to your own function

Writing a user-defined function allows you to isolate and package together a code block, so that you can apply that code block to different sets of input data. We have already made use of some built-in functions like `sin(...)` and `plot(...)` by calling them; this chapter will deal with creating and using your own functions.

- 5.1** Concepts: Abstraction and Encapsulation
- 5.2** Black Box View of a Function
- 5.3** MATLAB Implementation
 - 5.3.1 General Template
 - 5.3.2 Function Definition
 - 5.3.3 Storing and Using Functions
 - 5.3.4 Calling Functions
 - 5.3.5 Returning Multiple Results
 - 5.3.6 Auxiliary Local Functions
 - 5.3.7 Encapsulation in MATLAB Functions
 - 5.3.8 Global Variables
- 5.4** Engineering Example—Measuring a Solid Object

5.1 Concepts: Abstraction and Encapsulation

A **function** is an implementation of procedural abstraction and encapsulation. **Procedural abstraction** is the concept that permits a code block that solves a particular subproblem to be packaged and applied to different data inputs. This is exactly analogous to the concept of data abstraction we discussed in Chapter 3 where individual data items are gathered to form a collection. We have already used a number of built-in procedural abstractions in the form of functions. All the mathematical functions that compute—for example, the sine of a collection of angles or the maximum value of a vector—are procedural abstractions. They allow us to apply a code block about which we know nothing to data sets that we provide. To make use of a built-in function, all we have to do is provide data in the form the function expects, and interpret the results according to the function’s specification.

Encapsulation is the concept of putting a wrapper around a collection that you wish to protect from outside influence. Functions encapsulate the code they contain in two ways: the variables declared within the function are not visible from elsewhere, and the function’s ability to change the values of variables (otherwise known as causing side effects) is restricted to its own code body.

5.2 Black Box View of a Function

The most abstract view of a function can be seen in Figure 5.1. It consists of two parts: the definition of the interface by which the user passes data items to and from the function, and the code block that produces the results required by that interface. A function definition consists of the following components:

- A name that follows the same syntactic rules as a variable name
- A set of 0 or more parameters provided to the function
- Zero or more results to be returned to the caller of the function

The basic operation of a function begins before execution of the function actually starts. If the function definition requires n parameters, the calling

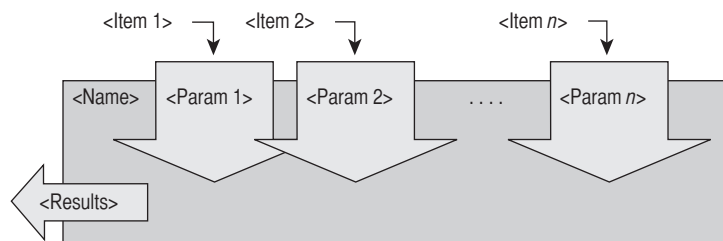


Figure 5.1 Black box view of a function

instructions first prepare n items of data from its workspace to be provided to the function. These data are then passed to the function, the code body is executed, and the results are returned to the caller.



5.3 MATLAB Implementation

In this section first we consider the general template for implementing functions and then the MATLAB implementation of that template.

5.3.1 General Template

The general layout of a function definition is shown in Template 5.1.

The `<return info>` section for most functions involves providing the name(s) of the results returned followed by an = sign. If more than one result is to be returned, they are defined in a vector-like container. If nothing is to be returned from this function, both the result list and the = sign are omitted.

The `<function name>` is a name with the same syntactic rules as a variable name, and will be used to invoke the code body.

The `<parameters>` section is a comma-separated list of the names of the data to be provided to the function.

The `<documentation>` section is one or more lines of comments that describe what the function does and how to call it. These lines will appear in two situations:

- All the documentation lines up to the first non-document line are printed in the Command window when you type the following:

```
>> help <function name>
```
- The first line is listed next to the file name in the Current Directory listing

5.3.2 Function Definition

MATLAB functions must be stored in a separate file located in a directory accessible to any script or function that calls it. The file containing the definition of a function named `function_name` must be `<function_name>.m`. For the general user, the Current Directory is the normal place to store it. Listing 5.1 illustrates a typical MATLAB function called `cylinder` that consumes two parameters, the height and radius of a cylinder, and produces the return variable `volume`.

Template 5.1 General template for a function

```
function <return info> <function name> (<parameters>)
<documentation>
    <code body>    % must return the results
```

Listing 5.1 Function example

```

1. function volume = cylinder(height, radius)
2. % function to compute the volume of a cylinder
3. % volume = cylinder(height, radius)
4.     base = pi * radius^2;
5.     volume = base * height;
6.

```

In Listing 5.1:

Line 1: The MATLAB function definition is introduced by the key word `function`, followed by the name of the return variable (if any) and the `=` sign.

Line 2: All comments written immediately after the function header are available to the Command window when you enter:

```
>>help <function_name>
```

The first comment line also appears in the Current Directory window as an indication of the basic purpose of the function.

Line 3: It is a good idea to include in the comments a copy of the function header line, sometimes referred to as the Application Programmer Interface (API), to remind a user exactly how to use this function.

Line 4: Although encapsulation rules forbid access to the caller's variables, the code body still has access to all built-in MATLAB names (for example, `pi`, as used here).

Line 5: You must make at least one assignment to the result variable.

Line 6: The function definition needs no `end` statement. The code body terminates either at the end of the file or at the next function definition in the same file.

Try saving and testing the `cylinder` function in Exercise 5.1.

**Exercise 5.1** Saving and testing the `cylinder` function***Do It Yourself***

Enter the function definition from Listing 5.1 in the Text Editor and save it as `cylinder.m` in your Current Directory. Then enter the following experiments in the Command window. Notice that the first help line appears next to this file name in the Current Directory.

```

>> help cylinder
function to compute the volume of a cylinder
    volume = cylinder(height, radius)
>> cylinder(1, 1)
ans =
    3.1416
>>

```

5.3.3 Storing and Using MATLAB Functions

All MATLAB functions must be created like scripts in an m-file. When the file is first created, it must be saved in an m-file with the same file name as the function. For example, the function in Listing 5.1 named `cylinder` must be saved in a file named `cylinder.m`.

Once the file has been saved, you may invoke the function by entering its name and parameters of the right type and number in the Command window, in a script, or in other function definitions. If you do not specify an assignment for the result of the function call, it will be assigned to the variable `ans`.

5.3.4 Calling Functions

When a function is defined, the user provides a list of the names of each data item expected to be provided by the caller. These are called the **formal parameters**. When this function is called from the Command window or a code block, the caller must provide the same number of data values expected by the function definition. These are the **actual parameters**, and can be generated in the following ways:

- Constants
- Variables that have been defined
- The result of some mathematical operation(s)
- The result returned from other functions

When the actual parameters have been computed, copies of their values are assigned as the values of the formal parameters the function is expecting. Values are assigned to parameters by position in the calling statement and function definition.

The process of copying the actual parameters into the formal parameters is referred to as “passing by value”—the only technique provided in MATLAB for passing data into a function. Some languages provide an alternative technique—“passing by reference”—wherein the storage location for the parameters remains in the caller’s workspace. This is usually a bad thing, allowing deliberate or accidental assignments to “reach back” into the scope of the calling code and thereby perhaps causing undesirable side effects. However, restricting parameter access to passing by value can result in poor program performance.

Once the parameter names have been defined in the function’s workspace, the function’s code body is executed, beginning with the first instruction. If return variables have been defined for the function, every exit from the code body must assign valid values for the results.

Although the number of parameters is usually fixed, most languages, including MATLAB, provide the ability to deal with a variable number of

parameters, both incoming and returning. The built-in MATLAB function `nargin` computes the actual number of parameters provided by the user in the current function call. If the function is sensitive to `nargin`, the user calling this function can provide any values he deems important, and allow the function to set default values for the unnecessary parameters.

Similarly, the function `nargout` computes the number of storage variables actually provided by the user. So if one or more of the results requires extensive computation or user interaction and the caller has not asked for that data, the computation can be omitted.

5.3.5 Returning Multiple Results

MATLAB is unique among programming languages in providing the ability to return more than one result from a function. The multiple results are specified as a vector of variable names: `[area, volume]`, as shown in Listing 5.2. Assignments must be made to each of the result variables. However, the calling program is not required to make use of all the return values.

In Listing 5.2:

Line 1: Multiple results are specified as a vector of variable names, each of which must be assigned from the code body.

Exercise 5.2 shows how to invoke a function that can return multiple results.

Listing 5.2 `cylinder` function with multiple results

```
1.function [area, volume] = cylinder(height, radius)
2. % function to compute the area and volume
3. %                                     of a cylinder
4. % usage: [area, volume]=cylinder(height, radius)
5.     base = pi .* radius.^2;
6.     volume = base .* height;
7.     area = 2 * pi * radius .* height + 2 * base;
```



Exercise 5.2 Testing multiple returns

Do It Yourself

Adapt the original `cylinder` function as shown in Listing 5.2 and perform the following tests in the Command window:

```
>> [a, v] = cylinder(1, 1)
a =
    6.2832
v =
    3.1416
>> cylinder(1, 1)
ans =
    6.2832
```

continued on next page

```
>> a = cylinder(1, 1)
a =
    6.2832
>> v = cylinder(1, 1)
v =
    6.2832
>>
```

Notice that the normal method to access the multiple answers is to put the names of the variable to receive the results in a vector. The names may be any legal variable name, and the values are returned in the order of the results defined. If you choose less than the full number of results (or none at all), the answers that are specified are allocated from left to right from the available results. As with parameter assignment, the results are allocated by position in these vectors. Although we called the variable `v` in the last test, it still receives the value of the first result, `area`. If you really only want the second result value, you must provide a dummy variable for the first, by making the call something like this:

```
[junk, v] = cylinder(1, 1);
```

5.3.6 Auxiliary (Local) Functions

MATLAB uses the name of the file to identify a function; therefore, every function should normally be saved in its own m-file. However, there are times when auxiliary functions are needed to implement the algorithm contained in the main function in a file. If this auxiliary function is only used in the main function, it can be written in the same file as its calling function after the definition of the main function. By convention, many people append the word `local_` to the name of local functions. These are also referred to as “helper functions.” Calling code can reach only the first function defined in an m-file. Other functions in the m-file, the auxiliary functions, can only be called only from the first function or other auxiliary functions in the same file.

5.3.7 Encapsulation in MATLAB Functions

Encapsulation is accomplished in most modern languages, including MATLAB, by implementing the concept of variable scoping.

Variable scoping defines the locations within your Command window, MATLAB system, and m-files to which variables have access. It is related to the Current Workspace window. When using the Command window or running a script m-file, and you access the value of a variable, the system will reach into your Current Workspace and then into the MATLAB system libraries to find its current value. This is referred to as **Global Scope**. When

you write a function, its local variables, including the internal names of its parameters, are not included in your Current Workspace, and it does not look into your Current Workspace for values of variables it needs. This is referred to as **Local Scope**, wherein the variables within a function are not visible from outside and the function is unable to cause side effects by making assignments to outside variables.

To illustrate variable scoping, do Exercise 5.3.

5.3.8 Global Variables

Because MATLAB always copies the input data into the function's workspace, there are occasions when it is very inefficient to pass data into and out of a function. Global variables must be defined in both the calling

script and the function using the key word `global`. For example, suppose we collect a large volume of data in a variable `buffer` and do not want to copy the whole buffer into and out of a function that processes that data. In this case we must declare the variable to be global in both the calling space and the called function by placing the following line of code before the variable is first used in both places:

```
global buffer
```

The function will then be able to access and modify the values in `buffer`

without having to pass it in and out as a parameter.

Style Points

1. Before you include a function in a complex algorithm, you should always test its behavior in isolation in a script. This test script should validate not only the normal operation of the function, but also its response to erroneous input data it might receive.
2. Although any legal MATLAB instruction is permitted within the code body of a function, it is considered bad form (except temporarily for debugging purposes) to display values in the Command window.
3. We also actively discourage the use of the `input(...)` function within the code body. If you need to input some values to test a function, do so from the Command window or a test script.



Exercise 5.3 Observing variable scoping

Do It Yourself

Put a break point at Line 6 of your version of the code in Listing 5.2, and then re-run the function by entering:

```
>> [a, v] = cylinder(1, 1)
```

Notice that the logic stops at that break point and the Text Editor displays an arrow. The Workspace window shows you the values of `height`, `radius`, and `base` but none of the variables you left in the workspace for the Command window. The function has no access to other workspaces.

Observe that as you step through the function, the variables appear in the Workspace window and are updated. When you return from the `cylinder` function to display the results, the Workspace for the function disappears. The calling environment has no access to the variables within the function.

This feature must be used with caution, however, because any function with global access to data is empowered to change that data counter-intuitively.

5.4 Engineering Example—Measuring a Solid Object

Problem:

Consider the disk shown in Figure 5.2. It has a radius R , height h , and eight cylindrical holes each of radius r bored in it. This might be a component of a machine that must be painted and then assembled with other components. During the process of designing this machine, we may need to know the weight of this disk and the amount of paint required to finish it. The weight and the amount of paint for the machine is the sum of the values for each component. Since the weight of our disk is proportional to its volume and the amount of paint is proportional to its “wetted area,” we need the volume and area of this disk.

Write a script to compute the volume of the disk and its wetted area.

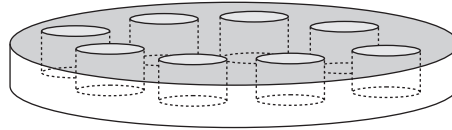


Figure 5.2 Disk with holes

Solution:

Listing 5.3 shows the code that solves this problem.

Listing 5.3 Volume and area of a disk

```
1. clear
2. clc

3. h = 1:5;      % set a range of disk thicknesses
4. R = 25;
5. r = 3;

6. [Area Vol] = cylinder(h, R) % dimensions of large disk
7. [area vol] = cylinder(h, r) % dimensions of the hole

8. % compute remaining volume
9. Vol = Vol - 8*vol
```

continued on next page

```

10. % the wetted area is a little messier. If we total the
11. % large disk area and the areas of the holes, we get the
12. % wetted area of the curved edges inside and out.
13. % However, for each hole, the top and bottom areas have
14. % been included not only in the top and bottom of the big
15. % disk, but also as the contributions of each hole.
16. % From the sum of the top areas, we therefore have to
17. % remove 32 times the hole top area

18. Area = Area + 8*(area - 2*2*pi*r.^2)

```

In Listing 5.3:

Lines 3–5: Set up the disk sizes. Notice that the script works fine with a vector of disk thicknesses to check the behavior as thickness varies.

Hint

If you experiment with this script a little, you will discover the power of vector processing for rapidly determining the sensitivity of results to different parameters. The mathematics will not work if you provide vectors for more than one of the given data items. However, vectors supplied for each of them in turn provide insight into the sensitivity of the results to each parameter.

Line 6: Area and volume of the large disk.

Line 7: Area and volume of one hole.

Line 9: Volume computation.

Lines 10–17: Document the logic behind the area computation.

Line 18: The area computation.

Listing 5.4 shows the results when this code is run. Notice that for thin disks, the area is smaller with the holes. However, as the thickness increases, the area with the holes is larger than without, as one would expect.

Listing 5.4 Volume and area results

```

Area = 4,084 4,241 4,398 4,555 4,712
Vol = 1,963 3,927 5,890 7,854 9,817
area = 75 94 113 132 151
vol = 28 57 85 113 141
Vol = 1,737 3,474 5,212 6,949 8,687
Area = 3,782 4,090 4,398 4,706 5,014

```



Chapter Summary

This chapter showed you how to encapsulate a code block to allow it to be re-used:

- Functions are defined in a file of the same name using the key word `function` to distinguish them from scripts
- Parameters are copied in sequence into the function and given the names of the formal parameters

- Results are returned to the caller by assigning value(s) to the return variable(s)
- Variables within the function can be accessed only in the function's code block unless they are declared global
- Helper functions accessible only to functions within the same file may be added below the main function and otherwise obey the same rules as the main function



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
()	Used to identify the formal and actual parameters of a function	5.3.2, 5.3.4
help	Invokes help utility	5.3.1
function	Identifies an m-file as a function	5.3.2
nargin	Determines the number of input parameters actually supplied by a function's caller	5.3.4
nargout	Determines the number of output parameters actually requested by a function's caller	5.3.4
global var	Defines a variable as globally accessible	5.3.8



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. All data used by a function must be passed in as parameters to the function.
2. The name of the first function in an m-file must match the name of the file containing its definition.
3. The first documentation line appears in the Current Directory listing.
4. Functions must consume at least one parameter.
5. The calling code must provide assignments for every result returned from a function.
6. The names of auxiliary functions must begin with `local_`.

Fill in the Blanks

1. _____ permits a code block to be packaged and referred to collectively rather than individually.
2. Values of the _____ parameters are copied to define the _____ parameters inside the function.
3. If more than one result is to be returned from a function, they are defined in a _____.
4. _____ describes the situation where the variables within a function are not visible from outside, and the function is unable to cause side effects by making assignments to outside variables.
5. Calling code can only reach the _____ function in an m-file. Other functions in the m-file can only be called from the _____ or _____.



Programming Projects

1. Write a function called `checkFactor` that takes in two numbers, and checks if they are divisible, i.e. if the first is divisible by the second. You may assume that both numbers are positive. Your function should return a logical value, `true` or `false`.

Hint

`mod(x, y)` gives the remainder when `x` is divided by `y`.

For example,

```
checkFactor(25,6) should return false.
checkFactor (9,3) should return true.
checkFactor (3,9) should return false.
```

2. Write and test the code for the function `mysteryFunction` that consumes a vector, `v`, and produces a new vector, `w`, of the same length where each element of `w` is the sum of the corresponding element in `v` and the previous element of `v`. Consider the previous element of `v(1)` to be 0.

For example,

```
mysteryFunction( 1:8 ) should return
    [1 3 5 7 9 11 13 15]
mysteryFunction([1:6].^2) should return
    [1 5 13 25 41 61]
```

3. Coming off a respectable 7-6 record last year, your football team is looking to improve on that this season. They have contacted you and asked for your help projecting some of the scenarios for their win-loss record. They want you to write a function called `teamRecord` which takes in two parameters - `wins`, and `losses`, and returns two values - `season`, and `wPercentage`. `season` should be a logical result that is `true` for a winning season. `wPercentage` is the percentage of games won (ranging from 0 to 100).

For example,

```
teamRecord(3, 9) should return season = false,
                                wPercentage = 25
teamRecord(10, 2) should return season = true,
                                wPercentage = 83.3
```

4. Write a function called `classAverage` that takes in an array of numbers and, after normalizing the grades in such a way that the highest corresponds to 100, returns the letter grade of the class average. This curve is the same method used to generate `normTests` in Chapter Problem 2. The grade ranges are as follows:

```
average > 90    => A
80 <= average < 90 => B
70 <= average < 80 => C
60 <= average < 70 => D
average < 60    => F
```

For example:

```
classAverage( [70 87 95 80 80 78 85 90 66 89 ...
               89 100] ) should return B
classAverage( [50 90 61 82 75 92 81 76 87 41 ...
               31 98] ) should return C
classAverage( [10 10 11 32 53 12 34 74 31 30 ...
               26 22] ) should return F
```

5. Write a function called `myMin4` that will take in 4 numbers and returns the minimum value and an index showing which parameter it was. You may not use the built-in `min()` function.

For example:

```
myMin4(1,3,5,7) should return 1 and 1
myMin4(8,9,2,4) should return 2 and 3
```

6. Write the function `meansAndMedian` which takes in a vector of numbers and returns the arithmetic and geometric means, as well as the median. You may not use the built-in functions `mean()`,

Hint

the built-in function `sort()` might help to compute the median of the vector.

`median()`, or `geomean()`. However, you could type "help geomean" to familiarize yourself with computing the geometric mean of a group of numbers.

7. Given an array of numbers that could be negative, write a function `posavg(a)` to calculate and return the average (mean) of the nonnegative numbers in the single dimensional array, `a`. One such solution is:

```
mean(a(find(a>0)))
```

In order to test your understanding of class concepts, re-implement the `posavg(a)` function using iteration. You may not use the built-in functions `sum(...)`, `find(...)`, or `mean(...)` in your solution.

8. Write a function called `sumAndAverage`. It should take in an array of numbers and return the sum and average of the array in that order.

For example:

```
sumAndAverage([3 2 3 2]) should return 10 and 2.5
sumAndAverage([5 -5 2 8 0]) should return 10 and 2
sumAndAverage([]) should return 0 and 0
```

9. You are already familiar with the logical operators `&&` (and) and `||` (or), as well as the unary negation operator `~` (not). In a weakly typed language such as MATLAB, the binary states `true` and `false` could be equivalently expressed as a 1 or a 0, respectively. Let us now consider a ternary number system, consisting of the states `true(1)`, `maybe(2)`, and `false(0)`. The truth table for such a system is shown below. Implement the truth table by writing the functions, `f=tnot(x)`, `f=tand(x,y)`, and `f=tor(x,y)`. You may not assume that only valid input numbers will be entered.

<code>x</code>	<code>y</code>	<code>tnot(x)</code>	<code>tand(x,y)</code>	<code>tor(x,y)</code>
1	1	0	1	1
1	0	0	0	1
1	2	0	2	1
0	1	1	0	1
0	0	1	0	0
0	2	1	2	0
2	1	2	2	1
2	0	2	2	0
2	2	2	2	2

10. Write a function called `multiSum(A)`. This particular function should take in a $N \times M$ array, `A`, and return four results:

`A` $1 \times M$ vector with the sum of the columns,
`A` $N \times 1$ vector with the sum of the rows, and
 Two numbers containing the sums of the two diagonals, the major diagonal first.

For example:

```
columnSum([1 2 3; 4 5 6; 7 8 9]) should return
[12 15 18], [6 15 24]', 15 and 15
columnSum([0 2 3; 4 0 6; 7 8 0]) should return
[11 10 9], [5 10 15]', 0 and 10
```

```
columnSum(eye(5,5)) should return  
[1 1 1 1 1], [1 1 1 1 1]', 5 and 1  
columnSum([]) should return [], [], 0 and 0
```

11. You are playing a game where you roll a die 10 times. If you roll a five or six seven or more times, you win 2 dollars, four or more times, you win 1 dollar, and if you roll a 5 or 6 three or less times, you win no money. Write a function called `diceGame` that takes in a vector representing the dice values and returns the amount of money won.

For example:

```
diceGame([5 1 4 6 5 5 6 6 5 2]) should return 2  
diceGame([2 4 1 3 6 6 6 4 5 3]) should return 1  
diceGame([1 4 3 2 5 3 4 2 6 5]) should return 0
```

Note: This function should work for any length vector.

Character Strings

Chapter Objectives

This chapter discusses the nature, implementation, and behavior of character strings in MATLAB:

- The internal workings of character strings as vectors
- Operations on character strings
- Converting between numeric and character string representations
- Input and output functions
- The construction and uses for arrays of strings

To this point in the text, we have seen the use of character strings that we can store in variables and display in the Command window. In reality, we have already seen a significant amount of character manipulation that we have taken for granted. The m-files we use to store scripts and functions consist of lines of legible characters separated by an invisible “new-line” character.

Introduction

This chapter presents the underlying concept of character storage and the tools MATLAB provides for operating on character strings.

We need to distinguish two different relationships between characters and numbers:

1. *Individual characters have an internal numerical representation:* The visible character shapes we see in windows are created as a collection of white and black dots by special software called a **character generator**. Character generators allow us to take the underlying concept of a character—say, “w”—and “draw” that character on screen or paper in accordance with the rules defined by the current font. A complete study of fonts is beyond the scope of this discussion, but we need to understand how computers in general and MATLAB in particular represent that “underlying concept” of a character. This is achieved by representing each individual character by a numerical equivalent. Not long ago, there were many different representations. Today,

- 6.1 Character String Concepts: Mapping and Casting
- 6.2 MATLAB Implementation
 - 6.2.1 Slicing and Concatenating Strings
 - 6.2.2 Arithmetic and Logical Operations
 - 6.2.3 Useful Functions
- 6.3 Format Conversion Functions
 - 6.3.1 Conversion from Numbers to Strings
 - 6.3.2 Conversion from Strings to Numbers
- 6.4 Character String Operations
 - 6.4.1 Simple Data Output: The `disp(...)` Function
 - 6.4.2 Complex Output
 - 6.4.3 Comparing Strings
- 6.5 Arrays of Strings
- 6.6 Engineering Example—Encryption

the dominant representation is the one defined by the American Standard Code for Information Interchange (ASCII). In this representation, the most common uppercase and lowercase characters, numbers, and many punctuation marks are represented by numbers between 0 and 127. A complete listing of the first 255 values is included in Appendix B.

2. *Strings of characters represent numerical values to the user:* Numerical values are stored in MATLAB in a special, internal representation for efficient numerical computation as described in Appendix C. However, whenever we need to see the value of that number in the Command window, that internal representation is automatically converted by MATLAB into a character string representing its value in a form we can read. For example, if the variable `a` contained the integer value 124, internally that number could be stored in a single byte (8 bits) with a binary value of 011111100. Not a very meaningful representation, but efficient internally for performing arithmetic and logical operations. For the user to understand that value, a MATLAB function must convert it to the three printable characters: `'124'`. Similarly, when we use the `input(...)` function, the set of characters that we enter is automatically translated to the internal number representation.



6.1 Character String Concepts: Mapping Casting, Tokens and Delimiting

Here we see the MATLAB tools that deal with the first relationship between characters and numbers: the numerical representation of individual characters. The basic idea of **mapping** is that it defines a relationship between two entities. The most obvious example of mapping is the idea that the function $f(x) = x^2$ defines the mapping between the value of x and the value of $f(x)$. We will apply that concept to the process of translating a character (like “A”) from its graphical form to a numerical internal code. **Character mapping** allows each individual graphic character to be uniquely represented by a numerical value.

Casting is the process of changing the way a language views a piece of data without actually changing the data value. Under normal circumstances, a language like MATLAB automatically presents a set of data in the “right” form. However, there are times when we wish to force the language to treat a data item in a specific way. For example, if we create a variable containing a character string, MATLAB will consistently display it as a character string. However, we might want to view the underlying numerical representation as a number, in which case we have to cast the variable containing the characters to a numerical data type.

MATLAB implements casting as a function with the name of the data type expected. In essence, these functions implement the mapping from one character representation to another.

A **token** is a collection of characters to which we may wish to attach meaning. Obvious examples of tokens are the name of a MATLAB variable or the characters representing the values of a number to be used in an expression.

A **delimiter** is a character used to separate tokens. The space character, for example, can delimit variables and numbers in an expression.



6.2 MATLAB Implementation

When you enter a string in the Command window or the editor, MATLAB requires that you delimit the characters of a string with a single quote mark ('). Note that you can include a single quote mark within the string by doubling the character. For example, if you entered the following in the Command window:

```
>> refusal = 'I can''t do that!'
```

The result displayed would be

```
refusal = I can't do that
```

Exercise 6.1 illustrates the concept of casting between data types `char` and `double`.



Exercise 6.1 Character casting

Do It Yourself

Enter the following in the Command window and study the results:

```
>> uint8('A') % uint8 is an integer data type
               % with values 0 - 255
ans =
    65
>> char(100) % char is the character class
ans =
    d
>> char([97 98 99 100 101])
ans =
abcde
>> double('fred')
ans =
    102    114    101    100
>> fred = 'Fred'
fred =
Fred
>> next = fred + 1
next =
    71    115    102    101
>> a = uint8(fred)
a =
    70    114    101    100
>> name = char(a + 1)
name =
Gsfe
```


In Exercise 6.1 the casting function `uint8(...)` takes a character or character string and converts its representation to a vector of the same length as the original string. Then the casting function `char(...)` takes a number or vector and converts it to a string representation. The casting function `double(...)` appears to act in the same way as `uint8(...)`, but it actually uses 64 bits to store the values.

Single quotes delimit a string to be assigned to the variable `fred`. Notice that when a string is presented as a result, the delimiters are omitted. When you apply arithmetic operations to a string, the operation is illegal on characters; therefore, an implicit casting to the numerical equivalent occurs. You can perform any mathematical operation on the vector and use the cast, `char(...)` to cast it back to a string.

6.2.1 Slicing and Concatenating Strings

Strings are internally represented as vectors; therefore, we can perform all the usual vector operations on strings. Try it in Exercise 6.2.

6.2.2 Arithmetic and Logical Operations

Mathematical operations can be performed on the numerical mapping of a character string. If you do not explicitly perform that casting first, MATLAB will do the cast for you, and create a result of type `double` (not usually suitable for character values). Note that `char('a' + 1)` returning `'b'` is an accident of the character type mapping.

Logical operations on character strings are also exactly equivalent to logical operations on vectors, with the same automatic casting. Exercise 6.3 gives you an opportunity to try it yourself.



Exercise 6.2 Character strings

Do It Yourself

Enter the following in the Command window and study the results:

```
>> first = 'Fred'
first =
Fred
>> last = 'Jones'
last =
Jones
>> name = [first, ' ', last]
name =
Fred Jones
>> name(1:2:end)
ans =
Fe oe
>> name(end:-1:1)
ans =
senoJ derF
```



Exercise 6.3 Character string logic

Do It Yourself

Enter the following in the Command window:

```
>> n = 'fred'
n =
fred
>> n > 'g'
ans =
     0     1     0     0
```

6.2.3 Useful Functions

The following functions are useful in analyzing character strings:

- `ischar(a)` returns `true` if `a` is a character string
- `isspace(ch)` returns `true` if the character `ch` is the space character



6.3 Format Conversion Functions

Now we turn to the second relationship between characters and numbers: using character strings to represent individual number values. We need two separate capabilities: converting numbers from the efficient, internal form to legible strings, and converting strings provided by users of MATLAB into the internal number representation. MATLAB provides a number of functions that transform data between string format and numerical format.

6.3.1 Conversion from Numbers to Strings

Use the following built-in MATLAB functions for a simple conversion of a single number, `x`, to its string representation:

- `int2str(x)` if you want it displayed as an integer value
- `num2str(x, n)` to see the decimal parts; the parameter `n` represents the number of decimal places required—if not specified, its default value is 3

Frequently you need better control over the data conversion, and the function `sprintf(...)` provides fine-grained control. The MATLAB version of `sprintf(...)` is very similar to the C / C++ implementation of this capability. The first parameter to `sprintf` is a **format control string** that defines exactly how the resulting string should be formatted. A variable number of **value parameters** follow the format string, providing data items as necessary to satisfy the formatting.

Basically the format string contains characters to be copied to the result string; however, it also contains two types of special entry introduced by the following two special characters:

- The `'%'` character introduces a conversion specification, indicating how one of the value parameters should be represented. The most common conversions are `%d` (integer), `%f` (real), `%g` (general), `%c` (character) and `%s` (string). A number may be placed immediately after the `%` character to specify the minimum number of characters in the conversion. If more characters than the specified minimum are required to represent the data, they will be added. In addition, the `%f` and `%g` conversions can include `'.n'` to indicate the number of decimal places required. If you actually require a `'%'` character, it must be doubled, for example, `'%%'`. MATLAB processes each of the value parameters in turn, inserting them in the result string according to the corresponding conversion specification. If there are more parameters than conversion specifications in the format control string, the format control string is repeated.
- The `'\'` character introduces format control information, the most common of which are `\n` (new line) and `\t` (tab). If the `'\'` character is actually required in the result string, it should be doubled, for example, `'\\'`.

Consider the following statements:

```
A = [4.7 1321454.47 4.8];
index = 1;
v = 'values';
str = sprintf('%8s of A(%d) are \t%8.3f\t%12.4g\t%f\n'...
    v, index, A(index,1), A(index,2), A(index,3))
str =
    values of A(1) are      4.700      1.321e+006      4.800000
```

The first conversion, `'%8s'`, took the value of the first parameter, `v`, allowed eight spaces for its conversion, and copied its contents to the result. Since this was a string conversion, the characters were merely copied. The characters `' of A('` were then appended to the output string. The second conversion, `'%d'`, took the value of the second parameter, `index`, and converted it as an integer with the minimum space allocated. The characters `' )are'` were then appended to the output string, followed by a tab character that inserted enough spaces to bring the next characters to a column that is an even multiple of eight. The following three conversions appended the next three value parameters converted with three decimal places, a general conversion with at least 12 spaces and 4 decimal places, and the default numerical conversion. Finally, a new line character was inserted into the string.

6.3.2 Conversion from Strings to Numbers

Conversion from strings to numbers is a much messier conversion, and it should be avoided if possible. When possible, allow MATLAB's built-in

function `input(...)` to do the conversion for you. If you have to do the conversion yourself, use the function `sscanf(...)`.

The function `input(str)` presents the string parameter to the user in the Command window and waits for the user to type some characters and the  key, all of which are echoed in the Command window. Then it converts the input string according to the following rules:

- If the string begins with a numerical character, MATLAB converts the string to a number
- If it begins with a non-numeric character, MATLAB constructs a variable name and looks for its definition
- If it begins with an open bracket, '[', a vector is constructed
- If it begins with the single quote character, MATLAB creates a string

If a format error occurs, MATLAB repeats the prompt.

This behavior can be modified if 's' is provided as the second parameter, `input(str, 's')`, in which case the complete input character sequence is saved as a string. Exercise 6.4 demonstrates a number of capabilities of the `input(...)` function.



Exercise 6.4 The `input(...)` function

Do It Yourself

Enter the following into the Command window and notice the behavior:

```
>> fred = 'Fred';
>> n = input('Enter a number: ')
Enter a number: 5
n =
    5
>> n = input('Enter a number: ')
Enter a number: fred
n =
Fred
>> n = input('Enter a number: ')
Enter a number: lsdf
??? lsdf
Error: Missing MATLAB operator.
Enter a number: sldf
??? Error using ==> input
Undefined function or variable 'sldf'.
Enter a number: char(fred - 2)
n =
Dpcb
>> n = input('Enter a number: ')
Enter a number: 'ABCD'
n =
ABCD
>> n = input('Enter a number: ', 's' )
Enter a number: ABCD
n =
```

In Exercise 6.4 first we define the variable `fred`. Then MATLAB attempts to interpret the result either as a number or as the name of an existing variable. Since the variable `fred` was defined (although not a number), it was assigned correctly to the variable `n`. MATLAB attempts to distinguish between a variable and a number by the first digit. Here, the information entered was an illegal variable name beginning with a number. When `input(...)` detects an error parsing the text entered, it automatically resets and requests a new entry.

On the second attempt, although this is a correctly formed variable name, its value is not known. On the third attempt, the `input(...)` function actually treats the string entered as an expression, to be evaluated by the same process as MATLAB parses the Command window entries.

If you actually want a string literal entered, it must be enclosed in the string delimiters. If you are sure you want a string literal entered, the second parameter, `'s'`, forces MATLAB to return the string entered without attempting to parse it.

The function `sscanf(...)`, unfortunately, is quite different from the C / C++ mechanization. In its simplest form, `cv = sscanf(str, fmt)` scans the string `str` and converts each data item according to the conversion specifications in the format string `fmt`. Each item discovered in `str` produces a new row on the result array, `cv`, a column vector. Unfortunately, if you convert strings this way, each character in the string becomes a separate numerical result in the output vector. A peculiar feature of MATLAB allows you to substitute the character `'*'` for the conversion size parameter to suppress any strings in the input string. For example:

Style Points

We strongly recommend that you restrict the use of this function to converting a single numerical value. Clearly, string extraction is best accomplished by using the `input(str)` and `sscanf(...)` string manipulation functions.

```
str = 'are      4.700      1.321  4.800000'
B = sscanf( str, '%*s %f %f %f')

B =
    4.7000
    1.3210
    4.8000
```



6.4 Character String Operations

As with the string-to-number conversions, input and output in the Command window can be accomplished with simple functions that have little flexibility or with complex functions that have better control.

6.4.1 Simple Data Output: The `disp(...)` Function

We have already seen the use of the `disp(...)` function to present data in readable form in the Command window. As the exercises indicate, it can present the values of any variable, regardless of type, or of strings constructed

**Exercise 6.5** The `disp(...)` function**Do It Yourself**

Enter the following in the Command window:

```
>> a = 4;
>> disp(a)
      4
>> disp(['the answer is ', a])
the answer is _
>> disp(['the answer is ', int2str(a)])
the answer is 4
```

by concatenation. Note, however, that an explicit number conversion is required to concatenate variables with strings. Try Exercise 6.5.

Note that although you can concatenate strings for output, conversion from the ASCII code is not automatic; the second result produced a character whose ASCII code is 4. You must use the simple string conversion functions to enforce consistent information for concatenation.

6.4.2 Complex Output

The function `fprintf(...)` is similar to `sprintf(...)`, except that it prints its results to the Command window instead of returning a string. `fprintf(...)` returns the number of characters actually printed. Exercise 6.6 demonstrates this.

6.4.3 Comparing Strings

Since strings are readily translated into vectors of numbers, they may be compared in the obvious way with the logical operators we used on numbers.

**Exercise 6.6** `fprintf(...)` and `sprintf(...)`**Do It Yourself**

Enter the following into the Command window:

```
>> a = 42;
>> b = 'fried okra';
>> n = fprintf('the answer is %d\n cooking %s', ...
               a,          b);

the answer is 42
  cooking fried okra
n =
    37
>> s = sprintf('the answer is %d\n cooking %s\n', ...
               a,          b)
```

continued on next page

```

s =
the answer is 42
  cooking fried Okra
>> str = input('Enter the data: ', 's');
Enter the data: 42 3.14159 -1
A = sscanf( str,'%f')
A =
    42.0000
     3.1416
    -1.0000
>>

```

However, there is the restriction that either the strings must be of the same length or one of them must be of length 1 before it is legal to compare them

Common Pitfalls

The `if` statement uses a logical expression as its controlling test; therefore, it is bound by the same comparison rules as those applied to vectors. Two strings being compared must be of the same length, and all of the comparisons must match to result in a logical `true`. Frequently, we expect the `if` statement to compare strings of unequal length. However, this will cause an error the first time two strings of unequal length are compared. You should use the `switch` statement, which will correctly compare strings of unequal length in the case tests.

with these operators. To avoid this restriction, MATLAB provides the C-style function `strcmp(<s1>, <s2>)` that returns `true` if the strings are identical and `false` if they are not. Unfortunately, this is not quite the same behavior as the C version, which does a more rigorous comparison returning -1, 0, or 1. You can try a character string comparison in Exercise 6.7.



Exercise 6.7 Character string comparison

Do It Yourself

Enter the following into the Command window and study the results:

```

>> 'abcd' == 'abcd'
     1     1     1     1
>> 'abcd' == 'abcde'
??? Error using ==> eq
Array dimensions must match for binary array op.
>> strcmp('abcd', 'abcde')
ans =
     0
>> strcmp('abcd', 'abcd')
ans =
     1
>> 'abc' == 'a'
ans =
     1     0     0
>> strcmpi('ABcd', 'abcd')
ans =
     1

```

In Exercise 6.7 we see that strings of the same length compare exactly to vectors returning a logical vector result. You cannot use the equality test on strings of unequal length. `strcmp(...)` deals gracefully with strings of unequal length. As with vectors, the equality test works if one of the inputs is a single character. For case-independent testing, use `strcmpi(...)`.



6.5 Arrays of Strings

Since a single character string is stored as a vector, it seems natural to consider storing a collection of strings as an array. The most obvious way to do this, as shown in previous examples, has some limitations, for which there are nice, tidy cures built into MATLAB. Consider the example shown in Exercise 6.8. Character

arrays can be constructed by either of the following:

- As a vertical vector of strings, all of which must be the same length
- By using a special version of the `char( & )` cast function that accepts a variable number of strings with different lengths, pads them with blanks to make all rows the same length, and stores them in an array of characters

Common Pitfalls

Trying to concatenate strings of unequal length vertically into column arrays of strings will cause errors because the vertical concatenation must use rows of equal length. Use the version of the `char(...)` function that pads the strings with spaces.



Exercise 6.8 Character string arrays

Do It Yourself

Enter the following into the Command window:

```
>> v = ['Character strings having more than'
        'one row must have the same number '
        'of columns just like arrays!      ']

v =
Character strings having more than
one row must have the same number
of columns just like arrays!
>> v = ['MATLAB gets upset'
        'when rows have'
        'different lengths']
??? Error using ==> vertcat
All rows in the bracketed expression must have the
same number of columns.

>> eng=char('Timoshenko','Maxwell','Mach','von Braun')
eng =
Timoshenko
Maxwell
Mach
von Braun
>> size(eng)
ans =
     4     10
```


6.6 Engineering Example—Encryption

The Problem:

As public access to information becomes more pervasive, there is increasing interest in the use of encryption to protect intellectual property and private communications from unauthorized access. The following discussion is based on no direct knowledge of the latest encryption technology. However, it illustrates a very simple approach to developing an algorithm that is immune to all but the most obvious, brute-force code-breaking techniques.

Background:

Historically, simple encryption has been accomplished by substituting one character for another in the message, so that 'Fred' becomes 'Iuhg' when substituting the letter three places down the alphabet for each letter in the message. More advanced techniques use a random letter selection to substitute new letters. However, any constant letter substitution is vulnerable to elementary code-cracking techniques based on the frequency of letters in the alphabet, for example.

The Solution:

We propose a simple algorithm where a predetermined random series is used to select the replacement letters. Since the same letter in the original message is never replaced by the same substitute, no simple language analysis will crack the code. The MATLAB `rand(...)` function is an excellent source for an appropriate random sequence. If the encryption and decryption processes use the same value to seed the generator, the same sequence of apparently random (**pseudo-random**) values will be generated. Since the seed can take on $2^{31}-2$ values, it is virtually impossible to determine the decryption without knowing the seed value. The seed (i.e., the decryption key) can be transmitted to anyone authorized to decrypt the message by any number of ways.

Furthermore, since there are abundant different techniques for generating pseudo-random sequences, the specific generation technique must be known in addition to the seed value for successful decryption. Listing 6.1 shows the code for encrypting and decrypting by this technique, and two attempts to decrypt—once with the wrong key and once with the wrong generator.

Listing 6.1 Encryption exercise

```
1. clear
2. clc
```

continued on next page

```

3. disp('original text')
4. txt = ['For example, consider the following:' 13 ...
5.   'A = [4.7 1321454.47 4.8];' 13 ...
6.   'index = 1;' 13 ...
7.   'v = 'values'';' 13 ...
8.   'str = sprintf('%8s of A(%d) are \t%8.3f ' 13 ...
9.     v, index, A(index,1) ' 13 ...
10.  'str = ' 13 ...
11.  ' values of A(1) are 4.700' 13 ...
12.  'The first conversion, '%8s'', took the value' ...
13.  ' of the first ' ...
14.  'parameter, v, allowed 8 spaces. ' 13 ]

%% encryption section
15. rand('state', 123456)
16. loch = 33;
17. hich = 126;
18. range = hich+1-loch;
19. rn = floor( range * rand(1, length(txt) ) );
20. change = (txt>=loch) & (txt
21.   enc = txt;
22.   enc(change) = enc(change) + rn(change);
23.   enc(enc > hich) = enc(enc > hich) - range;
24.   disp('encrypted text')
25.   encrypt = char(enc)

%% good decryption
26. rand('state', 123456);

27. rn = floor( range * rand(1, length(txt) ) );
28. change = (encrypt>=loch) & (encrypt<=hich);
29. dec = encrypt;
30. dec(change) = dec(change) - rn(change) + range;
31. dec(dec > hich) = dec(dec > hich) - range;
32. disp('good decrypt');
33. decrypt = char(dec)

%% bad seed
34. rand('seed', 123457);

35. rn = floor( range * rand(1, length(txt) ) );
36. change = (encrypt>=loch) & (encrypt<=hich);
37. dec = encrypt;
38. dec(change) = dec(change) - rn(change) + range;
39. dec(dec > hich) = dec(dec > hich) - range;
40. disp('decrypt with bad seed')
41. decrypt = char(dec)

%% different generator
42. rand('seed', 123456)
43. rn = mod(floor( range * abs(randn(1, length(txt) ))/10 ),
...
                                range);

```

continued on next page

```
44. change = (encrypt>=loch) & (encrypt<=hich);  
45. dec = encrypt;  
46. dec(change) = dec(change) - rn(change) + range;  
47. dec(dec > hich) = dec(dec > hich) - range;  
48. disp('decrypt with wrong generator')  
49. decrypt = char(dec)
```

In Listing 6.1:

Lines 4–14: This is the original text taken from earlier in this chapter. Multiple lines of characters can be concatenated as shown. The number 13 inserted in the string is the numerical equivalent of the new line escape sequence, `'\n'`.

Line 15: Seeds the random generator with a known value.

Lines 16–18: Set the upper and lower bounds and the range of the characters we will convert. This range excludes 32, the space character, and 13, the new line character. This choice was deliberate—it leaves the encrypted text with the appearance of a character substitution algorithm since all the characters are printable, and seem to be grouped in words.

Line 19: Generates the random values between 0 and `range-1`.

Line 20: Identifies the indices of the printable characters.

Line 21: Makes a copy of the original text.

Line 22: Adds the random offsets to those characters we intend to change.

Line 23: If this pushes a character value above the maximum printable character, here it brings it back within range.

Lines 24 and 25: Display the encrypted text. Notice that no two characters of the original text are replaced by the same character.

Lines 26–29: Begin the decryption by seeding the generator with the same value, creating the same random sequence, finding the printable characters, and copying the original file to the decrypt string.

Lines 30 and 31: We must subtract the random sequence from the encrypted string and correct for the underflow. However, there are some numerical issues involved. It is best to add the `range` value to all the letters while subtracting the random offsets, and then bring back those values that remain above the highest printable character.

Lines 32 and 33: Display the decrypted values and test that they are identical to the original.

Lines 34–41: Attempt to decrypt with the same code but a bad seed.

Lines 42–49: Attempt to decrypt with the right seed but a different generator—in this case, MATLAB's normal random generator limited to positive values within the letter range of interest.

Listing 6.2 shows the output from this encryption exercise.

Listing 6.2 Encryption exercise results

```

original text
txt =
For example, consider the following:
A = [4.7 1321454.47 4.8];
index = 1;
v = 'values';
str = sprintf('%8s of A(%d) are \t%8.3f
             v, index, A(index,1)
str =
    values of A(1) are    4.700
The first conversion, '%8s', took the value
of the first parameter, v, allowed 8 spaces.

encrypted text
encrypt =
@;J_a,Q/V_Q X/|IW?*q %;{ $Ctr:$&r3>
5 - v$zh uvqzmE@P(N Bh}.H
_3lc> / b/
q ( <0"z<F[Wg
eWP _ )QrWEyeV153u 2} -H]7I gbm U>}U,5b
    km cof+:J 28K?d8`49{
Dkp m
    t.7[S\ Zh Lmk; v<      .?&!S
:s6 #p.x5 W{aycv0${n! M;sc:\ U[QT nft ySET6
    fK ms_ @|MFn J,PSyKuM"+ G# 3`VL^sJ f .|2B8W?

good decrypt
decrypt =
For example, consider the following:
A = [4.7 1321454.47 4.8];
index = 1;
v = 'values';
str = sprintf('%8s of A(%d) are \t%8.3f
             v, index, A(index,1)
str =
    values of A(1) are    4.700
The first conversion, '%8s', took the value
of the first parameter, v, allowed 8 spaces.

decrypt with bad seed
decrypt =
tDQ <6VfMiS^ }lFI92/P c`@ eYrW%Q^2t+
6 L 4x5> B$rQ4XHpG# G;*<r
[8La^ ( $J
E 7 Mjpbq~,I>(
GH4 1 ^YgRpX}}>0&v [$ n|pA| />O OML-jKc
    3C f*q(WK (3oQQC\HpS
*a= +
    og>FK> .u Od]C :wf    J$"qQ
.&i wZwPo .=";~s6!-5g ~d7C}t >Q<z s}c HHZZh
    7u \uV f_h84 p(M&3kzGnP ci r1jnRm| m t?!>FP;

```

continued on next page

```
decrypt with wrong generator
decrypt =
>lE o-P:'P=p :xLjV+bi {!d 3)[Az$~c7<
' l fny& tHWB Vve6o
(mRdc T W$
= Q Z"'$;e[@{
/<* / )tokB]h:;ODu o^ 4=A0- z@0 P"!+axV
_U `LJsrrj ^,Ia7]V3bq
_P* 5
|v^)Z+ ?! Ld` } 5$) DKN54
o-U @^%'o u[YOzAV06JV i\(E^I X)cC -X` uu2.\
dq 7:D diq#l Y+sDW*FiF" $u eeF832} ? E,wc97*
```



Chapter Summary

This chapter discussed the nature, implementation, and behavior of character strings. We learned the following:

- Character strings are merely vectors of numbers that MATLAB presents to the user as single characters
- We can perform on strings the same operations that can be performed on vectors; if mathematical operations are performed, MATLAB converts the characters to double values
- We can convert between string representations of numbers and the numbers themselves using built-in MATLAB functions
- MATLAB provides functions that convert numbers to text strings for presentation in the Command window
- Arrays of strings can be assembled using the `char(...)` function



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
'...'	Encloses a literal character string	6.2
char(...)	Casts to a character type	6.2, 6.5
double(a)	Casts to type double	6.2
ischar(ch)	Determines whether the given object is of type char	6.2.3
isspace(a)	Tests for the space character	6.2.3
uint8(...)	Casts to unsigned integer type with 8 bits	6.2
int2str(a)	Converts an integer to its numerical representation	6.3.1

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>num2str(a,n)</code>	Converts a number to its numerical representation with <code>n</code> decimal places	6.3.1
<code>disp(...)</code>	Displays matrix or text	6.4.1
<code>fprintf(...)</code>	Prints formatted information	6.4.2
<code>input(...)</code>	Prompts the user to enter a value	6.3.2
<code>sscanf(...)</code>	Formats input conversion	6.3.2
<code>sprintf(...)</code>	Formats a string result	6.3.1
<code>strcmp(s1, s2)</code>	Compares two strings; returns <code>true</code> if equal	6.4.3
<code>strcmpi(s1, s2)</code>	Compares two strings without regard to case; returns <code>true</code> if equal	6.4.3



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. Casting changes the value of a piece of data.
2. The ASCII code maps individual characters to their internal numerical representation.
3. Because the single quote mark (`'`) delimits strings, you cannot use it within a string.
4. If you attempt mathematical operations on a character string, MATLAB will throw an error.
5. The function `disp(...)` can display multiple values to the Command window.
6. The function `strcmp(...)` throws an error if the two strings are of unequal length, unless one of them is a single character.
7. The `switch` statement will correctly compare strings of unequal length in the `case` tests.

Fill in the Blanks

1. Numerical values are stored in MATLAB in _____ for efficient numerical computation.

2. Most common _____, _____, and many _____ are represented in ASCII by the numbers _____.
3. The function _____ casts a string to a vector of the same length as the string containing the numerical mapping of _____.
4. The function `fprintf(...)` requires a _____ that defines exactly how the resulting string should be formatted and a variable number of _____.
5. Since the _____ statement tests a logical expression, it _____ test strings of unequal length.
6. A special version of the _____ cast function accepts strings with different lengths, _____, and stores them in an array of characters.



Programming Projects

1. Solve the following introductory problems on strings.
 - a. Write a function `dayName` that consumes a parameter, `day`, containing the numerical value of a day in the month of September 2008. Your function should return the name of that day as a string. For example:
 `dayName( 8 )` should return `'Monday'`
 - b. You are now given a variable named `days`, a vector that contains the numeric values of days in the month of September 2008. Write a script that will convert each numeric value in the vector `days` into a string named `daysOfWeek` with the day names separated by a comma and a space.

Hint

You should probably be concatenating the day names and the delimiters.

For example:

if `days = [8, 9, 10]`, `daysOfWeek` should be
`'Monday, Tuesday, Wednesday'`

Notice that there is no separator before the first day name or after the last one.

2. Consider the problem the MATLAB system has in parsing the string:

```
'V=[1 2 3 4; 5,6, 7;8; 9 10]'
```

Your task is to use `strtok` to parse this line and construct the array it represents. You will write a function `arrayParse` that consumes a string and returns two variables: a string that is the variable name and an array.

- a. Tokenize the string first using '=' as the delimiter to isolate the variable name and the expression to be evaluated. Return the variable name to the user and save the rest of the line as the variable `str1` for further processing. You may assume that there are no spaces outside the characters '['].
- b. Tokenize `str1` with '[' and ']' to remove the concatenation operators and save the first token as `str2`.

Hint

To understand string tokenizing, in the Command Window, enter `>> help strtok`.

- c. Tokenize `str2` using ';' as the delimiter. This will produce 0 or more strings that represent the rows of the array. Save each in the variable `rowString`. You may assume for now that the first row is the longest one.

- d. Using nested while loops, tokenize each `rowString` with ',' and '' as delimiters and use `sscanf(...)` to extract the numerical value of each array entry. Save it as `rowEntry`.
- e. Concatenate the `rowEntry` elements horizontally to produce each row of the array. If the row is too short, pad it with zeros.
- f. Concatenate each row vertically to produce the resulting array and return that array to the caller.
- g. Test the function with cases like:

```
empty=[]
row=[1 2 3 4]
diag=[0 0 0 1; 0 0 1; 0 1; 1]
```

3. Write a function called `DNAcomplement` that consumes a set of letters as a character array that form a DNA sequence such as 'gattaca'. The function will produce the complement of the sequence so that a's become t's, g's become c's, and vice versa. The string 'gattaca' would therefore become 'ctaattg'. You may assume that all the letters in the sequence will be lowercase and that they will all be either a, t, g, or c.

Note: You may be tempted to use iteration for this problem, but you don't need it.

4. The function `rot(s, n)` is a simple Caesar cipher encryption algorithm that replaces each English letter in places forward or

backward along the alphabet in the strings. For example, the result of `rot('Baz!',3)` is `'Edc!'`. An encrypted string can be deciphered by simply performing the inverse rotation on it, i.e., `rot('Edc!',3)`, which rotates each English letter in the strings three places to the left. Numbers, symbols, and non-letters are not transformed. Implement the following function:

```
function rotatedText=rot(text,n)
```

To assist you as you solve this problem, you could write several functions as local functions in the `rot.m` file:

```
isUppercaseLetter(letter), getUppercaseLetter(n),  
getLowercaseLetter(n), and getPosition(letter). You may also  
wish to use the built-in MATLAB functions isletter(...), find(...),  
and mod(...).
```

5. You have a big problem. In one of your CS courses, your professor decides that the only way you will pass the class is if you write a MATLAB function to get him out of a mess. All the grades in his class have been accidentally stored into one long string of characters containing only the letters A, B, C, D, F, and Y.
 - a. Your job is to write a function called `CrazyGrade` that will take in the string and flip the grades according to the following specifications:
 - A becomes F
 - B becomes D
 - C remains unchanged
 - D becomes B
 - F becomes A
 - Y becomes W

Your function should take in a string and return an inverted string. You may assume that the string will only consist of valid letter grades. For example,

```
CrazyGrade('BADDAD') should return 'DFBBFB'
```

```
CrazyGrade('BAYBAY') should return 'DFWDFW'
```

- b. To make matters worse, he wants you to organize this modified grade set. Write a function called `GradeDist` to bunch together all the similar grades (put all the A's next to each other, B's next to each other, etc.) Then calculate and return the professor's grade distribution. Your function should take in a string and return a string with all similar grades grouped together, along with an array containing percentage values from A's all the way to F's.

For example, if there are 15% A's, 16% B's, 33% C's, 16% D's, 16% F's and 4% W's, `GradeDist` should return `[15 16 33 16 16 4]`.

Cell Arrays and Structures

Chapter Objectives

This chapter discusses the nature, implementation, and behavior of collections that may contain data items of any class, size, or shape. We will deal with two different heterogeneous storage mechanisms:

- Those accessed by index (cell arrays)
- Those accessed by name (structures)

In addition, we will consider collecting structures into arrays of structures.

Introduction

This chapter covers data collections that are more general and flexible than the arrays we have considered so far. Heterogeneous collections may contain objects of any type, rather than just numbers. Consequently, none of the collective operations defined for numerical arrays can be applied to cell arrays or structures. To perform any operations on their contents, the items must be extracted one at a time and replaced if necessary. We will consider three different mechanisms for building heterogeneous collections: cell arrays index their contents with a numerical index; structures index their contents with a symbolic index; and structure arrays index structures with a numerical index.

- 7.1** Concept: Collecting Dissimilar Objects
- 7.2** Cell Arrays
 - 7.2.1 Creating Cell Arrays
 - 7.2.2 Accessing Cell Arrays
 - 7.2.3 Using Cell Arrays
 - 7.2.4 Processing Cell Arrays
- 7.3** MATLAB Structures
 - 7.3.1 Constructing and Accessing One Structure
 - 7.3.2 Constructor Functions
- 7.4** Structure Arrays
 - 7.4.1 Constructing Cell Arrays
 - 7.4.2 Accessing Structure Elements
 - 7.4.3 Manipulating Structures
- 7.5** Engineering Example—Assembling a Structure

7.1 Concept: Collecting Dissimilar Objects

Heterogeneous collections permit objects of different data types to be grouped in a collection. They allow data abstraction to apply to a much broader range of content. However, the fact that the contents of these collections may be of any data type severely restricts the operations that can be performed on the collections as a whole. Whereas a significant number of arithmetic and logical operations can be performed on whole number arrays, algorithms that process heterogeneous collections must deal with the data contents one item at a time.

7.2 Cell Arrays

Cell arrays, as the name suggests, have the general form of arrays and can be indexed numerically as arrays. However, each element of a cell array should be considered as a container in which one data object of any class can be stored. They can be treated as arrays of containers for the purpose of concatenation and slicing. However, if you wish to access or modify the contents of the containers, the cells must be accessed individually.

7.2.1 Creating Cell Arrays

Cell arrays may be constructed in the following ways:

- By assigning values individually to a variable indexed with braces:

```
>> A{1} = 42
A =
    [42]
```

- By assigning containers individually to a variable indexed with brackets:

```
>> B[1] = {[4 6]};
B =
    [1x2 double]
```

- By concatenating cell contents using braces { . . . }:

```
>> C = {3, [1,2,3], 'abcde'}
C =
    [3] [1x3 double] 'abcde'
```

- By concatenating cell containers:

```
>> D = [A B C {'xyz'}]
D =
    [42] [1x2 double] [3] [1x3 double] 'abcde' 'xyz'
```

Based on these examples, we observe the following:

- A cell array can contain any legal MATLAB object
- Just as with number arrays, cell arrays can be created “on the fly” by assigning values to an indexed variable

When the values from a cell array are displayed, their appearance is different from that of the contents of a number array. Individual numbers are shown in brackets, for example, `[3]`; larger numerical arrays display their size, for example, `[1x3 double]`; and character strings are displayed with the enclosing quotes, for example, `'abcde'`.

7.2.2 Accessing Cell Arrays

Since cell arrays can be considered as conventional arrays of containers, the containers can be accessed and manipulated normally. For example, continuing the previous examples, we have the following:

```
>> E = D(2) % parentheses - a container
E =
    [4 6]
```

However, braces are used to access the contents of the containers as follows:

```
>> D{2} % braces - the contents
ans =
    4    6
```

If the right-hand side of an assignment statement results in multiple cell arrays, the assignment must be to the same number of variables. The built-in MATLAB function `deal(...)` is used to make these allocations. Exercise 7.1 shows its use.



Exercise 7.1 Cell arrays

Do It Yourself

Enter the following commands in the Command window and study the results:

```
>> A = { 3, [1,2,3] 'abcde'}
A =
    [3]    [1x3 double] 'abcde'
>> A{1:2}
ans =
    3
ans =
    1    2    3
>> [x y] = A{1:2}
x =
    3
y =
    1    2    3
>> B = A{1:2}
??? Illegal right-hand side in assignment.
    Too many elements.
>> B([1 3]) = A([1 2])
B =
    [3]    []    [1x3 double]
```

continued on next page

```

>> B{[1 3]} = A{[1 2]}
??? Illegal right-hand side in assignment.
    Too many elements.
>> [a, b, c] = deal(A{:})
a =
    3
b =
    1    2    3
c =
abcde
>> [a, b] = deal(A)
a =
    [3]    [1x3 double]    'abcde'
b =
    [3]    [1x3 double]    'abcde'
>> B = A(1:2)
B =
    [3]    [1x3 double]
>> for i = 1:2
    s(i) = sum(A{i})
end
s =
    3
s =
    3    6
>> F{2} = 42
F =
    []    [42]
>> F{3} = {42}
F =
    []    [42]    {1x1 cell}

```

Notice the following observations:

- When we extract the contents of multiple cells using `A{1:2}`, this results in multiple assignments being made. These multiple assignments must go to separate variables. This is the fundamental mechanism behind returning multiple results from a function.
- These multiple assignments cannot be made to a single variable; sufficient storage must be provided either as a collection of variables or explicitly as a vector.
- Cell arrays can be “sliced” with normal vector indexing assignments as long as the sizes match on the left and right sides of the assignment. Any unassigned array elements are filled with an empty vector.
- The assignment `B{[1 3]} = A{[1 2]}` that produced an error needs some thought. Since `A{[1 2]}` produces two separate assignments, MATLAB will not assign the answers, even to the right number of

places in another cell array. The `deal(...)` function is provided to capture these multiple results in different variables. Notice the difference between `A{:}` and `A` as a parameter to `deal(...)`. When `deal(...)` is provided with a parameter other than a collection of cells, it copies that parameter to each variable.

- Assignments work normally if cell arrays are treated as vectors and the extraction of items can be indexed—`s` is a vector of the sums of the elements in `A`.
- Finally, notice that when accessing cell arrays, it is normal to have braces on one side or the other of an assignment; it is rarely appropriate to have braces on both sides of an assignment. The result here is that a cell array is loaded into the third container in the cell array.

7.2.3 Using Cell Arrays

There are a number of uses for cell arrays in MATLAB, some of which will be evident in upcoming chapters. For now, the following examples will suffice:

- Containing lists of possible values for `switch/case` statements, as we saw in Chapter 4
- Substituting for parameter lists in function calls

For example, suppose you have a function `largest(a, b, c)` that consumes three variables and produces the largest of the three values provided. It can be used in the following styles, as shown in Listing 7.1.

In Listing 7.1:

Lines 1–3: Set the values of `A`, `B`, and `C`.

Line 4: A conventional function call that results in a value of 6 for `N`.

Line 5: The same function call implemented as a cell array, returning the same answer.

7.2.4 Processing Cell Arrays

The general template for processing cell arrays is shown in Template 7.1.

Listing 7.1 Using cell arrays of parameters

```
1. A = 4;
2. B = 6;
3. C = 5;
4. N = largest(A, B, C)
5. params = { 4, 6, 5 };
6. N = largest(params{1:3})
```

Template 7.1 General template for processing cell arrays

```

<initialize result>
for <index specification>
    <extract an element>
    <check the element accordingly>
    <process the element accordingly>
end
<finalize result>

```

Checking the class of the element can be achieved in one of two ways:

- The function `class(item)` returns a string specifying the item type that can be used in a switch statement
- Individual test functions can be used in an `if... elseif` construct; examples of the individual test functions are `isa(item, 'class')`, `iscell(...)`, `ischar(...)`, `islogical(...)`, `isnumeric(...)`, and `isstruct(...)`

For example, suppose you are provided with a cell array and have been asked for a function that finds the total length of all the vectors it contains. The function might look like that shown in Listing 7.2.

In Listing 7.2:

Line 1: Typical function header accepting a cell array as input.

Line 3: Initializes the result.

Line 4: Traverses the whole cell array.

Line 5: Extracts each item in turn.

Line 6: Determines whether this item is of type `double`.

If so, it proceeds to line 7.

Line 7: Accumulates the number of items in this array. Recall that the `size(...)` function returns a vector of the sizes of each dimension. The total number of numbers is therefore the product of these values.

Listing 7.2 Cell array processing example

```

1. function ans = totalNums(ca)
2. % count the numbers in a cell array
3.     ans = 0 ;
4.     for in = 1 :length(ca)
5.         item = ca{in} ;      % extract the item
6.         if isnumeric(item) % check if a vector
7.             ans = ans + prod(size(item));
8.         end
9.     end

```



7.3 MATLAB Structures

Where cell arrays implemented the concept of homogeneous collections as indexed collections, structures allow items in the collection to be indexed by field name. Most modern languages implement the concept of a structure in a very similar style. The data contained in a structure is referenced by field name, for example, `item1`. The rules for making a field name are the same as those for a variable. Fields of a structure, like the elements of a cell array, are heterogeneous—they can contain any MATLAB object.

First we will see how to construct and manipulate one structure, and then how to aggregate individual structures into an array of structures.

7.3.1 Constructing and Accessing One Structure

To set the value of items in a structure `A`, the syntax is as follows:

```
>> A.item1 = 'abcde'
A =
    item1: 'abcde'
>> A.item2 = 42
A =
    item1: 'abcde'
    item2: 42
```

Notice that MATLAB displays the elements of an emerging structure by name. Fields in a structure are accessed in the same way—by using the dotted notation.

```
>> A.item2 = A.item2 ./ 2
A =
    item1: 'abcde'
    item2: 21
```

You can determine the names of the fields in a structure using the built-in function `fieldnames(...)`. It returns a cell array containing the field names as strings.

```
>> names = fieldnames(A)
names =
    'item1'
    'item2'
```

Fields can also be accessed “indirectly” by setting a variable to the name of the field, and then using parentheses to indicate that the variable contents should be used as the field name:

```
>> fn = names{1};
>> A.(fn) = [A.(fn) 'fg']
A =
    item1: 'abcdefg'
    item2: 21
```


Common Pitfalls

Be careful. `rmfield(...)` returns a new structure with the requested field removed. It does not remove that field from your original structure. If you want the field removed from the original, you must assign the result from `rmfield(...)` to replace the original structure:

```
>> A = rmfield(A, 'item1')
A =
    item2: 21
```

You can also remove a field from a structure using the built-in function `rmfield(...)`. Exercise 7.2 gives you an opportunity to understand how to build structures.

Here we build a typical structure that could be used as one entry in a telephone book. Since phone numbers usually contain punctuation, we could store them as strings. Notice that since a

structure may contain any object, it is quite legal to make a structure containing

**Exercise 7.2** Building structures***Do It Yourself***

Suppose that you want to use structures to maintain your address book in MATLAB. In the Command window, enter the following commands:

```
>> entry.first = 'Fred'
entry =
    first: 'Fred'
>> entry.last = 'Jones';
>> entry.phone = '(123) 555-1212'
entry =
    first: 'Fred'
    last: 'Jones'
    phone: '(123) 555-1212'
>> entry.phone
ans =
    (123) 555-1212
>> date.day = 31;
>> date.month = 'February';
>> date.year = 1965
date =
    day: 31
    month: 'February'
    year: 1965
>> entry.birth = date
entry =
    first: 'Fred'
    last: 'Jones'
    phone: '(123) 555-1212'
    birth: [1x1 struct]
>> entry.birth
ans =
    day: 31
    month: 'February'
    year: '1965'
>> entry.birth.year
ans =
    1965
```

a date and insert that structure in the date field of the entry. The structure display function, however, does not display the contents of the structures.

7.3.2 Constructor Functions

This section discusses functions that assign their parameters to the fields of a structure and then return that structure. You do this, as opposed to “manually” entering data into structures, for the following reasons:

- Manual entry can result in strange behavior due to typographical errors or having fields in the wrong order
- The resulting code is generally more compact and easier to understand
- When constructing collections of structures, it enforces consistency across the collections

There are two approaches to the use of constructor functions: using built-in MATLAB capabilities and writing your own constructor. MATLAB has a built-in function, `struct(...)`, that consumes pairs of entries (each consisting of a field name as a string and a cell array of field contents) and produces a structure. If all the cell arrays have more than one entry, this actually creates a structure array, as discussed in Section 7.4.1.

The following command would construct the address book entry created in the previous section. Note the use of ellipses (...) to indicate to the MATLAB machinery that the logic is continued onto the next line.

```
>> struct('first','Fred', ...
'last','Jones', ...
'phone','(123) 555-1212', ...
'birth', struct( 'day', 31, ...
                 'month', 'February', ...
                 'year', 1965 ))
ans =
    first: 'Fred'
    last: 'Jones'
    phone: '(123) 555-1212'
    birth: [1x1 struct]
```

This is useful in general to create structures, but the need to repeat the field names makes this general purpose approach a little annoying. We can create a special purpose function that “knows” the necessary field names to create multiple structures in an organized way.

Listing 7.3 shows the code for a function that consumes parameters that describe a CD and assembles a structure containing those attributes by name.

In Exercise 7.3 you can try your hand at using this function to construct a CD structure and then verify the structure contents.

Listing 7.3 Constructor for a CD structure

```
function ans = makeCD(gn, ar, ti, yr, st, pr)
% integrate CD data into a structure
    ans.genre = gn ;
    ans.artist = ar ;
    ans.title = ti;
    ans.year = yr;
    ans.stars = st;
    ans.price = pr;
```

**Exercise 7.3** A CD structure***Do It Yourself***

In the Command window, enter the following commands to create one entry of CD information:

```
>> CD = makeCD('Blues', 'Charles, Ray', &
'Genius Loves Company', 2004, 4.5, 15.35 )
CD =
    genre: 'Blues'
   artist: 'Charles, Ray'
    title: 'Genius Loves Company'
     year: 2004
    stars: 4.5000
   price: 15.3500
>> flds = fieldnames(CD)
flds =
    'genre'
   'artist'
    'title'
     year'
    'stars'
   'price'
>> field = flds{2}
field =
    artist
>> CD.(field)
ans =
    Charles, Ray
```

**7.4 Structure Arrays**

To be useful, collections like address books or CD collections require multiple structure entries with the same fields. This is accomplished by forming an array of data items, each of which contains the same fields of information. MATLAB implements the concept of structure arrays with the properties described in the following paragraphs.

7.4.1 Constructing Structure Arrays

Structure arrays can be created either by creating values for individual fields, as shown in Exercise 7.4; by using MATLAB's `struct(...)` function to build the whole structure array, as shown in Listing 7.4; or by using a custom function to create each individual structure, as shown in Listing 7.5. This latter listing illustrates these concepts by implementing a collection of CDs as a structure array using the function `makeCD(...)` from Listing 7.3.

In Listing 7.4:

Lines 1–5: Build cell arrays containing field values for five CDs.

Line 6: Uses the built-in `struct(...)` function to create the CD collection.

7.4.2 Accessing Structure Elements

Like normal arrays or cell arrays, items can be stored and retrieved by their index in the array. As structures are added to the array, MATLAB forces all



Exercise 7.4 Building a structure array “by hand”

Do It Yourself

```
>> entry(1).first = 'Fred';
>> entry(1).last = 'Jones';
>> entry(1).age = 37;
>> entry(1).phone = ' (123) 555-1212';
>> entry(2).first = 'Sally';
>> entry(2).last = 'Smith';
>> entry(2).age = 29;
>> entry(2).phone = '(000) 555-1212'
entry =
1x2 structure array with fields:
    first
    last
    age
    phone
```

Listing 7.4 Building a structure array using `struct(...)`

```
1. genres = {'Blues', 'Classical', 'Country' };
2. artists = {'Clapton, Eric', 'Bocelli, Andrea', &
'Twain, Shania' };
3. years = { 2004, 2004, 2004 };
4. stars = { 2, 4.6, 3.9 };
5. prices = { 18.95, 14.89, 13.49 };
6. cds = struct( 'genre', genres, &
'artist', artists, &
'year', years, &
'stars', stars, &
'price', prices);
```

Listing 7.5 Building a structure array using a custom constructor

```
% extracts from http://www.cduniverse.com/ 12/30/04
cds(1) = makeCD('Blues', 'Clapton, Eric', ...
'Sessions For Robert J', 2004, 2, 18.95 )
cds(2) = makeCD('Classical', ...
'Bocelli, Andrea', 'Andrea', 2004, 4.6, 14.89 )
cds(3) = makeCD('Country', 'Twain, Shania', ...
'Greatest Hits', 2004, 3.9, 13.49 )
cds(4) = makeCD('Latin', 'Trevi, Gloria', ...
'Como Nace El Universo', 2004, 5, 12.15 )
cds(5) = makeCD('Rock/Pop', 'Ludacris', ...
'The Red Light District', 2004, 4, 13.49 )
cds(6) = makeCD('R & B', '2Pac', ...
'Loyal To The Game', 2004, 3.9, 13.49 )
cds(7) = makeCD('Rap', 'Eminem', ...
'Encore', 2004, 3.5, 15.75 )
cds(8) = makeCD('Heavy Metal', 'Rammstein', ...
'Reise, Reise', 2004, 4.2, 12.65 )
```

elements in the structure array to implement the same field names in the same order. Elements can be accessed either manually (not recommended) or by creating new structures with a constructor and adding them (recommended).

If you elect to manipulate them manually, you merely identify the array element by indexing, and use the `.field` operator. For example, for the CD collection `cds`, we could change the price of one of them as follows:

```
>> cds(3).price = 11.95
cds =
1x31 struct array with fields:
    genre
   artist
   title
   year
   stars
   price
```

This is a little hazardous when making manual additions to a structure array. A typographical error while entering a field name results in all the structures having that bad field name. For example, consider this error:

```
>> cds(3).prce = 11.95
cds =
1x31 struct array with fields:
    genre
   artist
   title
   year
   stars
   price
  prce
```

You have accidentally added a new field to the whole collection. You can check this by looking at one entry:

```
>> cds(1)
ans =
    genre: 'Blues'
  artist: 'Sessions For Robert J'
   title: 'Clapton, Eric'
    year: 2004
   stars: 2
   price: 18.9500
    prce: []
```

If this happens, you can use the `fieldnames(...)` function to determine the situation, and then the `rmfield(...)` function to remove the offending entry.

```
>> fieldnames(cds)
ans =

    'genre'
    'artist'
    'title'
    'year'
    'stars'
    'price'
    'prce'
```

```
>> cds = rmfield(cds,'prce')
cds =
1x32 struct array with fields:
    genre
   artist
    title
    year
   stars
   price
```

It is best to construct a complete structure and then insert it into the structure array. For example:

```
>> newCD = makeCD( 'Oldies', 'Greatest Hits', ...
    'Ricky Nelson', 2005, 5, 15.79 );
> cds(8) = newCD
cds =
1x8 struct array with fields:
    genre
   artist
    title
    year
   stars
   price
```

If you insert that new CD beyond the end of the array, as one might expect, MATLAB fills out the array with empty structures:

```
>> cds(50) = newCD
cds =
1x50 struct array with fields:
```

Common Pitfalls

A few very understandable but sneaky errors occur when adding structures that have been created “manually” rather than by means of a standardized constructor function. If the new structure has fields not in the original structure, or extra fields, MATLAB gives a slightly obscure error: “Subscripted assignment between dissimilar structures.” Perhaps more puzzling, this same error occurs if all the fields are present, but are in the wrong order. See the following example:

```
>> it = cds(1);
>> it = rmfield(it, 'price');
>> it = rmfield(it, 'stars');
>> it.price = 19.95;
>> it.stars = 4
it =
genre: 'Blues'
artist: 'Sessions For Robert J'
title: 'Clapton, Eric'
year: 2004
price: 19.9500
stars: 4
>> cds(1) = it
??? Subscripted assignment between
dissimilar structures.
the latest versions of MATLAB take care of
field re-ordering and do not cause this error.
Note: the latest versions of MATLAB takes care of field re-
ordering and does not cause this error.
```

```
genre
artist
title
year
stars
price
>> cds(49)
ans =
    genre: []
   artist: []
    title: []
     year: []
    stars: []
    price: []
```

7.4.3 Manipulating Structures

Structures and structure arrays can be manipulated in the following ways:

Single values can be changed using the “.” (dot) notation directly with a field name:

```
>> cds(5).price = 19.95;
```

or indirectly using the “.” (dot) notation with a variable containing the field name:

```
>> fld = 'price';
>> cds(5).(fld) = 19.95;
```

or by using MATLAB’s equivalent built-in functions:

`nms = fieldnames(str)` returns a cell array containing the names of the fields in a structure or structure array.

```
>> flds = fieldnames(cds)
```

`if = isfield(str, <fldname>)` determines whether the given name is a field in this structure or structure array.

```
>> if isfield(cds, 'price') ...
```

`str = setfield(str, <fldname>, <value>)` returns a new structure array with the specified field set to the specified value.

```
>> cds(1) = setfield(cds(1), ...
    'price', 19.95);
```

`val = getfield(str, <fldname>)` returns the value of the specified field.

```
>> disp(getfield(cds(1), 'price') );
```

`str = rmfield(str, <fldname>)` returns a new structure array with the specified field removed.

```
>> noprice = rmfield(cds, 'price');
```

Values across the whole array can be retrieved using the "." notation by accumulating them into arrays; either into cell arrays:

```
>> titles = {cds.title};
```

or if the values are all numeric, into a vector:

```
>> prices = [cds.price];
```

Exercise 7.5 provides some practice in manipulating structure arrays using the above CD collection as an example.

Notice that after extracting the price values into a vector, all the normal vector operations—in this case, `sum(...)`—can be utilized.



Exercise 7.5 The CD collection

Do It Yourself

Retrieve and run the script named `buildCDs.m` from the book's Companion Website (www.aw.com/smith_mat). In the Command window, enter the following commands to create your collection of CD information:

```
>> cds(5)
ans =
    genre: 'Rock/Pop'
   artist: 'Ludacris'
    title: 'The Red Light District'
     year: 2004
    stars: 4
   price: 13.4900
>> flds = fieldnames(collection)
flds =
    'genre'
   'artist'
    'title'
    'year'
    'stars'
    'price'
cds(5).strs = 0.5;
>> cds(5)
ans =
    genre: 'Rock/Pop'
   artist: 'Ludacris'
    title: 'The Red Light District'
     year: 2004
    stars: 4
   price: 13.4900
    strs: 0.5
```

continued on next page


```

>> cds(1)
ans =
    genre: 'Blues'
   artist: 'Clapton, Eric'
    title: 'Sessions For Robert J'
    year: 2004
   stars: 2
   price: 18.9500
    strs: []
>> cds = rmfield(cds, 'strs');
>> cds(1)
ans =
    genre: 'Blues'
   artist: 'Clapton, Eric'
    title: 'Sessions For Robert J'
    year: 2004
   stars: 2
   price: 18.9500
>> sum([cds.price])
ans =
    409.1100

```

7.5 Engineering Example—Assembling a Structure

Many large buildings today have steel frames as their basic structure. Engineers perform the analysis and design work for each steel component and deliver these designs to the steel company. The steel company manufactures all the components, and prepares them for delivery to the building site. At this point, each component is identified only by a unique identifier string stamped and/or chalked onto that component. For even a modest-sized building, this transportation may require a significant number of truckloads of components.

The question we address here is how to decide the sequence in which the components are delivered to the building site so that components are available when needed, but not piled up waiting to be used.

Consider the relatively simple structure shown in Figure 7.1. The components have individual labels, and we can obtain from the architect the identities of the components that are connected together. The construction needs to start from the fixed point A. We need to analyze this information and compute the order in which the components would be used to assemble the structure.

The data will be organized as a structure array with one entry for each component. One of the fields in that structure will be a cell array of the names of the components to which this component is connected.

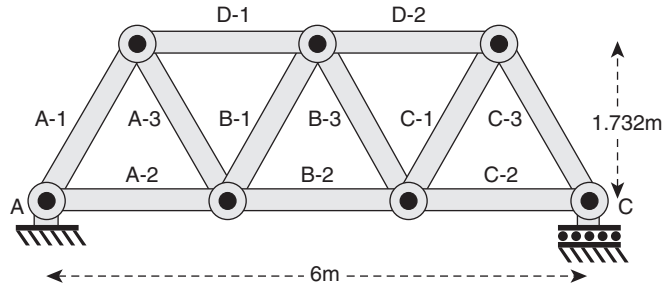


Figure 7.1 Simple structure assembly

The code in Listing 7.6 shows the solution to this problem.

Listing 7.6 Connectivity of a structure

```
clear
clc

1. data(1) = beam('A-1', 0.866, 0.5, ...
    {'A', 'A-2', 'A-3', 'D-1'} );
2. data(2) = beam('A-2', 0, 1, ...
    {'A', 'A-3', 'B-1', 'B-2'} );
3. data(3) = beam('A-3', 0.866, 1.5, ...
    {'A-1', 'A-2', 'B-1', 'D-1'} );
4. data(4) = beam('B-1', 0.866, 2.5, ...
    {'A-2', 'A-3', 'B-2', 'B-3', 'D-1', 'D-2'} );
5. data(5) = beam('B-2', 0, 3, ...
    {'A-2', 'A-3', 'B-1', 'B-3', 'C-1', 'C-2'} );
6. data(6) = beam('B-3', 0.866, 3.5, ...
    {'B-1', 'B-2', 'C-1', 'C-2', 'D-1', 'D-2'} );
7. data(7) = beam('C-1', 0.866, 4.5, ...
    {'B-2', 'B-3', 'C-2', 'C-3', 'D-2'} );
8. data(8) = beam('C-2', 0, 5, ...
    {'B-2', 'B-3', 'C-1', 'C-3', 'C'} );
9. data(9) = beam('C-3', 0.866, 5.5, ...
    {'C-1', 'C-2', 'D-2', 'C'} );
10. data(10) = beam('D-1', 1.732, 2, ...
    {'A-1', 'A-3', 'B-1', 'B-3', 'D-2'} );
11. data(11) = beam('D-2', 1.732, 4, ...
    {'B-1', 'B-3', 'C-1', 'C-3', 'D-1'} );

12. conn = 'A';
13. clist = {conn};
14. while true
15.     index = 0;
    % find all the beams connected to conn
16.     for in = 1:length(data)
17.         str = data(in);
18.         if touches(str, conn)
19.             index = index + 1;
20.             found(index) = str;
21.         end
22.     end
```

continued on next page

```

        % eliminate those already connected
23.     for jn = index:-1:1
24.         if ison(found(jn).name, clist)
25.             found(jn) = [];
26.         else
27.             clist = [clist {found(jn).name}];
28.         end
29.     end
30.     if length(found) > 0
31.         conn = nextconn( found, clist );
32.     else
33.         break;
34.     end
35. end
36. disp('the order of assembly is:')
37. disp(clist)

```

In Listing 7.6:

Lines 1–11: Construct the structure array using the `beam(...)` constructor function below.

Line 12: The current connection point, `conn`—originally, the point A.

Line 13: Initializes the connection list, a cell array of names.

Line 14: An infinite loop to be exited with `break` statements.

Lines 15–22: Traverse the components to make a structure array, `found`, containing all the components connected to the current connection point, `conn`.

Lines 23–29: Go through the `found` array, removing any component already on the connected list and appending the names of those not removed to the connected list.

Lines 30–33: We will exit the `while` loop when there are no new components found; until then, choose the next component to connect to.

The support functions for this script are assembled for convenience into Listing 7.7.

Listing 7.7 Support functions

```

1. function ans = beam( nm, xp, yp, conn )
   % construct a beam structure with fields:
   % name - beam name
   % xp, yp - coordinates of its centroid
   % conn - cell array - names of adjacent beams
   % usage: ans = beam( nm, xp, yp, conn )
2. ans.name = nm;
3. ans.pos = [xp, yp];
4. ans.connect = conn;

```

continued on next page

```

5. function res = touches(beam, conn)
% does the beam touch this connecting point?
% usage: res = touches(beam, conn)
6. res = false;
7. for in = 1:length(beam.connect)
8.     item = beam.connect{in};
9.     if strcmp(item,conn)
10.         res = true; break;
11.     end
12. end

13. function res = ison( nm, cl )
% is this beam on the connection list,
%     a cell array of beam names
% usage: res = ison( beam, cl )
14. res = false;
15. for in = 1:length(cl)
16.     item = cl{in};
17.     if strcmp(item, nm)
18.         res = true; break;
19.     end
20. end

21. function nm = nextconn( fnd, cl )
% find a connection name among
%     those found not already connected
% usage: nm = nextconn( fnd, cl )
22. for in = 1:length(fnd)
23.     item = fnd(in);
24.     cn = item.connect;
25.     for jn = 1:length(cn)
26.         nm = cn{jn};
27.         if ~ison(nm, cl)
28.             break;
29.         end
30.     end
31. end

```

In Listing 7.7:

Lines 1–4: Constructor for one structure defining one component.

Line 5: Function to determine whether a component is connected to the component with the name provided.

Line 6: Presume not connected.

Lines 7–8: Extract each `item` in turn from the component's connections

Lines 9–10: If this item matches the connection, return `true`.

Line 13: A similar function to determine whether a particular string is on the connection list, a cell array of strings.

Line 14: Presumes `false`.

Lines 15 and 16: Extract each item in turn from the cell array.

Lines 17 and 18: Check the item against the name provided, and return `true` if it matched.

Line 21: Functions to find the next connection to use based on the latest components found—the “outer edges” of the emerging structure—and its not being already on the connected list.

Lines 22–24: Extract one component at a time from the found list, and extract its connection list.

Lines 25 and 26: Extract each name in turn from that connection list.

Lines 27–29: If this component is not on the connected list, exit and return it. (Notice that the return variable `nm` has been set.)

Here is the resulting output:

```
data =
```

```
1x11 struct array with fields:
```

```
    name
    pos
  connect
```

```
the order of assembly is:
```

```
'A' 'A-2' 'A-1' 'D-1' 'A-3' 'B-2' 'B-1' 'D-2' 'B-3' 'C-2' 'C-1' 'C-3'
```



Chapter Summary

This chapter covered the nature, implementation, and behavior of two heterogeneous collections:

- Cell arrays are vectors of containers; their elements can be manipulated either as vectors of containers, or individually by inserting or extracting the contents of the container using braces in place of parentheses.
- The elements of a structure are accessed by name rather than by indexing, using the dot operator, '.', to specify the field name to be used.
- Structures can be collected into structure arrays whose elements are structures all with the same field names. These elements can then be indexed and manipulated in the same manner as the cells in a cell array.



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>{ }</code>	Defines a cell array	7.2
<code>.</code>	Used to access fields of a structure	7.3.1
<code>(variable)</code>	Used to allow a variable to be used as a structure field	7.3.1
<code>class(obj)</code>	Determines the data type of an object	7.2.4
<code>deal(...)</code>	Distributes cell array results among variables	7.2.2
<code>getfield (str,<fld>)</code>	Extracts the value of the field <fld>	7.4.3
<code>isa(obj, str)</code>	Tests for a given data type	7.2.4
<code>ischar(ch).</code>	Determines whether the given object is of type <code>char</code>	7.2.4
<code>iscell(...).</code>	Determines whether the given object is a cell	7.2.4
<code>isfield (str, <fld>)</code>	<code>true</code> if the string is a field in this structure	7.4.3
<code>islogical(...)</code>	Determines whether the given object is of type <code>logical</code>	7.2.4
<code>isnumeric(...).</code>	Determines whether the given object is of type <code>double</code>	7.2.4
<code>isstruct(...).</code>	Determines whether the given object is a structure	7.2.4
<code>str = setfield (str,<fld>, <value>)</code>	Constructs a structure in which the value of the field <fld> has been changed	7.4.3
<code>struct(...)</code>	Constructs a structure from <fieldname> <value> pairs of parameters	7.3.2



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. Of all the collective operations defined for numerical arrays, only logical operations can be applied to a whole cell array.
2. A cell array or a structure can contain any legal MATLAB object.
3. You gain access to the contents of a cell by using braces, `{...}`.

4. Since the contents of a structure are heterogeneous, we can store other structures in any structure.
5. The statement `rmfield(str, 'price')` removes the field 'price' and its value from the structure `str`.
6. The statement `getfield(str, <fldname>)` returns the value of the specified field.
7. You cannot extract and process all of the values of a field in a structure array.

Fill in the Blanks

1. To perform any operations on the contents of a heterogeneous collection, the items must be _____ and _____ if necessary.
2. Cell arrays can be treated for the purpose of concatenation and slicing as _____ of _____.
3. The assignment `B{3} = {42}` results in the third entry in the cell array `B` to be a _____.
4. If a variable called `field` contains the value of a field of a structure `str`, the expression _____ will set the value of that field to 42.
5. MATLAB has a built-in function _____ that consumes pairs of entries, each consisting of a _____ and a _____, and produces a structure array.



Programming Projects

1. Write a function named `cellParse` that takes in a cell array with each element being either a string (character array), or a vector (containing numbers), or a boolean value (logical array of length 1). Your function should return the following:
 - `nStr` : the number of strings
 - `nVec`: the number of vectors
 - `nBool`: the number of boolean values

Hint

Type `'help cell/sort.m'` at the command line to get more information about sorting cell arrays of strings.

- `cString`: a cell array of all the strings in alphabetical order
- `vecLength`: the average length of all the vectors
- `allTrue`: true if all the boolean values are true and false otherwise

For example,

```
[a b c d e f] =
cellParse( { [1 2 3], true, 'hi there!', ...
             42, false, 'abc' } )
should return a = 2, b = 2, c = 2, d = {'abc', 'hi there!'},
             e = 2, and f = false.
```

2. It turns out that since you've become experts on rating clothing, Acme Clothing Company has hired you to rate their clothes. Clothes are now represented as structures instead of vectors with the fields (all of which are numbers between 0 and 5):

Condition, Color, Price, Matches, and Comfort

Acme has a much simpler way of rating their clothes than you used before:

$$\text{Rating} = 5 * \text{Condition} + 3 * \text{Color} + 2 * \text{Price} + \text{Matches} + 9 * \text{Comfort}$$

You have a script called `makeClothes.m` that will create a structure array called `acmeClothes` that contains clothes structures. You are to write a script called `rateClothes` that will add a `Rating` field and a `Quality` field to each of the structures in the `acmeClothes` array. The `Rating` field in each structure should contain the rating of that particular article of clothing. The `Quality` field is a string that is 'premium' if the `Rating` is over 80, 'good' over 60, 'poor' over 20, and 'liquidated' for anything else.

Notes:

1. You **MUST** use iteration to solve this problem.
2. To make things easy, just place the line `makeClothes` at the top of your script so you're guaranteed to have the correct `acmeClothes` array to work with.
3. The fields are case sensitive, so make sure you capitalize them.
3. You have been hired by a used-car dealership to modify the price of cars that are up for sale. You will get the information about a car, and then change its price tag depending on a number of factors.

Write a function called `usedCar` that takes in a structure with the following fields:

Make: A string that represents the make of the car. (e.g.: 'Toyota Corolla')

year: A number that corresponds to the year of the car. (e.g.: 1997)

cost : A number that holds the marked price of the car (e.g.: 7000)

Miles : The number of miles clocked.(e.g.: 85,000)

Accidents: The number of accidents the car has been in.(e.g.: 1)

Your function should return a structure with all the above fields, with **exactly** the same names. It should have the same make, year, accidents and miles. Here are the changes you must make:

1. Add 5,000 to the cost if the car has clocked less than 20,000 miles.
2. Subtract 5,000 if it has clocked more than 100,000 miles.
3. Reduce the price by 10,000 for every accident.
4. This problem deals with structures that represent dates.
 - a. First, write a MATLAB function called `createDate` that will take in three numeric parameters. The first parameter represents the month, the second represents the day, and the third the year. The function should return a structure with at least the following fields:
 - Day:** a number
 - Month:** a 3 character string containing the first three characters of the month name
 - Year:** a number containing the year.

For example,

`it = createDate(3,30,2008)` should return a structure containing at least:

```
Day: 30
Month: Mar
Year: 2008
```

- b. Write a function called `printDate` that displays a date in the form:


```
Mar 30, 2007
```

Hints

- It might help to add a field to the data class.
- The third date does not have to be chronologically later than the first date.

- c. Write a function `inBetween` that will take in three date structures. The function should return `true` if the second date is between the first and third dates, otherwise the function should return `false`.
- d. Write a function called `issorted` that takes in a single parameter, an array of date structures. This function should return `true` if all the dates in the array are in a chronological order

(regardless of whether they are in ascending or descending order), otherwise the function should return `false`.

- e. Write a test script that creates an array of date structures, prints out each date, and then states whether or not the dates are in order.
5. Your university has added a new award for students that were "almost there" last semester and just missed getting into the Dean's List. Write a function called `almost` that iterates through an array of student structures that it takes in, and returns an array of names of those that have a semester GPA between 2.9 and 2.99 (inclusive). The student structure has the following fields:

```
Name - string (eg.: 'George P. Burdell')
Semester_GPA - decimal number (eg.: 2.97)
Cumulative_GPA - decimal number (eg.: 3.01)
```

6. MATLAB has a built-in ability to perform mathematical operations on complex numbers. However, there are times when it is useful to treat complex numbers as a structure. Write a set of functions with the following capability and a script to verify that they work correctly:

```
cmplx = makeComplex(real, imag)
res = cmplxAdd( cmpxa, cmpxb )
res = cmplxMult( cmpxa, cmpxb )
```

7. In terms of atomic physics, every electron has four numbers associated with it, called the quantum numbers. These are 'principal' (energy), 'azimuthal' (angular momentum), 'magnetic' (orientation of angular momentum) and 'spin' (particle spin) quantum numbers. Wolfgang Pauli hypothesized (correctly) that no two electrons in an atom can have the same set of four quantum numbers, i.e., if the Principal, Azimuthal, and Magnetic numbers are the same for two electrons, then it is necessary for the electrons to have different spin numbers.

You need to write a function called `spinSwitch` that takes in two structures and returns both structures. Each structure represents an electron in a hydrogen atom and has the following fields:

```
principal (this is always > 0)
azimuthal (a number)
magnetic (A number)
spin (a string with value 'up' or 'down')
```

Your function will compare the values in the two structures and check if they all have the same values for the four fields. If true, you

are required to switch the spin of the second structure. You also have to add a field called 'energy' to both structures. The value stored in this field must be $(-2.18 \cdot 10^{18}) / (n^2)$, where n is the value of the principal quantum number for that electron. You have to return both the structures with the energy field added to both, so that the one with the higher energy is first. If the energies are equal, return the one with the 'up' spin first. If both have the same spin and the same energy, the order does not matter.

File Input and Output

Chapter Objectives

This chapter discusses three levels of capability for reading and writing files in MATLAB, each including a discussion of the circumstances under which they are appropriate:

- Saving and restoring the workspace
- High-level functions for accessing files in specific formats
- Low-level file access programs for general-purpose file processing

Reading and writing data in data files is fundamental to the utility of programming languages in general, and MATLAB in particular. In addition to the obvious need to save and restore scripts and functions (covered in Chapter 2), here we consider three types of activities that read and write data files.

- MATLAB has the basic ability to save your workspace (or parts of your workspace) to a file and restore it later for further processing.
- There are high-level functions in MATLAB that consume the name of a file in any one of a number of popular formats and produce an internal representation of the data from that file in a form ready for processing. Almost all these functions have an equivalent write function that will write a new file in the same format after you have manipulated the data.
- However, we also need to deal with lower-level capabilities for manipulating text files that do not have recognizable structures.

Introduction

This chapter discusses files that contain workspace variables, spreadsheet data, and text files containing delimited numbers and plain text. Subsequent chapters will discuss image files and sound files. For information on the other file formats, consult the help documentation for details of their usage.

MATLAB also has the ability to access binary files—files whose data is not in text form—but the interpretation of binary data is beyond the scope of this text, and we will not consider binary files here. Refer to MATLAB help for information about binary files.

- 8.1** Concept: Serial Input and Output (I/O)
- 8.2** MATLAB Workspace I/O
- 8.3** High-level I/O Functions
 - 8.3.1 Exploration
 - 8.3.2 Excel Spreadsheets
 - 8.3.3 Delimited Text Files—Numerical Data Only
- 8.4** Low-level File I/O
 - 8.4.1 Opening and Closing Files
 - 8.4.2 Reading Text Files
 - 8.4.3 Examples of Reading Text Files
 - 8.4.4 Writing Text Files
- 8.5** Engineering Example—Spreadsheet Data

8.1 Concept: Serial Input and Output (I/O)

We frequently refer to the process of reading and writing data files as Input/Output (I/O). We have already seen and used examples of file I/O to store and retrieve data and programs. Your script and function files were stored in your current directory, and could be invoked from there by name from the Command window. In general, any computer file system saves and retrieves data as a sequential (serial) stream of characters, as shown in Figure 8.1. Mixed in with the characters that represent the data are control characters (“delimiters”) that specify the organization of the data. When a program opens a file by name for reading, it continually requests values from the file data stream until the end of the file is reached. As the data is received, the program must identify the delimiting characters and reformat the data to reconstruct the organization of the data as represented in the file.

Similarly, when writing data to a file, the program must serialize the data, as shown in Figure 8.2. To preserve the organization of the data, the appropriate delimiting characters must be inserted into the serial character stream. The purpose of the file I/O functions discussed in this chapter is to encapsulate these fundamental operations into a single MATLAB function, or at least into a manageable collection of functions.

8.2 MATLAB Workspace (I/O)

MATLAB provides the tools to save your complete workspace to a file with the `save` command and reload it with the `load` command. If you provide a file name with the `save` command, a file of that name will be written into your current directory in such a form that a subsequent `load` command with that file name will restore the saved workspace. If you do not provide a file name, MATLAB saves the workspace as `matlab.mat`.



Figure 8.1 An input stream



Figure 8.2 An output stream

You can also identify specific variables that you want to save—either by listing them explicitly or by providing logical expressions to indicate the variable names. For example:

```
> > save mydata.mat a b c*
```

would save the variables `a` and `b` and any variable beginning with the letter `c`. For more details, consult the MATLAB help documentation.

In a practical sense, however, this is very rarely an appropriate approach to saving work because it saves the results but not the code that generated the results. It is almost always better to save the scripts and raw data that created the workspace. For example, this is a good idea when you have a lengthy computation (perhaps one run overnight) to prepare data for a display. You could split that script into two halves. The first half would do the overnight calculation and save the workspace. The second part can then read the workspace quickly, and you can develop sophisticated ways to display the data without having to re-run the lengthy calculations.



8.3 High-Level I/O Functions

We turn to the general case of file I/O in which we expect to load data from external sources, process that data, and perhaps save data back to the file system with enhancements created by MATLAB.

When you try to process data from some unknown source, it is difficult to write MATLAB code without some initial exploration of the nature and organization of the data. So a good habit is to explore the data in a file by whatever means you have available and then commit to processing the data according to your observations.

Most programming languages require the programmer to write detailed programs to read and write files, especially those produced by other application programs or data acquisition packages. Fortunately for MATLAB programmers, much of this messy work has been built into special file readers and writers. Table 8.1 identifies the type of data, the name of the appropriate reader and writer, and the internal form in which MATLAB returns the data.

8.3.1 Exploration

The types of data of immediate interest are text files and spreadsheets. In Table 8.1 notice that the delimited text files are presumed to contain numerical values, whereas the spreadsheet data may be either numerical data stored as doubles or string data stored in cell arrays. Typically, text files are delimited by a special character (comma, tab, or anything else) to designate the column divider, and a new-line character to designate the rows. Once the data is

Table 8.1 File I/O functions

File Content	File Extension	Reader	Writer	Data Format
Plain text	Any	textscan	fprintf	Specified in the function calls
Comma-separated numbers	CSV	csvread	csvwrite	Double array
Tab-separated text	TAB	dlmread	dlmwrite	Double array
General delimited text	DLM	dlmread	dlmwrite	Double array
Excel worksheet	XLS	xlsread	xlswrite	Double or cell array
Lotus 1-2-3 worksheet	WKL	wklread	wklwrite	Double or cell array
Scientific data in Common Data Format	CDF	cdfread	cdfwrite	Cell array of CDF records
Flexible Image Transport System data	FITS	fitsread		Primary or extension table data
Data in Hierarchical Data Format	HDF	hdfread		HDF or HDF-EOS data set
Extended Markup Language (XML)	XML	xmlread	xmlwrite	Document Object Model node
Image data	Various	imread	imwrite	True color, grayscale, or indexed image
Audio file	AU or WAV	auread or wavread	auwrite or wavwrite	Sound data and sample rate
Movie	AVI	aviread		MATLAB movie

imported, all of MATLAB's normal array and matrix processing tools can be applied. The exception to this rule is the plain text reader that must be provided with a format specifier to define the data, and the names of the variables in which the data is to be stored.

So when you are approached with a file, the file extension (the part of the file name after the dot) gives you a significant clue to the nature of the data. For example, if it is the output from a spreadsheet, you should open the data in that spreadsheet program to explore its contents and organization. Typically, spreadsheet data will not open well in a plain text editor. If you do not recognize the file extension as coming from a spreadsheet, try opening the file in a plain text editor like Notepad, and see if the data is legible. You should be able to discern the field delimiters or the format of each line if the file contains plain text.

8.3.2 Excel Spreadsheets

Excel is a Microsoft product that implements spreadsheets. Spreadsheets are rectangular arrays containing labeled rows and columns of cells. The data in the cells may be numbers, strings, or formulae that combine the data values in other cells. Because of this computational capability, spreadsheets can be used to solve many problems, and most offer flexible plotting packages for presenting the results in colorful charts. There are occasions, however, when we need to apply the power of MATLAB to the data in a spreadsheet.

The MATLAB reader for Excel spreadsheets gives you a significant amount of flexibility in retrieving the data from the spreadsheet. Consider the typical set of data in an Excel spreadsheet named `grades.xls` shown in Table 8.2. The MATLAB `xlsread(...)` function does a good job of separating out the text and numerical portions of the spreadsheet. The parameter consumed by `xlsread(...)` is the name of the file; you can ask for up to three return variables: the first will hold all the numerical values in an array of doubles, the second will hold all the text data in cell arrays, and the third, if you request it, will hold both string and numerical data in cell arrays (try Exercise 8.1).

Table 8.2 Sample Excel spreadsheet

name	age	grade
fred	19	78
Joe	22	83
Sally	98	99
Charlie	21	56
Mary	23	89
Ann	19	51



Exercise 8.1 Reading Excel data

Do It Yourself

Enter the following in the Command window and observe the results:

```
>> [nums txt raw] = xlsread('grades.xls')
nums =
    19    78
    22    83
    98    99
    21    56
    23    89
    19    51
```

continued on next page


```

txt =
    'name'      'age'      'grade'
    'fred'      ''         ''
    'joe'       ''         ''
    'sally'     ''         ''
    'charlie'   ''         ''
    'mary'     ''         ''
    'ann'      ''         ''

raw =
    'name'      'age'      'grade'
    'fred'      [ 19]      [ 78]
    'joe'       [ 22]      [ 83]
    'sally'     [ 98]      [ 99]
    'charlie'   [ 21]      [ 56]
    'mary'     [ 23]      [ 89]
    'ann'      [ 19]      [ 51]
    'ann'      [ 19]      [ 51]

```

The reader first determines the smallest rectangle on the spreadsheet containing all of the numerical data; we will refer to this as the number rectangle. Then it produces the following results:

1. The first returned result is an array with the same number of rows and columns as the number rectangle and containing the values of all the numeric data in that rectangle. If there are non-numeric values within that rectangle, they are replaced by `NaN`, the built-in MATLAB name for something that is not a number.
2. The second returned result is a cell array with the same size as the original spreadsheet, containing only the string data; to ensure the consistency of this cell array, all numbers present are replaced by the empty string.
3. The third returned result is a cell array also with the same size as the original spreadsheet, containing both the strings and the numbers. Cells that are blank are presumed to be numerical, and are assigned as a cell containing `NaN`.

Frequently, after processing data with MATLAB, you are ready to write the results back to a spreadsheet. Excel spreadsheets can be written using:

```
xlswrite(<filename>, <array>, <sheet>, <range>)
```

where `<filename>` is the name of the file, `<array>` is the data source (a cell array), `<sheet>` is the sheet name, and `<range>` is the range of cells in Excel cell identity notation. The sheet name and range are optional.

8.3.3 Delimited Text Files—Numerical Data Only

If information is not available specifically in spreadsheet form, it can frequently be presented in text file form. If the data in a text file is numerical

values only, and is organized in a reasonable form, MATLAB can read the file directly into an array. It is necessary that the data values are separated (delimited) by commas, spaces, or tab characters, as shown in Listing 8.1. Rows in the data are separated as expected by the new-line character. These values might be saved in a file named `nums.txt`. This type of numerical data (not strings) in general delimited form can be read using `dlmread(file, delimiter)`, where the delimiter parameter is a single character that can be used to specify an unusual delimiting character. However, the function can usually determine common delimiter situations without specifying the parameter.

Common Pitfalls

It is best not to provide the delimiter unless you have to. Without it, MATLAB will assume that repeated delimiters—like tabs and spaces—are single delimiters. If you do specify a delimiter, MATLAB will assume that repeated delimiter characters are separating different, absent field values.

This function produces a numerical array containing the data values. Try reading delimited files in Exercise 8.2.

Notice that the array elements where data is not supplied are filled with zero.

Delimited data files can be written using:

```
dlmwrite( <filename>, <array>, )
```

where `<filename>` is the name of the file, `<array>` is the data source (a numerical array), and `<dlm>` is the delimiting character. If the delimiting character is not specified, it is presumed to be a comma.

Listing 8.1 Sample delimited text file

```
19, 78, 42
22, 83, 100
98, 99, 34
21, 56, 12
23, 89
19, 51
```



Exercise 8.2 Reading delimited files

Do It Yourself

Enter the following in the Command window and observe the results:

```
>> A = dlmread('nums.txt')
A =
    19    78    42
    22    83   100
    98    99    34
    21    56    12
    23    89     0
    19    51     0
```



8.4 Lower-Level File I/O

Some text files contain data in mixed format that are not readable by the high-level file reading functions. MATLAB provides a set of lower-level I/O functions that permit general purpose text file reading and writing. The following is a partial discussion of these functions that is sufficient for most text file processing. In general, the file must be opened to return a file handle to be used by subsequent functions. After the file contents have been manipulated, the file must be closed to complete the activity. Because these are lower-level functions used in combination to solve problems, we will need to present the behavior of several of them before we can show examples of their use.

8.4.1 Opening and Closing Files

To open a file for reading or writing, use:

```
fh = fopen( <filename>, <purpose> )
```

where `fh` is a file handle used in subsequent function calls to identify this particular I/O stream, `<filename>` is the name of the file, and `<purpose>` is a string specifying the purpose for opening the file.

The most common purposes are `'r'` to read the file, `'w'` to write it, or `'a'` to append to an existing file. See the MATLAB help files for more complex situations. If the purpose is `'r'`, the file must already exist; if `'w'` and the file already exists, it will be overwritten; if `'a'` and the file already exists, the new data will be appended to the end.

The consequence of failure to open the file is system dependent. In the standard version on a PC, this is indicated by returning a file handle of `-1`.

To close the file, use the following:

```
fclose( fh )
```

8.4.2 Reading Text Files

To read a file, three levels of support are provided: reading whole lines with or without the new line character, parsing into tokens with delimiters, or parsing into cell arrays using a format string.

- To read a whole line including the new line character, use:

```
str = fgets( fh );
```

which will return each line as a string until the end of the file, when the value `-1` is returned instead of a string. To leave out each new-line character, use `fgetl(...)` instead (the last character is a lowercase `L`).

- To parse each line into tokens (elementary text strings) separated by white space delimiters, use a combination of `fgetl(...)` and the `tokenizer` function:

```
[tk, rest] = strtok( ln );
```

where `tk` is string token, `rest` is the remainder of the line, and `ln` is a string to be parsed into tokens.

- To parse a line according to a specific format string into a cell array, use:

```
ca = textscan( fh, <format> );
```

where `ca` is the resulting cell array, `fh` is the file handle, and `<format>` is a format control string we used for `sscanf(...)` in Chapter 6.

8.4.3 Examples of Reading Text Files

To illustrate the use of these functions for reading a text file, the script shown in Listing 8.2 shows a script that will list any text file in the Command window.

In Listing 8.2:

Line 1: Fetches the name of a file.

Line 2: Opens the file and returns the file handle.

Line 3: Initializes the `while` loop control variable.

Line 4: When the file read reaches the end of the file, it returns `-1` instead of a string.

Line 5: Reads a string, including the end of line character.

Line 6: Classic loop-and-a-half logic that determines whether there is a line to process.

Listing 8.2 Script to list a text file

```
1. fn = input( 'file name: ', 's' );
2. fh = fopen( fn, 'r' );
3. ln = '';
4. while ischar( ln )
5.     ln = fgets( fh );
6.     if ischar( ln )
7.         fprintf( ln );
8.     end
9. end
10. fclose( fh );
```

If so, it proceeds to line 7.

Line 7: Displays that line.

Line 10: Closes the file when finished.

As an example of the use of a tokenizer, consider the code shown in Listing 8.3, which performs the same function as Listing 8.2 but uses tokens.

Line 5: Uses `fgetl(...)` instead of `fgets(...)` because the tokenizer does not need the new line character.

Line 7: Initializes the resulting cell array.

Line 8: The tokenizer will be finished when it leaves an empty line as the result.

Line 9: Creates a token from the remains of the line and puts the remains back into the variable `ln`.

Practice using file listers in Exercise 8.3.

Exercise 8.3 shows the difference in output results between the conventional listing script and the tokenizing lister. With the tokenizer, we see each individual token (really, each word in a normal text file) separately listed.

Listing 8.3 Listing a file using tokens

```

1. fn = input( 'file name: ' , 's' );
2. fh = fopen( fn, 'r' );
3. ln = '';
4. while ischar( ln )
5.     ln = fgetl( fh );
6.     if ischar( ln )
7.         ca = [];
8.         while ~isempty( ln )
9.             [tk, ln] = strtok( ln );
10.            ca = [ca {tk}];
11.        end
12.        disp( ca );
13.    end
14. end
15. fclose( fh );

```



Exercise 8.3 Using file listers

Do It Yourself

Enter the following in the Command window and observe the results:

```

>> listFile
file name: listFile.m
fn = input('file name: ','s');
fh = fopen(fn,'r');
ln = '';
while ischar(ln)

```

continued on next page

```

        ln = fgets(fh);
        if ischar(ln)
            fprintf(ln);
        end
    end
    fclose(fh);

>> tokenize
file name: listFile.m
'fn' '=' 'input('file' 'name:' ' ','s');'
'fh' '=' 'fopen(fn,'r');'
'ln' '=' ' ';
'while' 'ischar(ln) ' ' '=' 'fgets(fh);'
'if' 'ischar(ln)'
'fprintf(ln);'
'end'
'end'
'fclose(fh);'

```

Since the `textscan(...)` approach is data dependent, we will see practical examples of its use in Chapter 10.

8.4.4 Writing Text Files

Once a file has been opened for writing, the `fprintf(...)` function can be used to write to it by including its file handle as the first parameter. For example, Listing 8.4 is a minor alteration to Listing 8.2, copying a text file instead of listing it in the Command window.

In Listing 8-4:

Line 2 Fetches the output file name.

Line 4: Opens the output file.

Line 9: Adds the `oh` parameter to direct the output to the specified file.

Line 13: Closes the output file.

Listing 8.4 Script to copy a text file

```

1. ifn = input( 'input file name: ', 's' );
2. ofn = input('output file name: ', 's' );
3. ih = fopen( ifn, 'r' );
4. oh = fopen( ofn, 'w' );
5. ln = '';
6. while ischar( ln )
7.     ln = fgets( ih );
8.     if ischar( ln )
9.         fprintf( oh, ln );
10.    end
11. end
12. fclose( ih );
13. fclose( oh );

```

8.5 Engineering Example—Spreadsheet Data

Frequently, engineering data is provided in spreadsheets. Here we will adapt the structure assembly problem from Chapter 7. The script for that solution created the data using a constructor function. Consider the situation in which the data is provided in a spreadsheet such as that shown in Figure 8.3. We have to start by examining the layout of the data and the process necessary to extract what we need. Bearing in mind the three results returned from `xlsread(...)`, first we determine which of the three is most appropriate:

- The `{xlsread(...)}` function is going to include all the numerical cells from the spreadsheet in the numerical array. This is awkward because there are numbers in the first column, and since the primary interest in this problem is not the numerical data, we will not use the numerical array directly.
- However, this is not exclusively a text processing problem, since we need the numerical coordinates, so the second, text-only result is not what we need.
- Therefore, in this particular application, we will process the raw data provided by `xlsread(...)`, giving both the string and numerical data.

The other concern is that there are a different number of connections on each row of the sheet. When a connection is present, it is a string. When it is not there, we refer to the behavior of the raw data to discover that the contents of empty cells appear as `NaN` of type `double`.

We need a function that will read this file and produce the same model of the structure used in Chapter 7. Such a function is shown in Listing 8.5.

	A	B	C	D	E	F	G	H	I	J
1	Item	Name	X	Y	Connected to					
2	1	A-1	0.866	0.5	A	A-2	A-3	D-1		
3	2	A-2	0	1	A	A-3	B-1	B-2		
4	3	A-3	0.866	1.5	A-1	A-2	B-1	D-1		
5	4	B-1	0.866	2.5	A-2	A-3	B-2	B-3	D-1	D-2
6	5	B-2	0	3	A-2	A-3	B-1	B-3	C-1	C-2
7	6	B-3	0.866	3.5	B-1	B-2	C-1	C-2	D-1	D-2
8	7	C-1	0.866	4.5	B-2	B-3	C-2	C-3	D-2	
9	8	C-2	0	5	B-2	B-3	C-1	C-3	C	
10	9	C-3	0.866	5.5	C-1	C-2	C-3	C		
11	10	D-1	1.732	2	A-1	A-3	B-1	B-3	D-2	
12	11	D-2	1.732	4	B-1	B-3	C-1	C-3	D-1	

Figure 8.3 Data in a spreadsheet

Listing 8.5 Reading structure data

```

1. function data = readStruct(filename)
% read a spreadsheet and produce a
%       structure array:
%       name - the second column value
%       pos - columns 3 and 4 in a vector
%       connect - cell array with the remaining
%               data on the row

2. [no no raw] = xlsread(filename);
3. [rows cols] = size(raw);
% ignore the first row and column
4. out = 1;
5. for row = 2:rows
6.     str.name = raw{row,2};
7.     str.pos = [raw{row,3} raw{row,4}];
8.     cni = 1;
9.     conn = {};
10.    for col = 5:cols
11.        item = raw{row, col};
12.        if ~ischar(item)
13.            break;
14.        end
15.        conn{cni} = item;
16.        cni = cni + 1;
17.    end
18.    str.connect = conn;
19.    data(out) = str;
20.    out = out + 1;
21. end

```

In Listing 8.5:

Line 1: The function consumes the file name and produces a structure array with the fields described in the following comments.

Line 2: Reads the spreadsheet and keeps only the raw data.

Line 3: Gets the rows and columns in the raw data; we need to ignore the top row and left column.

Line 4: Initializes the output index for the structure array.

Line 5: Ignoring the first row, traverses all the remaining rows.

Line 6: The component name is in the second column.

Line 7: The coordinates of the component are in the third and fourth columns.

Lines 8–9: Initializes the search for the connections for this component. It is important to empty the array `conn` before each pass to avoid “inheriting” data from a previous row.

Lines 10–11: Extracts each item in turn from the row.

Lines 12–14: If the item is not of class `char`, this is the blank cell at the end of the row; the `break` command exits the `for` loop moving across the row.

Lines 15–16: Otherwise, it stores the connection and keeps going.

Lines 18–20: When the connections are complete, it stores them in the structure, stores the structure in the structure array, and continues to the next row.

Line 21: When the rows are completed, the data is ready to return to the calling script.

To test this function, replace the structure array construction in lines 1–11 of Listing 7.7 in Chapter 7 with the following line:

```
data = readStruct('Structure_data.xls');
```

The script will then produce the same results as before.



Chapter Summary

We have described three levels of capability for reading and writing files in MATLAB:

- The save and load operators allow you to save variables from the workspace and restore them to the workspace
- Specialized functions read and write spreadsheets and delimited text files
- Lower-level functions provide the ability to open and close files, and to read and write text files in any form that is required



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>NaN</code>	Not a number	8.3.2
<code>load</code>	Loads the workspace from a file	8.2
<code>save</code>	Saves variables in a file	8.2
<code>csvread(file)</code>	Reads comma-separated text files	8.3
<code>csvwrite (file, data)</code>	Writes comma-separated text files	8.3
<code>dlmread (file, dlm)</code>	Reads text files separated by the given delimiting character(s)	8.3
<code>dlmwrite (file, data, dlm)</code>	Writes text files separated by the given delimiting character(s)	8.3

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>[nums, txt, raw] = xlsread(file)</code>	Reads an Excel spreadsheet	8.3.2
<code>xlswrite (file, data, sheet, range)</code>	Writes an Excel spreadsheet in a specific row/column range	8.3.2
<code>fh = fopen (file, fl)</code>	Opens a text file for reading or writing	8.4.1
<code>fclose(fh)</code>	Closes a text file	8.4.1
<code>fgetl(fh)</code>	Reads a line, omitting the new-line character	8.4.2
<code>fgets(fh)</code>	Reads a line, including the new-line character	8.4.2
<code>[tk rest] = strtok(str, dlm)</code>	Extracts a token from a string and returns the remainder of the string	8.4.2
<code>ca = textscan (fh, format)</code>	Acquires and scans a line of text according to a specific format	8.3, 8.4.2
<code>fprintf(...)</code>	Writes to the console, or to plain text files	8.3, 8.4.4



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. All data files should be treated as a sequential series of characters.
2. When you save a workspace, you are actually saving the scripts that generate the data in the workspace.
3. MATLAB reads strings from tab- or comma-delimited files by recognizing the double quotes that delimit strings.
4. If you use `fopen(...)` to open an existing file and write to it, the original data in the file will be overwritten.
5. The function `fgets(fh)` does not always return a string.

Fill in the Blanks

1. In general, data files contain text that represents the _____ of the data and control characters that specify the _____ of the data.

2. The MATLAB `xlsread(...)` function returns three results: the _____ in a _____, the _____ in a _____, and _____ in a _____.
3. When using `dlmread(...)` to populate a _____, any unassigned values are _____.
4. When using `fopen(...)`, the consequence of failure to open the file is _____.



Programming Projects

1. Write a script that performs the following operations:
 - a. Set the value of variables `a`, `b`, `c1`, `c2`, `c3` and `x`. The values don't matter, except you should set `c2` to 42.
 - b. Save the values of all the variable except `x` to `mydata.mat` using the MATLAB save operation.
 - c. Set the value of `c2` to -99
 - d. Load `myData.mat` and check that `c2` is now 42.
 - e. Clear all variables.
 - f. Load `myData.mat` again and note that the variable `x` is not present.
2. One requirement for all freshmen classes is an issue of a 'standing' during the middle of the term. The results are either a Satisfactory (s) or Unsatisfactory (u). Since you are the office employee in charge of issuing these grades you decide to write a MATLAB function called `midtermGrades` to help you. You discover that the grades are on an Excel spreadsheet organized like this:

Each student is represented by one row on the spreadsheet. Unfortunately, since these sheets are created by different instructors, they are not necessarily consistent in their layout. The first row will contain the following 6 strings in any order: 'name', 'math', 'science', 'english', 'history' and 'cs'. Under the name column will be a string with the student's name. Grades in the other columns can be 'A', 'B', 'C', 'D', 'F', or 'W'. A student's grade is 's' if there are more A's, B's and C's than not.

Your function should print out grades ready to be entered consisting of a table:

```
Name    S/U
...
...
```

3. Write a function called `genStats` that will compute statistics for a set of class grades. The grades will be stored in an Excel file and your function will compute statistics and then write the grades along with the statistics to an Excel file.

You may assume that the Excel file will have a format similar too:

Student Name	Exam1	Exam2	Exam3	...
student 1	100	76	45	...
student 2	34	83	89	...

The first row is the header row and the first column is the list of students. There may be any number of exam grades and there may be any number of students. Although, you may assume that there will be at least one student and that there will be at least one exam. Also, every student will have a grade for every exam.

Your function should only have one input (a string containing the file name of the grades Excel file) and no outputs. You must write your function to perform the following steps:

- a. Calculate the average grade of each student (across the rows) and store it in a new column called 'Average' (to the right of the last exam grade) .
- b. Calculate the deviation of each student's overall average (calculated in step a) from the maximum student average and store it in a new column called 'Deviation' (to the right of the 'Average' column). Note that deviation is just the difference between the maximum student average and a student's overall average.
- c. Calculate the average of each column's data (each exam, the averages calculated in step a, and the deviations calculated in step b, then store these averages below the last row of the original data and name that row 'Total Average'.
- d. Write the original data along with all of the new data to a file named 'Stats_<name_of_original_file>.xls' (so if the inputted file name was 'Student_Grades.xls', the new data would be written to the file named 'Stats_Student_Grades.xls').
- e. Construct an Excel file with suitable test data and use it to test your function.

4. Write a function called `replaceStr`. Your function should take in the following (in this order):

`filename`: A string that corresponds to the name of a file

`wordA`: A string that is a word (contains no spaces)

`wordB`: Another string that is also a word (contains no spaces)

Your function should do the following:

- a. Read the file a line at a time.
- b. On each line, replace every occurrence of `wordA` with `wordB`.
- c. Write the modified text file with the same name as the original file, but prepended with `'new_'`. For instance, if the input filename was `'data.txt'`, the output filename would be `'new_data.txt'`.
- d. Prepare a test file by downloading a text file from the Internet. For example, the complete works of Shakespeare are accessible at <http://www.william-shakespeare.info>
- e. Examine the file for repeated words and test your function by writing a script that replaces frequently repeated words.

Recursion

Chapter Objectives

This chapter discusses the following basic ideas of recursive programming:

- Three basic characteristics must be present for a recursive function to work
- Exceptions are a powerful mechanism for detecting and trapping errors
- A wrapper function is used to set up the recursion
- Other forms of recursion occur in special circumstances

Introduction

Recursion is an alternative technique by which a code block can be repeated in a controlled manner. In Chapter 4, we saw repetition achieved by inserting control statements in the code (either `for` or `while`) to determine how many times a code block would be repeated. Recursion uses the basic mechanism for invoking functions to manage the repetition of a block of code.

While some problems are naturally solved by iterative solutions, there are many problems for which a recursive solution is elegant and easily understood.

Frequently, a recursive function needs a “wrapper function” to set up the recursion correctly, and to check for erroneous data conditions that might cause errors. The actual recursive function then becomes a private helper function.

- 9.1** Concept: The Activation Stack
 - 9.1.1 A Stack
 - 9.1.2 Activation Stack
 - 9.1.3 Function Instances
- 9.2** Recursion Defined
- 9.3** Implementing a Recursive Function in MATLAB
- 9.4** Exceptions
 - 9.4.1 Historical Approaches
 - 9.4.2 Generic Exception Implementation
 - 9.4.3 MATLAB Implementation
- 9.5** Wrapper Functions
- 9.6** Tail Recursion
- 9.7** Mutual Recursion
- 9.8** Generative Recursion
- 9.9** Examples of Recursion
 - 9.9.1 Detecting Palindromes
 - 9.9.2 Fibonacci Series
 - 9.9.3 Zeros of a Function
- 9.10** Engineering Example—Robot Arm Motion

9.1 Concept: The Activation Stack

In order to understand recursive programming, we must look deeper into the mechanism by which function calls are mechanized. Calling any function depends on a special kind of stack built into the architecture of the Central Processing Unit (CPU). This is called the **activation stack**. It enables the CPU to determine which functions are active or suspended awaiting the completion of other function calls. To understand the activation stack, first we consider the basic concept of a stack.

9.1.1 A Stack

A stack is one of the fundamental data structures of computer science. It is best modeled by considering the trays at the front of the cafeteria line. You cannot see how many trays there are on the stack, and the only access you have to them is to take a tray off the stack or put one on. So a stack is a collection of objects of arbitrary size with a restricted number of operations we are allowed to perform on that collection (see Figure 9.1). Unlike a vector, where it is permissible to read, add, or remove items anywhere in the collection, we are only allowed the following operations with a stack:

- Push an object onto the stack
- Pop an object off the stack
- Peek at the top object without removing it
- Check whether the stack is empty

9.1.2 Activation Stack

The core concept that enables any function (especially a recursive function) to operate is the concept of an activation stack. The activation stack is the means by which the operating system allocates memory to functions for local storage. Typically, local storage is required by a function for the following reasons:

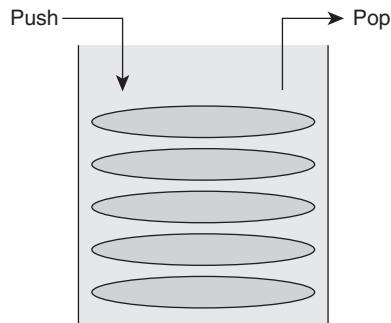


Figure 9.1 Behavior of a stack

- Storing the location of the function to be evaluated
- Storing the location in memory to return to when the function execution completes
- Storing copies of the function parameter values
- Providing space for the values of any local variables defined within the function

When the user starts a specific application program (in MATLAB terms, a script or an entry in the Command window), the operating system allocates enough memory to load the application and a block of memory to contain its activation stack. When that application calls a function, the information specific to that function call is assembled into an object called a **stack frame** and pushed onto the activation stack. The calling program is then suspended and control is passed to the function specified in the frame on the top of the stack. When that function completes, its frame is popped off the stack and destroyed, and control is returned to the frame beneath, which is now the top of the stack. If an active function calls another function, this process is repeated. The calling function is suspended, a stack frame is pushed onto the activation stack for the new function, and the original function is suspended until the new function completes.

Note that in a MATLAB implementation, the stack frame is the storage environment for the current workspace. Each new function call receives a fresh workspace from the activation stack that initially contains the copies of the actual parameters bearing the formal parameter names.

9.1.3 Function Instances

In Chapter 2 we discussed the difference between the type of data defined by its class, and an object—an instance of that class assigned to a variable. In the same way, we draw the distinction between the `.m` file that defines the behavior of a function and the instance(s) of that function that result when the function is called. Each new instance of a function has its own workspace that occupies a temporary stack frame allocated from the activation stack.



9.2 Recursion Defined

Following the previous logic, in principle there is no reason why a function could not in fact “call itself,” and this is the logical basis for recursive programming. Of course, as with iterative programming, if there is no mechanism to stop the recursion, the process would repeat endlessly. In the case of endless recursion, since space is being consumed on the activation stack, the operating system will eventually terminate the process when the memory allocated for the activation stack is exhausted.

The canonical illustration of recursion is the computation of n factorial. We could view the calculation of $5!$ in the following ways:

```
5! = 5 * 4 * 3 * 2 * 1
5! = 5 * 4!
```

The second representation is the recursive view, which warrants a closer view as follows:

```
n! = n * (n-1)!
```

This definition would not be complete, however, without realizing that it must stop somewhere. This is easy with the factorial function because $0!$ is defined mathematically as 1.

In this example we see the three necessary characteristics of a recursive function:

1. There must be a terminating condition to stop the process
2. The function must call a clone of itself
3. The parameters to that call must move the function toward the terminating condition

The word *clone* is important here—a recursive function really does not “call itself,” because it requests a new stack frame and passes different parameters to the instance of the function that occupies the new frame.

9.3 Implementing a Recursive Function in MATLAB

Template 9.1 shows the general template for recursive functions in MATLAB.

The following general guidelines indicate how the recursive template is implemented:

- The `<function_name>`, like the name of any other function, may be any legal variable name

Template 9.1 General template for a recursive function

```
function <result> = <function_name> (<formal_params>)
<documentation>
    if <terminating condition 1>
        <result> = <initial value 1>
    elseif <terminating condition 2>
        <result> = <initial value 2>
        . . .
    else
        <result> = <operation> ...
            (<formal_params>, ...
             <function_name> (<new_params>))
    end
```

- The variable `<result>` may be any legal variable name or a vector of variable names
- As usual with functions, you should supply at least one line of `<documentation>` to define its purpose and implementation
- Each exit from the function must assign values to all the result variables
- The first design decision is to determine the condition(s) under which the recursive process should stop, and how to express this as the `<terminating condition N>` tests
- The `<initial value N>` entries are the value(s) of the result(s) at the terminating condition(s)
- The second design decision is to determine the `<operation>`—the specific mathematical or logical operation that must be performed to combine the current formal parameters with the result of the recursive call to create a new value of the `<result>`
- The last design decision is to determine how to compute the `<actual_params>` of the recursive call to ensure that the process moves towards at least one of the `<terminating condition N>` states

The MATLAB implementation of the factorial function is shown in Listing 9.1.

In Listing 9.1:

Line 3: Shows a diagnostic print call, which if not commented, enables you to observe the sequence of events.

Line 4: Shows the terminating condition.

Line 5: Shows the result at termination.

Line 7: Calls a clone of the function, which moves closer to termination by reducing `N` and computing the current result by multiplication.

Exercise 9.1 provides an analysis of recursive behavior. In particular, notice that all the mathematical operations are performed as the activation stack “unwinds.”

Listing 9.1 Function to compute `N` factorial

```

1. function result = fact(N)
2. % recursive computation of N!
3. %     fprintf('fact( %d )\n', N); % testing only
4.     if N == 0
5.         result = 1;
6.     else
7.         result = N * fact(N - 1);
8.     end

```



Exercise 9.1 Analyzing recursive behavior

Do It Yourself

1. Create the `fact(...)` function from Listing 9.1 in MATLAB, uncomment the print line, and run it from the Command window:

```
>> fact(4)
fact( 4 )
fact( 3 )
fact( 2 )
fact( 1 )
fact( 0 )
ans =
    24
```

2. Put a break point at Line 4, define some arbitrary variables in your main workspace and run `fact(2)`. The function should pause in the first stack frame. The only variable in the workspace is `N` with a value 2.
3. Find the “step into” button and click it. Since `N` is not 0, the arrow should move to Line 7.
4. Click again, and the workspace should change to a new workspace with the value `N = 1`—you just called a clone of the original function with its own stack frame. There should be a second, transparent arrow at Line 7 to indicate that some clone of this function is waiting at that point for a result.
5. Continue stepping into functions until you return from the copy where `N = 0`. When this return happens you return to the frame with `N = 1`, the frame “underneath,” at Line 7, and are then able to compute the first result.
6. Further stepping will return from each stack frame until you finally return to your script’s workspace.



9.4 Exceptions

We digress here to discuss how programs deal with unexpected circumstances. Exceptions are a powerful tool for gracefully managing runtime errors caused by programming errors or bad data. The general need for an exception mechanism might best be established by way of an example. Suppose you write a program that requests some data from a user, and then launches a significant number of nested function calls—perhaps even a recursive function—to perform analysis on the data received. Somewhere in the depths of these function calls, the program divides something by a value, but in this instance that value is zero. The cause of this problem is probably bad data entered by the user. However, the effect is discovered deep in the activation stack in the middle of some obscure numerical computation.

9.4.1 Historical Approaches

Early languages attempted to deal with this problem in one of two equally unpleasant ways:

- Some languages require any mathematical function that might produce an error to return the status of that calculation to the calling function. They allow errors to be reported and processed, but they have two unpleasant consequences: using up the ability of a function to return a value and calling this function, which means choosing between testing for errors and solving the problem locally and passing the error condition back to its calling function in the hope that somewhere the error will be dealt with.
- Perhaps worse than this are the languages that use a globally accessible variable, such as `error`, to report status. For example, if `error` were normally set to 0, an error could be announced by setting its value to something other than 0 to indicate the nature of the failure. This frees the function from needing to return status, but it does not relieve the calling function of the need to check whether the `error` value is bad, or solving the problem, or elevating it. Furthermore, if an error does occur within a function, since it is now still returning a value, what value should it return if it is unable to complete its assigned calculation?

9.4.2 Generic Exception Implementation

By contrast, most enlightened languages provide an exception mechanism whereby if an error occurs, regardless of how deep in the activation stack, program implementation is immediately suspended in the current stack frame. The activation stack below this frame is then searched for the frame of a program that has “volunteered” to process this type of exception. When it is found, all the stack frames above this frame are removed from the stack and the code in the exception handling mechanism is activated. If no such frame is discovered, the overall program aborts with an error code.

The following mechanisms are necessary to implement the exception mechanism effectively:

- *Throwing an exception.* Whenever a problem occurs, the operating system is asked to suspend operations at that point in the activation stack and go back down the stack without completing any of the functions looking for a function equipped to handle the specific exception. If no such function is found, the program is terminated and an exception is returned to the operating system.
- *Catching an exception.* A function that is able to deal with a specific exception uses a `try ... catch` construct to identify the suspect

code and resolve the problem. Between `try` and `catch`, it puts a code block that contains the suspect code. After the `catch` statement that usually identifies the particular exception, there is a code block that should fix the problem.

Depending on the specific language implementation, the exception-catching mechanism usually offers facilities both for determining exactly where the exception occurred and for reconstructing the activation stack with all the variable values as they were at the time of the exception.

In the previous example, the general template for successfully interacting with the user is shown in Template 9.2. The successful Boolean flag will be set only if the data is processed without error. It does not matter how deep in the data processing code the error occurs—the user interface catches the error, reports it to the user, and prompts the user for better data.

9.4.3 MATLAB Implementation

MATLAB implements a simplified version of the most general form of exception processing. The `try ... catch ... end` construct is fully supported. However, unlike some languages, MATLAB does not distinguish between the kinds of exception that can be thrown.

- All built-in functions throw exceptions when they discover error conditions—attempting to open a nonexistent file for reading, for example—and expect the programmer to catch these exceptions if they are recoverable.
- To throw an exception manually, the program calls the `error(...)` function that takes one parameter, a string defining the error. If the exception is not caught, the string provided is displayed in red to the user. If the exception is caught, that string is ignored.
- To handle an exception, a code block we suspect might throw an exception is placed between `try` and `catch` statements. If no error occurs in the code block, the `catch` statement is ignored. If an

Template 9.2 General template for processing exceptions

```

successful = false
while <not successful>
    try
        <request data from the user>
        <process the data>
        successful = true
    catch
        <announce the error to the user>
    end
end
end

```

exception is thrown from that code block, however, execution is suspended at that point. No further processing is performed, no data are returned from functions, and the code in the closest `catch` block is executed up to the associated `end` statement. To determine the cause of the exception, you can use the `lasterror` function. It returns the textual information provided at the exception and a structure array describing the activation stack.

- In more complex situations where this function may not be able to actually handle the error, a further exception can be thrown from the `catch` block. This exception will escape from this `try ... catch` block, and must be caught (if at all) by another function or script deeper in the activation stack.

Listing 9.2 illustrates a simple example. The objective is to have the user define a triangle by entering a vector of three sides, and to calculate the angle between the first two sides. The `acosd(...)` function computes the inverse cosine of a ratio. If that ratio is greater than one, there is something seriously wrong with the triangle, and `acosd` returns a complex number. This script detects that the answer is complex and throws an exception.

In Listing 9.2:

Lines 1 and 2: We will repeat the attempts to compute the angle of a triangle until successful.

Line 3: Begins the suspect code.

Line 8: Detects the problem with the data.

Line 9: Throws the exception. In this case, the exception occurs visibly in this script. However, the `try ... catch` behavior is the same if the exception occurs deep in a set of nested function calls.

Listing 9.2 MATLAB script using exception processing

```

1. OK = false;
2. while ~OK
3.     try
4.         side = input('enter a triangle: ');
5.         a = side(1); b = side(2); c = side(3);
6.         cosC = (c^2 - a^2 - b^2)/(2 * a * b);
7.         angle = acosd(cosC);
8.         if imag(angle) ~= 0
9.             error('bad triangle')
10.        end
11.    catch
12.        disp('bad triangle - try again')
13.    end
14.    OK = true;
15. end
16. fprintf('the angle is %f\n', angle)

```

Line 11: The end of the suspect code block and the beginning of the exception handler—in this case, it's a warning to the user that the data is bad.

Line 14: This line is reached only if the suspect code block executed correctly, in which case we can exit the `while` loop.

You have an opportunity to work with exception processing in Exercise 9.2.

You may have noticed that the `input(...)` function has its own internal exception handling. If you ever enter some data that causes a parsing error, you see that error announced, and are then offered another opportunity to enter correct data.



Exercise 9.2 Processing exceptions

Do It Yourself

Put the code from Listing 9.2 in a script and execute it, using the following data:

```
enter a triangle: [3 4 8]
bad triangle - try again
enter a triangle: [3 4 6]
the angle is 62.720387
```

Then, edit the script to remove the `try` statement and the `catch` block and repeat the test.

Style Points

1. You should allow the exception-processing mechanism to simplify the structure of your code. Rather than attempting to detect every possible data error and return error condition, perhaps from deeply nested function calls, allow the exception mechanism to return control directly to the code that can deal with the problem.
2. Exception processing is for processing events that occur outside the normal thread of execution. It may be tempting at times to use the exception mechanism as a clever means of changing the normal flow of program control, but resist that temptation. It produces ugly, untraceable code and should be avoided.



9.5 Wrapper Functions

Consider the factorial function again for a moment—specifically, ask how you would deal with a user who accidentally called for the factorial of a negative number or of a number containing a fractional part. Our original

recursive `fact(...)` function is not protected from these programmer errors. There are three possible strategies for dealing with this situation:

1. *The legalist approach* ignores the bad values, lets the user's program die, and then responds to user complaints by pointing out that the documentation clearly indicates that you should not call for the factorial of a negative number. Usually this is not the best approach from the customer relations viewpoint or from the technical support effort viewpoint, especially since recursive code that hangs up typically crashes with a stack overflow—not the easiest symptom to diagnose!
2. *In-line coding* builds into the code a test for N less than zero (or fractional) and exits with a meaningful error message. Although this is an improvement over the first choice because it exits gracefully, the test is in a bad place. The function is recursive; therefore, the code for that test is repeated as many times as the function is called. While modern computers are fast enough that one would probably not notice the difference, in general this is a poor implementation that punishes those who are using the function correctly with the same test each time the recursive function is called.
3. *A wrapper function* is the best solution.

A wrapper function is called once to perform any tests or setup that the recursion requires, and then to call the recursive function as a helper to the main function call. While there is a small computational cost to using a wrapper, it is only executed once rather than each time the recursive function is called. Template 9.3 illustrates this idea.

Template 9.3 General template for a wrapper function

```
function <result> = <function_name> (<formal_params>)
<documentation>
    if <bad_condition>
        <throw exception>
    else
        <result> = <private_name> (<actual_params>)
    end

function <result> = <private_name> (<formal_params>)
<documentation>
    if <terminating condition 1>
        <result> = <initial value 1>
    elseif <terminating condition 2>
        <result> = <initial value 2>
    . . .
    else
        <result> = <operation> ...
            (<formal_params>, ...
             <private_name> (<new_params>) )
    end
```

The first function named `<function_name>` is actually the wrapper function with the return result, parameters, and documentation expected by the caller. It makes whatever tests are necessary to validate the input data, clean it up if necessary, and call the helper function named `<private_name>`.

Listing 9.3 is the MATLAB implementation of the factorial function with protection from bad data.

In Listing 9.3:

- Line 1: To the outside world, this is the function actually called.
(Ugly secret: even if the name is not the same name as the file, the first function in the file is always executed first.)
- Line 3: Checks for negative and fractional inputs.
- Line 4: Throws an exception if the data is bad.
- Line 6: Calls the recursive version if the data is valid.
- Line 8: Blank line for clarity—not required by MATLAB.
- Line 9: Definition of the recursive function. By convention, some MATLAB users tend to give the prefix `local_` to private functions like this, but this has no significance to the system.

Exercise 9.3 gives you an opportunity to work with the protected factorial.

Listing 9.3 Wrapper implementation for the factorial function

```

1. function result = fact(N)
2. % computation of N!
3.     if (N < 0) || ((N - floor(N)) > 0)
4.         error('bad parameter for fact');
5.     else
6.         result = local_fact(N);
7.     end
8.
9. function result = local_fact(N)
10. % recursive computation of N!
11. %     fprintf('fact( %d )\n', N);
12.     if N == 0
13.         result = 1;
14.     else
15.         result = N * local_fact(N - 1);
16.     end

```



Exercise 9.3 Writing the protected factorial

Do It Yourself

Write the `fact(...)` function as shown in Listing 9.3 and test it in the Command window:

```

>> fact(-1)
??? Error using ==> fact

```

continued on next page

```

bad parameter for fact
>> fact(.5)
??? Error using ==> fact
bad parameter for fact
>> fact(4)
ans =
    24

```

9.6 Tail Recursion

Although “normal” recursion is an effective way of using the function call mechanism to manage repetition, it can consume significant amounts of the activation stack. Tail recursion is a technique for accomplishing recursive behavior without consuming activation stack resources. It requires a wrapper function to initialize the recursion and uses an extra parameter in the function call to carry the emerging result of the recursion.

The driving characteristic of a tail recursive solution is that every exit from the recursive helper function must either return a result or be a standalone call to the recursive helper with no operations to be performed on it. All mathematical and logical operations must occur in computing the new parameters for the recursive call. When tail recursion is detected, almost all modern language compilers (including MATLAB) recognize this situation and construct the executable code to operate in place in the existing stack frame. This avoids pushing unnecessary duplicate stack frames onto the activation stack.

Template 9.4 is the general template for tail recursive processing. Notice that the wrapper function takes on the added responsibility of calling the helper function with an additional parameter that is the initial result.

Listing 9.4 is the MATLAB code for a tail recursive version of the factorial function.

Template 9.4 General template for tail recursive processing

```

function <result> = <function_name> (<formal_params>)
<documentation>
    <result> = <private_name> (<actual_params>, <init>)

function <result> = <private_name> (<params>, <current>)
<documentation>
    if <terminating condition>
        <result> = <current>
    else
        <result> = <private_name> (<new_params>, ...
                                <operation> (<params>));
    end

```

Listing 9.4 Tail recursive factorial implementation

```

1. function ans = fact(N)
2. % computation of N!
3.     ans = local_fact(N, 1);
4.     end
5.
6. function ans = local_fact(N, res)
7. %     fprintf('local_fact( %d, %d )\n', N, res);
8.     if N == 0
9.         ans = res;
10.    else
11.        ans = local_fact(N - 1, N * res);
12.    end

```

In Listing 9.4:

Line 3: The wrapper function calls the local, recursive factorial function with an extra parameter that is the initial value of the factorial result.

Line 7: A diagnostic statement for use in determining the sequence of events.

Line 8: Checks for the termination condition.

Line 9: If we are terminating, then all the computation has been done—the second parameter is the answer!

Line 11: Where all the work is done—the recursive call must stand by itself for tail recursion to work. The first parameter moves toward the termination as before and the second parameter is the computed ongoing result.

Try working with the tail recursive factorial in Exercise 9.4.

In Exercise 9.4 notice that all the computation is accomplished while “going forward” into the recursive process so that at the last recursive call the process is finished.

**Exercise 9.4** Writing the tail recursive factorial**Do It Yourself**

Write the `fact(...)` function as shown in Listing 9.4 with the `print` function uncommented, and test it in the Command window:

```

>> fact(4)
local_fact( 4, 1 )
local_fact( 3, 4 )
local_fact( 2, 12 )
local_fact( 1, 24 )
local_fact( 0, 24 )
ans =
    24

```



9.7 Mutual Recursion

Recursion is not always accomplished by a function directly calling itself. On rare occasions, the logic of a solution calls for function A to call function B, and then function B calls function A. Of course, at least one of the functions must be seeking the terminating condition.

The canonical example of this situation is a pair of odd/even functions, although there are more direct ways to determine odd and even using the remainder when the number is divided by 2. The function `odd(N)` can test for 0 and 1 to terminate, and then call `even(N-1)`. Similarly, the function `even(N)` can test for 0 and 1 to terminate, and then call `odd(N-1)`.



9.8 Generative Recursion

A discussion of recursion would not be complete without at least mentioning the concept of generative recursion. Consider a program that models the behavior of a billiard ball rolling on a billiard table and bouncing off the cushions. Does it represent the basic characteristics of recursion?

- There is definitely a terminating condition in which the ball reaches one of the pockets.
- One could certainly represent the repetitive process of updating the position of the ball as a recursive, even tail recursive, function.
- Is this process always moving toward the terminating condition? In one sense, obviously not, because the ball is sometimes moving away from a pocket and sometimes toward it. In fact, if friction is suitably represented in the model, the ball may roll to a stop without ever reaching a pocket.

However, in a broader sense, we could argue that the ball is approaching the terminating condition to the degree that falling into a pocket is in the future, and as long as time is elapsing in the model, the ball is moving toward the time when it is in a pocket. The fact that physics may intervene and prevent that future event from actually occurring does not detract from the ball's hopeful search for a resting place. There would be two terminating conditions: one with the ball in the pocket and the other with the ball stationary. Therefore, generative recursion is a recursive process in which there is a terminating condition, but the model of the process permits an unsuccessful attempt to achieve that condition.



9.9 Examples of Recursion

We conclude this chapter with three examples of recursive programming: detecting palindromes, computing the Fibonacci series of numbers, and

finding zeros of a function. The examples are followed by a practical engineering example of the use of zero finding.

9.9.1 Detecting Palindromes

We might want to determine whether a word or phrase received as a string is a palindrome, that is, whether it is spelled the same forward and backward. One could design a recursive function named `isPal(<string>)` as follows:

- The function `isPal(<string>)` terminates if the `<string>` has zero or one character, returning `true`.
- It also terminates if the first and last characters are not equal, returning `false`.
- Otherwise (first and last are equal), the function returns `isPal(<shorter string>)`, where the shorter string is obtained by removing the first and last characters of the original string.
- Clearly, since the string is always being shortened, the recursive solution is approaching the terminating condition.

The MATLAB implementation of the palindrome detector is shown in Listing 9.5.

In Listing 9.5:

Line 3: The successful terminating condition is when the length of the string is under 2.

Line 5: The failure condition is when the first and last characters do not match.

Line 8: To move toward termination, remove the first and last characters that have already been checked.

We should observe further that a serious student of palindromes might know that real palindromes contain spaces, punctuation marks, and uppercase and lowercase characters. We leave it as an exercise for you to write a wrapper function that cleans up strings containing these issues before passing the string to the recursive palindrome detector.

Listing 9.5 Recursive palindrome detector

```

1. function ans = isPal(str)
2. % recursive palindrome detector
3.     if length(str) < 2
4.         ans = true;
5.     elseif str(1) ~= str(end)
6.         ans = false;
7.     else
8.         ans = isPal(str(2:end-1));
9.     end

```

9.9.2 Fibonacci Series

The Fibonacci series was originally named for the Italian mathematician Leonardo Pisano Fibonacci, who was studying the growth of rabbit populations in the eleventh century. He hypothesized that rabbits mature one month after birth, after which time each pair would produce a new pair of rabbits each month. Starting with a pair of newborn rabbits free in a field, he wanted to calculate the rabbit population after a year. Figure 9.2 illustrates the calculation for the first six months, counting rabbit pairs. It soon becomes clear that the number of rabbits in month N comprises the number in month $N-1$ (since in this ideal example, none of them die) plus the new rabbits born to the mature pairs (shown in boxes in the figure). Since the rabbits mature after a month, the number of mature pairs that produce a new pair is the number of rabbits in the month before, $N-2$. So the algorithm for computing the population of pairs after N months, $\text{fib}(N)$, is recursive:

- There is a terminating condition: when $N = 1$ or $N = 2$, the answer is 1
- The recursive condition is: $\text{fib}(N) = \text{fib}(N-1) + \text{fib}(N-2)$
- The solution is moving toward the terminating condition, since as long as N is a positive integer, computing $N-1$ and $N-2$ will move towards 1 or 2.

The MATLAB implementation of the Fibonacci function is shown in Listing 9.6.

The algorithm produces the Fibonacci series: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ..., giving a population after a year of 144.

Truthfulness requires pointing out that while computing the Fibonacci series recursively is a very nice, conceptually simple approach, it is a nightmare as far as the computational load on your processor. Do not try to compute beyond about 27 numbers in the series. An iterative solution, while less elegant, runs in linear time rather than exponential.

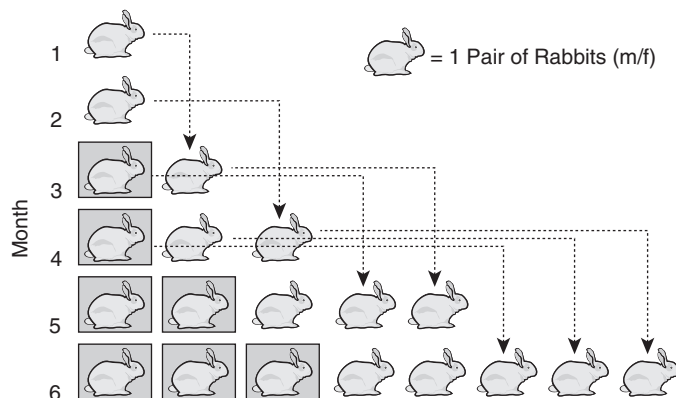


Figure 9.2 Computing rabbit populations

Listing 9.6 The Fibonacci function

```

1. function result = fib(N)
2. % recursive computation the Nth Fibonacci number
3.     if N == 1 || N == 2
4.         result = 1;
5.     else
6.         result = fib(N-1) + fib(N-2);
7.     end

```

A closely related phenomenon is the golden ratio or golden number computed as the limit of the ratio of successive Fibonacci series values—approximately 1.618034—that has been found to occur in nature. To the surprise of naturalists, this series of numbers occurs in nature in a remarkable number of circumstances. Consider Figure 9.3 for example, where a set of squares placed side by side in a rotating sequence is drawn using the Fibonacci series for the size of each square. The resulting geometric figure is a close approximation to the logarithmic spiral so frequently found in nature, such as the nautilus shell pictured in the figure.

9.9.3 Zeros of a Function

Frequently we need to solve nonlinear equations by seeking the values of the independent variable that produced a zero result. There are a number of well-known numerical techniques for achieving this goal. We will examine a recursive approach to determining the zeros of functions. However, especially when there are multiple zero crossings, it is very helpful to have a good initial estimate of the location(s) of the crossing(s). As an example, consider a function $f(x)$. We will use the function given by:

$$f(x) = 0.0333x^6 - 0.3x^5 - 1.3333x^4 + 16x^3 - 187.2x + 172.9$$

as plotted in Figure 9.4. However, this algorithm will work for any function of x . We assume that the continuous line describes the exact function, and the

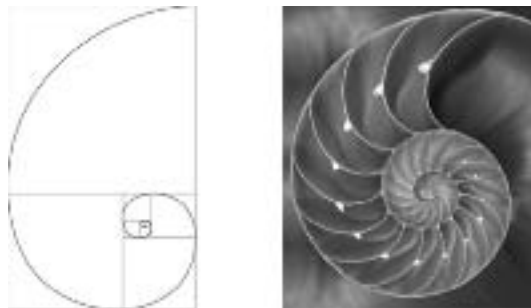


Figure 9.3 *Fibonacci in nature*

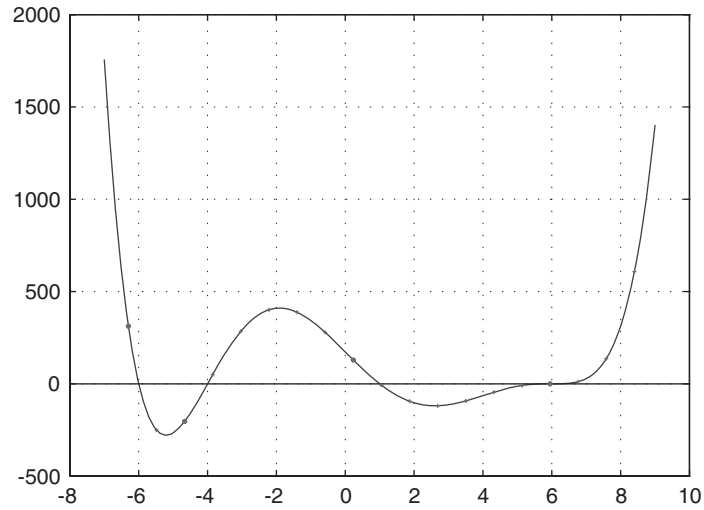


Figure 9.4 A function $f(x)$

plus marks indicate locations for which we have measurements. Clearly, there are a number of zero crossings of this function, including a very messy looking crossing at around $x = 6$.

We will find the exact value of one of the zeros of this function by first estimating the zero crossings, and then using a recursive technique for refining a better estimate to arbitrary levels of accuracy.

Estimating Critical Points of a Function First, we need to compute an approximation to the roots of this equation. These approximations will be found by finding the x values at which adjacent values of the function change sign. The technique for determining where adjacent points change sign is simply to multiply adjacent values of $f(x)$ and find where that product is not positive, as shown in Listing 9.7.

In Listing 9.7:

Line 1: Establishes samples of x .

Line 2: Computes $y = f(x)$.

Line 3: Detects the indices where the zero crossings occur by

Listing 9.7 Initial zero crossings

```
1. px = linspace(-6.3, 8.4, 19);
2. py = f(px);
3. zeros = find(py(1:end-1) .* py(2:end) <= 0)
4. disp('zeros occur just after')
5. px(zeros)
6. root = findZero([px(zeros(3)) px(zeros(3)+1)])
```

shifting y to the right by one slot, and shortening the original by one to keep the vector size equal.

Lines 4 and 5: Displays the zero crossing estimates.

Line 6: Calls the recursive function to refine the third root of this equation.

Listing 9.7 produces the following results, which can be verified by observing the circled data points shown in Figure 9.4:

```
zeros occur just after
ans =
    -6.3000    -4.6667     0.2333     5.9500
```

Having observed these results, we decide to compute the exact value of the first positive root, occurring at the third crossing.

Recursive Refinement of the Estimate The recursive function to find the third root of $f(x)$ works on the principle of binary division. It consumes a vector of adjacent values of x that are guaranteed to have values of $f(x)$ of opposite sign. The three characteristics of recursion implemented here are as follows:

- The terminating condition is when the two x values are within acceptable error—in this case, 0.001
- Otherwise, we find the middle of this x range, mx , find its $f(mx)$, and then make the recursive call either with $[x(1) \ mx]$ or $[mx \ x(2)]$, depending on the sign of mx
- This will always converge because each recursive call halves the distance between the x limits

In general, this method is a little slower than Newton's method, which uses the slope of $f(x)$ to compute the next estimate. However, it is very strong and immune from the instability suffered by Newton's method on undulating data. The function that solves this problem is shown in Listing 9.8.

Listing 9.8 Recursive root finding

```
1. function pt = findZero(x)
% x is a lower-upper pair guaranteed to have
%   y values of opposite sign
%   return the x coordinate of the root
2. if abs(x(1)-x(2)) < .001
3.     pt = x(1);
4. else
5.     mx = sum(x)/2;
6.     my = f(mx);
7.     if my*f(x(1)) <= 0
8.         pt = findZero([x(1) mx]);
9.     else
10.        pt = findZero([mx x(2)]);
11.    end
12. end
```

In Listing 9.8:

- Line 1: The function consumes a pair of x limits and produces the x root.
- Lines 2–4: Check for the terminating condition and return the x root.
- Lines 5 and 6: Calculate the x and y values of the midpoint of the x range.
- Line 7: Checks the sign of the y value of the midpoint.
- Line 8: If different from the first limit, it makes the recursive call with the first limit and the midpoint.
- Lines 9 and 10: Otherwise, it uses the range from the midpoint to the second limit.

This function computes the correct crossing at $x = 1.00$.

9.10 Engineering Example—Robot Arm Motion

Here we consider the problem of programming the arm of a robot to move in a straight line. Consider the arm shown in Figure 9.5. It consists of two jointed limbs of length r_1 and r_2 at angles α and β , respectively, to the horizontal.

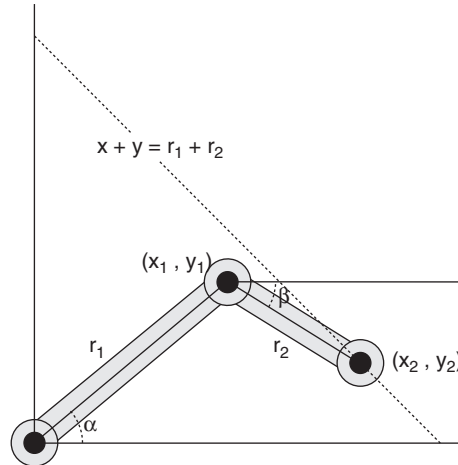


Figure 9.5 *The robot arm problem*

Overall Objective

The ultimate challenge of this situation is to calculate the sequence of values of α and β that will guide the end of the arm along the straight line:

$$x + y = r_1 + r_2 \quad (1)$$

However, this complete problem is more complex than necessary for this point in the text. First we will address a necessary component of the problem.

Immediate Objective

The subproblem we address here is to determine for a given value of α , the value β that will place the end of the arm at some place on the line. The algebra and trigonometry of this problem are quite simple. The position of the end of the arm, $[x_2 \ y_2]$, is expressed as:

$$x_2 = r_1 \cos \alpha + r_2 \cos \beta \quad (2)$$

$$y_2 = r_1 \sin \alpha + r_2 \sin \beta \quad (3)$$

Combining these two relationships with Equation (1) gives the equation for $F(\beta)$, the difference between the end point derived from β and the straight line. We need to solve this for $F(\beta) = 0$:

$$F(\beta) = r_1 \cos \alpha + r_2 \cos \beta + r_1 \sin \alpha + r_2 \sin \beta - (r_1 + r_2) \quad (4)$$

If we are given values for r_1 , r_2 , and the angle α , we will use the method of Section 9.9.3 to find the value(s) of β that satisfies this equation. By inspecting Figure 9.5, we might expect two answers—one with a small negative value and one “bending backwards” at an angle greater than 90° .

Figure 9.6 shows a plot of this function for $r_1 = 4$, $r_2 = 3$, and $\alpha = 30^\circ$. The zero crossings of this function confirm our intuition that there are two values of β that satisfy the equation for small, positive values of α : one around -30° and one around 110° .

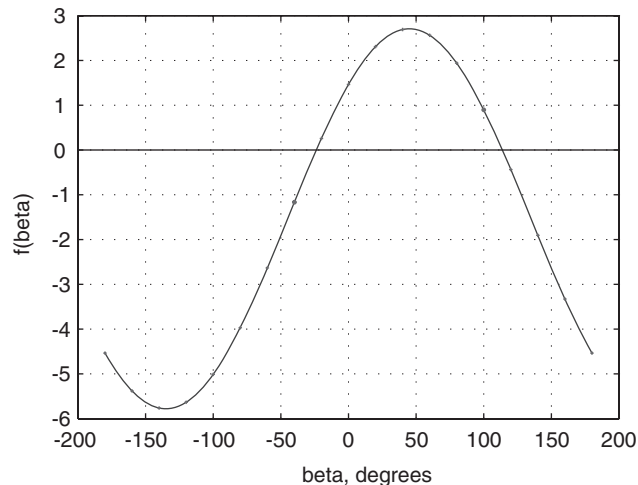


Figure 9.6 The relationship between β and the value of $F(\beta)$

The Solution to the Subproblem

As before, since there is no analytical solution to this function, we will find the approximate location of the zero crossings and then use a recursive function to find the exact roots. The script that accomplishes this is shown in Listing 9.9.

In Listing 9.9:

Lines 3–8: Establish the parameters of the problem as `global` variables to avoid the overhead cost of passing them into recursive functions.

Lines 9 and 10: Sample the possible range of beta values with enough values to identify the zero crossings, and compute the corresponding values of $F(\beta)$.

Lines 11–13: Estimate the zero locations by multiplying adjacent function values and display the results.

Line 14: Calls the recursive function to find the zero crossing.

Running this script produces the following:

```
r1 =
    4
r2 =
    3
alpha =
    0.5236
zeros =
    8    15
zeros occur just after
ans =
   -0.6981    1.7453
zero =
   -0.4152   -0.0009
```

Listing 9.9 Finding arm position

```
1. clear
2. clc

3. global r1
4. r1 = 4
5. global r2
6. r2 = 3
7. global alpha
8. alpha = pi/6 % 30 deg
9. beta = linspace(-pi, pi, 19);
10. pf = fab(beta);
11. zeros = find(pf(1:end-1) .* pf(2:end) <= 0)
12. disp('zeros occur just after')
13. beta(zeros)
%
14. zero = findZeroAB([beta(zeros(1)) ...
                     beta(zeros(1)+1)])
```

The function for which we are seeking the zero is shown in Listing 9.10.

In Listing 9.10:

Lines 2–4: Gain access to the global parameters.

Line 5: Computes the left-hand side of Equation (4).

The function that finds the zero crossings of `fab(beta)` is shown in Listing 9.11.

In Listing 9.11:

Line 2: Computes the y values corresponding to the x limits.

Lines 3 and 4: Check the terminating condition and return the $[x \ y]$ coordinates of the result.

Lines 6 and 7: Find the x and y values of the midpoint.

Lines 8 and 9: If the midpoint is on the opposite side of the x axis from the lower limit, make a recursive call using these limits.

Line 11: Otherwise, makes the recursive call using the midpoint and the upper limit.

Listing 9.10 Function for zeros

```
1. function res = fab(beta)
% f(beta) = r1 (cos(alpha) + sin(alpha) - 1)
%          + r2 (cos(beta) + sin(beta) - 1)
2. global r1
3. global r2
4. global alpha
5. res = r1 * (cos(alpha) + sin(alpha) - 1) ...
        + r2 * (cos(beta) + sin(beta) - 1);
```

Listing 9.11 Recursive zero finder

```
1. function pt = findZeroAB(x)
% x is a lower-upper pair guaranteed to have
%          y values of opposite sign
2. y = fab(x);
3. if abs(x(1)-x(2)) < .001
4.     pt = [x(1) y(1)];
5. else
6.     mx = sum(x)/2;
7.     my = fab(mx);
8.     if my*y(1) < 0
9.         pt = findZeroAB([x(1) mx]);
10.    else
11.        pt = findZeroAB([mx x(2)]);
12.    end
13. end
```

Reflection

A modest amount of code is all that is required to create an elegant solution to a nontrivial problem. The structure of the recursive function shown in Listing 9.9 clearly reflects the standard recursive template, and that function can be used to find zeros of any continuous function defined in `fab(x)`.



Chapter Summary

This chapter discussed the three basic principles of recursive programming that must be present for a recursive program to succeed:

- There must be a terminating condition
- The function must call a clone of itself
- The parameters of that clone must move the function toward the terminating condition

We have also seen some other important capabilities as follows:

- Exceptions are declared either within MATLAB system functions or by the user using the `error(...)` function; they are trapped and perhaps remedied using `try ... catch` code blocks
- A wrapper function is used to set up a recursive solution by validating the incoming data
- Tail recursion is a programming style that enables the recursive computation to happen on the way onto the stack, and usually results in no stack usage at all
- Mutual recursion can happen in rare circumstances, and must still obey the fundamental tenets of recursion



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>catch</code>	End of a suspect code block where the exception is trapped	9.4.3
<code>error(str)</code>	Announces an error with the string provided	9.4.3
<code>global var</code>	Defines a variable as globally accessible	9.10
<code>lasterror</code>	Provides a structure describing the environment from which an exception was thrown	9.4.3
<code>try</code>	Begins a block of suspect code from which an exception might be thrown	9.4.3

**Self Test**

Use the following questions to check your understanding of the material in this chapter:

True or False

1. We limit the functionality of a stack in order to protect the data from corruption.
2. The only way to remove a stack frame from the activation stack is to exit from the function instance hosted by that frame.
3. All the math operations in a recursive function are performed as the activation stack unwinds.
4. Exception processing can be used as a clever means of changing the normal flow of program control.
5. The name of the first function in a function definition m-file must match the name of the file.

Fill in the Blanks

1. Recursion is _____ by which a code block can be repeated in a controlled manner.
2. Very frequently, a recursive function needs a _____ to set up the recursion correctly and to _____.
3. Exceptions are a powerful tool for managing _____ caused by either _____ or _____.
4. A wrapper function is called once to perform _____ that the recursion requires, and then to call the recursive function _____.
5. Every exit from a tail recursive function must return either a _____ or the result of a _____.



Programming Projects

1. For this problem you will be required to write three functions: `recurSum`, `recurProd` and `fibVector`. The first one will take in a vector and compute the sum of the elements of the vector. The second one will take in a vector and compute the product of the elements of the vector. The third one will take in a number, n , and return a vector containing the first n terms of the Fibonacci sequence. You must use recursion to complete these functions. You may not use `for` loops, `while` loops or the MATLAB functions `sum`, `prod` or `factorial`. Your function headers should be:

```
function ans = recurSum(arr)
function ans = recurProd(arr)
function vec = fibVector (num)
```

2. Write a recursive function called `oddfact(n)` which takes in a number and returns the factorial of the odd numbers between the given number and 1.

For example:

```
oddfact(4) returns 3
oddfact(9) returns 945 = 9*7*5*3*1
```

3. Consider the problem of structures with nested fields.
 - a. Write a function called `tracker` that takes in a structure and returns the number of levels at which it has a field called 'Inner'. Each of these fields can also be structures having a field called 'Inner', but at each level there can be only one field called 'Inner'. The innermost structure will not contain a field called 'Inner'. You must use recursion. Your function header should be:

```
function num = tracker(astruct)
```

Hint

Use the 'isfield' function.

- b. Create a structure with at least three levels of recurring fields and use it to test your `tracker` function.

4. Create a recursive function with a wrapper to protect it from illegal values. The function name should be `recursiveFib`. It should take in a number n and return the n th Fibonacci number. You should ensure that n is a non-negative integer, and announce an error if that is not the case.

Fibonacci numbers are defined as:

```
F(n) = 0                if n = 0
F(n) = 1                if n = 1
F(n) = F(n-1) + F(n-2) otherwise.
```


This produces the following sequence of numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55...

For example:

```
a = recursiveFib(0) should return 0
b = recursiveFib(1) should return 1
c = recursiveFib(-1) should cause an error
d = recursiveFib(8) should return 21
```

5. Create and test a function called `recursiveMin` that takes in a vector and returns the element with the minimum value and the index of that element as separate returned values, much as the standard `min(...)` function. If the input vector is of length zero, your function should return two empty vectors. If the input vector contains two minimum elements of equal value, your function should return the index of the first element. Create suitable test cases and use the built-in function `min(...)` only to test your answers.

For example:

```
[m n] = recursiveMin([]) should return [] and []
[m n] = recursiveMin([5]) should return 5 and 1
[m n] = recursiveMin([5 2]) should return 2 and 2
[m n] = recursiveMin([2 5 2]) should return 2 and 1
[m n] = recursiveMin([2 5 2 1 6 7]) should return
                                1 and 4
```

6. Write a recursive function named `sumRings` that consumes an `m` by `n` array and outputs a vector of the sums of its consecutive rings. If the input array is a `1xN` or `Mx1` (row or column vector), find the sum of its elements. If the input array is empty, the output should be an empty vector (*not* zero).

You may *only* use the following built-in functions: `size`, `sum`, `isempty`, or `length`. You must use recursion to solve this problem.

For example:

```
A = [1 2 3
      4 5 6
      7 8 9];
sumA = sumRings(A) should return [40 5]
B = [1 2 3 4
      5 6 7 8
      9 10 11 12];
sumB = sumRings(B) should return [65 13]
C = [1 2 3 4
      5 6 7 8
      9 10 11 12
      13 14 15 16];
sumC = sumRings(C) should return [102 34]
D = [1 2 3 4]';
sumD = sumRings(D) should return 10
```

Principles of Problem Solving

Chapter Objectives

This chapter presents an overview of framing the solutions to problems:

- We begin with simple problems that can be solved in a single step
- We continue to strategies for solving more complex problems involving data collections by dividing the solution into the following fundamental operations that can be performed on any collection of data:
 - Inserting
 - Traversing
 - Building
 - Mapping
 - Filtering
 - Summarizing
 - Searching
 - Sorting

Then we will briefly discuss how to combine these fundamental tools to solve more complex data manipulation problems.

Introduction

Programming is really all about applying the computer as a tool to solve problems. One of the most difficult tasks facing novice programmers is the blank sheet of paper. Faced with a problem you have never seen before, how do you start to solve it? The problem-solving style recommended in this text is first to identify the basic character of the data and the basic operation(s) we are asked to perform. If these two ideas are clear, we can create a template or outline of the solution, and begin to fill in the blanks.

- 10.1** Solving Simple Problems
- 10.2** Assembling Solution Steps
- 10.3** Summary of Operations
 - 10.3.1 Basic Arithmetic Operations
 - 10.3.2 Inserting into a Collection
 - 10.3.3 Traversing a Collection
 - 10.3.4 Building a Collection
 - 10.3.5 Mapping a Collection
 - 10.3.6 Filtering a Collection
 - 10.3.7 Summarizing a Collection
 - 10.3.8 Searching a Collection
 - 10.3.9 Sorting a Collection
- 10.4** Solving Larger Problems
- 10.5** Engineering Example—Processing Geopolitical Data

As we gain more experience with the language, we have more computing tools to apply, and we can attack larger, more complex problems. We now have sufficient tools available to consider a more principled approach to data manipulation and problem solving. We will begin with the typical plan for solving simple problems in one step, and then continue to consider assembling multiple steps to solve more complex problems.

10.1 Solving Simple Problems

In Chapter 2 we saw the basic plan for solving simple problems:

- Define the input data
- Define the output data
- Discover the underlying equations to solve the problem
- Implement the solution
- Test the results
- Repair the code until it conforms to the specifications

This plan works whenever the problem is simple enough to be able to visualize the complete solution. Typically, however, problems are more complex and require a number of steps to be assembled.

10.2 Assembling Solution Steps

Problem complexity frequently comes in the form of data collections that need to be transformed into other collections or summarized as intermediate results. Identifying the operation(s) that will create the output from the input requires some experience. The rest of this chapter provides some guidelines for identifying elementary solution steps that can be combined to create solutions to many complex problems.

10.3 Summary of Operations

First, we document the operations we expect to be able to perform on collections. Table 10.1 lists the generic operations, a brief description of each, and a discussion of the consequences. The following paragraphs illustrate these fundamental operations, using the array of structures from Chapter 7 as examples. The discussion of each step takes the form of a written description, a flowchart, and a template for writing the code.

Style Points

It is conceivable—and in fact, a common practice—to combine multiple operations into one computing module, but it is poor abstraction and leads to code that is hard to understand and/or debug.

Table 10.1 Taxonomy of solution steps

Operation	Description	Consequence
Insert	Inserts one item into a collection	An updated collection of data
Traverse	Touches each item of data in the collection	The collection is unchanged— frequently used to display or copy a collection
Build	Creates a collection from a data source (external file or traversing another collection); usually accomplished by inserting one item at a time	A new collection of data
Map	Changes the content of some or all of the items in the collection	A new collection of the same length, but the content of some or all items is changed
Filter	Removes some items from the collection	A new collection with reduced length, but the content of the items remains unchanged
Fold	Touches the entire collection, summarizing the contents with a single result	A single result summarizing the collection in some way (e.g., sum, max, or mean)
Search	Traverses the collection until an item matches a given search criterion and then stops, returning a result	A single result or the indication that the result was not achieved
Sort	Puts the collection in order by some specific criterion	A new collection of the same length in order

10.3.1 Basic Arithmetic Operations

The simple problem solution described in Section 10.1 frequently needs to be used as part of a larger problem solution. We include this activity in this list for completeness.

10.3.2 Inserting into a Collection

Inserting an item into a collection is a process usually used to build or maintain a collection of information. In MATLAB, we have seen four basic data collection types to which insertion applies: vectors, arrays, cell arrays, and structure arrays. We will discuss the peculiarities of each collection, and then the common processing algorithm that can be used to insert a new entry into the collection.

- **Vectors** are very flexible collections in MATLAB, and suffer only from the obvious limitation that one can add only numbers to a vector

- **Arrays** are as flexible as vectors, except that they require that new data be inserted a row or column at a time, and that the size of the row or column must match the existing array dimensions
- **Cell arrays** can be indexed like number arrays, and can contain any object; however, to compare one element to another usually requires a special-purpose comparison function
- **Structure arrays** as a collection behave like cell arrays, except that any structure inserted must have the same fields in the same order as those in the existing structure

In general, inserting into any of these collections involves insertion into the front of the collection, the back of the collection, or at some position in the middle by a specific comparison method.

Inserting into the front is accomplished by concatenating the new element before the existing collection. For example, adding `item` to the front of an existing cell array, `ca`, is accomplished as follows:

```
>> ca = [{item} ca] % note the braces needed for a cell array
```

Inserting at the back is accomplished by concatenating the new element after the existing collection. For example, adding `item` to the back of an existing cell array, `ca`, is accomplished as follows:

```
>> ca = [ca {item}] % note the braces needed for a cell array
```

Inserting in order is usually accomplished using a `while` loop. If we are inserting `item` into a collection `coll`, we will use a `while` loop to find the index of the insertion point, `ins`, and then concatenate the three parts of the new collection. Figure 10.1 shows the flowchart that applies here, and Template 10.1 shows the template for the general solution.

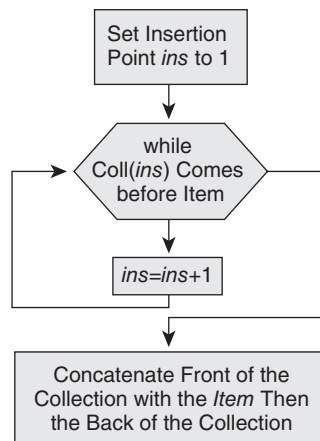


Figure 10.1 *Inserting in order*

Template 10.1 Template for inserting

```

%inserting item into a collection coll

<set insert point, ins, at the front>
while <insertion point in coll and
      item comes before coll(ins)>
    <move insertion point forward>
<end of the while loop>
<concatenate coll before ins with item and
  coll at and beyond ins>

```

For example, adding *item* in order to a vector, *v*, is accomplished as follows:

```

ins = 1;
while ins <= length(v) && before(item, v(ins))
    ins = ins + 1;
end
v = [v(1:ins-1) item v(ins:end)]

```

where *before(a,b)* is a generic comparator that determines whether *a* comes before *b* in the ordering scheme. Notice that this covers the cases where *item* must be the first or last item in the collection. Consequently, we could include the case of front or back insertion by having *before(a,b)* return *true* for inserting in the front and *false* for inserting at the back.

10.3.3 Traversing a Collection

Traversal involves moving across all elements of a collection and performing some step (not necessarily the same step) on each element without changing that element. Figure 10.2 and Template 10.2 illustrate the flowchart and basic template for traversing a collection. They assume that you are doing something like writing a file that needs to be initialized and finalized. These two steps may not always be required.

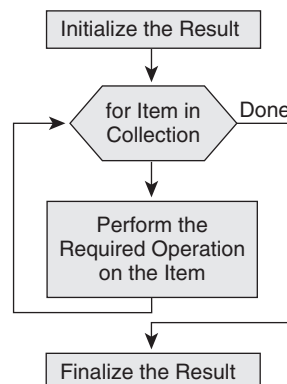


Figure 10.2 *Traversing a collection*

Template 10.2 Template for traversing

```

<initialize the result>
for item <across the whole collection>
    <operate on the item>
<end of the loop>
<finalize the result>

```

Listing 10.1 illustrates a function to write to a text file a selected attribute for all the CDs in a collection. Since some attributes are numeric and some strings, we need a helper function that is sensitive to the type of data to convert the data to string form. Such a helper function is listed in Listing 10.2. This function could be invoked with the following line:

```
>> printCDs(collection, 'artists.txt', 'artist');
```

In Listing 10.1:

Line 6: We open the file of the name provided for writing, and return a file handle, `fh`, to be supplied to subsequent functions to identify the file.

Line 7: Shows the simplest form of the `for` loop, taking each item in turn from the CD collection.

Line 8: Shows the general form of the `fprintf(...)` usage where the first parameter is the handle of the file to which the output will be sent. As we have already seen, if this handle is omitted, the output goes to the Command window.

Line 9: We invoke the helper function `toString(...)` to convert the attribute to a string before being written to the output file. Structure fields can be accessed indirectly via a variable by putting the variable in parentheses when selecting the structure attribute (`CD.(attrib)`).

Line 11: This closes the file to make it accessible to any text editor.

Listing 10.2 shows the basic form of a helper function that will convert data objects of any class to the string representation you select.

Listing 10.1 Listing one field to a text file

```

1. function printCDs(CDs, filename, attrib)
2. % usage: printCDs(CDs, attrib, type)
3. % print the specified attribute of all the CDs
4. %   attrib must be one of the attributes as
5. %                               a character string
6. fh = fopen(filename, 'w');
7. for CD = CDs
8.     fprintf( fh, '%s: %s\n'], ...
9.           attrib, toString(CD.(attrib)) );
10. end
11. fclose(fh);

```

Listing 10.2 The `toString(...)` helper function

```

1. function str = toString(item)
2. % convert an object of any class to string form
3. if isa(item, 'char')
4.     str = ['' item ''];
5. elseif isa(item, 'double')
6.     if length(item) == 1
7.         str = sprintf('%g', item );
8.     else
9.         str = '[';
10.        for in = 1:length(item)
11.            str = [str sprintf(' %g', item(in) )];
12.        end
13.        str = [str ' '];
14.    end
15. elseif isa(item, 'struct')
16.    nms = fieldnames(item);
17.    str = [];
18.    for in = 1:length(nms)
19.        nm = nms{in};
20.        str=[str nm ': ' toString(item.(nm)) 13];
21.    end
22. else
23.    str = 'unknown data';
24. end

```

In Listing 10.2:

Lines 3 and 4: We first form character strings by surrounding them with single quotes.

Lines 5–7: Scalars are shown with `%g` conversion without punctuation.

Lines 8–14: Vectors are surrounded by brackets (this version does not deal gracefully with arrays of more than one dimension).

Lines 15–21: Each field of a structure is listed by name, using recursive calls to `toString(...)`. The function does not follow the normal recursive template, because it depends on the natural character of structures eventually to terminate (not contain other structures).

Line 23: A trap for data classes not yet covered by this function.

For a second traversal example, consider the need to save a collection to a text file after making modifications to the original collection. This is a little tricky because the modifications may include adding or deleting fields. Therefore, the text file must contain the field names and types. The function shown in Listing 10.3 accomplishes this. It is called by the following line:

```
>> writeCDs(collection, 'myCDs.txt');
```


Listing 10.3 Function to write CDs to a text file

```

1. function writeCDs(CDs, ilename)
2. % write a text file from which a general
3. % structure can be retrieved
4. fh = fopen(ilename, 'w');
5. attribs = fieldnames(CDs);
6. fprintf(fh, '%d\t%d\n', ...
           length(CDs), length(attribs) );
7. for index = 1:length(attribs)
8.     fprintf(fh, '%s\t', attribs{index} );
9. end
10. fprintf(fh, '\n');
11. for index = 1:length(attribs)
12.     att = attribs{index};
13.     type{index} = class(CDs(1).(att));
14.     fprintf(fh, '%s\t', type{index} );
15. end
16. fprintf(fh, '\n');
17. for CD = CDs
18.     for index = 1:length(attribs)
19.         switch class(CD.(attribs{index}))
20.             case 'char'
21.                 str = [ '' CD.(attribs{index}) '' ];
22.             case 'double'
23.                 str=sprintf('%g', ...
                              CD.(attribs{index}) );
24.             end
25.             fprintf(fh, '%s\t', str );
26.         end
27.         fprintf(fh, '\n');
28.     end
29. fclose(fh);

```

In Listing 10.3:

- Line 4: Opens the file for writing.
- Line 5: Retrieves the field names.
- Line 6: Writes the number of records and fields.
- Lines 7–10: Write the field names.
- Lines 11–16: Fetch and write the class of each field.
- Lines 17–28: Write the data for each structure.
- Line 29: Closes the file.

10.3.4 Building a Collection

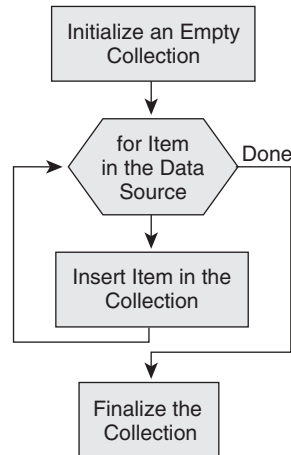
In practice, frequently we combine traversal of one collection and building of another to copy data from one collection into another. Building a collection is the process of beginning with an empty collection and assembling data elements by inserting them one at a time into the new collection. The size of the collection increases continually until the process is finished. Figure 10.3 and Template 10.3 illustrate the algorithm for building a collection.

Template 10.3 Template for building

```

<initialize the new collection>
for item <across the data source>
    <extract the item>
    <insert item in new collection>
<end of the loop>
<finalize the new collection>

```

**Figure 10.3** *Building a collection*

For example, the CD collection can be built by applying a custom read function to a text file containing the CD information with the following command:

```
>> collection = readCDs('myCDs.txt');
```

This function expects the data file—which was originally written by traversing a CD collection (see Listing 10.4)—in a specific format, as described in its header. The custom function to read the file is shown in Listing 10.4.

Listing 10.4 Custom structure read function

```

1. function CDs = readCDs(filename)
2. % read a text file describing the CD collection.
3. % The first line specifies the number of CDs and
4. % attributes
5. % The second line lists the attributes
6. % The 3rd line lists the class of each attribute
7. % Subsequent lines list the data for each CD,
8. %   one line per CD
9. fh = fopen(fileName, 'r');
10. ns = textscan(fh, '%d %d', 1);
11. noCDs = ns{1};
12. noAtts = double(ns{2});

```

continued on next page

```

13. temp = textscan(fh, '%s\t', noAtts );
14. att = temp{1};
15. temp = textscan(fh, '%s\t', noAtts );
16. type = temp{1};
17. form = '';
18. for at = 1:noAtts
19.     switch type{at}
20.         case 'char'
21.             form = [form '%q'];
22.         case 'double'
23.             form = [form '%f'];
24.     end
25.     if at < noAtts
26.         form = [form ' '];
27.     end
28. end
29. for line = 1:noCDs
30.     temp = textscan(fh, form, 1 );
31.     for at = 1:noAtts
32.         value = temp{at};
33.         switch type{at}
34.             case 'char'
35.                 value = value{1};
36.             end
37.             CD.(att{at}) = value;
38.         end
39.         CDs(line) = CD;
40.     end
41. fclose(fh);

```

In Listing 10.4:

Line 1: This function consumes a file name and produces an array of structures.

Lines 2–8: Description of the expected file format.

Line 9: Opens the file for reading—the result is a file handle, `fh`, to be used in subsequent file I/O calls.

Line 10: Uses `textscan(...)` to extract two numbers: the number of CDs and the number of attributes. The results are returned in a cell array.

Lines 11 and 12: Extract the sizes.

Lines 13 and 14: These lines read in and save the field names. When `textscan(...)` is provided with a third, numerical parameter, it performs a limited number of applications of the format string. Without that limitation, it would attempt to read the whole file. The results from each specified field in the format string are nested in the same cell array. So we find each field name in the first cell of the information returned.

Lines 15 and 16: These lines return the data types of each field in the same way.

Lines 17–28: These lines build the format string for reading each data record: '%q' for character strings and '%f' for numerical values. '%q' is a format parameter designed for `textscan(...)` to allow quoted strings to be read.

Lines 25–27: These lines provide for the spaces between format control parameters.

Lines 29–40: These lines actually extract and store the data values for each structure. If the data are characters, they must be extracted from the containing cell array (Lines 33–36).

Line 41: Close the data file.

Style Points

A simpler example of collection building occurred when we built the CD collection initially by repeated calls to the `makeCD` method, as shown in Chapter 7, Table 7.9, inserting each item at the end of the collection. However, while that example seems to simplify the process of building the collection, it really did not. The data for the function calls had to be extracted from a CD listing and edited to construct the function calls—normally not an efficient or effective way to compose a collection. Such hard-wiring should generally be avoided.

Because of the generality of the data, this is a slightly complex example of code to build a structure array. However, do not lose sight of the fact that it really is just scanning through a data file, extracting each structure in turn, and inserting that structure into the resulting collection.

10.3.5 Mapping a Collection

The purpose of mapping is to transform a collection by changing the data in

some or all of its elements according to some functional description without changing its length. It is distinct from traversal because its intent is to change the data elements. While many languages permit collections to be modified in place, MATLAB usually requires you to create a new collection. However, this is still considered mapping. The scalar mathematical and logical operations on vectors are good examples of mapping. Figure 10.4 and Template 10.4 illustrate the basic algorithm for mapping. As illustrated in the example of operations on vectors, mapping may involve combining two or more collections of the same length.

Template 10.4 Template for mapping

```
<initialize the result>
for item <across the whole collection>
    <extract the item>
    <modify the item>
    <insert modified item in the result>
<end of the loop>
<finalize the result>
```

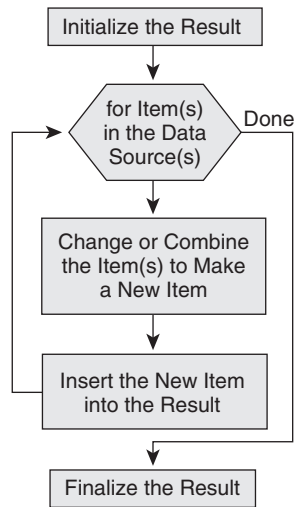


Figure 10.4 Mapping a collection

Listing 10.5 shows a function that will add an attribute to all the CDs reflecting the value of each CD as the ratio of the number of stars to the price. This function would be invoked by the following:

```
>> addedCDs = addValue(collection);
```

In Listing 10.5:

Line 4: Notice the use of the indexed form of the `for` loop in order to be able to store the new CD in the new collection.

Line 6: Notice the use of `eps` to guard against free CDs in the collection.

10.3.6 Filtering a Collection

Filtering involves removing items from a collection according to specified selection criteria. The data contents of the remaining items in the collection should not be changed, and the collection will usually be shorter than before. Typically, we filter vectors by applying built-in logical operations and then indexing with the results to produce new, shorter arrays. Figure 10.5 and Template 10.5 illustrate the general algorithm for filtering a collection.

Listing 10.5 Adding value to a CD collection

```

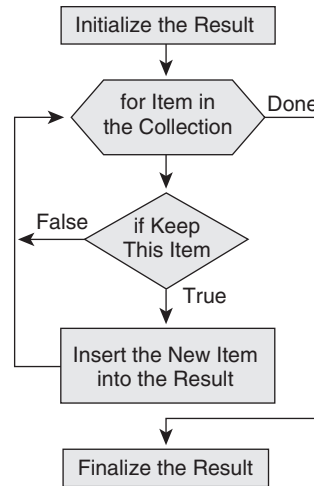
1. function newCDs = addValue( CDs )
2. % newCDs = addValue( CDs )
3. % add the value attribute
4. for index = 1:length(CDs)
5.     CD = CDs(index);
6.     CD.value = CD.stars / (CD.price + eps);
7.     newCDs(index) = CD;
8. end
  
```

Template 10.5 Template for filtering

```

<initialize the new collection>
for item <across the whole collection>
  <extract the item>
  if <keep the item>
    <insert item in new collection>
  <end if>
<end for>
<finalize the new collection>

```

**Figure 10.5** *Filtering a collection*

Listing 10.6 is the code for a generic function to filter CDs by a given attribute and value. It needs the auxiliary function illustrated in Listing 10.7 to compare two values uniformly. It could be invoked by this call:

```
>> noRayCharles = filterCDs(collection, 'artist', 'Charles, Ray');
```

Listing 10.6 Filtering a CD collection

```

1. function newCDs = filterCDs( CDs, attrib, value )
2. % newCDs = filterCDs( CDs, attrib, value )
3. % get rid of all CDs with the attribute
4. %           equaling the given value
5. index = 1;
6. for CD = CDs
7.     if ~valueEq( CD.(attrib), value )
8.         newCDs(index) = CD;
9.         index = index + 1;
10.    end
11. end
12. if index == 1
13.    newCDs = [];
14. end

```

In Listing 10.6:

Line 1: The good news is that this function will filter by removing any item that matches the given value of any attribute.

Line 5: This is a different technique for saving the new collection by keeping a separate index that is incremented when a new item is saved.

Line 7: The bad news is that generality brings a little complexity—we need a helper function to test for equality between objects of various types.

Lines 8 and 9: This is the process of saving an item to the new collection.

Lines 12–14: Protect us from the case where we have removed all the items, keeping none. Without this, MATLAB will throw an exception because `newCDs` is not defined until the first item is saved.

In Listing 10.7:

Line 1: A and B could be any data type. We address only the most common here. All common data types respond correctly to the normal equality test, except for strings.

Lines 4 and 5: Take care of string comparison.

10.3.7 Summarizing a Collection

Folding is the name given to summarizing a collection. It is a special case of traversal where all of the items in the collection are summarized as a single result. The collection is not altered in size or values by the operation. Totaling, averaging, or finding the largest element in a vector are typical examples of folding. For example, having mapped the CD collection to add a value field, we might want to find the CD with the best value. Figure 10.6 and Template 10.6 show the basic algorithm for folding a collection. The general form of a fold should be to initialize the summary value and then traverse the whole collection, updating the summary.

Listing 10.7 Helper function with a generic quality test

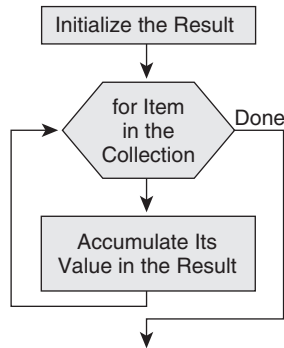
```
1. function ans = valueEq( A, B )
2. % are A and B equal, regardless of data type?
3. switch class(A)
4.     case 'char'
5.         ans = strcmp(A, B);
6.     otherwise
7.         ans = (A == B);
8. end
```

Template 10.6 Template for folding

```

<initialize the summary value>
for item <across the whole collection>
  <extract the item>
  <update the summary value>
<end for>
<finalize the summary value>

```

**Figure 10.6** *Folding a collection*

The function shown in Listing 10.8 finds the CD with the highest value in a collection. Note that we carefully avoided generality here by not letting the user specify the attribute. Had we permitted this, we would need another type-independent helper function to determine which of two values was greater.

When the summary involves calculating the maximum or minimum of something, it can be difficult to choose the right starting value. This function illustrates a tidy solution to this dilemma: picking the value of the first item, then comparing that to all the rest. The function would be called by the following:

```
>> bestCD = bestValue(collection);
```

Listing 10.8 Finding the best CD

```

1. function theCD = bestValue(CDs)
2. % find the CD with best value
3. theCD = CDs(1);
4. bestVal = theCD.value;
5. for index = 2:length(CDs)
6.     CD = CDs(index);
7.     if CD.value > bestVal
8.         theCD = CD;
9.         bestVal = CD.value;
10.    end
11. end

```

In Listing 10.8:

Lines 3 and 4: Set the first CD as the result.

Lines 6–10: Extract and test each item.

10.3.8 Searching a Collection

Searching is the process of traversing the collection and applying a specified test to each element in turn, terminating the process as soon as the test is satisfied. This is superficially similar to filtering, except that it is not necessary to touch all the elements of the collection; the search stops as soon as one element of the collection matches the search criteria. If the criteria are extremely complex, it is sometimes advisable to perform a mapping or folding before the search is performed. Figure 10.7 and Template 10.7 show one way to implement searching a collection using a `for` loop with a `break` exit. There are always two exit criteria from a search—finding what you seek or failing to find it. It can also be implemented with a `while` loop, but the multiple exit criteria make the code generally more complex.

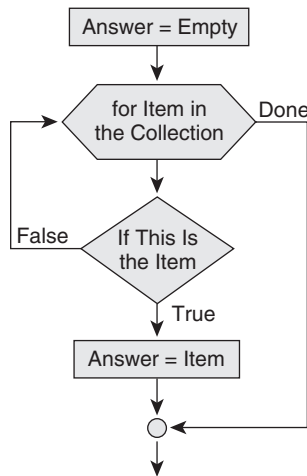


Figure 10.7 Searching a collection

Template 10.7 Template for searching

```

<initialize result to not succeeded>
for <item in the collection>
  if <found criteria>
    <set result to succeeded>
    <break the loop>
  <end if>
<end for>
<check for failure>

```

Listing 10.9 Searching for a specific CD

```
1. function theCD = findCD(CDs, attrib, value)
2. % find the specific CD
3. index = 1;
4. found = false;
5. while (index <= length(CDs)) && ~found
6.     CD = CDs(index);
7.     if valueEq(CD.(attrib), value)
8.         found = true;
9.         theCD = CD;
10.    else
11.        index = index + 1;
12.    end
13. end
14. if ~found
15.     theCD = [];
16. end
```

Listing 10.9 illustrates the search for a particular CD, using the helper function created to test for equality. It could be invoked by the following:

```
>> joshCD = findCD(addedCDs, 'artist', 'Groban, Josh')
```

In Listing 10.9:

Lines 3 and 4: Initialize terminating conditions.

Line 5: Notice the use of a `while` loop because we intend to terminate the search at success. There are two possible exits from the `while` loop—when the index passes the end of the collection, indicating failure, or when the CD has been found.

Line 6: Extracts the item.

Line 7: Checks for equality.

Line 11: Moves through the collection.

Lines 14 and 15: If we should exit the `while` loop without finding the CD, we need to return something to the user—null, the empty vector, is usually a good choice. The user can check for this by calling `isempty(...)`.

10.3.9 Sorting a Collection

Sorting involves reordering the elements in a collection according to a specified ranking function that defines which item “comes before” another. Sorting is computationally expensive. However, if a large collection of data is stable—items are added or removed infrequently—but is frequently searched for specific items, keeping the data sorted can greatly improve the efficiency of the searches. Chapter 16 is devoted to the details of sorting, but the concept is included here to complete the list of operations we can perform on a collection.

10.4 Solving Larger Problems

Problem statements are rarely simple enough to be able to seize one of the above steps and solve the whole problem. Usually, the solution involves choosing a number of known operations and performing those operations in order to solve the complete problem. Solution steps are combined in one of two ways—in sequence or nested. When considering the overall strategy for solving a problem, one might identify steps A and B as contributing to the solution. Your logical statement might say either “do A and then B” sequential steps—or “for each part of A, do B”—nested steps.

For example, consider the baseball card problem originally proposed in Chapter 1. You have collected over the years a huge number of baseball cards and you wish to find the names of the 10 “qualified” players with the highest lifetime batting average. To qualify, the players must have been in the league at least five years, had at least 100 plate appearances per year, and made less than 10 errors per year.

You have built a structure array containing the relevant information on the cards for each player, and we need to operate on this collection to solve the problem. Consider again the overall problem situation, as shown in Figure 10.8. The original data is a structure array containing all the player data. The final result is a list of 10 names of the qualified players with the highest batting averages. There may be more than one sequence of operations to solve this problem; some may be more efficient than others.

First, consider the operations that could be performed on the original data. Since the end result is a collection, it is unlikely that the first step would reduce the collection to one answer. This eliminates folding and searching. Since the collection is already built, we do not need to insert or build, leaving four possible operations to consider—traversal, mapping, filtering, and sorting (see Figure 10.9).

Now, consider the last operation—it seems reasonable that the last thing to do is a mapping—taking the 10 selected structures and extracting the names. Figure 10.10 shows this step. Here, we think about how to find these 10 structures. If we had a collection of qualified players sorted by their batting

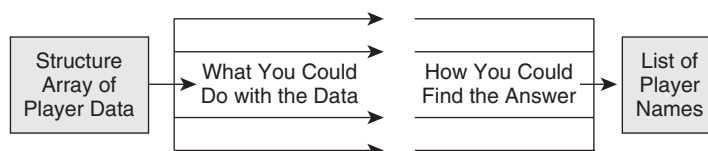


Figure 10.8 Generalized problem solving

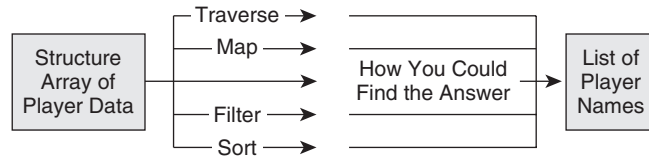


Figure 10.9 First step

average, we could accomplish this with a special filter taking the first 10 from these sorted, qualified players, as shown in Figure 10.11. Backing up one more step, we can see that the sorted collection we need is just a sort of the qualified players, as shown in Figure 10.12. Figure 10.13 shows the connection from the front to the back.

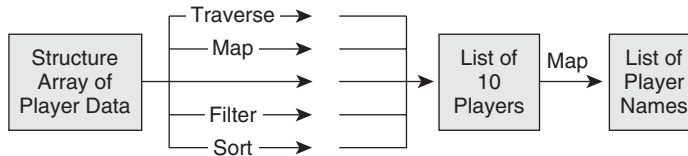


Figure 10.10 Step N

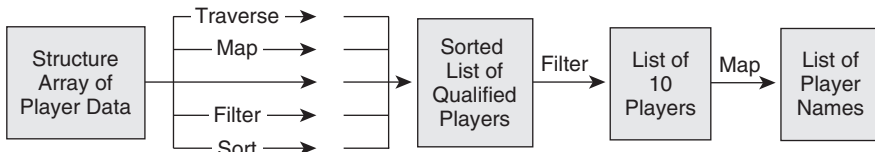


Figure 10.11 Step N-1

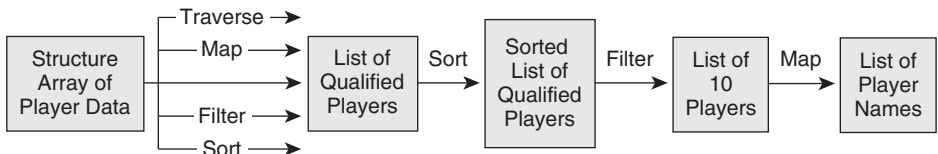


Figure 10.12 Step N-2

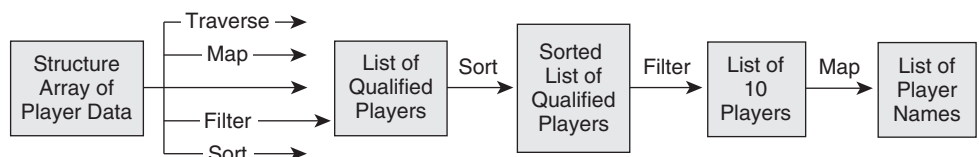


Figure 10.13 Connecting the dots



10.5 Engineering Example—Processing Geopolitical Data

Imagine that the U.S. government has still not passed the Fair Tax Act and you have decided to move your prosperous business overseas to the country with the most business-friendly environment. After considerable study, you decide that the best measure of friendliness would be to compute the rate of growth of the gross domestic product for candidate countries, subtract their rate of population growth, and use this measure to choose the best country. An Internet search provides an interesting source of data. Figure 10.14 shows an excerpt from a spreadsheet containing historical data for 154 countries from Penn World Table Version 6.1¹. The data columns of interest to us contain the following information:

- Country—Country name
- Code—Country code
- Year—Year in which the data in this row was recorded
- POP—Population that year
- XRAT—Exchange rate versus U.S. currency that year
- PPP—Purchasing power parity over GDP that year
- Cpdp—Real gross domestic product per capita that year

Figure 10.14 also illustrates one of the weaknesses of spreadsheets: they are inherently two dimensional, and the data in this case is three dimensional; each country has several sets of data as functions of the year when the information was recorded. Therefore, the data must be massaged into a form more useful to us. A careful examination of the data also reveals the following challenges:

- The years in which the data was available vary from country to country—most have data from 1950 to 2000
- There are some places within the numerical data where the values are not available, signified by the letters “na” at those locations

Our algorithm must take into account the variable number of years and the potential presence of strings within the data. Fortunately, the `xlswread(...)` function discussed earlier recognizes this situation and inserts `NaN` in the numerical data fields. To ensure clarity and reliability in our solution, we need a careful design for this data processing task as follows.

- Looking at the end result desired, eventually we need to fold a collection of data about each country, and choose the friendliest one.
- The information describing each country must include not only its name, but also vectors of the population and CGDP as a function of the year. It seems that a structure array by country would be an appropriate form for the data.

¹ Credit: Alan Heston, Robert Summers and Bettina Aten, Penn World Table Version 6.1, Center for International Comparisons at the University of Pennsylvania (CICUP), October 2002.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	Country	Code	Year	POP	XRAT	PPP	cgdp	cc	ci	cg	P	pc	pg	pi	openc	cgnp
2	Angola	AGO	1960	4816.00	0.03	0.01	542.68	76.75	8.84	9.45	17.51	13.07	24.03	49.11	36.98	na
3	Angola	AGO	1961	4884.19	0.03	0.00	564.37	74.23	7.92	9.85	17.36	13.18	23.65	48.67	35.23	na
4	Angola	AGO	1962	4955.35	0.03	0.00	573.94	75.48	6.76	10.55	17.28	13.41	23.65	50.44	38.79	na
5	Angola	AGO	1963	5028.69	0.03	0.01	593.72	73.68	5.72	13.56	17.73	13.90	24.04	52.06	38.69	na
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
36	Angola	AGO	1994	10627.18	59.51	53.32	1095.94	34.66	9.09	46.75	89.59	70.34	76.84	228.63	160.87	47.86
37	Angola	AGO	1995	10972.00	2750.23	1007.53	1244.73	41.86	9.43	57.65	36.63	29.58	31.91	96.85	146.58	48.19
38	Angola	AGO	1996	11316.94	128029.20	54873.28	1362.32	37.17	8.57	56.75	42.86	34.35	37.77	113.51	134.87	52.97
39	Angola	AGO	1997	na	na	na	na	na	na	na	na	na	na	na	na	59.09
40	Angola	AGO	1998	na	na	na	na	na	na	na	na	na	na	na	na	50.08
41	Angola	AGO	1999	na	na	na	na	na	na	na	na	na	na	na	na	44.52
42	Angola	AGO	2000	na	na	na	na	na	na	na	na	na	na	na	na	53.81
43	Albania	ALB	1991	3277.00	15.63	3.13	1605.36	82.81	6.92	36.66	20.04	23.78	12.02	17.70	54.89	98.10
44	Albania	ALB	1992	3225.00	75.03	10.53	1566.99	136.94	5.11	35.08	14.03	15.59	7.88	14.42	108.94	95.94
45	Albania	ALB	1993	3179.00	102.06	19.33	2031.94	109.39	12.54	25.73	18.94	20.80	10.51	20.01	77.14	99.02
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
5795	Zambia	ZMB	1998	9665.71	1862.07	744.91	800.69	85.12	13.75	14.14	40.00	39.54	33.22	49.87	68.86	93.36
5796	Zambia	ZMB	1999	9881.21	2388.02	941.87	765.24	91.82	15.30	12.54	39.44	39.02	31.89	48.14	66.55	94.97
5797	Zambia	ZMB	2000	10089.00	3110.84	1157.63	840.97	86.33	15.38	12.34	37.21	37.70	29.54	40.65	70.45	95.88
5798	Zimbabwe	ZWE	1954	3011.69	0.71	0.37	400.19	66.89	41.48	4.03	50.97	60.59	135.99	29.92	77.30	na
5799	Zimbabwe	ZWE	1955	3127.52	0.71	0.36	429.04	65.87	50.95	3.47	50.40	60.80	136.52	31.10	78.43	na
5800	Zimbabwe	ZWE	1956	3264.42	0.71	0.36	471.08	63.51	54.77	3.53	50.08	62.11	136.86	30.53	74.27	na
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
5842	Zimbabwe	ZWE	1998	12153.85	23.68	4.06	2799.85	77.66	10.75	13.39	17.16	14.97	22.03	26.87	91.96	93.25
5843	Zimbabwe	ZWE	1999	12388.32	38.30	6.12	2770.48	76.89	10.73	12.81	15.98	14.35	19.01	24.02	92.99	93.75
5844	Zimbabwe	ZWE	2000	12627.00	44.42	9.48	2607.03	69.23	8.62	22.44	21.33	19.26	23.63	31.96	62.61	96.62

Figure 10.14 Spreadsheet samples

- Therefore, before actually solving the problem, we have to build this structure.
- Having built the structure, the folding operation to find the friendliest country follows the folding template shown in Section 10.3.7.

Listing 10.10 shows the script that accomplishes this analysis, although most of the work is actually done in the following functions.

Listing 10.10 Country analysis

```
% build the country array

1. worldData = buildData('World_data.xls');

2. best = findBest(worldData);

3. fprintf('best country is %s\n', ...
           worldData(best).name)
```

Line 1: `worldData` will be a structure array containing the relevant data from the spreadsheet.

Line 2: `best` will be the index of the friendliest country according to the criteria defined in the function `findBest(...)`.

Line 3: Here we can look up and print the name of the best country.

Listing 10.11 lists the function that builds the country data. The algorithm violates the best style by taking advantage of the logical ordering of the data in the spreadsheet to traverse the data from the spreadsheets simultaneously, filter out the data for each country in turn, and then map the available data for that country into the emerging structure array.

In Listing 10.11:

Line 2: Reads the Excel spreadsheet—we need the numerical data and the text part for the names of the countries.

Listing 10.11 Building the country data

```

1. function worldData = buildData(name)

% read the spreadsheet into a data array
%           and a text cell array
2. [data txt] = xlsread(name);
3. country = ' '; % force the first data row
%           % to change the country
4. cntry_index = 0;
% Traverse the data and cell arrays producing
%   an array of structures,
%   one for each country

5. for row = 1:length(data)
% Because the text data in txt contains
% the header row of the spreadsheet,
% the data at a given row belongs to the country
% whose name is at txt{row+1}.
% if the country name changes,
% begin a new structure.
6.   if ~strcmp(txt{row+1}, country)
7.       col = 1;
8.       country = txt{row+1};
9.       cntry_index = cntry_index + 1;
10.      cntry.year = 1;
11.      cntry.pop = 1;
12.      cntry.gdp = 1;
13.   end
14.   cntry.name = country;
15.   cntry.year(col) = data(row, 1);
16.   cntry.pop(col) = data(row, 2);
17.   cntry.gdp(col) = data(row, 5);
18.   col = col + 1;
19.   worldData(cntry_index) = cntry;
20. end

```

Lines 3 and 4: Initialize the results of the traversal, setting an unknown country name and the initial country count.

Line 5: Traverses the rows of the numerical data.

Line 6: Since the numerical data skipped the header row, the name of the country corresponding to each row of data is in the text file at `row+1`. When the country changes, we step to the next country index, reset the year counter, `col`, for that country, and empty the structure used to accumulate the country data.

Line 7: Resets the counter that indexes the year storage for the current country.

Line 8: Saves the name of the new country to continue retrieving its data.

Line 9: Increases the country count.

Lines 10–12: Reset the structure used to store the vectors of data. This is crucial because the number of annual data items for all countries is not the same.

Lines 14–17: Add this row of data to the structure. Column 1 is the year, column 2 is the population, and column 5 is the CGDP.

Line 18: Moves to the next year.

Line 19: Saves all this in the structure array.

Listing 10.12 shows the function that finds the best country by folding the country structure array, together with the two supporting functions that provide the comparison criteria. Notice that the complexity of the data has forced the solution into nested folds: to fold the country data array, we have to summarize (fold) the annual data for each country.

In Listing 10.12:

Lines 2 and 3: As with any folding function that is looking for the maximum or minimum of a collection, the best place to start is the first item in the collection. The remaining items can then be compared to this one.

Listing 10.12 Folding the country data

```
1. function besti = findbest(worldData)
% find the index of the best country
% according to the criterion in the function
% fold
2. best = fold(worldData(1));
3. besti = 1;
4. for ndx = 2:length(worldData)
5.     cntry = worldData(ndx);
6.     tryThis = fold(cntry);
7.     if tryThis > best
8.         best = tryThis;
9.         besti = ndx
10.    end
11. end
```

continued on next page


```

12. function ans = fold(st)
% s1 is the rate of growth of population
13. pop = st.pop(~isnan(st.pop));
14. yr = st.year(~isnan(st.pop));
15. s1 = slope(yr, pop)/mean(pop);
% s2 is the rate of growth of the GDP
16. gdp = st.gdp(~isnan(st.gdp));
17. yr = st.year(~isnan(st.gdp));
18. s2 = slope(yr, gdp)/mean(gdp);
% Measure of merit is how much faster
%   the gdp grows than the population
19. ans = s2 - s1;

20. function s1 = slope(x, y)
% Estimate the slope of a curve
21. if length(x) == 0 || x(end) == x(1)
22.     error('bad data')
23. else
24.     s1 = (y(end) - y(1))/(x(end) - x(1));
25. end

```

Line 4: Loops through the remaining countries in the array.

Line 5: Extracts one structure.

Line 6: Computes its friendliness value.

Lines 7–10: If the result is improved, these lines update the stored values. The index `best_i` is returned when the loop finishes.

Line 12: This function computes the measure of friendliness for each country. The goal is to subtract the rate of population growth from the rate of growth of the GDP. So first we compute the rate of population growth.

Lines 13 and 14: These lines establish two local vectors containing the population value and the corresponding year without the values that are `NaN`, the places where "na" appears in the spreadsheet.

Line 15: Calls the helper function for the slope of this relationship, and non-dimensionalizes the result by dividing by the mean population.

Lines 16–18: Repeat the same logic for the non-dimensional rate of increase of the GDP.

Line 19: Returns the difference in growth rates.

Line 20: The function that estimates the rates of growth.

Lines 21 and 22: We have a problem if there is no data or if the value we will subsequently use as a divisor is zero.

Line 24: A very crude measure of the slope is to divide the difference between the first and last data points by the difference between the first and last `x` values. (We will be able to improve on this approach later.)

When we run this program, we see the following result:

```
>> best country is Equatorial Guinea
```

This may not be exactly the result we were hoping for. In Chapter 16 we will revisit this example with some better tools that will allow us to apply additional criteria to the best countries.



Chapter Summary

This chapter presented the fundamental operations that can be applied to problem solving:

- Using normal arithmetic operations with specific input and output values
- Inserting new elements in a collection
- Traversing a collection
- Building a collection by repetitive insertion
- Mapping a collection—changing the values of the data items in the collection, but not the number of them
- Filtering a collection—reducing the number of entries, but not changing the data contents of the collection
- Folding—summarizing the values in a collection into a single quantity
- Searching for a specific match in a collection
- Sorting a collection

Then we briefly discussed how to combine these fundamental tools to solve more complex data manipulation problems.



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. Copying the elements of a structure array into a cell array is a combination of traversal and insertion.
2. If you map a collection, you must change at least one of its elements.
3. When you filter a collection, at least one data element is changed.

4. The function `max(...)` is not folding because it returns two values.
5. You can use a `for` loop to search a collection even if you need to stop the search when you find the answer.
6. Sorting must involve putting the items in a collection in numerical order (ascending or descending).

Fill in the Blanks

1. The problem-solving style recommended in this text is to identify the _____ and the _____.
2. Building is the process of _____ and _____.
3. Mapping may involve combining _____ of the same length.
4. We _____ vectors by applying built-in logical operations and then indexing with the results to produce new, shorter arrays.
5. Totaling, averaging, or finding the smallest element in a vector are typical examples of _____.
6. There are almost always two exit criteria for a search: _____ or _____.
7. To save a collection to a text file, you _____ the collection _____ to the file.



Programming Projects

1. The purpose of this problem is to write a set of functions that calculate the volume of a slant cylinder with an irregular pentagonal cross section shown in Figure 1.

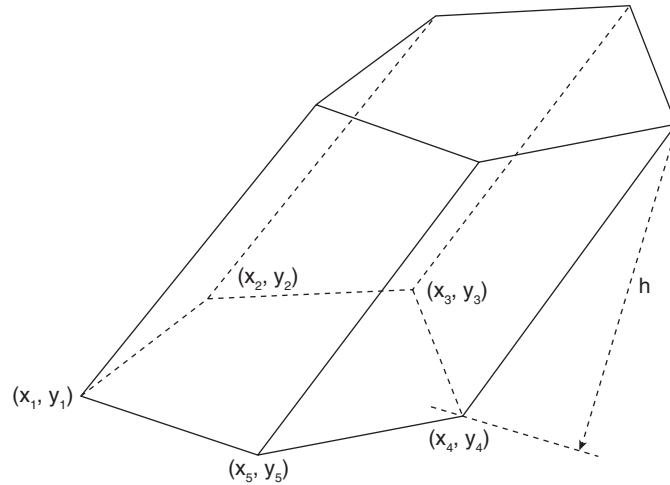


Figure 1 *The Slant Cylinder*

You will be given two vectors, x and y , containing the coordinates of the corners of the pentagon, and the value h , the vertical height of the cylinder. We will need to break this problem apart, writing functions to solve each part:

- a. The volume of the cylinder is the area of the pentagon multiplied by the vertical height; write a function `polyvol(x, y, h)` to solve this.
- b. The area of the pentagon is the sum of the areas of three triangles shown in Figure 2. So we need to write a function `pent_area(x, y)` that asks for the area of the three triangles and adds them together.
- c. Given the coordinates of the corners of a triangle, we need a function `tri_area(x, y)` to calculate the area of the triangle—see Figure 3. To compute the area of the triangle, we need the values of a , b , and c . So if we had the lengths of the lines, the area of the triangle is given by Heron's formula:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

where s is half the sum of a , b and c

- d. So we need a function `tri_side(x, y)` that computes the length of a line when given its end points.

Then, we can put the pieces back together by calling the functions with the right parameters, and then build and test `polyvol` using the test cases provided.

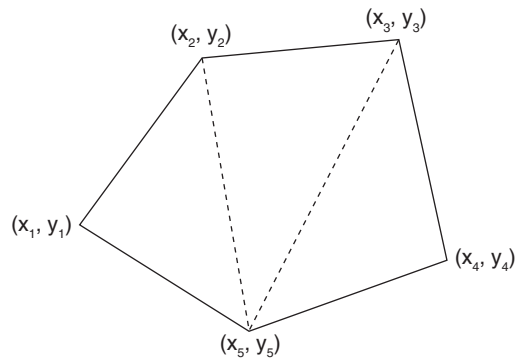


Figure 2 Break down the pentagon

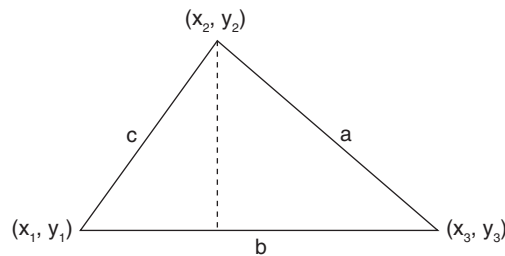


Figure 3 Area of a Triangle

2. This problem is about processing structure arrays. Write a function named `structSort` that sorts a structure array based on a given field that contains numerical values. Your function should take in a structure array and a string that should correspond to one of the fields of the structure array and return the original structure array sorted on the given field. It should check to be sure that the specified field name is in fact one of the fields of the structure array, and call the `error(...)` function if it is not.

Test your function by using the `buildCDs` script from Chapter 7, using the input function to specify the sorting field.

Plotting

Chapter Objectives

This chapter presents the principles and practice of plotting in the following forms:

- Basic two-dimensional line plots
- Two-dimensional parametric plots
- Three-dimensional line and parametric plots
- Basic three-dimensional surface plots
- Parametric surface plots
- Bodies of rotation

There is a much-quoted expression that “a picture is worth a thousand words,” and this is never more appropriate than when talking about data. In previous chapters we used some simple plot commands to display data to illustrate its behavior. The ability of MATLAB to present data reaches far beyond ordinary data plotting, and far beyond the limited confines of a textbook. This chapter will present the fundamental concepts of the different forms in which data can be presented, but it leaves to the reader the challenge of exploring the full range of capabilities available. You only really discover the power of MATLAB’s plotting capabilities when you have some unusual data to visualize.

- 11.1 Plotting in General**
 - 11.1.1 A Figure—The Plot Container
 - 11.1.2 Simple Functions for Enhancing Plots
 - 11.1.3 Multiple Plots on One Figure—Subplots
 - 11.1.4 Manually Editing Plots
- 11.2 2-D Plotting**
 - 11.2.1 Simple Plots
 - 11.2.2 Plot Options
 - 11.2.3 Parametric Plots
 - 11.2.4 Other 2-D Plot Capabilities
- 11.3 3-D Plotting**
 - 11.3.1 Linear 3-D Plots
 - 11.3.2 Linear Parametric 3-D Plots
 - 11.3.3 Other 3-D Plot Capabilities
- 11.4 Surface Plots**
 - 11.4.1 Basic Capabilities
 - 11.4.2 Simple Exercises
 - 11.4.3 3-D Parametric Surfaces
 - 11.4.4 Bodies of Rotation
 - 11.4.5 Other 3-D Surface Plot Capabilities
 - 11.4.6 Assembling Compound Surfaces
- 11.5 Interacting with Plotted Data**
 - 11.5.1 Brushing Data Items
 - 11.5.2 Linking a Plot to the Source Data
 - 11.5.3 Further Information
- 11.6 Engineering Example—Visualizing Geographic Data**
 - 11.6.1 Analyzing the Data
 - 11.6.2 Displaying the Data



11.1 Plotting in General

Before considering the details of how each plotting mode works, we should set the context. In this section we will discuss the general container for all graphical types, the figure, and some basic operations that apply to all figures—functions that enhance them, the ability to assemble subplots into a single figure, and the advisability of making manual changes to plots.

11.1.1 A Figure—The Plot Container

The fundamental container for plotting is a MATLAB figure. In a simple script, if you just start plotting data, *figure number 1* is automatically generated to present the data. You can manage the figures by calling the `figure` function. Each time `figure` is called, a new figure is made available, with the next higher figure number. The `figure` function actually returns a reference called a handle with which, in advanced plotting circumstances, you can change the attributes of the figure.

To clear the current figure, put the key word `clf` in the header of your script. To remove all the figures, put the key phrase `close all` at the beginning of your script.

11.1.2 Simple Functions for Enhancing Plots

We have already introduced `plot(x, y)`, the basic function that creates a simple plot of x versus y . The following functions can be used to enhance any of the plots discussed in this chapter. Note that they enhance an existing plot; they should all be called after the fundamental function that creates a plot figure.

- `axis <param>` provides a rich set of tools for managing the appearance of the axes including the following:
 - `tight` reduces the axes to their smallest possible size
 - `equal` sets the x and y scales to the same value
 - `square` makes the plot figure of equal width and height
 - `off` does not show the axes at all
- `axis([x1 xu y1 yu z1 zu])` overrides the automatic computation of the axis values, forcing the x -axis to reach from $x1$ to xu , the y -axis from $y1$ to yu , and the z -axis from $z1$ to zu . For 2-D plots, the z values should be omitted.
- `colormap <specification>` establishes a sequence of colors, the color map, to be used under a number of circumstances to cycle through a series of colors automatically. The legal specification values are listed in Appendix A.

- `grid on` puts a grid on the plot; `grid off` (the default) removes grid lines.
- `hold on` holds the existing data on the figure to allow subsequent plotting calls to be added to the current figure without first erasing the existing plot; `hold off` (the default) redraws the current figure, erasing the previous contents.
- `legend(...)` takes a cell array of strings, one for each of the multiple plots on a single figure, and creates a legend box. By default, that box appears in the top-right corner of the figure. However, this default can be overridden by explicitly specifying the location of the legend. See the MATLAB help files for a complete discussion of the legend options.
- `shading <spec>` defines the method for shading surfaces. See the MATLAB help files for a complete discussion of the shading specification options.
- `text(x, y, {z,}, str)` places the text provided at the specified (x, y) location on a 2-D plot, or at the (x, y, z) location on a 3-D plot.
- `title(...)` places the text provided as the title of the current plot.
- `view(az, el)` sets the angle from which to view a plot. The parameters are `az`, the azimuth, an angle measured in the horizontal plane, and `el`, the elevation, an angle measured upward from the horizontal. Both angles are measured in degrees.
- `xlabel(...)` sets the string provided as the label for the x-axis.
- `ylabel(...)` sets the string provided as the label for the y-axis.
- `zlabel(...)` sets the string provided as the label for the z-axis. (As we will see, all plots actually have a third axis.)

11.1.3 Multiple Plots on One Figure—Subplots

Within the current figure, you can place multiple plots with the `subplot` command, as shown in Figure 11.1.

The function `subplot(r, c, n)` divides the current figure into `r` rows and `c` columns of equally spaced plot areas, and then establishes the `n`th of these (counting across the rows first) as the current figure. You do not have to draw in all of the areas you specify. Figure 11.1 was generated by the code shown in Listing 11.1.

Listing 11.1 Creating a subplot

```
1. clf
2. x = -2*pi:.05:2*pi;
3. subplot(2,3,1)
4. plot(x, sin(x))
5. title('1 - sin(x)');
```

continued on next page


```
6. subplot(2,3,2)
7. plot(x, cos(x))
8. title('2 - cos(x)');

9. subplot(2,3,3)
10. plot(x, tan(x))
11. title('3 - tan(x)');

12. subplot(2,3,4)
13. plot(x, x.^2)
14. title('4 - x^2');

15. subplot(2,3,5)
16. plot(x, sqrt(x))
17. title('5 - sqrt(x)');

18. subplot(2,3,6)
19. plot(x, exp(x))
20. title('4 - e^x');
```

In Listing 11.1:

Line 1: `clf` clears the parameters of the current figure.

Line 2: Specifies a suitable range of x values.

Line 3: Sets the first subplot region.

Line 4: This is the simple version of the `plot(...)` function introduced earlier, plotting x against y and automatically creating the axes, creating subplot 1, the plot in the top-left corner. Note that although in the figure seen here the line is gray, when you run the script, the line will appear in its default color, blue.

Line 5: The `title(...)` function puts the specified string at the top of the plot as its title.

Lines 6–8: Create subplot 2, the second plot on the first row.

Lines 9–11: Create subplot 3, the first plot on the second row.

Lines 12–14: Create subplot 4, the second plot on the second row.

Lines 15–17: Create subplot 5, the first plot on the bottom row.

Lines 18–20: Create subplot 6, the second plot on the bottom row.

Style Points

All of these capabilities are also available to the script that creates the plots, and you are very likely to want to generate a plot more than once. Therefore, it is unwise to put a significant amount of manual effort into adjusting a plot. It is better to experiment with the manual adjustments and then find out how to make the same adjustments in the script that creates the plots. This also leaves you a permanent record of how the plot was generated.

11.1.4 Manually Editing Plots

When a figure has been created, you are free to manipulate many of its characteristics by using its menu items and tool bars. They provide the ability to resize the plot, change the view characteristics, and annotate it with legends, axis labels, lines, and text callouts.

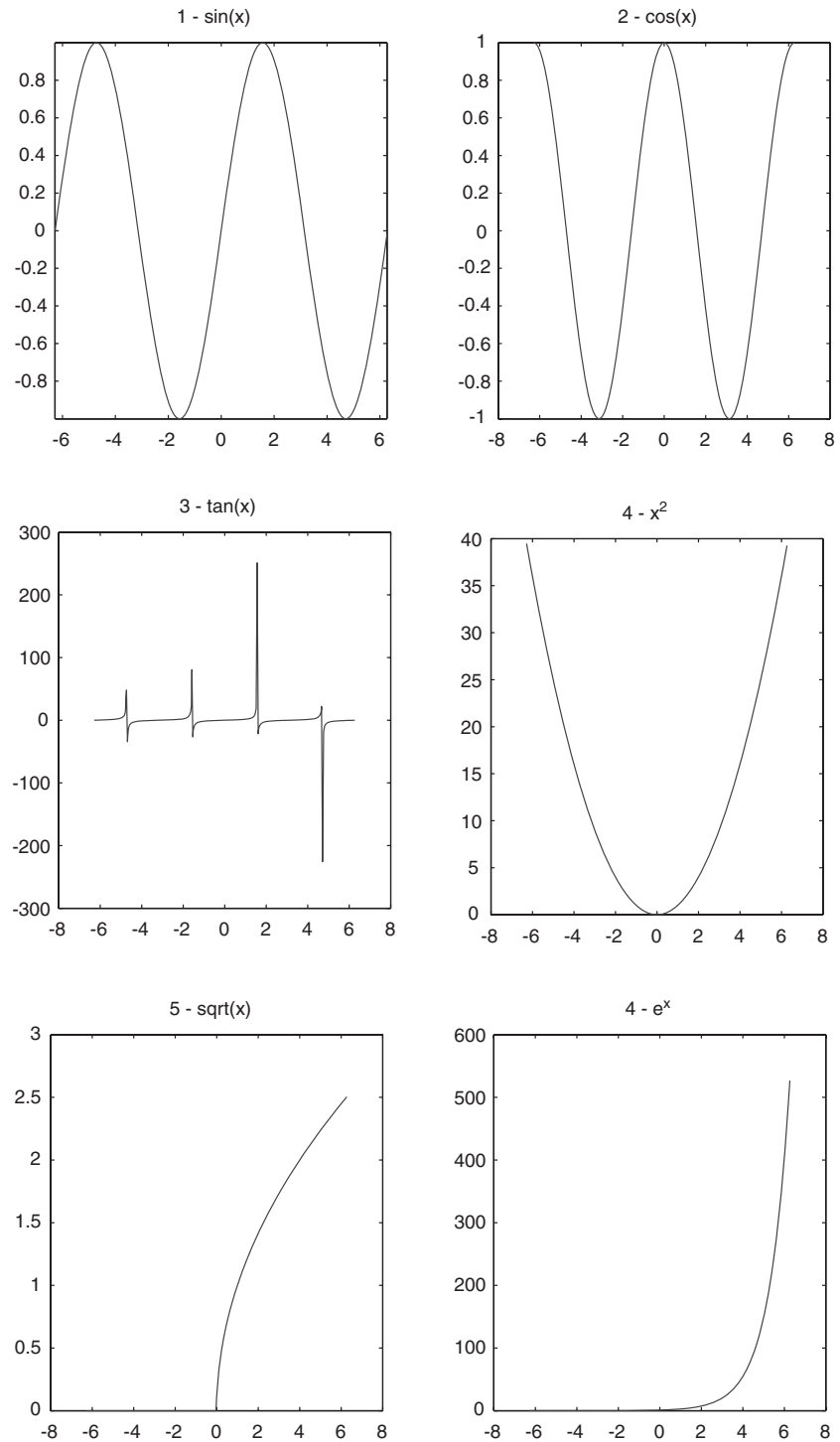


Figure 11.1 Plotting effects

11.2 2-D Plotting

11.2.1 Simple Plots

The basic function to use for 2-D plots is `plot(...)`. The usual way to use this function is to give it three parameters, `plot(x, y, str)`, where `x` and `y` are vectors of the same length containing the `x` and `y` coordinates, respectively, and `str` is a string containing one or more optional line color and style control characters. A complete list of these control characters is included in Appendix A. If the vector `x` is omitted, MATLAB assumes that the `x` coordinates are `1:N`, where `N` is the length of the `y` vector. If the `str` is omitted, the default line is solid blue. The function also permits multiple `(x, y, str)` data sets in a single function call. In this case, if the line style is not specified, the default line colors rotate through the active color map.

Since we have already seen basic 2-D plotting at work, it should be sufficient to observe and comment on the simple example seen in Figure 11.2, generated by the code shown in Listing 11.2.

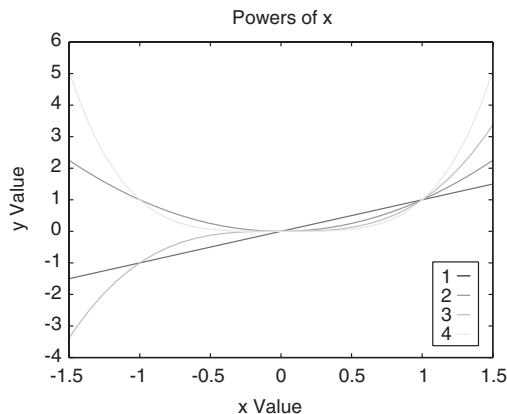


Figure 11.2 Powers of x

Listing 11.2 Simple 2-D plots

```
1. clear
2. clc
3. close all
4. x = linspace(-1.5, 1.5, 30);
5. y1 = x;
6. y2 = x.^2;
7. y3 = x.^3;
8. y4 = x.^4;
9. plot(x,y1,x,y2,x,y3,x,y4)
10. xlabel('x')
11. ylabel('value')
12. title('powers of x')
13. legend({'1', '2', '3', '4'}, ...
14. 'Location','SouthEast')
```

In Listing 11.2:

Line 3: Clears all previous plot results.

Lines 4–8: Create four curves to plot.

Line 9: Plots all four curves in one call. You might be wondering why we use this ponderous form of the plot function, where all the curves are generated with one call. It would indeed have been much simpler to iterate across the powers of x , and overlay the plots one at a time, using `hold on` to allow multiple plots to be overlaid. However, with individual plot calls, the colors of each line must then be set manually. Plotting in this form automatically cycles the colors and generates the data for the legend.

Lines 10–13: Add enhancements to the plot as noted above.

Line 14: One of many possible parameters to the `legend(...)` function—this one forces its location to the lower-right corner of the figure, out of the way of the data.

11.2.2 Plot Options

Figure 11.3 illustrates some of the options available to enhance 2-D plots. This figure was generated using the code shown in Listing 11.3. Study the code and the comments that follow.

Listing 11.3 Demonstrating plot options

```
1. clear
2. clc
3. close all

4. x = linspace(0, 2*pi);
5. subplot(2, 3, 1)
6. plot(x, sin(x))
7. axis([0 2*pi -0.5 0.5])
8. title('Changing Data Range on an Axis')

9. subplot(2, 3, 2)
10. plot(x, sin(x))
11. hold on
12. plot(x, cos(x))
13. axis tight
14. title('Multiple Plots with hold on')

15. subplot(2, 3, 3)
16. plot(x, sin(x), '-')
17. hold on
18. plot(x, cos(x), 'r:')
19. axis tight
20. title('Multiple Plots with hold on')
```

continued on next page

```

21. subplot(2, 3, 4)
22. N = 16;
23. xr = linspace(0, 1, N);
24. yr = xr + rand(1,N) - 0.5;
25. plot(xr, yr, 'r+')
26. title('Symbols for Real Data')

27. subplot(2, 3, 5)
28. x = 0:0.01:20;
29. y1 = 20*exp(-0.05*x).*sin(x);
30. y2 = 0.8*exp(-0.5*x).*sin(3*x);
31. plotyy(x,y1,x,y2, 'plot');
32. xlabel('Zero to 20 \musec.')
33. title('plotting on the Second Axis')

34. subplot(2, 3, 6)
35. pow = linspace(0, 1);
36. w = 10.^pow;
37. fr = complex(0, w);
38. h = (fr.^2 + 0.1*fr + 7.5) ...
    ./ (fr.^4 + 0.12*fr.^3 + 9*fr.^2);
39. loglog(w, abs(h))
40. title('log - log plots')

```

In Listing 11.3:

Lines 4–8: The first plot using `axis(...)` to override the default axis limits.

Lines 9–14: Plot multiple curves on one figure using `hold on` and `axis tight` to minimize the white space.

Lines 15–20: Multiple plots distinguished by line style, using the third parameter on `plot(...)` to control the color and line style: `'--'` specifies a dashed line in the default color, blue; `'r:'` specifies a dotted line in red.

Lines 21–26: Simulated real data plotted with symbols but no line: `'r+'` specifies red plus signs for the symbols. Adding a line specification such as `'r+--'` or `'r+--'` would draw both the symbols at each data point and the lines, solid and dashed, respectively.

Lines 27–33: Plot on the same figure with two sets of data with widely differing data ranges. Depending on the exact version of MATLAB, the axis control and label functions are complex and may not work well. See the MATLAB help documentation for `plotyy(...)` for details.

Lines 34–40: To plot data with a wide range of values, you can use logarithmic scales on either axis or, as in this case, both. For logarithmic axes only in x or y, use `semilogx(...)` or `semilogy(...)`.

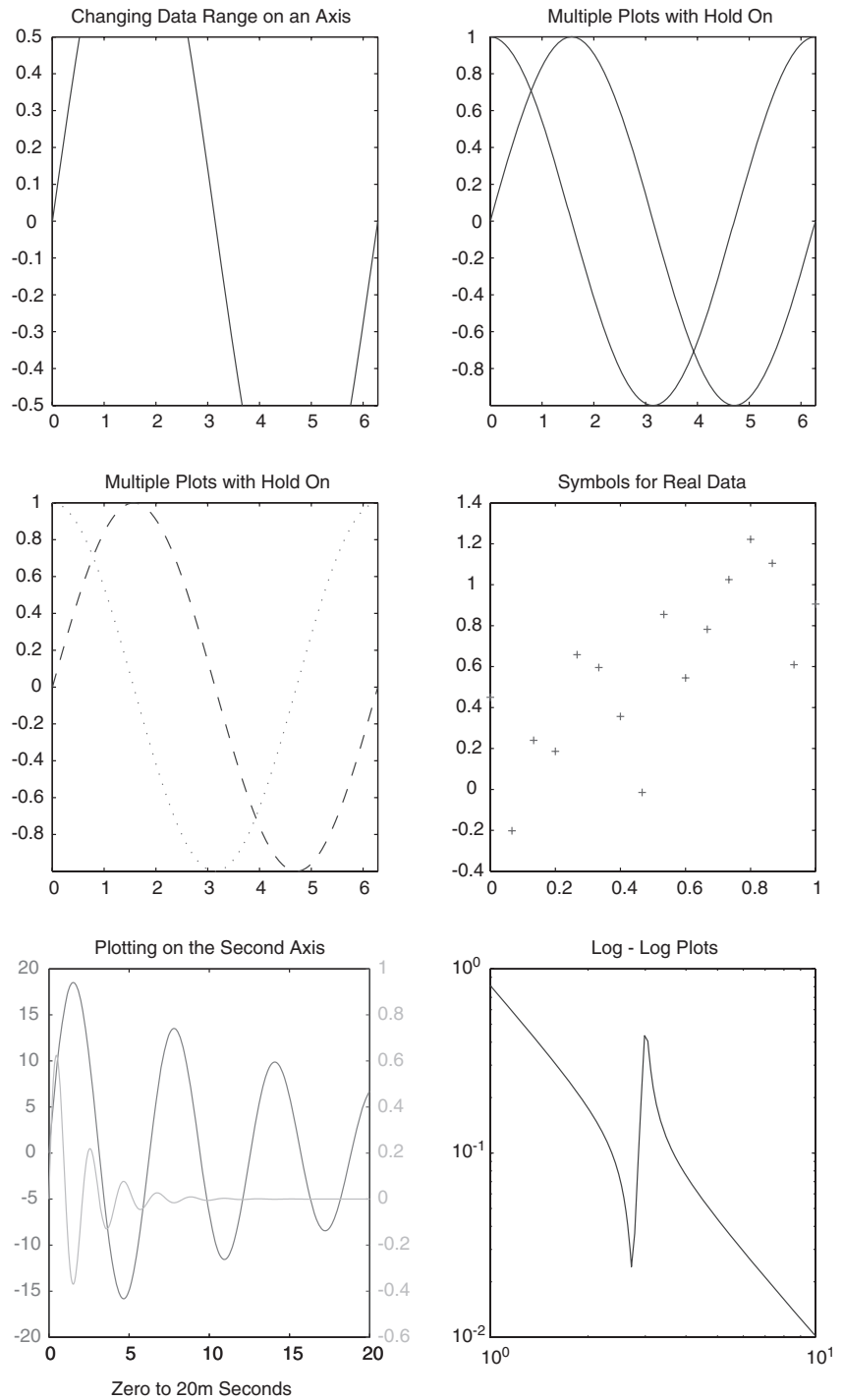


Figure 11.3 Examples of plotting styles

Style Points

By convention, good engineers are expected to represent the data with appropriate line styles to avoid misleading the reader. For example, if you have some raw data that is only valid at the measurement points, it should be plotted with symbols only. Connecting the data with a line would imply that the data has some interpolated values, which may not be the case. On the other hand, if you calculate a theoretical curve that is good throughout the range of x , it should be plotted as a continuous curve, perhaps even at a better resolution (more x values) than the raw data samples.

11.2.3 Parametric Plots

MATLAB plotting is not restricted to the situation where one axis is an independent variable and one is dependent. Parametric plots allow the variables on each axis to be dependent on a separate, independent variable. That independent variable will define a path on the plotting surface. Consider the plot shown in Figure 11.4, which presents a simple exercise in

transforming a specific circle into an airfoil. It was generated using the code shown in Listing 11.4.

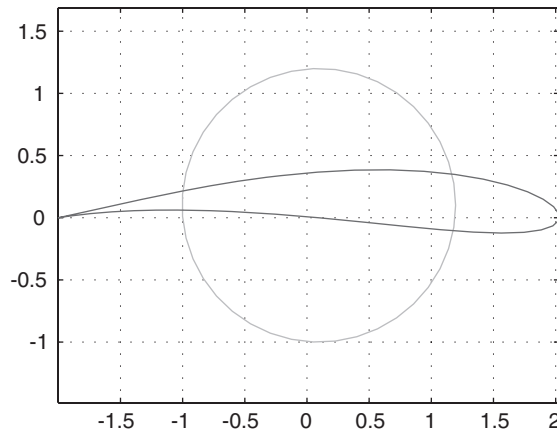


Figure 11.4 Parametric 2-D plot

Listing 11.4 Parametric plots

```
1. th = linspace(0, 2*pi, 40);
2. r = 1.1; g = .1;
3. cx = sqrt(r^2-g^2) - 1; cy = g;
4. x = r*cos(th) + cx;
5. y = r*sin(th) + cy;
6. plot( x, y, 'r' )
7. axis equal
8. grid on
9. hold on
10. z = complex(x, y);
11. w = z + 1./z;
12. plot( real(w), imag(w), 'k' );
```

In Listing 11.4:

Line 1: The independent variable in this case is the angle θ varying from 0 to 2π .

Line 2: The particular transformation we use here requires a circle with a radius, r , slightly greater than 1 offset by a small distance, g , from the x-axis, passing through the point $(-1, 0)$.

Line 3: We compute the center of the circle passing through the point $(-1, 0)$.

Lines 4–5: Show a standard polar-to-Cartesian coordinate transformation computing the coordinates of the circle.

Line 6: Plots the two dependent variables x and y with a red line.

Line 7: Equalizes the axes and forces the circle to be drawn correctly.

Line 8: Displays the grid and simplifies the assessment that the circle passes through the important point $(-1, 0)$.

Line 9: Here we want to add a second plot to the figure.

Lines 10–11: The Joukowski transformation is easiest when expressed in complex terms: if z is the path around the required circle, $w = z + 1/z$ traces a very credible looking airfoil shape.

Line 12: Adds the plot of w , and reverts from the complex plane to plot the real and imaginary parts colored in black.

11.2.4 Other 2-D Plot Capabilities

MATLAB can also create two dimensional pie charts, histograms, and bar graphs with the following functions:

- `bar(x, y)` produces a bar graph with the values in y positioned at the horizontal locations in x . The options available can be studied with `>> help bar`.
- `barh(x, y)` produces a bar graph with the values in y positioned at the horizontal locations in x . The options available can be studied with `>> help barh`.
- `fill(x,y,n)` produces a filled polygon defined by the coordinates in x and y . The fill color is specified by indexing n into the color map. The options available can be studied with `>> help fill`.
- `hist(y, x)` produces a histogram plot with the values in y counted into bins defined by x . The options available can be studied with `>> help hist`.
- `loglog(x, y)` plots x versus y on a log-log scale. For more options, see `>> help loglog`.
- `pie(y)` makes a pie chart of the values in y . For more options, see `>> help pie`.

- `polar(th, y)` makes a polar plot of the angle `th` (radians) with the radius `r` specified for each angle. For more options, see `>> help polar`.
- `semilogx(x, y)` plots `x` versus `y` with the `x` scale logarithmic. For more options, see `>> help semilogx`.
- `semilogy(x, y)` plots `x` versus `y` with the `y` scale logarithmic. For more options, see `>> help semilogy`.



11.3 3-D Plotting

Before attacking the details of plotting in three dimensions, it should be noted that even two-dimensional plots in MATLAB are actually 3-D plots. Consider the picture shown in Figure 11.5, which was generated originally as a 2-D plot of `sin(x)` versus `x`. By selecting the Rotate 3D icon on the tool bar and moving the mouse on your figure, it becomes apparent that what appeared to be a 2-D plot in the `x-y` plane is really a 3-D plot in the `x-y-z` plane “suspended in space” at `z = 0`.

11.3.1 Linear 3-D Plots

The simplest method of three-dimensional (3-D) plotting is to extend our 2-D plots by adding a set of `z` values. In the same style as `plot(...)`, `plot3(x, y, z, str)` consumes three vectors of equal size and connects the points defined by those vectors in 3-D space. The optional `str` specifies the color and/or line style. If the `str` is omitted, the default line is solid blue. The function also permits multiple `(x, y, z, str)` data sets in a single function call. In this case, if the line style is not specified, the default line colors rotate through the active color map.

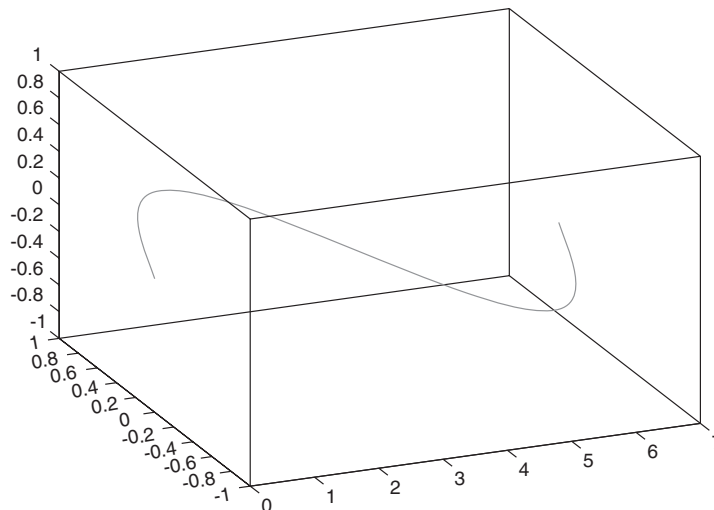


Figure 11.5 Rotated 2-D plot

Figure 11.6 shows three curves plotted in three dimensions, using the script shown in Listing 11.5. Each plot is in the z-x plane: the red curve at $y = 0$; the blue curve at $y = 0.5$; and the green curve at $y = 1$.

In Listing 11.5:

Line 1: Each plot has the same set of x values.

Lines 2–3: The y values for the first plot are all 0.

Lines 4–5: The second and third plots are $\sin(x)$ at different frequencies.

Lines 6–7: The y values of the second and third plots are all 0.5 and 1, respectively.

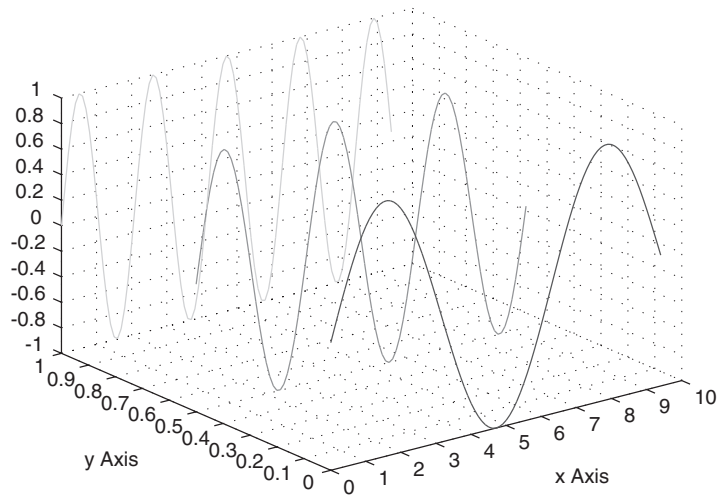


Figure 11.6 3-D lines

Listing 11.5 Simple 3-D line plots

```
1. x=0:0.1:3.*pi;
2. y1=zeros(size(x));
3. z1=sin(x);
4. z2=sin(2.*x);
5. z3=sin(3.*x);
6. y3=ones(size(x));
7. y2=y3./2;
8. plot3(x,y1,z1, 'r',x,y2,z2, 'b',x,y3,z3, 'g')
9. grid on
10. xlabel('x-axis')
11. ylabel('y-axis')
12. zlabel('z-axis')
```

11.3.2 Linear Parametric 3-D Plots

We can generalize the concept of parametric plots to 3-D, as shown in Figure 11.7, in which the x , y , and z values are mappings of some linear parameter. As with 2-D parametric plots, 3-D parametric plots allow the variables on each axis to be dependent on a separate, independent variable that defines a path in the plotting space.

On the left side, we draw a spiral as an example of a 3-D plot where two of the dimensions, x and y , are dependent on the third, independent parameter. The independent parameter in this example is the rotation angle, θ , varying from 0 to 10π (five complete revolutions). The x and y values are mapped as $\sin(\theta)$ and $\cos(\theta)$ —the classic means of describing a circle. The spiral effect is accomplished by plotting θ on the z -axis.

The right half of Figure 11.7 illustrates a fully parametric plot, where the values of all three coordinates are mappings of an independent parameter, t . This particular example is a plot of the 3-D motion of a particle receiving random impulses in all three axes. Note the use of text anchored in x - y - z space to label points on the graph. The figure is drawn using Listing 11.6.

In Listing 11.6:

Lines 2–5: Draw the spiral plot with a simple `plot3(...)` call.

Lines 8–10: Define random velocity increments in x , y , and z .

Listing 11.6 Linear parametric 3-D plots

```

1. subplot(1, 2, 1)
2. theta = 0:0.1:10.*pi;
3. plot3(sin(theta),cos(theta),theta)
4. title('parametric curve based on angle');
5. grid on

6. subplot(1, 2, 2)
7. N = 20;
8. dvx = rand(1, N) - 0.5 % random v changes
9. dvy = rand(1, N) - 0.5
10. dvz = rand(1, N) - 0.5
11. vx = cumsum(dvx); % integrate to get v
12. vy = cumsum(dvy);
13. vz = cumsum(dvz);
14. x = cumsum(vx); % integrate to get pos
15. y = cumsum(vy);
16. z = cumsum(vz);
17. plot3(x,y,z)
18. grid on
19. title('all 3 axes varying with parameter t')
20. text(0,0,0, 'start');
21. text(x(N),y(N),z(N), 'end');
```

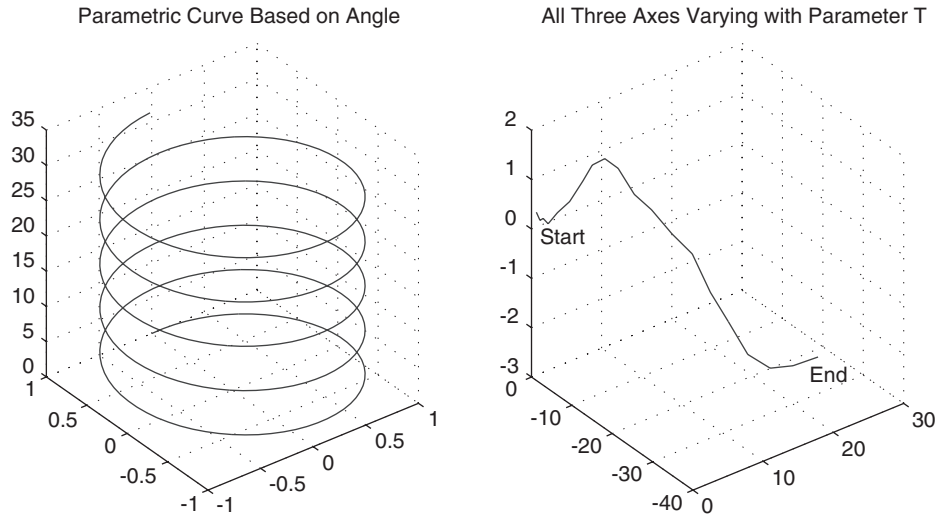


Figure 11.7 Parametric 3-D plots

Lines 11–13: Integrate to obtain the velocities, starting at 0. There will be a full discussion of integration in Chapter 15, Section 15.3.

Lines 14–16: Integrate again to compute the position in x, y, z space.

Lines 17–19: Plot and enhance the time history of the particle.

Lines 20 and 21: Add labels to indicate the start and end of the trace.

11.3.3 Other 3-D Plot Capabilities

MATLAB can also create three-dimensional pie charts, histograms, and bar graphs with the following functions:

- `bar3(x, y)` produces a bar graph with the values in y positioned at the horizontal locations in x . The options available can be studied with `>> help bar3`.
- `barh3(x, y)` produces a bar graph with the values in y positioned at the horizontal locations in x . The options available can be studied with `>> help barh`.
- `pie3(y)` makes a 3-D pie chart of the values in y . For more options, see `>> help pie3`.



11.4 Surface Plots

In Section 11.3.2 we saw that data can be generated for all three axes based on one linear parameter. However, the most dramatic graphics are produced

by a different group of 3-D graphics functions that produce images based on mapping a 2-D surface. The underlying surface is sometimes referred to as *plaid* because of its conceptual similarity to a Scottish tartan pattern. To design such a pattern, one needs only to specify the color sequence of the horizontal and vertical threads. In the same way, we specify a plaid by defining vectors of the row and column data configurations. The simplest surface plots are obtained by defining a *z* value for each point on an *x-y* plaid.

11.4.1 Basic Capabilities

Three fundamental functions are used to create 3-D surface plots:

- `meshgrid(x, y)` accepts the $x_{1 \times m}$ and $y_{1 \times n}$ vectors that bound the edges of the plaid, and replicates the rows and columns appropriately to produce $xx_{n \times m}$ and $yy_{n \times m}$, containing the *x* and *y* values (respectively) of the complete plaid. This enables us in general to compute mappings for the 3-D coordinates of the figure we want to plot.
- `mesh(xx, yy, zz)` plots the surface as white facets outlined by colored lines. The line coloring uses one of many color maps (listed in Appendix A), where the color is selected in proportion to the *zz* parameter. You can turn the white facets transparent with the command `hidden off`.
- `surf(xx, yy, zz)` plots the surface as colored facets outlined by black lines. The line coloring by default is selected in proportion to the *zz* parameter. You can remove the lines by using one of a number of *shading* commands listed in Appendix A.

11.4.2 Simple Exercises

We will consider some simple situations that illustrate many of the features of surface drawing.

Drawing a Cube In the first example, in order to understand the underlying logic, we will develop the basic concept of drawing surfaces *without* the help of the `meshgrid(...)` function. Figure 11.8 shows the coordinates of a cube of side 2 units centered at the origin. Listing 11.7 shows the code that plots two cubes from scratch—one with no top or bottom, and the other with a top and bottom added. Figure 11.9 shows the results from this script. On the left is the basic core of the solid cube. The `surf(...)` function works by drawing the line defined by the top row of the *xx*, *yy*, and *zz* arrays. Then it locates the line defined by the next row and makes a smooth surface between the two lines.

To add the top and bottom as illustrated on the right side, we add the points P and Q to the array. Although only one point each is required to define P and Q, the array must have the same number of columns in each row.

Therefore, P and Q must be replicated five times to keep the arrays rectangular. Physically, this has the following effect:

- Beginning at point P, drawing expanding squares until they reach ABCD
- “Sliding down” the sides of the cube to EFGH
- Shrinking that square down to the point Q

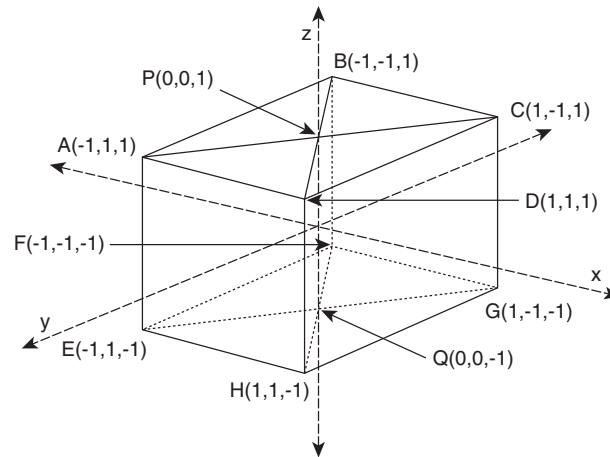


Figure 11.8 *A simple cube*

In Listing 11.7:

Lines 1–6: Establish the plaid defining the A-B-C-D plane (the first rows of each array) and the E-F-G-H plane (the second rows). Notice that the first corner is repeated on each row to close the figure shape.

Line 7: Selects the first plot.

Lines 8–12: Plot the cube sides.

Line 13: The x value of P.

Line 16: The x value of Q.

Line 17: The y value of P.

Line 20: The y value of Q.

Line 21: The z value of P.

Line 24: The z value of Q.

Line 25: Selects the second plot.

Lines 26–30: Plot the cube top, sides, and bottom.

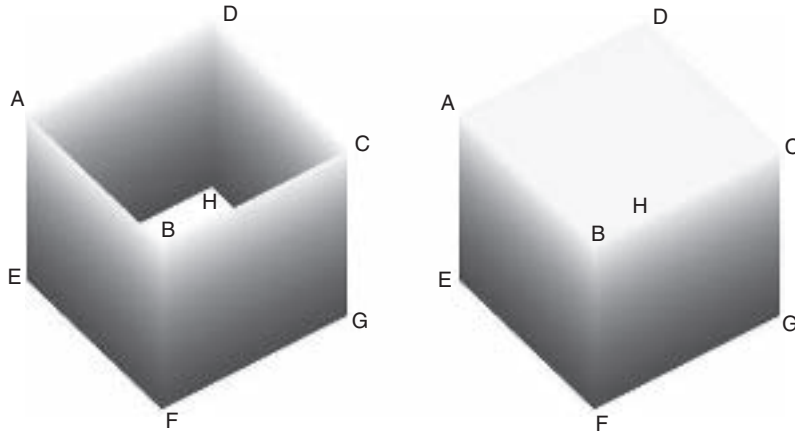


Figure 11.9 Solid cubes

Listing 11.7 Simple solid cube

```

1. xx = [-1 -1 1 1 -1 % A-B-C-D-A
2.      -1 -1 1 1 -1] % E-F-G-H-E
3. yy = [ 1 -1 -1 1 1 % A-B-C-D-A
4.       1 -1 -1 1 1] % E-F-G-H-E
5. zz = [ 1 1 1 1 1 % A-B-C-D-A
6.      -1 -1 -1 -1 -1] % E-F-G-H-E
7. subplot(1, 2, 1)
8. surf(xx, yy, zz)
9. axis equal
10. shading interp
11. view(-36, 44)
12. axis off

13. xx = [ 0 0 0 0 0 % P-P-P-P-P
14.      -1 -1 1 1 -1 % A-B-C-D-A
15.      -1 -1 1 1 -1] % E-F-G-H-E
16.      0 0 0 0 0] % Q-Q-Q-Q-Q
17. yy = [ 0 0 0 0 0 % P-P-P-P-P
18.       1 -1 -1 1 1 % A-B-C-D-A
19.       1 -1 -1 1 1] % E-F-G-H-E
20.      0 0 0 0 0] % Q-Q-Q-Q-Q
21. zz = [ 1 1 1 1 1 % P-P-P-P-P
22.       1 1 1 1 1 % A-B-C-D-A
23.      -1 -1 -1 -1 -1] % E-F-G-H-E
24.      -1 -1 -1 -1 -1] % Q-Q-Q-Q-Q
25. subplot(1, 2, 2)
26. surf(xx, yy, zz)
27. axis equal
28. shading interp
29. view(-36, 44)
30. axis off

```

A Simple Parabolic Dish We will continue with a simple example illustrating the use of `meshgrid(...)` to define the plaid. Consider how we might plot the data shown in Figure 11.10. Before we look at the code, consider what the picture represents. Clearly, the independent variables are x and y , each covering the

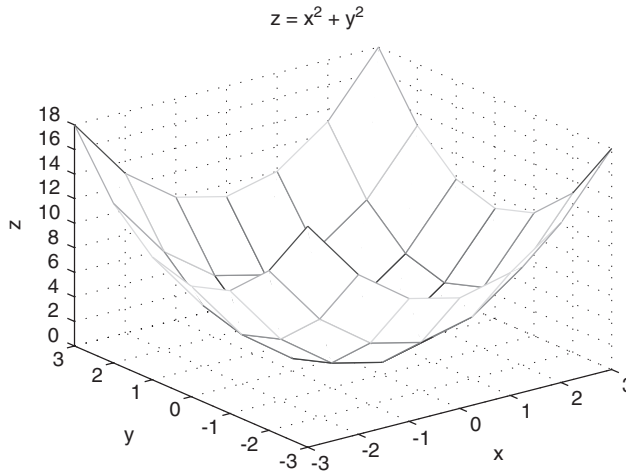


Figure 11.10 A mesh plot

range from -3 to 3 , each having seven discrete values. As the label indicates, the z values are calculated as the sum of x^2 and y^2 . There are not, however, 14 z values as the range of x and y values might suggest, but 49! In order to plot the 3-D shape of our parabolic bowl, we must have a z value for every point on the x - y surface. Each of these points has a value of x corresponding to the reading on the x -axis, and a value of y from the y -axis.

Therefore, the process of creating this plot has three parts:

1. Develop the underlying plaid specifying the x - y location of every point on the x - y plane.
2. Calculate the z values from the plaid.
3. Call some function that will accept the plaid and these z values to produce the required plot.

The code to accomplish this is shown in Listing 11.8.

In Listing 11.8:

Line 1: The x and y vectors define the edges of the plaid.

Listing 11.8 Simple surface plot

```

1. x=-3:3; y = x ;
2. [xx,yy]=meshgrid(x,y);
3. zz=xx.^2 + yy.^2;
4. mesh(xx,yy,zz)
5. axis tight
6. title('z = x^2 + y^2')
7. xlabel('x'),ylabel('y'),zlabel('z')
```

Line 2: Generates the plaid.

Line 3: In this particular example, we map only the z coordinate, leaving the plaid (xx and yy) as the x and y coordinates of the figure.

Line 4: `mesh(...)` is one of many functions that represent 3-D mappings of a plaid in different ways. Notice in the figure that the faces between line segments are solid white, and the line colors change with the z coordinate.

Try Exercise 11.1 and make your observations.



Exercise 11.1 Output from `meshgrid(...)`

Do It Yourself

Run the script shown in Listing 11.8 without the semicolon on Line 2, and observe the following:

```
xx =
    -3    -2    -1     0     1     2     3
    -3    -2    -1     0     1     2     3
    -3    -2    -1     0     1     2     3
    -3    -2    -1     0     1     2     3
    -3    -2    -1     0     1     2     3
    -3    -2    -1     0     1     2     3
    -3    -2    -1     0     1     2     3
yy =
    -3    -3    -3    -3    -3    -3    -3
    -2    -2    -2    -2    -2    -2    -2
    -1    -1    -1    -1    -1    -1    -1
     0     0     0     0     0     0     0
     1     1     1     1     1     1     1
     2     2     2     2     2     2     2
     3     3     3     3     3     3     3
```

Examine the values of `xx` and `yy`. Notice that in general, if `x` is length `m` and `y` is length `n`, the `xx` values consist of the `x` vector in rows replicated `n` times, and the `yy` values consist of the `y` vector as a column replicated `m` times. Together, they provide the underlying `x` and `y` values for the “floor” of the bowl plot from which the `z` values are computed to draw the picture.

Manipulating Plots Exercise 11.2 provides an opportunity for you to work with plotting.



Exercise 11.2 Plotting

Do It Yourself

To begin to discover the range of rendering capabilities available, perform the following simple exercise, based on the code shown in Listing 11.8:

1. Insert the line `hidden off` after `mesh(xx, yy, zz)`.

continued on next page

2. Change `mesh(xx, yy, zz)` to `surf(xx, yy, zz)`. Notice that the panels are now colored and the lines are black. This form is also insensitive to the hidden parameter.
3. Replace `hidden off` with `shading flat`, and notice that the lines have disappeared.
4. Replace `shading flat` with `shading interp`, and notice that the surface is now smoothly contoured.
5. Insert the line `colormap 'summer'` after `mesh(xx, yy, zz)`. There are a number of built-in color maps to handle the shading. Look up `help colormap` for details.
6. Do not forget to rotate your images and examine them from different points of view using the 3-D rotate tool bar icon.

For the next few exercises, we will replace our do-it-yourself data with one of MATLAB's many built-in data sets. Figure 11.11 shows a picture of the peaks data set created by the script shown in Listing 11.9.

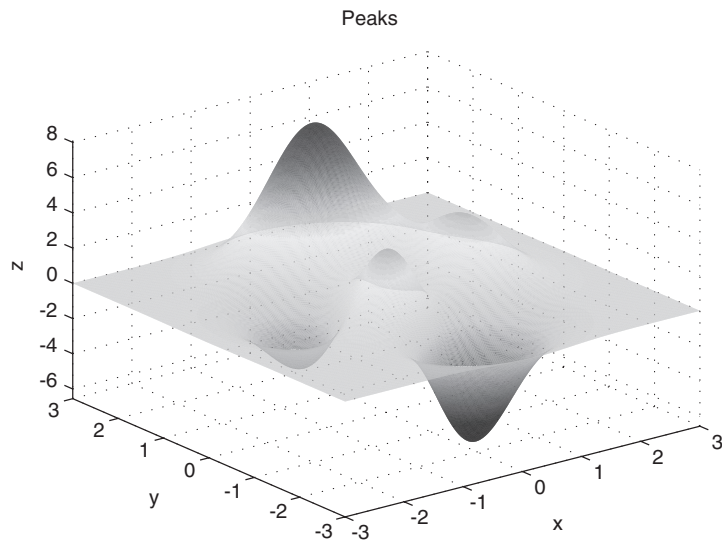


Figure 11.11 *A peaks plot*

Listing 11.9 Basic `peaks(...)` plot

```

1. [xx,yy,zz]=peaks(30);
2. surf(xx,yy,zz)
3. colormap 'default'
4. shading interp
5. axis tight
6. title('peaks')
7. xlabel('x'),ylabel('y'),zlabel('z')

```

In Listing 11.9:

Line 1: The function `peaks(n)` returns the complete `xx`, `yy`, `zz` data sets for drawing this odd-shaped function with `n` facets along each axis. Note that `meshgrid` is done internally in the `peaks(...)` function.

Line 3: The color map can be restored if necessary by specifying `'default'`.

Exercise 11.3 provides an opportunity to work with `peaks(...)`.

11.4.3 3-D Parametric Surfaces

Thus far, we have drawn beautiful pictures of typical 3-D plots in a Cartesian axis system where the x-y plaid was passed directly to the plotting function. The only coordinate actually mapped was the z-axis. We now return to the concept briefly introduced in Section 11.3.2 when discussing Figure 11.9. In that case, all three coordinate values were mappings of one parameter, `t`.



Exercise 11.3 `peaks(...)`

Do It Yourself

Continue with the next exercise using Listing 11.9 as your code baseline.

1. Change the `surf(xx, yy, zz)` call to `surfc(xx, yy, zz)`. Notice the contour lines plotted on the x-y plane base.
2. Add the line `view(-45, 60)`, which changes the viewing angle. This can program a different angle rather than having to manually rotate the image after it is drawn. If you rotate the image manually, the azimuth and elevation numbers in the lower-left corner are the values you need to enter in the `view(...)` function to replicate that view.
3. Add a color bar by inserting the command `colorbar`—see the help menu for the code that labels the color bar.
4. Change the `surfc(xx, yy, zz)` call to `surf(xx, yy, zz, yy)`. Here, we override the default color direction `z` with the `y` direction by supplying a fourth parameter to the `surfc(...)` function, specifying the color values explicitly rather than implicitly.
5. Change `surf(xx, yy, zz, yy)` to `surf(xx, yy, zz, del2(zz),)`. The `del2(...)` function computes the second derivative, or curvature, of the plot, so now the coloring highlights the areas of maximum curvature.
6. For an eye-catching effect, change the parameter to `peaks` to 120, and add the line `lightangle(60, 45)` at the bottom of the script. This illuminates the surface with a light at the specified azimuth and elevation angle (degrees). Figure 11.12 shows the results.
7. Try this effect without increasing the `peaks` parameter. You will see that the lighting effect highlights the facets on the figure due to the coarser resolution of the raw data.

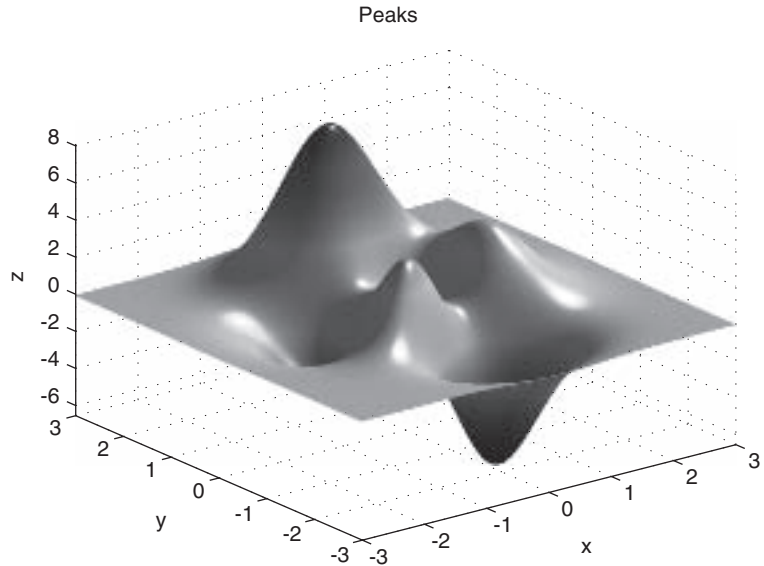


Figure 11.12 *Curvature color and lighting*

Consider first the construction of a cylinder as illustrated in Figure 11.13. One could consider this figure as a sheet of paper rolled up in a circular shape. We could visualize that piece of paper as a plaid of values, not of x - y in this case, but perhaps $x - \theta$. The range of x would be from 0 to the length of the cylinder, and the range of θ would be 0 to 360° .

To plot this, one would then merely need to create a plaid in x and θ , and then decide on the mapping from θ to the y and z values of the cylinder. The resulting picture is shown in Figure 11.14, and the code is shown in Listing 11.10.

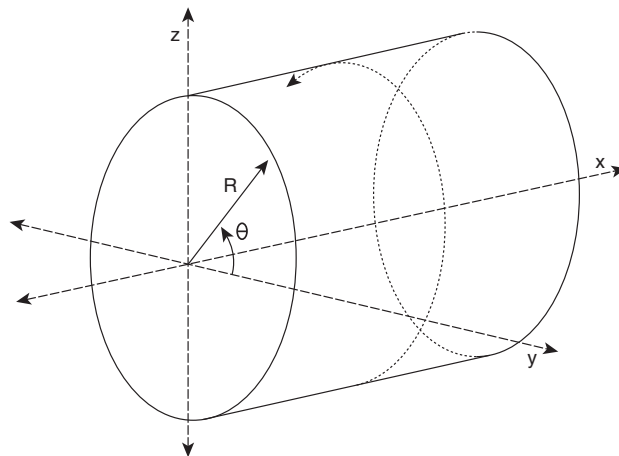


Figure 11.13 *Creating a cylinder image*



Figure 11.14 *A cylinder plot*

In Listing 11.10:

- Line 1: Constants to define the smoothness of the cylinder.
- Lines 2–4: Define a plaid in x and θ . Note that only two points are needed in the x direction because that contour is straight.
- Lines 5 and 6: The circular cross-section is achieved by using the parametric definition of a circle of a given radius.
- Line 9: Changes the color to a pleasant metallic scale.
- Line 10: Squares up and removes the axes.
- Line 11: Illuminates the figure.
- Line 12: Sets the transparency of the surface so that a portion of the hidden details can show through.

Listing 11.10 Constructing a cylinder

```

1. facets = 120; len = 2; radius = 1;
2. thr = linspace(0, 2*pi, facets);
3. xr = [0 len];
4. [x, th] = meshgrid( xr, thr );
5. y = radius * cos(th);
6. z = radius * sin(th);
7. surf(x, y, z);
8. shading interp
9. colormap bone
10. axis equal,axis tight,axis off
11. lightangle(60, 45)
12. alpha(0.8)
13. view(-20, 35)

```



Figure 11.15 *A copper sphere*

Now, we construct a sphere as shown in Figure 11.15, starting with the cylinder. However, instead of using a constant radius in the x direction, we will calculate the radius in that direction by rotating a second angle, ϕ , from 0 to 180° . Think of this as mapping or “wrapping” a plaid with two angles as the independent variables around the sphere. The coordinate in the x direction would be $r \cos \phi$, and the radii of the y - z circles would be $r \sin \phi$. The code for drawing this sphere is shown in Listing 11.11.

In Listing 11.11:

Line 1: The radius set here is the sphere radius.

Lines 2 and 3: Set the ranges of θ and ϕ .

Line 4: Builds the plaid in θ and ϕ .

Line 5: As ϕ rotates, the value of x varies as its cosine.

Lines 6 and 7: The radius of rotation about the x -axis varies as the sine of ϕ .

Listing 11.11 Constructing a sphere

```
1. facets = 120; radius = 1;
2. thr = linspace(0, 2*pi, facets); % range of theta
3. phir = linspace(0, pi, facets); % range of phi
4. [th, phi] = meshgrid( thr, phir );
5. x = radius * cos(phi);
6. y = radius * sin(phi) .* cos(th);
7. z = radius * sin(phi) .* sin(th);
8. surf(x, y, z);
9. shading interp
10. colormap copper
11. axis equal, axis tight, axis off
12. lightangle(60, 45)
```

11.4.4 Bodies of Rotation

The cylinder and sphere drawn in the above section are special cases of a more general form of solid body. Bodies of rotation are created by rotating a linear curve about a specified axis, that is, by rotating a general function $v = f(u)$ defined over a range of u values about the x or z axes. Note: this is perfectly general because rotating such a function about the y -axis would result merely in “sliding” the function across a flat surface in the x - z plane. We use z rather than y for the dependent variable here because in MATLAB 3-D plots, the z -axis is vertical.

Rotating Continuous Functions First we consider rotating a continuous function $v = f(u)$ about the x and z axes.

- To rotate $v = f(u)$ about the x -axis, we should rewrite this equation as $r = f(x)$. Figure 11.16 shows the logic of this rotation. The independent variable is x , and the values of y and z are computed as the usual polar-to-Cartesian conversion:

$$\begin{aligned}y &= r \cos(\theta) \\z &= r \sin(\theta)\end{aligned}$$

Notice that these are the two axes about which we are not rotating.

- To rotate $v = f(u)$ about the z -axis, we should rewrite this equation as $z = f(r)$. Figure 11.17 shows the logic of this rotation. The independent variable is now r , and the values of x and y are computed as the usual polar-to-Cartesian conversion:

$$\begin{aligned}x &= r \cos(\theta) \\y &= r \sin(\theta)\end{aligned}$$

Notice again that these are the two axes about which we are not rotating. Notice also a simple rule of thumb: if you rewrite $v = f(u)$

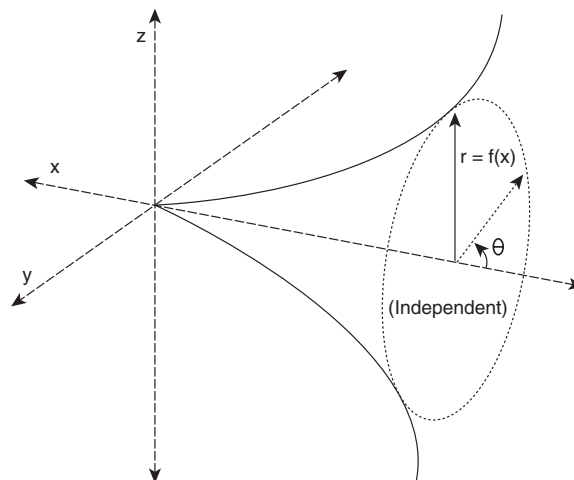


Figure 11.16 Rotating $v = f(u)$ about the x -axis

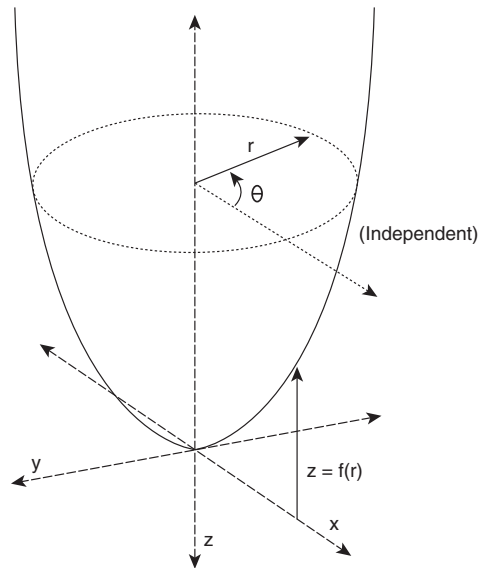
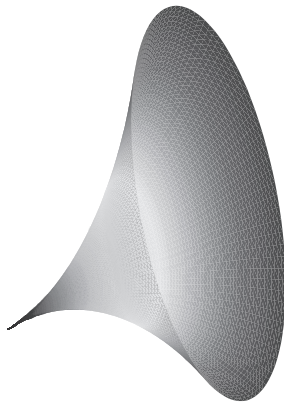


Figure 11.17 Rotating $v = f(u)$ about the z -axis

correctly for each rotation, the independent variable is always the parameter to $f(\dots)$.

Figure 11.18 shows the result of these rotations generated by the code shown in Listing 11.12.

x^2 Rotated about the x Axis



x^2 Rotated about the z Axis

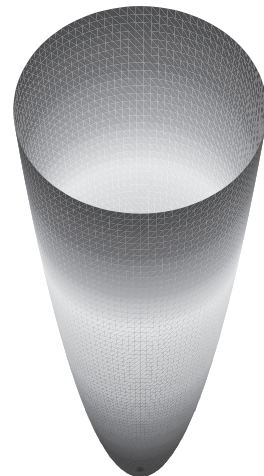


Figure 11.18 Rotation of u^2

Listing 11.12 Rotating $v = u^2$ about the x and z axes

```

1. facets = 100;
2. u = linspace(0, 5, facets);
3. th = linspace(0, 2*pi, facets);
4. [uu tth] = meshgrid(u, th);

5. % rotate about the x-axis
6. subplot(1, 2, 1)
7. rr = uu.^2;
8. yy = rr .* cos(tth);
9. zz = rr .* sin(tth);
10. surf(xx, yy, zz, xx);
11. shading interp, axis tight
12. xlabel('x'), ylabel('y'), zlabel('z')
13. title('u^2 rotated about the x-axis')
14. % rotate about the z-axis
15. subplot(1, 2, 2)
16. rr = uu;
17. zz = rr.^2;
18. xx = rr .* cos(tth);
19. yy = rr .* sin(tth);
20. surf(xx, yy, zz);
21. shading interp, axis tight
22. xlabel('x'), ylabel('y'), zlabel('z')
23. title('u^2 rotated about the z-axis')

```

In Listing 11.12:

Lines 1–4: Set up the plaid of u , the independent variable for the function, and θ for the rotations.

Lines 6–13: Compute the rotation about the x-axis. Notice that when rotating about a specific axis, that axis must be treated separately; the other two axes will always have the form of a polar-to-Cartesian transformation. In rotating about the x-axis, since u is the independent variable for our function, we only need to compute the yy and zz values.

Line 10: We use the fourth parameter to `surf(...)` to set the direction of color variation.

Lines 15–23: Compute the z-axis rotation. Some apparent sleight of hand is necessary here. In this case, the axis containing the independent variable is being rotated about the z-axis. Because the radius of the rotated surface is the original independent variable, uu , we copy uu to the variable $radius$. Then we define xx together with yy as the polar-to-Cartesian transformation to achieve the rotation. In this case, the z value of the surface is $f(u)$, u^2 .

Rotating Discrete Functions There is no need to restrict ourselves to continuous functions as the profiles for bodies of rotation. Figure 11.19 shows

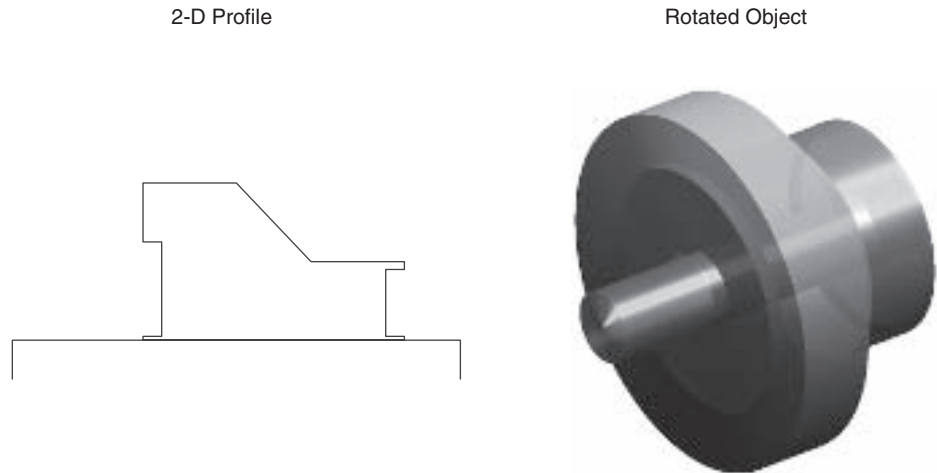


Figure 11.19 *Rotation of machine part*

the 2-D profile of a fictitious machine part and the picture created when that profile is rotated about the x-axis. The figure was generated by the code shown in Listing 11.13.

In Listing 11.13:

Lines 1–9: Define and plot the initial 2-D profile. Notice that although this is a continuous curve, MATLAB has accomplished

Listing 11.13 Rotating an irregular shape

```

1. u = [0 0 3 3 1.75 1.75 2 2 1.75 1.75 3 4 ...
2.      5.25 5.25 5 5 5.25 5.25 3 3 6 6];
3. v = [0 .5 .5 .502 .502 .55 .55 1.75 1.75 ...
4.      2.5 2.5 1.5 1.5 1.4 1.4 ...
5.      .55 .55 .502 .502 .5 .5 0];
6. subplot(1, 2, 1)
7. plot(u, v)
8. axis ([-1 7 -1 5]), axis off
9. title('2-D profile')

10. facets = 200;
11. subplot(1, 2, 2)
12. [uu tth] = meshgrid( u, linspace(0, 2*pi, facets) );
13. radius = meshgrid( v, 1:facets);
14. yy = radius .* cos(tth);
15. zz = radius .* sin(tth);
16. surf(xx, yy, zz);
17. shading interp
18. axis square, axis tight, axis off
19. colormap bone
20. lightangle(60, 45)
21. alpha(0.8)
22. title('rotated object')

```

something very strange. The coordinates of the third and fourth points are identical to those of the (end-fourth) and (end-third) points. Therefore, MATLAB has ignored the segment joining these two points in the profile picture and in the 3-D rendering.

Lines 10–22: Perform the rotation about the x-axis. The only unusual idea here is how to turn this discrete collection of points into the equivalent of $v = f(u)$. Line 12 shows an elegant way to solve this dilemma. After going through the `meshgrid(...)` to produce a plaid, the conversion `zz = f(xx)` would result in `zz` having the same plaid dimensions as `xx`. Since we already have all the `y` values available, we merely need to force it to conform to the dimensions of `xx` by running `meshgrid(...)` again, but keeping only the first result.

Rotating about an Arbitrary Axis Bodies of rotation are not confined to rotating about the `x`, `y`, or `z` axes. The simplest approach to rotating $z = f(x)$ about an arbitrary axis is as follows:

- Calculate the matrix that will place your axis of rotation along the `x`-axis (see Chapter 12)
- Transform `x` and `z` with that rotation
- Rotate the results about the `x`-axis
- Invert the transformation on the resulting surface

11.4.5 Other 3-D Surface Plot Capabilities

MATLAB can also create special-purpose plots with the following functions:

- `alpha(x)` sets the transparency of the surfaces. $0 \leq x \leq 1$, where 0 means completely transparent and 1 is opaque. The options available can be studied with `>> help alpha`.
- `contour(z)` produces a contour plot of the plaid surface defined by `z`. The options available can be studied with `>> help bar3`.
- `[x,y,z] = cylinder(n)` constructs the `meshgrid` for a cylinder with `n` facets in each direction. For more options, see `>> help cylinder`.
- `[x,y,z] = ellipsoid(n)` constructs the `meshgrid` for an ellipsoid with `n` facets in each direction. For more options, see `>> help ellipsoid`.
- `lightangle(az,el)` sets the angle of a light source (angles in degrees). For more options, see `>> help lightangle`.
- `meshc(x,y,z)` makes a mesh plot with contours below. For more options, see `>> help meshc`.
- `meshz(x,y,z)` makes a mesh plot with vertical line extensions. For more options, see `>> help meshz`.
- `pie3(y)` makes a 3-D pie chart of the values in `y`. For more options, see `>> help pie3`.

- `[x,y,z] = sphere(n)` constructs the meshgrid for a sphere with `n` facets in each direction. For more options, see `>> help sphere`.
- `surf(x,y,z)` makes a surface plot with contours below. For more options, see `>> help surf`.
- `surfz(x,y,z)` makes a surface plot with vertical line extensions. For more options, see `>> help surfz`.
- `waterfall(x,y,z)` makes a mesh plot with vertical line extensions only in the `x` direction. For more options, see `>> help surfz`.

11.4.6 Assembling Compound Surfaces

We can assemble more complex solid bodies by constructing simple surfaces and concatenating the data before submitting it to the rendering machine. The solid disk shown in Figure 11.20 was constructed by first drawing the front face as a body of rotation, then the curved surface, and then the back face. If the angles of rotation are maintained in a consistent direction, the three sets of `[x y z]` arrays can be concatenated and submitted to the rendering software as before. The code for this figure is shown in Listing 11.14.

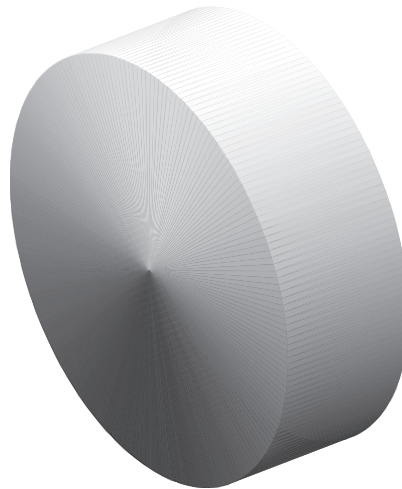


Figure 11.20 *A solid disk*

Listing 11.14 Drawing a solid disk

```

1. %basic parameters
2. facets = 200;
3. len = 2; radius = 3;
4. radial = [0, radius];
5. th = linspace(0, 2*pi, facets);
6. along = [0 len];

```

continued on next page

```

7. %build the front face
8. [r1, tth] = meshgrid( radial, th );
9. x1 = zeros(size(r1));
10. y1 = r1.*cos(tth);
11. z1 = r1.*sin(tth);

12. % build the curved surface
13. [l, tth] = meshgrid(along, th);
14. x2 = l;
15. y2 = radius*cos(tth);
16. z2 = radius*sin(tth);

17. % build the back face
18. x3 = len*ones(size(r1));
19. [r3, tth] = meshgrid(radial(end:-1:1), th);
20. y3 = r3.*cos(tth);
21. z3 = r3.*sin(tth);

22. % assemble and draw the three parts
23. x = [x1 x2 x3]; y = [y1 y2 y3]; z = [z1 z2 z3];
24. surf(x, y, z);
25. shading interp
26. colormap copper
27. axis equal, axis tight, axis off

```

In Listing 11.14:

- Lines 2 and 3: Show the basic parameters of the disk.
- Line 4: Shows the edge of the plaids in the radial direction.
- Line 5: Shows the edge of the plaids around the cylinder.
- Line 6: Shows the edge of the plaids along the cylinder.
- Lines 8–11: Build a disk for the front face from the center outward.
- Lines 13–16: Build a cylindrical surface along the x-axis.
- Lines 18–21: Build the back face from the rim inward to ensure continuity of the flow of data along the cylinder from the center of the front disk finishing at the center of the back disk.
- Line 23: Assembles the pieces by concatenating the x, y, and z values.
- Lines 24–27: Render the complete object.

Shapes of considerable complexity can be assembled this way. Consider, for example, the Klein bottle, a well-documented example of topological curiosity. The particular example shown in Figure 11.21 was constructed in the manner indicated above by assembling individually mapped articles.

The code is a little too complex to be included here, and is left as an exercise for the reader.



Figure 11.21 A Klein bottle



11.5 Interacting with Plotted Data

Two new features introduced with MATLAB 7.6 (R2008a) allow you to interact with the data presented in a plot. Brushing allows you to select portions of the data presented in a plot, and make changes to the values presented. Linking allows you to connect the plotted data to the underlying data source so that when you make changes to the plotted data, these changes are reflected in the data source.

11.5.1 Brushing Data Items

Suppose you were to execute the following simple script in MATLAB:

```
clear
clc
close all

th = linspace(1, 20*pi, 500);
y = sin(th);
plot(th, y)
```

The result would be the plot illustrated in Figure 11.22. Notice that in R2008a, there are two new tools on the tool bar, a brush symbol and a chain link symbol. There is also a pop-up notice offering more information about brushing and linking. Once you have studied the new features, you can close the pop-up permanently by clicking the 'x' on its right corner. Suppose for the sake of a simple example that we want to see the plot with one cycle in the middle of the plot missing. You would select the brush tool and highlight the data you want to modify. By right clicking that selected data, you see the choices shown on Figure 11.23. NaN is frequently a good choice because the plotting functions ignore any NaN

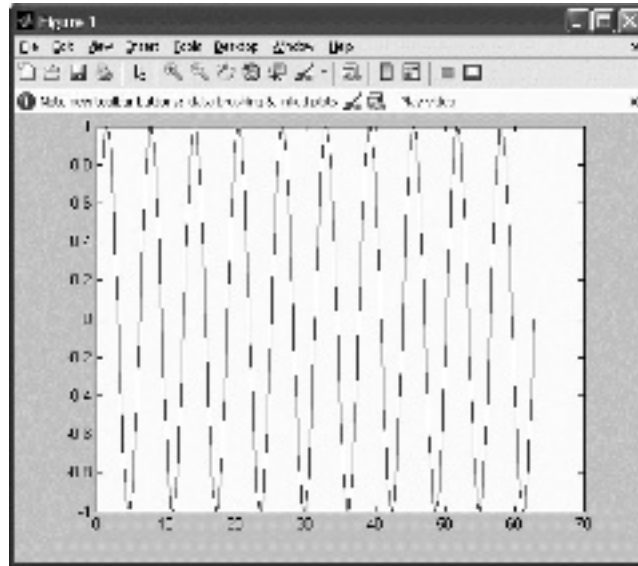


Figure 11.22 Original sine wave plot

values. Figure 11.24 shows the plot immediately after you have selected the NaN replacement option. Notice that the lines that would have connected the middle cycle are now missing.

11.5.2 Linking a Plot to the Source Data

While brushing alone allows you to modify the plotted data, you might want to capture the changes you make in the underlying source data. To

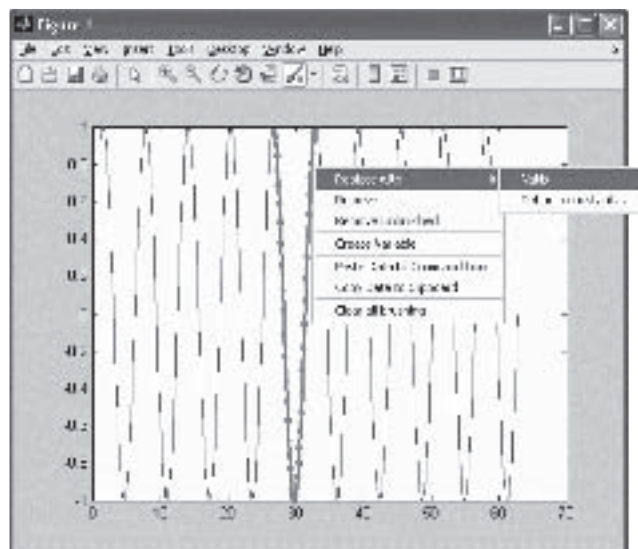


Figure 11.23 Selected data and the pop-up menus

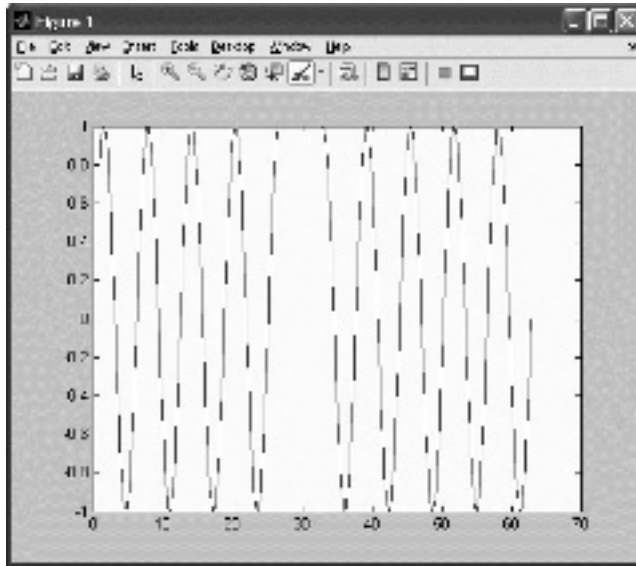


Figure 11.24 *Replacing values with NaN removes them from the plot*

accomplish this, you select the linking tool by clicking its icon. A pop-up window allows you to select the data source to which you want to link the plot. When you have made the selection, you see a reminder pop-up that allows you to make changes to the link selection. Figure 11.25 shows the plot window during the process of selecting the upper half of the first two cycles.

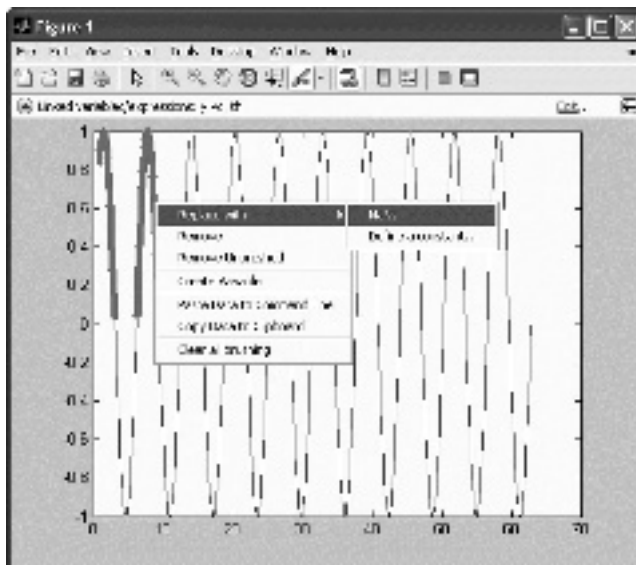


Figure 11.25 *Selecting a rectangular area of the plot*

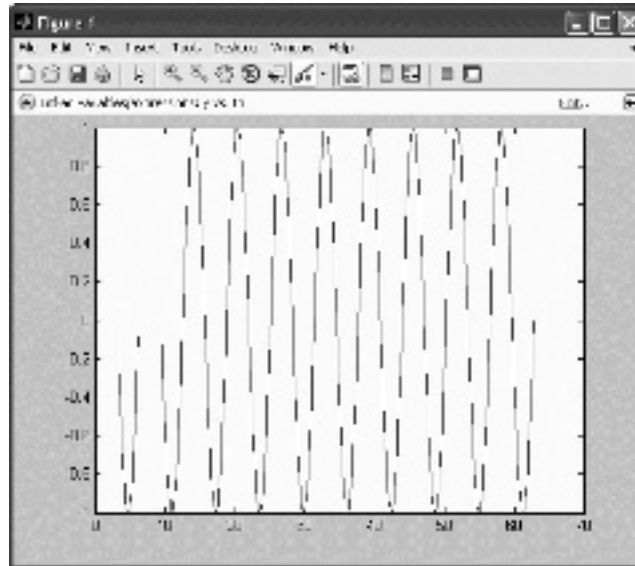


Figure 11.26 Rectangular area removed with Linking enabled

Figure 11.26 shows the result of this change.

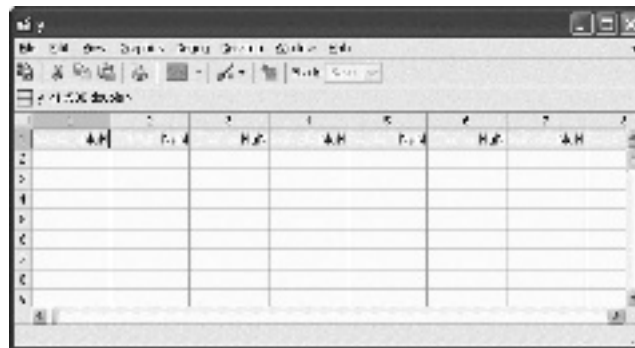


Figure 11.27 The underlying data has been changed to NaN

If you then went to the Workspace and double-clicked the variable `y`, you would see the NaN values inserted by the linking mechanism as shown in Figure 11.27.

11.5.3 Further Information

It is difficult in a textbook that is essentially static to demonstrate data interactions. An interested reader can view an on-line video presentation that demonstrates these capabilities in more detail at http://www.mathworks.com/products/demos/shipping/matlab/WhatsNewR2008a_GraphicsAndGUIBuilding.html

11.6 Engineering Example—Visualizing Geographic Data

You have been given two files of data: `atlanta.txt`, which presents the streets of Atlanta in graphical form, and `ttimes.txt`, which gives the travel times between Atlanta suburbs and the city center. You have been asked to present these data sets in a manner that will help to visualize and validate the data.

11.6.1 Analyzing the Data

First we proceed to determine the nature of the data by opening the files in a text editor and examining their format and content.

1. *Determine the file format:* The first step is to open the data files in a plain text editor (the MATLAB editor would work fine). The format appears to be consistent with that of a text file delimited by tab characters. Since there are no strings in the file, it should be suitable to be read using MATLAB's built-in `d1mread(...)` function.
2. *Discern the street map file content:* Table 11.1 shows the first few lines of the file `atlanta.txt` simplified by omitting certain irrelevant columns. The numbers in columns 3–6 are pairs, the first of the pair being a large negative number, and the second a smaller positive number. Assuming that each row of this file is a street segment, these could be the x-y coordinates of the ends of a line. A little thought confirms this guess when we realize that the latitude of Atlanta is $-84^{\circ} 42'$ relative to the Greenwich meridian, and its longitude is $33^{\circ} 65'$ —clearly, the values in these columns are 1,000,000 times the latitude and longitude of points within the city, probably each end of street

Table 11.1 Street map data

... ..	-84546100.00	33988160.00	-84556050.00	33993620.00	1.00 ...
... ..	-84546080.00	33988480.00	-84558400.00	33995480.00	1.00 ...
... ..	-84243880.00	33780010.00	-84249980.00	33800840.00	1.00 ...
... ..	-84243590.00	33780060.00	-84249740.00	33800840.00	1.00 ...
... ..	-84509920.00	33944340.00	-84517200.00	33958190.00	1.00 ...
... ..	-84510420.00	33944930.00	-84516490.00	33957280.00	1.00 ...
... ..	-84252840.00	33895840.00	-84247360.00	33899290.00	1.00 ...
... ..	-84247360.00	33899290.00	-84240250.00	33903630.00	1.00 ...
... ..	-84240250.00	33903630.00	-84216090.00	33911010.00	1.00 ...
... ..	-84216090.00	33911010.00	-84203990.00	33913990.00	1.00 ...

segments. Column 7 contains numbers mostly in the range 1–6, which could indicate the type of street. We could explore this idea by coloring each line according to that value.

- 3. *Discern the travel time file content:* Table 11.2 shows the first few lines of the file `ttimes.txt` simplified by omitting certain irrelevant columns. The same latitude/longitude values occur in columns 4 and 5, but they are not repeated, suggesting that the data in this file are in a different form. Examining the first two columns, the numbers in column 2 cycle repeatedly from 1 to 75, with column 1 counting the number of cycles up to 75. Furthermore, the values in column 5 are the same whenever column 1 is the same, and the values in column 4 are the same whenever the value in column 2 matches. This seems to be much like the plaid that results from a `meshgrid(...)` function call. The values in column 6 then become evident—they would be the *z* values of the plaid, and it seems reasonable to assume that they represent the travel time in minutes.

11.6.2 Displaying the Data

With this much understanding of the data sources, we proceed to solve the problem of presenting the data. The script shown in Listing 11.15 shows the code used to visualize these data files.

In Listing 11.15:

- Line 1: Reads the street map data.
- Lines 2–3: Extract the relevant columns and determine the size of the array.
- Line 4: Color symbols to use for the lines.
- Line 5: Traverses the rows of the file.

Table 11.2 Travel time data					
1	1	...	−84575725	33554573	14.34
1	2	...	−84569612	33554573	0
1	3	...	−84563499	33554573	0
1	4	...	−84557387	33554573	0
1	5	...	−84551274	33554573	51.66
1	6	...	−84545161	33554573	50.2
1	7	...	−84539049	33554573	49.4
1	8	...	−84532936	33554573	49.65
1	9	...	−84526823	33554573	0
1	10	...	−84520710	33554573	0

Listing 11.15 Map analysis script

```

% draw the streets

1. raw = dlmread('atlanta.txt');
2. streets = raw(:,3:7);
3. [rows,cols] = size(streets)
4. colors = 'rgbkcmo';
5. for in = 1:rows
6.     x = streets(in,[1 3])/1000000;
7.     y = streets(in,[2 4])/1000000;
8.     col = streets(in,5);
9.     col(col < 1) = 7;
10.    col(col > 6) = 7;
11.    plot(x,y,colors(col));
12.    hold on
13. end

% plot the travel times
14. tt = dlmread('ttimes.txt');
15. [rows,cols] = size(tt)
16. for in = 1:rows
17.     r = tt(in, 1); c = tt(in, 2);
18.     xc(r,c) = tt(in, 4)/1000000;
19.     yc(r,c) = tt(in, 5)/1000000;
20.     zc(r,c) = tt(in, 6);
21. end
22. surf(xc, yc, zc)
23. shading interp
24. alpha(.5)
25. grid on
26. axis tight
27. xlabel('Longitude')
28. ylabel('Latitude')
29. zlabel('Travel Time (min)')
30. view(-30, 45)

```

Lines 6 and 7: Extract the longitude and latitude in degrees.

Lines 8–10: Extract and limit the line colors.

Lines 11 and 12: Plot the street lines on the same figure.

Line 14: Read the travel times.

Line 15: Sizes the array.

Line 16: Constructs the plaid by traversing the array.

Line 17: Extracts the row and column numbers.

Lines 18–20: Extract the plaid values.

Lines 22–30: Plot and display the results.

Figure 11.28 shows the resulting plot. As a credibility check, the plot can be rotated to look straight down on the map. The travel time surface shows valleys of low travel times that follow the paths of the major expressways through the city.

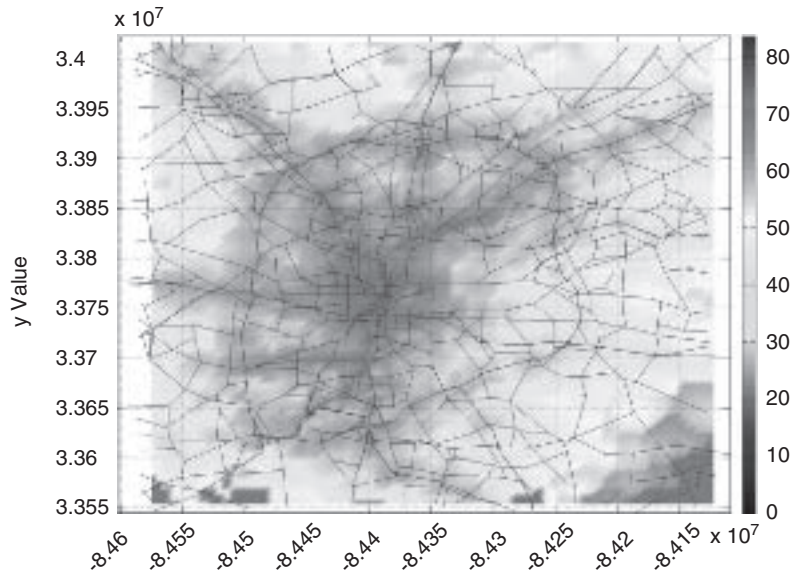


Figure 11.28 Atlanta travel times



Chapter Summary

This chapter presented the principles and practice of plotting:

- Basic two-dimensional line plots are accomplished by using `plot(x,y)`, where `x` is the independent variable and `y` the dependent variable
- Two-dimensional parametric plots are accomplished by using `plot(x,y)`, where both `x` and `y` are dependent on another independent variable
- Three-dimensional line and parametric plots are accomplished by using `plot3(x,y,z)`
- Basic three-dimensional surface plots are accomplished by building a plaid using `[xx yy] = meshgrid(x,y)`, computing the `zz` layer as a function of `xx` and `yy`, then plotting the surface using `mesh(xx, yy, zz)` OR `surf(xx, yy, zz)`
- Parametric surface plots, like parametric line plots, are achieved by building the plaid with two independent variables and making `xx`, `yy`, and `zz` functions of those independent variables
- Bodies of rotation are a special case of parametric surface plots where one of the independent variables is an angle with values between 0 and 2π
- (For MATLAB 7.6, R2008a) Brushing and linking allow interactions with plots to be reflected back into the underlying source data



Special Characters, Reserved Words, and Functions—2-D

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>axis(...)</code>	Freezes the current axis scaling for subsequent plots or specifies the axis dimensions	11.1.2
<code>bar</code>	Generates a bar graph	11.2.5
<code>barh</code>	Generates a horizontal bar graph	11.2.5
<code>clf</code>	Clears the current figure	11.1.1
<code>close all</code>	Closes all graphics windows	11.1.1
<code>colormap <spec></code>	Specifies a sequence of colors to be used when a cycle of color values is required	11.1.2
<code>figure</code>	Opens a new figure window	11.1.1
<code>fill(x,y,n)</code>	Fills a polygon defined by <code>x</code> and <code>y</code> with color index <code>n</code>	11.2.5
<code>grid off</code>	Turns the grid off	11.1.2
<code>grid on</code>	Adds a grid to the current and all subsequent graphs in the current figure	11.1.2
<code>hist</code>	Generates a histogram	11.2.5
<code>hold off</code>	Instructs MATLAB to erase figure contents before adding new information	11.1.2
<code>hold on</code>	Instructs MATLAB not to erase figure contents before adding new information	11.1.2
<code>loglog</code>	Generates an x-y plot, with both axes scaled logarithmically	11.2.5
<code>pie</code>	Generates a pie chart	11.2.5
<code>semilogx</code>	Generates an x-y plot, with the x-axis scaled logarithmically	11.2.5
<code>semilogy</code>	Generates an x-y plot, with the y-axis scaled logarithmically	11.2.5
<code>plot(...)</code>	Creates an x-y plot	11.1.2
<code>polar</code>	Creates a polar plot	11.2.4
<code>legend(ca)</code>	Adds a legend to a graph	11.1.2
<code>shading <spec></code>	Shades a surface according to the specification	11.1.2
<code>subplot(plts, n)</code>	Divides the graphics window into sections available for plotting	11.1.3
<code>text(x,y,{z,} str)</code>	Adds a text string to a graph	11.1.2
<code>title(str)</code>	Adds a title to a plot	11.1.2
<code>view(az,el)</code>	Sets the angle from which to view a plot; the parameters are <code>az</code> , the azimuth, and <code>el</code> , the elevation, both angles measured in degrees	11.1.2
<code>xlabel(str)</code>	Adds a label to the x-axis	11.1.2
<code>ylabel(str)</code>	Adds a label to the y-axis	11.1.2
<code>zlabel(str)</code>	Adds a label to the z-axis	11.1.2



Special Characters, Reserved Words, and Functions—3-D

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>alpha(x)</code>	Sets the transparency of the surfaces; $0 \leq x \leq 1$, where 0 means completely transparent and 1 is opaque	11.3.3
<code>bar3</code>	Generates a three-dimensional bar graph	11.3.3
<code>barh3</code>	Generates a horizontal three-dimensional bar graph	11.3.3
<code>contour</code>	Generates a contour plot	11.4.5
<code>cylinder(n)</code>	Constructs the plaid for a cylinder with <i>n</i> facets	11.4.5
<code>ellipsoid(n)</code>	Constructs the plaid for an ellipsoid with <i>n</i> facets	11.4.5
<code>cylinder(n)</code>	Constructs the plaid for a cylinder with <i>n</i> facets	11.4.5
<code>lightangle(az,el)</code>	Sets the angle of a light source, angles in degrees	11.4.5
<code>mesh(x,y,z)</code>	Generates a mesh plot of a surface	11.4.1
<code>meshc(x,y,z)</code>	Generates a mesh plot of a surface with a contour below it	11.4.5
<code>meshz(x,y,z)</code>	Generates a mesh plot of a surface with vertical line extensions	11.4.5
<code>meshgrid(r,c)</code>	Creates a plaid for 3-D plots	11.4.1
<code>peaks</code>	Creates a sample matrix used to demonstrate graphing functions	11.4.2
<code>pie3</code>	Generates a three-dimensional pie chart	11.3.3
<code>plot3(...)</code>	Generates a three-dimensional line plot	11.3.1
<code>sphere</code>	Example function used to demonstrate graphing	11.4.5
<code>surf(x,y,z)</code>	Generates a surface plot	11.4.1
<code>surfc(x,y,z)</code>	Generates a combination surface and contour plot	11.4.5
<code>waterfall(x,y,z)</code>	Generates a mesh plot of a surface with vertical line extensions in the x direction only	11.4.5



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. The `plot(...)` function needs only one parameter to function correctly.
2. Plot enhancement functions may be called before or after the function that plots the data.

3. You must provide plots for all the specified subplot areas.
4. When multiple data sets are used in a single `plot(...)` or `plot3(...)` call, all the lines are black.
5. `meshgrid(...)` accepts vectors of length `m` and `n` that bound the edges of the plaid and produces two arrays sized $m \times n$ giving the complete plaid.
6. To construct a parametric surface, both independent parameters must be angles.
7. When rotating a function about the y-axis, the variables along the x and y axes are computed from a classic polar-to-Cartesian conversion.
8. To compute a body of rotation, the curve must be a continuous, differentiable function.
9. Bodies of rotation are confined rotating about the x, y, or z axes.

Fill in the Blanks

1. Each time `figure` is called, a _____ is made available, with _____ figure number.
2. To prepare for plotting, put _____ or _____ at the beginning of your script.
3. Parametric plots allow the variables on each axis to be _____ on a _____, _____ variable.
4. The simplest surface plots are obtained by defining a _____ value for each point on _____.
5. We construct a sphere by wrapping a _____ with two _____ as the independent variables around the sphere.
6. Bodies of rotation are created by rotating a _____ about a _____.



Programming Projects

1. Write a script that creates six sub-plots in two columns each with three rows. Each plot should have an appropriate title and labels on the x and y axes. The plot in the top left sub-plot should be $y = \cos(\theta)$ for values of θ from -2π to 2π . Subsequent plots going across the rows before going down the columns should be of $y = 2\cos(\theta)$, $y = 3\cos(\theta)$, etc., to $y = 6\cos(\theta)$ over the same range of θ .
2. Your task is to create a script called `thisPlot`. This script should do the following:
 - a. Ask the user to enter in a positive number, N , greater than 5.
 - b. Calculate the factorial for each number from 1 to N . Each of these values should be stored into a vector.
 - c. Display a graph titled 'Logarithmic Growth', where the logarithms for each of the factorials are displayed.
 - d. Add to the graph a continuous linear line that follows the equation $y = x$ with x values from 1 to N .
 - e. Since the numbers will have different magnitudes, use `plotyy` to plot the linear values on the right hand axis.
3. Write a function called `sineGraph` that graphs a sine function four times between the interval `[start,stop]` on the same graph. The `start` and `stop` values should be parameters of the function. The number of points per interval will vary. More specifically:
 - The first time you graph the sine function, you should have two evenly spaced points, `start` and `stop`
 - The next plot should have 4 evenly spaced points—`start`, `stop`, and two points between them
 - The third should have 8 evenly spaced points and the fourth 256 points.
 - Make sure to add a legend and a title—'Multiple graphs on one plot'—and to label the axes. Make sure each line has a different color.
 - The function should return the x and y values for the 256 point set.

Test your function with the following intervals `[0,pi/2]`, `[0,2pi]`, `[0,4pi]`, `[0,16pi]`

4. This programming problem will compare the `surf(...)` and `mesh(...)` functions by putting two 3-D side by side plots for comparison using `subplot(...)`. You should label all axes accordingly (' x -axis', ' y -axis', etc.) and title your plot corresponding to the problem statement.

- a. On the left side, plot the function $f(x,y)=x^2*\cos(y)$ in the range $x= -5:5$ and $y= -5:5$ using `mesh` and name your plot 'Using Mesh'.
 - b. On the right side, plot the same function, in the same range, but using `surf`. Name your plot 'Using Surf'.
5. Georgia Tech wants to tear down the Campanile and build a new one that is ridiculously tall. However, before they build it, they need you to model it in MATLAB. Using the equation $z = 1/(x^2 + y^2)$ as the model, write a script that will plot the Campanile. Your domain should be $-.75 \leq x \leq .75$ and $-.75 \leq y \leq .75$ using an increment of .05 for each range. Set your axes such that all of the x, y domain is seen and z runs from 0 to 300. Use `surf(...)` to plot your image.
6. You are provided the file 'data.xls', which contains two columns of numbers. Each column contains the same number of elements. The first row contains the titles of the x and y values respectively. Create a script called `excelPlot` that plots the data in this file. The first column represents your x values and the second column is your y values. Read the numbers from the file and make a plot of the x vs. y values. Title your plot 'Excel Plot' and use the first row data to label the x and y axes. For example, the spreadsheet might look like:

	A	B
1	time(sec)	distance(ft)
2	0	0.84
3	0.2	0.23
4	0.4	0.77
5	0.6	1.06
6	0.8	1.28
7	1	1.48
8	1.2	1.63
9	...	...
10	...	...
11	...	...
12		

7. You just realized that February 14th has passed and you haven't gotten anything for your Valentine. Since your date is a CS major, sending the lucky person a MATLAB-coded heart seems like a cool and sincere thing to do. Make sure you follow each and every instruction carefully, or your heart will end up broken. Trust us.

You are going to write a script to draw this heart using the following steps:

- a. Create a plaid `[xx, yy]` using `x` values with range (0 to 2π , with an interval 0.05π) and `y` values with the range (0 to 1, with an interval 0.05).
 - b. Define the following variables:


```
c=[0.1 + 0.9*(pi-abs(xx - pi))/pi] .* yy
aa = c .* cos(xx)
bb = c .* sin(xx)
zz = (-2)*aa.^3 + (3/2)*c.^2 + 0.5
```
 - c. Plot `zz` against `aa` and `bb` using the `surf()` function with interpolated colors.
8. Write a function named `plotRotation` that takes in two vectors, `x` and `z`, and a vector `th`. Your function should plot three plots in the same figure by using the `subplot` command. The figure should have 1×3 plots. The plots should be as follows:
- a. `z` vs. `x`, titled '`z vs. x`'. Note that you will have to use `plot3()` to correctly plot this in the `xz` plane rather than the `xy` plane as `plot()` would do. Also, you should use `view(0, 0)` to make the plot produced by `plot3()` show up as 2D as shown in the example image.
 - b. `z` vs. `x` rotated around the `x` axis using `mesh()` with flat shading and a square axis, titled '`z vs. x about x using mesh`'.
 - c. `z` vs. `x` rotated around the `z` axis using `surf()` with `interp` shading and a square axis, titled '`z vs. x about z using surf`'.

For plots b and c, the input vector `th` should be used for your independent vector `theta` which is used to convert from polar to Cartesian coordinates. Don't forget to title and label each of the three plots.

Matrices

Chapter Objectives

This chapter shows matrices as logical extensions of arrays. You will learn about two specialized operations performed with matrices:

- Multiplication for coordinate rotation
- Division for solving simultaneous equations

Although the matrix operations that are the subject of this chapter can be performed on pairs of vectors or arrays that meet certain criteria, when using these operations, we tend to refer to the data objects as matrices. In most mathematical discussions, the words matrix and array can be used interchangeably, and rightly so, because they store data in exactly the same form. Moreover, almost all of the operations we can perform on an array can also be performed on a matrix—logical operations, concatenation, slicing, and most of the arithmetic operations behave identically. The fact that some of the mathematical operations are defined differently gives us a chance to think about an important concept that will increase in importance when we begin to consider object-oriented programming.

- 12.1** Concept: Behavioral Abstraction
- 12.2** Matrix Operations
 - 12.2.1 Matrix Multiplication
 - 12.2.2 Matrix Division
 - 12.2.3 Matrix Exponentiation
- 12.3** MATLAB Implementation
 - 12.3.1 Matrix Multiplication
 - 12.3.2 Matrix Division
- 12.4** Rotating Coordinates
 - 12.4.1 2-D Rotation
 - 12.4.2 3-D Rotation
- 12.5** Solving Simultaneous Linear Equations
 - 12.5.1 Intersecting Lines
 - 12.5.2 Curve Fitting
- 12.6** Engineering Examples
 - 12.6.1 Ceramic Composition
 - 12.6.2 Analyzing an Electrical Circuit



12.1 Concept: Behavioral Abstraction

Recall the following concepts:

- *Abstraction* is the ability to ignore specific details and generalize the description of an entity
- *Data abstraction* is the specific example of abstraction that we first considered whereby we could treat vectors of data (and later other collections like structures and arrays) as single entities rather than enumerating their elements individually
- *Procedural abstraction* are functions that collect multiple operations into a form; once they are developed, we can overlook the specific details and treat them as a “black box,” much as we treat MATLAB’s built-in functions

Behavioral abstraction combines data and procedural abstraction, encapsulating not only collections of data, but also the operations that are legal to perform on that data. One might argue that this is a new, irrelevant concept best ignored until “we just have to!” But consider the rules we have had to establish for what we can and cannot do with data collections we have seen so far. For example, can I add two arrays together? Yes, but only if they have the same number of rows and columns, or if one of them is a scalar (a 1×1 array). Can I add two character strings? Almost the same answer, except that each string is first converted to a numerical quantity and the result is a vector of numbers and not a string. Can I add two cell arrays? No.

So at least some, and maybe all, MATLAB data collections also “understand” the set of operations that are permitted on the data. This encapsulation of data and operations is the essence of behavioral abstraction. Therefore, we can distinguish arrays from matrices not by the data they collect, but by the operations that are legal to perform on them.



12.2 Matrix Operations

The arithmetic operations that differ between arrays and matrices are multiplication, division, and exponentiation.

12.2.1 Matrix Multiplication

Previously, when we considered multiplying two arrays, we called this scalar multiplication, and it had the following typical array operation characteristics:

- Either the two arrays must be the same size, or one of them must be scalar
- The multiplication was indicated with the `.*` operator

- The result was an array with the same size as the larger original array
- Each element of the result was the product of the corresponding elements in the original two arrays

This is best illustrated in Figure 12.1. Scalar division and exponentiation have the same constraints. Matrix multiplication, on the other hand, performed using the normal `*` operator, is an entirely different logical operation, as shown in Figure 12.2. The logical characteristics of matrix multiplication are as follows:

- The two matrices do not have to be the same size. The requirements are either:
 - One of the matrices is a scalar, in which case the matrix operation reduces to a scalar multiply.
 - The number of columns in the first matrix must equal the number of rows in the second. We refer to these as the **inner dimensions**. The result is a new matrix with the column count of the first matrix and the row count of the second.
- If, as illustrated, A is an $m \times n$ matrix and B is an $n \times p$ matrix, the result of $A * B$ is an $m \times p$ matrix.
- The item at (i, j) in the result matrix is the sum of the scalar product of the i th row of A and the j th column of B .

$$\begin{array}{c}
 A_{(m \times n)} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & & \ddots & \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \quad . * \quad B_{(n \times p)} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ \vdots & & \ddots & \\ b_{m1} & b_{m2} & \dots & b_{mn} \end{bmatrix} \\
 \\
 \longrightarrow \begin{bmatrix} a_{11} \times b_{11} & a_{12} \times b_{12} & \dots & a_{1n} \times b_{1n} \\ a_{21} \times b_{21} & a_{22} \times b_{22} & \dots & a_{2n} \times b_{2n} \\ \vdots & & \ddots & \\ a_{m1} \times b_{m1} & a_{m2} \times b_{m2} & \dots & a_{mn} \times b_{mn} \end{bmatrix}
 \end{array}$$

Figure 12.1 Matrix dot multiply

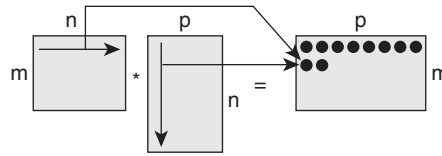


Figure 12.2 Mechanics of matrix multiplication

- Whereas with scalar multiplication $A \cdot B$ gives the same result as $B \cdot A$, this is not the case with matrix multiplication. In fact, if $A \cdot B$ works, $B \cdot A$ will not work unless both matrices are square, and even then the results are different. (Proof of this can be derived immediately from Figure 12.3 by eliminating the third row and column and exchanging a for b . All four terms of the result of $A \cdot B$ are different from $B \cdot A$.)
- Whereas with scalar multiplication the original array A can be recovered by dividing the result by B , this is not the case with matrix multiplication unless both matrices are square.
- The **identity matrix**, sometimes given the symbol I_n , is a square matrix with n rows and n columns that is zero everywhere except on its major diagonal, which contains the value 1. I_n has the special property that when pre-multiplied by any matrix A with n columns, or post-multiplied with any matrix A with n rows, the result is A . We will need this property to derive matrix division below. (The MATLAB function `eye(...)` generates the identity matrix.)

Figure 12.3 illustrates the mathematics for the case where a 3×2 matrix is multiplied by a 2×3 matrix, resulting in a 3×3 matrix.

$$\begin{aligned}
 A_{(m \times n)} &= \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{m1} & a_{m2} \end{bmatrix} * B_{(n \times p)} = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix} \\
 \longrightarrow & \begin{bmatrix} (a_{11} \times b_{11} + a_{12} \times b_{21}) & (a_{11} \times b_{12} + a_{12} \times b_{22}) & (a_{11} \times b_{13} + a_{12} \times b_{23}) \\ (a_{21} \times b_{11} + a_{22} \times b_{21}) & (a_{21} \times b_{12} + a_{22} \times b_{22}) & (a_{21} \times b_{13} + a_{22} \times b_{23}) \\ (a_{31} \times b_{11} + a_{32} \times b_{21}) & (a_{31} \times b_{12} + a_{32} \times b_{22}) & (a_{31} \times b_{13} + a_{32} \times b_{23}) \end{bmatrix}
 \end{aligned}$$

Figure 12.3 Matrix multiplication

12.2.2 Matrix Division

Matrix division is the logical process of reversing the effects of a matrix multiplication. The goal is as follows: given $A_{n \times n}$, $B_{n \times p}$, and $C_{n \times p}$, where $C = A * B$, we wish to define the mathematical equivalent of C/A that will result in B .

Since $C = A * B$, we are actually searching for some matrix $K_{n \times n}$ by which we can multiply each side of the above equation:

$$K * C = K * A * B$$

This multiplication would accomplish the division we desire if $K * A$ were to result in I_n , the identity matrix. If this were the case, pre-multiplying C by K would result in $I_n * B$, or simply B by the definition of I_n above. The matrix K is referred to as the inverse of A , or A^{-1} . The algebra for computing this inverse is messy but well defined. In fact, Gaussian Elimination to solve linear simultaneous equations accomplishes the same thing. MATLAB has both functions (`inv(A)`) and operators (“back divide,” `\`) that accomplish this. However, two things should be noted:

- This inverse does not exist for all matrices—if any two rows or columns of a matrix are linearly related, the matrix is singular and does not have an inverse
- Only nonsingular, square matrices have an inverse (just as a set of linear equations is soluble only if there are as many equations as there are unknown variables)

12.2.3 Matrix Exponentiation

For completeness, we mention here that matrix operations include exponentiation. However, this does not suggest that one would encounter $A_{n \times n}^{B_{n \times n}}$ in the scope of our applications. Rather, our usage of matrix exponentiation will be confined to A^k where k is any non-zero integer value. The result for positive k is accomplished by multiplying A by itself k times (using matrix multiplication). The result for negative k is accomplished by inverting A^{-k} . (There is, in fact, meaning in matrix exponentials with non-scalar exponents, but this involves advanced concepts with eigen values and eigenvectors. See the MATLAB help files for more detail.)



12.3 MATLAB Implementation

In this section, we see how MATLAB implements matrix multiplication and division. However, since applications that require matrix exponentiation are beyond the scope of this text, we will not look at its implementation in MATLAB.

12.3.1 Matrix Multiplication

Matrix multiplication is accomplished by using the “normal” multiplication symbol, as illustrated in Exercise 12.1.



Exercise 12.1 Matrix multiply

Do It Yourself

In the Command window, enter the following:

```
1. >> A = [2 5 7; 1 3 42]
A =
     2     5     7
     1     3    42
2. >> B = [1 2 3]
B =
     1
     2
     3
3. >> A * B
ans =
     33
    133
4. >> (1:2) * A
ans =
     4    11    91
5. >> I2 = eye(2)
I2 =
     1     0
     0     1
6. >> I2 * A
ans =
     2     5     7
     1     3    42
7. >> A*I2
??? Error using ==> mtimes
Inner matrix dimensions must agree.
8. >> A*eye(3)
ans =
     2     5     7
     1     3    42
```

In Exercise 12.1 we make the following observations:

- Entry 1 creates a 2×3 matrix, **A**
- Entry 2 creates a 3×1 matrix, **B**, a column vector
- Entry 3 indicates that this multiplication is legal because the columns in **A** match the rows in **B**
- Entry 4 shows that, likewise, it is legal to multiply a 1×2 vector by a 2×3 matrix
- Entry 5 creates an identity matrix
- Entry 6 shows that pre-multiplying **A** by this is legal because the inner dimensions match
- Entry 7 shows that post-multiplying **A** by $\mathbf{I}_2$ does not work because the inner dimensions do not match
- Entry 8 uses $\mathbf{I}_3$ to post-multiply legally

12.3.2 Matrix Division

Matrix division is accomplished in a number of ways, all of which are equivalent. Returning to the division problem described in Section 12.2.2, we know that A , B , and C are square matrices and $C = A * B$. If we are actually given the matrices C and B , we can compute A in one of the following ways:

- $A = C * \text{inv}(B)$ —using the MATLAB `inv(...)` function to compute the inverse of B
- $A = B \setminus C$ —“back dividing” B into C to produce the same result
- $A = C / B$ —apparently performing the same operation, but giving different answers

According to the MATLAB help system, the third way really computes $(C' \setminus B')'$.

The order in which the matrix multiply is done affects the value of the result; therefore, care must be taken to ensure that the appropriate inversion or division is used. Both multiplying by the inverse and the normal divide nullify the effect of the array that did the post-multiplication. Back dividing nullifies the effect of the matrix that did the pre-multiplication. See Exercise 12.2.



Exercise 12.2 Matrix divide

Do It Yourself

Create a script containing the following:

```
1. >> A = magic(3)
A =
     8     1     6
     3     5     7
     4     9     2
2. >> B = [1 26 24; 9 22 20; 5 12 16]
B =
     1    26    24
     9    22    20
     5    12    16
3. >> AB = A * B
AB =
    47    302    308
    83    272    284
    95    326    308
4. >> BA = B * A
BA =
    182    347    236
    218    299    248
    140    209    146
5. >> AB * inv(B)
ans =
    8.0000    1.0000    6.0000
    3.0000    5.0000    7.0000
    4.0000    9.0000    2.0000
```

continued on next page

```

6. >> AB / B
ans =
    8.0000    1.0000    6.0000
    3.0000    5.0000    7.0000
    4.0000    9.0000    2.0000
7. >> B \ BA
ans =
    8.0000    1.0000    6.0000
    3.0000    5.0000    7.0000
    4.0000    9.0000    2.0000
8. >> BA / B
ans =
   -4.3000   29.2000  -15.3000
   -9.9667   27.5333   -3.9667
   -5.7333   20.7667   -8.2333

```

In Exercise 12.2 we make the following observations:

- Entries 1 and 2 construct two 3×3 matrices, A and B
- Entries 3 and 4 pre-multiply and post-multiply B and A ; recall that we expect this to produce different answers
- Entry 5 shows that since we defined $\text{inv}(B)$ as that function that produces the result $B * \text{inv}(B) = I$, this should produce a matrix with the same values as A
- Entry 6 reveals that normal division by B should also produce a matrix with the same values as A
- Entry 7 shows that back dividing B into BA should also produce a matrix equal to A
- Entry 8 verifies that dividing BA by B works but does not return the matrix A



12.4 Rotating Coordinates

A common use for matrix multiplication is for rotating coordinates in two or three dimensions. Previously we have seen the ability to rotate a complete picture by changing the viewing angle. We can move and scale items on a plot by adding coordinate offsets or multiplying them by scalar quantities. However, frequently the need arises to rotate the coordinates of a graphical object by some angle. We can use matrix multiplication to rotate individual items in a picture in two or three dimensions.

12.4.1 2-D Rotation

The mathematics implementing rotation in two dimensions is relatively straightforward, as shown in Figure 12.4. If the original point location is (x, y) at angle α to the x axis and you wish to rotate that point by the angle θ about the origin of coordinates, the mathematics are as follows:

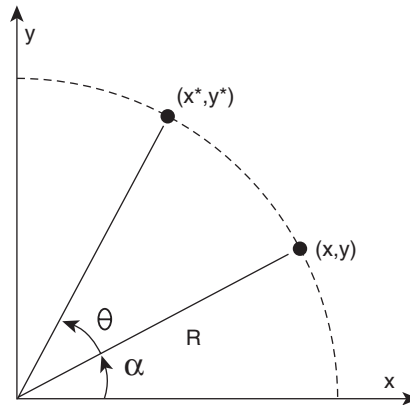


Figure 12.4 Rotating Cartesian coordinates

$$x^* = x \cos \theta - y \sin \theta$$

$$y^* = x \sin \theta + y \cos \theta$$

This can be expressed as a matrix multiplication as follows:

$$\begin{bmatrix} x^* \\ y^* \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

To rotate the x-y coordinates of a graphic object in the x-y plane about some point, P, other than the origin, you would do as follows:

1. Translate the object so that P is at the origin by addition
2. Perform the rotation by matrix multiplication
3. Translate the rotated object back to P by subtraction

Rotating a Line Listing 12.1 illustrates a simple script to rotate a line about the origin. It uses the function shown in Listing 12.2 to build the appropriate rotation matrix.

Listing 12.1 Script to rotate a line

```

1. pts = [3, 10
2.       1, 3];
3. plot(pts(1,:), pts(2,:))
4. axis ([0 15 0 15])
5. hold on
6. for angle = 0.05:0.05:1
7.     A = rotation(angle);
8.     pr = A * pts;
9.     plot(pr(1,:), pr(2,:))
10. end

```

Listing 12.2 Compute a rotation matrix

```

1. function transformation = rotation(angle)
2. % rotate by the angle (radians)
3. transformation = [ cos(angle), -sin(angle)
4.                  sin(angle),  cos(angle) ];

```

In Listing 12.1:

Lines 1 and 2: Considering the form of the rotation equations, we need to define the points where the x values are in the first row and the y values are in the second row.

Line 3: Plots the line in its original location from (3, 1) to (10, 3).

Lines 4 and 5: Fix the axes at a suitable size.

Line 6: Iterates across a selection of angles (in radians).

Line 7: Uses the function in Listing 12.2 to define the matrix of rotation.

Line 8: Rotates the original line by the current angle.

Line 9: Plots the rotated line.

Figure 12.5 shows the plot resulting from this script.

Twinkling Stars As a second example, consider the problem of simulating twinkling stars. One way to accomplish this is to draw two triangles for each star rotating in opposite directions. The script shown in Listing 12.3 accomplishes this.

In Listing 12.3:

Line 1: Sets the number of stars and the initial rotation angle.

Lines 2–6: Establish the location, size, and rotation speed of each star.

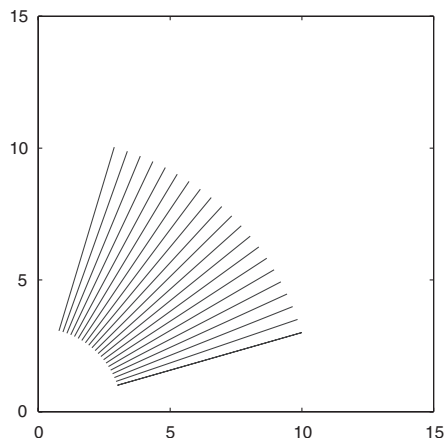


Figure 12.5 *Line rotations*

Listing 12.3 Simulating stars

```

1. nst = 20; th = 0;
2. for ndx = 1:nst
3.     pos(ndx,:) = rand(1,2)*10;
4.     scale(ndx) = rand(1,1) * .9 + .1;
5.     rate(ndx) = rand(1,1) * 3 + 1;
6. end
7. while true
8.     for str = 1:nst
9.         star(pos(str,:), ... % location
10.            scale(str), ... % scale
11.            th, ... % basic angle
12.            rate(str)) % angle multiplier
13.     end
14.     colormap autumn
15.     axis equal; axis([-0.5 10.5 -0.5 10.5])
16.     axis off; hold off
17.     th = mod(th + .1, 20*pi);
18.     pause(0.1)
19. end

```

Line 7: Continues drawing until interrupted by Ctrl-C.

Lines 8–13: Draw each star at the current rotation (see Listing 12.4 for the `star(...)` function).

Line 14: Chooses a color map with yellow as the first color.

Lines 15 and 16: Show the normal display environment setup.

Line 17: Updates the angle of rotation.

Line 18: Waits for the figure to be displayed.

Listing 12.4 Drawing one star

```

1. function star(pt, sc, v, th)
% draw a star at pt(1), pt(2),
%     scaled with sc, at angle v*th

2. triangle(1, v*th, pt, sc)
3. hold on
4. triangle(-1, v*th, pt, sc)

5. function triangle( up, th, pt, sc )
6. pts = [-.5    .5    0    -.5; % x values
7.        -.289 -.289 .577 -.289]; % y values
% rotation matrix
8. A = sc * [cos(th), -sin(th); sin(th), cos(th)];
9. thePts = A * pts;
10. fill( thePts(1,:) + pt(1), ...
11.      up*thePts(2,:) + pt(2), 1);

```

In Listing 12.4:

Line 1: Draws one star at location $[pt(1), pt(2)]$ with scale sc , rotation speed v , and angle th .

Lines 2–4: Invoke the helper function `triangle(...)` to draw two triangles rotating in opposite directions.

Line 5: Draws one triangle with the following parameters: up , with values 1 for upright and -1 for point down; th , the scaled rotation angle; and pt and sc , which are passed directly through from the `star(...)` function.

Lines 6 and 7: Are coordinates of an equilateral triangle.

Line 8: Computes the rotation matrix and applies the scaling factor.

Line 9: Rotates and scales the points of the triangle.

Lines 10 and 11: Call the function `fill(...)` to fill the triangle, offsetting the x and y coordinates by the original location of the triangle, and scaling y by the up multiplier to invert the triangle if necessary.

The results of this script are shown in Figure 12.6.

12.4.2 3-D Rotation

The mathematics implementing rotation in three dimensions is a natural extension of the 2-D rotation case. We present here a simple way to make this extension. The 2-D rotation in Section 12.4.1 that rotates by the angle θ in the x - y plane is actually rotating about the z axis. The 3-D equivalent of this transformation is as follows:

$$\begin{bmatrix} x^* \\ y^* \\ z^* \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

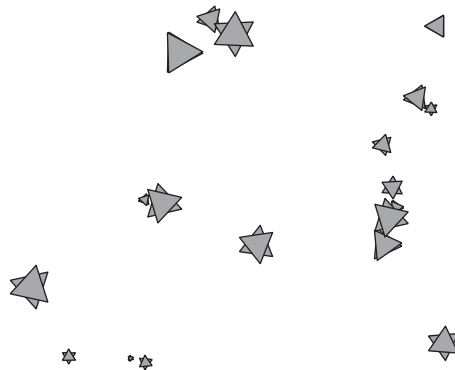


Figure 12.6 Stars

We can establish similar transformations by the angle ϕ about the y-axis, and ψ about the x-axis as follows:

$$\begin{bmatrix} x^* \\ y^* \\ z^* \end{bmatrix} = \begin{bmatrix} \cos \phi & 0 & -\sin \phi \\ 0 & 1 & 0 \\ \sin \phi & 0 & \cos \phi \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

and:

$$\begin{bmatrix} x^* \\ y^* \\ z^* \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \psi & -\sin \psi \\ 0 & \sin \psi & \cos \psi \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

Once these matrices are constructed, the rotations can be combined by multiplying the matrices together:

$$x^* = R_x * R_y * R_z * x$$

where R_x , R_y , and R_z are the matrices of rotation about the x, y, and z axes, respectively.

An example of a script to rotate the solid cube drawn in Chapter 11 is shown in Listing 12.5. The major problem with rotating solid objects is that the coordinates of the object are defined as arrays of points. However, the rotation matrices need each set of coordinates in single rows. To accomplish this, we will use the `reshape(...)` function to translate the coordinates to and from the row vectors necessary for the coordinate rotation.

Listing 12.5 Rotating a solid cube

```

1. xx = [ 0  0  0  0  0  0;
2.      -1 -1  1  1 -1;
3.       -1 -1  1  1 -1;
4.        0  0  0  0  0]
5. yy = [ 0  0  0  0  0;
6.       -1  1  1 -1 -1;
7.       -1  1  1 -1 -1;
8.        0  0  0  0  0]
9. zz = [ 1  1  1  1  1;
10.      1  1  1  1  1;
11.     -1 -1 -1 -1 -1;
12.     -1 -1 -1 -1 -1]
13. [r c] = size(xx);
14. ln = r*c; % length of reshaped vector
15. th = 0; ph = 0; ps = 0;
16. dth = 0.05; dph = 0.03; dps = 0.01;
17. go = true
18. while go
19.     surf(xx+4, yy, zz)
20.     shading interp; colormap autumn

```

continued on next page


```

21.     hold on; alpha(0.5)
22.     Rz = [cos(th) -sin(th)  0
23.           sin(th)  cos(th)  0
24.           0         0      1];
25.     Ry = [cos(ph)  0  -sin(ph)
26.           0         1      0
27.           sin(ph)  0   cos(ph)];
28.     Rx = [ 1         0         0
29.           0         cos(ps) -sin(ps)
30.           0         sin(ps)  cos(ps)];
31.     P(1,:) = reshape(xx, 1, ln);
32.     P(2,:) = reshape(yy, 1, ln);
33.     P(3,:) = reshape(zz, 1, ln);
34.     Q = Rx*Ry*Rz*P;
35.     qx = reshape(Q(1,:), r, c);
36.     qy = reshape(Q(2,:), r, c);
37.     qz = reshape(Q(3,:), r, c);
38.     surf(qx, qy, qz)
39.     shading interp
40.     axis equal; axis off; hold off
41.     axis([-2 6 -2 2 -2 2])
42.     lightangle(40, 65); alpha(0.5)
43.     th = th+dth; ph = ph+dph; ps = ps+dps;
44.     go = ps < pi/4
45.     pause(0.03)
46. end

```

In Listing 12.5:

Lines 1–12: Build the coordinates of the cube centered at the origin.

Lines 13 and 14: Determine the length of the linearized row vector for the `reshape(...)` function.

Lines 15 and 16: Set up the three rotation angle parameters—the initial values and the increments.

Lines 17 and 18: Repeat the drawing loop until the variable `go` is reset.

Lines 19–21: Draw one cube not rotated four units down the x-axis.

Lines 22–30: Set up the rotation matrices.

Lines 31–33: Reshape the `x`, `y`, and `z` arrays into linear form.

Line 34: Performs the rotation.

Lines 35–37: Reshape the original arrays.

Lines 38–42: Draw the rotated cube.

Line 43: Updates the rotation angles.

Line 44: Shows the terminating condition.

Line 45: Pauses to give the figure time to draw.

The results after running this script are shown in Figure 12.7. Notice that the mechanization of the top face has caused a “wrapped parcel” effect on the light reflections off that surface.

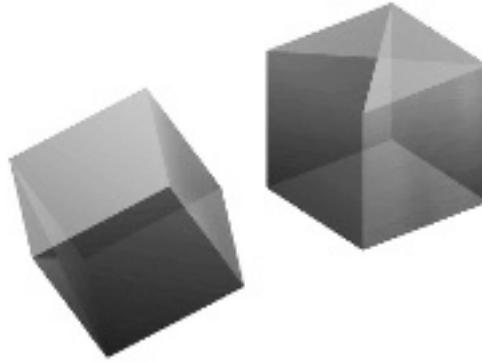


Figure 12.7 Solid cubes



12.5 Solving Simultaneous Linear Equations

A common use for matrix division is solving simultaneous linear equations. To be soluble, simultaneous linear equations must be expressed as N independent equations involving N unknown variables, x_i . The equations will also involve $N * (N + 1)$ constant values, A_{ij} and c_i , where i and j each take on values from 1 to N . They are usually expressed in the following form:

$$A_{11} x_1 + A_{12} x_2 + \dots + A_{1N} x_N = c_1$$

$$A_{21} x_1 + A_{22} x_2 + \dots + A_{2N} x_N = c_2$$

$$\cdot \quad \cdot \quad \cdot \quad \cdot$$

$$\cdot \quad \cdot \quad \cdot \quad \cdot$$

$$A_{N1} x_1 + A_{N2} x_2 + \dots + A_{NN} x_N = c_N$$

In matrix form, they can be expressed as follows:

$$A_{N \times N} * X_{N \times 1} = C_{N \times 1}$$

from which, since all of the values in A and C are constants, we can immediately solve for the column vector X by back division:

$$X = A \backslash C$$

or by using the matrix inverse function:

$$X = \text{inv}(A) * C$$

12.5.1 Intersecting Lines

A typical example of a simultaneous equation problem might take the following form. Consider two straight lines on a plot with the following general form:

$$A_{11}x + A_{12}y = c_1$$

$$A_{21}x + A_{22}y = c_2$$

These lines intersect at some point $[x, y]$ that is the solution to both of these equations. The equations can be rewritten in matrix form as follows:

$$A * V = c$$

where c is the column vector $[c_1 \ c_2]'$ and v is the required result, the column vector $[x \ y]'$. The solution is obtained by eliminating A from the left side of the equation using the back divide operator as follows:

$$V = A \setminus c$$

Recall that back divide, like the `inv(...)` function, will fail to produce a result if the matrix is singular, that is, has two rows or columns that have a linear relationship. In the specific example of two intersecting lines, this singularity occurs when the two lines are parallel, in which case there is no point of intersection. Listing 12.6 shows one solution to a pair of simultaneous equations.

Listing 12.6 Plotting line intersections

```

1. % equations are y = m1 x + c1
2. %           y = m2 x + c2
3. %   in matrix form:
4. %   [ -m1 1;   * [xp;  = [c1
5. %       -m2 1 ]   yp]   c2]
6. ax = [-0.5 6]; ay = [-4.5 18];
7. % plot the two lines
8. m1 = 3; c1 = -2;
9. y1 = m1*ax + c1;
10. m2 = -2; c2 = 9;
11. y2 = m2*ax + c2;
12. plot(ax, y1)
13. hold on
14. plot(ax, y2, 'b-')
15. % solve for the intersection point
16. A = [-m1 1; -m2 1];
17. c = [c1; c2];
18. P = A\c;
19. % draw intersection identification lines
20. ix = P(1); iy = P(2);
21. plot([ix ix], [0 iy*1.2], 'r:')
22. plot([0 ix*1.2],[iy iy], 'r:')
23. % draw the axes
24. plot(ax, [0 0], 'k');

```

continued on next page

```

25. axis([ax ay])
26. plot([0 0], ay, 'k');
27. legend({'Line 1','Line 2','Intersect'}, ...
         'Location','NorthWest' )

```

In Listing 12.6:

Line 6: Sets the x and y limits of the plot.

Lines 8–14: Plot the original lines.

Line 16: Sets the simultaneous equation matrix.

Line 17: Shows the right-hand side of the equation.

Line 18: Solves the linear equations—P(1) is the x value; P(2) is the y value.

Lines 21 and 22: Plot the lines identifying the intersection point.

Lines 24–27: Complete the plot.

Figure 12.8 shows the result of this script.

12.5.2 Curve Fitting

A specific example of simultaneous equation solution occurs when seeking the best polynomial fit to a set of data points. This fitting a polynomial equation of order n to a set of x and y values involves a search for the $n + 1$ coefficients C_1 of x^n through x^0 that minimize the sum of the squares of the difference between actual y values and the y values of the polynomial at the given x values. The polynomial is of the form:

$$y = C_n x^n + C_{n-1} x^{n-1} \dots + C_1 x + C_0$$

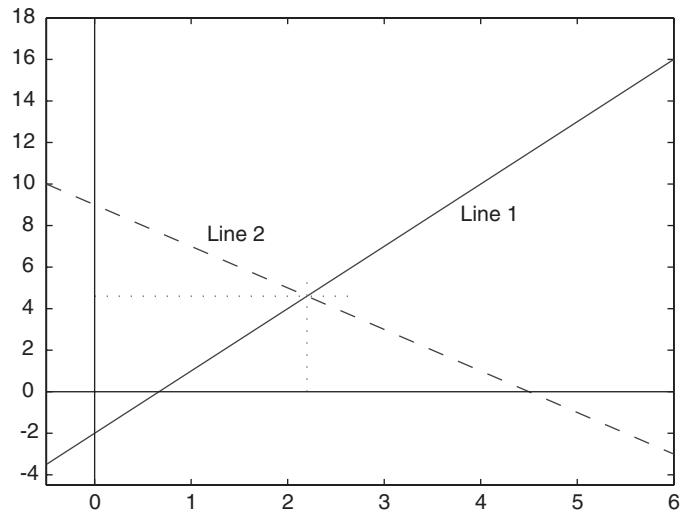


Figure 12.8 Lines intersecting

The solution, found in any good calculus book, produces the following set of simultaneous equations:

$$\begin{bmatrix} \Sigma x^{2n} & \Sigma x^{2n-1} & \dots & \Sigma x^{n+2} & \Sigma x^{n+1} & \Sigma x^n \\ \Sigma x^{2n-1} & \Sigma x^{2n-2} & \dots & \Sigma x^{n+1} & \Sigma x^n & \Sigma x^{n-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ \Sigma x^{n+1} & \Sigma x^n & \dots & \Sigma x^3 & \Sigma x^2 & \Sigma x \\ \Sigma x^n & \Sigma x^{n-1} & \dots & \Sigma x^2 & \Sigma x & n \end{bmatrix} * \begin{bmatrix} C_n \\ C_{n-1} \\ \vdots \\ C_1 \\ C_0 \end{bmatrix} = \begin{bmatrix} \Sigma x^n y \\ \Sigma x^{n-1} y \\ \vdots \\ \Sigma xy \\ \Sigma y \end{bmatrix}$$

This is an equation of the general form $A_{(n+1) \times (n+1)} * C_{(n+1) \times 1} = B_{(n+1) \times 1}$. To solve this, we merely perform the back divide in the usual way:

`C = A \ B`

The example shown in Listing 12.7 illustrates the computation of a linear fit to a set of data. It reads a file of fuel consumption records and computes the curve fit of order 1, a linear curve fit. The resulting plot is shown in Figure 12.9; the crosses show the raw data, and the line is the least squares linear fit.

In Listing 12.7:

Line 3: Reads in the data.

Lines 4 and 5: Extract the data.

Lines 6–10: Plot the original data as red x's.

Line 11: Builds the simultaneous equation matrix.

Line 12: Builds the right-hand side of the equation.

Line 13: Solves for the coefficients. The best fit is

`y = a(1) * x + a(2).`

Listing 12.7 Linear curve fit

```
1. % the file gasoline.dat contains two data
2. % columns:  the miles driven and the gas used.
3. r = dlmread('gasoline.dat');
4. x = r(:,2)';
5. y = r(:,1)';
6. plot(x, y, 'rx');
7. hold on
8. title('gas vs miles');
9. ylabel('miles');
10. xlabel('gallons');
11. c = [ sum(x.^2) sum(x); sum(x) 1];
12. rhs = [sum(x.*y); sum(y)]
13. a = c \ rhs;
14. plot(x, a(1)*x + a(2))
15. fprintf('average mpg = %f\n', a(1) );
```

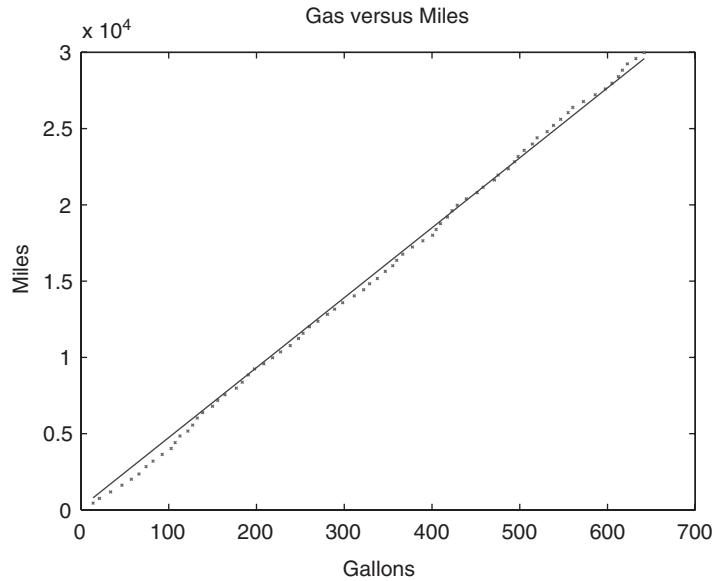


Figure 12.9 *Linear curve fit*

Line 14: Plots the best fit line.

Line 15: The slope happens to be the average fuel consumption.

As we will see later, MATLAB has built-in functions that perform polynomial curve fits.



12.6 Engineering Examples

The following examples illustrate applications of the matrix capabilities discussed in this chapter.

12.6.1 Ceramic Composition

Industrial ceramics plants require mixtures with precise formulations in order to produce products of consistent quality. For example, a factory might require 100 kg of a mix consisting of 67 percent silica, 5 percent alumina, 2 percent calcium oxide, and 26 percent magnesium oxide. However, the raw material provided is not pure quantities of these materials. Rather, they are delivered as batches of material that consist of the required components in different proportions. Each batch of raw materials is analyzed to determine their composition, and we will need to do the analysis to determine the proportions of the raw materials to mix in order to accomplish the appropriate formulation. The raw materials we will use here are feldspar, diatomite, magnesite, and talc. Table 12.1 illustrates a typical analysis of the composition of these compounds.

Table 12.1 Compound compositions				
	Silica	Alumina	CaO	MgO
Feldspar	0.6950	0.1750	0.0080	0.1220
Diatomite	0.8970	0.0372	0.0035	0.0623
Magnesite	0.0670	0.0230	0.0600	0.8500
Talc	0.6920	0.0160	0.0250	0.2670

For example, if we mixed W_f kg of feldspar, W_d kg of diatomite, W_m kg of magnesite, and W_t kg of talc, the amount of silica would be $0.695 W_f + 0.897 W_d + 0.067 W_m + 0.692 W_t$. Repeating this equation for the other components produces a matrix equation that reduces to:

$$\begin{bmatrix} 67 \\ 5 \\ 2 \\ 26 \end{bmatrix} = \begin{bmatrix} 0.695 & 0.897 & 0.067 & 0.692 \\ 0.175 & 0.0372 & 0.023 & 0.016 \\ 0.008 & 0.0035 & 0.06 & 0.025 \\ 0.122 & 0.0623 & 0.025 & 0.267 \end{bmatrix} * \begin{bmatrix} W_f \\ W_d \\ W_m \\ W_t \end{bmatrix}$$

which is an equation of the form:

$$C = A * W$$

We find the appropriate amounts of the raw material by solving these equations:

$$W = A \backslash B$$

A script that works this problem is shown in Listing 12.8.

In Listing 12.8:

- Lines 1–4: Matrix A is the transpose of the original data table.
- Line 5: Shows the required composition in kg.
- Line 6: Shows the computed weights of the raw materials in kg, which produces the following result:

$$W = \begin{bmatrix} 16.0083 & 35.3043 & 15.1766 & 33.5108 \end{bmatrix}$$

Listing 12.8 Analyzing ceramic composition

```
1. A = [0.6950    0.8970    0.0670    0.6920
2.      0.1750    0.0372    0.0230    0.0160
3.      0.0080    0.0035    0.0600    0.0250
4.      0.1220    0.0623    0.8500    0.2670]

5. B = [67 5 2 26]

6. W = (inv(A) * B)'
```

12.6.2 Analyzing an Electrical Circuit

Figure 12.10 illustrates a typical electrical circuit with two voltage sources connected to five resistors with three closed loops. The voltages and resistances are given. We are asked to determine the voltage drop across R_1 . Solution techniques apply Ohm's Law to the voltage drops around each closed circuit. When this technique is applied, the equations are as follows:

$$\begin{aligned} V_1 &= i_1 * R_1 + (i_1 - i_2) * R_4 \\ 0 &= i_2 * R_2 + (i_2 - i_3) * R_5 + (i_2 - i_1) * R_4 \\ -V_2 &= i_3 * R_3 + (i_3 - i_2) * R_5 \end{aligned}$$

When these three equations are manipulated to isolate the three currents, we have the following matrix equation:

$$\begin{bmatrix} V_1 \\ 0 \\ -V_2 \end{bmatrix} = \begin{bmatrix} R_1 + R_4 & -R_4 & 0 \\ -R_4 & R_2 + R_4 + R_5 & -R_5 \\ 0 & -R_5 & R_3 + R_5 \end{bmatrix} * \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix}$$

which is in the form $V = A * i$, and can be solved as usual by:

$$i = A \setminus V$$

The script to accomplish this is shown in Listing 12.9.

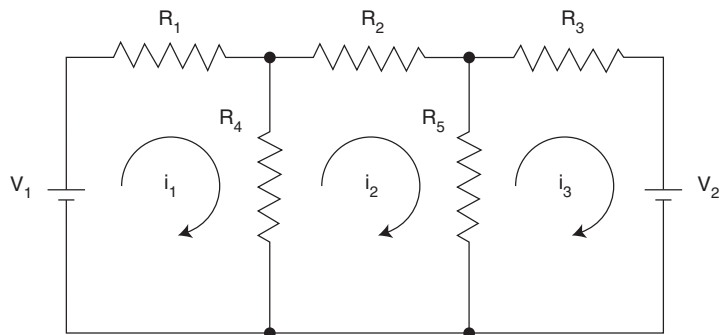


Figure 12.10 Typical electrical circuit

Listing 12.9 Analyzing an electrical circuit

```

1. R1 = 100; R2 = 200; R3 = 300
2. R4 = 400; R5 = 500
3. V1 = 10; V2 = 5
4. A = [R1+R4  -R4      0
5.      -R4   R2+R4+R5 -R5
6.       0    -R5     R3+R5]
7. B = [V1; 0; -V2]
8. curr = inv(A) * B
9. fprintf('drop across R1 is %6.2f volts\n', ...
10.        curr(1) * R1 );
```


In Listing 12.9:

Lines 1–3: Set up the parameters of the problem.

Lines 4–7: Set up the coefficient matrices.

Lines 8–10: Solve the problem.

Running this script produces the following printout:

```
curr =
    0.0283
    0.0104
    0.0003
drop across R1 is    2.83 volts
```


Chapter Summary

This chapter presented two specialized operations performed with matrices:

- Matrix multiplication can be used for two- and three-dimensional coordinate rotations by building the appropriate rotation matrices
- Matrix division is used for solving simultaneous equations by setting up the equations in the general form $B = A * x$, where the known matrix A is $n \times n$ and the known column vector B is $n \times 1$; the unknown vector x is then found by $x = A \backslash B$ or $x = \text{inv}(A) * B$

Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
*	Matrix multiplication	12.2.1
/	Matrix division	12.2.2
\	Matrix back division	12.2.2
^	Matrix exponentiation	12.2.3
eye(n)	Computes the identity matrix	12.2.1
inv(a)	Computes the inverse of a matrix	12.2.3
reshape(a,r,c)	Changes the row/column configuration of the array a	12.4.2



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. All MATLAB classes exhibit some form of behavioral abstraction.
2. Matrix multiplication requires that the inner dimensions match.
3. The results of $A * B$ and $B * A$ are identical.
4. Both $A * A^{-1}$ and $A^{-1} * A$ return the identity matrix.
5. Multiplying $\text{inv}(A) * B$ is logically equivalent to B / A .
6. Fitting a polynomial to a set of data always reduces to a set of simultaneous linear equations.

Fill in the Blanks

1. Behavioral abstraction combines _____ abstraction and _____ abstraction.
2. The result of a matrix multiplication is a new matrix with the _____ count of the first matrix and the _____ count of the second.
3. To rotate a graphic object in the x-y plane about some point, P, other than the origin, you first _____, then _____, and then _____.
4. To be soluble, simultaneous linear equations must be expressed as _____ equations involving _____ variables, x_i , and _____ values.



Programming Projects

1. This is a set of simple matrix manipulations.
 - a. Create a five by six matrix, A, that contains random numbers between 0 and 10.
 - b. Create a six by five matrix, B, that contains random numbers between 0 and 10.

- c. Find the inverse of matrix $A \cdot B$ and store it in the variable, c .
- d. Without iteration, create a new matrix D that is the same as A except that all values less than 5 are replaced by zero.
- e. Using iteration, create a new matrix F that is the same as A except that all values less than 5 are replaced by zero.
- f. Create a new matrix G that is the matrix A with the columns reversed.

For example:

if A is $\begin{bmatrix} 1 & 2 & 3; & 3 & 2 & 5; & 1 & 7 & 4 \end{bmatrix}$, G should be
 $\begin{bmatrix} 3 & 2 & 1; & 5 & 2 & 3; & 4 & 7 & 1 \end{bmatrix}$

- g. Find the minimum value amongst all the elements in A and store your answer in the variable n .
2. World leaders have decided to come up with a single currency for the world. This new currency, called the Eullar, is defined by the following:

Seven dollars and three Euros make seventy-one Eullars.
 One dollar and two Euros make twenty Eullars.

You are a reputed economist and your job is to find out the value of a dollar in terms of Eullars.

3. Write a function called `determinant`. This function will take in a 3×3 matrix of numbers and return the determinant of the matrix. The formula for the determinant of a 3×3 matrix is as follows:

Given a matrix in the form

$$M = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$$

The determinant can be found given the following formula

$$D = m_{11}(m_{22}m_{33} - m_{32}m_{23}) - m_{12}(m_{21}m_{33} - m_{31}m_{23}) + m_{13}(m_{21}m_{32} - m_{31}m_{22})$$

Note: For this problem, you may make use of the `det` function. You may not use the `det` function on the original matrix M , passed into your function; however you can break the matrix into smaller units and then use the `det` function to solve for the determinant of those individual matrices. In short, you may not use the `det` function to do all the work for you.

4. As the enthusiastic and motivated student that you are, you decided to go out and buy plenty of pens for all your classes this semester. This spending spree unfortunately occurred before you

realized your engineering classes seldom required the use of “ink”. So now, you’re left with four different types of pens and no receipt—you only remember the total amount you spent, and not the price of each type of pen. You decide to get together with three of your friends who coincidentally did the same thing as you, buying the same four types of pens and knowing only the total amount.

In order to find the price of each individual pen, you create a matrix called “pens”, where each column represents a different type of pen and each row represents a different person.

```
pens = [3  6  2  5    % you
        4  7  5  2    % friend 1
        1  3 12  6    % friend 2
        2  8  2  4 ] % friend 3
```

You then generate a column vector `totals` that contains the amount of money each of you spent on the pens:

```
totals = [19.60 18.78 25.59 19.26]'
```

Write a script to find the prices of each type of pen.

5. Write a function called `rotateLine` that takes in two vectors, `x` and `y`, of the same length that represent a set or ordered pairs that could be used to plot a line. Your function should also take in a third parameter, `theta`, representing an angle in degrees. Your function should return `xprime` and `yprime` where `xprime` and `yprime` represent the line that is `x` and `y` rotated about the origin by the angle `theta`.

Hint

Use the 2-D rotation matrix found in section 12.4.1.

For example:

```
x = [ 7 7 11 11 7];
y = [-5 -9 -9 -5 -5];
[xprime yprime] = rotateLine(x, y, 90) returns
xprime = [5      9      9      5      5]
yprime = [7      7     11     11     7]
```

6. Write a function named `solveSystem` that has three inputs:
 - two vectors consisting of the coefficients `[a b c]` of two line equations of the form

$$ax + by = c$$
 - a vector of `x` values
 - a. The function should output a vector giving the `x` and `y` values of the point of intersection between the two lines. If the lines are parallel, return the empty vector.

- b. Your function should also plot the two lines using the inputted vector of x values as x . In addition, on the same graph, plot the intersection point of the two lines. Make the first line blue, the second line red, and the intersection point a magenta diamond (type `'help plot'` in the MATLAB window for more info). Also, make sure that you label your plot appropriately with:
- Title: `'Intersection of Two Lines'`
 - x and y axis labels
 - A legend `'Line 1'` and `'Line 2'`

For example:

`solveSystem([1 -1 -1], [3 1 9], -5:5)` should produce `[2 3]`

Images

Chapter Objectives

This chapter covers:

- The basic representation of images
- How to read, display, and write JPEG image files
- Some basic operations on images
- Some advanced image processing techniques

Introduction

The graphical techniques we have seen so far have been two- and three-dimensional plots, whose basic concept is to write in places on the screen where data are required and to leave the rest of the screen blank. These images are easily generated when we have a mathematical model of the data and wish to represent it graphically. However, many sensors observing the world do not have that underlying model of the data. Rather, they passively generate two-dimensional representations that we see as images, leaving the interpretation of those images to a human observer.

This chapter discusses some of the elementary processes that can be applied to images in order to begin to extract meaning from them.

- 13.1** Nature of an Image
- 13.2** Image Types
 - 13.2.1 True Color Images
 - 13.2.2 Gray Scale Images
 - 13.2.3 Color Mapped Images
 - 13.2.4 Preferred Image Format
- 13.3** Reading, Displaying, and Writing Images
- 13.4** Operating on Images
 - 13.4.1 Stretching or Shrinking Images
 - 13.4.2 Color Masking
 - 13.4.3 Making a Collage
 - 13.4.4 Creating a Kaleidoscope
 - 13.4.5 Images on a Surface
- 13.5** Engineering Example—Detecting Edges

13.1 Nature of an Image

Before we confine ourselves to practical, computational reality, we need to understand the general nature of an image. The easiest answer would be that an image is a two-dimensional sheet on which the color at any point can have essentially infinite variability. However, since we live in a digital world, we will immediately confine ourselves to the conventional representation of images required for most digital display processors, as shown in Figure 13.1. We can represent any image as a 2-D, $M \times N$ array of points usually referred to as picture elements, or **pixels**, where M and N are the number of rows and columns respectively. Each pixel is “painted” by blending variable amounts of the three primary colors: red, green, and blue. (Notice that this is not the same blending process used in painting with oils or water colors, where the third primary color is yellow and the combination process is reversed—increasing amounts of the primary colors tends toward black, not white.)

The resolution of a picture is measured by the number of pixels per unit of picture width and height. This governs the fuzziness of its appearance in print, and controls the maximum size of good-quality photo printing. The color resolution is measured by the number of bits in the words containing the red, green, and blue (RGB) components. Since one value generally exists for each of the $M \times N$ pixels in the array, increasing the size of the pixel color will have a significant effect on the stored size of the image. Typically, 8 bits (values 0–255) are assigned to each color.

MATLAB has a data type, `uint8`, which uses 8 bits to store an unsigned integer in the range 0–255. It is unsigned because we are not interested in negative color values, and to specify the sign value would cost a data bit and reduce the resolution of the data to 0–127. By combining the three color values, there are actually 2^{24} different combinations of color available to a true-color image—many more possible combinations than the human eye can distinguish.

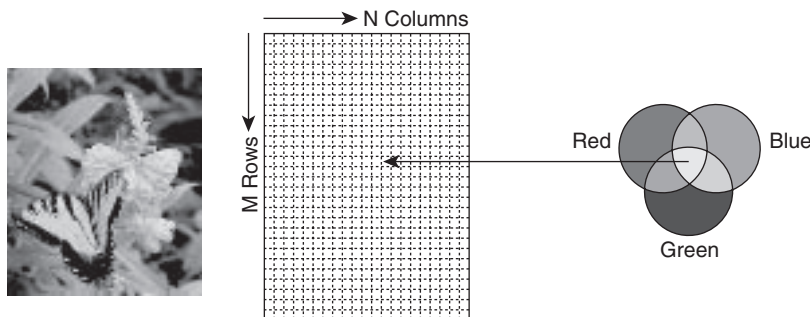


Figure 13.1 *The nature of images*



13.2 Image Types

Our sources for images to process are data files captured by imaging devices such as cameras, scanners, and graphic arts systems, and these image files are provided in a wide variety of formats. According to the MATLAB documentation, it recognizes files in TIFF, PNG, HDF, BMP, JPEG (JPG), GIF, PCX, XWD, CUR, and ICO formats. The various file formats are usually identified by their file extensions.

While this seems a bewildering collection of formats, MATLAB provides one image reading function that converts these file formats to one of three internal representations: true color, gray scale, or color mapped images.

13.2.1 True Color Images

True color images are stored according to the scheme shown in Figure 13.2 as an $M \times N \times 3$ array where every pixel is directly stored in three layers of the 3-D array. The first layer contains the red value, the second layer the green value, and the third layer the blue value. The advantage of this approach, as the name suggests, is that every pixel can be represented as its true color value without compromise. The only disadvantage is the size of the image in memory because there are three color values for every pixel.

13.2.2 Gray Scale Images

Gray scale images are also directly stored, but save the black-to-white intensity value for each pixel as a single `uint8` value rather than three values.

13.2.3 Color Mapped Images

Color mapped, or indexed, images keep a separate color map either 256 items long (for maximum economy of memory) or up to 32,768 items long. Each item in the color map contains the red, blue, and green values of a color, respectively.

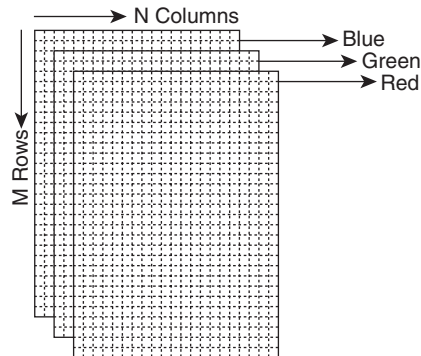


Figure 13.2 A true color image

As illustrated in Figure 13.3, the image itself is stored as an $M \times N$ array of indices into the color map. So, for example, a certain pixel index might contain the value 143. The color to be shown at that pixel location would be the 143rd color set (RGB) on the color map.

If the color map is restricted to 256 colors, each pixel can be drawn at the same color resolution as a true color image, as three 8-bit values, but the choice of colors is very restricted, and normal pictures of scenery—sky, for instance—take on a “layered color” appearance. Color mapped images can be used effectively, however, to store cartoon pictures economically where limited color choices are not a problem. Using a larger color map provides a larger, but still sometimes restrictive, range of color choices, but since the indices in the picture array must be 16-bit values and the color map is larger, the memory size advantages of this method of storage are diminished.

Computationally, it is possible to convert a color mapped image to true color, but true color or black and white images cannot normally be converted to color mapped format without loss of fidelity in the color representation.

13.2.4 Preferred Image Format

In order to avoid confusion in the format of images, we will confine our discussions to one specific image file format that is prevalent at the time of writing and that provides a nice compromise between economy of storage as an image file and accessibility within MATLAB. We will discuss files compressed according to a standard algorithm originally proposed by the Joint Photographic Experts Group (JPEG). When MATLAB reads JPEG images, they are decoded as true color images; when MATLAB writes them, they are again encoded in compressed form. If you take the time to look, the file size for a typical JPEG file is 30 times less than the size you would need to store the $M \times N \times 3$ bytes of the image. As we will see later, this compression does not come without cost.

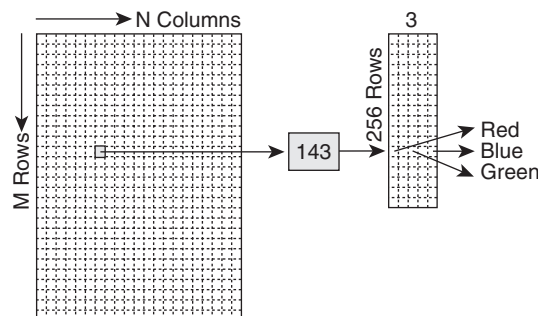


Figure 13.3 A color mapped image



13.3 Reading, Displaying, and Writing Images

MATLAB uses one image reading function, `imread(...)`, for all image file types. We will not attempt to describe its behavior for all the image file types—the MATLAB help file for `imread(...)` is extremely detailed if you need to read files other than the most common.

To read a file named *myPicture.jpg*, we use the following command:

```
>> pic = imread('myPicture.jpg', 'jpg')
```

where the result, `pic`, is an $M \times N \times 3$ `uint8` array of pixel color values, and the second parameter, `'jpg'`, provides the format of the file explicitly. This parameter is optional; MATLAB usually infers the file format correctly from the file contents.

Once the picture has been read into MATLAB, you can display it in a figure window with fixed size, just like a plot with the `image(...)` function. To display the picture represented by `pic`, for example, use the following command:

```
>> image(pic)
```

The `image(...)` function actually stretches or shrinks the image to fit the size of the normal plot figure, a behavior you normally desire; however, occasionally, you want the plot figure to match the actual image size and add axes showing the number of pixel rows and columns. Beginning with MATLAB 7.6 (R20008a), the systems also provide the `imshow(...)` function, which presents the image without stretching, shrinking, or axes (unless the figure window is too small).

Similarly, there is one function for writing files: `imwrite(...)`, which can be used to write most common file formats. If we have made some changes to `pic`, the internal representation of the image, we could write a new version to the disk by using the following:

```
>> imwrite(pic, 'newPicture.jpg', 'jpg')
```

where the third parameter, `'jpg'`, is required to specify the output format of the file. Reading and writing color mapped images is more complicated. Refer to the MATLAB help files for these operations, and for the many options available in writing JPEG files.



13.4 Operating on Images

Since images are stored as arrays, it is not surprising that we can employ the normal operations of creation, manipulation, slicing, and concatenation. We will note one particular matrix operation that will be of great value before examining some applications of array manipulation related to image processing.

13.4.1 Stretching or Shrinking Images

In earlier chapters we have seen the basic ability of MATLAB to use index vectors to extract rows and columns from an array. Now we extend these ideas to understand how to uniformly shrink or stretch an array to match an exact size. Consider, for example, `A`, a `rows × cols` array. Assume for a moment that the vertical size is good, but we want to stretch or shrink the image horizontally to `newRows`—a number that might be larger or smaller than `rows`. We use `linspace(...)` to create an index vector as follows:

```
>> rowVector = linspace(1, rows, newRows)
```

where the third parameter is the required size of the new array. In general, this will contain fractional values, and MATLAB will accept these for indexing but will issue warnings. Therefore we need to round the results as follows:

```
>> rowVector = round(rowVector)
```

Then we can use this vector to shrink or stretch the array `A` as follows:

```
>> newA = A(rowVector,:)
```

Clearly, this can be applied to both dimensions simultaneously, as shown in Exercise 13.1.



Exercise 13.1 Working with image stretching

Do It Yourself

```
>> m = magic(7); m = m(1:5,:)
m =
    30    39    48     1    10    19    28
    38    47     7     9    18    27    29
    46     6     8    17    26    35    37
     5    14    16    25    34    36    45
    13    15    24    33    42    44     4
>> [rows, cols] = size(m)
rows =
     5
cols =
     7
>> a = m(2:2:end, 3:3:end)
a =
     7    27
    16    36
>> RFactor = 1.43; CFactor = 0.75
RFactor =
    1.4300
CFactor =
    0.7500
>> rowVec = ...
```

continued on next page

```

round(linspace(1, rows, RFactor*rows))
rowVec =
    1     2     2     3     4     4     5
>> colVec = ...
round(linspace(1, cols, CFactor*cols))
colVec =
    1     3     4     6     7
>> m(rowVec, colVec)
ans =
    30     48     1     19     28
    38     7     9     27     29
    38     7     9     27     29
    46     8    17    35     37
     5    16    25    36     45
     5    16    25    36     45
    13    24    33    44     4
>>

```

In this exercise, first we create a 5×7 array `m` and determine its size. Then we illustrate the “normal” slicing operations by extracting the array `a` as the even rows, and every third column of `m`. Next we generalize this array slicing by stretching the number of rows by a factor 1.43 and shrinking the number of columns by a factor 0.75, which should give us an array 7.15×5.25 .

This is accomplished by building a row index vector, `rowVec`, and a column index vector, `colVec`, according to the algorithm above. Notice that the stretching is achieved by repeating selected values in the index vector, and shrinking is achieved by omitting some. When these index vectors are applied to the original array, `m`, the result is a new array with the anticipated size rounded down to the nearest whole number.

13.4.2 Color Masking

As an example of image manipulation, consider the image shown in Figure 13.4. This is a 2400×1600 JPEG image that can be taken with any good digital camera. However, the appearance of the Vienna garden is somewhat marred by the fact that the sky is gray, not blue. Fortunately, we have a picture of a cottage, as shown in Figure 13.5, with a nice, clear blue sky. So our goal is to replace the gray sky in the Vienna garden with the blue sky from the cottage picture.

Initial Exploration Before we can do this, however, we need to explore the Vienna picture to determine how to distinguish the gray sky from the rest of the picture. In particular, there are patches of sky visible between the tree branches that must be changed as well as the open sky. Listing 13.1 illustrates a good way to accomplish this. Here we display the image in one figure, choose a representative row in the image that includes some sky showing through the tree (we chose row 350), and then plot the red, blue, and green values of the pixels across that row. Figure 13.6 shows the resulting plot.



Figure 13.4 *A garden in Vienna*



Figure 13.5 *A cottage in Oxfordshire*

Listing 13.1 Exploring the sky situation

```
1. clear
2. clc
3. close all

4. v = imread('Vienna.jpg');
5. image(v)
6. figure
7. row = input('which row? ');
8. red = v(row, :, 1);
9. gr = v(row, :, 2);
10. bl = v(row, :, 3);
11. plot(red, 'r');
12. hold on
13. plot(gr, 'g');
14. plot(bl, 'b');
```

In Listing 13.1:

Lines 1–3: The usual graphics setup.

Line 4: Reads the image.

Line 5: Displays the image.

Line 6: Determines a suitable row (350 is a good choice).

Line 7: Makes a new plotting figure.

Lines 8–10: Extract the three color layers for the chosen row.

Lines 11–14: Plot the three colors. Since we omitted one of the axis values, MATLAB makes the correct assumption that the x values are the integers `1:length(Y)`, which give us the horizontal pixel number across the row.

Initial Analysis As we examine Figure 13.6, we see that the red, green, and blue values for the open sky are all around 250 because the sky is almost white. However, the color “spikes” that correspond to the color values of the sky elements that show through the tree are actually lower. We could decide, for example, to define the sky as all those pixels where the red, blue, and green values are all above a chosen threshold, and we could comfortably set that threshold at 160.

There is one more important consideration. It would be unfortunate to turn the hair of the lady (the author’s wife) blue, and there are fountains and walkways that might also logically appear to be “sky.” We can prevent this embarrassment by limiting the color replacement to the upper portion of the picture above row 700.

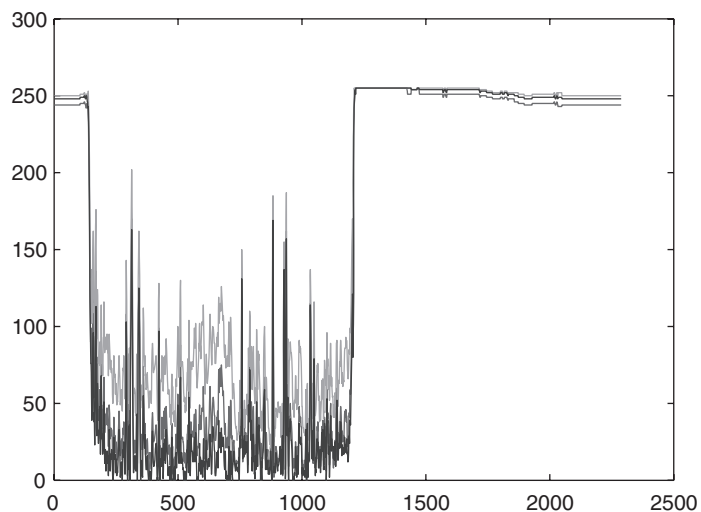


Figure 13.6 Plot of the color values on one row of the Vienna image

Final Computation So we are ready for the code that will replace the gray sky with blue. The code in Listing 13.2 accomplishes this, and Figure 13.7 shows the resulting image.

In Listing 13.2:

Lines 1–3: Clear the environment.

Lines 4 and 5: Read the two images.

Line 6: Sets the arbitrary threshold.

Lines 7–13: Build a logical mask to replace the appropriate pixels from the cottage picture into the Vienna picture. This is the only



Figure 13.7 *The Vienna garden with a blue sky*

Listing 13.2 Replacing the gray sky

```
1. clear
2. clc
3. close all

4. v = imread('Vienna.jpg');
5. w = imread('Witney.jpg');
6. thres = 160;
7. layer = (v(:,:,1) > thres) ...
8.         & (v(:,:,2) > thres) ...
9.         & (v(:,:,3) > thres);
10. mask(:,:,1) = layer;
11. mask(:,:,2) = layer;
12. mask(:,:,3) = layer;
13. mask(700:end, :, :) = false;
14. nv = v;
15. nv(mask) = w(mask);
16. image(nv);
17. imwrite(nv, 'newVienna.jpg', 'jpg')
```

messy part of the solution. We need to build a logical, 3-D array of the same size as both images. However, since the criterion for detecting the sky is a 2-D layer, we have to build that first and then populate each color of the mask with that layer.

Lines 7–9: Determine which pixels are sky as a 2-D layer.

Lines 10–12: Build the required mask by putting this logical layer in each color plane.

Line 13: Refuses to replace any pixels below row 700.

Line 14: Copies the original image.

Line 15: Replaces the sky.

Line 16: Shows the image.

Line 17: Writes out the JPEG result.

Post-operative Analysis We realize that this is not quite the end of the story, because a wire has suddenly become evident in the picture. Furthermore, if we take a close look at the wire in Figure 13.8, we see a number of disturbing things:

- The sky is by no means uniform in color—justifying the assertion that color mapped images do not have enough different colors to draw a true sky effectively
- The color of the wire is not far removed from the color of some parts of the blue sky—so replacing slightly darker blue would be problematic
- There is a light colored “halo” around the wire that is actually a result of the original JPEG compression of the image so that even if we did replace the darker colors, the ghost of the wire would still be visible

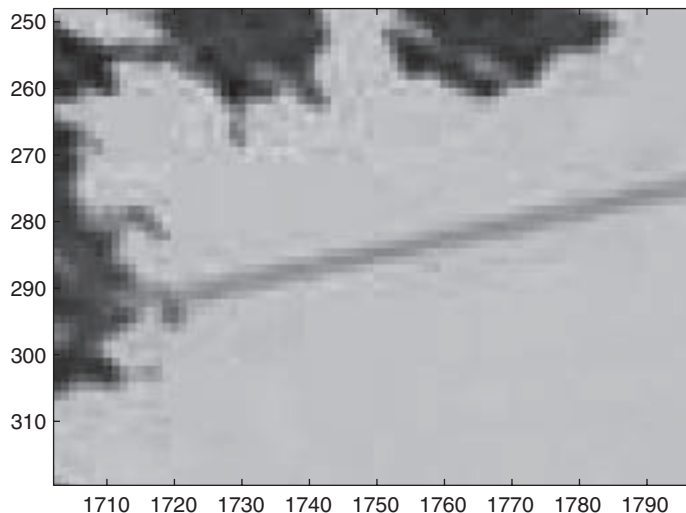


Figure 13.8 *Magnified image of the wire*

So pixel replacement will probably not solve our wire problem. We will take a different approach to solve this problem in Chapter 15.

13.4.3 Making a Collage

As a second example of image manipulation, consider the two images shown in Figures 13.9 and 13.10, each of which is a 1600×1200 JPEG image. However, the cute factor is a little low. What we really want for our family album is a collage consisting of the author's cute dog without most of the background and the lake picture. Furthermore, it would be nice to have a brown frame and a light yellow mat setting off the two pictures.



Figure 13.9 *The original cute dog picture*



Figure 13.10 *The original lake scene*

Before we begin coding, we draw a diagram in detail to understand what we are trying to achieve. The design diagram is shown in Figure 13.11, the resulting collage is shown in Figure 13.12, and the code is shown in Listing 13.3.

Notice that two different manipulations are required. Cropping will eliminate the extraneous background in the dog picture; however, since the lake picture is twice the size of the dog picture, it has to be shrunk by a factor of two in each dimension. The easiest way to accomplish that is to eliminate every other row and every other column of the lake picture. This will reduce its resolution, but still retain enough detail to be pleasant to the eye. If these parameters had not been exact, the technique discussed in Section 13.4.1 could be used to provide a generalized shrinking of the image.

Study the notes in the code for a full understanding of the implementation of this picture. In Listing 13.3:

Lines 4 and 5: Read the JPEG files that are stored as 8-bit unsigned integers of type `uint8`.

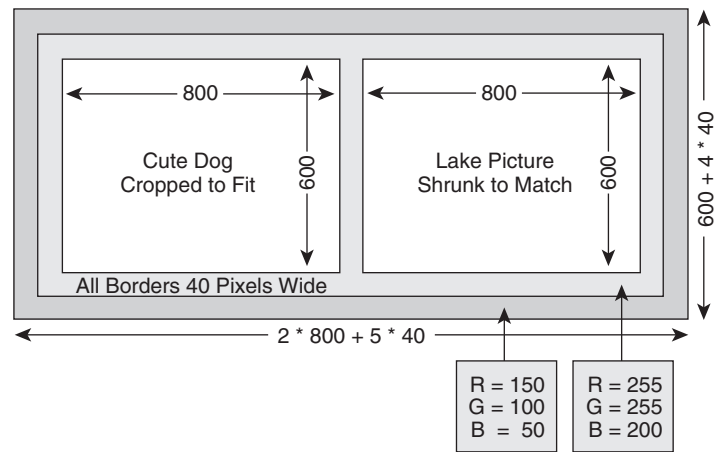


Figure 13.11 *Design of the collage*



Figure 13.12 *The final collage*

Listing 13.3 Making a collage

```

1. clear
2. clc
3. clf

4. dog = imread('Pep.jpg');
5. lake = imread('Lake.jpg');

6. [rows, cols, clrs] = size(dog);
7. pheight = rows/2; pwidth = cols/2
8. border = 40;
9. overallw = 2*pwidth + 5*border;
10. overallh = pheight + 4*border;
11. frameR = 150; frameG = 100; frameB = 50;
12. matR = 255; matG = 255; matB = 200;
13. % make the outside frame
14. frame = uint8(ones( overallh, overallw ));
15. collage = uint8(zeros( overallh, overallw, 3 ));
16. collage(:, :, 1) = frame * frameR;
17. collage(:, :, 2) = frame * frameG;
18. collage(:, :, 3) = frame * frameB;
19. % insert the mat
20. mat = uint8(ones( overallh - 2*border, ...
21.                  overallw - 2*border ));
22. collage(border+1:overallh-border,...
23.         border+1:overallw-border,1) = mat*matR;
24. collage(border+1:overallh-border,...
25.         border+1:overallw-border,2) = mat*matG;
26. collage(border+1:overallh-border,...
27.         border+1:overallw-border,3) = mat*matB;
28. % crop the dog picture
29. left = 180; % a judgment call
30. dogCr = dog( end - pheight + 1: end,...
31.             left + 1: left + pwidth, :);
32. collage(2*border+1:2*border + pheight,...
33.         2*border+1:2*border + pwidth, :) = dogCr;
34. % shrink the lake picture
35. lakeSh = lake( 1:2:end, 1:2:end, :);
36. collage(2*border+1:2*border + pheight, ...
37.         3*border+pwidth+1:3*border+2*pwidth,:) ...
38.         = lakeSh;
39. image(collage)
40. axis equal
41. axis off
42. imwrite(collage, 'collage.jpg', 'jpg' )

```

Line 6: Obtains the size of the dog picture.

Line 7: Gives the height and width of each picture.

Lines 9 and 10: Compute the overall height and width of the image using the design shown in Figure 13.11.

Lines 11 and 12: Set the colors of the frame and mat between the pictures.

Hint

An easy way to choose color values is to open any software package that draws or sets the color of text and use its utilities that allow you to set arbitrary colors. They usually provide the RGB values for any given color.

Line 13: Now we build the collage starting from the outside and working inwards.

Line 14: Makes the 2-D layer that is the size of the overall picture. Since `ones(...)` naturally produces an array of type `double`, we have to cast it to type `uint8` by calling the function `uint8(...)`.

Line 15: Creates an empty collage of the right shape.

Lines 16–18: Load the three color layers with the color values specified for the frame.

Lines 19–27: Repeat the process for the mat. The rectangle for the mat is two borders smaller (one each side/end) than the frame, and must be positioned one border in from the frame edge in the collage.

Lines 28–31: Looking at the dog picture, we must decide where to cut off the background. Considering the rows, it is pretty clear that the bottom rows should be kept. However, we need to cut off a little to the left and a lot to the right, so setting a variable for the left cutoff allows one to look at the results and adjust the position of the dog in the cropped picture.

Lines 32 and 33: The cropped dog picture is inserted into the collage two borders from the top and two borders from the left.

Line 35: Shrinks the lake picture by removing every other row and every other column.

Lines 36–38: The shrunken lake picture is inserted into the collage two borders from the top and three borders plus the width of the dog picture from the left.

Lines 39–42: In cleaning up, the two axis commands make sure that the x and y scales are equal to avoid distortion in the image, and turn off the unwanted numerical boundaries. The image is then written out as a JPEG file.

Common Pitfalls

Be careful requesting the size of 3-D (and more) arrays. If you leave off variables—as here, you might be tempted not to ask for the number of colors because you know it's three—the `size(...)` function multiplies together the remaining dimension sizes. So `[r,c] = size(dog)` would return `r = 1200` and `c = 4800`!

13.4.4 Creating a Kaleidoscope

Figure 13.13 illustrates the geometric manipulation necessary to create a particular kaleidoscope picture. We start with a square image, *sqbutter.jpg*. A non-square image can always be made square by stretching or cropping, as defined in Section 13.4.1. We will divide the picture into four quadrants and then halve each quadrant with a diagonal toward the center of the picture.

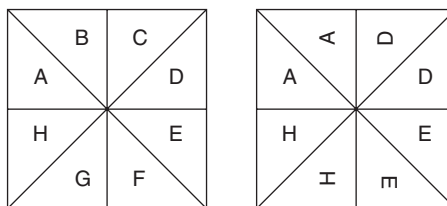


Figure 13.13 *Layout of the collage*

We will keep those halves that touch the horizontal centerline, and overwrite the other half of the quadrant with its mirror image reflected in the diagonal.

Two of MATLAB's built-in functions make this process quite simple:

- `tril(A)` takes a 2-D array and returns its lower-left diagonal half, with zeros in the triangle above the diagonal
- `rot90(A, n)` takes a 2-D array and rotates it by $n \times 90^\circ$ counter-clockwise; if n is not specified, it is assumed to be 1

Figure 13.14 shows the results. The overall logic flow of the solution matches that shown in Figure 13.13:

- Disassemble the picture into four quadrants
- Rotate each quadrant the appropriate number of 90° turns
- Mirror the quadrant
- Rotate the mirrored quadrant back to its original orientation
- Reassemble the picture

Listing 13.4 shows the code that manages the overall logic.



Figure 13.14 *The butterfly kaleidoscope*

Listing 13.4 Making a kaleidoscope

```

1. sb = imread('sqbutter.jpg');
2. subplot( 1, 2, 1);
3. image(sb);
4. cols = length(sb);
5. mid = cols/2
6. subplot( 1, 2, 2);
7. img = [diagMirror( sb(1:mid,      1:mid, :), 0 ) ...
8.        diagMirror( sb(1:mid,      mid+1:end, :), 3 );
9.        diagMirror( sb(mid+1:end, 1:mid, :), 1 ) ...
10.       diagMirror( sb(mid+1:end, mid+1:end, :), 2 )];
11. image(img);

```

In Listing 13.4:

Line 1: Reads the butterfly image.

Line 2: Starts the left subplot.

Line 3: Shows the original image in the left half of the plot.

Line 4: Finds the largest dimension to use as the number of columns (the image might not initially be square).

Line 5: Computes the center point of the picture.

Line 6: Switches to the right picture.

Lines 7–10: These lines simultaneously do the work of dividing the picture into quadrants, calling the function that mirrors each one, and reassembling the quadrants by concatenation to build the new picture.

Listing 13.5 shows the detailed code for rotating and reflecting each quadrant.

Listing 13.5 Mirroring on a diagonal

```

1. function sq = diagMirror(A, code)
2. % mirror this square image diagonally
3. % the parameter code represents the number of
4. %           90 deg left rotations

5. for c = 1:3 % tacky to do a layer at a time, but
6.           % tril must see a 2-D array
7.     layer = A(:, :, c);
8.     trin = tril(rot90(layer, code));
9.     % rotate the image back after mirroring
10.    sq(:, :, c) = rot90(trin + trin', 4 - code);
11. end

```

In Listing 13.5:

Line 5: Unfortunately, since our built-in tools operate in only two dimensions, we have to rotate each color plane separately.

Line 7: Extracts a color layer.

Line 8: Rotates the image the specified number of times (depends on the original quadrant) and strips out the lower triangle.

Line 10: The transpose operator performs the required mirroring about the diagonal and the addition merges the original and the mirror because the other triangle is zero. Finally, it rotates the image back to its original orientation and replaces the original color layer in the image.

13.4.5 Images on a Surface

In Chapter 11 we saw how to create a surface representing solid objects and, in particular, how to create a spherical image that rotates with lighting. Spectacular effects can be created by “pasting” images onto these surfaces, as will be illustrated in this last example. Here, we are given an image of the surface of the earth using Mercator projection, shown in Figure 13.15.¹

It is important to use the Mercator projection, named for the sixteenth-century Flemish cartographer Gerardus Mercator, because this projection keeps the lines of latitude and longitude on a rectangular grid. This allows a correct representation of the map as it is pasted onto the spherical surface. However, it also presents a challenge because in this projection, the north and south poles would be stretched unreasonably across the top and bottom of the map. This map, therefore, leaves off the region near the poles, and we have to replace those regions.

The objective of this exercise is to paste this image onto a rotating globe. The trick to accomplishing this is to use a feature of the `surf(...)` function whereby the image is supplied in a specific form as the fourth parameter, as follows:

```
surf(xx, yy, zz, img)
```

It will replace the normal coloring scheme of the surface with the image under the following conditions:



Figure 13.15 *Map projection*

¹ The file *earth_s.jpg* is provided as part of the MATLAB system.

- The rows and columns of the image match the rows and columns of the `xx`, `yy`, `zz` plaid
- The image supplies the red, green, and blue layers in the same form as true color images
- The color values, however, must be of type `double` in the range `0..1`

We use the following MATLAB tricks to improve the appearance of the image on the globe:

- `material dull` reduces the reflectivity of the surface.
- `handle = light('Color',[1,1,1])` creates a custom light source with a source with the specified properties. The `handle` provided by this function is passed as the first parameter to the `lightangle(...)` function to manipulate the position of this light rather than the default light. See MATLAB help `light` for more options on this function.

In the following code, rather than stretching the image to the size of the plaid, we choose to size the plaid to the image, thereby preserving all the image resolution. Clearly, in different circumstances where the size of the plaid is specified, the image can be stretched to suit those dimensions. The code to accomplish this is shown in Listing 13.6.

Listing 13.6 Rotating a globe

```

1. WM = imread('figure_13_15_map_projection.jpg');
2. WM(:,end+1,:) = WM(:,1,:);
3. snow = mean(mean(WM(1,:,:)));
4. [WMr, WMc, clr] = size(WM);
5. rowsperdeglat = WMr/170
6. add = floor(rowsperdeglat * 5)
7. addlayer = uint8(ones(add, WMc) * snow);
8. toAdd(:,:,1) = addlayer;
9. toAdd(:,:,2) = addlayer;
10. toAdd(:,:,3) = addlayer;
11. worldMap = [toAdd; WM; toAdd];
12. [nlat nlong clr] = size(worldMap)
13. lat = double(0:nlat-1) * pi / nlat;
14. long = double(0:nlong-1) * 2 * pi / (nlong-1);
15. [th phi] = meshgrid(long, lat);
16. radius = 10;
17. zz = radius * cos(phi);
18. xx = radius * sin(phi) .* cos(th);
19. yy = radius * sin(phi) .* sin(th);
20. wM = double(worldMap) / 256;
21. surf(xx, yy, zz, wM);
22. shading interp
23. axis equal, axis off, axis tight
24. material dull
25. handle = light('Color',[1,1,1]);
26. th = 0;

```

continued on next page


```
27. while true
28.     th = th - 1;
29.     view([th 20]);
30.     lightangle(handle, th+50, 20)
31.     pause(.001)
32. end
```

In Listing 13.6:

Line 1: Reads the JPEG image.

Line 2: Enables good closure at the image edge by copying the first column of the map beyond the last column.

Line 3: Computes the mean image intensity of the snow on the top edge of the image. This will be used to fill the circles at the north and south poles.

Line 4: Fetches the size of the map.

Line 5: To calculate the size of the circles at the poles, we assume that the map takes us to $\pm 85^\circ$ of latitude, so we need the equivalent of 5° at the top and bottom of the map. This line calculates how many rows represent 1° of latitude.

Line 6: Shows the number of rows to add to the map.

Line 7: Computes the values of a single color layer by making an array with `ones(...)` using the number of rows to add and the number of map columns, and multiplying by the snow intensity.

Lines 8–10: Build the strips to add to the globe map by copying this layer to the red, green, and blue layers of a new image array.

Line 11: Prepares the complete map by concatenating this image to the top and bottom of the map.

Line 12: Retrieves the size of this map.

Lines 13 and 14: Prepare the vectors defining the plaid by spreading the map dimensions across π radians in latitude and 2π radians in longitude.

Lines 15–19: Prepare the sphere.

Line 20: Scales the image to double values between 0 and 1 as required by `surf(...)`.

Lines 21–23: Draw the surface as usual, using the image as the color distribution.

Lines 24 and 25: With the usual `lightangle(...)` implementation with the default light, the globe does not look right. These two lines set the reflectivity of the surface and create a custom light with a lower intensity.

Lines 26–29: The perpetual rotation with the angle θ moving backward one degree at a time.

Line 30: This keeps the custom light in the same position relative to the observer.

Line 31: The usual pause to allow the drawing to take place for each iteration.

A snapshot of the globe as it is rotating is shown in Figure 13.16.



Figure 13.16 *Globe*

13.5 Engineering Example—Detecting Edges

While images are powerful methods of delivering information to the human eye, they have limitations when being used by computer programs. Our eyes have an astonishing ability to interpret the content of an image, such as the one shown in Figure 13.17. Even a novice observer would have no difficulty



Figure 13.17 *C-130 in flight*

seeing that it is a picture of an aircraft in flight. An experienced observer would be able to identify the type of aircraft as a Lockheed C-130, and perhaps some other characteristics of the aircraft.

While our eyes are excellent at interpreting images, computer programs need a lot of help. One operation commonly performed to reduce the complexity of an image is edge detection, in which the complete image is replaced by a very small number of points that mark the edges of “interesting artifacts.”

Figure 13.18 shows the results from a simple program attempting to paint the outline of the aircraft in black by putting a black pixel at an identified edge. The key element of the algorithm is the ability to determine unambiguously whether a pixel is part of the object of interest or not. An edge is then defined as a pixel where some of the surrounding pixels are on the object and some are not. The image selected for this exercise makes edge detection simple since the aircraft is everywhere darker than the surrounding sky.



Figure 13.18 *Result of edge detection*

The script used to generate this picture is shown in Listing 13.7. The basic approach of the algorithm is to use MATLAB's built-in array processing to detect the edges across the whole image at once. To accomplish this, we create four arrays, each one row and one column less than the original image, and each offset by one pixel, as illustrated in Figure 13.19. The array `pic` is in the original location, `pt` is one row up from that location, `p1` is one row left, and `pt1` is one row left and up. If we now collapse these arrays on top of each other, we are simultaneously comparing the values of a square of four pixels across the whole image (less one row and one column).

Listing 13.7 Edge detection

```
1. pic = imread('C-130.jpg');
2. [rows, cols, cl] = size(pic);

3. amps = uint16(pic(:,:,1))...
4.       + uint16(pic(:,:,2))...
5.       + uint16(pic(:,:,3));
```

continued on next page

```

6. up = max(max(amps))
7. dn = min(min(amps))
8. fact = .5
9. thresh = uint16(dn + fact * (up - dn))

10. pix = amps(2:end, 2:end);
11. pt1 = amps(1:end-1, 1:end-1);
12. pt = amps(1:end-1, 2:end);
13. pl = amps(2:end, 1:end-1);

14. alloff= and(and((pix > thresh), ( pt > thresh)),...
15.           and(( pl > thresh), (pt1 > thresh)));
16. allon = and(and((pix <= thresh), ( pt <= thresh)),...
17.           and(( pl <= thresh), (pt1 <= thresh)));
18. edges = and(not(allon), not(alloff));
19. layer = uint8(ones(rows-1, cols-1) *255);
20. layer(edges) = 0;
21. outline(:, :, 1) = layer;
22. outline(:, :, 2) = layer;
23. outline(:, :, 3) = layer;
24. image(outline)
25. imwrite(outline, 'c-130 edges.jpg', 'jpg')

```

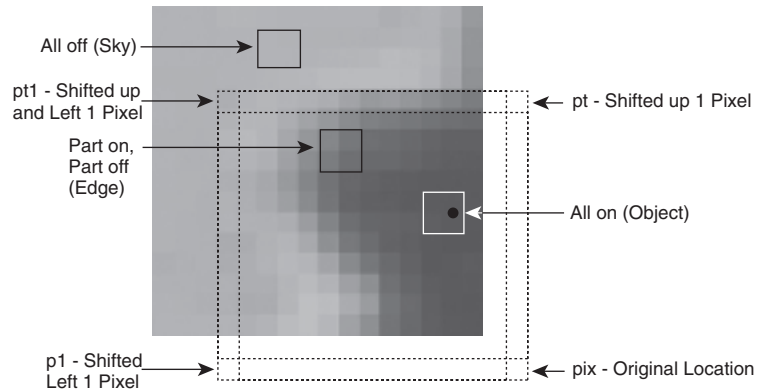


Figure 13.19 *Overlapping picture layers*

In Listing 13.7:

Lines 1 and 2: Read the original image and determine its size.

Lines 3–5: Construct an array of size `rows` x `cols` containing the total color intensity of each pixel. The class `uint16`, using two bytes instead of one, is big enough for the sum of three `uint8`s.

Lines 6–9: Rather than guess an amplitude threshold, we compute a threshold halfway between the maximum and minimum intensities across the picture.

Lines 10–13: Set up the four overlapping arrays offset by a pixel each.

Lines 14 and 15: The logical array `allOff` will be true wherever all four adjacent pixels have an intensity above the threshold—these are on the sky.

Lines 16 and 17: The logical array `allOn` will be true wherever all four adjacent pixels have an intensity below the threshold—these are on the aircraft.

Line 18: The pixels we are looking for are those where the pixel is neither completely sky nor completely aircraft.

Line 19: Makes a white image the same size as the logical arrays.

Line 20: Sets the edges to black.

Lines 21–25: Put that layer into the RGB layers, show the image, and write it to the disk.

Observation Clearly, while there is much more to be done with this data for it to be useful, the complexity of this image has been reduced from 12 million `uint8` values with no real meaning to a small number of data values that outline an object of interest. Algorithms beyond the scope of this text could be used to convert these outlining points to polynomial shapes. These shapes could then be matched against projections of 3-D models to actually identify the object in the picture.



Chapter Summary

This chapter covered the following:

- Images represented internally in MATLAB in bit-mapped, gray scale, or true color form
- Image files that come in a large variety of formats; MATLAB provides a single reader function and a single writer function to manipulate all the common image types
- Common operations on images, including cropping, stretching or shrinking, and concatenating and pasting an image onto a surface
- An engineering example showing how edge detection begins the process of extracting meaning from an image



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>image(pic)</code>	Displays an image in a figure of fixed dimensions with axes	13.3
<code>imread(file)</code>	Reads an image file	13.3
<code>imshow(pic)</code>	Displays an image in a figure of variable dimensions without axes (not available in all MATLAB versions)	13.3
<code>imwrite(data, file, format)</code>	Writes an image file	13.3
<code>handle = light ('Property', 'value', ...)</code>	Creates a light source with the specified properties	13.4.5
<code>linspace(from, to, n)</code>	Defines a linearly spaced vector	13.2.1, 13.4.1
<code>material <spec></code>	Sets the material quality of a surface	13.4.5
<code>rot90(A,n)</code>	Rotates A by 90° clockwise n times	13.4.4
<code>tril(A)</code>	Reduces A to its lower triangular half with zeros in the upper triangle	13.4.4
<code>uint8/16</code>	Unsigned integer type with the specified number of bits	13.1



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. An image whose color values are all 0 will be all white on the screen.
2. MATLAB uses one image reader for all image file types.
3. The normal operations of creation, slicing, and concatenation can be used to manipulate images.
4. `rot90(A)` rotates a 3-D array by 90° clockwise.
5. Edge detection dramatically reduces the amount of data to be processed by image identification software.

Fill in the Blanks

1. Each pixel of a true color image is stored as _____ values of type _____ containing values _____.
2. Gray scale images store the black-to-white intensity value for each _____ as a _____.
3. When MATLAB reads JPEG files, they are _____ as _____ images containing _____.
4. Once a picture has been read into MATLAB, you can display it in a _____ with the _____ function.
5. _____ an image discards pixels around one or more edges; _____ an image discards pixels within its borders; _____ an image involves replicating pixels.
6. The _____ operator mirrors an array about its _____.

**Programming Projects**

1. As an introduction to image problems, perform the following manipulations:
 - a. Find a suitable JPEG image file. Read it with the function `imread(...)`, display it with `image(...)` and store the result in `A`.
 - b. Create a copy of `A`, flip the image from left to right, and display it in a new figure.
 - c. Create a copy of `A`, swap the values for red and blue, and display it in a new figure.
 - d. Create a copy of `A`, stretch the image to 4 times its original size (twice as many rows and twice as many columns), and display it in a new figure.
 - e. Create a copy of `A` and then shrink the image to 0.7 its original size in each dimension and display it in a new figure.
2. An image could be scrambled by doing the following in order:
 - a. multiple quadrant flips:
 - top left quadrant becomes bottom right quadrant
 - top right quadrant becomes bottom left quadrant
 - bottom right quadrant becomes top left quadrant
 - bottom left quadrant becomes top right quadrant

In other “words”:

1	2
3	4

becomes:

4	3
2	1

- b. The image is flipped upside down.
- c. The red values are swapped with the green values.
- d. The blue values are flipped left to right.

Write a function called `imageScrambler` that takes in an image array and a string. If the string is equal to `'scramble'`, your function should scramble the image according to the above method and return the modified image in array form. If the string is equal to `'unscramble'`, your function should unscramble the image by reversing the above method and return the modified array.

Otherwise, your function should return the array untouched. You may assume that the image array provided will always contain an even number of rows and columns.

Test your solution by writing a script that reads a selected image, `A`, ensures that there is an even number of rows and columns, and displays the image `A`. It should then scramble `A` returning the result in `B` and display `B`. Finally, it should unscramble `B` and display the result, `C`. Compare `C` to the original image `A`. If your code is correct, they should be identical.

3. You are provided an image and your job is to convert the full-sized image to a smaller one. Normally when image processing software is required to resize an image, a complex resizing algorithm is used to accomplish the conversion. We will attempt to duplicate this conversion. Write a function called `resizeMe` that takes in a string as an input corresponding to an image file name. The function should then resize the image to 1.414 times its original size in each dimension and display it. Additionally your function should use the built-in function `imwrite(...)` to write the new image to a file. The name of the new file will be the original file name preceded by `'LG'`. For example, if the original filename is called `'yellow_bird.jpg'`, the new file should be called `'LGyellow_bird.jpg'`.

4. Write a function called `rotate` that takes in an image array and a number. The number represents the number of times the function will rotate the image clockwise by 90 degrees. A negative number signifies counter-clockwise rotation and a positive one signifies clockwise rotation. Remember image matrices have to be of type `uint8`, so make sure to cast the result at the end.
5. We have obtained new intelligence that the Housing Department has plans to renovate all the rooms in the dorms with a new prototype. However, the prototype has been encoded into 3 separate images to avoid rival students from finding out about it and thus seeking refuge here. Each image only contains one layer of color (e.g., `roomScrambledRed.jpg` only contains the Red layer). As a loyal student, it is your job to reconstruct a new image out of these 3 images.
 - a. Create a script called `room`, and read the three layers using `'imread'`. Create the new matrix `ReconImage` with the three layers and display it using `'imshow'`.
 - b. After detailed analysis of the image, you find that it is also scrambled. Using advanced cryptography and whizbang mathematical formulas, you have come to the conclusion that the 4 quadrants of the image have been re-arranged. Manipulate the composite image from part a. and re-arrange the pieces to form the proper image. Display it using `subplot(...)`, below the first image.
6. For this exercise, you will visit—at least in MATLAB—a place you have always wanted to go.
 - a. Find or take a picture of yourself with a plain background such as a green screen, using the JPEG image format. It would be a good idea not to wear the color of the background.
 - b. Find a JPEG image of the place you want to go and decide on the rectangle in that scene where your image should appear. Save the width and height of the rectangle and the row and column of its top left corner.
 - c. Re-size your image to be the width and height of the rectangle.
 - d. Use the color masking technique of section 13.4.2 to copy your image without the green screen into the selected rectangle of your dream scene.

Hint

Rotating counter-clockwise once is the same as rotating clockwise three times.

Processing Sound

Chapter Objectives

This chapter discusses the following:

- How sound is physically recorded and played back, and MATLAB's internal storage of sound
- Operations that can be performed with the original time trace
- The ability to transform the data into the frequency domain, and the physical significance of the transformed data
- Operations that can be performed in the frequency domain

- 14.1** The Physics of Sound
- 14.2** Recording and Playback
- 14.3** MATLAB Implementation
- 14.4** Time Domain Operations
 - 14.4.1 Slicing and Concatenating Sound
 - 14.4.2 Musical Background
 - 14.4.3 Changing Sound Frequency Poorly
 - 14.4.4 Changing Sound Frequency Well
- 14.5** The Fast Fourier Transform
 - 14.5.1 Background
 - 14.5.2 MATLAB Implementation
 - 14.5.3 Simple Spectral Analysis
- 14.6** Frequency Domain Operations
 - 14.6.1 Analyzing Instrument Sounds
 - 14.6.2 Adding Sounds to the Spectrum
 - 14.6.3 Manipulating the Spectrum
- 14.7** Engineering Example—Oil Rig Structural Integrity



14.1 The Physics of Sound

Any sound source produces sound in the form of pressure fluctuations in the air. While the air molecules move infinitesimal distances in order to propagate the sound, the important part of sound propagation is that pressure waves move rapidly through the air by causing air molecules to “jostle” each other. These pressure fluctuations can be viewed as analog signals—data that have a continuous range of values. These signals have two attributes: their amplitude and their frequency characteristics.

In absolute terms, sound is measured as the **amplitude** of pressure fluctuations on a surface like an ear drum or a microphone. However, the primary characteristic of these data is their dynamic range. Our ears are able to detect small sounds with amplitudes around 10^{10} (10 billion) times smaller than the loudest comfortable sound. Sound intensity is therefore usually reported logarithmically, measured in decibels where the intensity of a sound in decibels is calculated as follows:

$$I_{DB} = 10 \log_{10}(I / I_0)$$

where I is the measured pressure fluctuation and I_0 is a reference pressure usually established as the lowest pressure fluctuation a really good ear can detect, 2×10^{-4} dynes/cm².

Also, sounds are pressure fluctuations at certain **frequencies**. The human ear can hear sounds as low as 50 Hz and as high as 20 kHz. Voices on the telephone sound odd because the upper frequency is limited by the telephone equipment to 4 kHz. Typically, hearing damage due to exposure to excessive sound levels causes an ear to lose sensitivity to high and/or low frequencies.



14.2 Recording and Playback

Early attempts at sound recording concentrated first on mechanical, and later magnetic, methods for storing and reproducing sound. The phonograph/record player depended on the motion of a needle in a groove as a cylinder or disk rotated at constant speed under the playback head. Not surprisingly, when you see the incredible dynamic range required, even the best stereos could not reproduce high-quality sound. Later, analog magnetic tape in various forms replaced the phonograph, offering less wear on the recording and better, but still limited, dynamic range. Digital recording has almost completely supplanted analog recording, and will be the subject of this chapter.

Of course, sound amplitude in analog form is unintelligible to a computer—it must be turned into an electrical signal by a microphone, amplified to suitable voltage levels, digitized, and stored, as shown in Figure 14.1. The key to successful digital recording and playback—whether by digital tape machines, compact disks, or computer files—is the design of the analog-to-digital (A/D)

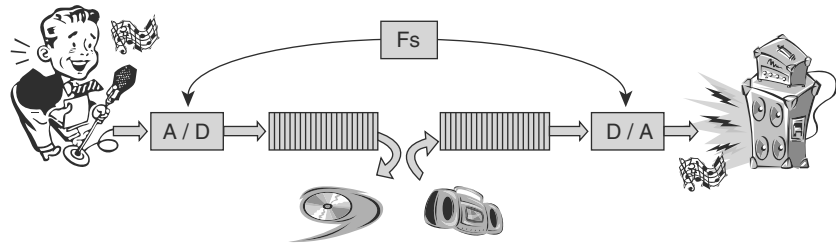


Figure 14.1 *Mechanics of sound recording and playback*

and digital-to-analog (D/A) devices. The reader should remember that this is still low-level data. Each word coming out of the A/D or going into the D/A merely represents the pressure on the microphone at a point in time.

The primary parameter governing the sound quality is the **recording rate**—how quickly the mechanism records samples of the sound (the sampling rate). The background theory of sampling is beyond the scope of this text. Interested readers should research Nyquist on a good search engine. For our purposes, we want the sampling rate to be twice the highest frequency you are likely to encounter, usually around 20,000 samples per second.

The other parameter, the **resolution** of the recorded data, has remarkably little effect on the quality of the recording to an untrained ear. The resolution is usually either 8 bits (−128 to 127) or 16 bits (−32,768 to 32,767). While 8-bit resolution ought to offer very limited dynamic range, and theoretically should be used only for recording speech, in practice it produces excellent reproductions of music also. However, 16-bit resolution, which still does not have sufficient resolution to capture the complete 100dB range discussed above, reproduces high-quality music superbly.

These parameters must be received from the functions that read pre-recorded files of sound data. To be able to play such a file, we must receive not only the data stream, but also information indicating the sample frequency, f_s , and the word size.



14.3 MATLAB Implementation

MATLAB offers the tools for reading sound files in two formats: `.wav` files (`wavread(...)`), and `.au` files (`auread(...)`). Both return three variables: the vector of sound values in the range (−1 ... 1), the sampling frequency in Hz (samples per second), and the number of bits used to record the data (8 or 16).

To play a sound file, MATLAB provides the function `sound(data, rate)` where `data` is the vector of sound values, and `rate` is the playback frequency, usually the frequency at which the sound values were recorded.

We will see that the function `sound(...)` passes the data directly to the computer's sound card, and does not wait for the card to finish playing the



Exercise 14.1 The effect of data resolution

Do It Yourself

Enter the following commands in the Command window:

```
>> [b16, F16, n] = wavread('brooklyn16.wav');
>> sound(b16, F16)
>> [b8, F8, n] = wavread('brooklyn8.wav');
>> sound(b8, F8)
```

sound. A number of .wav files are included on the book's Companion Website (www.aw.com/smith_mat) to demonstrate certain aspects of sound files. Try Exercise 14.1, which compares two recordings of the identical sound bite recorded at 16-bit and 8-bit resolution, respectively.

Exercise 14.1 compares sound files stored in 8-bit and 16-bit form by reading and playing each version. Unless you have very good audio equipment and a really good musical ear, in spite of the discussion of the ideal physics, it is very hard to distinguish between 8-bit and 16-bit recording quality.



14.4 Time Domain Operations

First we consider three kinds of operations on sound files in the time domain: slicing, playback frequency changes, and sound file frequency changes.

14.4.1 Slicing and Concatenating Sound

Consider the problem of constructing an “audio ransom note” by choosing and assembling words from published speeches. This exercise is strictly experimental in nature, but can produce some interesting results. There are a number of Web sites that offer free .wav files containing memorable speech fragments from movies, TV shows, and cartoons.

The Companion Website (www.aw.com/smith_mat) contains a sampling of such files. In particular, it has the *Apollo 13* speech, “Houston, we have a problem”; “Frankly, my dear ...” from *Gone with the Wind*; and “You can’t handle the truth” from *A Few Good Men*. Exercise 14.2 describes the process of assembling parts of these speeches into a semi-coherent conversation. Consider the following exercises that could be turned into scripts.

The first part of Exercise 14.2 reads the *Apollo 13* speech, plays the speech, and plots the data (with the data index as x-axis). The resulting plot is shown in the left half of Figure 14.2. Since the sound actually includes more than we need, the next step is to crop this file to keep only the words we need. By listening to the speech using the function `sound(...)`, and judiciously zooming and panning the plot, it is possible to narrow down the location in



Exercise 14.2 Locating the first part of the speech

Do It Yourself

Step 1: Find the location of problem speech.

```
>> [houston, Fsh] = wavread('a13prob.wav');
>> subplot(1, 2, 1)
>> plot(houston);
>> sound(houston, Fsh);
```

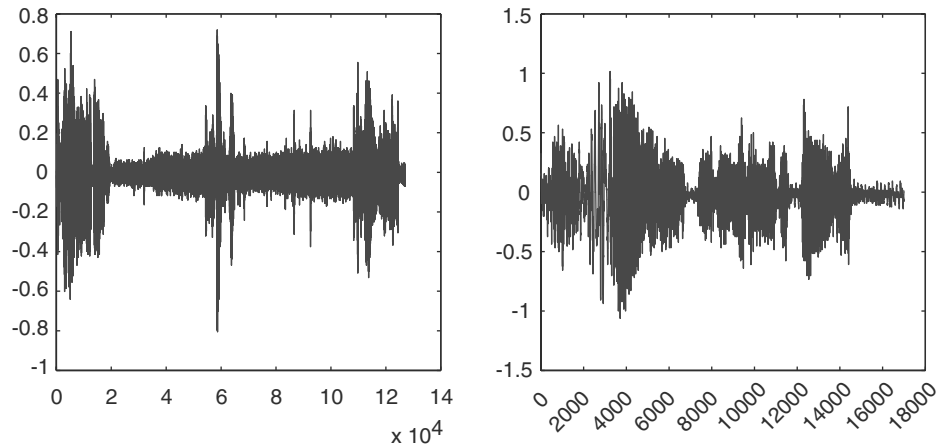


Figure 14.2 Apollo 13 speech

the file where the problem speech starts, at about 111000. In Exercise 14.3 you will extract the first part of the speech.

In Exercise 14.3 we truncate the speech file to the words we need and also, realizing that the amplitude of these words is a little low, raise its amplitude by a factor of 2.

In Exercise 14.4, by a similar process, we remove “my dear” from the “frankly, my dear ...” speech, reducing its amplitude by one-half, which results in Figure 14.3.



Exercise 14.3 Extracting the first part of the speech

Do It Yourself

Step 2: Extract the problem speech.

```
>> clip = 110000;
>> prob = houston(clip:end)*2;
>> subplot(1, 2, 2)
>> plot(prob)
```



Exercise 14.4 Extracting the second part

Do It Yourself

Step 3: Remove “my dear”.

```
>> figure
>> [damn, Fsd] = wavread('givdamn2.wav');
>> subplot(1, 2, 1)
>> plot(damn);
>> lo = 4500;
>> hi = 8700;
>> sdamn = [damn(1:lo); damn(hi:end) ] * .5;
>> subplot(1, 2, 2)
>> plot(sdamn);
```

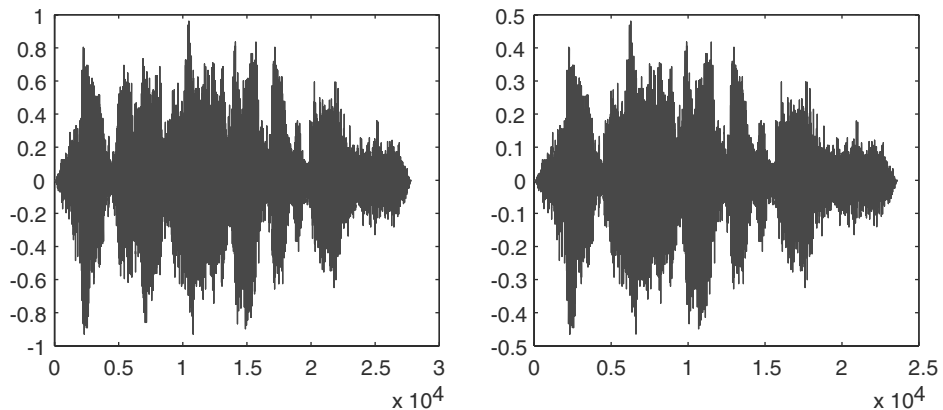


Figure 14.3 *Gone with the Wind* speech

Finally, in Exercise 14.5, we assemble the complete speech by concatenating these two fragments with the speech from *A Few Good Men*. The resulting picture is shown in Figure 14.4.

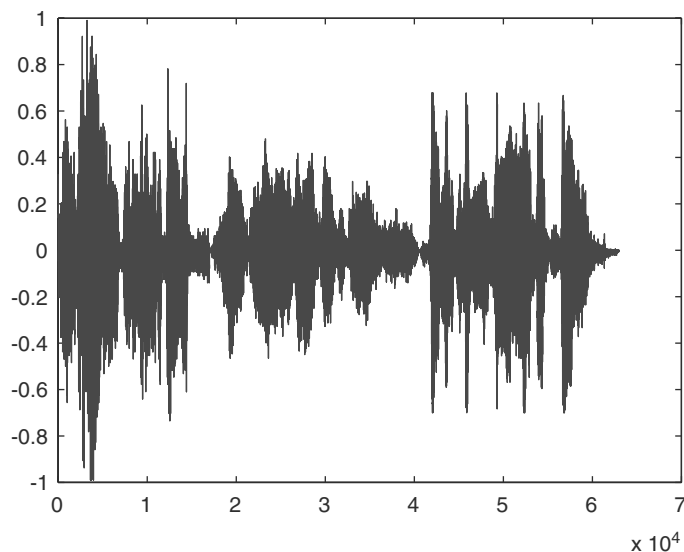


Exercise 14.5 Completing the speech

Do It Yourself

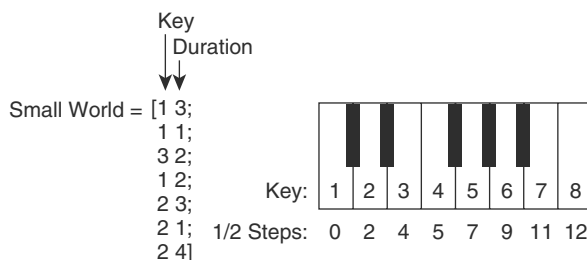
Step 4: Assemble the speech.

```
>> [truth, Fst] = wavread('truth1.wav');
>> speech = [prob; sdamn; truth * .7];
>> figure
>> plot(speech);
>> sound(speech, Fst);
```

Figure 14.4 *The complete speech*

14.4.2 Musical Background

For good historical reasons, music is usually described graphically on a music score. The graphics describe for each note to be played its pitch and its duration, together with other notations indicating how to introduce expression and quality into the music. However, this graphical notation is not amenable to the simple representation of music we need for these experiments. Rather, we will use the representation illustrated in Figure 14.5. The right side of this figure shows a standard piano keyboard, the index of each white note, and the number of half steps necessary to achieve the pitch of each note. On the left side of the figure, we see the method to be used in this text to describe simple tunes. It will consist of an array with two columns and n rows, where n is the number of notes to be played for each tune. The first column is the key number to play, and the second column is the number of beats each note should be held.

Figure 14.5 *Musical notes*

The examples to follow will manipulate the file `piano.wav` to produce a snippet of music. This file is a recording¹ of a single note played on a piano. Other files provided in the resource center are the same note played on a variety of instruments. There are two ways to accomplish this, as follows:

1. Playing each note at a different playback frequency
2. Stretching or shrinking each note to match the required note pitch and playing them all at the same playback frequency

The first way is easier to understand and code, but very inflexible; the second method is a little more difficult to implement, but completely extensible.

Musically speaking, if a sound is played at twice its natural frequency, it is heard as one musical octave higher. The steps from one note to the next higher octave are divided into 7 increments: 5 whole note steps and 2 half note steps, for a total of 12 half note steps. These 12 half steps are logarithmically divided where the frequency change between half-note steps is $2^{1/12}$.

14.4.3 Changing Sound Frequency Poorly

We begin by implementing a scale and a tune by changing the playback frequency. There are a few practical issues to deal with in this simple example:

- Since different notes are played at different frequencies, we cannot concatenate different notes in the same sound file.
- Changing the playback frequency also changes the duration of the note because it consumes more or fewer data items in a given period of time. Therefore, we have to manage the number of data items sent to the `sound(...)` function.
- Since the `sound(...)` function starts playing the sound, but immediately continues to the next line of code, it is necessary to pause until the sound is finished playing before continuing to process the next sound.

Play a Scale First, we examine one way to play a scale—keys 1–8 played in order at a steady pace. Consider the code shown in Listing 14.1 that plays a major scale on the piano (the white notes seen in Figure 14.5) and saves the frequencies and durations for subsequent use.

Listing 14.1 Playing a scale poorly

```
1. [note, Fs] = wavread('instr_piano.wav');
2. count = floor(length(note)/8);
3. wait = count / Fs;
4. whole = 2^(1/6);
5. half = 2^(1/12);
6. for i = 1:8
```

continued on next page

¹ <http://chronos.ece.miami.edu/~dasp/samples/samples.html>.

```

7.      cnt(i) = count;
8.      freq(i) = Fs;
9.      sound(note(1: count), Fs);
10.     pause(wait)
11.     if (i == 3) || (i == 7)
12.         Fs = Fs * half;
13.         count = round(count * half);
14.     else
15.         Fs = Fs * whole;
16.         count = round(count * whole);
17.     end
18. end

```

In Listing 14.1:

Line 1: Reads the piano note file.

Line 2: We arbitrarily decide that the duration of each note will be one-eighth of the length of the piano note.

Line 3: We compute the wait time in seconds between notes.

Lines 4 and 5: Compute the frequency ratios for half notes and whole notes.

Line 6: Plays the eight white notes one at a time.

Lines 7 and 8: Save the length and frequency of each note.

Line 9: Plays the note.

Line 10: Pauses to allow the sound to be completed.

Lines 11–17: Update the duration and frequency. Referring to Figure 14.5, after the third and seventh notes, there is a half-note step; for all the others, it is a whole-note step. We increase the playback frequency by that factor, and to maintain the duration of the note, we also increase the number of samples played by the same factor.

Play a Simple Tune Now we extend the above script by using an array of note frequencies and durations to play the simple tune diagrammed in Figure 14.5. A script that extends the code from Listing 14.1 to play this tune is shown in Listing 14.2.

Listing 14.2 Play a tune fragment poorly

```

1. smallworld = [ 1 3; 1 1; 3 2; 1 2; 2 3; 2 1; 2 4]
2. l = length(smallworld);
3. for i = 1:l
4.     f = freq(smallworld(i,1));
5.     n = cnt(smallworld(i,1)) * smallworld(i,2);
6.     sound(note(1:n), f);
7.     pause(wait*smallworld(i,2));
8. end

```

In Listing 14.2:

- Line 1: Shows the tune array diagrammed in Figure 14.5.
- Lines 2 and 3: Play the notes for each row in the array.
- Line 4: Extracts the frequency for the current note previously stored in the vector `freq(...)`.
- Line 5: Computes the note duration as the stored duration multiplied by the number of note steps required by the tune.
- Lines 6 and 7: Play the note and pause as before.

14.4.4 Changing Sound Frequency Well

This time-domain exercise is to manipulate the piano note file to produce a snippet of music at the same playback frequency throughout. In order to change the note frequency without changing the playback frequency, we have to remove or add the appropriate number of data samples from or to the original data file. This is exactly the same process as we used to stretch or shrink images in Section 13.4.1. Use Exercise 14.6 to experiment with this technique for playing notes at different pitches.

In Exercise 14.6, first we read and play the note at its natural frequency. Then we raise its pitch by removing about one-third of the samples, and then lower the pitch by an octave by doubling the number of samples.

Play a Scale Listing 14.3 shows a script that uses this capability to play the same scale as before on the piano. It repeatedly shortens the vector `newNote` to increase the frequency of the note played.

In Listing 14.3:

- Lines 1–3: Read the note and set the step multipliers.
- Line 4: Plays eight notes of the scale.
- Lines 5 and 6: Play the note and pause.
- Lines 7–11: Choose the appropriate factor.
- Line 12: Shrinks the note file by the chosen factor.



Exercise 14.6 Note pitch experiment

Do It Yourself

Changing a note frequency by indexing:

```
>> [note Fs] = wavread('instr_piano.wav');
>> sound(note, Fs);
>> sound(note(ceil(1:1.3:end)), Fs);
>> sound(note(ceil(1:0.5:end)), Fs);
```

Listing 14.3 Play a scale by shrinking the note

```

1. [note, Fs] = wavread('instr_piano.wav');
2. half = 2^(1/12);
3. whole = half^2;
4. for index = 1:8
5.     sound(note, Fs);
6.     pause(.5);
7.     if (index == 3) || (index == 7)
8.         mult = half;
9.     else
10.        mult = whole;
11.    end
12.    note = note(ceil(1:mult:end));
13. end;

```

Play a Simple Tune We now write a script to build a playable .wav file using the note shrinking technique. The script is shown in Listing 14.4. It uses the array `steps` to decide how many half-tone steps are necessary to reach the *n*th note on the scale, and uses the array `doremi` to define the tune. The array `doremi` is illustrated in Figure 14.5—the first column specifies the pitch (the note on the scale), and the second the duration in “beats.” The script sets the beat time to be 0.2 seconds.

The goal of the script is to put the notes into a single sound array called `tune`, as illustrated in Figure 14.6, rather than playing the notes “on the fly.”

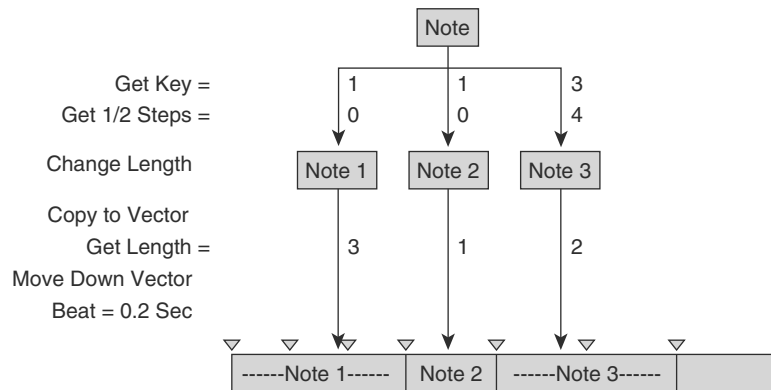


Figure 14.6 Building a tune file

This is accomplished as follows:

- Create an empty array, `tune`, of the appropriate length (the length of the original note plus the total number of beats in the song)
- Initialize `storeAt` to store the first note at the start of the tune
- Iterate across the `tune` definition array `doremi` with the following steps:

- Start with the original note
- Get the key index to decide how many times to raise the note array by half a step
- Raise the note to the right pitch and save it as theNote
- Add that theNote vector to the tune vector, starting at storeAt
- Move the storeAt variable down the tune vector a distance equivalent to the duration of that note

When all the notes have been added to the tune file, play the tune and save it as a .wav file.

In Listing 14.4:

Lines 1–6: Read the file and set up the parameters.

Line 7: A vector defining how many half-steps it takes to set the frequency of notes 1–8.

Line 8: The time between notes of length 1—the beat of the tune.

Line 9: The number of samples to play for one beat of the tune.

Line 10: Begins storing notes at the beginning of the tune.

Line 11: Inserts each note in the song file into the tune file.

Line 12: Fetches the key number.

Line 13: Extracts the number of half-steps required for this note.

Line 14: Stretches the bass note by this multiplier.

Lines 15 and 16: Compute where the end of the note will be stored.

Listing 14.4 Building a tune file

```

1. [note, Fs] = wavread('instr_piano.wav');
2. half = 2^(1/12);
3. doremi = [1 3; 2 1; 3 3; 1 1; 3 2; 1 2; 3 4; 2 3;
4.           3 1; 4 1; 4 1; 3 1; 2 1; 4 8; 3 3; 4 1;
5.           5 3; 3 1; 5 2; 3 2; 5 4; 4 3; 5 1; 6 1;
6.           6 1; 5 1; 4 1; 6 4];
7. steps = [0 2 4 5 7 9 11 12];
8. dt = .2;
9. nCt = floor(dt*Fs);
10. storeAt = 1;
11. for index = 1:length(doremi)
12.     key = doremi(index,1);
13.     pow = steps(key);
14.     theNote = note(ceil(1:half^pow:end));
15.     noteLength = length(theNote);
16.     noteEnd = storeAt + noteLength - 1;
17.     tune(storeAt:noteEnd,1) = theNote;
18.     storeAt = storeAt + doremi(index,2) * nCt;
19. end
20. sound(tune, Fs)
21. wavwrite(tune, Fs, 'dohAdeer.wav')

```

Line 17: Copies the note into the tune file.

Line 18: Advances the `storeAt` index down the tune file by the beat count multiplied by the beats required for this note.

Lines 20 and 21: Play the complete tune and save it as a `.wav` file.



14.5 The Fast Fourier Transform

Typically, the time history display of a sound shows you the amplitude of the sound as a function of time but makes no attempt at showing the frequency content. While this works for slicing voice files, we are often more interested in the frequency content of a sound file, for which we need a different presentation—a spectrum display.

14.5.1 Background

In general, a spectrum display shows the amount of sound energy in a given frequency band throughout the duration of the sound analyzed but ignores the time at which the sound at that frequency was generated. Many acoustic amplifiers (see Figure 14.7) include two features that allow you to customize the sound output:

- A spectral display that changes values as the sound is played, indicating the amount of sound energy (vertically) in different frequency bands (horizontally)
- Filter controls to change the relative amplification in different frequency bands

In the following paragraphs, we will consider only the analysis of the sound frequency content. The ability to reshape the sound frequency content as the sound plays is beyond the scope of this text.



Figure 14.7 A typical spectrum display

To achieve the motion of the spectrum display, software to analyze a segment of the sound file runs periodically and updates the spectrum display. Typically, perhaps 20 times a second, 1/20th second of sound file is analyzed and transformed. The software used for this conversion is known as the Fourier transform.

The mathematics of the Fourier transform is beyond the scope of this book. However, we can make use of the tools it offers without concerning ourselves with the mathematics. There are a number of implementations of this transform; perhaps the most commonly used is the Fast Fourier Transform (FFT). The FFT uses clever matrix manipulations to optimize the mathematics needed to generate the forward (time to frequency) and reverse (frequency to time) transforms.

14.5.2 MATLAB Implementation

Figure 14.8 illustrates the overall process of transforming between the time domain and frequency domain. It starts with a simple sound file, a vector of N sound values in the range $(-1.0$ to $1.0)$, which, if played back at a sample frequency F_s entries per second, reproduces the sound. Examining the parameters involved, we see the following:

N	the number of samples
F_s	the sampling frequency
Δt	the time between samples, computed as $1/F_s$
$T_{\max}$	the maximum time is $N \times \Delta t$

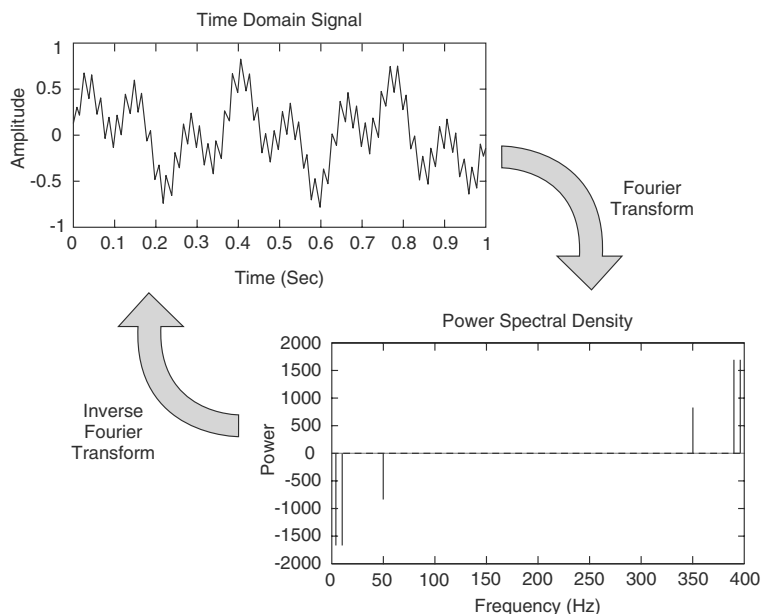


Figure 14.8 *Mechanics of the Fourier transform*

The FFT consumes a file with these characteristics and produces a frequency spectrum with a corresponding set of characteristics. The frequency spectrum consists of the same number, N , of data points, each of which is a complex value with real and imaginary parts. (While many displays actually plot the magnitude of the spectrum values, to accomplish the inverse transform, the complex values must be retained.) The frequency values are “folded” on the plot so that zero frequency occurs at either end of the spectrum, and the maximum frequency occurs in the middle, at spectrum data point $N/2$.

The equivalent characteristics for the spectrum data are as follows:

- N the number of samples
- Δf the frequency difference between samples, computed as
 $1/t_{\max}$
- $f_{\max}$ the maximum frequency is $1/2 \times N \times \Delta f$

In MATLAB, the FFT is mechanized using the function `fft(...)`, which consumes the time history and produces the complex spectrum file. The inverse FFT function, `ifft(...)`, takes a spectrum array with these parameters and reconstructs the time history. This pair of functions provides a powerful set of tools for manipulating sound files.

14.5.3 Simple Spectral Analysis

Listing 14.5 illustrates a script that creates 10 seconds of an 8 Hz sine wave, plots the first second of it, performs the FFT, and plots the real and imaginary parts of the spectrum. Notice the following:

- A sine wave in the time domain transforms to a line in the frequency domain because all its energy is concentrated at that frequency—8 Hz in this example.
- Since the FFT is a linear process, multiple sine or cosine waves added together at different frequencies have additive effects in the spectrum.
- The resulting spectrum is complex (with real and imaginary parts) and symmetrical about its center, the point of maximum frequency. On the MATLAB plot, of course, one cannot make the frequency axis labels reduce from the center to the end.
- The real part of the spectrum is mirrored about the center; the imaginary part is mirrored and inverted.
- The phase of the complex spectrum retains the position of the sine wave in the time domain—it would be totally real for a cosine wave symmetrically placed in time, and totally imaginary for a sine wave in the same relationship.
- The mathematics involved in computing the magnitude of the spectrum value requires integrating $\sin^2 \omega t$ in time that results in $t/2$. Because t is 10,000 samples, the steady state value of the spectral amplitude is 5,000.

The script in Listing 14.5 creates three subplots: the original sine wave, and then the amplitude and phase of the spectrum.

In Listing 14.5:

Lines 1–5: Set up the time domain signal.

Lines 6–10: Plot the front part of the time trace.

Line 11: Performs the FFT.

Lines 12–14: Set up the frequency plots.

Lines 15–19: Plot the spectrum real part.

Lines 20–24: Plot the spectrum imaginary part.

Figure 14.9 shows the result from running this script. It confirms the earlier statement that the real part of the spectrum is mirrored about the center frequency, and the imaginary part is mirrored and inverted.

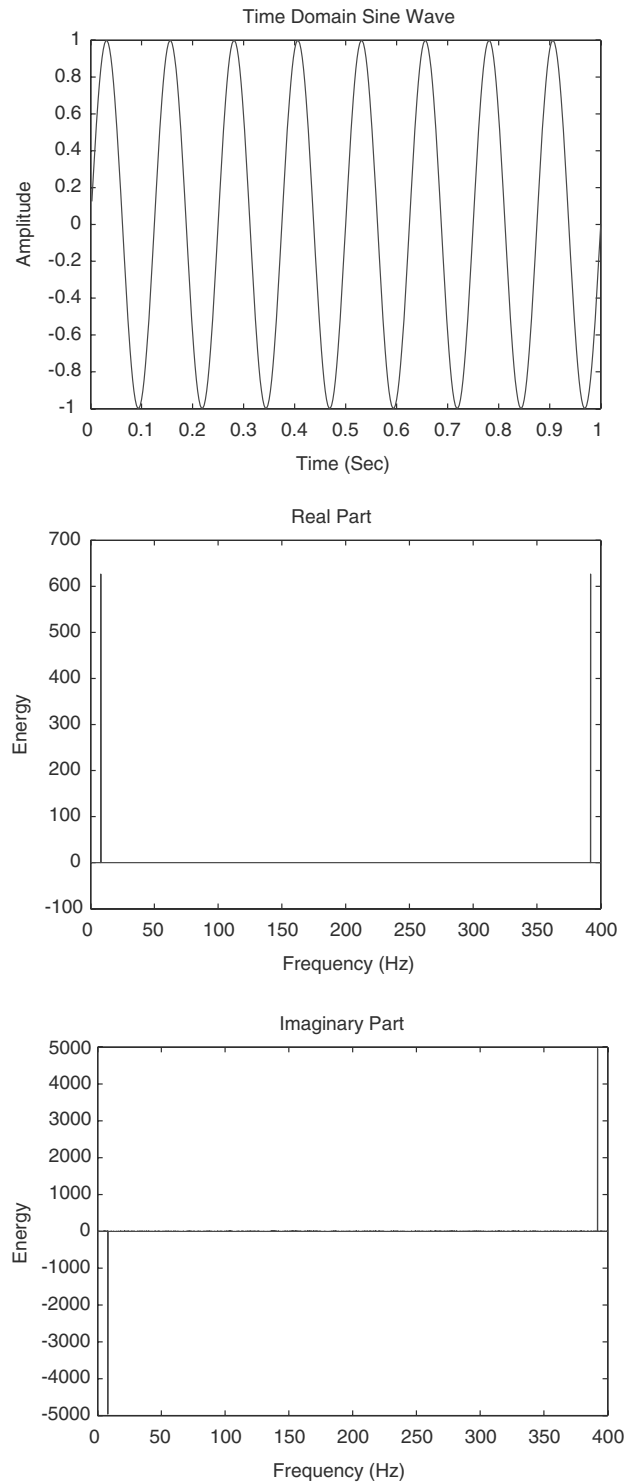
Listing 14.5 FFT of a sine wave

```

1. dt = 1/400                % sampling period (sec)
2. pts = 10000               % number of points
3. f = 8                     % frequency
4. t = (1:pts) * dt;         % time array for plotting
5. x = sin(2*pi*f*t);
6. subplot(3, 1, 1)
7. plot(t(1:end/25), x(1:end/25));
8. title('Time Domain Sine Wave')
9. ylabel('Amplitude')
10. xlabel('Time (Sec)')
11. Y = fft(x);              % perform the transform
12. df = 1 / t(end)          % the frequency interval
13. fmax = df * pts / 2
14. f = (1:pts) * 2 * fmax / pts; % frequencies for plotting

15. subplot(3, 1, 2)
16. plot(f, real(Y))
17. title('Real Part')
18. xlabel('Frequency (Hz)')
19. ylabel('Energy')
20. subplot(3, 1, 3)
21. plot(f, imag(Y))
22. title('Imaginary Part')
23. xlabel('Frequency (Hz)')
24. ylabel('Energy')

```

**Figure 14.9** *FFT of a sine wave*



14.6 Frequency Domain Operations

We will consider three applications in the frequency domain—analyzing the spectral quality of different musical instruments, inserting a sine wave in the frequency domain, and a simple form of digital filtering.

14.6.1 Analyzing Instrument Sounds

The intent of this section is to develop a plot showing the spectra of a selection of different musical instruments. We will first build a function that plots the spectrum for a single instrument, and then build the script to create all the plots.

Listing 14.6 shows a function that reads the .wav file of an instrument from the music samples in the University of Miami's Audio and Signal Processing Laboratory.² All the instruments are carefully playing a note at about 260 Hz.

In Listing 14.6:

Line 1: Shows a function consuming two strings: the name of the instrument and the title of the plot.

Lines 2–5: Read the file and set up the plot parameters.

Line 6: Performs the FFT and computes the absolute value.

Lines 7 and 8: Scale the plot to be a percentage of the maximum energy at any frequency.

Lines 9–16: Set up and plot the first 10 percent of the spectrum.

The script that uses this function to plot the instrument data is shown in Listing 14.7.

Listing 14.6 Plotting the spectrum of one instrument

```
1. function inst(name, ttl)
   % plot the spectrum of the instrument with
   % the given name, with the given plot title
2. [x, Fs] = wavread(['instr_' name '.wav']);
3. N = length(x);
4. dt = 1/Fs; % sampling period (sec)
5. t = (1:N) * dt; % time array for plotting
6. Y = abs(fft(x)); % perform the transform
7. mx = max(Y);
8. Y = Y * 100 / mx;
9. df = 1 / t(end) ; % the frequency interval
10. fmax = df * N / 2 ;
11. f = (1:N) * 2 * fmax / N;
12. up = floor(N/10);
13. plot(f(1:up), Y(1:up) );
14. title(ttl)
15. xlabel('Frequency (Hz)')
16. ylabel('Energy')
```

² <http://chronos.ece.miami.edu/~dasp/samples/samples.html>

Listing 14.7 Script to plot eight-instrument spectra

```

1. rows = 4; cols = 2
2. subplot(rows, cols, 1)
3. inst('sax', 'Saxophone');
4. subplot(rows, cols, 2)
5. inst('flute', 'Flute');
6. subplot(rows, cols, 3)
7. inst('tbone', 'Trombone');
8. subplot(rows, cols, 4)
9. inst('piano', 'Piano');
10. subplot(rows, cols, 5)
11. inst('tpt', 'Trumpet');
12. subplot(rows, cols, 6)
13. inst('mutetpt', 'Muted Trumpet');
14. subplot(rows, cols, 7)
15. inst('violin', 'Violin');
16. subplot(rows, cols, 8)
17. inst('cello', 'Cello');

```

In Listing 14.7:

Line 1: Sets up the subplot configuration.

Lines 2–17: Each pair of lines makes the subplot of one instrument.

The results are shown in Figure 14.10. It is interesting to notice the following:

- All the instruments have significant amounts of energy at even multiples of the source frequency
- The flute and piano are perhaps the purest tones with the least harmonics
- The brass instruments have most of their energy in the second, third, and fourth harmonics rather than the base frequency
- The strings have very strong third harmonic components
- Those instruments with noticeable vibrato when played (flute, violin, and cello) show a marked fuzziness in their spectral peaks

Notice the following:

- None of the instruments produces a pure tone. The lowest frequency at which there is energy is usually called the fundamental frequency, and successive peaks to the right at multiples of the fundamental frequency are referred to, for example, as the first, second, and third harmonics.
- Several instruments have much more energy in the harmonics than in the fundamental frequency.
- “Families” of instruments have similar spectral shapes—the strings, for example, have strong fundamental and second harmonic energy. In principle, these characteristic spectral “signatures” can be used to synthesize the sound of instruments, and even to identify individual instruments when played in groups.

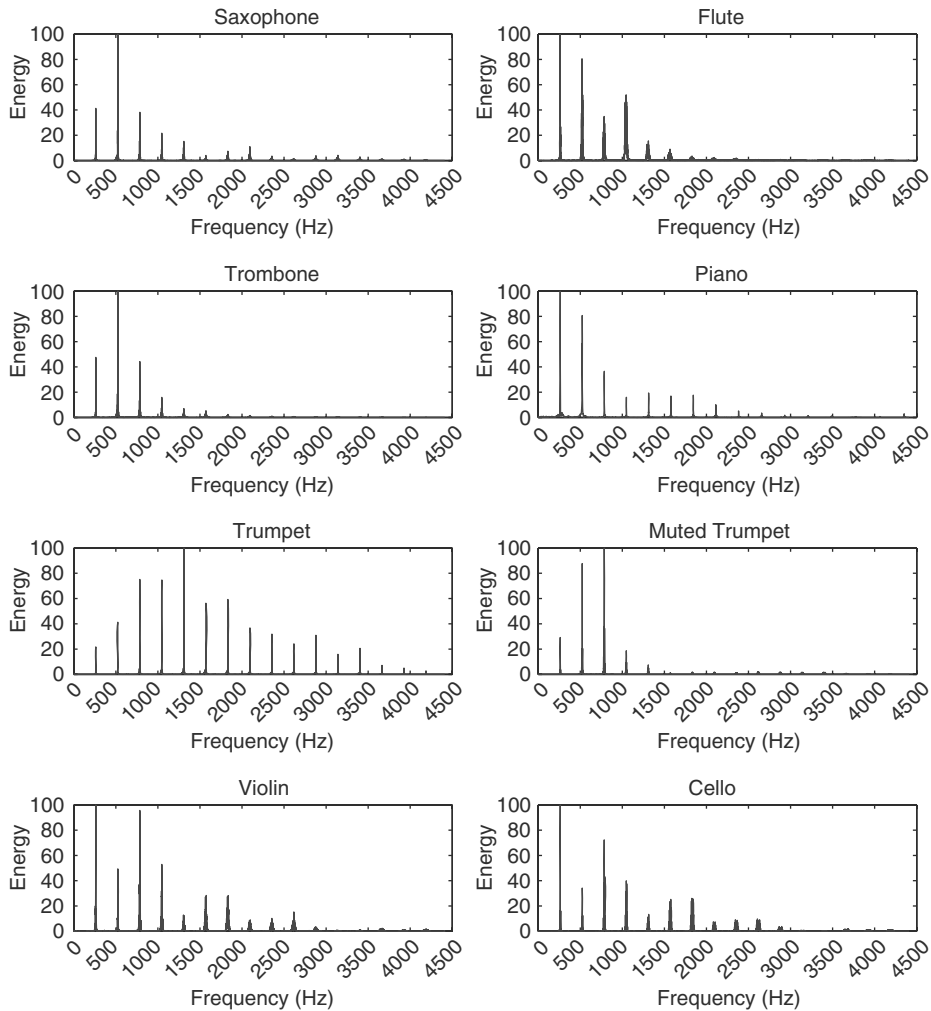


Figure 14.10 *Instrument spectra*

14.6.2 Adding Sounds to the Spectrum

The goal of this section is to understand how to build a signal with two sine waves (3 Hz and 8 Hz) in the time domain, perform the FFT, add a 50 Hz wave in the spectrum, and perform the inverse FFT to construct the new time history. The script to accomplish this is shown in Listing 14.8.

Listing 14.8 Adding a signal to the spectrum

```
1. rows = 2; cols = 2;
   % set up two sine waves
2. dt = 1/400; % sampling period (sec)
3. N = 10000; % number of points
```

continued on next page

```

4. f1 = 3; f2 = 8; % signal frequencies
5. t = (1:N) * dt; % time array for plotting
6. x = sin(2*pi*f1*t) + sin(2*pi*f2*t);
7. subplot(rows, cols, 1)
8. plot(t(1:(1/dt)), x(1:(1/dt)))
9. title('Original Signal')
10. xlabel('Time (Sec)')
11. Y = fft(x); % perform the transform
12. df = 1 / t(end); % the frequency interval
13. fmax = df * N / 2;
14. f = (1:N) * 2 * fmax / N; % frequencies
15. subplot(rows, cols, 2) % plot the spectrum
16. plot(f(1:N/3), abs(Y(1:N/3)))
17. title('Spectrum')
18. xlabel('Frequency (Hz)')
    % find the maximum value and location
19. level1 = max(abs(Y));
20. newF = 50;
21. newBasic = 50 / df;
22. newI1 = newBasic + 1;
23. newI2 = N - newBasic + 1;
24. newV = complex(0, -level1/2);
25. Y(newI1) = newV;
26. Y(newI2) = -newV; % imaginary part negated
27. subplot(rows, cols, 4) % plot the new spectrum
28. plot(f(1:N/3), abs(Y(1:N/3)))
29. title('Spectrum with Added Signal')
30. xlabel('Frequency (Hz)')
31. y = ifft(Y);
32. subplot(rows, cols, 3)
33. plot(t(1:(1/dt)), y(1:(1/dt)))
34. title('Reconstructed the New Signal')
35. xlabel('Time (Sec)')

```

In Listing 14.8:

Line 1: Sets up the subplot configuration.

Lines 2–5: Set up the time domain parameters.

Line 6: The time trace consisting of two added frequencies.

Lines 7–10: Plot the original signal.

Line 11: Performs the FFT, preserving the complex values.

Lines 12–18: Set up and plot the absolute values of the spectrum.

Line 19: Finds the peak level on the plot.

Lines 20 and 21: Compute the basic index of the position of a 50 Hz signal.

Lines 22 and 23: The positions of the mirrored spectral spikes at 50 Hz.

Line 24: The complex values (all imaginary) to insert into the spectrum at half the maximum level.

Lines 25 and 26: Insert the spikes—one is negated because they are imaginary.

Lines 27–30: Plot the updated spectrum.

Line 31: Performs the inverse FFT.

Lines 32–35: Plot the new time trace with the 50 Hz signal added.

Figure 14.11 shows the results.

14.6.3 Manipulating the Spectrum

Adding pure sine waves to the spectrum is reasonably straightforward as long as the symmetry of the spectrum components is preserved. It is possible to consider more complex operations on the spectrum as long as these operations are performed in areas of the spectrum where there is no significant sound energy.

Consider, for example, an apparently simple project. If we compare the spectra of the trumpet and the muted trumpet shown on the third row of Figure 14.10, we might consider operating on the spectrum to apply “digital

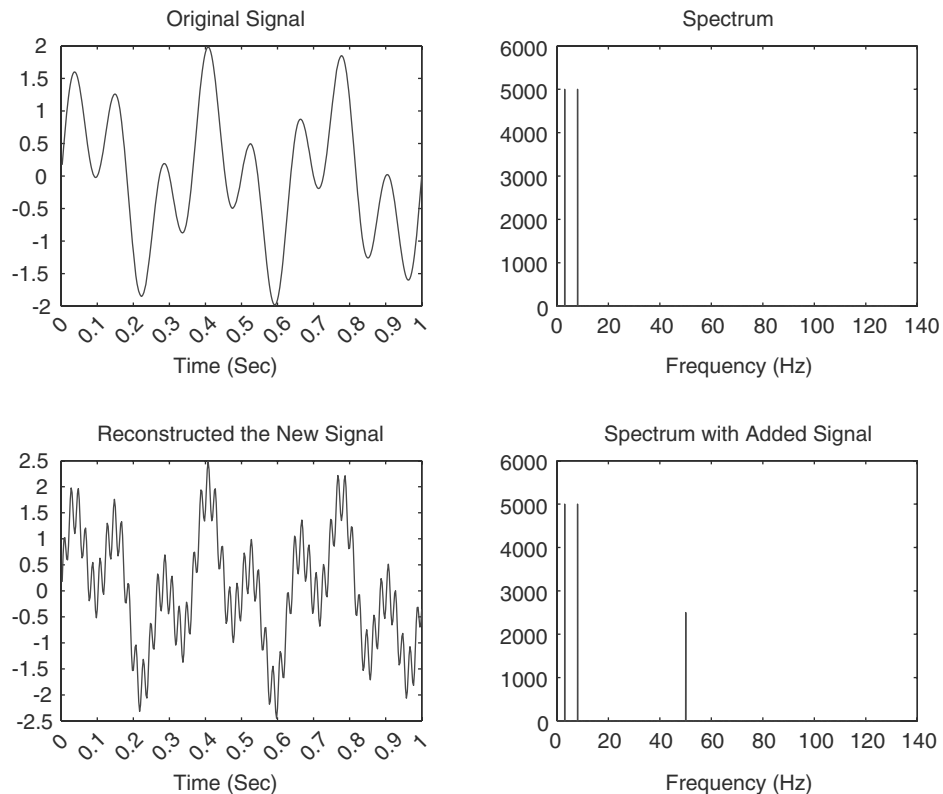


Figure 14.11 *Adding a signal*

muting” to the trumpet spectrum. It seems that this could be achieved by reducing the third harmonic by 80 percent, and eliminating all the energy in the trumpet spectrum above it.

The code shown in Listing 14.9 implements a crude approximation to such an effort, manually defining the location of the third harmonic, reducing that to 20 percent of its original size, and then zero filling the spectrum above that to remove all other harmonics.

In Listing 14.9:

Lines 1–4: Read the trumpet file and set up the parameters.

Lines 5 and 6: Play the sound and wait for it to finish.

Listing 14.9 Synthetic trumpet muting

```

1. [x, Fs] = wavread('instr_tpt.wav');
2. N = length(x);
3. dt = 1/Fs ;           % sampling period (sec)
4. t = (1:N) * dt; % time array for plotting
5. sound(x, Fs)
6. pause(t(N));
7. Y = fft(x);           % perform the transform
8. df = 1 / t(end); % the frequency interval
9. fmax = df * N / 2;
10. f = (1:N) * 2 * fmax / N;
11. subplot(1, 2, 1);
12. frac = floor(N/10);
13. plot(f(1:frac), abs(Y(1:frac)));
14. title('trumpet original')
15. xlabel('frequency (Hz)')
16. ylabel('energy')
17. fcut = 925; %Hz
18. icut = floor(fcut / df);
19. foff = 1100; %Hz
20. ioff = floor(foff / df);
21. Y(icut:end-icut) = Y(icut:end-icut) * 0.2;
22. Y(ioff:end-ioff) = 0;
23. subplot(1, 2, 2);
24. plot(f(1:frac), abs(Y(1:frac)));
25. title('truncated trumpet')
26. xlabel('frequency (Hz)')
27. ylabel('energy')
28. y = abs(ifft(Y));
29. y = y / max(y);
30. disp('synthetic muted');
31. sound(y, Fs);
32. pause(t(N));
33. disp('real muted');
34. [x, Fs] = wavread('instr_mutetpt.wav');
35. sound(x / (max(x)*4), Fs)

```

Lines 7–16: Perform the FFT and plot the first 10 percent of the spectrum.

Lines 17–20: Since the second harmonic lies at 800 Hz, and the third at around 1070 Hz ($4 * \text{the fundamental } 266 \text{ Hz frequency}$), we estimate the positions where the energy must be cut between the second and third harmonics, and where it must be zero, and compute the corresponding index values.

Line 21: Reduces the energy where it should be cut by 80 percent, not forgetting to leave in place the mirrored portion of the spectrum.

Line 22: Zeros out the remaining energy, again leaving the mirrored portion untouched.

Lines 23–27: Plot the digitally muted spectrum.

Line 28: Performs the inverse FFT and takes the absolute value to eliminate any stray complex values introduced by asymmetry in the spectrum.

Line 29: Scales the volume to match the original sound volume.

Lines 30–32: Play the synthetic muted trumpet.

Lines 33–35: Play the real muted trumpet.

Figure 14.12 suggests that the resulting spectrum is superficially similar to a muted trumpet. However, if you run the script and listen, even an untrained ear would observe that the sound is not really the same as a real muted trumpet. The subtleties of level in the harmonics are sufficient to be noticeable even to an untrained ear.

Common Pitfalls

Changes in the spectrum have to be made under circumstances where they do not introduce sudden discontinuities in the spectral levels. These sudden discontinuities cause large, resonant “ringing” patterns in the time domain that can overpower the audio effects being created. The experiments performed in this section succeeded because they were either deliberately introducing spikes to add a single frequency, or changing the spectrum where there was no energy at the edges of the changes.

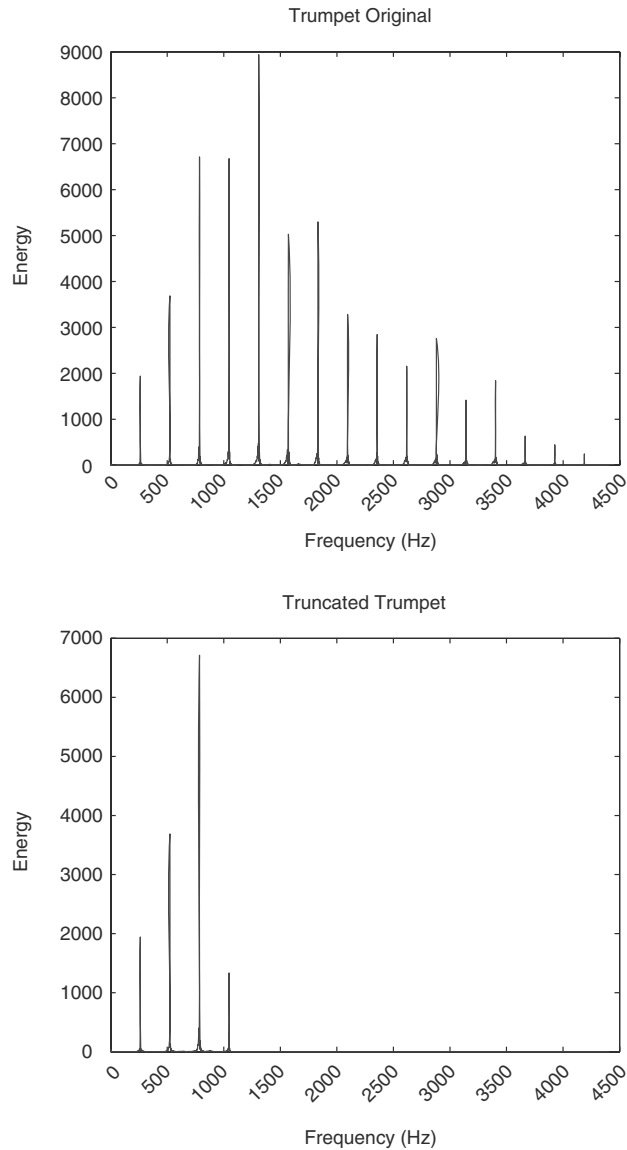


Figure 14.12 *Synthetic muted trumpet*

14.7 Engineering Example—Oil Rig Structural Integrity

Like icebergs, large off-shore oil rigs have an imposing amount of structure visible above the surface of the ocean, but even more structure beneath the surface. The Petronius Platform in the Gulf of Mexico, operated by Chevron and Marathon Oil, is arguably one of the world's tallest structures at 610 meters (over 2,000 feet) tall. Only 75 meters of its height is above the surface of the

sea. It cost half a billion dollars to build and delivers 50,000 barrels of oil and two million cubic meters of gas every day. Figure 14.13 shows a typical oil rig on which several hundred workers are permanently housed. Especially in hostile climate conditions, these structures vibrate significantly and are susceptible to structural damage due to corrosion and metal fatigue. A number of such rigs have sustained enough structural damage that they physically collapsed, at the cost of lives, loss of oil production, and the resources required to replace the structures.



Figure 14.13 *Oil platform*

The problem we address here is the need to determine whether the structural integrity has been compromised. The size of the structures and the depth of the water make most normal inspection procedures either physically impossible or prohibitively expensive. One technique that has shown promise is the analysis of the modes of vibration of the structure. Figure 14.14 shows the configuration of an experiment performed in 1981 to determine the feasibility of this approach. It shows the configuration of a “small” oil rig model, a shaker to vibrate the structure, a transducer to record the vibration of the structure, and the location of some deliberately induced structural damage.

The premise of this technique is that any flexible structure, like a violin string, has modes of vibration with characteristic frequencies. If the physical characteristics of the structure change—like changing the length of the violin string by placing a finger on the string—we expect the frequency of the modes of vibration to change. We can detect the frequency of these modes of vibration by capturing the vibrations at the transducer for a period of time and then performing an FFT on the data. Figure 14.15 shows the results of one such experiment.

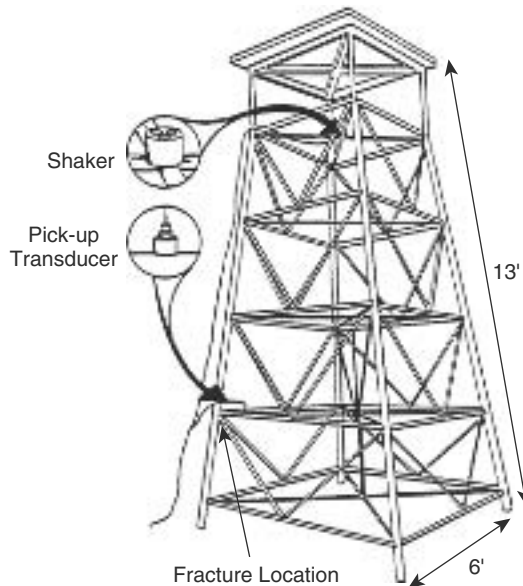


Figure 14.14 Model diagram

Figure 14.15 illustrates many of the practical difficulties associated with using this technique:

- The amount of noise generated across the spectrum makes the peaks hard to distinguish
- The noise caused by the instrumentation, such as the 60 Hz peak
- The number and resolution of the peaks

The goal of studies such as this is to determine whether the instrumentation can detect damage to a real oil rig before that damage causes structural failure, and whether the natural impact of the sea and wind can provide enough excitation of the structure to avoid the need to artificially stimulate the structure.

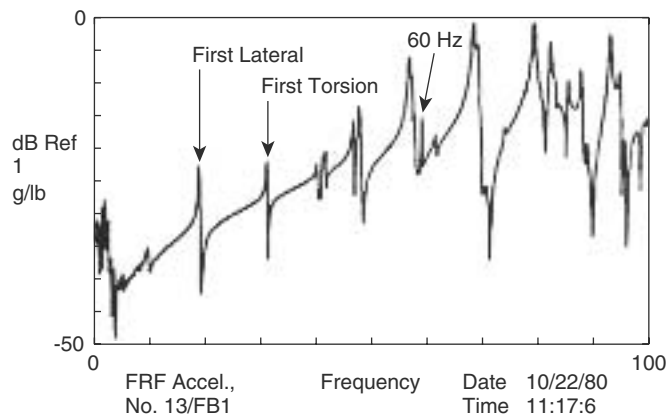


Figure 14.15 Analysis results

As a footnote, one of the oil rigs lost in the North Sea was being analyzed by an early implementation of this capability. It took a long time to resolve the legal liability issues associated with the failure of the monitoring equipment to detect the structural problem in time.

Chapter Summary

This chapter presented the following:

- Sounds are read into MATLAB with specific readers that provide a time history and sampling frequency
- Sounds can be played through the computer’s sound system and saved to disk as a sound file ready for playing on any digital player
- We can slice and concatenate sounds to edit speeches and change the frequency of the sound to change its pitch
- We can analyze the frequency content of sound using the Fast Fourier Transform (FFT)
- We can modify the spectra by adding, deleting, or changing the sound levels at chosen frequencies under certain controlled conditions

Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>[data Fs nb]</code> <code>= auread(file)</code>	Reads an .au sound file in .wav format	14.3
<code>auwrite((data, Fs, nb, file)</code>	Writes a sound file in .au format	14.3
<code>fft(ftime)</code>	Performs the Fast Fourier Transform on a sound file	14.5.2
<code>ifft(ffreq)</code>	Performs the inverse Fourier transform on a spectrum file	14.5.2
<code>sound(data, Fs)</code>	Plays a sound file	14.3
<code>[data Fs nb] =</code> <code>wavread(file)</code>	Reads a .wav sound file in .wav format	14.3
<code>wavwrite(data, Fs, nb, file)</code>	Writes a sound file in .wav format	14.3

**Self Test**

Use the following questions to check your understanding of the material in this chapter:

True or False

1. Playing a sound file at double the recorded sample frequency raises its pitch by an octave.
2. Removing every other sample from a sound file lowers the pitch by an octave.
3. The resolution of the recorded data has a significant effect on the quality of the recording.
4. After performing an FFT, the zero frequency occurs at either end of the spectrum and the maximum frequency occurs in the middle.
5. Since the mathematics of the FFT are linear, the spectrum of a sound added in the time domain is also added in the frequency domain.

Fill in the Blanks

1. Sound pressure fluctuations have two attributes: their _____ and their _____ characteristics.
2. Each word coming out of the _____ or going into the _____ merely represents the _____ on the microphone at a point in time.
3. The steps from one note to the next higher octave are divided into _____ increments: _____ whole note steps and _____ half note steps, for a total of _____ half note steps.
4. A spectrum display shows the amount of _____ in a given _____ throughout the duration of the sound analyzed.



Programming Projects

1. These are fundamental exercises with sound files. You should not hard-code any of the answers for this problem, and you should not need iteration.
 - a. Select and read a suitable .wav file and save the sound values and sampling frequency.
 - b. Create a new sound that has double the frequency of the original sound and store your answer in the variable `sound_Double`.
 - c. Create a new sound that is the same as the original except that the pitch is raised by 5 half tones. Store your answer in the variable `raised_pitch`.
 - d. We need a figure showing two views each of these three sounds, created using `subplot`. In the left column, plot the original sound, `sound_Double`, and `raised_pitch`, labeling each plot accordingly. In the right column, plot the first quarter of the values of the power spectrum of each sound with the proper frequency values on the horizontal axis.
 - e. Play each of the sounds in the following order: original sound, `sound_Double`, and `raised_pitch` each at the original sampling frequency.
2. Write a function that will accept a string specifying a sound file and do the following:
 - a. Play back the sound.
 - b. Plot the sound in the time domain, titling and labeling your plot appropriately.
 - c. Compute the frequency with the most energy in this file. Validate your answer by plotting the lower quarter of the frequencies of the Fourier transform of the sound. Don't forget that the Fourier Transform is complex; you will need to reason with and plot the absolute value of the spectrum.
 - d. Test this function with suitable .wav files.
3. Write a function named `plotSound` that takes in the name of a sound file and produces a 1×2 figure with two plots. The first plot should be a plot of the sound in the time domain. The second plot should be a plot of the sound in the frequency domain. Your function should not return anything.

Label the first plot 'Time Domain' and label its axes appropriately. Label second plot 'Frequency Domain' and label its axes appropriately. The Time Domain plot should be an amplitude vs. time plot. For simplicity make sure your time vector starts at Δt (delta time) and goes to $n \cdot \Delta t$ (tmax) where n is the number of samples.

The Frequency Domain plot should be a power vs. frequency plot where power is the absolute value of the FFT of the amplitude values. For simplicity make sure your frequency vector starts at Δf (delta frequency) and goes to $n \cdot \Delta f$ ($2 \cdot f_{\max}$).

4. In this exercise, we will write a script to create an instrument sound from scratch.
 - a. Create a vector, τ , of time values from 0 to 2 seconds with length 40,000 samples.
 - b. Convert the frequency of middle C (261.6 Hz) to ω radians per second.
 - c. Compute a sound sample as $\cos(\omega \tau)$ over the range of τ in part a.
 - d. Play that sound at a sample frequency of 20,000 and verify that it sounds “about right.”
 - e. Perform the Fourier transform on the sound vector, establish the correct axis values and prove that the sound is exactly Middle C.
5. A sound can be scrambled by doing the following in order:
 - a. The sound is reduced to $1/3$ of its original “volume.” Specifically, the amplitude was reduced to $1/3$ of its original.
 - b. The sound is reversed. In other words, it would play backwards.
 - c. A 0.6 second gap of silence is added in the middle and then at the end.
 - d. The final sound vector is written to a sound file with a sampling frequency of $1.5 \cdot f_s$, where f_s was the original sampling frequency.

Write a function called `mysteryScrambler` that takes in the name of a sound file and a control string. If the string is 'scramble', your function should scramble the sound according to the above method and then write it to a file named `scrambled_<name of sound file>`. If the string is 'unscramble', your function should unscramble the sound assuming that it was scrambled using the above method; this means that your function will have to do the opposite of each step in the reverse order. Your function should then write the unscrambled sound to a file named `unscrambled_<name of sound file>`. If the string is neither 'unscramble' nor 'scramble', your function should do nothing.

Test your solution by finding and playing an original sound, scrambling it and playing the result, and then unscrambling it and playing the result.

6. Write a function named `playNote` that takes in a string representing a note on the piano. Your function should return a vector representing the amplitude values of the note in addition to the correct sampling frequency to be used to play it back. You should

do this by modifying the sound in the provided `c_note.wav` file which is Middle C played on the piano. Note that the returned sampling frequency should be the same as that in `c_note.wav`.

Here is a list of all the possible note names representing notes that your function should work with and below that is the number of half steps above/below the middle C for that note:

c	c#	d	d#	e	f	f#	g	g#	a	a#	b
-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
cc	cc#	dd	dd#	ee	ff	ff#	gg	gg#	aa	aa#	bb
ccc											
0	1	2	3	4	5	6	7	8	9	10	11
12											

where `cc` is the middle C, `ccc` is 1 octave above it and `c` is 1 octave below it. Similarly, `ff` is 1 octave higher than `f`, `dd` is 1 octave higher than `d` and so on.

For example:

`[y1 fs] = playNote('cc');` should return a vector such that `sound(y1, fs)` should sound like middle C

7. Finally, you will use these tools to play your favorite song.
 - a. Find the music for your favorite song and translate it into the symbology of Problem 14.6.
 - b. Write a script that uses the `playNote` function to play your song on the piano.
 - c. Modify `playNote` to use your synthetic instrument from Problem 14.4 and save it as `playSynthetic`.
 - d. Write a script that uses `playSynthetic` to play your song in futuristic style.

Numerical Methods

Chapter Objectives

This chapter discusses the implementations of four common numerical techniques:

- Interpolating data
- Fitting polynomial curves to data
- Numerical integration
- Numerical differentiation

Introduction

Real world data is rarely in such a form that you can use it immediately. Frequently, the data must be manipulated according to the user's actual needs:

- If the data samples have correct values but are not close enough together to be used directly, we can use interpolation to compute data points between the samples provided.
- There are occasions where the data-gathering facilities add some amount of noise to the data. To minimize the effects of the noise, we can compute the coefficients of a polynomial function that best matches the data.
- There are also times when the data must be integrated or differentiated to derive the quantities of interest.

15.1 Interpolation

15.1.1 Linear Interpolation

15.1.2 Cubic Spline Interpolation

15.1.3 Extrapolation

15.2 Curve Fitting

15.2.1 Linear Regression

15.2.2 Polynomial Regression

15.2.3 Practical Application

15.3 Numerical Integration

15.3.1 Determination of the Complete Integral

15.3.2 Continuous Integration Problems

15.4 Numerical Differentiation

15.4.1 Difference Expressions

15.4.2 MATLAB Implementation

15.5 Engineering Example—Analyzing Rocket Data

15.6 Engineering Example—Cleaning Up Images

15.1 Interpolation

If our data samples have correct values but are not close enough to be used directly, we can use either linear or cubic interpolation to compute data points between the samples provided. For example, the MATLAB plotting functions use linear interpolation to draw the lines between data points.

In general, interpolation is a technique by which we estimate a variable's value between known values. In this section we present the two most common types of interpolation: linear interpolation and cubic spline interpolation. In both techniques, we assume that we have a set of data points that represents a set of x - y coordinates for which y is a function of x ; that is, $y = f(x)$. We then have a value of x that is not part of the data set for which we want to find the y value. Figure 15.1 illustrates the definition of the interpolation problem.

15.1.1 Linear Interpolation

Linear interpolation is one of the most common techniques for estimating data values between two given data points. With this technique we assume that the function between the points can be represented by a straight line drawn between the points, as shown in Figure 15.2. Since we can find the equation of a straight line defined by the two known points, we can find y for any value of x . The closer the points are to each other, the more accurate our approximation is likely to be. Of course, we could use this equation to extrapolate points past our collected data. This is rarely wise, however, and often leads to significant errors.

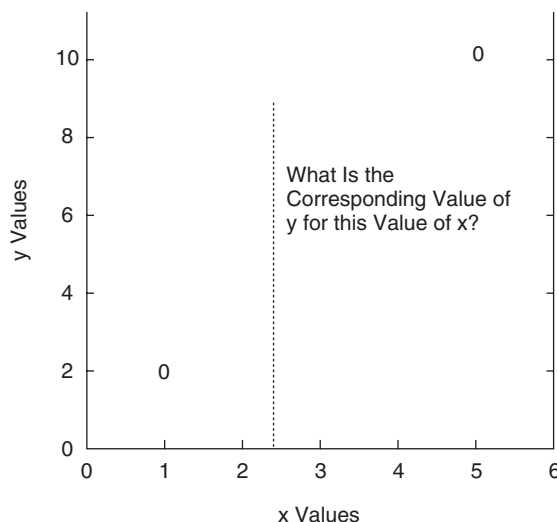


Figure 15.1 The interpolation problem

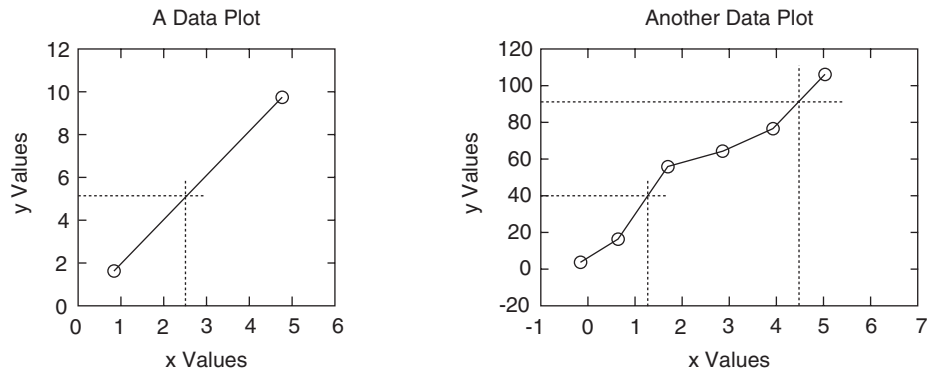


Figure 15.2 *Linear interpolation*

The MATLAB function that performs linear interpolation is as follows:

```
new_y = interp1(x, y, new_x)
```

where the vectors *x* and *y* contain the original data values and the vector *new_x* contains the point(s) for which we want to compute interpolated *new_y* values. The *x* values should be in ascending order, and the *new_x* values should be within the range of the *x* values. Note that the last character in the name `interp1` is the numeric 1 (one), not a lowercase L.

These forms are demonstrated in Exercise 15.1.

In Exercise 15.1 we use the data illustrated in Figure 15.2. First we take a single interpolated reading from the data at $x = 1.5$, and then we plot circles spaced 0.2 units apart on the *x*-axis, as shown in Figure 15.3. Notice that the circles fall on the straight lines between the given data values. Finally, we attempt to extrapolate to the point $x = -0.5$, and see that MATLAB returns `NaN`.



Exercise 15.1 Linear interpolation

Do It Yourself

Enter the following MATLAB commands:

```
>> x = 0:5;
>> y = [0, 20, 60, 68, 77, 110];
>> interp1(x, y, 1.5)
ans =
    40
>> new_x = 0:0.2:5;
>> new_y = interp1(x,y,new_x);
>> plot(x, y, new_x, new_y, 'o')
>> axis([-1,7,-20,120])
>> title('linear Interpolation Plot')
>> xlabel('x values') ; ylabel('y values')
>> interp1(x, y, -0.5)
ans =
NaN
```

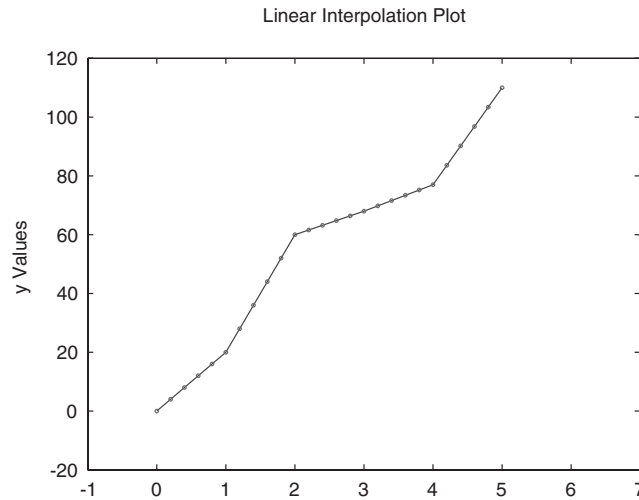


Figure 15.3 *Linear interpolation*

For the advanced student, MATLAB allows us to provide a fourth parameter to the `interp1` function that must be a string that modifies its behavior. The choices are as follows:

- 'nearest' nearest neighbor interpolation
- 'linear' linear interpolation—the default
- 'spline' piecewise cubic spline interpolation (see Section 15.1.2)
- 'pchip' shape-preserving piecewise cubic interpolation
- 'cubic' same as 'pchip'
- 'v5cubic' the cubic interpolation from MATLAB 5, which does not extrapolate, and uses 'spline' if x is not equally spaced

MATLAB also provides two-dimensional (`interp2`) and three-dimensional (`interp3`) interpolation functions, which are not discussed here. Refer to the MATLAB help feature for more information.

15.1.2 Cubic Spline Interpolation

A **cubic spline** is a smooth curve constructed to go through a set of points. The curve between each pair of points is a third-degree polynomial that has the general form:

$$x = a_{x0}t^3 + a_{x1}t^2 + a_{x2}t + a_{x3} \text{ and}$$

$$y = a_{y0}t^3 + a_{y1}t^2 + a_{y2}t + a_{y3}$$

where t is a parameter ranging from 0 to 1 between each pair of points. The coefficients are computed so that this provides a smooth curve between the two points and a smooth transition between the adjacent curves. Figure 15.4

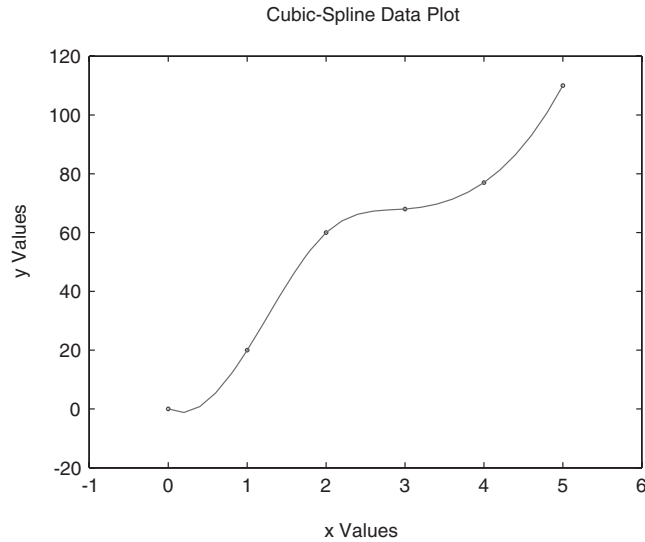


Figure 15.4 Cubic spline interpolation

shows a cubic spline smoothly connecting six points using a total of five different cubic equations.

The MATLAB function that performs linear interpolation is as follows:

```
new_y = spline(x, y, new_x);
```

Style Points

A good convention to adopt is shown in Figure 15.4:

- Use symbols to plot data points that are real values with no associated information connecting them
- Draw lines between data points only when there is an analytic relationship that connects the data points

Here, we use a circle symbol for the raw data to emphasize the original source of the information, and a smooth line for the spline curve to indicate that we are assuming a possibly erroneous but continuous relationship between data points.

where the vectors x and y contain the original data values, and the vector x_{new} contains the point(s) for which we want to compute interpolated y_{new} values. The x values should be in ascending order, and the x_{new} values should be within the range of the x values.

The curve in Figure 15.4 was created using the code shown in Listing 15.1.

Listing 15.1 Spline interpolation

```
1. x = 0:5;
2. y = [0, 20, 60, 68, 77, 110];
3. new_x = 0:0.2:5;
4. new_y = spline(x, y, new_x);
5. plot(x, y, 'o', new_x, new_y, '-')
6. axis([-1,6,-20,120])
7. title('Cubic-Spline Data Plot')
8. xlabel('x values'); ylabel('y values')
```

In Listing 15.1:

Lines 1 and 2: Show the original x and y values.

Line 3: Shows dense x values to define the curve.

Line 4: Computes the spline function.

Lines 5–8: Plot the original data and the smooth curve.

15.1.3 Extrapolation

A note of caution about extrapolation—attempting to infer the values of data points outside the range of data provided is problematic. Although logically your code may allow you to, you should never do it. The MATLAB `interp1` and `spline` functions behave differently in this respect.

As we saw previously, the `interp1` function refuses to supply results outside the range of the original x data. If you try, for every `new_x` value outside the range of the original x values, it will return `NaN`—not a number. This is actually quite nice because if you accidentally request interpolated data like this, the plot programs ignore `NaN` values.

The `spline` function, however, has no such scruples, and allows you to request any x values you want, using the equation of the closest line segment. So considering Figure 15.4, if you asked for the value at $x = -3$, it would use the segment between 0 and 1, which has a violent upswing at the lower end (see Exercise 15.2).

This might be what you want, but it looks odd! Chances are the data are not as accurate as you thought, and you probably need to fit a curve to the data, as explained in the following section.



Exercise 15.2 The evils of extrapolation

Do It Yourself

Following Listing 15.1, enter this code:

```
>> spline(x, y, -3)
ans =
    813.3333
```



15.2 Curve Fitting

There are occasions where the data-gathering facilities add some amount of noise to the data. To minimize the effects of the noise, we can compute the coefficients of a polynomial function that best matches the data. The choice of the order of the polynomial must be made by the user, depending upon our understanding of the underlying physics that generated the data.

Assume that we have a set of data points collected from an experiment. After plotting the data points, we find that they generally fall in a straight line. However, if we were to try to draw a straight line through the points, probably only a couple of the points would fall exactly on the line. A least-squares curve fitting method could be used to find the straight line that is the closest to the points, by minimizing the distance from each point to the straight line. Although this line can be considered a “best fit” to the data points, it is possible that none of the points would actually fall on the line of best fit. (Note that this method is very different from interpolation, because the lines used in interpolation actually fall on all of the original data points.)

In the following section, we will discuss fitting a straight line to a set of data points, and then we will discuss fitting a polynomial of higher order.

15.2.1 Linear Regression

Linear regression is the process that determines the linear equation that is the best fit to a set of data points in terms of minimizing the sum of the squared distances between the line and the data points. To understand this process, first we consider the same set of data values used previously, and attempt to “eyeball” a straight line through the data. Assume, for example, that $y = 20x$ is a good estimate of the curve. Listing 15.2 shows the code to plot the points and this estimate.

In Listing 15.2:

Lines 1 and 2: Show the original data points.

Line 3: Is our eyeball estimate.

Lines 4–9: Plot the original data and the estimate.

Looking at the results in Figure 15.5, it appears that $y = 20x$ is a reasonable estimate of a line through the points.

We really need the ability to compare the quality of the fit of this line to other possible estimates, so we compute the difference between the actual y value and the value calculated from the estimate:

```
dy = [0, 0, 20, 8, -3, 10]
```

Listing 15.2 Eyeball linear estimation

```
1. x = 0:5;
2. y = [0 20 60 68 77 110];
3. y2 = 20 * x;
4. plot(x, y, 'o', x, y2);
5. axis([-1 7 -20 120])
6. title('Linear Estimate')
7. xlabel('Time (sec)')
8. ylabel('Temperature (degrees F)')
9. grid
```

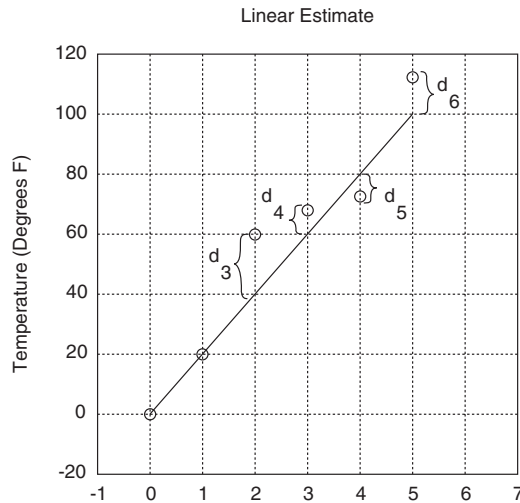


Figure 15.5 An eyeball estimate of a linear fit

It turns out that the best way to make this comparison is by the **least squares technique**, whereby the measure of the quality of the fit is the sum of the squared differences between the actual data points and the linear estimates. This sum can be computed with the following command:

```
>> sum_sq = sum(dy^2)
```

For the above set of data, the value of `sum_sq` is 573. As we will see, MATLAB can automatically produce the best linear fit shown in Figure 15.6 whose sum of squares is 356.82, a significant improvement over our original guess.

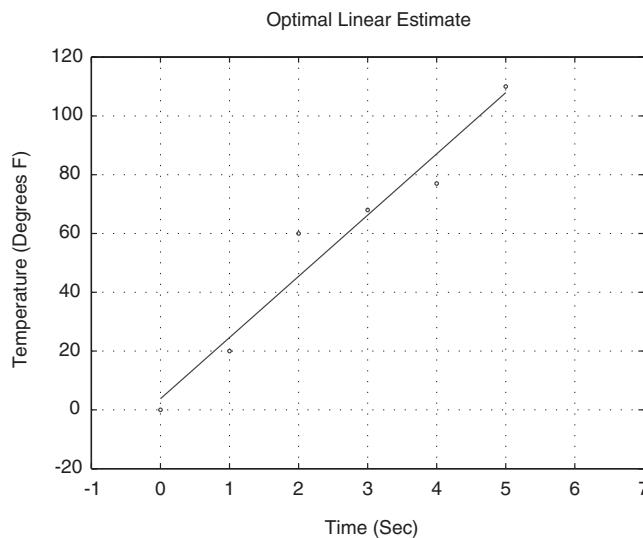


Figure 15.6 Linear curve fit

15.2.2 Polynomial Regression

Linear regression is a special case of the polynomial regression technique. Recall that a polynomial with one variable can be written by using the following formula:

$$f(x) = a_0x^n + a_1x^{n-1} + a_2x^{n-2} + a_3x^{n-3} + \dots a_{n-1}x + a_n$$

The degree of a polynomial is equal to the largest value used as an exponent.

MATLAB provides a pair of functions to compute the coefficients of the best fit to a set of data, and then interpolate on those coefficients to produce the data to plot:

- `coef = polyfit(x, y, n)` computes the coefficients of the polynomial of degree `n` that best matches the given `x` and `y` values. The function returns the coefficients, `coef`, in descending powers of `x`. For the least squares calculation to work, the length of `x` should be greater than `n - 1`. If this is not the case, the coefficients are still computed, but the curve passes through all the data points.
- `new_y = polyval(coef, new_x)` can then be used to interpolate on these coefficients for the `y` value(s) corresponding to any `x` value(s). Note that there is nothing to prevent you from using these coefficients for extrapolation.

Exercise 15.3 illustrates fitting the best straight line to the data used in Section 15.1.1.

So the first-order polynomial that best fits our data is as follows:

$$f(x) = 20.8286x + 3.7169$$

We could interpolate the values of `new_x` with:

```
new_y = coef(10 * new_x + coef(2))
```

or we could use the function `polyval`:

```
new_y = polyval(coef, new_x)
```

We can use our new understanding of the `polyfit` and `polyval` functions to write a program to study the improvement in the curve fit as `n` increases, as shown in Listing 15.3.



Exercise 15.3 Optimal linear fit

Do It Yourself

For example, again using the data from Section 15.1.1:

```
>> x=0:5; y=[0,20,60,68,77,110]
>> polyfit(x, y, 1)
ans =
    20.8286    3.7169
```

Listing 15.3 Higher-order fits

```

1. x = 0:5;
2. fine_x = 0:.1:5;
3. y = [0 20 60 68 77 110];
4. for order = 2:5
5.     y2=polyval(polyfit(x,y,order), fine_x);
6.     subplot(2,2,order-1)
7.     plot(x, y, 'o', fine_x, y2)
8.     axis([-1 7 -20 120])
9.     ttl = sprintf('Degree %d Polynomial Fit', ...
                    order );
10.    title(ttl)
11.    xlabel('Time (sec)')
12.    ylabel('Temperature (degrees F)')
13. end

```

In Listing 15.3:

Lines 1–3: Set up the data sets.

Line 4: Studies second- through fifth-order fits.

Line 5: Combines `polyfit` and `polyval` calls to compute the new `y` values.

Lines 6–12: Plot the results. Notice the use of `sprintf(...)` to make a dynamic title for the plots.

The results are shown in Figure 15.7. Notice that with six points, the fifth-order fit goes through all the data points.

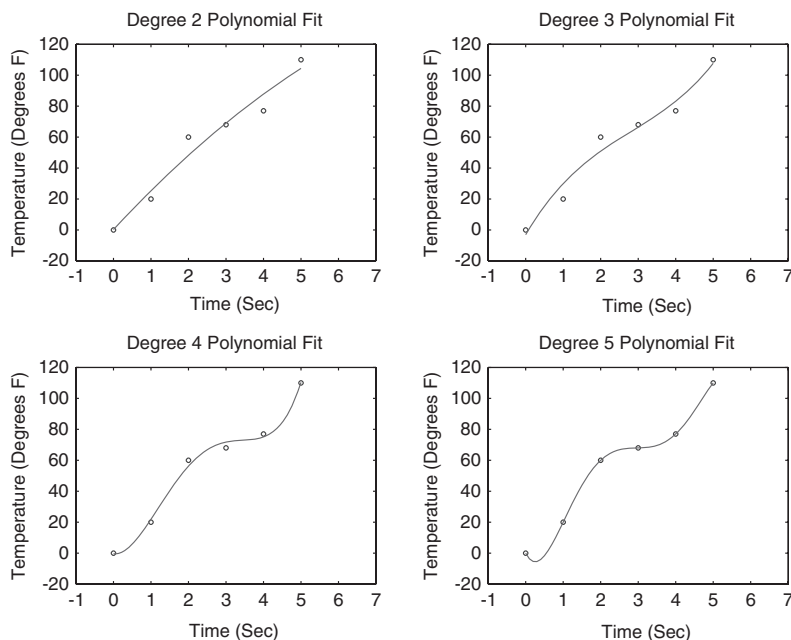


Figure 15.7 Higher-order polynomial fits

15.2.3 Practical Application

We return briefly to the problem of replacing the blue sky in Chapter 13. The sky we used to replace the gray skies of Vienna has a power line we need to remove. We can use polynomial curve fitting to create an artificial sky with exactly the same characteristics as the blue sky in the cottage picture, but without the wire. We merely need to process each row of the sky, fit a second-order curve to it, interpolate a new sky row from the parameters, and replace the row in the sky. The code to perform this is shown in Listing 15.4.

In Listing 15.4:

- Line 1: Reads the original cottage picture.
- Line 2: Obtains its sizes.
- Line 3: Shows the x values for the curve fitting.
- Line 4: Makes a copy of the original picture.
- Line 5: Converts to top 700 rows where the sky is.
- Line 6: Treats each color individually.
- Line 7: The polynomial approximation needs each row as a double vector.
- Lines 8 and 9: Compute a synthetic row.
- Line 10: Puts the row into the new sky.
- Lines 13 and 14: Show and save the new image.

Figure 15.8 shows the cottage picture updated with a smooth sky. Notice that the chimneys have been smeared off, but this does not affect the part of the sky needed for the Vienna picture. This synthetic sky is ready to be used in the script to replace the original sky (see Listing 13.1). Figure 15.9 shows the Vienna picture with a clear blue synthetic sky.

Listing 15.4 Removing the power line from the sky

```

1. p = imread('Witney.jpg');
2. [rows, cols, cls] = size(p);
3. x = 1:cols;
4. sky = p;
5. for row = 1:700
6.     for color = 1:3
7.         cv = double(p(row, :, color));
8.         coef = polyfit(x, cv, 2);
9.         ncr = polyval(coef, x);
10.        sky(row,:,color) = uint8(ncr);
11.    end
12. end
13. image(sky)
14. imwrite(sky, 'sky.jpg');
```



Figure 15.8 *Updated sky*



Figure 15.9 *Updated picture*



15.3 Numerical Integration

The integral of a function $f(x)$ over the interval $[a, b]$ is defined to be the area under the curve of $f(x)$ between a and b , as shown in Figure 15.10. If the value of this integral is K , the notation to represent the integral of $f(x)$ between a and b is as follows:

$$K = \int_a^b f(x) dx$$

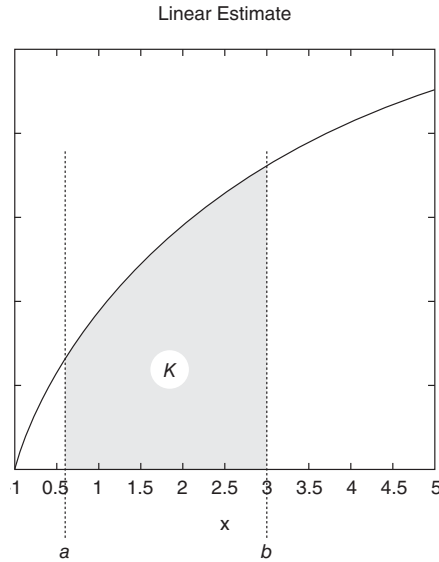


Figure 15.10 *Integration of $f(x)$*

For many functions, this integral can be computed analytically. However, for a number of functions, this is not possible, and we require a numerical technique to estimate its value. We look at two different scenarios:

- Two different techniques for computing the complete integral with various degrees of accuracy
- A technique for evaluating the continuous integral of $f(x)$

15.3.1 Determination of the Complete Integral

Two of the most common numerical integration techniques estimate $f(x)$ either with a set of piecewise linear functions or with a set of piecewise parabolic functions. If we use piecewise linear functions, we can compute the area of the trapezoids that compose the area under the piecewise linear function. This technique is called the **trapezoidal rule**. If we use piecewise quadratic functions, we can compute and add the areas of these components. This technique is called **Simpson's rule**.

The Trapezoidal Rule If we represent the area under a curve by trapezoids, as illustrated in Figure 15.11, and if the interval $[a, b]$ is divided into n equal sections, then the area can be approximated by the following formula:

$$K_T = \frac{b-a}{2n} (f(x_0) + 2f(x_1) + 2f(x_2) + \dots + 2f(x_{n-1}) + f(x_n))$$

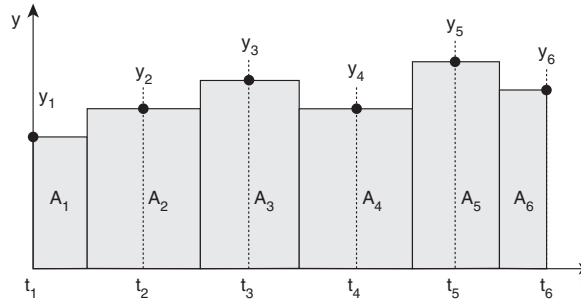
where the x_i values represent the end points of the trapezoids and where $x_0 = a$ and $x_n = b$. Listing 15.5 shows a MATLAB function that computes this integral.

Listing 15.5 Trapezoidal integration

```

1. function K = trapezoid( v, a, b )
2. % h = trapezoid(v, a, b )
3. K = (b-a) * (v(1) + v(end) + ...
    2*sum(v(2:end-1))) / (2*(length(v) - 1) );

```

**Figure 15.11** Discrete integration

Simpson's Rule If the area under a curve is represented by areas under quadratic sections of a curve, and if the interval $[a, b]$ is divided into $2n$ equal sections, then the area can be approximated by the formula (Simpson's rule):

$$K_s = \frac{h}{3} (f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \dots + 2f(x_{2n-2}) + 4f(x_{2n-1}) + f(x_{2n}))$$

where the x_i values represent the end points of the sections, $x_0 = a$ and $x_{2n} = b$, and $h = (b - a) / (2n)$.

Listing 15.6 shows a MATLAB function to integrate using Simpson's rule.

15.3.2 Continuous Integration Problems

We now consider a slightly different scenario. If $f(t)$ is the rate of change of $F(t)$ defined as $f(t) = dF(t)/dt$, then given $f(t)$, we can find $F(t)$ according to the following formula:

$$F(t) = \int_{t_0}^t f(x) dt$$

For example, we might be given data that represents the velocity of a sounding rocket, such as is plotted in Figure 15.12. We need to approximate the altitude of the rocket over time by integrating this data.

Listing 15.6 Simpson's rule integration

```

1. function K = simpson( v, a, b )
2. % h = simpson(v, a, b )
3. K = (b-a) * (v(1) + v(end) + ...
    4*sum(v(2:2:end-1)) + ...
    2*sum(v(3:2:end-2)) ) / (3*(length(v) - 1) );

```

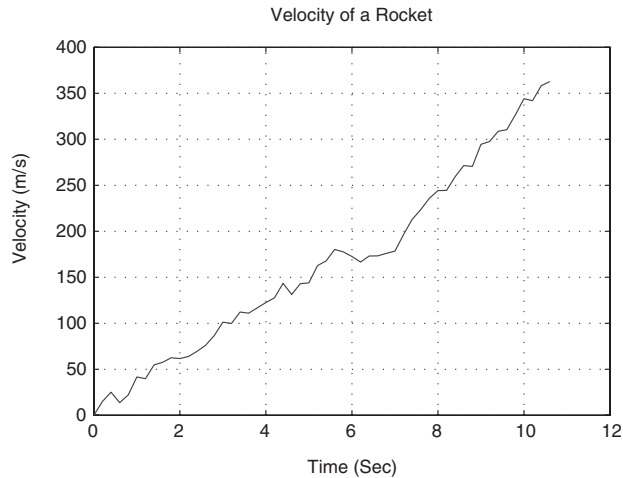


Figure 15.12 *Velocity of a rocket*

To perform this kind of integral, MATLAB provides the function `y = cumsum(v)` that computes the cumulative sum of the vector `v`. The result, `y`, is a vector of the same length as `v` where `y(i)` is the sum of `v(1:i)`. If the data values, `v`, are regularly sampled, the integral is found by multiplying `cumsum(v)` by the time interval. If they are not regularly sampled, you have to compute the `cumsum(...)` of the scalar product of and the vector of time differences.

Listing 15.7 shows the function that computes this continuous integral, making use of `cumsum(...)`.

Listing 15.7 Integrating rocket velocity

```

1. v=[ 0.0 15.1 25.1 13.7 22.2 41.7 ...
      39.8 54.8 57.6 62.6 61.6 63.9 69.6 ...
      76.2 86.7 101.2 99.8 112.2 111.0 ...
      116.8 122.6 127.7 143.4 131.3 143.0 ...
      144.0 162.7 167.8 180.3 177.6 172.6 ...
      166.6 173.1 173.3 176.0 178.5 ...
      196.5 213.0 223.6 235.9 244.2 244.5 ...
      259.4 271.4 270.5 294.5 297.6 ...
      308.7 310.5 326.6 344.1 342.0 358.2 362.7 ];
2. lv = length(v);
3. dt = 0.2;
4. t = (0:lv-1) * dt;
5. h = dt * cumsum(v);
6. plot(t, v, t, h/5)
7. legend({'velocity' 'altitude/5' })
8. title('velocity and altitude of a rocket')
9. xlabel('time (sec)'); ylabel('v (m/s), h(m/5)')
10. fprintf('cumsum height: %g\n', h(end));
11. fprintf('trapezoidal height: %g\n', ...
           trapezoid(v, t(1), t(end)));
12. fprintf('Simpson's Rule height: %g\n', ...
           simpson(v, t(1), t(end)));

```


In Listing 15.7:

- Line 1: Shows the original velocity data.
- Lines 2–4: Parameters for plotting.
- Line 5: Performs the integration.
- Lines 6–9: Plot the results.
- Lines 10–12: Validate the three integration techniques by checking the results.

Figure 15.13 shows the resulting plot. The MATLAB results in the Command window are:

```
cumsum height: 1848.5
trapezoidal height: 1811.85
Simpson's Rule height: 1811.14
```

The continuous integration produces results within 2 percent of the “accurate” integration techniques.

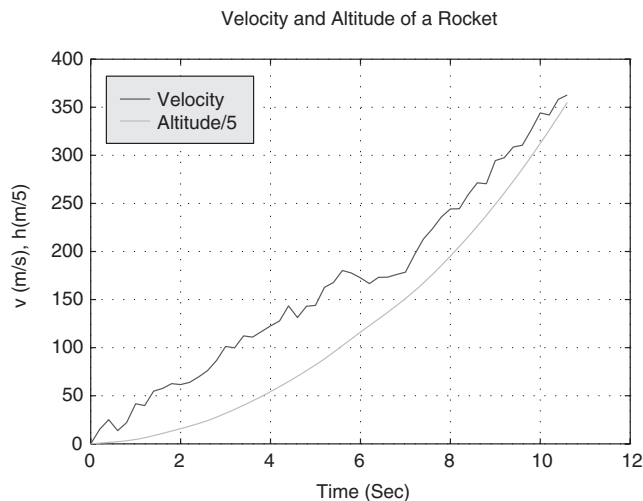


Figure 15.13 Rocket velocity and altitude



15.4 Numerical Differentiation

The derivative of a function $f(x)$ is defined to be a function $f'(x)$ that is equal to the rate of change of $f(x)$ with respect to x . The derivative can be expressed as a ratio, with the change in $f(x)$ indicated by $df(x)$ and the change in x indicated by dx , giving us the following:

$$f'(x) = \frac{df(x)}{dx}$$

There are many physical processes for which we want to measure the rate of change of a variable. For example, velocity is the rate of change of position (as in meters per second), and acceleration is the rate of change of velocity (as in meters per second squared).

The derivative $f'(x)$ can be described graphically as the slope of the function $f(x)$, which is defined to be the slope of the tangent line to the function at the specified point. Thus, the value of $f'(x)$ at the point a is $f'(a)$, and it is equal to the slope of the tangent line at the point a , as shown in Figure 15.14.

15.4.1 Difference Expressions

In general, numerical differentiation techniques estimate the derivative of a function at a point x_k by approximating the slope of the tangent line at x_k using values of the function at points near x_k . The approximation of the slope of the tangent line can be done in several ways, as shown in Figure 15.15.

Figure 15.15(a) assumes that the derivative at x_k is estimated by computing the slope of the line between $f(x_{k-1})$ and $f(x_k)$, as in the following:

$$f'(x_k) = \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

This type of derivative approximation is called a **backward difference approximation**. Figure 15.15(b) assumes that the derivative at x_k is estimated by computing the slope of the line between $f(x_k)$ and $f(x_{k+1})$, as in the following:

$$f'(x_k) = \frac{f(x_{k+1}) - f(x_k)}{x_{k+1} - x_k}$$

This type of derivative approximation is called a **forward difference approximation**. Figure 15.15(c) assumes that the derivative at x_k is estimated by computing the slope of the line between $f(x_{k-1})$ and $f(x_{k+1})$, as in the following:

$$f'(x_k) = \frac{f(x_{k+1}) - f(x_{k-1})}{x_{k+1} - x_{k-1}}$$

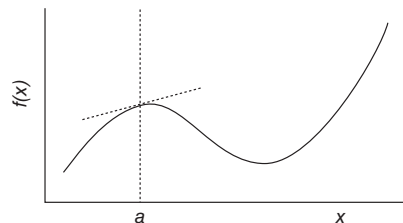


Figure 15.14 Slope of a curve

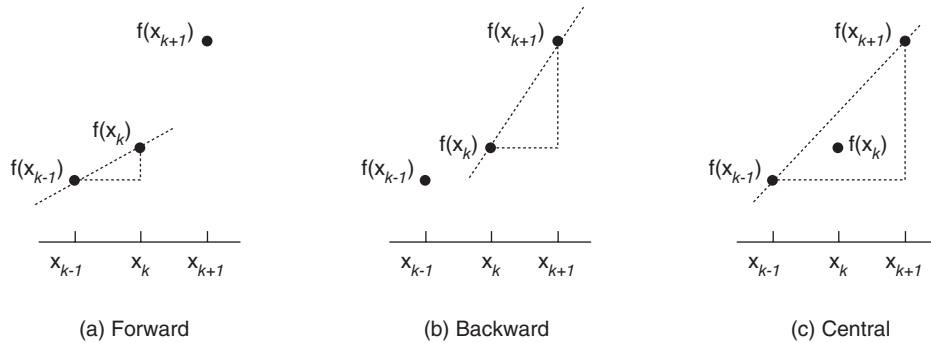


Figure 15.15 Differentiation techniques

This type of derivative approximation is called a **central difference approximation**, and we usually assume that x_k is halfway between x_{k-1} and x_{k+1} . The quality of all of these types of derivative computations depends on the distance between the points used to estimate the derivative; the estimate of the derivative improves as the distance between the two points decreases.

15.4.2 MATLAB Implementation

To facilitate differentiation, MATLAB provides the `diff(...)` function, which computes differences between adjacent values in a vector, generating a new vector with one less value than the original:

`dv = diff(V)` returns $[V(2)-V(1), V(3)-V(2), \dots, V(n)-V(n-1)]$

An approximate derivative dy/dx can be computed by using `diff(y)./diff(x)`. Depending on the application, this can be used to compute the forward, backward, or central difference approximation. The solution to the forward difference is shown in Listing 15.8.

Listing 15.8 Differentiating a function

```
1. x = -7:0.1:9;
2. f = polyval([0.0333,-0.3,-1.3333,16,0,-187.2,0], x);
3. plot(x, f)
4. hold on
5. df = diff(f)./diff(x);
6. plot(x(2:end), df, 'g')
7. grid
8. legend({'f(x)', 'f'(x)'})
```

In Listing 15.8:

Lines 1–4: Establish and plot $f(x)$.

Line 5: The difference expression—returns a vector one shorter than the original.

Lines 6–8: Plot the differences. By plotting at $x(2:end)$, we accomplish the forward difference version where the slope at $x(i)$ is the previous differential value.

The results are shown in Figure 15.16. We could accomplish the backward difference approach by plotting the values at $x(1:end-1)$. The central difference version is left as an exercise for the reader. Suffice it to note that if the samples $x(i)$ are close enough, it really doesn't matter which approximation we use.

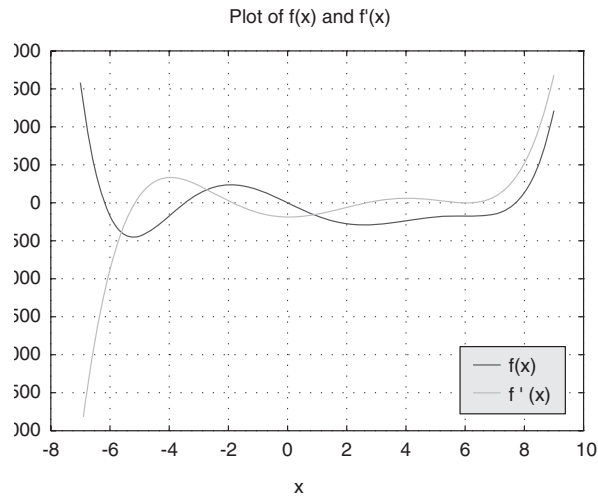


Figure 15.16 Differentiation

15.5 Engineering Example—Analyzing Rocket Data

Returning to the sounding rocket example from the previous sections, suppose we are asked to find the acceleration of the rocket as well as its velocity and altitude. As we have seen, when faced with some data of unknown origin, it is wise to perform an initial survey before launching the actual solution. In our initial survey, we make a raw plot of the rocket velocity and attempt to integrate and differentiate to obtain the altitude and acceleration. The complete script (using the velocity values from above) is shown in Listing 15.9.

Listing 15.9 Raw rocket values

```
1. v = [ 0.0 15.1 25.1 13.7 22.2 41.7 ...
        39.8 54.8 57.6 62.6 61.6 63.9 69.6 ...
        76.2 86.7 101.2 99.8 112.2 111.0 ...
        116.8 122.6 127.7 143.4 131.3 143.0 ...
```

continued on next page

```

144.0 162.7 167.8 180.3 177.6 172.6 ...
166.6 173.1 173.3 176.0 178.5 ...
196.5 213.0 223.6 235.9 244.2 244.5 ...
259.4 271.4 270.5 294.5 297.6 ...
308.7 310.5 326.6 344.1 342.0 358.2 362.7 ];
2. lv = length(v); dt = 0.2; t = (0:lv-1) * dt;
3. h = dt * cumsum(v);
4. acc = diff(v) ./ diff(t);
5. plot(t, v, t, h/5, t(2:end), acc)
6. legend({'velocity' 'altitude/5' 'acceleration'})
7. title('vel, alt and acc measurements of a rocket')
8. xlabel('time (sec)')
9. ylabel('v (m/sec), h(m/5) and acc(m/sec^2)')

```

In Listing 15.9:

Lines 1–3: As before, compute the velocity and altitude.

Line 4: Differentiates to estimate the acceleration.

Lines 5–9: Plot the three results.

The resulting plot is shown in Figure 15.17.

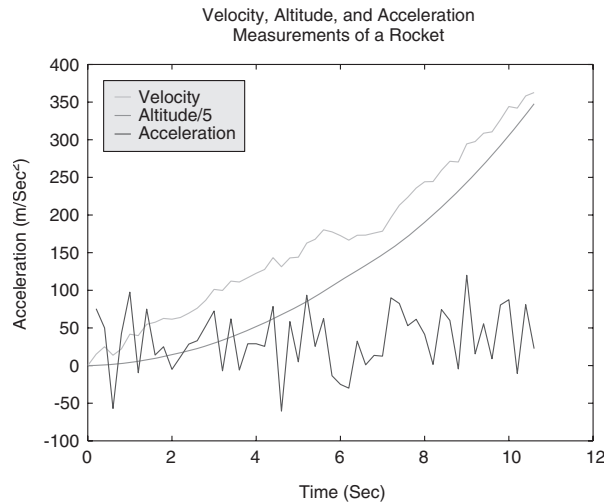


Figure 15.17 *Rocket velocity, altitude, and acceleration*

Having completed the initial survey, we take a look at the data. The results illustrate the problems associated with differentiating inherently noisy signals—whereas integration always smoothes the velocity data, differentiating noisy data amplifies the undesirable effects of the noise.

Our first thought might be to improve the quality of our results by fitting a curve to the velocity data before differentiation. The question must be, however, what order of polynomial to use to fit the data. So we need some further analysis—a short study to assess the right polynomial order. Such a study is shown in Listing 15.10.

Listing 15.10 Curve fitting experiment

```

1. v =[ 0.0 15.1 25.1 13.7 22.2 41.7 ...
      39.8 54.8 57.6 62.6 61.6 63.9 69.6 ...
      76.2 86.7 101.2 99.8 112.2 111.0 ...
      116.8 122.6 127.7 143.4 131.3 143.0 ...
      144.0 162.7 167.8 180.3 177.6 172.6 ...
      166.6 173.1 173.3 176.0 178.5 ...
      196.5 213.0 223.6 235.9 244.2 244.5 ...
      259.4 271.4 270.5 294.5 297.6 ...
      308.7 310.5 326.6 344.1 342.0 358.2 362.7 ];
2. dt = 0.2; t = (0:length(v)-1) * dt;
3. for plt = 1:6
4.     order = plt*3;
5.     y = polyval(polyfit(t, v, order), t);
6.     subplot(2, 3, plt)
7.     plot(t, v, t, y);
8.     str = sprintf('order %d', order);
9.     title(str);
10.    legend({'velocity' 'fitted vel'})
11.    xlabel('time (sec)')
12.    ylabel('vel (m/s)')
13. end

```

In Listing 15.10:

Lines 1–2: Set up $v(t)$ as before.

Line 3: Makes six subplots.

Line 4: The plots will fit the data from order 3 to order 18.

Line 5: Performs the fit and interpolation.

Lines 6–12: Plot the results.

The results are plotted in Figure 15.18. Clearly, with sufficient data samples, we can push the order of the fit to match the data quite well, including the odd-looking “kink” in the middle.

To reach the obvious (erroneous) conclusion of this experiment, we would write a script to differentiate the smoothed velocity. Listing 15.11 shows this code, together with an extra plot that will reveal the error of our thinking.

Listing 15.11 Differentiating the smoothed data

```

1. v =[ 0.0 15.1 25.1 13.7 22.2 41.7 ...
      39.8 54.8 57.6 62.6 61.6 63.9 69.6 ...
      76.2 86.7 101.2 99.8 112.2 111.0 ...
      116.8 122.6 127.7 143.4 131.3 143.0 ...
      144.0 162.7 167.8 180.3 177.6 172.6 ...
      166.6 173.1 173.3 176.0 178.5 ...
      196.5 213.0 223.6 235.9 244.2 244.5 ...
      259.4 271.4 270.5 294.5 297.6 ...
      308.7 310.5 326.6 344.1 342.0 358.2 362.7 ];

```

continued on next page

```

2. dt = 0.2; t = (0:length(v)-1) * dt; g = 9.81;
3. acc = diff(v) ./ diff(t);
4. vs = polyval(polyfit(t, v, 12), t);
5. accs = diff(vs) ./ diff(t);
6. treal = [0 6 6.2 7 7.2 t(end)];
7. areal = g * [3 3 -1 -1 5 5];
8. plot(t, v, t(2:end), acc, ...
        t(2:end), accs, treal, areal)
9. title('attempt to smooth the acceleration')
10. xlabel('time (sec)')
11. ylabel('v (m/sec) and acc(m/sec^2)')

```

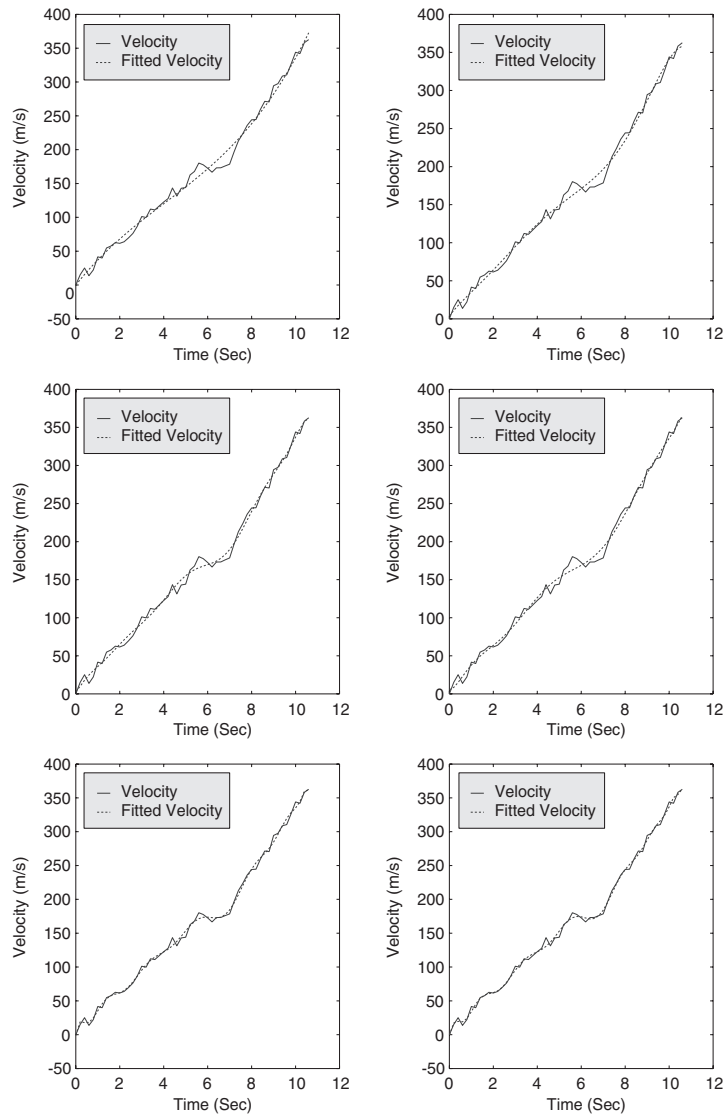


Figure 15.18 *Fit order study*

In Listing 15.11:

Lines 1–3: Velocity and acceleration plots as before.

Line 4: Computes a twelfth-order smooth velocity.

Line 5: Differentiates the smoothed velocity.

Lines 6 and 7: The actual acceleration used to generate the data.

Lines 8–11: Plot the four lines.

Figure 15.19 shows the resulting plot.

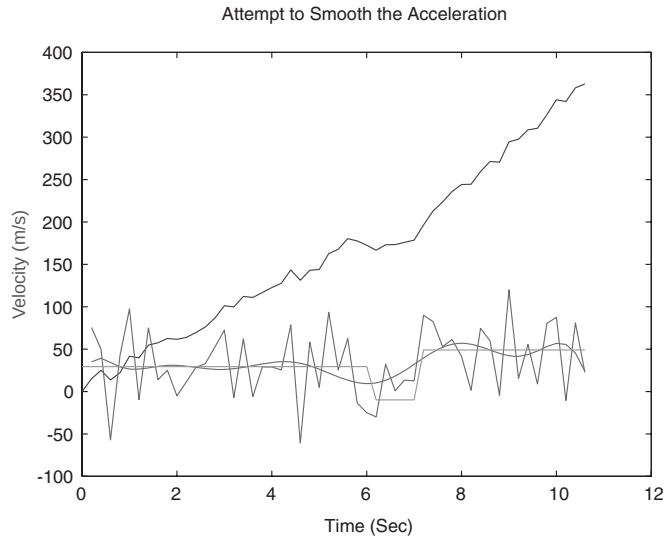


Figure 15.19 *Study conclusion*

This appears to have somewhat improved the situation, but when we take a careful look, guided by the plot that shows the actual acceleration, we see the truth about the actual acceleration used to compute the rocket velocity profile. Being a two-stage rocket, it consisted of six seconds of acceleration at $3g$, one second of falling at $-g$ while the first stage motor detached, and the remaining time at $5g$. As we can now see, while the curve fitting managed to smooth out the worst of the computed spikiness in the data and it was trying to track the discontinuity in the acceleration profile, it was also beginning to model the shape of the noise on the velocity rather than the underlying truth. Although we could reexamine the choice of order of the polynomial and try a less aggressive fit, we would find that the data would track the noise less carefully, and also not follow the central velocity oddity. We seem to be trapped between fitting to the artifact we want and smoothing out artifacts we do not want.

The Clue: The fact that we needed to go to a twelfth or higher order to fit the data should have been a strong clue that something else was going on. Ballistic flight is usually a second-order phenomenon—

perhaps third or fourth order if we allow for fuel to burn off and friction—but not twelfth order.

Moral: Curve fitting can sometimes help to clean up nasty looking data. However, there may be some clues in the source of the data (in our case, a sounding rocket that could be multistage) and the character of the data (in our case, the odd discontinuity between burns) that might lead to different and better interpretation of the data. Looking again at the velocity profile in the light of the actual data, it can be seen that the velocity profile is not as noisy as first appeared. Rather, it is two segments of linear acceleration with a discontinuity in the middle. We leave it to the reader to construct piecewise fits to the data to improve the acceleration estimates.

15.6 Engineering Example – Cleaning Up Images

In this example, we will see how interpolation in two dimensions can be used to improve the quality of images. The specific image defect we address is the pixilation caused by having insufficient pixels for the image. Advanced image processing techniques remove this effect by a mathematical process called antialiasing. We will see how MATLAB's interpolation tools produce a comparable effect.

We begin with a small piece of an old picture where there are too few pixels on the image to give a good rendering (Figure 15.20a) leaving a pixilated effect. We then define a grid of pixels five times finer than the original grid and use MATLAB's two dimensional interpolation function `interp2(...)` to interpolate the red, green, and blue values at the intermediate points. The second image (Figure 15.20b) shows the result of 2-D linear interpolation where the pixilation has almost completely disappeared. The third image (Figure 15.20c) shows the result of 2-D spline interpolation where the splined curves have sharpened the color contrast as well as performing the antialiasing function.

In Listing 15.12:

Lines 1-3: Read in the image and display it in the first subplot.

Line 4: Determines the size of the image.

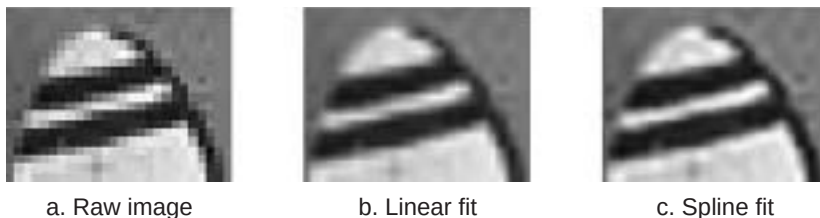


Figure 15.20 Three views of the tail of an aircraft

Line 5: Establishes the edge vectors for the original image.

Lines 6–7: Define the new edge vectors with 5 times increase in the number of points across the image.

Line 8: Repeats this code once for linear, and once for spline interpolation.

Lines 9–10: Treat each color layer separately.

Line 11: Determine whether this is linear or spline interpolation.

Lines 12–14: Do the linear interpolation and set the plot title.

Lines 15–19: Do the spline interpolation.

Line 20: Reassemble the new image one layer at a time.

Lines 22–23: Draw the second and third frames of the image.

Listing 15.12 Image Interpolation

```

1. img = imread('connie_tail.jpg');
2. subplot(1,3,1); imshow(img)
3. title('a. raw image')
4. [rows cols junk] = size(img);
5. oc = 1:cols; or = 1:rows;
6. nc = linspace(1, cols, 5*cols);
7. nr = linspace(1, rows, 5*rows);
8. for frame = 2:3
9.     for clr = 1:3
10.         layer = double(img(:,:,clr));
11.         if frame == 2
12.             newLayer = ...
13.                 interp2(oc, or, layer, nc', nr);
14.             txt = 'b. linear';
15.         else
16.             newLayer = ...
17.                 interp2(oc, or, layer, nc', nr, 'spline');
18.             txt = 'c. spline';
19.         end
20.         newImg(:,:,clr) = uint8(newLayer);
21.     end
22.     subplot(1,3,frame); imshow(newImg)
23.     title([txt, ' fit'])
    end

```



Chapter Summary

In this chapter, we saw the implementations of four common numerical techniques:

- We can estimate data points between given data values using linear (interp1/2/3) or spline interpolation
- We can smooth noisy data by fitting polynomial curves of suitable order to the raw data

- Given, for example, the velocity of an object over time, we can determine its position by integrating using `cumsum`
- We can differentiate to generate its acceleration



Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>NaN</code>	Not a number	15.1.1
<code>cumsum(v)</code>	Computes the integral of a function, assuming that x is 1	15.3.2
<code>diff(v)</code>	Computes the differences between adjacent values in a vector	15.4.2
<code>interp1(x, y, nx)</code>	Computes linear and cubic interpolation	15.1.1
<code>interp2(x, y, z, nx, ny)</code>	Computes linear and cubic interpolation	15.1.1
<code>interp3(x, y, z, v, nx, ny, nz)</code>	Computes linear and cubic interpolation	15.1.1
<code>polyfit(x, y, n)</code>	Computes a least-squares polynomial	15.2.2
<code>polyval(c, x)</code>	Evaluates a polynomial	15.2.2
<code>spline(x, y)</code>	Spline interpolation	15.1.2



Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. MATLAB functions permit extrapolation beyond the limits of the original independent variable.
2. The cubic spline is a series of parametric curves.
3. You cannot extrapolate the equations generated by curve fitting.
4. You should always match the order of a parametric curve fit to the underlying physics of the data.
5. Simpson’s rule is more accurate then the trapezoidal rule for integrating a function.
6. Numerical differentiation produces a vector that is the same length as the original vector.
7. You can only find the solutions to equations that are polynomial in structure.

Fill in the Blanks

1. _____ is the technique by which we estimate a variable's value between known values.
2. Nth-order polynomial regression determines the _____ of order n that minimize the _____ between the line and the data points.
3. The _____ makes the slope at x_k the _____ of the line between x_{k-1} and x_{k+1} .
4. To compute the continuous integral of a data set that is not regularly sampled, you have to compute the _____ of the _____ of _____ and _____.
5. You can find the _____ of a set of data (the points of zero slope) by first _____ the data and then _____ of the result.

**Programming Projects**

1. Do the following basic exercises with numerical methods.
 - a. Define two vectors x_i and y_i of the same length where the x_i values are monotonically increasing and the y_i values are somehow related to the x_i values. Then define a new vector x with closer spacing than x_i and extending below and above the range of x_i . Find the y values corresponding to the x -values in x_i by linear interpolation. On the same figure, plot the original y_i vs. x_i as red circles, and y vs. x as a black line. What do you observe about the visible range of the x values?
 - b. Repeat the exercise in part a using the `spline(...)` function to interpolate. Explain the difference in the range of the resulting y vs. x plot.
 - c. Use `polyfit(...)` to find the coefficients of the third order polynomial that best fits the points represented by vectors x_i

and y_i and then use `polyval(...)` to evaluate that curve at the x points. As before, plot y_i vs. x_i as red circles and y vs. x as a black line.

- d. Approximate the derivative, $dx_y = dy/dx$, for the vectors x_i and y_i using the `diff(...)` function and plot y_i vs. x_i . Since `diff(...)` reduces the length of the vector by one, you will have to plot dx_y vs. either $x_i(1:end-1)$, $x_i(2:end)$ or compute x_m , the mid-points of x_i .
 - e. Find y_p , the cumulative sum of the elements in dx_y , and add this to the plot of part d. With the exception of a constant offset, this curve ought to track the original plot of y_i vs. x_i .
 - f. Use `cumtrapz` to find the area under the curve represented by y_p vs. x_i with the trapezoidal method of approximation. Compare this result to the ending value of the y_p curve.
2. Write a function, `bestFit`, that takes in a vector of x -coordinates and a vector of y -coordinates. Your function should fit a polynomial curve to the data. The degree of the polynomial should be the smallest degree polynomial with an average error (the average of the absolute value of the difference between the new y -coordinates and the original y -coordinates) less than 2. Your function should return:
 - the vector of coefficients of your polynomial
 - the vector of new y -coordinates which is the polynomial evaluated at the original x -coordinates, and
 - the vector of the error magnitudes of your polynomial.
 Write a test program to provide reasonable data to your function and plot the original data (in blue), the curve-fitted data (in green), and the error (in red) on one figure. Title your plot and label your axes accordingly, including a legend.

For example:

```
[coeff newy err] =
    bestFit(0:5, [0 20 25 40 52 67]) should return:
coeff = [0.4722 -3.4881 18.9444 0.9048]
newy = [0.905 16.83 28.62 39.1 51.1 67.45]
err = [0.905 3.17 3.62 0.905 0.905 0.45]
```

3. You have been approached by the Rambling Wreck club to test the performance of the Rambling Wreck. You are provided with the test results of the car for 10 trial runs in the form of a vector a that contains the displacement of the car from the origin at that second. The first element is the displacement at the 0th second, the second element is the displacement at the 1st second and so on. Write a script called `testWreck` that displays a plot of the velocity of the

Hint

Use `diff()`.

Rambling Wreck over time during the test run. You could test your script using:

```
d=[0 20 35 50 60 55 30 25 15 5];
```

4. Engineers often use tabulated data for various calculations. An important method that any good engineer should be able to apply to tabulated data is Interpolation. In thermodynamics, the properties of a gas can be known when two of its properties have been fixed. You are required to come up with a continuous MATLAB function being given the tabulated data below measured where the pressure is 0.10 MPa:

Temperature (deg C)	Specific Volume (cu meters / Kg)
99.63	1.694
100	1.696
120	1.793
160	1.984
200	2.172
240	2.359
280	2.546
320	2.732
360	2.917
400	3.103
440	3.288
500	3.565

Write a function called `lookup` that consumes three parameters: the above table in an array `table`, a number `value` and a logical control value `getTemp`. If `getTemp` is `true`, the function interpolates the value as a specific volume and returns the corresponding temperature. Otherwise, it interpolates the value as a temperature and returns the corresponding specific volume.

Note: Your function IS NOT TO extrapolate the data (i.e., your function should return `NaN` if the user tries to obtain values outside the range of the table values).

5. Mathematically speaking, a critical point occurs when the derivative of a function equals zero. It is possible that a local minimum or a

local maximum occurs at a critical point. A local minimum is a point where the function's value to the left and right of it are larger, and a

Hint

Find the derivative to solve this problem. You know a local minimum or maximum occurs when the derivative goes from positive to negative or negative to positive.

local maximum is a point where the function's value to the left and right of it are smaller. You are going to write a function that finds the local minimum and maximum points. Call the function `find_points`. It should take in a vector of `x` values and a vector of `y` values. It should return two vectors. The first vector

should contain the `x` values where the minimum points occur while the second vector should contain the `x` values where the maximum points occur.

You may assume that the local minimum and maximum occur at integer `x` values. This will take care of any decimals created when using `diff`. For example,

```
If x=linspace(-8,2,1000) and y=x.^2+6*x+3;
    [min_p max_p]=find_points(x,y) should return:
        min_p = -3, max_p = []
If x=linspace(-5,5,1000) and y=x.^3-12*x;
    [min_p max_p]=find_points(x,y) should return:
        min_p = 2, max_p = -2
```

You could plot `x` and `y` to confirm the answers.

6. Now that we used the derivative it only makes sense that you are going to write a function that finds integrals. Call your function `find_integral`. Your function should take in a vector of `x` and `y` values as Problem 15.3 does and should plot the integral and also return the total area under the function. You are to use the trapezoidal rule to find the integrals.

For example:

```
If x=linspace(0,5,1000); and y=2*x+5;
    find_integral(x,y) should return 50.0000
```

7. When learning about sound you may have asked yourself how the digital wave file is converted to an actual sound. The sample points of the wave file need to be joined together to create a continuous function. To accomplish this, your computer (or CD player) uses a digital to analog converter. This chip will take the digital samples and create a continuous analog signal that will then get sent to your speakers. There are different methods to connect the points but one way you could do it is to use `spline` interpolation. This provides an advantage over linear interpolation since it will create a more sinusoidal looking wave.

You are going to recreate this conversion using MATLAB. Write a function called `fill_in` that takes in a vector of samples and returns a vector with 10 times the number of samples in the input. This means there will be 9 new points between the original samples. The values of the samples should be interpolated using `spline`.

You should test your function by also producing a plot of the original and interpolated data on the same figure. You could use:

```
fill_in([0.75,-0.2,-0.8,0.7,-0.4,0,-0.6,0.7]);
```

as a test case.

Sorting

Chapter Objectives

This chapter discusses:

- A technique for comparing the performance of algorithms
- A range of algorithms for sorting a collection of data
- Application areas in which these algorithms are most appropriate

First we will digress from the main thread of problem solving using MATLAB to discuss an “engineering algebra” for measuring the cost of an algorithm in terms of the amount of work done. Then we will consider a number of sorting algorithms, using this technique to assess their relative merits.

- 16.1** Measuring Algorithm Cost
 - 16.1.1 Specific Big O Examples
 - 16.1.2 Analyzing Complex Algorithms
- 16.2** Algorithms for Sorting Data
 - 16.2.1 Insertion Sort
 - 16.2.2 Bubble Sort
 - 16.2.3 Quick Sort
 - 16.2.4 Quick Sort in Place
 - 16.2.5 Merge Sort
 - 16.2.6 Bucket Sort
- 16.3** Performance Analysis
- 16.4** Applications of Sorting Algorithms
 - 16.4.1 Using MATLAB's Internal Sort Algorithm
 - 16.4.2 Insertion Sort
 - 16.4.3 Bubble Sort
 - 16.4.4 Quick Sort
 - 16.4.5 Merge Sort
 - 16.4.6 Bucket Sort
- 16.5** Engineering Example—A Selection of Countries



16.1 Measuring Algorithm Cost

How many times do you ask yourself, “Just how good is my algorithm?” Probably not very often, if ever. After all, we have been creating relatively simple programs that work on a small, finite set of data. Our functions execute and return an answer within a second or two. You may have noticed that some of the image processing scripts take a number of seconds to run.

However, as the problems become more complex and the volume of data increases, we need to consider whether we are solving the problem in the most efficient manner. In extreme cases, processes that manipulate huge amounts of data like the inventory of a large warehouse or a national telephone directory might be possible only with highly efficient algorithms.

Big O is an algebra that permits us to express how the amount of work done in solving a problem relates to the amount of data being processed. It is a gross simplification for software engineering analysis purposes, based on some sound but increasingly complex theory. (Interested readers should look up little-O, Big-Ω, little-ω, and Big-Θ.)

Big O is a means of estimating the worst case performance of a given algorithm when presented with a certain number of data items, usually referred to as N . In fact, the actual process attempts to determine the limit of the relationship between the work done by an algorithm and N as N approaches infinity.

We record the Big O of an algorithm as $O(<\text{expression in terms of } N>)$. For example, $O(1)$ describes the situation where the computing cost is independent of the size of the data, $O(N)$ describes the situation where the computing cost is proportional to the size of the data, and $O(2N)$ describes the situation where the computing cost doubles each time one more piece of data is added. At this point, we should also observe some simplifying assumptions:

- We are not concerned with constant multipliers on the Big O of an algorithm. As rapidly as processor performance and languages are improving, multiplicative improvements can be achieved merely by acquiring the latest hardware or software. Big O is a concept that reports qualitative algorithm improvement. Therefore, we choose to ignore constant multipliers on Big O analyses.
- We are concerned with the performance of algorithms as N approaches infinity. Consequently, when the Big O is expressed as the sum of multiple terms, we keep only the term with the highest power of N .

16.1.1 Specific Big O Examples

On the basis of algorithms we have already discussed, we will look at examples of the most common Big O cases.

O(1)—Independent of N O(1) describes the ideal case of an algorithm or logical step whose amount of work is independent of the amount of data. The most obvious example is accessing or modifying an entry in a vector. Since all good languages permit direct access to elements of a vector, the work of these simple operations is independent of the size of the vector.

O(N)—Linear with N O(N) describes an algorithm whose performance is linearly related to N. Copying a vector of size N is an obvious example, as is searching for a specific piece of data in such a vector. One might argue that occasionally one would find the data as the first element. There is an equal chance that we would be unlucky and find the item as the last element. On average, we would claim that the performance of this search is the mean of these numbers: $(N + 1) / 2$. However, applying the simplification rules above, we first reject the 1 as being N to a lower power, leaving $N/2$, and then reject the constant multiplier, leaving O(N) for a linear search.

O(logN)—Binary Search Consider searching for a number—say, 86—in a sorted vector such as that shown in Figure 16.1. One could use a linear search without taking advantage of the ordering of the data. However, a better algorithm might be as follows:

1. Go to the middle of the vector (approximately) and compare that element (59) to the number being sought.
2. If this is the desired value, exit with the answer.
3. If the number sought is less than that element, then, since the data are ordered, we can reject the half of the array to the right of, and including the 59.
4. Similarly, if the number sought is greater than that element, we can reject the half of the array to the left of and including the 59.
5. Repeat these steps with the remaining half vector until either the number is found or the size of the remaining half is zero.

Now consider how much data can be covered by each test—a measure of the work done as shown in Table 16.1.

In general, we can state that the relationship is expressed as follows:

$$N = 2^{W-1}$$

7	12	42	59	71	86	104	212
---	----	----	----	----	----	-----	-----

Figure 16.1 Binary search

Table 16.1 Work done in a binary search

Comparisons	Coverage, N
1	1
2	2
3	4
4	8
5	16
⋮	⋮
⋮	⋮
W	2^{W-1}

However, we need the expression for the work, W , as a function of N . Therefore, we take the log base 2 of each side so that:

$$W = \log_2 N + 1$$

Now we simplify by removing the 1, and then realizing that we can convert $\log_2 N$ to $\log_x N$ merely by multiplying by $\log_2 x$. Consequently, since we reject constant multipliers as irrelevant, we lose interest in representing the specific base of the logarithm, leaving the work for a binary search as $O(\log N)$.

$O(N^2)$ —Proportional to N^2 $O(N^2)$ describes an algorithm whose performance is proportional to the square of N . It is a special case of $O(N \times M)$, which describes any operation on an $N \times M$ array or image.

$O(2^N)$ —Exponential Growth or Worse Occasionally we run across very nasty implementations of simple algorithms. For example, consider the recursive implementation of the Fibonacci algorithm we discussed in Section 9.9.2. In this implementation, $\text{fib}(N) = \text{fib}(N - 1) + \text{fib}(N - 2)$. So each time we add another term, the previous two terms have to be calculated again, thereby doubling the amount of work. If we double the work when 1 is added to N , in general the Big O is $O(2^N)$. Of course, in the case of this particular algorithm, there is a simple iterative solution with a much preferable performance of $O(N)$.

16.1.2 Analyzing Complex Algorithms

We can easily calculate the Big O of simple algorithms. For more complex algorithms, we determine the Big O by breaking the complex algorithm into simpler abstractions, as we saw in Chapter 10. We would continue that process until the abstractions can be characterized as simple operations on defined collections for which we can determine their Big Os. The Big O of the

overall algorithm is then determined from the individual components by combining them according to the following rules:

- If two components are sequential (do A and then do B), you add their Big O values
- If components are nested (for each A, do B), you multiply their Big O values

For example, we will see the merge sort algorithm in Section 16.2.5. It can be abstracted as follows:

Perform a binary division of the data ($O(\log N)$) and *then for each* binary step (of which there are $O(\log N)$), merge all the data items ($O(N)$).

This has the general form:

Do A, then for each B, do C

which, according to the rules above, should result in $O_A + O_B * O_C$. The overall algorithm therefore costs $O(\log N) + O(N) * O(\log N)$.

We remove the first term because its growth is slower, leaving $O(N \log N)$ as the overall algorithm cost.



16.2 Algorithms for Sorting Data

Generally, sorting a collection of data will organize the data items in such a way that it is possible to search for a specific item using a binary search rather than a linear search. This concept is nice in principle when dealing with simple collections like an array of numbers. However, it is more difficult in practice with real data. For example, telephone books are always sorted by the person's last name. This facilitates searching by last name, but it does not help if you are looking for the number of a neighbor whose name you do not know. That search would require sorting the data by street name.

There are many methods for sorting data. We present five representative samples selected from many sorting algorithms. First we describe each algorithm, and then we compare their performance and suggest engineering circumstances in which you would apply each algorithm. Note that in all these algorithms the comparisons are done using functions (for example, `gt(...)`, `lt(...)`, or `equals(...)`) rather than mathematical operators. This permits collections containing arbitrarily complex objects to be sorted merely by customizing the comparison functions.

16.2.1 Insertion Sort

Insertion sort is perhaps the most obvious sorting technique. Given the original collection of objects to sort, it begins by initializing an empty

collection. For example, if the collection were a vector, you might allocate a new vector of the same size and initialize an “output index” to the start of that vector. Then the algorithm traverses the original vector, inserting each object from that vector in order into the output vector. This usually requires “shuffling” the objects in the new vector to make room for the new object. Figure 16.2 illustrates the situation where the first four numbers of the original vector have been inserted into the new vector; the algorithm finds the place to insert the next number (10) and then moves the 12 across to make space for it.

Listing 16.1 shows the MATLAB code for insertion sort on a vector of numbers. The algorithm works for any data type for which the function `lt(A,B)` compares two instances.

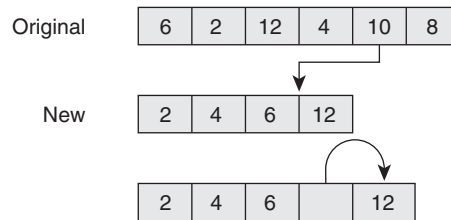


Figure 16.2 *Insertion sort in progress*

Listing 16.1 The insertion sort function

```

1. function b = insertionsort(a)
2. % This function sorts a column vector,
3. % using the insertion sort algorithm
4. b = []; i = 1; sz = length(a);
5. while i <= sz
6.     b = insert(b, a(i,1) );
7.     i = i + 1;
8. end

9. function a = insert(a, v)
10. % insert the value v into column vector a
11. i = 1; sz = length(a); done = false;
12. while i <= sz
13.     if lt(v, a(i,1))
14.         done = true;
15.         a = [a(1:i-1); v; a(i:end)];
16. break;
17. end
18.     i = i + 1;
19. end
20. if ~done
21.     a(sz+1, 1) = v;
22. end

```

In Listing 16.1:

Line 4: Initializes the result and the `while` loop parameters.

Line 6: Calls the helper function to insert the latest value into the output vector.

Line 9: Shows the helper function that does the hard work.

Line 11: Initializes the `while` loop

Line 13: The general purpose test to determine whether the inserted variable belongs ahead of the `i`th element.

Line 14: We need the `done` flag set on exit to guard against not finding a place for `v`.

Line 15: Insert `v` into the array.

Line 16: Exit from the `while` loop—nothing left to do.

Line 18: Updates the loop index.

Lines 20–22: If there is no place to put `v` (it is bigger than all the elements), put it at the end.

Later we will refer to the selection sort algorithm that is similar in concept to insertion sort. Rather than sorting as the new data is put into the new vector, however, the selection sort algorithm repeatedly finds and deletes the smallest item in the original vector and puts it directly into the new vector. Both insertion sort and selection sort are $O(N^2)$ if used to sort a whole vector.

16.2.2 Bubble Sort

Where insertion sort is easy to visualize, it is normally implemented by creating a new collection and growing that new collection as the algorithm proceeds. Bubble sort is conceptually the easiest sorting technique to visualize, and is usually accomplished by rearranging the items in a collection in place. Given the original collection of N objects to sort, it makes $(N - 1)$ major passes through the data. The first major pass examines all N objects in a minor pass, and subsequent passes reduce the number of examinations by 1. On each minor pass through the data, beginning with the first data item and moving incrementally through the data, the algorithm checks to see whether the next item is smaller than the current one. If so, the two items are swapped in place in the array.

At the end of the first major pass, the largest item in the collection has been moved to the end of the collection. After each subsequent major pass, the largest remaining item is found at the end of the remaining items. The process repeats until on the last major pass, the first two items are compared and swapped if necessary. Figure 16.3 illustrates a bubble sort of a short vector. On the first pass, the value 98 is moved completely across the vector to the rightmost position. On the next pass, the 45 is moved into position.

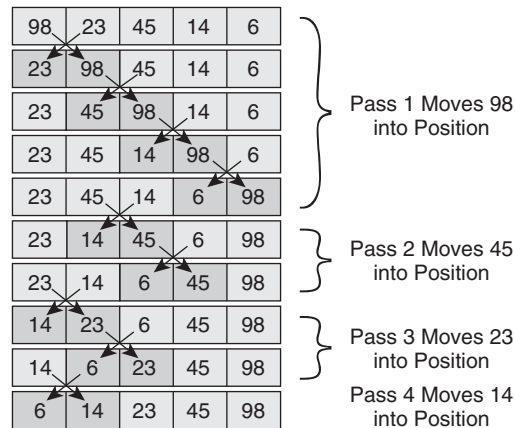


Figure 16.3 Bubble sort

On the third pass, the 23 reaches the right position, and the last pass finishes the sort.

Listing 16.2 shows the MATLAB code for bubble sort on a vector of numbers. The algorithm works for any data type for which the function `gt(A,B)` compares two instances. Since bubble sort performs $(N - 1) * N/2$ comparisons on the data, it is also $O(N^2)$. Some implementations use a flag to determine whether any swaps occurred on the last major pass, and terminate the algorithm if none occurred. However, the efficiency gained by stopping the algorithm early has to be weighed against the cost of setting and testing a flag whenever a swap is accomplished.

Listing 16.2 Bubble sort

```

1. function bubblesort()
2. %
3. % This function sorts the column array b in place,
4. % using the bubble sort algorithm
5. %
6. global b
7. N = length(b);
8. right = N-1;
9. for in = 1:(N-1)
10.   for jn = 1:right
11.     if gt(b(jn), b(jn+1))
12.       tmp = b(jn); % swap b(jn) with b(jn+1)
13.       b(jn) = b(jn+1);
14.       b(jn+1) = tmp;
15.     end
16.   end
17.   right = right - 1;
18. end

```

In Listing 16.2:

Line 6: In order to be able to access the array in place, we pass it as a global variable instead of as a parameter.

Lines 7 and 8: Since each pass puts the largest element in place, we can reduce the item count by 1 each time. This initializes the size of the first pass.

Line 9: Shows the loop for the $N - 1$ major passes.

Line 10: Shows the loop for each major pass.

Line 11: Checks the current item against its neighbor.

Lines 12–14: Swap the current item with its neighbor. By doing this in place, the largest item is always considered the current item until it reaches the end.

Line 17: Shortens the row after each major pass because the largest item in the last pass is placed at the right-hand end.

16.2.3 Quick Sort

As its name suggests, the quick sort algorithm is one of the fastest sorting algorithms. It is designed to sort an array “in place.” The quick sort algorithm is recursive, and uses an elegant approach to subdividing the original vector. Figure 16.4 illustrates this process. The algorithm proceeds as follows:

- The terminating condition occurs when the vector is of length 1, which is obviously sorted.
- A “pivot point” is then chosen. Some sophisticated versions go to a significant amount of effort to calculate the most effective pivot point. We are content to choose the first item in the vector.
- The vector is then subdivided by moving all of the items less than the pivot to its left, and all those greater than the pivot to its right, thereby placing the pivot in its final location in the resulting vector.

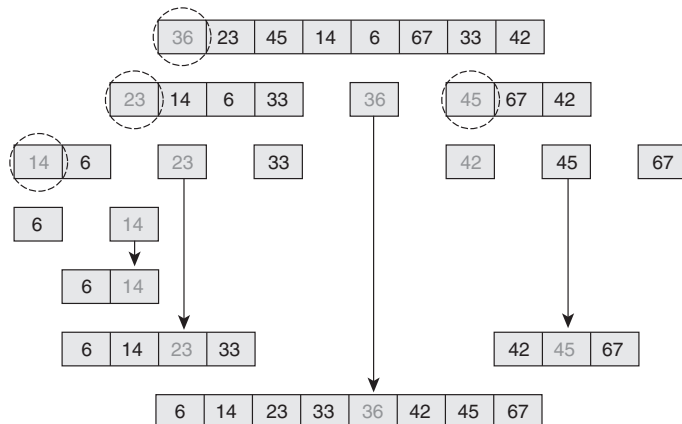


Figure 16.4 Quick sort

- The items to the left and right of the pivot are then recursively sorted by the same algorithm.
- The algorithm always converges because these two halves are always shorter than the original vector.

Listing 16.3 shows the MATLAB code for the quick sort algorithm. The partitioning algorithm looks a little messy, but it is just performing the array adjustments. It starts with *i* and *j* outside the vector to the left and right. Then it keeps moving each toward the middle as long as the values at *i* and *j* are on the proper side of the pivot. When this process stops, *i* and *j* are the indices of data items that are out of order. They are swapped, and the process is repeated until *i* passes *j*. Quick sort is $O(N\log N)$. As with the previous techniques, this algorithm applies to collections of any data type for which the functions `lt(A,B)` and `gt(A,B)` compare two instances.

Listing 16.3 Quick sort

```

1. function a = quicksort(a, from, to)
2. % This function sorts a column array,
3. % using the quick sort algorithm
4.   if (from < to)
5.     [a p] = partition(a, from, to);
6.     a = quicksort(a, from, p);
7.     a = quicksort(a, p + 1, to);
8.   end

9. function [a lower] = partition(a, from, to)
10. % This function partitions a column array
11.   pivot = a(from); i = from - 1; j = to + 1;
12.   while (i
13.     i = i + 1;
14.     while lt(a(i), pivot)
15.       i = i + 1;
16.     end
17.     j = j - 1;
18.     while gt(a(j), pivot)
19.       j = j - 1;
20.     end
21.     if (i < j)
22.       temp = a(i); % this section swaps
23.       a(i) = a(j); % a(i) with a(j)
24.       a(j) = temp;
25.     end
26. end
27. lower = j;

```

In Listing 16.3:

Line 1: Each recursive call is provided with the vector to sort and the range of indices to sort. These are initially `from = 1` and `to = length(a)`.

Line 4: The terminating condition for the recursion is when the subvector to sort has size 1—that is, when `from == to`.

Line 5: The helper function performs three roles—it places the pivot in the right place, moves the smaller and larger values to the correct sides, and returns the location of the pivot to permit the recursive partitioning.

Lines 6 and 7: Show recursive calls to sort the left and right parts of the vector.

Line 9: Shows the helper function.

Line 11: Initializes the variables.

Line 12: The outer loop continues until `i` passes `j`.

Lines 13–16: Skip `i` forwards over all the items less than the pivot.

Lines 17–20: Skip `j` backwards over all the elements greater than the pivot.

Line 21: If `i` passed `j`, the partitioning is complete.

Lines 22–24: Otherwise, `i` is indexing an item greater than the pivot, and `j` is indexing one less than the pivot. By swapping the contents of `a(i)` and `a(j)`, we rectify both inequities and can continue the inner loop.

Line 27: When the loop exits, both `i` and `j` are indexing the pivot.

There is one performance caution about quick sort. Its speed depends on the randomness of the data. If the data are mostly sorted, its performance reduces to $O(N^2)$.

16.2.4 Quick Sort in Place

A careful study of the performance of the quick sort algorithm indicates that while it performs as $O(N \log N)$ for small collections, this quick sort implementation starts to look like $O(N^2)$ when N gets large. This is because MATLAB requires the arrays to be copied to return them from a function call. An alternative implementation using a global variable `b` to contain the array being sorted is shown in Listing 16.4. The majority of the comments above apply also to this version of the algorithm.

Listing 16.4 Quick sort in place

```
1. function quicksortip(from, to)
2. %
3. % This function sorts a column array,
4. % using the quick sort algorithm in place
5. %
6. if (from
7.     p = partition(from, to);
8.     quicksortip(from, p);
9.     quicksortip(p + 1, to);
10. end
```

continued on next page

```

11. function lower = partition(from, to)
12. global b
13. pivot = b(from); i = from - 1; j = to + 1;
14. while (i < j)
15.     i = i + 1;
16.     while lt(b(i), pivot)
17.         i = i + 1;
18.     end
19.     j = j - 1;
20.     while gt(b(j), pivot)
21.         j = j - 1;
22.     end
23.     if (i
24.         temp = b(i); % this section swaps
25.         b(i) = b(j); % b(i) with b(j)
26.         b(j) = temp;
27.     end
28. end
29. lower = j;

```

In Listing 16.4:

Lines 1–9: The array is neither passed in nor returned.

Line 12: Global access to the array.

16.2.5 Merge Sort

Merge sort is another $O(N \log N)$ algorithm that achieves speed by dividing the original vector into two “equal” halves. Equality, of course, is not possible when there is an odd number of objects to be sorted, in which case the length of the “halves” will differ by at most 1. The heart of the merge sort algorithm is the technique used to reunite two smaller sorted vectors. This function is called “merge.” Its objective is to merge two vectors that have been previously sorted. Since the two vectors are sorted, the smallest object can only be at the front of one of these two vectors. The smallest item is removed from its place and added to the result vector. This merge process continues until one of the two halves is empty, in which case the remaining half (whose values all exceed those in the result vector) is copied into the result.

The merge sort algorithm, like quick sort, is recursive, as shown in Figure 16.5 and as follows:

- The terminating condition is a vector with length less than 2, which is already sorted
- The recursive part invokes the merge function on the recursive call to merge the two halves of the vector
- The process converges because the halves are always smaller than the original vector

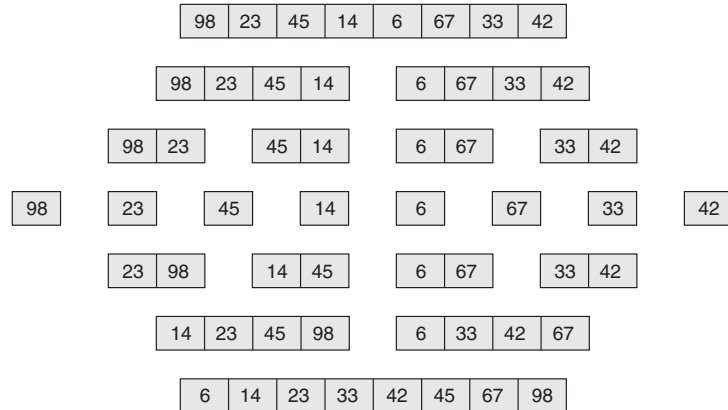


Figure 16.5 Merge sort

The code for merge sort is shown in Listing 16.5.

Listing 16.5 Merge sort

```

1. function b = mergesort(a)
2. % This function sorts a column array,
3. % using the merge sort algorithm
4. b = a; sz = length(a);
5. if sz > 1
6.     szb2 = floor(sz / 2);
7.     first = mergesort(a(1 : szb2));
8.     second = mergesort(a(szb2+1 : sz));
9.     b = merge(first, second);
10. end

11. function b = merge(first, second)
12. % Merges two sorted arrays
13. i1 = 1; i2 = 1; out = 1;
14. % as long as neither i1 nor i2 past the end,
15. % move the smaller element into a
16. while (i1 <= length(first)) & (i2 <= length(second))
17.     if lt(first(i1), second(i2))
18.         b(out,1) = first(i1); i1 = i1 + 1;
19.     else
20.         b(out,1) = second(i2); i2 = i2 + 1;
21.     end
22.     out = out + 1;
23. end
24. % copy any remaining entries of the first array
25. while i1 <= length(first)
26.     b(out,1) = first(i1); i1 = i1 + 1; out = out + 1;
27. end
28. % copy any remaining entries of the second array
29. while i2 <= length(second)
30.     b(out,1) = second(i2); i2 = i2 + 1; out = out + 1;
31. end

```

In Listing 16.5:

Line 4: Initializes the parameters.

Line 5: The terminating condition is an array of length 1, which does not need sorting.

Line 6: Divides the array in half.

Lines 7 and 8: Sort the halves of the array.

Line 9: Merges the two sorted halves.

Line 11: Shows the helper function to merge sorted arrays.

Line 13: Initializes the `while` loop.

Line 16: Stays in this loop until one of the two arrays is used up.

Lines 17–20: Choose the smaller item to add to the result.

Lines 25 and 26: Copy the remains of the first array.

Lines 29 and 30: Copy the remains of the second array.

As discussed for quick sort, this algorithm would greatly benefit from the use of a global variable to contain the collection being sorted. We leave that improvement as an exercise.

16.2.6 Bucket Sort

A discussion of sorting techniques would not be complete without discussing bucket sort. This is also an $O(N \log N)$ algorithm that is frequently used for sorting physical piles of papers, such as students' examination papers. The process begins with a stack of unsorted papers, each with an identifier consisting of a number or a unique name. One pass is made through the stack separating the papers into piles based on the first digit or character of the identifier. Subsequent passes sort each of these piles by subsequent characters or digits until all the piles have a small number of papers that can be sorted by insertion or selection sorts. The piles can then be reassembled in order. Figure 16.6 illustrates the situation at the end of the second sorting pass when piles for the first digit have also been separated by the second digit.

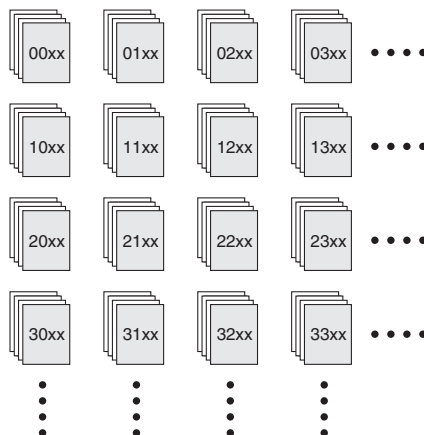


Figure 16.6 Bucket sort

There are a number of reasons why this technique is popular for sorting papers, including the following:

- There is a minimal amount of “paper shuffling” or bookkeeping
- The logarithm in the $O(N\log N)$ is either 10 (numerical identifier) or 26 (alphabetic identifier), thereby providing a “constant multiplier” speed advantage
- Once the first sorting pass is complete, one can use multi-processing (in the form of extra people) to perform the remaining passes in parallel, thereby reducing the effective performance to $O(N)$ (given sufficient parallel resources)



16.3 Performance Analysis

In order to perform a comparison of the performance of different algorithms, a script was written to perform each sort on a vector of increasing length containing random numbers. The script started with a length of 4 and continued doubling the length until it reached 32,768. To obtain precise timing measurements, each sort technique was repeated a sufficient number of times to obtain moderately accurate timing measurements with MATLAB's internal millisecond clock. In order to eliminate common computation costs, it was necessary to measure the overhead cost of the loops themselves and subtract that time from the times of each sort algorithm. Note that in order to show the results of the MATLAB internal sort, its execution time was multiplied by 1,000.

Figure 16.7 shows a typical plot of the results of this analysis, illustrating the relative power of $O(N\log N)$ algorithms versus $O(N^2)$ algorithms. The plot on a log-log scale shows the relative time taken by the selection sort,

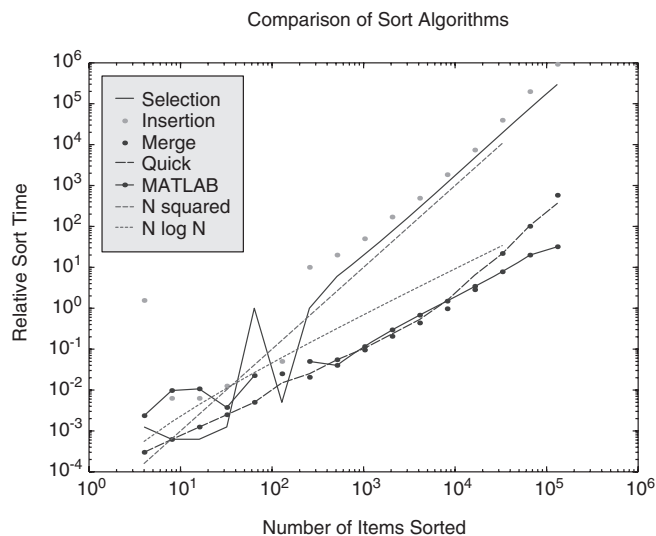


Figure 16.7 Sort study results

insertion sort, bubble sort, merge sort, quick sort, and quick sort in place algorithms, together with MATLAB's internal sort method. Also on the chart are plotted trend lines for $O(N^2)$ and $O(N\log N)$ processes. We can make the following observations from this chart:

- Since the scales are each logarithmic, it is tempting to claim that there is “not much difference” between $O(N^2)$ and $O(N\log N)$ algorithms. Looking closer, however, it is clear that for 100,000 items, the $O(N^2)$ sorts are around 1,000 times slower than the $O(N\log N)$ algorithms.
- The performance of most of the algorithms is extremely erratic below 100 items. If you are sorting small amounts of data, the algorithm does not matter.
- The selection sort, bubble sort, and insertion sort algorithms clearly demonstrate $O(N^2)$ behavior.
- The performance of the merge sort and quick sort algorithms is less clear. They appear to have $O(N\log N)$ performance when N is in the modest size range, but are tending toward $O(N^2)$ as N gets above 50,000. This is an artifact of the MATLAB implementation of functions that does not permit a function direct access to an array passed as a parameter, as discussed above. Every recursive call is therefore penalized by having to copy the result array back to the calling function. The quick sort in place algorithm has a well-defined $O(N\log N)$ performance.
- Clearly, the internal MATLAB sort function, in addition to being 1,000 times faster than any of the coded algorithms, is closely tracking the $O(N\log N)$ performance curve, indicating that it is programmed efficiently to take advantage of direct access to the array data.



16.4 Applications of Sorting Algorithms

This section discusses the circumstances under which you might choose to use one or another of the sorting algorithms presented above. We assert here without proof that the theoretical lower bound of sorting is $O(N\log N)$. Consequently, we should not be looking for a generalized sorting algorithm that improves on this performance. However, within those constraints, there are circumstances under which each of the sorting techniques performs best. As we saw in the analysis above, the MATLAB internal sort routine is blindingly fast, and should be used whenever possible. The subsequent paragraphs show the applicability and limitations of the other sort algorithms if you cannot use `sort(...)`.

16.4.1 Using MATLAB's Internal Sort Algorithm

The first and most obvious question is why one would not always use the built-in MATLAB `sort(...)` function. Clearly, whenever that function works, you should use it. Its applicability seems to be limited to sorting numbers in an array, and you will come across circumstances when you need to sort more complex items. You might, for example, have a structure array of addresses and telephone numbers that you wish to sort by last name, first name, or telephone number. In this case, it seems that the internal sort program does not help, and you have to create your own sort.

Extracting and Sorting Vectors and Cell Arrays However, a closer examination of the specification of the sort function allows us to generalize the application of `sort(...)` significantly. When you call `sort(v)`, it actually offers you a second result that contains the indices used to sort `v`. So in the case where you have a cell array or a structure array and your sort criteria can be extracted into a vector, you can sort that vector, and use the second result, the indexing order, to sort the original array. Furthermore, if you can extract character string data into a cell array, the internal sort function will sort that cell array alphabetically.

For example, consider again the CD collection from Chapter 10. We might want to find the most expensive CD in our collection, and then make a list of artists and titles ordered alphabetically by artist. The script to accomplish this is shown in Listing 16.6.

In Listing 16.6:

- Line 1: Uses the previous script to build the CD collection.
- Line 2: Extracts the prices into a vector.
- Line 3: Sorts the prices (in ascending order).
- Line 4: The most expensive CD will be the last in ascending order.
- Line 5: Extracts the artists into a cell array.
- Line 6: Sorts the cell array in alphabetical order.
- Lines 7–10: Traverse the ordered list, printing the artist and title.

Listing 16.6 Sorting CDs

```
1. buildCDs
2. prices = [cds.price];
3. [np order] = sort(prices);
4. mostExpensive = cds(order(end))

5. artists = {cds.artist};
6. [junk order] = sort(artists);
7. for index = order
8.     cd = cds(index);
9.     fprintf('%30s:%s\n', cd.artist, cd.title)
10. end
```

The resulting output is shown in Table 16.2.

If the sort criteria are more complex, you will have to customize the comparison functions and use one of the following sort algorithms.

Table 16.2 CD display

```

mostExpensive =
  genre: 'Heavy Metal'
  artist: 'Bon Jovi'
  title: '100,000,000 Bon Jovi Fans Can't Be Wrong'
  year: 2004
  stars: 3.8000
  price: 47.1500

2Pac:Loyal To The Game
2Pac:Loyal To The Game
2Pac:Loyal To The Game
Alison Krauss And Union Station:Lonely Runs Both Ways
Alison Krauss And Union Station:Lonely Runs Both Ways
Anthrax:Greater Of Two Evils
Ashanti:Concrete Rose
Bocelli, Andrea:Andrea
Bon Jovi:100,000,000 Bon Jovi Fans Can't Be Wrong
Charles, Ray:Genius Loves Company
Charles, Ray:Spirit Of Christmas
Charles, Ray:Very Best Of Ray Charles
Clapton, Eric:Sessions For Robert J
Cure:Three Imaginary...
Dream Theater:Live At Budokan
El Gran Combo:Aqui Estamos y De Verdad
Eminem:Encore
Eminem:Encore
Fey:La Fuerza Del Destino
Groban, Josh:Closer
Ivy Queen:Real
Killers:Hot Fuss
Ludacris:The Red Light District
Ludacris:The Red Light District
Nas:Street's Disciple
Nas:Street's Disciple
Phantom Of The Opera:The Phantom Of The Opera: Deluxe
Edition
Rammstein:Reise, Reise
Rogers, Kenny:Gift
T.I.:Urban Legend
T.I.:Urban Legend
Trans-Siberian Orchestra:Beethoven's Last Night
Trevi, Gloria:Como Nace El Universo
Twain, Shania:Greatest Hits
U2:How To Dismantle An Atomic Bomb
Yankee, Daddy:Barrio Fino

```

16.4.2 Insertion Sort

Insertion sort is the fastest means of performing incremental sorting. If a small number of new items—say, M —are being added to a sorted collection of size N , the process will be $O(M * N)$, which will be fastest as long as $M < \log N$.

16.4.3 Bubble Sort

Bubble sort is the simplest in-place sort to program, and is fine for small amounts of data. The major advantage of bubble sort is that in a fine-grained multi-processor environment, if you have $N/2$ processors available with access to the original data, you can reduce the Big O to $O(N)$.

16.4.4 Quick Sort

As its name suggests, this is the quickest of the sorting algorithms, and should normally be used for a full sort. However, it has one significant disadvantage: its performance depends on a fairly high level of randomness in the distribution of the data in the original array. Consequently, if there is a significant probability that your original data might be already sorted, or partially sorted, your quick sort is not going to be quick. You should use merge sort.

16.4.5 Merge Sort

Since its algorithm does not depend on any specific characteristics of the data, merge sort will always turn in a solid $O(N \log N)$ performance. You should use it whenever you suspect that quick sort might get in trouble.

16.4.6 Bucket Sort

It is theoretically possible to write the bucket sort algorithm in MATLAB to attempt to take advantage of its apparent performance improvements over the more conventional algorithms shown previously. However, some practical problems arise:

- In practice, the manipulation of the arrays of arrays necessary to sort by this technique is quite complex
- The performance gained for manual sorts by “parallel processing” stacks using multiple people cannot be realized
- The logic for extracting the character or digit for sorting is going to detract from the overall performance

Therefore we recommend that the use of bucket sort be confined to manually sorting large numbers of physical objects.

16.5 Engineering Example—A Selection of Countries

In the Engineering Application problem in Section 10.5, we attempted to find the best country for a business expansion based on the rate of growth of the GNP for that country versus its population growth. The initial version of the program returned the suggestion that the company should move to Equatorial Guinea. However, when this was presented to the Board of Directors, it was turned down, and you were asked to bring them a list of the best 20 places to give them a good range of selection.

We should make two changes to the algorithm:

- Originally we used a crude approximation to determine the slope of the population and GNP curves. However, now we know that `polyfit` can perform this slope computation accurately, and we will substitute that computation.
- We will use the MATLAB sort function to find the 20 best countries.

The code to accomplish this, a major revision of the code in Chapter 10, is shown in Listing 16.7.

Listing 16.7 Updated world data analysis

```

1. worldData = buildData('World_data.xls');
2. n = 20;
3. bestn = findBestn(worldData, n);
4. fprintf('best %d countries are:\n', n)
5. for best = bestn(end:-1:1)
6.     fprintf('%s\n', worldData(best).name)
7. end

8. function bestn = findBestn(worldData, n)
    % find the indices of the n best countries
    % according to the criterion in the function
    % fold
    % we first map world data to add the field
    %   growth
9. for ndx = 1:length(worldData)
10.    centry = worldData(ndx);
11.    worldData(ndx).growth = fold(centry);
12. end
    % now, sort on this criterion
13. values = [worldData.growth];
14. [junk order] = sort(values);
    % filter these to keep the best 10
15. bestn = order(end-n+1:end);

16. function ans = fold(st)
    % s1 is the rate of growth of population
17. pop = st.pop(~isnan(st.pop));
18. yr = st.year(~isnan(st.pop));
19. s1 = slope(yr, pop)/mean(pop);
    % s2 is the rate of growth of the GDP

```

continued on next page

```

20. gdp = st.gdp(~isnan(st.gdp));
21. yr = st.year(~isnan(st.gdp));
22. s2 = slope(yr, gdp)/mean(gdp);
    % Measure of merit is how much faster
    % the gdp grows than the population
23. ans = s2 - s1;

24. function s1 = slope(x, y)
    % Estimate the slope of a curve
25. if length(x) == 0 || x(end) == x(1)
26.     error('bad data')
27. else
28.     coef = polyfit(x, y, 1);
29.     s1 = coef(1);
30. end

```

In Listing 16.7:

Line 1: Reads the spreadsheet of world data.

Line 2: Selects the number of countries to present.

Line 3: Calls the function that will return the indices of the *n* best countries.

Lines 4–7: Print the list of country names in reverse order (the best first).

Line 8: Shows an updated version of the original function to return the indices of the *n* best countries.

Lines 9–12: Map the `worldData` structure array, adding to each a field called `growth` that contains the criterion specified in the `find` function.

Line 13: Extracts the values of `growth` for each country.

Line 14: Sorts these, keeping the order.

Line 15: Returns the last *n* countries that will have the highest `growth` values.

Lines 16–23: The `find` function unchanged from Chapter 10.

Lines 24–27: Unchanged beginning to the `slope` function.

Lines 28 and 29: Use `polyfit` to compute an accurate slope and return it to the calling function.

Common Pitfalls

A deceptively simple question arises: Should you expect the `worldData` at line 6 of Listing 16.7 to contain the field `growth`? Actually, it will not. Although it appears that the function `findBestn` adds this field to `worldData`, it is working with a copy of the `worldData` structure array that is not returned to the calling script.

The results from running this version are shown in Table 16.3. This seems to be an acceptable list of possibilities to take back to the Board of Directors.

Table 16.3 Updated world data results

Best 20 countries are:

Estonia	Lebanon
St. Kitts & Nevis	Malta
Albania	Cyprus
Vietnam	Tajikistan
Croatia	Taiwan
Kazakhstan	Korea, Republic of
Azerbaijan	Grenada
Uzbekistan	Ireland
Georgia	Portugal
Dominica	Antigua

 Chapter Summary

This chapter discussed:

- A technique for comparing the performance of algorithms
- A range of algorithms for sorting a collection of data
- Application areas in which these algorithms are most appropriate

 Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>polyfit(x, y, n)</code>	Computes a least-squares polynomial	16.5
<code>[v,in] = sort(v)</code>	Sorts the vector <code>v</code> (a vector or a cell array of strings)	16.4.1

 Self Test

Use the following questions to check your understanding of the material in this chapter:

True or False

1. When computing the Big O of sequential operations, you retain only the one that grows fastest with N.

2. All search algorithms have $O(N)$.
3. No sort algorithm can perform better than $O(N \log N)$.
4. All sorting algorithms with $O(N^2)$ traverse the complete data collection N times.
5. Quick sort in reality should be listed as $O(N^2)$.

Fill in the Blanks

1. _____ is an algebra that permits us to express how the amount of _____ done in solving a problem relates to the amount of _____ being processed.
2. Any algorithm that traverses, maps, folds, or filters a collection is $O(\text{_____})$.
3. _____ sort and _____ sort perform with $O(N \log N)$.
4. MATLAB's internal `sort(...)` returns the _____ and a _____ that allow you to sort any collection from whose elements one can derive a _____.



Programming Projects

1. In this series of exercises, you will write a script that performs some measurements that justify the need to sort a vector. Background:
 - Since this need is only apparent when processing large amounts of data, we will use a vector of at least 100,000 elements.
 - MATLAB has two built-in functions that collectively provide stop-watch capability: `tic` and `toc`. The function `tic` starts the stop-watch, and `toc` returns the number of seconds since the last `tic`.
 - However, since the time difference is measured to the resolution of milliseconds, we will have to repeat the experiments a number of times to achieve accurate measurements.
 - Since this involves a `for` loop that itself will cost time, we must also measure the overhead of the `for` loop and subtract that from the computational cost of the operation we are timing.

- a. Begin your script by deciding on the vector size, `N`, and the number of repetitions, `reps`, and making a vector, `vec`, of random integers between 0 and `N`.
- b. Write an empty function, `junk`, that consumes a vector of size `N` and a number and returns a number.
- c. Use `tic` and `toc` to compute the time for a `for` loop that does nothing but pass `vec` and `round(N*rand(1,1))` to `junk`. Adjust the value of `reps` until this loop takes about a second to run. Store this cost in the variable `overhead`.
- d. Write a function, `linearSearch`, that consumes a vector and a number and performs a linear search on the vector looking for that number. It should return either the position of that number in the vector or an empty vector if the number is not present.
- e. Add a loop to your script like the loop in step c, except calling `linearSearch`. Subtract `overhead` from the computed time and save it as `linearSearchTime`.
- f. Now sort the vector `vec` using the MATLAB `sort(...)` function and store it in a new vector, `sortedVec`.
- g. Write the function `binarySearch` that consumes a vector and a number and uses the algorithm in section 16.1.1 to find the position of the number in the vector returning its position or the empty vector.
- h. Now, repeat step e, calling your new `binarySearch` function, saving the time as `binarySearchTime`.
- i. Plot the times of the linear and binary searches for a range of values of `N`. What do you learn from these plots?

Searching Graphs

Chapter Objectives

This chapter demonstrates algorithms for finding good paths through a graph. However, first we need to understand the following:

- How to construct and use two special forms of data collection: queues and priority queue
- How to build a model of a graph
- How to traverse and search a graph

Introduction

The collections we have considered so far—vectors, arrays, structures, and cell arrays—have essentially been collections of fixed size. In practice, of course, many MATLAB operations disguise the fixed character of the collections by performing complex operations that are invisible to the user to change the size of the collection.

The ultimate goal of this chapter is to discuss the most general form of dynamic data structure: the graph. In computer science, a **graph** is a collection of **nodes** connected by **edges**, as opposed to the engineering view of a graph that is produced by plotting, for example, $f(x)$ against x . To process graphs effectively, we must first consider two simpler concepts: a **queue** and a **priority queue**.

17.1 Queues and Priority Queues

17.1.1 The Nature of a Queue

17.1.2 Implementing Normal Queues

17.1.3 Priority Queues

17.1.4 Testing Queues

17.2 Graphs

17.2.1 Graph Examples

17.2.2 Processing Graphs

17.2.3 Building Graphs

17.2.4 Traversing Graphs

17.2.5 Searching Graphs

17.2.6 Breadth-First Search (BFS)

17.2.7 Dijkstra's Algorithm

17.2.8 Testing Graph Search Algorithms

17.3 Engineering Application—Route Planning



17.1 Queues and Priority Queues

We first consider the nature and implementation of queues, special collections that enable us to process graphs efficiently. We experience the concept of a queue every day of our lives. A line of cars waiting for the light to turn green is a queue; when we stand in line at a store or send a print job to a printer, we experience typical queue behavior. In general, the first object entering a queue is the first one to exit the other end.

17.1.1 The Nature of a Queue

Formally, we refer to a queue as a first in/first out (FIFO) collection, as illustrated in Figure 17.1. The most general form of a queue is permitted to contain any kind of object, that is, an instance of any data type or class. Typically, operations on a queue are restricted to the following:

- `enqueue` puts an object into the queue
- `dequeue` removes an object from the queue
- `peek` copies the first object out of the queue without removing it
- `isempty` determines that there are no items in the queue

17.1.2 Implementing Normal Queues

Although there are many ways to implement a queue, a cell array is a good choice because it is a linear collection of objects that may be of any type. If we establish a queue using a cell array, the mechanizations are trivial as follows:

- `qEnq` concatenates data at the end of the cell array
- `qDeq` removes the item from the front of the cell array and returns that item to the user
- `peek` merely accesses the first item in the cell array
- `isempty` is the standard MATLAB test for the empty vector

Clearly, because all the cell array operations are also accessible to the programmer, nothing prevents an unscrupulous programmer from using other operations on the queue—for example, adding an item to the front of the queue rather than the back to effectively “jump in line.” There are implementations beyond the scope of this text that use object-oriented

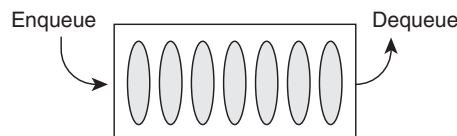


Figure 17.1 A queue

programming techniques to encapsulate the data and restrict the available operations on that data to those that implement the required functionality. However, for our purposes, the “open” cell array implementation is sufficient. Functions that perform the enqueue and dequeue operations for a queue are shown in Listing 17.1.

In Listing 17.1:

Line 1: Obviously, these two trivial functions should actually be in separate files. Both functions must return the updated queue because they receive copies of the original queue.

Line 2: Concatenates the enqueued item with a container (the braces) at the back of the cell array.

Line 3: The `dequeue` function must return the new queue and the item being removed.

Line 4: We return the first item on the cell array and remove it by returning the rest.

17.1.3 Priority Queues

There are times when we wish ordinary queues were priority queues. For example, at your favorite coffee bar when you simply want a cup of regular coffee and there are 10 people ahead of you ordering time-consuming steamed milk drinks, or at the printer where you wait an hour for one page while someone prints large sections of an encyclopedia. The only difference between an ordinary queue and a priority queue is in the `enqueue` algorithm. On a priority queue, the `enqueue` function involves adding the new item in order to the queue, as illustrated in Figure 17.2. For the `enqueue` function to

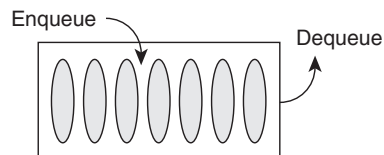


Figure 17.2 A priority queue

Listing 17.1 Enqueue and dequeue functions

```
1. function q = qEnq(q, data)
   % enqueue onto a queue
2.    q = [q {data}];

3. function [q ans] = qDeq(q)
   % dequeue
4.    ans = q{1};
5.    q = q(2:end);
```

add in order, there must be a means of comparing two objects. Here, we use the function `islt` that generally should be able to compare any two objects. In this implementation, it is sufficient to be able to compare numbers or structures that contain either the fields `key` or `dod`. Clearly, this can be extended as necessary to compare any two objects.

The code for `islt` is shown in Listing 17.2.

In Listing 17.2:

Line 1: Shows a function that consumes two objects and returns a Boolean result.

Line 2: Captures the data type (class) of the first object.

Line 3: Initializes the result.

Line 4: Checks that the second object is the same data type—otherwise, the `false` answer is returned.

Line 5: Decides how to compare the objects based on the data type.

Lines 6 and 7: Numbers are easily compared.

Line 8: Selects different structures to compare based on fields in the structure.

Lines 9 and 10: If the field `key` is present, compare these.

Lines 11 and 12: If the field `age` is present, compare these.

Lines 13–18: Show that an error exits.

The `enqueue` function that uses `islt` to compare objects for a priority queue is shown in Listing 17.3.

Listing 17.2 Comparing two objects

```

1. function ans = islt(a, b)
% comparing two objects
2. acl = class(a);
3. ans = false;
4. if isa(b, acl)
5.     switch acl
6.         case 'double'
7.             ans = a < b;
8.         case 'struct'
9.             if isfield(a, 'key')
10.                ans = a.key < b.key;
11.            elseif isfield(a, 'dod')
12.                ans = age(a) < age(b);
13.            else
14.                error('comparing unknown structures')
15.            end
16.        otherwise
17.            error(['can't compare ' acl 's'])
18.        end
19. end

```

Listing 17.3 Priority queue enqueue function

```

1. function pq = pqEnq(pq, item)
   % enqueue in order to a queue
2.     in = 1;
3.     at = length(pq)+1;
4.     while in <= length(pq)
5.         if islt(item, pq{in})
6.             at = in;
7.             break;
8.         end
9.         in = in + 1;
10.    end
11.    pq = [pq(1:at-1) {item} pq(at:end)];

```

In Listing 17.3:

Line 1: Shows the same signature as the enqueue function for ordinary queues.

Line 2 and 3: Initialize the while loop parameters.

Line 4: Moves the index `in` down the existing queue until it falls off the end of the cell array or finds something smaller than the `item` to insert. This second exit is implemented with a `break` statement.

Line 5: Checks whether the `item` is less than the current entry in the queue.

Lines 6 and 7: If so, mark the spot and exit the loop.

Lines 8 and 9: Otherwise, keep moving down the queue.

Line 11: Inserts the `item` in a container between the front part of the queue and the remains of the queue from `at` to the `end`.

17.1.4 Testing Queues

It is always advisable to test utility functions thoroughly before using them in complex algorithms. First we will build a simple utility for presenting the contents of any cell array, and then we will write a script to test the queues.

In order to observe the results from testing the queues, we need a function that will convert a cell array to a string for printing. Although it is tempting to try to write a single function to accomplish this, we achieve more maintainable code by separating the cell array traversal from the details of converting each item to a string. The first function, `CAToString`, which traverses the cell array, is shown in Listing 17.4.

Listing 17.4 Converting a cell array to a string

```

1. function str = CAToString(ca)
   % Traverse a cell array to make a string
2. str = '';
3. for in = 1:length(ca)
4.     str = [str toString(ca{in}) 13];
5. end

```

In Listing 17.4:

- Line 1: The function consumes any cell array and returns a string.
- Line 2: Initializes the string.
- Line 3: Traverses the cell array.
- Line 4: Extracts each item from the container, uses the `toString` utility function below to convert it to a string, and appends it to the end of the output string together with a new-line character (the ASCII value 13).

Of course, the real effort in creating this string is the second function that converts each individual item from the cell array to its string representation. This is shown in Listing 17.5.

In Listing 17.5:

- Line 1: Java programmers might recognize the concept of converting an object to its string equivalent.
- Lines 2 and 3: If the object is a string, surround it with single quotes.

Listing 17.5 Converting any object to a string

```

1. function str = toString(item)
% turn any object into its string representation
2. if isa(item, 'char')
3.     str = ['' item ''];
4. elseif isa(item, 'double')
5.     if length(item) == 1
6.         str = sprintf('%g', item );
7.     else
8.         str = '[';
9.         for in = 1:length(item)
10.            str = [str ...
                    sprintf(' %g', item(in) ) ];
11.        end
12.        str = [str ' '];
13.    end
14. elseif isa(item, 'struct')
15.     if isfield(item, 'name')
16.         str = item.name;
17.     else
18.         nms = fieldnames(item);
19.         str = [];
20.         for in = 1:length(nms)
21.             nm = nms{in};
22.             str = [str nm ': ' ...
                    toString(item.(nm)) 13];
23.         end
24.     end
25. else
26.     str = 'unknown data';
27. end

```

Lines 4–6: Individual scalar numbers are printed in %g form.

Lines 7–12: Vectors are enclosed in braces.

Line 14: Recursively uses `toString` to print the fields of a structure.

Lines 15 and 16: A special case wherein if there is a `name` field in the structure, the value of that field is used for the string.

Lines 18–23: Extract the field names and iterate through them one at a time, creating a string by appending each field name with its value and a new line.

Listing 17.6 illustrates a test script that exercises most of the available functions for queues and priority queues using numbers. However, a queue can contain any object you can display, and a priority queue can contain any object you can display and compare to another of the same type.

In Listing 17.6:

Line 1: Initializes a queue.

Lines 2–4: Enqueue 10 numbers.

Line 5: Displays the resulting queue.

Lines 6 and 7: Dequeue and print one value and the remaining queue.

Line 8: Peeks at the head of the queue and verifies that we have not changed its contents.

Line 9: Creates a priority queue.

Lines 10–14: Enqueue 10 random integers.

Line 15: Lists the queue to show that they were enqueued in order.

Listing 17.6 Testing the queues

```

1. q = [];
2. for ix = 1:10
3.     q = qEnq(q, ix);
4. end
5. CAToString(q)
6. [q ans] = qDeq(q);
7. fprintf('dequeue -> %d leaving \n%s\n', ...
    ans, CAToString(q) );
8. fprintf('peek at queue -> %d leaving \n%s\n', ...
    q{1}, CAToString(q) );

9. pq = [];
10. for ix = 1:10
11.     value = floor(100*rand);
12.     fprintf(' %g:', value );
13.     pq = pqEnq(pq, value );
14. end
15. fprintf('\npriority queue is \n%s\n', ...
    CAToString(pq) );

```

The results from this test are shown in Table 17.1.

Table 17.1 Results from testing the queues	
ans =	peek at queue -> 2 leaving
1	2
2	3
3	4
4	5
5	6
6	7
7	8
8	9
9	10
10	95: 23: 60: 48: 89: 76:
dequeue -> 1 leaving	45: 1: 82: 44:
2	priority queue is
3	1
4	23
5	44
6	45
7	48
8	60
9	76
10	82
	89
	95

17.2 Graphs

This chapter focuses on processing a graph—the most general form of dynamic data structure, an arbitrary collection of nodes connected by edges. A street map is a good example of a graph (the intersections are the nodes and the streets are the edges). The edges may be **directional** to indicate that the graph can be traversed along that edge in only one direction (like a one-way street). The edges may also have a value associated with them to indicate, for example, the cost of traversing that edge. We refer to this as a **weighted graph**. For a street map, this cost could either be the distance, or in a more sophisticated system, the travel time—a function of the distance, the speed limit, and the traffic congestion. Graphs are not required to be completely connected, and they can contain **cycles**—closed loops in which the unwary algorithm could become trapped. Graphs also have no obvious starting and stopping points. A **path** on a graph is a connected list of edges that is the result of traversing a graph.

17.2.1 Graph Examples

A simple graph is shown in Figure 17.3. In the figure the connection points A . . . F are the nodes and the edges are the interconnecting lines, which are

17.2.2 Processing Graphs

In designing algorithms that operate on graphs in general, we need to consider the following constraints:

- With cycles permitted in the data, there is no natural starting point like the beginning of a cell array. Consequently, the user must always specify a place on the graph to start a search, as well as the place to end.
- There are no natural “leaf nodes” where a search might have to stop and back up. Consequently, an algorithm processing a graph must have a means of determining that being at a given node is the “end of the line.” Typically, this is accomplished by maintaining a collection of visited nodes as it progresses around the graph. Each time a node is considered, the algorithm must check to see whether that node is already in the visited collection. If so, it refuses to return to that node. The algorithm must backtrack if it reaches a node from which there is no edge to a node that has not already been visited.
- Whereas on a cell array there is only one feasible path from one node to another, there may be many possible paths between two nodes on a graph. The best algorithms that search for paths must take into account a comparison between paths to determine the best one.

The upcoming sections discuss the following:

- Constructing a graph from the standpoint of understanding the structure of the representation
- Breadth-first search that always finds the path with the least number of nodes
- Optimal search that generates the best path through a graph based on the cost of each edge

For a simple, consistent example, consider the graph shown in Figure 17.5.

We might want to find a path from node A to node H. Since there are no arrows shown on the graph, we can assume that all edges are bidirectional.

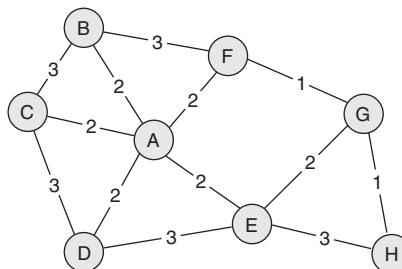


Figure 17.5 A weighted graph

The first step would be to create a representation of that graph. The second step would be to find a suitable way to search that graph for the best path.

17.2.3 Building Graphs

We need to consider graphs as two collections of data as follows:

- A list of n nodes with the properties of identity (a name) and position
- An $n \times n$ adjacency matrix that specifies the weight of the edge from each node to any other node

If one node is not reachable from another, by convention we will specify that weight as 0. This is actually a rather intimidating structure to build “by hand.” In order to facilitate reliable construction of the adjacency matrix, we start with a simpler description of the graph shown in Figure 17.5. This graph can be described initially with the following data:

- `cost` a vector of size $m \times 1$ containing the weights for each of m edges
- `dir` a vector of size $m \times 1$ indicating the directionality of each edge as follows:
 - 2 two-way edge
 - 1 one way in a positive direction
 - -1 one way in the other direction
- `node` a matrix of size $n \times \text{rows}$ containing the edge indices for each node. For example, if `node(i, j)` contains x , it says that the i th node connects to the x th edge. If x is 0, there is no connection. The value `rows` is the maximum number of nodes that can be reached from any other node.
- `coord` a matrix of size $n \times 2$ containing the x-y coordinates of each node that is used only for the graphical representation of the graph.

The script shown in Listing 17.7 starts with the above representation of the graph and calls the function `grAdjacency(...)` to produce the adjacency matrix. We will save this script as the constructor script `makeGraph.m`. Again referencing Figure 17.5, the sequence of edges used in this script are as follows:

A-B, A-C, A-D, A-E, A-F, B-F, B-C, C-D, D-E, F-G, E-G, G-H, E-H

Listing 17.7 Constructing a simple graph

```
% edge weights
cost = [2 2 2 2 2 3 3 3 3 1 2 1 3];
% edge directions
dir = [2 2 2 2 2 2 2 2 2 2 2 2 2];
% connectivity
```

continued on next page

```

node = [ 1  2  3  4  5; ...    % edges from A
        1  6  7  0  0; ...    % edges from B
        2  7  8  0  0; ...    % edges from C
        3  8  9  0  0; ...    % edges from D
        4 11 13  9  0; ...    % edges from E
        5  6 10  0  0; ...    % edges from F
       10 11 12  0  0; ...    % edges from G
       12 13  0  0  0];      % edges from H

% coordinates
coord = [ 5 6; ... % A
         3 9; ... % B
         1 6; ... % C
         3 1; ... % D
         6 2; ... % E
         6 8; ... % F
         9 7; ... % G
        10 2]; % H

A = grAdjacency( node, cost, dir )

```

Large adjacency matrices usually contain very little data relative to their size. Consequently, to store them as a conventional $n \times n$ array is to waste most of the storage space, and may even cause memory problems for the processor. Recognizing this eventuality, MATLAB provides a special class, `sparse`, that stores a matrix as lists of row and column indices and the associated value. All normal array and matrix operations can be applied to a sparse matrix. The assumption is that any value not specifically allocated in a sparse matrix contains a zero. This is consistent with the earlier treatment of vectors and arrays where unknown values are filled with 0.

The function `grAdjacency(...)` that converts graph data from this legible form to the adjacency matrix form builds a sparse matrix by establishing three vectors of the same length—the row index, the column index, and the value of each point in the sparse matrix. The code to accomplish this is shown in Listing 17.8.

Listing 17.8 Creating an adjacency matrix

```

1. function A = grAdjacency( node, cost, dir )
   % compute an adjacency matrix.
   % it should contain the weight from one
   % node to another (0 if the nodes
   % are not connected)
2. [m cols] = size(node);
3. n = length(cost);
4. k = 0;
   % iterate across the edges
   % finding the nodes at each end of the edge
5. for is = 1:n
6.     iv = 0;
7.     for ir = 1:m
8.         for ic = 1:cols

```

continued on next page

```

9.          if node(ir, ic) == is
10.             iv = iv + 1;
11.             if iv > 2
12.                 error(...
                    'bad intersection matrix');
13.             end
14.             ij(iv) = ir;
15.         end
16.     end
17. end
18. if iv ~= 2
19.     error(sprintf( ...
        'didn't find both ends of edge %d', is));
20. end
21. t = cost(is);
22. if dir(is) ~= -1
23.     k = k + 1;
24.     ip(k) = ij(1); jp(k) = ij(2); tp(k) = t;
25. end
26. if dir(is) ~= 1
27.     k = k + 1;
28.     ip(k) = ij(2); jp(k) = ij(1); tp(k) = t;
29. end
30. end
31. A = sparse( ip, jp, tp );

```

In Listing 17.8:

Line 1: Shows a function consuming the `node`, `cost`, and `direction` arrays defined above. The locations of the nodes are needed only for plotting.

Lines 2–4: Show initial parameters, where `k` is the number of entries in the sparse matrix.

Line 5: Iterates down the list of edges.

Line 6: Initializes the number of nodes found connected to the edge.

Lines 7 and 8: Iterates across the nodes and columns of the node array, looking for the nodes connected to the edge.

Lines 9 and 10: When we find the edge value, we want to save that node index.

Lines 11–13: There can be only two ends to an edge; any more indicates a bad data set.

Line 14: Saves each end in the local variable `ij`.

Lines 18–20: When we finish the traversal, there must be a node at each end of the edge.

Line 21: Retrieves the cost of this edge.

Lines 22–25: Since bidirectional edges must be in the matrix twice, we check to see if the edge is bidirectional or forward, and enter the forward path in the sparse matrix.

Lines 26–28: Similarly, the reverse path is entered only if the edge is not forward.

Line 31: Constructs the sparse adjacency matrix.

17.2.4 Traversing Graphs

We first develop an iterative template for traversing a graph, then adapt this template to deal with some practical issues of path search, and finally, we mechanize this template for two search techniques. In its simplest form, the template for graph traversal is shown in Template 17.1.

In Template 17.1:

Line 1: This algorithm uses a queue to serialize the nodes to be considered. The first in/first out behavior of the queue causes the nearest nodes to emerge before the nodes farther away.

Line 2: Since all nodes have equal status on a graph, graph traversal must be provided with the node from which to begin the traversal. We enqueue that node to begin the traversal.

Lines 3 and 4: Show the typical `while` loop traversal, initializing a result.

Lines 5 and 6: Extract and process one node.

Lines 7 and 8: Traverse the edges from this node. There must be an indication for each problem of which order to use in selecting the edges to the children of the current path.

Lines 9 and 10: Because the graph can contain cycles, the mechanism for preventing the algorithm from becoming trapped requires that we enqueue only those nodes that have not already been visited.

Template 17.1 Template for graph traversal

```

1. < create a queue >
2. < enqueue the start node >
3. < initialize the result >
4. while < the queue is not empty >
5.     < dequeue a node >
6.     < operate on the node >
7.     < for each edge from this node >
8.         < retrieve the other node >
9.         if < not already used >
10.             < enqueue the other node >
11.         end
12.     end
13. end
14. < return the result >

```

The choice of queue type governs the behavior of the traversal. If a simple queue is used, the traversal will happen like ripples on a pond from the starting node, touching all the nearest nodes before touching those farther away.

To illustrate the use of Template 17.1, we will print the names of all the nodes of the graph in Figure 17.5 in breadth-first order starting from node E, assuming that all edges are bidirectional. When choosing the edges to the next child node, the child nodes should be taken in alphabetical order as follows:

- Fortunately, because the adjacency matrix was built with the nodes in alphabetical order, the request will naturally produce them in alphabetical order.
- To make sure a node is not revisited, we will keep a list of the visited nodes, beginning with the start node.
- We find the children from the non-zero entries in the adjacency matrix—they are already in alphabetical order.
- We then traverse these children, adding to the queue those not found on the visited list and adding each to the visited list.

The script for this is shown in Listing 17.9.

Listing 17.9 Breadth-first graph traversal

```

1. makeGraph
   % Constructs an adjacency matrix
2. start = 5;
   % start is a node number (in this case, 'E')
   % Create a queue and
   % enqueue a path containing home
3. q = qEnq([], start);
   % initialize the visited list
4. visited = start;
   % initialize the result
5. fprintf('trace: ')
   % While the queue is not empty
6. while ~isempty(q)
   % Dequeue a path
7.     [q thisNode] = qDeq(q);
   % Traverse the children of this node
8.     fprintf('%s - ', char('A'+thisNode-1));
9.     children = find(A(thisNode,:) ~= 0);
10.    for aChild = children
        % If the child is not on the path
11.        if ~any(aChild == visited)
            % Enqueue the new path
12.            q = qEnq(q, aChild);
            % add to the visited list
13.            visited = [visited aChild];
14.        end % if ~any(eachchild == current)
15.    end % for eachchild = children
16. end % while q not empty
17. fprintf('\n');

```

In Listing 17.9:

- Line 1: Invokes the script in Listing 17.7 to build the adjacency matrix.
- Line 2: The user-defined starting node—E.
- Line 3: Enqueues the starting node on a new queue.
- Line 4: Initializes the visited list.
- Line 5: Initializes the result—in this case, a printout.
- Line 6: Shows the `while` loop.
- Line 7: Dequeues a node.
- Line 8: In this case, processing the node involves printing its label and a dash.
- Line 9: The non-zero values from the row in the adjacency matrix corresponding to this edge give us the children of this node.
- Line 10: Traverses the children.
- Lines 11–13: If they are not already on the visited list, enqueue them and put them on it.
- Line 17: Completes the result when the queue is empty.

The results from this script are as follows:

```
trace: E - A - D - G - H - B - C - F -
```

which, referring to Figure 17.5, is a “ripple” traversal from node E outward, taking children in alphabetical order as specified.

17.2.5 Searching Graphs

In order to search a graph rather than traverse it, we have to make the following changes to Template 17.1:

- Since we need to return the complete path between the start and target, the queue has to contain that path
- Rather than use a global visited list, we can use the path taken from the queue to determine whether a child node is causing a cycle
- The order of the nodes on the path has the starting node at the front of the path and the new node at the end

17.2.6 Breadth-First Search (BFS)

Frequently, we actually need the path with the smallest number of nodes between the starting and ending nodes. For example, because changing trains involves walking and waiting, the best path on a railway map (such as the one shown in Figure 17.4) is that with the fewest changes, even if the resulting path is longer.

To search for the path with the least nodes, we need a function that performs a Breadth-First Search (BFS) on a graph. In order to be able to use MATLAB's built-in graph plotting program, the answer returned should be an adjacency matrix showing the computed path. The function to perform this search is shown in Listing 17.10.

In Listing 17.10:

Line 1: Shows a function consuming an adjacency matrix, the starting and destination node indices.

Line 2: Initializes the queue.

Line 3: Shows the usual `while` loop.

Listing 17.10 Breadth-first graph search

```

1. function D = grBFS(A, home, target)
% A - an adjacency matrix
% home - starting node index
% target - ending node index
% produce the adjacency matrix D of the
% breadth-first path from home to target.
% If no such path exists, return []
% Create a queue and enqueue a path containing home
2. q = qEnq([], home);
% While the queue is not empty
3. while ~isempty(q)
    % Dequeue a path
4.     [q current] = qDeq(q);
    % If this path reaches the destination, stop
5.     if current(end) == target    % success exit
6.         D = sparse([0]);
        % build a new adjacency matrix
        % from the path
7.         for ans = 1:length(current)-1
8.             D(current(ans), current(ans+1)) = 1;
9.         end
10.        return;    % exit the function
11.    end % if current == target
    % Traverse the children of the last node
12.    thisNode = current(end);
13.    children = find(A(thisNode,:) ~= 0);
14.    for thisChild = children
        % If the child is not on the path
15.        if ~any(thisChild == current)
            % Enqueue the new path
16.            q = qEnq(q, [current thisChild]);
17.        end % if ~any(thisChild == current)
18.    end % for thisChild = children
19. end % while q not empty
% If we reach here we never found a path
20. D = [];

```

Line 4: The queue now contains a vector of the node indices on the current path.

Line 5: If the node dequeued (`current(end)`) is the target, the function creates a new adjacency matrix representing the path from the home node to the target.

Line 6: Creates an empty sparse matrix.

Lines 7–9: Add to it the edges between each node in the path.

Line 10: Exits the function.

Line 12: Otherwise, recovers the last node.

Line 13: Retrieves its children.

Lines 14–18: Traverse the children as before, checking for their presence on the current path. When a child is enqueued, it is appended to the end of the current path and the whole path is enqueued.

The BFS path from A to H is shown in Figure 17.6. Note that it found the path with the least number of nodes.

17.2.7 Dijkstra's Algorithm

Although the minimal number of nodes is sometimes the right answer, frequently there is a path that uses more nodes but has a smaller overall cost. This is evident from a quick glance at Figure 17.5: the path A–F–G–H has a lower cost than the A–E–H path found by the BFS algorithm, which actually ignores the edge weights. Many algorithms exist for finding the optimal path through a graph. Here we illustrate the algorithm attributed to the Dutch computer scientist Dr. Edsger Dijkstra. Perhaps it is not the most efficient algorithm, but for our purposes, this approach has the virtue of being a minor extension to the `while` loop algorithms used for BFS.

- The major differences arise from the use of a priority queue in place of the normal queue used in the BFS algorithm. As previously noted, priority queues differ from basic queues only to the extent that the `enqueue` method puts the data in order, rather than at the tail.

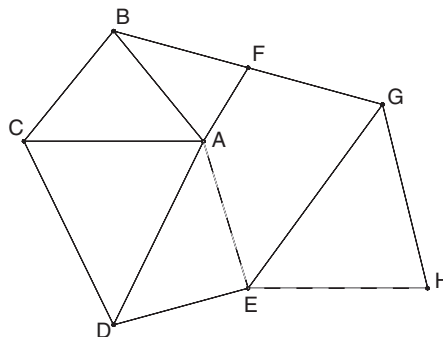


Figure 17.6 Breadth-first result

- The ordering criterion required by the algorithm is to place the paths in increasing order of path cost (total weight).

The objects contained in the priority queue need to contain not only the path used for BFS, but also the total path weight. For this we will use a structure with fields `nodes` and `key`, and implement a small collection of helper functions. The priority queue and its helper function to compare two structures with a key are shown in Listings 17.2 and 17.3; `Path(...)` to construct a new entry and `pthGetLast(...)` to get the index of the last node on the path are shown in Listing 17.11.

In Listing 17.11:

Line 1: Shows a function to construct a path structure from its components.

Lines 2 and 3: Build the structure.

Line 4: Shows a function to retrieve the last node from a path.

Line 5: Since the path nodes start at the path origin, the last entry is the node we need.

The function that performs Dijkstra's algorithm is shown in Listing 17.12.

Listing 17.11 Helper functions for Dijkstra's algorithm

```
1. function ret = Path(nodes, len)
   % Path constructor
2.     ret.nodes = nodes;
3.     ret.key = len;

4. function ret = pthGetLast(aph)
   % Returns number of last node on a path
5.     ret = aph.nodes(end);
```

Listing 17.12 Code for Dijkstra's algorithm

```
1. function D = grDijkstra(A, home, target)
   % A - an adjacency matrix
   % home and target - node indices.
   % Create a priority queue
   % Enqueue a path with containing home with length 0
2.   pq = pqEng([], Path(home, 0));
3.   while ~isempty(pq)
   % Dequeue a path
4.     [pq current] = qDeq(pq);
   % If this path is to the destination quit
5.     if pthGetLast(current) == target
6.       D = sparse(0);
7.       answer = current.nodes;
8.       for ans = 1:length(answer)-1
9.         D(answer(ans), answer(ans+1)) = 1;
10.      end
```

continued on next page

```

11.         return;
12.     end % if last(current) == target
    % put the children of the last node in the path
13.     endnode = pthGetLast(current);
14.     children = A(endnode,:);
15.     children = find(children ~= 0);
16.     for achild = children
17.         len = A(endnode, achild);
    % If the child is not on the path
18.         if ~any(achild == current.nodes)
    % Add it to the path
19.             clone = Path( [clone.nodes achild] ...
20.                 current.key + len;
    % Enqueue the new path to that length
21.             pq = pqEnq(pq, clone);
22.         end % if ~any child == current.nodes
23.     end % for achild = children
24. end % if pq not empty
% If we reach here we never found a path
25. D = [];

```

In Listing 17.12:

Line 1: Shows a function consuming an adjacency matrix, the starting and destination node indices.

Line 2: Initializes the priority queue with a starting node and zero cost.

Line 3: Shows the usual `while` loop.

Line 4: Shows that the queue now contains a path structure.

Lines 5–12: If the node dequeued is the target, the function creates a new adjacency matrix representing the path from the home node to the target.

Line 13: Otherwise, it recovers the last node.

Line 14: Retrieves its children.

Lines 15–23: Traverse the children as before, checking for their presence on the current path. When a child is enqueued, it is appended to the end of the current path, and the whole path is enqueued.

The optimal path from A to H is shown in Figure 17.7. Note that it found the path with the least cost.

17.2.8 Testing Graph Search Algorithms

The script that develops both search path solutions is shown in Listing 17.13. It requests the starting and ending node letters from the user and then incrementally plots the original graph, the BFS solution, and the optimal solution. The pause between plots allows the individual paths to be examined. Without a parameter, `pause` waits for any keyboard character.

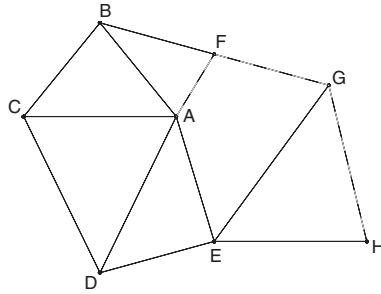


Figure 17.7 Dijkstra's result

In Listing 17.13:

- Line 1: Constructs the adjacency matrix.
- Line 2: Forces your way into the `while` loop once.
- Line 3: Keeps going until the user enters an illegal starting node.
- Lines 4 and 5: Plot the graph using the adjacency matrix and the coordinates stored for the nodes.
- Lines 6–10: Put letters by the nodes.
- Line 11: Sets the axes.
- Lines 12 and 13: Get the starting node.
- Lines 14–16: If valid, get the target node.
- Line 17: Shows the original graph and waits for a character.
- Lines 18–20: Compute and plot the BFS solution.
- Lines 21–24: Compute and plot the optimal solution.
- Lines 25 and 26: Repeat as necessary.

Listing 17.13 Testing graph search algorithms

```

1.  makeGraph; % call script to make the graph:
2.  start = 1;
3.  while start > 0
4.      gplot(A, coord, 'ro-')
5.      hold on
6.      for index = 1:length(coord)
7.          str = char('A' + index - 1);
8.          text(coord(index,1) + 0.2, ...
9.              coord(index,2) + 0.3, str);
10.     end
11.     axis([0 11 0 10]); axis off; hold on
12.     ch = input('Starting node: ', 's');
13.     start = ch - 'A' + 1;
14.     if start > 0
15.         ch = input('Target node: ', 's');
16.         target = ch - 'A' + 1;
17.         disp('original graph'); pause
18.         D = grBFS( A, start, target);

```

continued on next page

```

19.          gplot(D, coord, 'go-')
20.          disp('BFS result'); pause
21.          D = grDijkstra( A, start, target);
22.          gplot(D, coord, 'bo-')
23.          disp('Optimal result'); pause
24.          hold off
25.      end
26.  end

```



17.3 Engineering Application—Route Planning

One obvious application is to use these algorithms for route planning in a city. Figure 17.8 shows the results from a Dijkstra's path search on a small city street map. Although this particular search used street length as the cost of each edge, it is not difficult to visualize a system architecture whereby individual vehicles could receive real-time data feeds indicating the actual speed of vehicles on the streets and wait times at intersections. The cost of using a street in terms of travel times could then be factored into the calculation. An advanced mapping system in vehicles would then be able to recommend alternate routes to avoid bottlenecks on city streets, thereby saving significant amounts of time and energy now lost idling in city traffic.

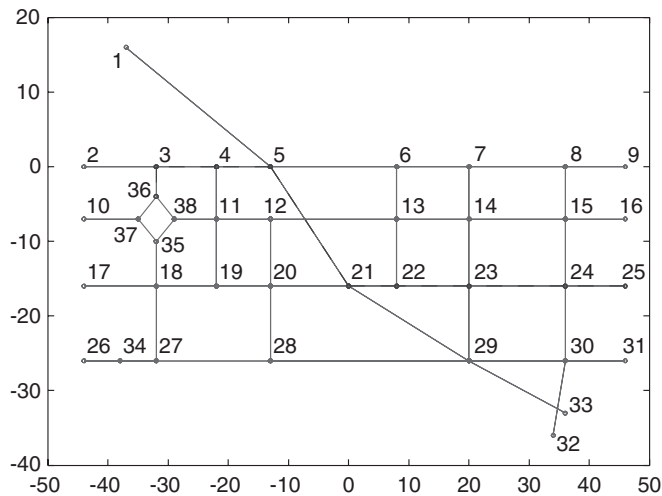


Figure 17.8 Street map results

Chapter Summary

This chapter demonstrated effective algorithms for finding good paths through a graph, and included the following:

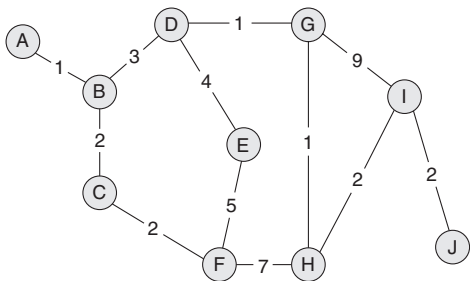
- How to construct and use queues and priority queues as the underlying mechanism for graph traversal
- The basic use of an adjacency matrix for defining a graph
- How to use a queue to develop traversal and search results on any kind of graph

Special Characters, Reserved Words, and Functions

Special Characters, Reserved Words, and Functions	Description	Discussed in This Section
<code>break</code>	A command within a loop module that forces control to the statement following the innermost loop	17.1.3
<code>any()</code>	True if any of the values in <code>a</code> , a logical vector, is true	17.2.4
<code>class(obj)</code>	Determines the data type of an object	17.1.3
<code>gplot(A,coord)</code>	Plots a graph from an adjacency matrix and an array of coordinates	17.2.8
<code>isa(obj, str)</code>	Tests for a given data type	17.1.3
<code>sparse(r,c,value)</code>	Builds a sparse matrix from vectors of row and column indices and the associated value	17.2.3

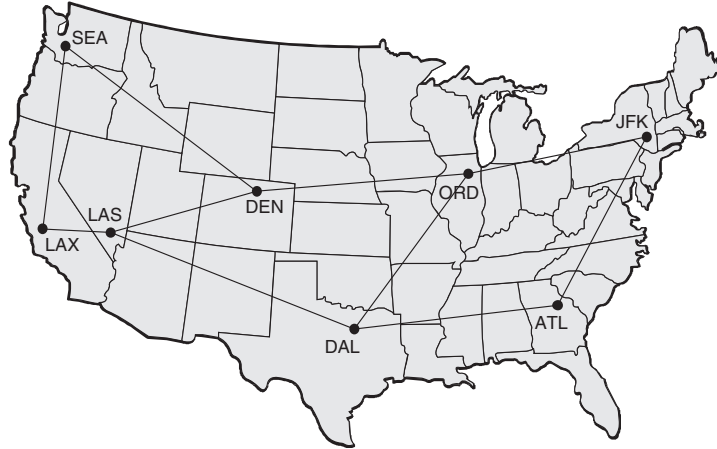
Programming Projects

1. Consider the graph shown in the figure below.



- a. Write a script to construct this graph.

- b. Compute the shortest path and the path with least nodes between nodes A and J.
 - c. Verify the answer by manually executing the BFS and Dijkstra's algorithms.
2. Consider the graph shown in the figure below.



- a. Obtain the route pattern for a small airline.
 - b. Write the code to build a model of their route map.
 - c. Write a script that asks the user for an origin and destination and computes the shortest route and the route with the fewest connections.

Object-Oriented Programming

Chapter Objectives

This chapter discusses the object-oriented paradigm, including:

- Object-oriented programming (OOP) in general
- How MATLAB implements OOP

This chapter is not intended as a full treatment of the art and science of object-oriented design and object-oriented programming (OOP). Rather, it presents some basic definitions of terms used in OOP and the implementation in MATLAB of some simple constructs. Subsequent chapters will extend these ideas to illustrate how dynamic data structures may be constructed and manipulated using OOP.

- 18.1** Object-Oriented Programming
 - 18.1.1 OO Background
 - 18.1.2 Definitions
 - 18.1.3 Concepts
 - 18.1.4 MATLAB Observations
- 18.2** Categories of Classes
 - 18.2.1 Modeling Physical Things
 - 18.2.2 Modeling Collections
 - 18.2.3 Objects within Collections
- 18.3** MATLAB Implementation
 - 18.3.1 MATLAB Classes
 - 18.3.2 MATLAB Objects
 - 18.3.3 MATLAB Attributes
 - 18.3.4 MATLAB Methods
 - 18.3.5 Encapsulation in MATLAB Classes
 - 18.3.6 Inheritance in MATLAB Classes
 - 18.3.7 MATLAB Parent Classes
 - 18.3.8 MATLAB Child Classes
 - 18.3.9 Polymorphism in MATLAB
- 18.4** Example—Modeling Bank Accounts
 - 18.4.1 The Base Class
 - 18.4.2 Inheritance by Extension
 - 18.4.3 Inheritance by Redefinition
- 18.5** Practical Example—Vehicle Modeling
 - 18.5.1 A Vehicle Hierarchy
 - 18.5.2 The Containment Relationship



18.1 Object-Oriented Programming

In Chapters 1–17 we saw MATLAB as a means for manipulating **arrays**—sometimes in the guise of vectors or matrices, character strings, or cell arrays. Furthermore, the manipulation performed was conducted by writing scripts that sometimes call functions—either the functions built into MATLAB or functions we create ourselves. This type of programming is referred to as being in the procedural paradigm—functions and scripts as a form of procedure.

This chapter considers a different paradigm altogether—the object-oriented (OO) paradigm. In this programming style, we still begin with a script, but the scripts that we write will usually create and interact with **objects** rather than **arrays**.

18.1.1 OO Background

Languages that express the essential elements of the OO paradigm have been around since the 1960s when Simula was first developed. However, in the 1980s and 1990s, as massive software projects and especially graphical user interfaces (GUIs) became common, OO emerged as the paradigm of choice for designing and developing large software systems. Major software systems (like the various releases of Microsoft Windows) faced enormous design and integration challenges that could not be met by conventional programming practices. They needed language-imposed management of the interaction between large and small collections of programs and data.

A secondary requirement in efficiently developing large software systems is the ability to reuse core software modules without rewriting their entire contents. OO principles allow core modules to be reused in three ways:

- Reused intact, because the definitions of how to use them are precisely recorded
- Reused and extended, adding specific custom capabilities not found in the original module, but using all of the original capabilities
- Reused and redefined, replacing a few attributes of the general module by more specific definitions, while retaining all of the original characteristics

The OO paradigm emerged as the framework that made large, reliable software systems possible. Note that OO encompasses far more than the syntax of any particular language, and far more than its core concepts, which are touched briefly in this section of the book. OO is primarily a design issue. OO design takes advantage of the tenets and concepts of OO languages to produce good software system designs, from which good software systems can be built.

Many books are available to students wishing to pursue this subject further. However, a book on computer science concepts would be incomplete without a serious treatment of, and some practical exposure to, OO concepts. As we study OO concepts and their MATLAB implementation,¹ it should be in the light of realizing the power of these concepts, and their place in the development of significant software systems.

18.1.2 Definitions

In general, we will use the following definitions:

- A **class** is the generic description of something. For example, a Toyota Prius indicates the nature of the car and the fact that one must specify a color and body style, and defines the functional relationships between, for example, the speed and fuel consumption.
- An **object** is an instance of a class in the same way that one specific Toyota Prius is an instance or example of the Toyota Prius design, having a specific identification number, color and body style, and more concretely, its own values for its current location, speed, fuel contents, and so on.
- An **attribute** is a data component of the class definition that will have a specific value in an object of that class at a given time. For example, all cars have a speed, but you must examine a particular car to determine its current speed.
- A **method**, much like a MATLAB function, is a procedural abstraction attached to a class that enables manipulation of the attributes of a particular object. Unlike a MATLAB function, methods have access not only to their own workspace, but also to the attributes of the class of which they are a part.
- **Encapsulation** is actually the core of good OO design. It is the process of packaging attributes and methods in such a way that you define and control the interfaces through which outside users (other objects) can access the attributes of your objects.
- **Inheritance** is the characteristic that enables the reuse and/or extension of core facilities. It is the facility by which a general, core class can be extended to add more specific attributes and methods, perhaps redefining the behavior of a few existing methods, but retaining access to the original, unmodified behavior.
- A **parent class** is the class from which other classes inherit characteristics.

¹ The MATLAB implementations in this text were designed to remain as close as possible to a style that permits ready translation to more conventional OO language implementations (Java, C++, or C#). Where possible, we will note typical Java implementations of the MATLAB software artifacts.

- **Child classes** are classes derived from parent classes by inheritance.² Of course, grandchild classes can also inherit from child classes, and so on.
- **Polymorphism**³ is the ability to treat all the children of a common parent as if they were all instances of that parent. It has two important aspects, as follows:
 1. All objects that are children of a parent class must be treated collectively as if they were all instances of the parent
 2. Individually, the system must reach the specific child methods when called for

18.1.3 Concepts

Behavioral abstraction is the central concept in OO programming. In Section 3.2.1 we discussed **data abstraction** as the ability to group disparate data items as arrays or structures that permit us to discuss “the temperature readings for July” or “the information about that CD” without having to enumerate all the details. In Section 5.1 we also referred to functions as implementing **procedural abstraction** whereby we could collect a number of recurring instructions, group them, and invoke them without reiterating all the details.

Behavioral abstraction combines these two abstractions, allowing us to encapsulate not only related data items, but also the operations that are legal to perform on those data items. In the terms defined above, we can visualize a class as the encapsulation of a number of attributes with the methods that operate on those attributes to describe the behavior of an object.

An **abstract data type (ADT)** is a technique for describing the general character of a class without descending into specifics. The format of an ADT does not have to be graphical—there are good textual techniques for accomplishing the same objective. The specific ADT format we will use in this text is shown in Figure 18.1, where the data defined for the class are enclosed in the box and the methods for operating on the data are identified by the rectangles. We will use this form to define the overall behavior of classes before diving into the detailed implementation.

18.1.4 MATLAB Observations

The basic MATLAB functionality we have already used exhibits some interesting behaviors. Consider, for example, the plus operator, `A + B`.

² Some OO languages permit child classes to inherit from more than one set of parents—a practice that can cause significant logical challenges. Other languages (Java, for one) enforce single inheritance chains, but provide alternate mechanisms (interfaces) for enforcing other common behaviors. We will see interface-like characteristics in our MATLAB code examples.

³ From a Greek word meaning “taking many forms.”

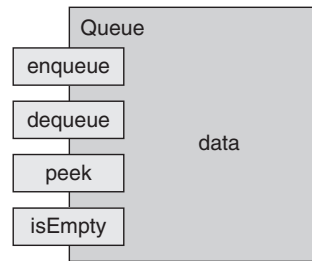
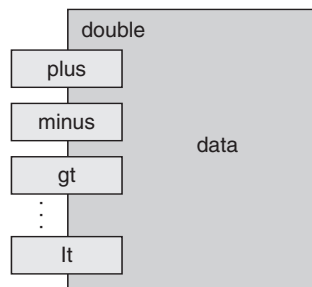


Figure 18.1 ADT illustration

Depending on the actual nature of A and B , this might have very different effects. For example, if A is an array, its actual type (referred to in the MATLAB workspace window as its class) is `double`. MATLAB actually interprets $A + B$ as `plus(A, B)`; in other words, it applies the `plus` method to the object A with B as the second parameter. Depending on the nature of B , this operation may have very different results:

- If A is 1×1 (really, a scalar value), the operation will succeed when B is just about anything
- If A is any sized array, the operation will succeed only if B is either a scalar or the same size as A
- Otherwise, the operation will fail with an error message

These seem to be obvious statements, but they are actually quite profound and fundamental to our understanding of OO. This means that what we have previously considered to be an “open collection” of numbers is actually a collection “protected from the world” by a set of methods that “understand” what can and cannot be done with the data (see Figure 18.2).

Figure 18.2 ADT for the class `double`

18.2 Categories of Classes

In general, classes fall into two categories: classes that model individual objects (physical or abstract), such as vehicles, bank accounts, or GUI widgets;

and classes that model collections of things, such as arrays or queues. It is important to keep these model categories separate, but to provide for the fact that collection classes need to be able to hold objects of many different types—individual objects and other collection objects.

For example, if we intended to model traffic flow in a city, we might begin by considering the city map as a collection of streets and intersections. The role of a street object is to contain and organize the vehicles moving on that street; the role of an intersection object (which might temporarily contain vehicles) is primarily to move vehicles from one street to another. Since one of the attributes of each vehicle should be the route it is currently following, the intersection should query a vehicle to discover which direction it should turn.

18.2.1 Modeling Physical Things

We talk generically about the vehicles on the streets. In general, a vehicle would have a size, heading, location, speed, and route plan. It would have a method for moving along a street, and perhaps a generic method for drawing the vehicle when necessary. Specific vehicles would have specific constraints—motorcycles might have a higher maximum speed, for example. If we were drawing the vehicles on the streets, each specific vehicle would also have its own specific drawing method. So a vehicle class would encapsulate the basic data about a specific vehicle with methods like `draw(...)` and `move(...)`.

18.2.2 Modeling Collections

When we look closer at the behavior of vehicles on a street (excluding passing, accidents, and so on), they behave as if they are in a queue. They join at the back of the queue, and arrive in order at the intersection at the other end of the street. So the streets really are containers of collections of vehicles with a specific behavior that we might describe as a queue. Therefore, a queue class would encapsulate objects it currently contains with methods like `enqueue(...)` and `dequeue(...)`, as shown in Figure 18.1.

In a totally different application, if we were, for example, emulating a Polish notation calculator, we would need a stack to hold the intermediate values. We might visualize a stack⁴ class `MStack`, as shown in Figure 18.3. By convention, we refer to the process of adding data to a queue as enqueueing and the process of adding data to a stack as pushing. We refer to removing data from queues and stacks as dequeuing and popping, respectively.

Finally, there are advanced processing applications that require a special kind of queue called a priority queue. Imagine, for example, a process for organizing print jobs where one does not have to wait for small print jobs to

⁴ Actually, since MATLAB has a predefined stack class, we will name our stack class `MStack`.

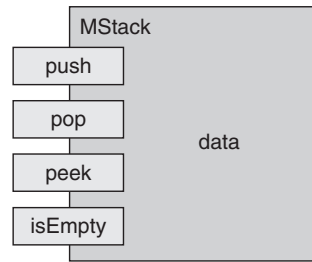


Figure 18.3 ADT for a stack

print while long print jobs are being processed. Jobs would be enqueued for printing, and small jobs would be put in the queue ahead of larger jobs. This is an example of a priority queue.

18.2.3 Objects within Collections

A street will process each vehicle generically by traversing its queue contents, but the vehicle-specific characteristics will govern their actual behavior. In upcoming chapters we will discuss the MATLAB implementation of such a model that integrates objects with object collections; but first, we will separately consider modeling objects and collections with simple examples.



18.3 MATLAB Implementation

MATLAB does a credible job implementing the OO language characteristics detailed in Section 18.1. Consider the very simple pair of classes shown in Figure 18.4. Both the parent class, `Fred`, and the child class, `FredChild`, contain one data item and host one method named `add`. The parent `add` method adds the value provided to its local data storage. The child `add` method adds to its own data and to the parent's data.

The following paragraphs indicate in this context how MATLAB implements the OO characteristics enumerated in Section 18.1.

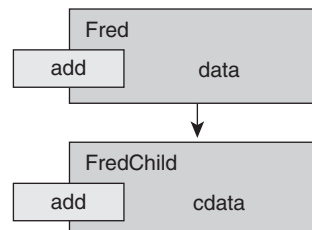


Figure 18.4 Example classes

18.3.1 MATLAB Classes

In MATLAB a class is a collection of functions (the methods of that class) stored as M-files in a specific subdirectory of the current workspace. For example, if we were modeling the class named `Fred` illustrated in the figure, all of the public methods (those you want to be accessible from outside the class) must be stored in a directory named `@Fred`. There are two ways to withhold public access to methods. If you need a private utility for use by only one of `Fred`'s methods, it can be included as a local function inside that method definition file. Utilities to be shared by more than one method but not from outside the class can be stored in a subdirectory named `private`. MATLAB requires the following methods to be present:

- A constructor named the same as the class (`Fred` in our example) that may contain parameters used to initialize the object and that returns the completed object.⁵ However, it must make provisions for two features: calling the constructor with no parameters and calling the constructor to copy another object of the same class—accomplished by calling the constructor with the object to copy as the only parameter. The `Fred` constructor is shown in Listing 18.1.
- A `display(...)` method that is used by MATLAB whenever an assignment is made to a variable of this class without the semicolon to suppress the display.
- A `char(...)` method⁶ that returns a string describing the content of this object. Typically, the `display(...)` method merely contains `disp(char(...))`, but this method also allows functions like `fprintf(...)` to display the object as a string.
- `set` and `get` methods for any attributes accessible by child classes.
- Other methods required by the class specification.

18.3.2 MATLAB Objects

An **object** in MATLAB is created in a test script or a method of another class by assigning to a variable a call to the constructor of the required class with or without initializing parameters. For example, `myFred = Fred` would create an object `myFred` with default values of its attributes. Having created the object, its methods are called by invoking them by name with `myFred` as the first parameter.⁷ For example, to display the object, one might use the following:

```
fprintf('the object is %s\n ', char(myFred) );
```

⁵ Java constructors normally do not return the object—they just initialize its attributes.

⁶ Java enthusiasts might recognize this as `toString()` except that MATLAB does not automatically invoke `char(...)` if it is expecting a string but sees an object.

⁷ Most OO implementation languages use the “structure access” style for invoking methods on an object like: `myFred.char()`.

18.3.3 MATLAB Attributes

Attributes in MATLAB are stored in a structure we will refer to as the **attribute structure** that actually becomes typed as the object. The structure is made into an object by casting its type in the class constructor. This is shown in Listing 18.1.

In Listing 18.1:

Line 1: The constructor for any class looks exactly like a function returning an object of that class.

Line 2: To indicate the purpose of this particular function, we include `@Fred\` in the first comment line to indicate that this constructor should be stored in that directory.

Lines 3–5: Show the remainder of the documentation.

Line 6: If no arguments were provided, this is the default constructor that must populate the attributes of the class with default values. Since MATLAB uses default constructors internally, this logic must always be there.

Line 7: Sets the default contents of the attribute named `value`. Notice that at this point we have created a structure named `frd`.

Line 8: We change the data type of the structure from `struct` to `Fred`, thereby restricting external access to the attributes of any object.

Line 9: This could also be a copy constructor designed to make a copy of another object of type `Fred`. We detect this by checking the class of the parameter.

Line 10: Since MATLAB always passes by value, the data item is already a copy of the original object, so just return that.

Listing 18.1 Constructor for the `Fred` class

```

1. function frd = Fred(data)
2. % @Fred\Fred class constructor.
3. % frd = Fred(data) creates a Fred containing data
4. % could also be a copy constructor:
5. %   newF = Fred(oldF) copies the old Fred oldF
6. if nargin == 0           % did the user provide data?
7.     frd.value = 0;       % set the attribute
8.     frd = class(frd,'Fred'); % cast as a Fred
9. elseif isa(data, 'Fred') % copy constructor?
10.    frd = data;
11. else
12.    frd.value = data; % data provided
13.    frd = class(acct,'Fred');
14. end

```

Lines 12 and 13: This is the real constructor, which is very similar to the default constructor except the attribute is set to the data provided.

18.3.4 MATLAB Methods

A method in MATLAB is written like a function, is stored in the class folder, and obeys all the normal rules of functions. The object attributes are presented to the method as its first parameter (which is actually an attribute structure). If the method changes any attributes of the object, it is necessary to return the updated attribute structure to the calling program. This is an ugly dilemma. Most conventional OOP languages implement method calls by treating the methods of a class like attributes of a structure, and allow the user to write calling code as follows:

```
myObject = Fred(4);  
myObject.add(3);
```

As a result, we would optimistically imagine that the value stored in `myObject` would be 7. However, MATLAB cannot invoke methods in this form because it has no mechanism for treating functions as attributes. Therefore, it must provide the object as a parameter (usually the first one) to a regular function call. Consequently, the above code has to be implemented as follows:

```
myObject = Fred(4);  
add(myObject, 3);
```

So we ask ourselves whether `myObject` now contains the value 7. Of course not, because the function `add(...)` received a copy of `myObject` and has no access to the original. We would be tempted to concede defeat and write the example for a third time as follows, where the function has to return an updated copy of the original object:

```
myObject = Fred(4);  
myObject = add(myObject, 3);
```

Although this last approach works, at least from the user's perspective, it is far removed from the original concept of an object as one with persistent data. Furthermore, it takes away the ability to easily return a result from an object method. Fortunately, there is a construct in MATLAB that allows the call `add(myObject,3)` to tunnel back to the caller's workspace and modify the original object. In this text we have chosen the tunneling back approach shown in Listing 18.2, which illustrates the code to add something to the data stored in an object of type `Fred`. After performing the addition, the following line:

```
assignin('caller', inputname(1), frd)
```

is the command that tunnels back. First it obtains the name of the variable provided to this method using `inputname(1)`, and then it assigns the updated attribute structure `frd` to that name in the caller's workspace.

Listing 18.2 Fred's add method

```

1. function add(frd, data)
2. % @Fred\add to add data to its value.
3. % add(frd, data)
4.     frd.value = frd.value + data;
5.     assignin('caller', inputname(1), frd);

```

In Listing 18.2:

Line 1: Shows the function header with an object of class `Fred` as the first parameter.

Lines 2 and 3: Show the documentation.

Line 4: Modifies the copy of the original object.

Line 5: Tunnels back to the caller's workspace, finds the variable provided as the first parameter name, and copies our new object to that variable.

18.3.5 Encapsulation in MATLAB Classes

Encapsulation is accomplished in MATLAB by storing all the methods for a class in the folder named for the class (`@Fred` in the example).

18.3.6 Inheritance in MATLAB Classes

Inheritance is sometimes referred to as an "IS-A" relationship because it defines a child class that exhibits all the behavior of the parent class as well as its own unique characteristics. It is accomplished in the MATLAB constructor for the child class. A local variable is first initialized as an instance of the parent class. The class cast method that creates the new child object includes this parent instance as a third parameter. The end result of this manipulation is an additional attribute in the child class that has the same name as the parent class and contains a reference to the parent object. When the child class needs access to the methods of the parent,⁸ it refers to these methods by way of this reference. Listing 18.3 shows the code for the child constructor.

Listing 18.3 Fred child constructor

```

1. function fc = FredChild(pd, cd)
2. % @FredChild class constructor.
3. %   fc = FredChild (pd, cd) creates a FredChild
4. %       whose parent value is pd, and local
5. %       attribute is cd

```

continued on next page

⁸ MATLAB child classes cannot directly access the attributes of the parent. They are accessed and modified only by way of the parent's set and get methods.

```

6. % This could also be a copy constructor:
7. %   fc = FredChild ( ofc ) copies the object ofc
8. if nargin == 0
9.     super = Fred;
10.    fc.cdata = 0;
11.    fc = class(fc, 'FredChild', super);
12. elseif isa(pd, 'FredChild')
13.    fc = pd;
14. else
15.    super = Fred(pd);
16.    fc.cdata = cd;
17.    fc = class(fc, 'FredChild', super);
18. end

```

In Listing 18.3:

Lines 1–8: Show a typical header block for this constructor.

Line 9: When constructing a default child, we first construct a local parent object named `super` with default contents.

Line 10: We set the child data items to default values.

Line 11: We include the parent object in the class cast to establish the parent-child relationship. The actual result of including `super` is to establish another field in the `FredChild` structure named `Fred` whose content is a `Fred` object.

Lines 12 and 13: Show the copy constructor.

Line 15: The real constructor passes the `pd` parameter to create the parent object.

Line 16: Sets the child data field.

Line 17: As with the default constructor, we establish the parent-child link by passing the parent object to the class cast.

18.3.7 MATLAB Parent Classes

A **parent class** is the class from which other classes inherit characteristics.

18.3.8 MATLAB Child Classes

Child classes are derived from parent classes by inheritance.⁹ Of course, grandchild classes can also inherit from child classes. A child class cannot directly access the attribute structure of its parent. However, it can access the methods of the parent class by way of an attribute with the name of the parent class. This logic needs careful study. Examine the diagram shown in Figure 18.5, the explanatory notes, and the code for the `add` method of the child class

⁹ Some OO languages permit child classes to inherit from more than one set of parents—a practice that can cause significant logical challenges. Other languages (Java, for one) enforce single inheritance chains, but provide alternate mechanisms (interfaces) for enforcing other common behaviors. We will see interface-like characteristics in our MATLAB code examples.

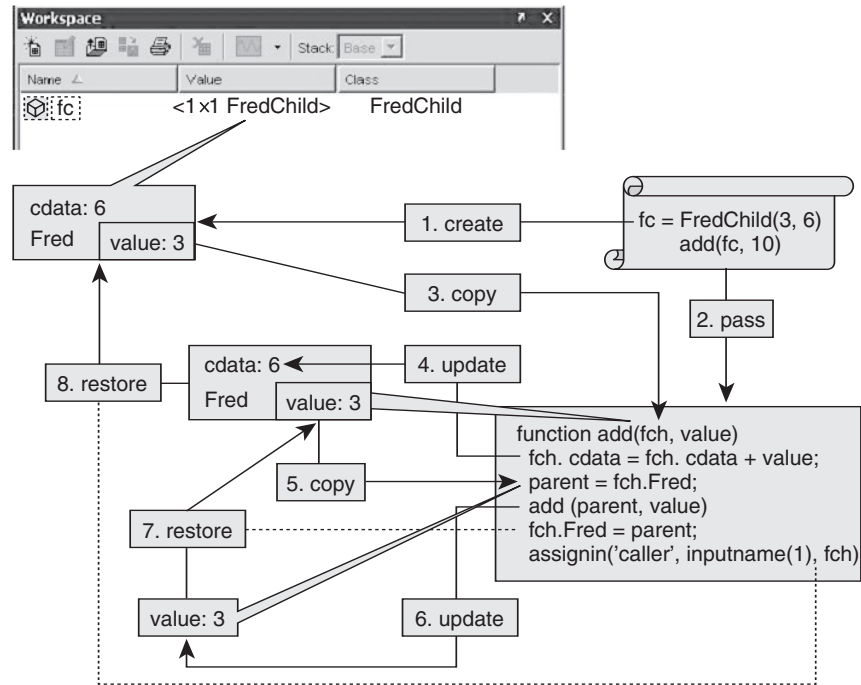


Figure 18.5 Functionality of a child class

shown in Listing 18.4. Note that for illustrative purposes, this method is intended to add the provided data to the attributes of both parent and child. This method might be invoked in the following code:

```
fc = FredChild(3, 6);
add(fc, 10);
```

Figure 18.5 illustrates the sequence of creation and access for these two “simple” operations. The notes below follow the figure callouts.

1. When a script creates an object, a structure whose class name is the name of the child class appears in the workspace. Its attributes are the local data values for the child object and a special attribute with the name of the parent class. That parent class attribute contains the parent data items defined for this object.
2. When a child method is called, since the parameters are passed by value, the numerical parameters are copied in directly.
3. The original object is copied into the workspace of the child method, so the child method is working on a copy of the original object.
4. The child method can directly modify those parts of the attribute structure that belong to that child.

Listing 18.4 child class add method

```

1. function add(fc, value)
2. % @FredChild\add
3. %   add this value to both the child's and
4. %   parent's stored data
5.   fc.cdata = fc.cdata + value;
6.   parent = fc.Fred;
7.   add(parent, value)
8.   fc.Fred = parent;
9.   assignin('caller', inputname(1), fc);

```

5. The child reaches the methods of the parent class via the attribute named for the parent class—in this case, `fc.Fred`. This attribute contains the attributes of the parent object that are defined for this child class. Because this parent structure is an attribute structure, it cannot be directly passed as a parameter to the parent's methods. Rather, we must first extract a copy of the parent's attribute structure.
6. We call the parent method with that copy, which modifies our local copy of the parent attributes.
7. Then, we put the copy of the parent attributes back into our local attribute structure.
8. Finally, since this method is also working with a copy of this object's attributes, these updated attributes must be copied back to the caller.

In Listing 18.4:

Lines 1–4: Show a suitable header block for this method.

Line 5: Adds the value to the child's data attribute.

Line 6: This line needs explanation. We really want to say the following:

```
add(fc.Fred, value)
```

However, this will tunnel back to change the value of `fc.Fred`, and the addressing capability of `assignin(...)` is restricted to elementary variables; it cannot reach structure fields. So we have to extract the parent, add to it, and copy the result back to the parent.

Line 7: Adds to the parent's attribute.

Line 8: Restores the local parent structure.

Line 9: Copies the result back to the caller.

18.3.9 Polymorphism in MATLAB

There are two crucial aspects of **polymorphism**:

1. All objects that are children of a parent class must be treated collectively as if they were all instances of the parent

2. Individually, the system must reach the specific child methods when called for

MATLAB achieves the first very naturally because it ignores the type of all data until forced to operate on that data. The power of this polymorphic approach is this: Throughout MATLAB, all data objects are self-aware—of their data type and the methods they can implement. Therefore, the second objective is achieved because when MATLAB calls a method on a particular object, it goes to the definition of that object for the method implementation.

A simple example might suffice. In the `Fred` class, in addition to the `add` and constructor methods discussed, we implemented a `display(...)` method and a `char(...)` method to show the contents of objects of type `Fred`. In the `FredChild` class there is no `display(...)` method, but there is a `char(...)` method to return the contents of this child and its parent class. Listing 18.5 shows a simple test script for a child class.

In Listing 18.5:

Lines 1 and 2: Show the typical beginning of a script.

Line 3: Shows an additional initializer to remove previous class definitions. If this is not present, for some reason MATLAB balks when you define a class you have previously defined.

Line 4: Creates an instance of the parent and because the semicolon is missing, shows it by calling `display(...)` on the parent class. As shown in Listing 18.6, the `display(...)` method calls `char(...)` on this object (a parent class) that acts exactly like a `char` cast of the object, producing a character string.

Line 5: Creates and displays a child object. Note that this uses the `display` method on the parent because the child does not have one, but calls the `char(...)` method on the child. Note also that the child `char(...)` method invokes the parent's `char(...)` method.

Line 6: Changes the data in both parent and child by adding 10 to them.

Line 7: Displays the child again.

Listings 18.6, 18.7, and 18.8 show the parent `display` method and the `char` methods for the parent and child.

Listing 18.5 Fred child test program

```
1. clear
2. clc
3. clear classes

4. f = Fred(20)
5. fc = FredChild(3, 6)
6. add(fc, 10)
7. fc
```

Listing 18.6 Parent's `display(...)` method

```
1. function display(frd)
2. % @Fred\display method
3. disp(char(frd))
```

In Listing 18.6:

Lines 1 and 2: Show the typical header.

Line 3: Calls the `char(...)` method for whichever object is provided as a parameter, and passes the resulting string to the standard MATLAB `disp(...)` method.

In Listing 18.7:

Lines 1 and 2: Show the typical header.

Line 3: Creates a string using `sprintf(...)`, which incorporates the parent data attribute.

In Listing 18.8:

Lines 1 and 2: Show the typical header.

Line 3: Calls the `char(...)` method of the parent to retrieve that string, and then creates a string using `sprintf(...)`, which incorporates both child and parent data.

Listing 18.9 shows the results of the script in Figure 18.5.

Listing 18.7 Parent's `char(...)` method

```
1. function str = char(frd)
2. % @Fred\char method
3. str = sprintf('Fred with %d', frd.value );
```

Listing 18.8 Child's `char(...)` method

```
1. function str = char(frdc)
2. % @FredChild\char method
3. str = ...
    sprintf('FredChild with %d containing %s', ...
           frdc.cdata, char(frdc.Fred) );
```

Listing 18.9 Results in the Command window

```
1. Fred with 20
2. FredChild with 6 containing Fred with 3
3. FredChild with 16 containing Fred with 13
```



18.4 Example—Modeling Bank Accounts

The overall goal of this section is to show a slightly more practical model than the previous illustrations without having to write too much code, and then illustrate inheritance by extension, and by redefining a small part of the code. In particular, the emphasis will be on designing and gathering functionality in the parent class in order to minimize the amount of content in the child classes.

We will first see the ADT and MATLAB model for a simple bank account class, and then consider two techniques for extending that class—using extension to model a savings account, and then using redefinition to model an overdraft-protected account.

18.4.1 The Base Class

Figure 18.6 illustrates the ADT for a basic `BankAccount` class. While there are a number of attributes of a real account associated with the identity and ownership of that account, we will keep things as simple as possible by considering just the balance on an account as the attribute of interest. The `deposit` and `withdraw` methods provide normal access to the account balance for external users. The `getBalance` and `setBalance` methods provide balance access for the derived classes.

All of the following files will be stored in the folder `@BankAccount` except the test program, which should be in the work directory above the object directories. Listing 18.10 shows the constructor setting up the single attribute, the balance in the account. Listings 18.11, 18.12, 18.13, 18.14, 18.15, and 18.16 show the code for the methods identified in Figure 18.6 together with the standard `display(...)` and `char(...)` methods.

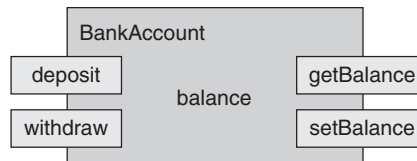


Figure 18.6 ADT for the `BankAccount` class

Listing 18.10 `BankAccount` Constructor

```

1. function acct = BankAccount(data)
2. % @BankAccount\BankAccount class constructor.
3. %   ba = BankAccount(amt) creates a bank account
4. %                               with balance amt
5. % could also be a copy constructor:
6. %   ba = BankAccount( oba ) copies the account oba
  
```

continued on next page

```

7. if nargin == 0
8.     acct.balance = 0;
9.     acct = class(acct, 'BankAccount');
10. elseif isa(data, 'BankAccount')
11.     acct = data;
12. else
13.     acct.balance = data;
14.     acct = class(acct, 'BankAccount');
15. end

```

In Listing 18.10:

Lines 1–6: Show the constructor header.

Lines 7–9: Show the default constructor.

Lines 12–15: Show the real constructor.

In Listing 18.11:

Lines 1 and 2: Show the method header.

Line 3: Invokes the `setBalance(...)` method (Listing 18.14) to add the deposit to the current balance.

Line 4: Returns the new object with the updated balance.

Access to important data like the balance of an account should go through access methods to allow for validation, writing audit trails, and so on to be added in a central location.

In Listing 18.12:

Lines 1 and 2: Show the method header.

Line 3: In a typical account, withdrawals are limited to the amount in

Listing 18.11 The `BankAccount` `deposit` method

```

1. function deposit(acct, amount)
2. % @BankAccount\deposit to the account
3.     setBalance(acct, acct.balance + amount);
4.     assignin('caller', inputname(1), acct);

```

Listing 18.12 The `BankAccount` `withdraw` method

```

1. function gets = withdraw(acct, amount)
2. % @BankAccount\withdraw from the account
3.     gets = amount;
4.     if gets > acct.balance
5.         gets = acct.balance;
6.     end
7.     setBalance(acct, acct.balance - gets);
8.     assignin('caller', inputname(1), acct);

```

Listing 18.13 BankAccount getBalance method

```
1. function ans = getBalance(ba)
2. % @BankAccount\getBalance - get the balance
3. ans = ba.balance;
```

Listing 18.14 BankAccount setBalance method

```
1. function setBalance(acct, amount)
2. % @BankAccount\setBalance - set the balance
3.     acct.balance = amount;
4.     assignin('caller', inputname(1), acct);
```

Listing 18.15 BankAccount display method

```
1. function display(ba)
2. % @BankAccount\disp for the BankAccount class
3. % displays string representation of the account
4. disp(char(ba));
```

the account. Consequently, we must limit the amount returned to the current balance. We initialize what the user gets to what he asked for. Lines 4–6: If this is more money than the user has, give him only the current balance.

Line 7: Sets the new balance.

Line 8: Updates the object.

In Listing 18.13:

Lines 1 and 2: Show the method header.

Line 3: Shows a simple process of extracting the balance from the object. Although we do not need this method in the parent class, child classes do not have access to the parents' attributes and therefore must use this accessor.

In Listing 18.14:

Lines 1 and 2: Show the method header.

Line 3: Sets the balance to the given amount.

Line 4: Returns the object to the caller.

In Listing 18.15:

Lines 1–3: Show the method header.

Line 4: Invokes the `char(...)` method on the object provided. Every object could operate with the same `display(...)` method, if there were some way to accomplish this.

Listing 18.16 BankAccount char method

```

1. function s = char(ba)
2. % @BankAccount\char for the BankAccount class
3. % returns string representation of the account
4. s = sprintf('Account with $%.2f\n', ba.balance );

```

In Listing 18.16:

Lines 1–3: Show the method header.

Line 4: Creates a string representing the attributes of this object. This string should be sufficiently generic to enable its use by children classes.

Listing 18.17 shows a typical script to test the behavior of the BankAccount class.

In Listing 18.17:

Line 1: Creates a bank account with a \$1,000 initial deposit. With no semicolon, the `display(...)` method is called to show the result.

Line 2: Shows the \$20.11 deposit. Although there is no semicolon, this is not an assignment and therefore no output is generated.

Line 3: Shows the `printout` directly using the `char(...)` method to describe the object.

Line 4: Shows the \$200 withdrawal from the account.

Line 5: Demonstrates that the amount was withdrawn and shows the remaining balance.

Listing 18.18 shows the result from running this test script.

Listing 18.17 BankAccount test script

```

1. moola = BankAccount(1000)
2. deposit(moola, 20.11)
3. fprintf('deposit 20.11 -> %s\n', char(moola) );
4. gets = withdraw(moola, 200);
5. fprintf('withdraw 200 -> $%.2f; %s\n', ...
    gets, char(moola) )

```

Listing 18.18 Test results

```

Account with $1000.00
deposit 20.11 -> Account with $1020.11
withdraw 200 -> $200.00; Account with $820.11

```

18.4.2 Inheritance by Extension

Having invested significant effort in preparing the `BankAccount` class to be extended, writing a `SavingsAccount` class involves only the constructor and two methods. The `SavingsAccount` class will have all the characteristics of a `BankAccount`, plus the ability to calculate interest periodically. For simplicity, we will omit any time-sensitive calculations, and presume that the `calcInterest` is run only when appropriate. The interest will accumulate at a specified rate, and will be applied only if the account balance exceeds a given minimum balance.

The `SavingsAccount` class and its parent are shown in Figure 18.7. The only new code needed will be for the constructor (Listing 18.19), the method that calculates the interest (Listing 18.20), and the `char(...)` method to display its content (Listing 18.21). Even that method will invoke the parent `char(...)` method for most of the work.

All the following files except the test script must be stored in the directory `@SavingsAccount`. Notice the following important characteristics:

- The `calcInterest` method does not have to specifically request the `getBalance(...)` method of the parent; because there is no `getBalance` method on this class, the parent supplies it.
- Similarly, the test program merely invokes the `deposit` method. Since the `SavingsAccount` does not have one, the parent `deposit` method is used.
- On the other hand, the `char(...)` method does need to invoke the parent `char(...)` method explicitly in order to avoid recursive behavior.
- Perhaps most challenging, notice that there is no `display(...)` method in this class, so the parent's `display(...)` method is called. However, since it calls a `char(...)` method, we have to ask: Which `char(...)` method is invoked? The answer, of course, is that the `SavingsAccount char(...)` method is used because the object provided is a `SavingsAccount` first and a `BankAccount` second.

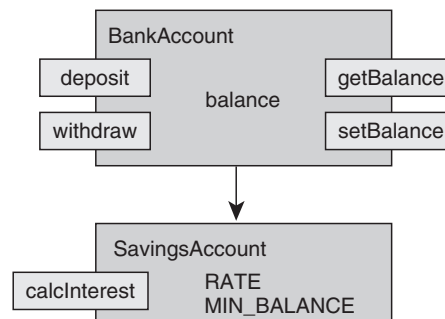


Figure 18.7 *The SavingsAccount class*

Listing 18.19 SavingsAccount constructor

```

1. function acct = SavingsAccount(data)
2. % @SavingsAccount\SavingsAccount constructor.
3. %   ba = SavingsAccount(amt) creates an account
4. %       with balance amt
5. % could also be a copy constructor:
6. %   ba = SavingsAccount( osa ) copies account osa
7. if nargin == 0
8.     super = BankAccount;
9.     acct.RATE = 5;
10.    acct.MIN_BALANCE = 1000;
11.    acct = class(acct, 'SavingsAccount', super);
12. elseif isa(data, 'SavingsAccount')
13.    acct = data;
14. else
15.    super = BankAccount(data);
16.    acct.RATE = 5;
17.    acct.MIN_BALANCE = 1000;
18.    acct = class(acct, 'SavingsAccount', super);
19. end

```

In Listing 18.19:

Lines 1–6: Show the constructor header.

Line 7: Detects the need for the default constructor logic.

Line 8: Creates a default parent object.

Lines 9 and 10: Set the default local attributes (constants in this example).

Line 11: Sets the class of the default object with the parent object included.

Lines 12 and 13: Show the copy constructor.

Lines 15–19: Show the real constructor, which differs only in passing the initial balance to the parent constructor.

Hint

You can make a child class containing no local attributes by creating the parent object and then passing the empty vector to the class cast in line 18 as follows:

```

acct = class(struct([]),
    'SavingsAccount', super);

```

In Listing 18.20:

Lines 1–3: Show the method header.

Listing 18.20 SavingsAccount calcInterest method

```

1. function int = calcInterest(acct)
2. % @SavingsAccount\calcInterest
3.     calculate the interest
4. if getBalance(acct) > acct.MIN_BALANCE
5.     int = acct.RATE * getBalance(acct) / 100;
6. else
7.     int = 0;
8. end

```

Line 4: You can earn interest only if your balance is above the specified minimum balance.

Line 5: Calculates the interest earned during this time period.

Line 7: No interest is earned on small balances.

In Listing 18.21:

Lines 1–3: Show the method header.

Line 4: Invokes the parent `char(...)` method for the balance information; this function adds only the account type. We could, of course, extend this to include the local attributes.

The code shown in Listing 18.22 tests the savings account logic.

In Listing 18.22:

Line 1: Creates a savings account with an initial deposit of \$2,000. This calls the parent `display(...)` method.

Line 2: Invokes the parent's `deposit` method to add another \$3,000.

Line 3: Displays this account information.

Line 4: Invokes our `calcInterest(...)` method.

Line 5: Deposits this interest back into the account.

Line 6: Shows the updated information.

The output from this test is shown in Listing 18.23.

Listing 18.21 `SavingsAccount char(...)` method

```
1. function s = char(sa)
2. % @SavingsAccount\char for the SavingsAccount
3. % returns string representation of the account
4. s=sprintf( 'Savings %s', char(sa.BankAccount) );
```

Listing 18.22 `SavingsAccount` tests

```
1. sa = SavingsAccount(2000)
2. deposit(sa, 3000);
3. fprintf('deposit 3000 -> %s\n', char(sa) );
4. intrst = calcInterest(sa);
5. deposit(sa, intrst);
6. fprintf('deposit interest %.2f -> %s\n', ...
           intrst, char(sa) );
```

Listing 18.23 `SavingsAccount` test results

```
Savings Account with $2000.00
deposit 3000 -> Savings Account with $5000.00
deposit interest 250.00
               -> Savings Account with $5250.00
```

18.4.3 Inheritance by Redefinition

We now consider a further extension of the `BankAccount` family—a `SavingsAccount` with a guaranteed overdraft, as shown in Figure 18.8. Once overdraft privileges have been authorized, users of this account can withdraw as much as they want. Of course, if the resulting balance is negative, the bank applies an overdraft charge, thereby taking away even more money than is available.

The coding for this account will follow the general guidelines used above—a new constructor (Listing 18.24), a method for permitting overdrafts to occur (Listing 18.25), and a new `char(...)` method (Listing 18.26). The data for this class will include a Boolean value indicating that overdraft has been approved for this particular account object. However, this account will also need its own `withdraw` method to implement the new withdrawal rules (Listing 18.27).

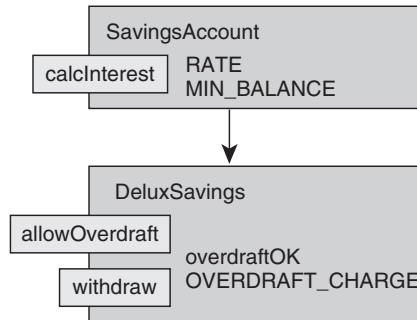


Figure 18.8 The `DeluxSavingsAccount` class

Listing 18.24 `DeluxSavingsAccount` constructor

```

1. function acct = DeluxSavingsAccount(data)
2. % DeluxSavingsAccount class constructor.
3. %   ba = DeluxSavingsAccount(amt) creates an
4. %       account with balance amt
5. % could also be a copy constructor:
6. %   ba = DeluxSavingsAccount( oda ) copies oda
7. if nargin == 0
8.     super = SavingsAccount;
9.     acct.overdraftOK = false;
10.    acct.OVERDRAFT_CHARGE = 20;
11.    acct=class(acct,'DeluxSavingsAccount', super);
12. elseif isa(data,'DeluxSavingsAccount')
13.    acct = data;
14. else
15.    super = SavingsAccount(data);
16.    acct.overdraftOK = false;
17.    acct.OVERDRAFT_CHARGE = 20;
18.    acct=class(acct,'DeluxSavingsAccount', super);
19. end
  
```

In Listing 18.24:

Lines 1–6: Show the constructor header.

Lines 7–11: Show the default constructor defining the local attributes.

Lines 12 and 13: Show the copy constructor.

Lines 15–18: Show the real constructor.

In Listing 18.25:

Lines 1–3: Show the method header.

Line 4: Sets the overdraft permission attribute to the value provided.

Line 5: Returns the updated object.

In Listing 18.26:

Lines 1–3: Show the method header.

Lines 4 and 5: Set the output string to 'OK' or 'off'.

Line 6: Creates the string.

In Listing 18.27:

Lines 3 and 4: Check to see if there is enough money, or if overdraft is allowed.

Lines 5–7: Either way, we can use the parent's withdraw method, but the parent object must be extracted and then replaced.

Line 9: Assumes that the penalty is \$0.

Listing 18.25 DeluxSavingsAccount allowOverdraft method

```

1. function allowOverdraft(acct, value)
2. % @DeluxSavingsAccount\allowOverdraft
3. % approve overdraft on DeluxSavingsAccount
4.     acct.overdraftOK = value;
5.     assignin('caller', inputname(1), acct);

```

Listing 18.26 DeluxSavingsAccount char(...) method

```

1. function s = char(sa)
2. % @DeluxSavingsAccount\char method
3. % returns string representation of the account
4. if sa.overdraftOK, strng = 'OK';
5. else, strng = 'off'; end
6. s = sprintf( 'Delux %s with overdraft %s', ...
               char(sa.SavingsAccount), strng );

```

Listing 18.27 Redefined withdraw method

```

1. function gets = withdraw(acct, amount)
2. % @DeluxSavingsAccount\withdraw method
3.   if (amount < getBalance(acct))
4.       || ~acct.overdraftOK
5.       parent = acct.SavingsAccount;
6.       gets = withdraw(parent, amount);
7.       acct.SavingsAccount = parent;
8.   else
9.       penalty = 0;
10.      if amount > getBalance(acct)
11.          penalty = acct.OVERDRAFT_CHARGE;
12.      end
13.      setBalance(acct,
14.          getBalance(acct) - amount - penalty);
15.   end
16.   assignin('caller', inputname(1), acct);

```

Lines 10–12: However, if the user is asking for more than the balance, the penalty amount applies.

Lines 13 and 14: Update the balance using the parent's `getBalance(...)` and `setBalance(...)` methods.

Line 16: Returns the updated object as usual.

Notice the following observations: Like its predecessor, the `char(...)` method of the `DeluxSavingsAccount` class lets its parent classes generate much of the string, adding locally only the additional information. Also, the redefined `withdraw(...)` method invokes the parent's `withdraw(...)` method if it can, but changes the balance by way of its `get` and `set` methods when it has to. In particular, observe that when it can use the parent `withdraw(...)` method, it only reaches back one level to the `SavingsAccount` parent. The fact that this parent does not implement `withdraw(...)` and passes the call to its parent is not a concern for this child class.

Listing 18.28 shows a script to test the `DeluxSavingsAccount` class.

Listing 18.28 Testing the `DeluxSavingsAccount`

```

1. dsa = DeluxSavingsAccount(2000)
2. deposit(dsa, calcInterest(dsa) )
3. fprintf('deposit interest -> %s\n', char(dsa) );
4. gets = withdraw(dsa, 3000);
5. fprintf('try to get 3000 -> $%8.2f; %s\n', ...
6.     gets, char(dsa) )
7. deposit(dsa, 500);
8. fprintf('deposit 500 -> %s\n', char(dsa) );
9. allowOverdraft(dsa, true)
10. gets = withdraw(dsa, 3000);
11. fprintf(...'enable overdraft and try again -> $%8.2f; %s\n', ...
12.     gets, char(dsa) )

```

In Listing 18.28:

- Line 1: Creates a `DeluxSavingsAccount` object.
- Line 2: Uses the parent's `deposit` and `calcInterest` methods.
- Line 3: Shows the result.
- Line 4: Attempts to withdraw \$3,000 without overdraft enabled.
- Line 5: Shows the result—getting only the balance and an empty account.
- Line 6 and 7: Deposit another \$500 and show the account.
- Line 8: Enables overdrafts.
- Line 9: Tries again for the \$3,000.
- Line 10: Displays the final result.

Listing 18.29 shows the result of testing the `DeluxSavingsAccount` class.

Listing 18.29 Result of testing the `DeluxSavingsAccount`

```
Delux Savings Account with $2000.00
  with overdraft off
deposit interest -> Delux Savings Account with $2100.00
  with overdraft off
try to get 3000 -> $ 2100.00; Delux Savings Account with $0.00
  with overdraft off
deposit 500 -> Delux Savings Account with $500.00
  with overdraft off
enable overdraft and try again -> $ 3000.00; Delux Savings
Account with $-2520.00
  with overdraft OK
```



18.5 Practical Example—Vehicle Modeling

It would be wrong to leave this discussion without an example of a more practical application of these techniques. Figure 18.9 illustrates the inheritance hierarchy for a collection of vehicles that might be used, for example, in a traffic simulation.

18.5.1 A Vehicle Hierarchy

To simulate traffic dynamically, every vehicle must be able to be drawn, and to move in accordance with the rules that govern its behavior. Notice that many of the “simpler” actual vehicles do not contain their own move methods. Given a speed and heading for one of these vehicles, the generic vehicle move method can be applied. However, they all contain their own draw methods, because in general, drawing specific objects requires specific behavior in that draw method.

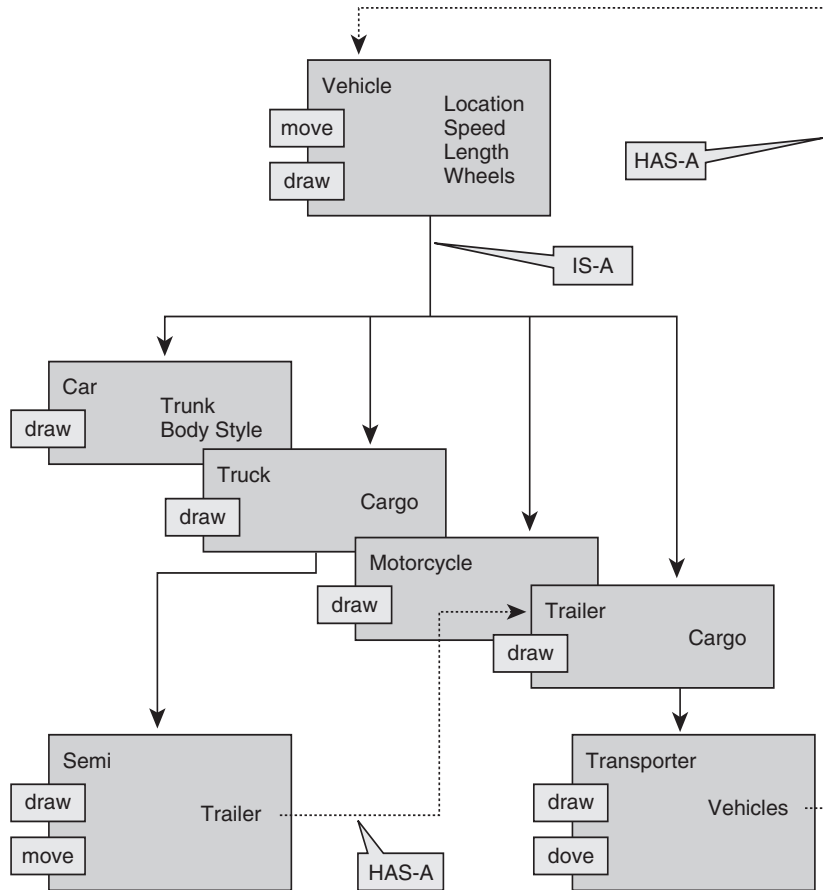


Figure 18.9 A vehicle hierarchy

The more complex vehicles (a semi pulling a trailer, for example) do have their own draw methods, however, because in addition to moving the semi, the trailer must be moved in such a way that it remains attached to the semi. Similarly, a transporter trailer containing other vehicles must move all the other vehicles in a manner consistent with their remaining on the trailer.

18.5.2 The Containment Relationship

There is one other consideration to note. Any practical hierarchy actually reflects two different relationships between classes. We discussed the inheritance (IS-A) relationship in the previous paragraphs—it is all about inheriting data and methods from a parent class, and is implemented in the child constructor. Now we illustrate the other relationship—that of containment (HAS-A), used to indicate a relationship of possession between objects. Two examples follow:

1. A semi class has a trailer attached. There is no inheritance of data or methods between the semi and trailer classes. Rather, one of the attributes peculiar to the semi is a trailer object—that attribute can be accessed by `set` and `get` methods, permitting the semi to move and draw its trailer.
2. A transporter contains a collection of vehicles. Its `move` and `draw` methods must be able to traverse that collection, calling the `move` or `draw` methods for the children as appropriate.



Chapter Summary

This chapter presented the object-oriented paradigm, including class categories and MATLAB implementations:

- Fundamentals of Object-Oriented Programming (OOP)
- How MATLAB implements OOP
- Examples of inheritance and polymorphism
- An example of vehicle modeling

Linked Lists

Chapter Objectives

This chapter discusses:

- Dynamic data structures in general
- A simple dynamic data structure—the linked list
- The MATLAB implementation of a linked list
- Useful implementations of linked lists: queues, stacks, and priority queues

- 19.1** Dynamic Data Structure Concepts
- 19.2** Linked Lists
- 19.3** MATLAB Implementation of Linked Lists
 - 19.3.1 The LLNode Class
 - 19.3.2 The LinkedList Class
- 19.4** Processing Linked Lists Recursively
- 19.5** Implementing Linked List Methods in MATLAB
 - 19.5.1 Building a Linked List
 - 19.5.2 Traversing a Linked List
 - 19.5.3 Mapping a Linked List
 - 19.5.4 Filtering a Linked List
 - 19.5.5 Folding a Linked List
 - 19.5.6 Searching a Linked List
- 19.6** Applications of Linked Lists
 - 19.6.1 Queues
 - 19.6.2 Stacks
 - 19.6.3 Priority Queues

19.1 Dynamic Data Structure Concepts

Having dealt with the concept of modeling specific concrete or abstract objects, we turn to the process of defining dynamically sized collections of objects. The goal is to provide mechanisms that organize collections containing any kind of object—MATLAB basic classes, arrays or structures, cell arrays, or instances of our own classes as defined by the rules discussed in Chapter 18.

We will consider three concepts important to dynamic data structures: static memory allocation on the activation stack, dynamic memory allocation from the heap, and dynamically linking objects to create dynamic structures.

19.1.1 Static Memory Allocation

In Chapter 1 we discussed the allocation of memory to application programs on a processor in general. The first of these approaches is the activation stack (or simply the stack). The stack is used to allocate memory frames that contain data local to a script or function call. Each overall application program (such as MATLAB) is allocated one memory block for use as its stack. Each time a function is called, a frame of memory is allocated from the stack; when the function terminates, that memory is released from the stack as discussed in Chapter 9.

For example, in the middle of a solution to Programming Project 4 in Chapter 5, the activation stack might look like that shown in Figure 19.1. The script containing the variables `u`, `s`, and `g` calls the function `roots` with parameters `A`, `B`, and `C`. In the core of that function, the square root of a value is called. This would be the state of the stack before the `sqrt` function completes. The shaded blocks indicate static data storage on the stack for the variables allocated in the script and functions.

19.1.2 Dynamic Memory Allocation

The second source of memory discussed in Chapter 1 is the heap, which is a single block of memory available to all the applications to be allocated and deallocated from an application upon request. For example, when we created a `BankAccount` object for test purposes in Chapter 18, the command was as follows:

```
moola = BankAccount(1000)
```

sqrt	x =	815.2				
roots	A =	49.05	B =	-40	C =	40
Ex6_1	u =	40	s =	40	g =	9.81

Figure 19.1 An activation stack

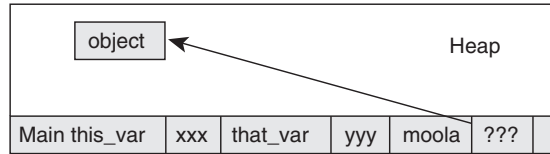


Figure 19.2 *Dynamically allocated memory*

Figure 19.2 illustrates what actually happens when that command is executed:

1. A block of memory large enough to hold the data structure for a `BankAccount` is requested from the heap
2. The data are initialized by the code in the constructor
3. The variable `moola` actually becomes a pointer, or reference, to that block of memory
4. The allocated memory is retained as long as there is a reference somewhere (not necessarily the same one)
5. When the last reference to that memory block is destroyed, the block is returned to the operating system's heap

We cannot leave this concept without considering the “ordinary” variables in Figures 19.1 and 19.2. Although it is convenient to introduce them initially as if they were statically allocated,¹ recall that all MATLAB entities are in fact instances of classes. Even single numbers are 1×1 arrays. Each of the variables illustrated should in fact be viewed in the same style as the variable `moola`, as shown in Figure 19.2.

19.1.3 Dynamically Linked Memory

The power of these structures becomes evident when you consider the “simple” structure shown in Figure 19.3. References to objects on the heap are not confined to stack frames. Dynamically created objects can contain references to other similar, or dissimilar, objects. This chapter will demonstrate the implementation of the simplest of these structures—the linked list.



19.2 Linked Lists

Linked lists are linear data structures implemented exactly as shown in Figure 19.3. Before we examine the details of this implementation, it is important to identify a watershed in the code presentation style used in this text. To this point, we have illustrated examples by showing all the code necessary to

¹ This is true in Java where primitive variables of type `int`, `double`, and `Boolean` are in fact stored on the stack.

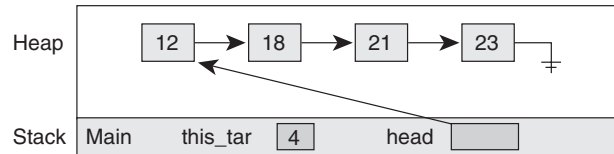


Figure 19.3 *Dynamically linked data*

implement the examples. So far this has been possible because the code has been quite simple, and necessary because there was no good metaphor for summarizing the code. Now the code will be more complex, but we have a metaphor for describing the code more concisely than including all the listings—the ADT.

Here we will review the ground rules under which we will use the ADTs as metaphors for the bulk of the routine code,² freeing us to concentrate on the methods of general interest. Consider one of the two basic classes necessary for implementing linked lists: the `LLNode` class, as shown in Figure 19.4. When we include an ADT in this form, the implementation implied by its presence will be as follows:

- A class with the given name (`LLNode`) stored in the appropriate directory (`@LLNode`)
- Data items with the names and order shown in the ADT
- Access methods with names built as shown, capitalizing the name of the data items
- A constructor expecting initial values of the data items in the order indicated (`data` then `next`); if any of the data items are not provided to the constructor, by default the values will be set to null (`()`)
- Other “expected” methods (`char` or `display` in this example) may be discussed if they have any interesting content
- Other public methods will be identified on the left side of the ADT and the code presented and discussed

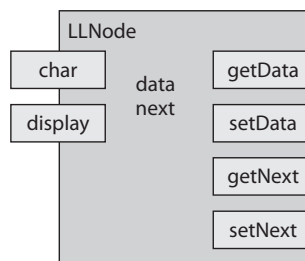


Figure 19.4 *ADT for the `LLNode`*

² The necessary source code is available at the Addison-Wesley Instructor Resource Center (www.aw.com/irc).



19.3 MATLAB Implementation of Linked Lists

Two classes are necessary to implement linked lists. The links themselves are all instances of the `LLNode` class. The container of all the linked list methods is a `LinkedList` class.

19.3.1 The `LLNode` Class

The purpose of the `LLNode` class is to contain references to data items and connect them in a linear list. The data items can be of any type, with two possible constraints. If the data are of type `double`, we have to make a special case in the `char(...)` method. Some methods that process the contents of the linked list may put other requirements on the contained objects.

Hint

When you are debugging code by stepping through it with the debugger, it is easy to become confused about the class to which the current method belongs. You can make life easier in this respect by always putting the class to which this method belongs in the documentation lines. Here, we adopt the convention of specifying the directory containing the method in the second line of the method definition, as shown in Listing 19.1.

Since the `LLNode` class has no interesting methods, according to the ground rules mentioned earlier, we need to include no code listings except the `char(...)` method, as shown in Listing 19.1. Here, we have to make a special case of contents with class `double` because `char(d)` when `d` is type `double` attempts to convert that `double` value to the ASCII code equivalent. If

`d` is of any other type, `char(d)` will convert it to a string as needed.

In Listing 19.1:

- Lines 1–3: Show the typical method header.
- Line 4: Determines whether the node contains a double value.
- Line 5: If so, it uses the `%g` conversion to obtain a general string conversion.
- Line 7: Otherwise, calls the `char(...)` method on the object data.

19.3.2 The `LinkedList` Class

The purpose of the `LinkedList` class is to contain the head of the linked list and all the methods for processing the list. Figure 19.5 shows the ADT of the

Listing 19.1 `LLNode char(...)` method

```
1. function s = char(n)
2. % @LLNode\char(n) is the string representation
3. %           of n.data
4. if isa( n.data, 'double')
5.     s = sprintf('%g', n.data);
6. else
7.     s = char(n.data);
8. end
```

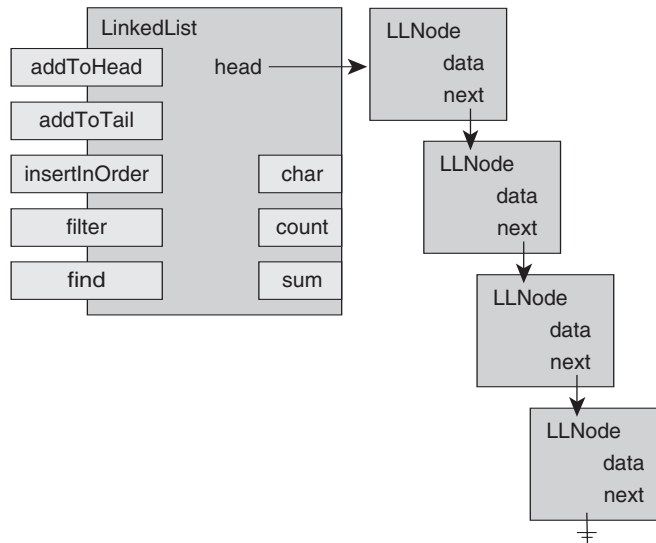


Figure 19.5 ADT for the LinkedList class

LinkedList class and its relationship to the LLNode class. The relationship is one of containment (HAS-A), not inheritance (IS-A). We will consider the methods of the LinkedList class in the context of the taxonomy of collection operations first introduced in Chapter 10.

19.4 Processing Linked Lists Recursively

We could view the definition of a linked list as one of the following:

- A linked list is empty (`[]`)
- A linked list is an LLNode containing some data and another linked list

Since the definition of a linked list is recursive in the same style as $n!$ (see Chapter 9), we could expect that we should be processing the list recursively. Further, we could argue that we could model the template of a function for

Template 19.1 Template for processing a linked list

```

<function and return> processNode(here)
% recursive processing
if isempty(here)
    <reached the end>
else
    ... getData(here) ...
    ... processNode( getNext(here) ) ...
end
  
```

processing a linked list after that same structure, as shown in Template 19.1. See the notes that follow.

- The parameter passed to the recursive function could be named *here* to remind us that the recursive program is moving through the list
- We test for the empty state with MATLAB's built-in function `isempty(...)`
- There will be some actions to take when we reach the end of the list
- Until then, we may do something with the *data* value in the current node, and then recursively process using the *next* value of the node

We might also consider the view shown in Figure 19.6, where processing the whole list is accomplished by repetitively processing the values of a single node:

- Check for the end of the list (*here* is null)
- Make the recursive call via *next*
- Combine the result from the recursive call with the *data* at this node
- Return the result to the calling program

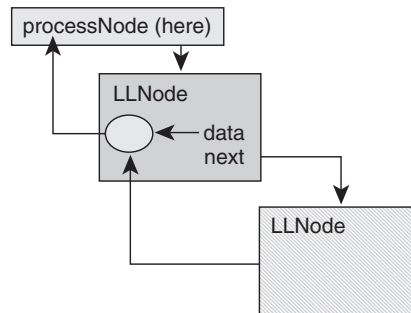


Figure 19.6 *Recursively processing a linked list*



19.5 Implementing Linked List Methods in MATLAB

This section discusses the implementation of the methods of a linked list and some derived classes.

19.5.1 Building a Linked List

There are three techniques for building a linked list. The one you use depends on how you intend the data to be inserted. The most straightforward is adding to the head of the list. However, this presents the last data item added as the first to be seen when traversing or searching the list. Consequently,

we will also consider adding to the tail of the list and inserting in order. While adding to the head or tail of the list imposes no constraints on the nature of the data in the list, adding in order presumes that we have a means for comparing two of the objects being inserted.

Adding to the Head of a List Listing 19.2 shows the method for adding to the head of a list. As long as the user has supplied the parameter `data`, the method creates a new `LLNode` containing the `data` provided and the existing head of the list and makes that the new list head.

In Listing 19.2:

Lines 1–4: Show the method header.

Line 5: Checks that the caller supplied the data.

Line 6: All the logic for adding to the head of a list relates to the head of the list; therefore, recursion is not necessary. The `setHead(...)` and `getHead(...)` methods will be part of the class definition, but not specifically listed here. We create a new `LLNode` containing the `data` provided with the original list as its `next` field, and place this as the new head of the list.

Line 7: Returns the new linked list.

Adding to the Tail of a List Theoretically it is possible to add to the tail of a linked list either iteratively or recursively. Recall the “myopic view” illustrated in Figure 19.6, and then apply that view when adding to the tail of a list. The `addToTail` method in the `LinkedList` class merely passes the head of the list to its recursive local function, `addToTailR`, and stores the result as the new head of the list when it returns. The helper function performs the addition of the data according to the logic described below.

All the processing is done by the recursive module using the myopic view illustrated in Figure 19.6 and Template 19.1. Consider, for example, the existing list shown in Figure 19.7.

If we wanted to add the value 10 to the tail of the list, the first recursive call comes by way of the head of the list to the first node, the one containing

Listing 19.2 Adding to the head of a linked list

```

1. function addToHead(ll, data)
2. % @LinkedList\addToHead.
3. %   addToHead(ll, data) adds data to
4. %       the head of a LinkedList object
5.   if nargin > 1
6.       setHead(ll, LLNode(data, getHead(ll)));
7.       assignin('caller', inputname(1), ll);
8.   end

```

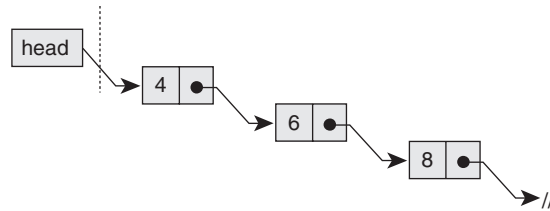


Figure 19.7 The initial list

the value 4 as shown in Figure 19.8. Since we are seeking the end of the list, the recursive calls continue.

Hint

At first glance, this program looks a little strange. You might ask why we do not merely replace the final next field with a new `LLNode` containing the new data. One of the frustrations of working with MATLAB is its insistence on passing parameters by value so that every function and every method is working with a copy of the parameters passed. So it is certainly possible to replace that last next field in a copy of the last node, but not in the node itself. That copy must be returned to the caller and stored, and the circular argument continues back to the head of the list. When you follow that logic to its sad conclusion, you have to replace every node in every dynamic data structure that is changed by the current algorithm.

Finally, the recursive calling reaches the end of the list, as illustrated in Figure 19.9. The action at the end of the list is to create a new node with the data provided (the 10) and null for the next node.

Each recursive call then returns to its caller a copy of its original node linked to the emerging list provided from the recursive call. Finally, as shown in Figure 19.10, the new list fully formed is provided to the wrapper function that stores the new list as its head value.

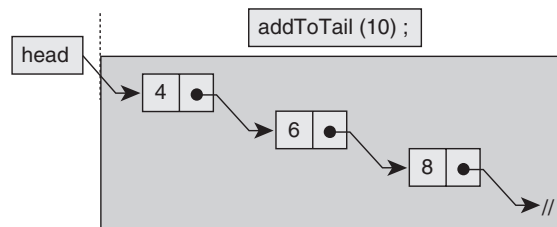


Figure 19.8 The first recursive call

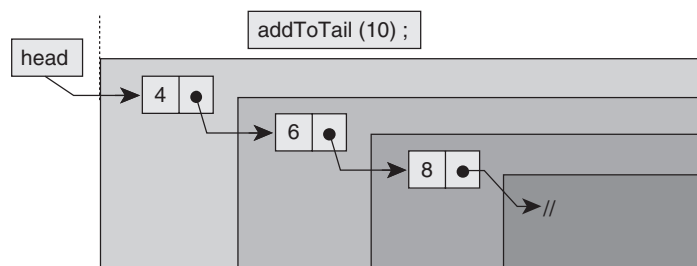


Figure 19.9 At the end of the list

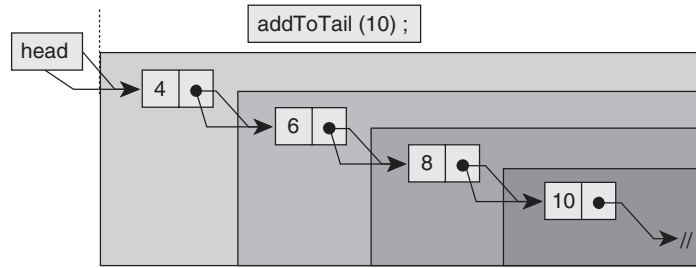


Figure 19.10 *Returning the new list*

Listing 19.3 shows the recursive method for adding to the tail of a linked list.

In Listing 19.3:

Lines 1–4: Show the method header.

Line 5: Makes sure the caller provided the data parameter.

Line 6: Passes the head of the list to the recursive helper function and sets the returned list as the new list head.

Line 7: Returns the new `LinkedList` object to the caller.

Line 9: Shows the recursive helper function. Every exit from this function must return an `LLNode` object.

Line 11: Checks for the terminating condition: the end of the list.

Line 12: Creates the new node for the end of the list.

Lines 14 and 15: Create a new node containing the data from the original node and the result of another recursive call as the next field.

Listing 19.3 Adding to the tail of a linked list

```

1. function addToTail(ll, data)
2. % @LinkedList\addToTail.
3. %   addToTail(ll, data) adds data to the
4. %       tail of a LinkedList object
5. if nargin > 1
6.     setHead( ll, addToTailR(getHead(ll), data) );
7.     assignin('caller', inputname(1), ll);
8. end

9. function nl = addToTailR(here, data)
10. % recursive add to tail
11. if isempty(here)
12.     nl = LLNode(data);
13. else
14.     nl = LLNode( getData(here), ...
15.                 addToTailR(getNext(here), data));
16. end

```

Adding to the List in Order Adding to the list in order is structurally similar to adding to its end, since we must allow for the possibility that the data must be added at its end. It must, however, also allow for finding a link that must come after the new data in the list. In this case, the method builds a new node containing the new data item, with the complete list from here to the end as the next link.

This process also places an additional demand on the data in the list. Whereas any data item can be added to the head or tail of the list, if the list is to be ordered, each data item must be able to respond to the test `A >= B`. Clearly, numbers and characters (including vectors) are equipped to do so. User-defined classes are enabled to respond to this operator by including the method `ge(A, B)`. Listing 19.4 shows the `ge(...)` method that enables a `BankAccount` (and any of its child classes) to be compared to another account, or to a numerical value.

In Listing 19.4:

Lines 1–3: Show the method header.

Line 4: Checks to see if the second parameter is a `BankAccount` object or one of its children.

Lines 6–8: Establish the value for comparing—either the balance of the given account or the data provided (presumed to be a number).

Line 10: Returns the comparison. This is not recursive because the `>=` operator is applied here to numerical values for which this operator is built in.

As with `addToTail(...)`, the `addInOrder(...)` method hands the head of the list to a recursive local function and retrieves the new head of the list. Listing 19.5 shows the method for adding to a list in order. This method also uses a recursive helper function.

In Listing 19.5:

Lines 1–4: Show the method header.

Lines 5–7: Call the recursive helper function and return the result.

Listing 19.4 `ge(...)` operator for the `BankAccount` class

```
1. function ans = ge(acct, data)
2. % @BankAccount\ge
3. % compare this account to another or to a number
4. if isa( data, 'BankAccount')
5.     % children also respond to this
6.     comparison = getBalance(data);
7. else
8.     comparison = data;
9. end
10. ans = (getBalance(acct) >= comparison);
```

Listing 19.5 Linked list addInOrder method

```

1. function addInOrder(ll, data)
2. % @LinkedList\addInOrder.
3. %   ll = addInOrder(ll, data) adds data in order
4. %                               in a LinkedList object
5. if nargin > 1
6.     setHead( ll, addInOrderR(ll.head, data) );
7.     assignin('caller', inputname(1), ll);
8. end

9. function nl = addInOrderR(here, data)
10. % recursive add in order
11. if isempty(here)
12.     nl = LLNode(data);
13. elseif getData(here) >= data
14.     nl = LLNode(data, here);
15. else
16.     nl = LLNode( getData(here), ...
17.                 addInOrderR(getNext(here), data));
17. end

```

Lines 9 and 10: Show the recursive helper function—a minor adaptation of `addtoTail(...)`.

Lines 11 and 12: Show the original terminating condition if this part of the list is empty. Recall that if the second parameter is not supplied to the `LLNode` constructor, its `next` field is presumed to be empty.

Lines 13 and 14: Show the added terminating condition where we discover that the current node contains data that ought to be ahead of the new data in the list. Here we create a new node with the remains of the list represented by `here` as its next field.

Line 16: Otherwise, we continue recursively down the list as before.

19.5.2 Traversing a Linked List

Since a traversal does not change the list, it is possible to perform a list traversal iteratively or recursively. In order to illustrate iterative processing on a list, we will implement the `char(...)` method on a `LinkedList` class iteratively using the `while` loop shown in Template 19.2.

Template 19.2 Iterative `LinkedList` processing template

```

<function and return> processLinkedList(theList)
% iterative processing
<initialize the loop exit test>
<initialize the data result>
while <stay in the loop>
    <process the data>
    <move forward>

```

continued on next page

```
end
<return the results>
```

Listing 19.6 shows the `char(...)` method code that directly matches Template 19.2:

- This function returns a character string `s`.
- The loop will move the variable here through the nodes of the list; its initial value will be the head of the list.
- The initial value of the result string is set to the identifier `'LL: '`.
- The loop continues as long as the `LLNode` reference here is not empty.
- We process the data by appending the `char(...)` conversion of the `LLNode` followed by a semicolon. Note that by asking for the `char(...)` of the `LLNode` rather than its data contents, we make it responsible for always returning a string. This permits the `LinkedList` to contain numbers—usually a problem because `char(n)` where `n` is class `double` will attempt an ASCII conversion of `n`.

In Listing 19.6:

Lines 1–3: Show the method header.

Line 4: Initializes the iterative variable that will move down the list.

Line 5: Initializes the output string.

Line 6: Uses the built-in `isempty(...)` test to terminate the iteration.

Line 7: Invokes the `char(...)` cast of the `LLNode` and concatenates the output with the emerging result.

Line 8: Moves down the list.

19.5.3 Mapping a Linked List

In general, mapping a list is performed not as a general service to all `LinkedList` users, but rather as a specialized utility written for derived classes that uses a `LinkedList` as their parent class. For example, suppose a bank

Listing 19.6 The `LinkedList` `char(...)` method

```
1. function s = char(ll)
2. % @LinkedList\char
3. % char(ll) is its string representation
4. here = getHead(ll);
5. s = 'LL: ';
6. while ~isempty(here)
7.     s = [s char(here) ';'];
8.     here = getNext(here);
9. end
```

keeps all its interest-bearing accounts in a class derived from the `LinkedList` class. Periodically, the bank needs to traverse that list and add the interest generated by each account. Because the length of the list remains unchanged but the contents do change, it is a mapping of the original list.

Listing 19.7 illustrates such an update method.

In Listing 19.7:

Lines 1–4: Show the method header.

Line 5: Computes the interest.

Line 6: Updates the account balance.

Line 7: Returns the new account object.

In order to update all the accounts in a list of `SavingsAccounts`, the mapping would follow the recursive template shown in Template 19.1 with the following tailoring required:

- As usual, the public method, `updateList`, merely launches the recursive helper, `updateR`, with the current head of the list and stores the new list returned as the new head
- The helper returns an empty list when it reaches the end of the old list
- Otherwise, it returns a new `LLNode` containing the result of updating the old data, using the recursive `updateR` call to update the rest of the list

The resulting code is shown in Listing 19.8.

Listing 19.7 `SavingsAccount` `update(...)` method

```
1. function update(acct)
2. % @SavingsAccount\update
3. %   update to the account by depositing
4. %   the interest due
5.   amount = calcInterest(acct);
6.   setBalance(acct, getBalance(acct) + amount);
7.   assignin('caller', inputname(1), acct);
```

Listing 19.8 Mapping a linked list

```
1. function updateList(acctList)
2. % @LinkedList\update
3. %   update to the list items by calling
4. %   its update method.
5.   setHead( acctList, updateR(getHead(acctList)) );
6.   assignin('caller', inputname(1), acctList);
```

continued on next page

```

7. function newLst = updateR(here)
8. % recursive update helper
9. if isempty(here)
10.     newLst = [];
11. else
12.     item = getData(here);
13.     update(item);
14.     newLst = LLNode( item, ...
15.                     updateR( getNext(here) ) );
16. end

```

In Listing 19.8:

Lines 1–4: Show the method header.

Line 5: Invokes the recursive helper function and replaces the list head.

Line 6: Returns the new list object.

Lines 7 and 8: Show the header for the helper function.

Lines 9 and 10: At the end of the list, return empty—we are not adding to the list.

Style Points

It is really important as you build more complex programs to use good abstraction. In the illustration of mapping, it is tempting to incorporate the update function into the `updateList` method because both are rather simple in form. However, the effort that creates separate, single-purpose functions is rewarded in code that is both easy to understand and reusable.

Lines 12 and 13: Extract and update the item. Recall that since the update function tunnels back, it must be called with a simple variable.

Lines 14 and 15: Create a new node with the updated item and the usual recursive call.

19.5.4 Filtering a Linked List

As with mapping a list, filtering can be performed only as a service to specialized collections derived from the `LinkedList` class. The contents of the derived class must provide a method `keep(...)` that determines whether to keep a particular item on the list.

For example, we continue the idea of a list of bank accounts in a `LinkedList`. Since we are not making any use of child account features, we can use any type of `BankAccount` and store the `keep(...)` method in the `BankAccount` class. Perhaps the bank wishes to remove all accounts with a negative balance for this list. They would merely have to provide the `keep(...)` method shown in Listing 19.9 and then implement the generic `LinkedList` filter.

Listing 19.9 `BankAccount keep(...)` method

```

1. function ans = keep(acct)
2. % @BankAccount\keep

```

continued on next page

```

3. %    keep an account with a non-negative balance
4.      ans = getbalance(acct) >= 0;

```

In Listing 19.9:

Lines 1–3: Show the method header.

Line 4: Shows the logic for keeping an account on the list. In more complex situations, this test can be tailored to a specific set of requirements.

To filter items in a generic list, we will again follow the recursive template shown in Template 19.1 with the following tailoring required:

- As usual, the public method, `filter`, launches the recursive helper, `filterR`, with the current head of the list and stores the new list returned
- The helper returns an empty list when it reaches the end of the old list
- Otherwise, if it should keep this data item, it returns a new `LLNode` containing the old data, using the recursive `updateR` call to update the rest of the list
- If it should not keep this data item, it returns the result from applying `updateR` recursively to the rest of the list

The resulting code is shown in Listing 19.10.

In Listing 19.10:

Lines 1–6: Show the usual wrapper function to set up the recursion.

Lines 7–9: Show the filtering helper function header.

Listing 19.10 Generic `LinkedList` `filter` method

```

1. function filter(ll)
2. % @LinkedList\filter
3. %    filter the list by removing all items
4. %    returning false from their keep method
5.     setHead( ll, filterR(getHead(ll)) );
6.     assignin('caller', inputname(1), ll);

7. function newLst = filterR(here)
8. % recursive filter helper
9. if isempty(here)
10.     newLst = [];
11. elseif keep(getData(here))
12.     newLst = LLNode( getData(here), ...
                        filterR( getNext(here) ) );
13. else
14.     newLst = filterR( getNext(here) );
15. end

```

Lines 9 and 10: Show the terminating condition returning empty.

Line 11: Checks whether we need to keep this item.

Lines 11 and 12: If so, we make a new node with the current item and the recursive call.

Line 14: Otherwise, we delete the current item by returning only the result of the recursive call without creating a new node.

19.5.5 Folding a Linked List

Unlike the previous list operations, folding can be performed as a service in the `LinkedList` class. We illustrate this by the `sum(...)` method. It constrains the contents of the list to those classes that respond to the method `plus(...)`. In general, `plus(...)` permits the programmer to add the current object to any other object. We will assume here that if the second object is not the same class as ours, it is a `double` value.

We continue the idea of a list of bank accounts in a `LinkedList`. Since we are not making any use of child account features, we can use any type of `BankAccount` and store the `plus(...)` method in the `BankAccount` class. Perhaps the bank wishes to sum all accounts in this list. They would merely have to provide the `plus(...)` method shown in Listing 19.11, and then implement the generic `LinkedList` `sum`.

Hint

We have used two specific examples of a general principle. The statement `'A >= B'` results in MATLAB calling `ge(A, B)`, and `'A + B'` results in a call to `plus(A, B)`. In general, all MATLAB operators have a “shadow” function that is actually called when the operator is applied. See MATLAB help for a full listing of these functions.

In Listing 19.11:

Lines 1–5: Show the method header.

Line 6: Checks whether the addend is a `BankAccount` (or child thereof).

Line 7: If so, extracts its balance.

Line 9: Otherwise, assumes the data item is a number.

Line 11: Returns the required sum.

Listing 19.11 `BankAccount` `plus(...)` method

```
1. function ans = plus(ba, item)
2. % @BankAccount\plus
3. % add this bank account to another item:
4. %   either another BankAccount, or something
5. %   else that can be added to a double
6. if isa(item, 'BankAccount')
7.     addend = getBalance(item);
8. else
9.     addend = item;
10. end
11. ans = addend + getBalance(ba);
```


To sum items in a generic list, we will again follow the recursive template shown in Template 19.1 with the following tailoring required:

- As usual, the public method, `sum`, merely launches the recursive helper, `sumR`, with the current head of the list and passes the total on to its caller.
- The helper returns 0 when it reaches the end of the old list.
- Otherwise, it returns the result from adding the current data value to the result of applying `sumR` recursively to the rest of the list.

The resulting code is shown in Listing 19.12.

In Listing 19.12:

Lines 1–3: Show the method header.

Line 4: Invokes the recursive sum and returns the result.

Lines 5 and 6: Show the header for the recursive helper.

Lines 7 and 8: Show the terminating condition—return zero when there are no nodes.

Lines 10 and 11: We should spend a line or two on the stunning elegance of this apparently simple statement. MATLAB is set up so that whenever the `+` operator is invoked, it just calls the `plus(...)` method of the first item with the second operand as the second parameter. Since `sumR(...)` always returns a number, this line of code will result in calling the `plus` method on whatever item is in the list with the current sum as the second parameter. If the list is a list of `BankAccount` objects, the `BankAccount plus(...)` method will be invoked. If it is a list of numbers, the built-in `plus` method for doubles will be called. This same `sum(...)` method could be applied to a linked list containing vectors as long as they all had the same length!

Listing 19.13 is a script to test the filtering and folding functions. Note that as far as outside appearances are concerned, filtering is done “in place” in the

Listing 19.12 Generic `LinkedList` `sum` method

```

1. function total = sum( list )
2. % @LinkedList\sum
3. % total the items in a list
4. total = sumR( getHead(list) );

5. function sum = sumR(here)
6. % recursive list adder
7. if isempty(here)
8.     sum = 0;
9. else
10.     sum = getData(here)...
11.         + sumR(getNext(here));
12. end

```

`LinkedList` class. So to keep a copy of all the accounts before applying the filter, we use the copy constructor as follows:

```
goodAccts = LinkedList(BAList);
```

In Listing 19.13:

Line 1: Creates an empty list.

Lines 2–4: Add three random accounts.

Lines 5–8: Add an overdrawn `DeluxSavingsAccount`.

Line 9: Uses the `char(...)` method to print the complete list.

Line 10: Makes a copy of the original account list.

Line 11: Filters the accounts.

Line 12: Uses the `LinkedList char(...)` method to display the filtered list.

Line 13: Folds the original list with the `sum(...)` method.

Listing 19.14 shows the result of this test.

A Powerful Thought Notice that the total of the original list is in fact the total of the four original account balances. This draws attention to a powerful

Listing 19.13 Testing the `filter` function

```
1. BAList = LinkedList;
2. addToHead(BAList, BankAccount)
3. addToHead(BAList, BankAccount(-1000))
4. addToHead(BAList, SavingsAccount(2000))
5. dsa = DeluxSavingsAccount(2000);
6. allowOverdraft(dsa, true);
7. gets = withdraw(dsa, 3000);
8. addToHead(BAList, dsa)
9. fprintf('accounts: %s\n', char(BAList) )
10. goodAccts = LinkedList(BAList);
11. filter(goodAccts);
12. fprintf('\ngood accounts: %s\n',char(goodAccts) )
13. total = sum(BAList)
```

Listing 19.14 Result of testing the `filter` function

```
accounts: Delux Savings Account with $-1020.00
with overdraft OK
Savings Account with $2000.00
Account with $-1000.00
Account with $0.00
good accounts: Savings Account with $2000.00
Account with $0.00
total =
-20
```

idea. We should ask whether it is sufficient for the `LinkedList` copy constructor to do as all previous copy constructors have done and return just a copy of the object provided. Follow this thought chain:

First Answer: “Oops, no—you cannot just return a copy of the `LinkedList` object, because it contains a reference to a chain of `LLNodes` in memory. If you just copy the head of that chain, you will end up with two `LinkedList` objects pointing to the same `LLNode` chain. If you modify either of these `LinkedList` objects, they will both have their `LLNode` chains changed.” True or False?

Second Answer: “Oops, no—False. Whenever you change the `LLNode` chain, MATLAB makes you copy it anyway, so it is actually perfectly safe to copy just the original list head. Whoever changes that chain gets back a new chain, not a modification of the original.”

Sad Reality: The current implementation of the `LinkedList` copy constructor actually does the deep copy by building a new `LLNode` chain.³

19.5.6 Searching a Linked List

Unlike the previous list operations, searching can be performed as a service in the `LinkedList` class. We illustrate this by the `find(...)` method. It constrains the contents of the list to those classes that respond to the `eq(...)` method. The `eq(...)` method is invoked whenever a program uses the `'=='` operator, and can be applied to primitive data types and added to all more complex classes. The user of this operation is required only to state how a user class will determine whether an object of that class responds to the equality test for some data. Bank account equality could be determined in general by account number, telephone number, and so on. In our simple example, we will look for a number in a list of numbers.

To find an item in a list, we will again follow the recursive template shown in Template 19.1 with the following tailoring required:

- As usual, the public method, `find`, launches the recursive helper, `findR`, with the current head of the list and the item to be found, `item`. When the recursive call completes, it passes the result to its caller.
- The helper returns `[]` when it reaches the end of the old list.
- Otherwise, if the item sought matches this item in the list, it returns that item.
- Otherwise, it returns the result from finding the item on the rest of the list, using `findR` recursively.

The resulting code is shown in Listing 19.15.

³ If you can explain this to your instructor, you deserve an A for the class and to be exempted from all future assignments.

Listing 19.15 LinkedList find method

```

1. function result = find( list, item )
2. %
3. % find an item on a list
4. result = findR( getHead(list), item );

5. function result = findR( here, item )
6. % recursive list adder
7. if isempty( here )
8.     result = [];
9. elseif getData( here ) == item
10.    result = getData( here );
11. else
12.    result = findR( getNext( here ), item );
13. end

```

In Listing 19.15:

Lines 1–3: Show the method header as a wrapper function.

Line 4: Returns the result of the recursive call. Rather than returning `true` or `false`, this function should do the more useful thing of returning *a copy* of the object found or `empty` if it is not there. (You cannot do what you really want to do and return a reference to the object actually in the list.)

Lines 5 and 6: Show the header for the recursive helper.

Lines 7 and 8: Show the terminating condition—if you reach the end of the list, return `empty`.

Line 9: Shows another elegant situation. This calls the `eq(...)` method on whatever item is in the list. In our test case, this is a number with a built-in `eq(...)` method, but exactly the same code will also call the `eq(...)` method on user-defined classes.

Line 10: Shows the object found.

Line 12: If not found, continues with the recursive calls.

Listing 19.16 shows the code to test the `find` method. It populates a `LinkedList` with random numbers between 0 and 19, and then checks to see if the value 10 is on the list. Running it repeatedly will verify that the `find` method is operating correctly.

Listing 19.16 Testing the find method

```

1. lst = LinkedList;
2. for it = 1:20
3.     num = floor( 20*rand );
4.     addToHead( lst, num );
5. end

```

continued on next page

```

6. fprintf('list is %s\n', char(lst));
7. ans = find(lst, 10);
8. if isempty(ans)
9.     ans = -1;
10. end
11. fprintf('find 10 returned %d\n', ans );

```

In Listing 19.16:

Line 1: Makes a new linked list.

Lines 2–5: Add 20 random integers.

Line 6: Displays the list.

Line 7: Finds the value 10 on the list.

Lines 8–10: Detect the failure case to avoid problems with `fprintf`.

Line 11: Shows the result.



19.6 Applications of Linked Lists

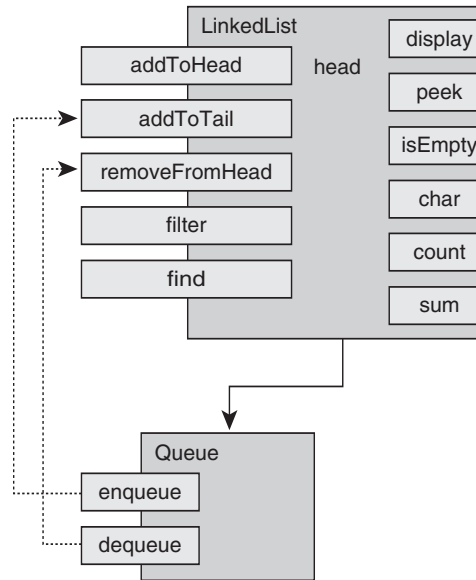
Aside from being a good introductory example of dynamic data structures, there are three useful children classes of the `LinkedList` class—the `Queue`, the `Stack`, and the `PriorityQueue`. In Chapter 17 we saw that two of these classes, the `Queue` and the `PriorityQueue`, are necessary to the algorithms for searching for paths in a graph. We might ask what is wrong with the queue and priority queue implementations that used cell arrays to store the queue data. There are two answers: one is weak because of the MATLAB OO implementation, but the other is very strong.

1. The slightly weak argument is that the efficiency of every cell array operation—adding to the head or tail or removing from the head—is always $O(N)$ because the cell array must be copied. The efficiency of removing from the head of a linked list is $O(1)$, and adding to the tail is also $O(1)$ in other implementations that use references, such as Java.
2. The stronger argument is that in the OO implementation, the data are completely encapsulated. The user has no access at all to the data except by way of the methods made public to the user.

The following sections describe the OO implementation of the `Queue`, `MStack`, and `PriorityQueue` classes. Unfortunately, although MATLAB does not currently implement the `Queue` and `PriorityQueue` classes, it does have an internal `Stack` class that forces us to change the name of our stack implementation.

19.6.1 Queues

Here we begin to see some concrete return on the investment made to create the `LinkedList` class. Once that `LinkedList` class has been created with all

**Figure 19.11** Queue implementation

the data support methods likely to be required, a `Queue` class can quickly be derived as a child class, as shown in Figure 19.11. All we need to do is implement the child class framework and two methods, each of which invokes the appropriate parent methods. To `enqueue`, we need to add to the tail of the list, and to `dequeue`, we remove from the head of the list.

The complete code for the `Queue` class is therefore its standard infrastructure plus the simple `enqueue(...)` and `dequeue(...)` methods, as shown in Listings 19.17 and 19.18. Since the method `peek(...)` is appropriate for all three child classes, this method (Listing 19.19) was added to the `LinkedList` class.

Listing 19.17 Queue enqueue method

```
function enqueue(q, data)
% @Queue\enqueue
% enqueue onto a queue
    addToTail(q, data);
    assignin('caller', inputname(1), q);
```

Listing 19.18 Queue dequeue method

```
function ans = dequeue(q)
% @Queue\dequeue
% dequeue from a Queue
    ans = removeFromHead(q);
    assignin('caller', inputname(1), q);
```

Listing 19.19 LinkedList peek method

```
function ans = peek(lst)
% @LinkedList\peek
% peek at the head of the list
    ans = getData(getHead(lst));
```

Listing 19.20 Testing the queue class

```
1. q = Queue;
2. fprintf('is queue empty? %d\n', isEmpty(q) );
3. for ix = 1:10
4.     enqueue(q, ix);
5. end
6. fprintf('is queue empty? %d\n', isEmpty(q) );
7. q
8. fprintf('dequeue -> %d leaving %s\n', ...
    dequeue(q), char(q) );
9. fprintf('peek at queue -> %d leaving %s\n', ...
    peek(q), char(q) );
```

Listing 19.20 shows the code to test the queue class.

In Listing 19.20:

Line 1: Creates an empty queue.

Line 2: Verifies that it is empty.

Lines 3–5: Put the numbers 1:10 into the queue.

Line 6: Verifies that the queue is not empty.

Line 7: Forces its display method to show us the queue.

Line 8: Shows that the first item into the queue was the first item out.

Line 9: Shows that the second item would be the next out, but is not removed by the peek(...) method.

Listing 19.21 shows the result of this test.

19.6.2 Stacks

Once we have a working `LinkedList` class, implementing a stack is also trivial, as illustrated in Figure 19.12. The `push(...)` method merely calls

Listing 19.21 Results from testing the queue class

```
is queue empty? 1
is queue empty? 0
Queue: 1;2;3;4;5;6;7;8;9;10;
dequeue -> 1 leaving Queue: 2;3;4;5;6;7;8;9;10;
peek at queue -> 2 leaving Queue: 2;3;4;5;6;7;8;9;10;
```

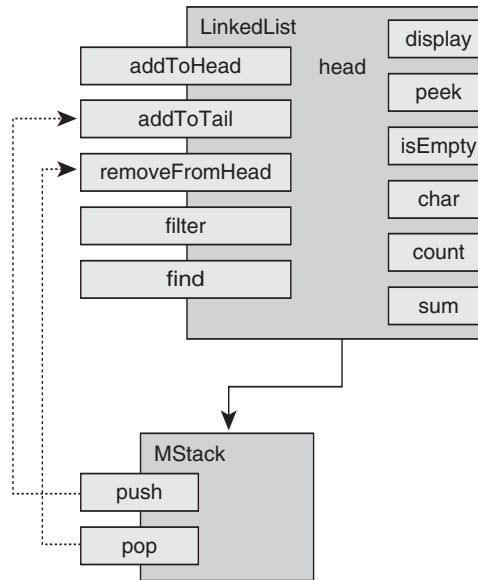


Figure 19.12 MStack implementation

`addToHead(...)`, and the `pop(...)` method calls `removeFromHead(...)`. As previously noted, we need to name our stack class something other than `stack` to avoid the MATLAB name conflict.

Listings 19.22 and 19.23 show the `push(...)` and `pop(...)` methods without the need for further explanation.

Listing 19.24 shows the code to test the `MStack` class, which is intended to follow on from the `Queue` class test script.

Listing 19.22 The Stack `push` method

```
function push(stk, data)
% @MStack\push
% push onto a stack
    addToHead(stk, data);
    assignin('caller', inputname(1), stk);
```

Listing 19.23 The Stack `pop` method

```
function ans = pop(stk)
% @MStack\pop
% pop from a stack
    ans = removeFromHead(stk);
    assignin('caller', inputname(1), 'stk');
```


Listing 19.24 Testing the MStack class

```

1. stk = MStack;
2. fprintf('is stack empty? %d\n', isEmpty(stk) );
3. for ix = 1:10
4.     push(stk, ix);
5. end
6. fprintf('is stack empty? %d\n', isEmpty(stk) );
7. stk
8. fprintf('pop from stack -> %d leaving %s\n', ...
    pop(stk), char(stk) );
9. fprintf('peek stack -> %d leaving %s\n', ...
    peek(stk), char(stk) );
19. while ~isEmpty(q)
20.     push(stk, dequeue(q));
21. end
22. fprintf('stack with whole queue is %s\n', ...
    char(stk) );

```

In Listing 19.24:

Line 1: Creates an MStack object.

Line 2: Checks that the empty test works.

Lines 3–5: Push the numbers 1 to 10.

Line 6: Checks that the stack is not empty.

Line 7: Forces the display(...) method to show the stack contents.

Line 8: The first out should be the last item in: the value 10, leaving the other 9 numbers on the stack.

Line 9: peek(...) should show the next out, but keep it on the stack.

Lines 19–21: Empty the queue (Listing 19.20) into the stack.

Line 22: Shows the whole stack with the queue coming out first in reverse order.

Listing 19.25 shows the results of this test.

19.6.3 Priority Queues

It is not surprising that implementing a priority queue can be accomplished by extending the Queue class, and invoking a different LinkedList method

Listing 19.25 Results from testing the MStack class

```

is stack empty? 1
is stack empty? 0
MStack: 10;9;8;7;6;5;4;3;2;1;
pop from stack -> 10 leaving MStack:
    9;8;7;6;5;4;3;2;1;
peek stack -> 9 leaving MStack: 9;8;7;6;5;4;3;2;1;
stack with whole queue is MStack:
    10;9;8;7;6;5;4;3;2;9;8;7;6;5;4;3;2;1;

```

for the `PriorityQueue enqueue(...)` method. This is an excellent example of extension by redefinition—allowing the parent classes to provide all the infrastructure and operational requirements except the one specific new capability required. Figure 19.13 illustrates this implementation, and Listing 19.26 illustrates the one method required.

Listing 19.27 shows a suitable script for testing the `PriorityQueue` class.

In Listing 19.27:

Line 1: Creates a priority queue.

Lines 2–6: Generate 10 random numbers between 1 and 100, display them, and enqueue them on the priority queue.

Line 7: Displays the priority queue.

Listing 19.28 shows the results from running this test script. Notice that the values are in priority order.

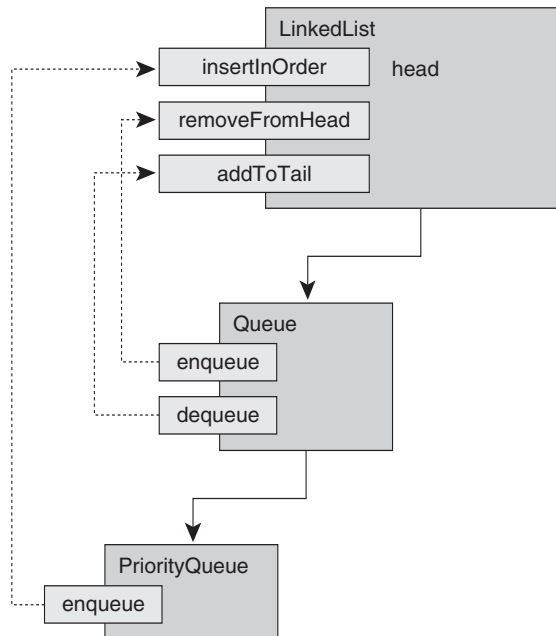


Figure 19.13 *PriorityQueue implementation*

Listing 19.26 `PriorityQueue enqueue` method

```
function enqueue(pq, data)
% @PriorityQueue\enqueue
% enqueue onto a queue
addInOrder(pq, data);
assignin('caller', inputname(1), pq);
```

Listing 19.27 PriorityQueue test script

```

1. pq = PriorityQueue;
2. for ix = 1:10
3.     value = floor(100*rand);
4.     fprintf(' %g:', value );
5.     enqueue(pq, value );
6. end
7. fprintf('\npriority queue is %s\n', char(pq) );

```

Listing 19.28 PriorityQueue test results

```

82: 91: 11: 81: 90: 15: 12: 76: 72: 65:
priority queue is PriorityQueue:
      11;12;15;65;72;76;81;82;90;91;

```

**Chapter Summary**

This chapter discussed:

- The overall concept of dynamic data structures
- The linked list as a specific structure
- The MATLAB implementation of a linked list
- Queues, stacks, and priority queues as useful applications created by extending the `LinkedList` class

**Programming Projects**

1. Write and test a method that copies all the entries from a `LinkedList` into a cell array.
 - a. What kind of process is this? (Folding? Mapping?)
 - b. Is this restricting the nature of the data in any way?
2. Write another method to copy the data, inserting each item *in order* into the cell array.
 - a. Is this now restricting the nature of the data in any way?
3. Rewrite the `char(...)` method for the `LinkedList` class in recursive form. Call it `rchar(...)` and test it thoroughly.
4. Rewrite the recursive `rchar(...)` method for the `LinkedList` class, concatenating the recursive result before the local data instead of after it. Test this version thoroughly. What do you learn about the ordering of the data?

5. Write and test a method to find the number of items in a `LinkedList`.
6. Write and test a method to find the average of the items in a `LinkedList`.
7. Write and test a method to find the bank account with the largest balance in a `LinkedList` of `BankAccounts`.
8. Write an iterative version of `find(...)` for the `LinkedList` class using a `while` loop.
9. Write an iterative version of `sum(...)` for the `LinkedList` class using a `while` loop.
10. Write the class `ArrayQueue` that conforms to the `Queue` ADT, but is implemented using a cell array instead of inheriting from the `LinkedList` class.
11. Write the class `ArrayStack` that conforms to the `Stack` ADT, but is implemented using a cell array instead of inheriting from the `LinkedList` class.
12. Attempt to write an iterative version of `addToTail` for a `LinkedList` class using a `while` loop to reach the end of the list.

Hint

Don't spend too long trying—it probably cannot be done. What inherent MATLAB behavior makes this impossible?

Binary Trees

Chapter Objectives

This chapter discusses:

- Dynamic data structure concepts
- The nature of a binary tree and its derivative—the binary search tree (BST)
- How to process binary trees in general
- Those operations defined for general data collections that are relevant to binary trees or BSTs

Introduction

We move to a slightly more complex dynamic data structure—the binary tree. Where each node of a linked list contains only one child node, a binary tree has two child nodes, normally referred to as the left and right nodes, as shown in Figure 20.1.

Within this structure, we need to consider two cases: a binary tree, where the data in the tree exhibit no apparent organization, and a binary search tree, where the data are organized for efficient searching.

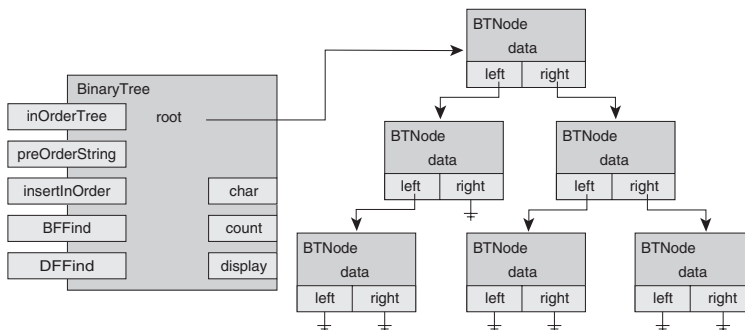


Figure 20.1 Structure of a binary tree

- 20.1** Dynamic Data Structure Concepts
 - 20.1.1 Graphs
 - 20.1.2 Paths
 - 20.1.3 Directed Acyclic Graphs
 - 20.1.4 Roots
 - 20.1.5 Leaves
 - 20.1.6 N-ary Trees
 - 20.1.7 Binary Trees
 - 20.1.8 Binary Search Trees
- 20.2** Processing Binary Trees
- 20.3** Processing Binary Search Trees
- 20.4** Traversing a Binary Tree
 - 20.4.1 Two Depth-First Traversals
 - 20.4.2 Breadth-First Traversal
- 20.5** Building a Binary Search Tree
- 20.6** Mapping a Binary Tree
- 20.7** Deleting Nodes from a Binary Search Tree
- 20.8** Folding a Binary Tree
- 20.9** Searching Binary Trees
 - 20.9.1 Breadth-First Search
 - 20.9.2 Depth-First Search
 - 20.9.3 Searching a Binary Search Tree
 - 20.9.4 Testing Binary Search Tree Functionality
- 20.10** Combining Complex Operations



20.1 Dynamic Data Structure Concepts

20.1.1 Graphs

The most general form of dynamic data structure is a graph—an arbitrary collection of nodes connected by edges. A street map is a good example of a graph, where the intersections are the nodes and the streets are the edges. The edges may be directional to indicate that the graph can be traversed at that point in only one direction (like a one-way street). The edges may also have a value associated with them to indicate, for example, the cost of traversing that edge. For a street map, this could be either the distance or, perhaps in a more sophisticated system, the travel time—a function of the distance, the speed limit, and the traffic congestion. Graphs are not required to be completely connected, and they can obviously contain cycles—closed loops in which the unwary algorithm could get trapped. Graphs also have no obvious starting and stopping points.

20.1.2 Paths

A path on a graph is a connected list of edges that is the result of traversing a graph.

20.1.3 Directed Acyclic Graphs

On a graph, if we place the constraint that no path is permitted to wrap around and complete a cycle but child nodes can still have multiple parents, we have a directed, acyclic graph (DAG). These graphs are conceptually simpler to process than full graphs because they must have at least one node at which paths will originate.

20.1.4 Roots

No cycles are permitted in; therefore, there must be at least one starting point like there is for the head of a linked list. We refer to that starting point as the root.

20.1.5 Leaves

Just as there is one node that begins the tree, in general there will be many nodes that terminate the traversal of a tree because nodes have no child links. We refer to these nodes as the leaves. A linked list, by contrast, has only one leaf.

20.1.6 N-ary Trees

While a DAG is less complex to process than a graph, we must still apply another simplification. A tree is best thought of as a DAG in which nodes may have only one parent. A good example would be a family tree tracing

the descendants from a common ancestor. In N-ary trees each node can contain an arbitrary number of links to other nodes.

20.1.7 Binary Trees

We make one more simplification before we can start analyzing the behavior of trees—it is convenient at the beginning to restrict the number of child nodes to two. This greatly reduces the amount of logic required to traverse a tree. An ancestral tree—tracing from a child back up through its lineage—is a good example of a binary tree.

20.1.8 Binary Search Trees

A binary search tree (BST) is an important specialization of a binary tree in which the data are ordered specifically to facilitate searching for information in the tree. We will discuss the impact of that ordering later. However, now is a good time to point out that if the data items in the tree are complex, one cannot order the data in the tree on all the components of the data. For example, it might be useful to organize student objects in a BST ordered by student IDs. As we will see, this greatly speeds the process of searching by ID for a student, but is not helpful when searching for other criteria, such as names or telephone numbers.



20.2 Processing Binary Trees in General

When discussing linked lists, we discovered that there are occasions in which one could iteratively process the list, especially if the list is not being changed. There are similar opportunities once we start processing graphs or trees. However, we first consider processing binary trees recursively. Figure 20.2 illustrates the content of the node with which we can construct a binary tree. Omitted for clarity are the `set/get` methods for the `data`, `left`, and `right` attributes.

We determine how to process a binary tree based on its formal definition:

- A binary tree is empty (`[]`)

or

- A `BTNode` contains a data item and two binary trees named `left` and `right`

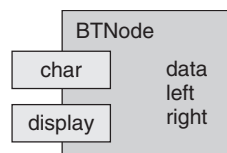


Figure 20.2 A binary tree node

Template 20.1 Template for processing a binary tree

```

<function and return> processBTNode(here)
% recursive processing
if isempty(here)
    <reached the end>
else
    ... getData(here) ...
    ... processBTNode( getLeft(here) ) ...
    ... processBTNode( getRight(here) ) ...
end

```

Further, we could argue that we could model the template of a function for processing a binary tree after this definition, as shown in Template 20.1.

Figure 20.3 illustrates the operation of this recursive process on a particular `BTNode`. Just as with processing linked lists, the accomplishments of a recursive operation on a binary tree are governed by the logic implemented in the recursive call. In general, processing a node involves a recursive call to process the left child, a recursive call to process the right child, and then some logic to combine these two results with the data in this particular node. The results are then returned to the calling node.

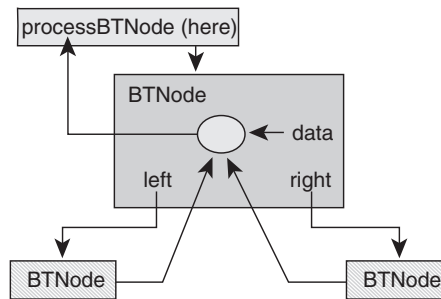


Figure 20.3 *Processing a binary tree*

20.3 Processing Binary Search Trees

The definition of a binary search tree (BST) is as follows:

- A BST is either empty (`[]`)
- or
- A `BTNode` contains a data item and two BSTs named `left` and `right` where all the data in the left subtree are less¹ than the data at this node, and all the data in the right subtree are greater than this node

¹ This restricts the nature of the data items to those that can respond to `gt(...)`, `lt(...)`, and `eq(...)`.

- Equal data items are not permitted in BSTs^{2 3}

Notice that this is a recursive definition so that the ordering requirement continues throughout the BST.

The template and diagram for processing a BST is identical to that shown in Template 20.1 and Figure 20.3. As we see specific applications processing these trees, there are times when we can take advantage of the ordering of the data to reduce the amount of processing.

In each of the following sections, some of the discussions may apply only to one type of tree or the other. The script that tests these operations is shown in Section 20.9.4.



20.4 Traversing a Binary Tree

We will consider three different techniques for traversing a binary tree. Two are depth-first traversals—so-called because each traversal reaches recursively down paths to the leaves of the tree. The other is a breadth-first traversal that is performed iteratively and travels across the tree generations.

20.4.1 Two Depth-First Traversals

When traversing a linked list, the process is simple because there is only one child. The only choice you can make is to append the recursive result from the rest of the list before or after the local data value. When traversing a binary tree, there are three data items to combine: the local data item (D), the result from the left branch (L), and the result from the right branch (R). In general, they can be combined in six different combinations: D—L—R, L—D—R, L—R—D, D—R—L, R—D—L, and R—L—D. All six will give different results, but only two are actually useful to implement.

- The L—D—R sequence is referred to as an “in-order traversal” because if the tree is a BST, it will visit the nodes in their assigned order in the tree. This is the standard traversal method, unless there is a good reason not to use it.⁴ Template 20.2 shows the template for in-order traversal.
- The D—L—R sequence is referred to as a “pre-order traversal” because it touches the data first before performing either recursion.

² This requirement drives the nature of the information stored in a BST—there needs to be a high probability that the index will be unique, so that equality becomes either a data item to be ignored or an error to be analyzed. Good indices would be a telephone number or student ID. Bad indices would be first and last names in any combination, since uniqueness is not guaranteed.

³ It is technically possible, but beyond the scope of this text, to construct a BST where the data item stored is a collection of items, all of which satisfy a specific equality test. The structure of the BST would then continue to satisfy this requirement, while its implementation would add equal items to the local collection. Traversal and search then become considerably more complex, of course.

⁴ For our current purposes, the only reason not to use an in-order traversal is if the traversal is for the purpose of serializing a BST (writing it to a file for archiving purposes). If you use an in-order traversal for this, the data will be in order. If you subsequently insert that data, an item at a time back into an initially empty BST, the result will not be the original BST, but rather a linear tree!

Template 20.2 Recursive BinaryTree traversal template

```

<function and return> inOrder(here)
% processing items in order
if isempty(here)
    <return the empty answer>
else
    <combine the following>
    ... inOrder(getLeft(here)) ...
    ... getData(here) ...
    ... inOrder(getRight(here)) ...
end

```

This form of traversal is used when serializing a BST because it preserves the tree structure.

By convention in this text, the `char(...)` conversion of these more complex collections will not traverse the collection. Specific methods traversing the collections will be used. Listing 20.1 shows the `inOrderString(...)` method that directly matches the above template. For comparison:

- As usual, the wrapper function merely invokes the recursive process with the tree root
- Both functions return a character string
- The recursive function concatenates the result of the left recursion with the `char(...)` of the local node and the result of the right recursion
- The `BTNode char(...)` function takes care of presenting the string value of double types using the `%g` conversion

In Listing 20.1:

Lines 1–3: Show the header for the wrapper function.

Listing 20.1 The BinaryTree `inOrderString(...)` method

```

1. function str = inOrderString(bt)
2. % @BinaryTree\inOrderString
3. %   str = inOrderString(bt)
4. str = RinOrderString(bt.root);

5. function str = RinOrderString(here)
6. %   recursive in-order processing
7. if isempty(here)
8.     str = '';
9. else
10.     str = [ RinOrderString(getLeft(here)) ' ' ...
              char(here) ' ' ...
              RinOrderString(getRight(here)) ];
11. end

```

Line 4: The wrapper function invokes the recursive helper function with the root of the tree.

Lines 5 and 6: Show the header for the recursive helper function.

Lines 7 and 8: At a leaf node, return an empty string.

Lines 9–11: Otherwise, concatenate a string using (in order) the left recursive call, the character conversion of the local `BTNode`, and the right recursive call.

Listing 20.2 shows a pre-order traversal that is identical to Listing 20.1 except for the order of the concatenation. In this case, the node character conversion precedes the left and right traversal.

20.4.2 Breadth-First Traversal

Breadth-first traversal permits data in the tree to be accessed a generation at a time. It uses a simple, but effective, iterative template that uses a queue to store the children of each generation. Template 20.3 shows the template for this process.

Listing 20.2 The `BinaryTree preOrderString(...)` method

```

1. function str = preOrderString(bt)
2. % @BinaryTree\preOrderString
3. %   str = preOrderString(bt)
4. str = RPreOrderString(bt.root);

5. function str = RPreOrderString(here)
6. %   recursive in-order processing
7. if isempty(here)
8.     str = '';
9. else
10.    str = [ char(here) ' ' ...
            RPreOrderString(getLeft(here)) ' ' ...
            RPreOrderString(getRight(here)) ];
11. end

```

Template 20.3 Template for `BFTraversal`

```

<function and return> BFTraversal(btree)
% @BinaryTree\BFTraversal
q = Queue;      % create a queue
enqueue(q, getRoot(btree) );
< initialize the result>
while !isEmpty(q)
    node = dequeue(q);
    <operate on getData(node)>
    enqueue(q, getLeft(node));
    enqueue(q, getRight(node));
end
< return the result>

```

Listing 20.3 shows the code that produces a string representation of the tree using a breadth-first traversal. This is slightly complicated by the need to keep up with the layers in the tree. What is pushed onto the queue is a cell array containing a number—the level of the tree—with the node to be processed.

- All the children are pushed with the value of the next level
- A new line is inserted into the string when the level changes
- Whenever the level changes, we halve the number of spaces to be written between the items on a row
- Since it is possible for leaves to appear in the printout, they are marked by '-'.

In Listing 20.3:

Lines 1–3: Show the usual function header.

Lines 4 and 5: Create a queue and enqueue the tree root.

Lines 6–9: Initialize the data. We set the current level to -1 in order to force a new line at the beginning of the output.

Line 10: Continues as long as there are nodes in the queue.

Line 11: Removes an entry. Each entry is a cell array containing the current level (generation) in the tree and the current node to process.

Listing 20.3 BFTString generator

```

1. function str = BFTString(bt)
2. % @BinaryTree\BFTString
3. % tree traversal including formatting lines
4. q = Queue;
5. enqueue(q, {0, bt.root} )
6. level = -1;
7. sp='';
8. str = '';
9. nsp = 2*length(sp);
10. while ~isEmpty(q)
11.     v = dequeue(q);
12.     if v{1} ~= level      % set the tree level
13.         level = v{1};
14.         str = [str '\n'];
15.         nsp = ceil(nsp / 2);
16.     end
17.     node = v{2}; % insert what we just dequeued
18.     if isempty(node)
19.         str = [str sp(1:nsp) '-'];
20.     else
21.         str = [str sp(1:nsp) char(node)];
22.         % enqueue the children of this node
23.         enqueue(q, {level+1, getLeft(node) } );
24.         enqueue(q, {level+1, getRight(node) } );
25.     end
26. end

```


layer of the tree. The only place one can safely add to a BST and retain its ordering is at a leaf node, which is usually found recursively. This code is listed in Listing 20.4. This code will function well for any tree content object that can respond to the equality test and the greater than test (that is, an object that has `gt(...)` and `eq(...)` methods defined).

In Listing 20.4:

Lines 1 and 2: Show the header for the wrapper function. As with any operation that changes a dynamic data structure, the whole structure is always replaced.

Line 3: Passes the root to start the recursive helper, and stores the result as the new root.

Line 4: Copies the local object back to the caller.

Lines 5 and 6: Show the header for the recursive helper. Notice that a new `BTNode` is created and returned from each exit point.

Lines 7 and 8: Show the terminating condition: we found a leaf node, and return a new `BTNode` containing the new data.

Lines 9 and 10: We must test for equality to avoid adding duplicate keys. A policy decision is needed to decide whether to announce an error here or merely (as in this code) exit without adding a new node by just returning what we were given. Notice that the existing node must still be returned to add the rest of that structure to the new resulting structure.

Line 11: We compare the current node's data to the value to insert to decide whether the insertion happens to the left or right of this node.

Lines 12–14: Whichever direction is chosen, a new node is created with the same data value as the current node, the same left or right subtree to the side that is unchanged, and a recursive call to the side that is changed.



20.6 Mapping a Binary Tree

In general, mapping a binary tree is performed not as a general service to all `BinaryTree` users, but rather as a specialized utility written for derived classes that uses a `BinaryTree` as their parent class. For example, suppose a bank keeps all its interest-bearing accounts in a class derived from the `BinaryTree` class. Periodically, they need to traverse that tree and add the interest generated by each account. Since the size of the tree remains unchanged, but the contents change, this is a mapping of the original tree.

20.7 Deleting Nodes from a Binary Search Tree

We do not, in general, filter binary trees. These trees are static in nature, and rarely need maintenance. If maintenance is required, this is usually done manually in the code that reconstructs the tree. We can, however, filter a BST in a restricted sense—removing one node at a time, while preserving the ordering of the BST. Figure 20.4 illustrates this process.

When removing an item from a BST, there are three cases to consider depending on the location in the tree of the item to be removed.

No children node: Consider, for example, removing the node containing 13. Since it has no children at all, it can be removed simply by replacing its link in the node containing 9 with [].

One child node: Removing a node with one child node (such as those with value 9 or 71) is also simple. We merely replace its parent node link with a link to the surviving child node.

Two children nodes: The logic for removing a node with both children present is a little more challenging. Consider removing the node with value 50. First, we must find the largest node in its left subtree, the value 42, or the smallest in its right subtree, the value 66. We place its node data in the node we are removing, and then delete that node from the tree. This latter deletion is one of the simpler cases with fewer than two children.

Listings 20.6 and 20.7 show the code for this.

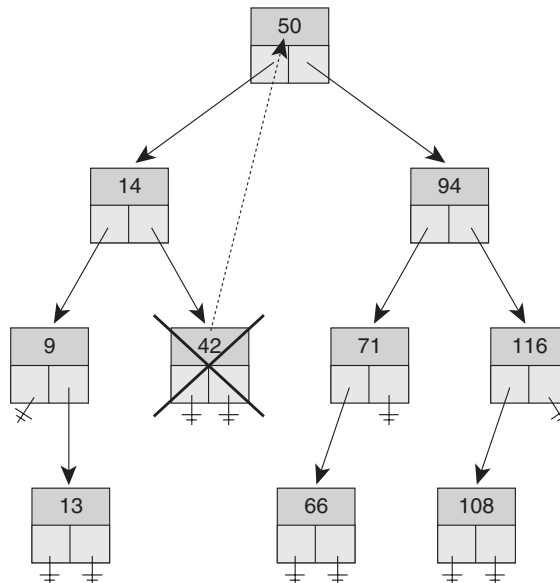


Figure 20.4 Deleting from a BST

Listing 20.6 Deleting from a BST Part 1—finding and repairing

```

1. function delete(bst, n)
2. % @BST\delete
3. setRoot(bst, rDelete(getRoot(bst), n) );
4. assignin('caller', inputname(1), bst);
5. function node = rDelete(here, n)
6. % recursive BST delete function
7. if isempty(here)           % not in the tree
8.     node = [];
9. elseif getData(here) == n % found it
10.    node = deleteThis(here);
11. elseif getData(here) > n % look left
12.    node = BTreeNode(getData(here), ...
                       rDelete(getLeft(here), n), ...
                       getRight(here));
13. else                       % look right
14.    node = BTreeNode(getData(here), ...
                       getLeft(here), ...
                       rDelete(getRight(here), n));
15. end

```

In Listing 20.6:

Lines 1–4: Show the typical wrapper function passing the root to the recursive helper, saving the result as the new root and returning the local object to the caller.

Lines 5 and 6: Show the header for the recursive delete function.

Lines 7 and 8: Show the terminating condition—we did not find the value sought—and return an empty node.

Lines 9 and 10: Show successful termination—we found the node required, and call `deleteThis(...)` (Listing 20.7) to compute the new subtree below this node.

Lines 11–15: The usual BST search creates a new `BTreeNode`, substituting the results of a right or left recursive call.

Listing 20.7 Deleting from a BST Part 2—finding the replacement

```

1. function node = deleteThis(here)
2. % @BST\deleteThis - delete this node
3. left = getLeft(here);
4. right = getRight(here);
5. if isempty(left)
6.     node = right;
7. elseif isempty(right)
8.     node = left;
9. else
10.    [value rest] = deleteLargest(left);
11.    node = BTreeNode(value, rest, right);
12. end

```

continued on next page

```

13. function [value node] = deleteLargest(here)
14. % find the largest in this tree
15. right = getRight(here);
16. if isempty(right)
17.     value = getData(here);
18.     node = getLeft(here);
19. else
20.     [value, node] = deleteLargest(right);
21. end

```

In Listing 20.7:

Lines 1 and 2: Show the header.

Lines 3 and 4: Retrieve the left and right nodes.

Lines 5 and 6: Show one of the easy cases—if the left is empty, return the right node. That might also be empty, of course.

Lines 7 and 8: Otherwise, the right node's being empty also gives us one of the easy cases.

Line 10: No such luck, so we have to go to a recursive helper to give us two things: the value of the node we want to delete and the tree that remains when we delete it. We provide it with the left subtree and expect the largest value back. We could also have passed the right subtree to a similar function that looks for the smallest value.

Line 11: Creates a new node with the value returned, the remains of the left subtree, and the untouched right subtree.

Lines 13 and 14: Show the recursive helper function that finds and deletes the largest node in this subtree.

Line 15: Gets the right node. The largest in this subtree will be the node that has no right child (it might have a left child).

Line 16: Shows the terminating condition: the right child is empty.

Lines 17 and 18: If not, return the value of this node and its left child that might be empty.

Line 20: Otherwise, keep going recursively to the right.



20.8 Folding a Binary Tree

Unlike the previous tree operations, folding can be performed as a service in the `BinaryTree` class. We illustrate this by the `count(...)` method. To count items in a binary tree, we will again follow the recursive template shown in Template 20.1, with the following tailoring required:

- As usual, the public method, `count(...)`, merely launches the recursive helper, `rCount`, with the root of the tree and passes the total to its caller

- The helper returns 0 when it reaches a tree leaf
- Otherwise, it returns the result from adding 1 to the result of applying `rCount` recursively to left and right subtrees

The resulting code is shown in Listing 20.8.

In Listing 20.8:

Lines 1 and 2: Show the function header.

Line 3: Returns the answer from the recursive call. Since we are not changing the tree, we do not need to return our object to the caller.

Lines 4 and 5: Show the header for the recursive helper.

Lines 6 and 7: Show the terminating condition—a leaf node. Return a count of 0.

Line 9: Otherwise, adds 1 for this node to the recursive counts for the left and right subtrees.

Listing 20.8 A generic `BinaryTree` `count` method

```

1. function ans = count(bt)
2. % @BinaryTree\count
3. ans = rCount(bt.root);

4. function ans = rCount(here)
5. % recursive counting a binary tree
6. if isempty(here)
7.     ans = 0;
8. else
9.     ans = 1 + rCount(getLeft(here)) ...
               + rCount(getRight(here));
10. end

```



20.9 Searching Binary Trees

We need to consider searching both the ordinary binary tree and the BST.⁵ In the same way as we categorized traversal, searching an ordinary binary tree may be accomplished either breadth-first or depth-first.

20.9.1 Breadth-First Search

The breadth-first search method `BFFind(...)` will follow the iterative, breadth-first template shown in Template 20.3, with the following tailoring required:

- First, it creates a queue and enqueues the tree root

⁵ We should observe again that a BST ordered by some particular data attribute can be searched efficiently for items with that particular attribute. However, to find items by some other criteria requires it to be treated as a regular binary tree.

- As long as that queue is not empty (indicating failure) or the dequeued item is not what we seek, it continues enqueueing the children of the current node

The resulting code is shown in Listing 20.9.

In Listing 20.9:

- Lines 1 and 2: Show the function header.
- Lines 3 and 4: Create a queue and enqueue the tree root.
- Line 5: Initializes the loop control variable.
- Line 6: Initializes the answer.
- Line 7: Shows loop-and-a-half repeats until the answer is found (success), or the queue is empty (failure).
- Line 8: Fetches a node.
- Line 10: Checks for the target (invokes the `eq(...)` method).
- Line 11: Continues the process only when this is not the target.
- Line 13–16: Enqueue the left child if it is not empty.
- Lines 17–20: Enqueue the right child if it is not empty.
- Lines 23 and 24: Return the node found rather than `true` or `false`. With complex objects, this allows access to other parts of the data found.

Listing 20.9 BFFind method

```

1. function ans = BFFind(bt, what)
2. % @BinaryTree\BFFind
3. q = Queue;
4. enqueue(q, bt.root )
5. found = false;
6. ans = [];
7. while ~found && ~isEmpty(q)
8.     node = dequeue(q);
9.     % check the data we just dequeued
10.    found = (node == what);
11.    if ~found
12.        % enqueue the children of this node
13.        left = getLeft(node);
14.        if ~isempty(left)
15.            enqueue(q, left );
16.        end
17.        right = getRight(node);
18.        if ~isempty(right)
19.            enqueue(q, right );
20.        end
21.    end
22. end
23. if found
24.    ans = getData(node);
25. end

```

20.9.2 Depth-First Search

Depth-first search can also be accomplished iteratively using an `MStack` instead of a `Queue`, as illustrated with the `DFFind(...)` method. The code illustrated in Listing 20.10 is similar in every way to Listing 20.9, except that an `MStack` class with `push(...)` and `pop(...)` is used in place of the `Queue` class with `enqueue(...)` and `dequeue(...)`.

20.9.3 Searching a Binary Search Tree

BST search takes advantage of the ordering of the data to eliminate half of the tree at each search step, as shown in Listing 20.11. Although this could be accomplished with the `while` loop template, the recursive template shown in Template 20.1 is a better choice for simpler code. Obviously, the data items in the tree must provide both an `eq(...)` method and a `gt(...)` method.

Listing 20.10 `DFFind` method

```

1. function ans = DFFind(bt, what)
2. % BinaryTree\DFFind
3. st = MStack;
4. push(st, bt.root )
5. found = false;
6. ans = [];
7. while ~found && ~isEmpty(st)
8.     node = pop(st);
9.     % check the data we just popped
10.    found = (node == what);
11.    if ~found
12.        % push the children of this node
13.        left = getLeft(node);
14.        if ~isEmpty(left)
15.            push(st, left );
16.        end
17.        right = getRight(node);
18.        if ~isEmpty(right)
19.            push(st, right );
20.        end
21.    end
22. end
23. if found
24.     ans = getData(node);
25. end

```

Listing 20.11 BST search

```

1. function ans = find(bst, n)
2. % @BST\find
3. ans = rFind( getRoot(bst), n);

```

continued on next page

```

4. function ans = rFind(here, n)
5. %
6. if isempty(here)
7.     ans = [];
8. elseif getData(here) == n
9.     ans = getData(here);
10. elseif getData(here) > n
11.     ans = rFind(getLeft(here), n);
12. else
13.     ans = rFind(getRight(here), n);
14. end

```

20.9.4 Testing Binary Search Tree Functionality

Listing 20.12 shows how to test the binary tree functions discussed so far in this chapter.

In Listing 20.12:

Line 1: Asks the user how many nodes for this tree.

Listing 20.12 Testing the BST functions

```

1. nn = input('How many nodes? ');
2. bst = BST( BTreeNode(50) );
3. for in = 1:nn
4.     insert(bst, ceil(100*rand));
5. end
6. fprintf('number is %d (started with %d)\n', ...
           count(bst), nn+1 );
7. fprintf('largest number is %d\n', ...
           largest(bst) );
8. fprintf('pre-order list is %s\n', ...
           preOrderString(bst) );
9. fprintf('in-order list is %s\n', ...
           inOrderString(bst) );
10. fprintf('breadth-first list: %s\n', ...
           BFTString(bst));
11. n = 1;
12. while n > 0
13.     n = input('number to remove: ');
14.     if n > 0
15.         ans = find(bst, n)
16.         BSTdelete(bst, n);
17.         fprintf('number is %d (started with %d)\n', ...
                   count(bst), nn+1 );
18.         fprintf('in-order list is %s\n', ...
                   inOrderString(bst) );
19.         fprintf(' bst with %d removed: %s\n', ...
                   n, BFTString(bst));
20.     end
21. end

```

Line 2: Constructs a BST with 50 as its root. This is actually a sneaky trick to give the BST some balance. Since we will be inserting values between 0 and 100, starting with a root of 50 gives the best chance of a roughly balanced tree.

Lines 3–5: Insert the requested number of values into the tree.

Line 6: Folds the tree by counting its nodes. Why might these numbers not match?

Line 7: Shows another fold to find the largest number. Would this function be different on a BST than on a binary tree? Would it be more efficient?

Lines 8–10: Show results from pre-order, in-order, and breadth-first traversals.

Lines 11–13: Show a loop-and-a-half, asking the user which node to delete from the tree.

Line 14: Shows that a negative entry terminates the loop.

Line 15: Searches the BST for the value requested.

Lines 16–19: Delete the value and display the count, in-order listing, and the breadth-first traversal of the result.



20.10 Combining Complex Operations

When the stated problem is a little more complex than the ordinary operations previously indicated, it is tempting to expand the code for a single operation. For example, suppose we were asked to put into a cell array the names of all foreigners in Prince William's ancestral tree. It would be possible to modify the `fold` method on the `FamilyTree` to apply the condition that the person's nationality be not British, and then add their name to a cell array of names. However, it seems that a regular traversal of the tree, extracting all the data into a cell array and then filtering that cell array to remove all the non-British subjects, might take a little longer but would be much more robust and comprehensible.



Chapter Summary

In this chapter, we have seen the following:

- Dynamic data structures can be created in many forms, the most general of which is the graph.
- Binary trees are defined recursively as dynamic collections where each node has at most two children.
- A binary search tree (BST) is a binary tree where at every node, all data less than that node value are in the left subtree, and all data greater than its value are in the right subtree.

- In general, binary trees are processed recursively, except when performing breadth-first operations where a queue can be used to organize an iterative solution.
- Because the organization of the data makes searching BSTs very efficient, most applications of interest relate to managing, traversing, and searching BSTs.

N-ary Trees and Graphs

Chapter Objectives

This chapter discusses:

- The nature of N-ary trees
- The MATLAB implementation of a number of the common operations on N-ary trees
- Some ideas on how to develop and test these concepts on graphs

- 21.1** N-ary Tree Concepts
- 21.2** Processing N-ary Trees
- 21.3** Traversing N-ary Trees
 - 21.3.1 Breadth-First Traversal
 - 21.3.2 Depth-First Traversal
- 21.4** Building N-ary Trees
- 21.5** Folding N-ary Trees
 - 21.5.1 Counting N-ary Trees
 - 21.5.2 Finding the Largest Item in N-ary Trees
- 21.6** Searching N-ary Trees
 - 21.6.1 Breadth-First Search
 - 21.6.2 Depth-First Search
- 21.7** Testing N-ary Trees
- 21.8** Graphs in OO Style
- 21.9** Concluding Observations

21.1 N-ary Tree Concepts

We move to an even more complex dynamic data structure—the N-ary tree, which removes the previous restriction on the number of children permitted. Where each node of a binary tree has two child nodes, an N-ary tree generally contains a collection of zero or more child nodes, as shown in Figure 21.1. We discussed the place of the N-ary tree concept among other dynamic data structures in Section 20.1.

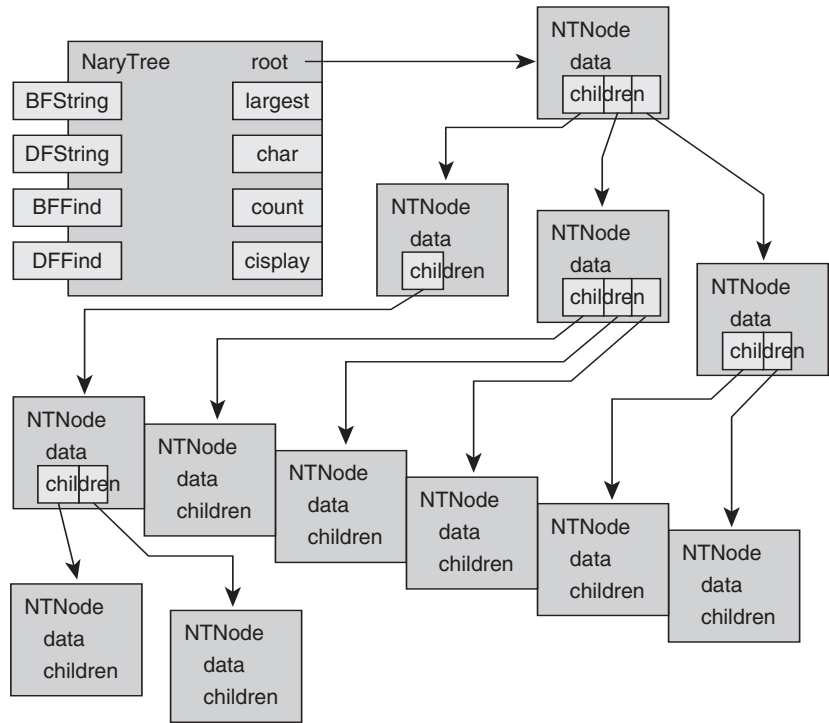


Figure 21.1 Structure of an N-ary tree

21.2 Processing N-ary Trees

As with binary trees, there are opportunities to use either iterative processes or recursive processes. First, we will consider processing N-ary trees recursively. Figure 21.2 illustrates the content of the node with which we

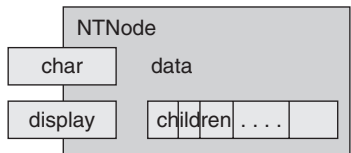


Figure 21.2 An N-ary tree node

construct an N-ary tree. Omitted for clarity are the `set/get` methods for the `data` and `children` attributes.

We determine how to process an N-ary tree based on its formal definition as follows:

An `N-aryTree` is an `NTNode` containing a data item and a collection (that might be empty) of `NTNodes` named `children`.¹

This is a slightly different formal definition than we have seen before, because it introduces the idea of a collection of child nodes. A very strong way to decompose this definition for processing is to consider two separate parts of the node—the data part and the collection of children—and to build a separate function for processing each part. The template for processing an N-ary tree then becomes mutually recursive, as shown in Template 21.1.

Figure 21.3 illustrates the operation of this recursive process on a particular `NTNode`. The children of this node are a collection of references to other

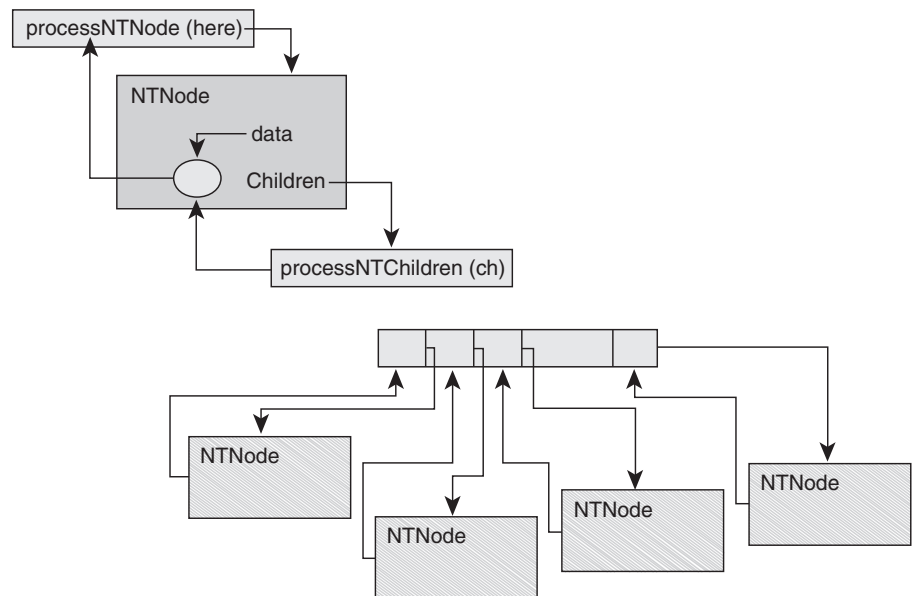


Figure 21.3 Processing an N-ary tree node

Template 21.1 Template for processing an N-ary tree

```
<function and return> processNaryTree(nt)
% process the whole tree
if <tree is empty>
    <return default result>
else
```

continued on next page

¹ Notice that having a collection of children actually simplifies the processes because the collection of children can be empty, thereby removing the possibility (except at the root of an empty tree) that the node to process is ever empty.

```

        <return processNTNode(root)>
    End

    <function and return> processNTNode(here)
    % mutually recursive processing
        ... getData(here) ...
        ... processNTChildren( getChildren(here) ) ...

    <function and return> processNTChildren(ch)
    % traverse the collection, processing each child
    <initialize the result>
    <traverse the collection>
        <extract the child node thisch>
        <insert processNTNode(thisch) in the result>
    <end traversal>
    <return the result>

```

NTNodes; therefore, we define a separate method for processing that collection² to accumulate the results from the child nodes. The main function then combines this summary of the child results with the data at this node, and returns the result to the calling node.



21.3 Traversing N-ary Trees

We will consider both depth-first and breadth-first techniques for traversing N-ary trees. For simplicity, we will adopt the iterative approach using queues and stacks to manage the traversal.

21.3.1 Breadth-First Traversal

Breadth-first traversal permits data in the tree to be accessed horizontally a generation at a time, and is the more intuitive presentation of data. It uses a simple, but effective, iterative template that uses a queue to store the children of each generation. Template 21.2 shows the template for this process. As with the binary tree breadth-first traversal, the need to present the data in generation order introduces some complexity into the code to manage not

Template 21.2 Template for N-ary tree breadth-first processing

```

<function and return> BFProcessNTree(ntree)
q = Queue;          % create a queue
q.enqueue(getRoot(ntree) );
< initialize the result>
while !isEmpty(q)
    node = dequeue(q);
    <operate on getData(node)>

```

continued on next page

² Although it is possible to use any collection capable of holding generalized objects, in this text we have chosen to use a cell array to hold the collection of children.

```

        <traverse the children>
            enqueue(q, <each child>);
        end
    end
    < return the result>

```

only the introduction of new lines in the output string, but also an attempt to use a variable number of spaces to present the data in tree form.

Listing 21.1 shows the code that produces a string representation of the tree using a breadth-first traversal.

In Listing 21.1:

Lines 1–3: Show the header for the `BFString` method on the `NaryTree` class.

Lines 4 and 5: Create a queue containing the tree root at level 0.

Listing 21.1 Breadth-first N-ary tree string generator

```

1. function str = BFString(nt)
2. % @NaryTree\BFString
3. % tree traversal including formating lines
4. q = Queue;
5. enqueue(q, {0, nt.root} )
6. level = -1;
7. sp = '                                     ';
8. nsp = length(sp)*2;
9. str = '';
10. while ~isEmpty(q)
11.     v = dequeue(q);
12.     % set the tree level
13.     if v{1} ~= level
14.         level = v{1};
15.         str = sprintf('%s\n', str);
16.         nsp = ceil(nsp / 2);
17.     end
18.     % insert the data we just dequeued
19.     node = v{2};
20.     if isempty(node)
21.         str = [str sp(1:nsp) '-'];
22.     else
23.         str = [str sp(1:nsp) char(node)];
24.         % enqueue the children of this node
25.         ch = getChildren(node);
26.         for ind = 1:length(ch)
27.             enqueue(q, {level+1, ch{ind} });
28.         end
29.     end
30. end

```

Line 6: Sets the original tree level to -1 to force a level transition for the root node.

Lines 7 and 8: Show a string of spaces for use in spacing the output, with the initial length twice its actual length (it will be halved immediately).

Line 9: Initializes the output.

Line 10: Continues as long as the queue is not empty.

Line 11: Dequeues a level and a node.

Line 13: If the dequeued level is not the current level, we have just dequeued the first node of the next generation.

Lines 14–16: So we set the new level, insert a new line in the output, and halve the number of spaces used.

Line 19: Recover the node itself.

Line 21: If it is empty, merely show it as '- '.

Line 23: Otherwise, append to the output the chosen number of spaces and the `char(...)` cast of this node.

Lines 25–27: Enqueue each child node at the next level.

21.3.2 Depth-First Traversal

In general, depth-first traversal seeks to probe deeply down a tree first, and to move laterally only when arrival at a leaf causes the algorithm to retreat back up the tree to the lowest unused branch. We call this “back-tracking.” It can be accomplished recursively. However, here we choose to show an iterative implementation using the `MStack` class to replace the `Queue` class used in breadth-first traversal.

Template 21.3 shows the template for depth-first traversal on an N-ary tree. It is the typical iterative approach using a stack to organize the data. In order to maintain the “natural” left-right appearance of the traversal, the

Template 21.3 Iterative N-ary tree depth-first template

```
<function and return> DFprocessNTree(ntree)
st = MStack;      % create a stack
push(st, getRoot(ntree) );
< initialize the result>
while !isEmpty(st)
    node = pop(st);
    <operate on getData(node)>
    <traverse the children in reverse order>
    push(st, <each child>);
end
end
< return the result>
```

children should be pushed onto the stack in reverse order so that they are popped off the stack in forward order.

Since by convention in this text, the `char(...)` conversion of these more complex collections will not traverse the collection, we must use specific methods traversing the collections to create strings. Listing 21.2 shows the `DFString(...)` method that performs the traversal using a stack.

In Listing 21.2:

Lines 1–3: Show the header for the `DFString` method on the `NaryTree` class.

Lines 4 and 5: Create a stack containing the tree root.

Line 6: Initializes the result.

Line 7: Continues as long as there are items on the stack.

Line 8: Pops the first item off the stack. Depth-first behavior is achieved because this is the last item pushed onto the stack. Backtracking happens automatically because leaf nodes push nothing fresh onto the stack.

Line 9: Appends the `char` cast of this node to the output.

Line 11: Gets all the children.

Lines 12 and 13: Iterate across the children (if any), pushing the children in reverse order. The order reversal is necessary only to enforce the expected appearance of going left to right across the children.

Listing 21.2 The N-ary tree `DFString(...)` method

```
1. function str = DFString(nt)
2. % @NaryTree\DFString
3. % depth-first tree traversal
4. st = MStack;
5. push(st, nt.root )
6. str = '';
7. while ~isEmpty(st)
8.     node = pop(st);
9.     str = [str ' ' char(node)];
10.    % push the children of this node
11.    ch = getChildren(node);
12.    for ind = length(ch):-1:1
13.        push(st, ch{ind} );
14.    end
15. end
```



21.4 Building N-ary Trees

Since the structure of N-ary trees usually has some specific meaning not evident from the data content itself, they are not usually built dynamically.

For our purposes, they will be created in the test script. The motivation example in this text will be a segment of the British royal family tree. A fuller version of this tree is provided at the Addison-Wesley Instructor Resource Center (www.aw.com/irc).

21.5 Folding N-ary Trees

Here we will illustrate two examples of mutually recursive methods to fold or summarize an N-ary tree. The first, `count(...)`, is of course independent of the content of the tree. The second, `largest(...)`, will find the “largest” item in the tree. Our test data for this will be a family tree containing person objects whose comparisons are performed on their age. Therefore, this method will find the oldest person in the family tree.

21.5.1 Counting N-ary Trees

To count items in an N-ary tree, we will follow the recursive template in Template 21.1 with the following tailoring required:

- The public method, `count`, checks for an empty tree, then launches the recursive helper, `countChild`, with the root of the tree and passes the total to its caller
- The `countChild` helper adds one to the result of counting its children
- The `countChildren` helper traverses the collection of children, accumulating the number of children in each subtree

The resulting code is shown in Listing 21.3.

Listing 21.3 NaryTree count method

```

1. function ans = count(nt)
2. % @NaryTree\count
3. if isempty(nt.root)
4.     ans = 0;
5. else
6.     ans = countChild(nt.root);
7. end

8. function ans = countChild(here)
9. % count the child node
10. ch = getChildren(here);
11. ans = 1 + countChildren( ch );

12. function ans = countChildren(ch)
13. % count the children nodes
14. ans = 0;
15. for index = 1:length(ch)
16.     ans = ans + countChild( ch{index} );
17. end

```

In Listing 21.3:

Lines 1 and 2: Show the method header.

Lines 3 and 4: Here we are setting up a pair of mutually recursive functions in which there is actually no terminating condition; therefore, we must deal with the empty tree in the wrapper function.

Line 6: Calls the recursive child processing function with the tree root.

Lines 8 and 9: Show the header for the child-processing function.

Line 10: Extracts the children.

Line 11: Calls the child-processing function to count the children, and then adds 1 for this node.

Lines 12 and 13: Show the header for the children processor.

Line 14: Initializes the answer.

Lines 15 and 16: Iterate across the children, calling the child-counting method on each and summing them. This call back to the child-processing method makes this and most operations on N-ary trees mutually recursive.

21.5.2 Finding the Largest Item in N-ary Trees

To find the largest item in N-ary trees, we will again follow the recursive Template 21.1 with the following tailoring required:

- The public method, `largest`, checks for an empty tree, then launches the recursive helper, `largestChild`, with the root of the tree and passes the result to its caller
- The `largestChild` helper invokes the `largestOfChildren` method to find its largest child (which could be empty if there are no children)
- The `largestOfChildren` helper uses a recursive call to the `largestChild` method to find the largest child in each subtree—it checks that there are children, sets the largest to the largest child in the first child subtree, and traverses the rest of the collection of children, comparing the largest child in each subtree

The resulting code is shown in Listing 21.4.

Listing 21.4 Finding the largest item in N-ary trees

```
1. function ans = largest(nt)
2. % @NaryTree\largest
3. % returns the largest item in the tree
4. if isempty(nt.root)
5.     ans = [];
6. else
7.     ans = largestChild(nt.root);
8. end
```

continued on next page

```

9. function ans = largestChild(here)
10. % returns the data item that is the largest child
11. thisch = getData(here);
12. ch = getChildren(here);
13. chl = largestOfChildren( ch );
14. if ~isempty(chl) && (chl > thisch)
15.     ans = chl;
16. else
17.     ans = thisch;
18. end

19. function ans = largestOfChildren(ch)
20. % returns the data item that is the largest child
21. l = length(ch);
22. if l == 0
23.     ans = [];
24. else
25.     ans = largestChild(ch{1});
26.     for index = 2:length(ch)
27.         chld = largestChild(ch{index});
28.         if chld > ans
29.             ans = chld;
30.         end
31.     end
32. end

```

In Listing 21.4:

Lines 1–3: Show the method header.

Lines 4 and 5: Show the empty case.

Line 7: Shows the recursive call returning the result.

Lines 9 and 10: Show the header for the child processor.

Lines 11 and 12: Extract the data and children.

Line 13: Calls the children processor for the largest child.

Line 14: Shows two interesting MATLAB features at work. Short-circuit evaluation ensures that if the largest child is empty (there are no children), we do not try to compare it, and the ' $>$ ' test automatically invokes the `get(...)` method on the first object.

Lines 15–17: Set the result accordingly to the largest child or this child.

Lines 19 and 20: Show the header for the child-processing function.

Line 21: Gets the number of children.

Lines 22 and 23: If none, return the empty string.

Line 25: Presets the answer to the largest of the first child.

Lines 26–31: Iterate through the remaining children, finding by mutual recursion their largest, and comparing that to the current largest, replacing it if necessary.



21.6 Searching N-ary Trees

We need to consider searching N-ary trees either breadth-first or depth first. As with the traversals, these searches are done iteratively with queues and stacks, respectively.

21.6.1 Breadth-First Search

The breadth-first search method `BFFind` will follow the iterative Template 21.2 with the following tailoring required:

- First, it creates a `queue` and enqueues the tree root
- As long as that `queue` is not empty (indicating failure) or the dequeued item is not what we seek, it continues enqueueing the children of the current node

The resulting code is shown in Listing 21.5.

In Listing 21.5:

Lines 1 and 2: Show the method header.

Lines 3 and 4: Create a queue containing the root.

Lines 5 and 6: Initialize a detection flag and the result.

Line 7: Continues as long as the answer is not found, and the queue has more data.

Line 8: Extracts a node from the queue.

Listing 21.5 NaryTree BFFind method

```

1. function ans = BFFind(nt, what)
2. % NaryTree\BFFind
3. q = Queue;
4. enqueue(q, nt.root )
5. found = false;
6. ans = [];
7. while ~found && ~isEmpty(q)
8.     node = dequeue(q);
9.     % check the data we just popped
10.    found = (node == what);
11.    if ~found
12.        % push the children of this node
13.        ch = getChildren(node);
14.        for ind = 1:length(ch)
15.            enqueue(q, ch{ind});
16.        end
17.    end
18. end
19. if found
20.    ans = getData(node);
21. end

```

Line 10: Automatically calls the `eq(...)` method, testing against the input parameter.

Lines 11–17: If this is not the answer, enqueue all the children of this node.

Lines 19 and 20: If we find the answer, return the data from that node.

21.6.2 Depth-First Search

Depth-first search can also be accomplished iteratively using an `MStack` instead of a `Queue`, and using Template 21.3, as illustrated with the `DFFind` method. The code is shown in Listing 21.6.

In Listing 21.6:

Lines 1 and 2: Show the method header.

Lines 3 and 4: Create a stack containing the root.

Lines 5 and 6: Initialize a detection flag and the result.

Line 7: Continues as long as the answer is not found and the stack has more data.

Line 8: Extracts a node from the stack.

Line 10: Automatically calls the `eq(...)` method, testing against the input parameter.

Lines 11–17: If this is not the answer, push all the children of this node.

Listing 21.6 NaryTree DFFind method

```

1. function ans = DFFind(nt, what)
2. % NaryTree\DFFind
3. st = MStack;
4. push(st, nt.root )
5. found = false;
6. ans = [];
7. while ~found && ~isEmpty(st)
8.     node = pop(st);
9.     % check the data we just popped
10.    found = (node == what);
11.    if ~found
12.        % push the children of this node
13.        ch = getChildren(node);
14.        for ind = length(ch):-1:1
15.            push(st, ch{ind});
16.        end
17.    end
18. end
19. if found
20.    ans = getData(node);
21. end

```

Lines 19 and 20: If we find the answer, return the data from that node.



21.7 Testing N-ary Tree Methods

To test these N-ary tree methods, we built a brief royal family database, as shown in Listing 21.7. These person objects are then collected into an N-ary tree by building the nodes illustrated in Listing 21.8 and tested using the script shown in Listing 21.9. All three scripts should be run in sequence. Clearly, `BFFind` can also be used for searching.

Listing 21.7 A brief royal family collection

```
William = Person('William', 1982, -1, [], ...
    'Prince of Wales');
Henry = Person('Henry', 1984, -1, [], ...
    'Prince of Wales');
Peter = Person('Peter', 1977, -1, [], ...
    'Phillips');
Zara = Person('Zara', 1981, -1, [], ...
    'Phillips');
Beatrice = Person('Beatrice', 1988, -1, [], ...
    'Princess of York');
Eugenie = Person('Eugenie', 1990, -1, [], ...
    'Princess of York');
Charles = Person('Charles', 1948, -1, [], ...
    'Prince of Wales');
Anne = Person('Anne', 1950, -1, [], ...
    'Princess Royal');
Andrew = Person('Andrew', 1960, -1, [], ...
    'Duke of York');
Edward = Person('Edward', 1964, -1, [], ...
    'Earl of Wessex');
David = Person('David', 1961, -1, [], ...
    'Viscount Linley');
Sarah = Person('Sarah', 1964, -1, [], ...
    'Lady Armstrong-Jones');
ElizabethII=Person('Elizabeth II', 1926, -1, [], ...
    'Queen of the United Kingdom ...');
Margaret = Person('Margaret', 1930, 2002, [], ...
    'Princess');
GeorgeVI = Person('George VI', 1895, 1952, [], ...
    'King of the United Kingdom ...');
```

Listing 21.8 A brief royal family tree

```
1. N1_01 = NTNode(William);
2. N1_02 = NTNode(Henry);
3. N1_03 = NTNode(Peter);
4. N1_04 = NTNode(Zara);
5. N1_05 = NTNode(Beatrice);
6. N1_06 = NTNode(Eugenie);
```

continued on next page

```

7. N2_02 = NTNode(Charles);
8. setChildren(N2_02,{N1_01, N1_02});
9. N2_03 = NTNode(Anne);
10. setChildren(N2_03,{N1_03, N1_04});
11. N2_04 = NTNode(Andrew);
12. setChildren(N2_04,{N1_05, N1_06});
13. N2_05 = NTNode(Edward);
14. N2_06 = NTNode(David);
15. N2_07 = NTNode(Sarah);

16. N3_01 = NTNode(ElizabethII);
17. setChildren(N3_01,{N2_02, N2_03, N2_04, N2_05});
18. N3_02 = NTNode(Margaret);
19. setChildren(N3_02,{N2_06, N2_07});

20. N4_01 = NTNode(GeorgeVI);
21. setChildren(N4_01,{N3_01, N3_02});

```

In Listing 21.8:

Lines 1–6: Create the leaf nodes, generation 1, using the variables from Listing 21.7.

Lines 7–15: Create the second generation nodes and set the appropriate children from the first generation.

Lines 16–19: Create the third generation nodes and set the appropriate children from the second generation.

Lines 20 and 21: Show that the root of this tree is King George VI.

Listing 21.9 Testing the royal family tree

```

1. family = NaryTree(N4_01)
2. fprintf('s has %d descendants!\n', ...
           show(getData(getRoot(family))), ...
           count(family) - 1 );
3. fprintf('oldest member is %s\n', ...
           show( largest(family) ) );
4. fprintf('depth-first list: %s\n', ...
           DFString(family));
5. fprintf('breadth-first list: %s\n', ...
           BFString(family));
6. lookFor = input('Depth First find: ', 's' );
7. while length(lookFor) > 0
8.     val = DFFind(family, lookFor);
9.     if isempty(val)
10.        disp('not found');
11.    else
12.        fprintf('depth-first find %s -> %s\n', ...
                  lookFor, show(val));
13.    end
14.    lookFor = input('Depth First find: ', 's' );
15. end

```

In Listing 21.9:

- Line 1: Creates an N-ary tree using the nodes connected in Listing 21.8.
- Line 2: Shows the result of counting the tree. We subtract 1 because when we count a tree, we include the root in the count. However, a normal definition of “descendants” excludes the root.
- Line 3: Shows the results from finding the oldest family member.
- Line 4: Shows the depth-first listing.
- Line 5: Shows the breadth-first listing.
- Line 6: Requests a person to search for.
- Line 7: While the user enters an answer, continue.
- Lines 8–13: Use depth-first find to locate the requested person and display the result.
- Line 14: Requests the next person.



21.8 Graphs Using OO

As we discussed in Chapter 17, graphs are the most general form of dynamic data structure. It would be splendid if we could flow smoothly from N-ary trees to graphs in the OO paradigm, showing natural extensions from the building, traversal, and path-finding capabilities to that for graphs. Unfortunately, we run across a fundamental limitation of MATLAB. It is not possible to create a dynamic data structure with a closed cycle or even with multiple parents to create a directed graph. This is why we resorted to using the adjacency matrix in Chapter 17 to describe the connectivity of a graph.

Perhaps a simple illustration will suffice. Suppose we tried to construct the simple fragment of a directed graph, shown in Figure 21.4, where nodes B and C are children of A, and D is a child both of B and C—a legal and common situation in constructing graphs. We will also assume that we can represent the nodes with `NTNode` structures.

In order to set up this experiment, we added a method to the `NTNode` structure that changes the data in an `NTNode` object—not something one would normally do—as shown in Listing 21.10.

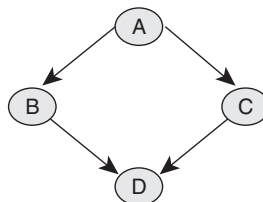


Figure 21.4 A fragment of a directed graph

Listing 21.10 NTNode setData method

```

1. function setData(ntn, item)
2. % @NTNode\setData - set the data of an NTNode
3.     ntn.data = item;
4.     assignin('caller', inputname(1), ntn);

```

In Listing 21.10:

Lines 1 and 2: Show the method header.

Line 3: Sets the data attribute in the local copy.

Line 4: Returns the local copy.

We can now run the script shown in Listing 21.11 to check the ability of an object to have multiple parents. Exercise 21.1 shows the results.

Listing 21.11 Testing multiple parents

```

1. clear
2. clc
3. clear classes

4. disp('make the four nodes')
5. A = NTNode('A')
6. B = NTNode('B')
7. C = NTNode('C')
8. D = NTNode('D')

9. disp('make B and C children of A')
10. setChildren(A, {B, C})
11. disp('make D a child of B')
12. setChildren(B, {D})
13. disp('make D a child of C')
14. setChildren(C, {D})

15. disp('get the children of B')
16. blist = getChildren(B)
17. disp('D should be the first child')
18. cb = blist{1}
19. disp('set the data in B's child')
20. setData(cb, 'child of B')
21. cb

22. disp('get the children of C')
23. clist = getChildren(C)
24. disp('D should be the first child')
25. cc = clist{1}
26. disp(['if D was a child of B and C, ' ...
        'this should say ''child of B'' '])
27. cc

```

In Listing 21.11:

Lines 1 and 2: Show the usual script header.

Line 3: This is useful when you have changed the class definitions as we did by adding a method to the `NTNode` class.

Lines 4–8: Create the four nodes.

Lines 9–14: Attempt to create the tree where D is a child of both A and C.

Lines 15–21: Find and modify the child of B.

Lines 22–27: Find and examine the child of C.

The test clearly failed. We found the child of B and changed the data at that node from 'D' to 'child of B'. However, when we extracted the child of C and displayed its data, the value was still 'D'. Now, we must think carefully why this test failed. The very attribute of MATLAB that provides encapsulation of the data also prevents this code from doing what we expected. MATLAB rightly insists that we access the attributes of an object by calling `set...` and `get...` methods. However, since these functions always pass parameters by value, we are always copying data.



Exercise 21.1 Testing multiple parents

Do It Yourself

Run the script in Listing 21.11 and observe the results carefully.

```
make the four nodes
A
B
C
D
make B and C children of A
make D a child of B
make D a child of C
get the children of B
blist =
    [1x1 NTNode]
D should be the first child
D
set the data in B's child
child of B
get the children of C
clist =
    [1x1 NTNode]
D should be the first child
D
if D was a child of B and C, this should say 'child of B' D
```

- In general, when we call the `setChildren(...)` method of `NTNode`, it is passed a copy of the cell array to be used as the children. By copying that cell array, we cannot avoid copying its contents.
- In particular, when setting the children of `b`, we create a copy of the original object, `d`. When setting the children of `c`, we make another copy of object `d`. So the tree produced is not a directed graph ending at `d`, but a normal N-ary tree with two leaf nodes, each of which is a copy of the original node, `d`.



21.9 Concluding Observations

As a final note, we might try to implement all these dynamic data structures not as MATLAB objects, but as simple structures with public functions instead of encapsulated methods to manipulate them. Although this approach allows us to create nodes such that we can have multiple parents of a single node, it is still unable to create graphs with cycles: it is not physically possible to code the circular assignments necessary to make `a` a child of `d` in the example of Exercise 21.1. Frequently, we suggest that such an effort is left as an exercise for the student; however, here it really is not a good use of your time. It is preferable to use an adjacency matrix to define a graph as we did in Chapter 17. If you wish to process your graphs with more efficient and fully encapsulated queues, replace the queues based on cell arrays in Chapter 17 with `Queue` or `Priority Queue` objects from Chapter 19.



Chapter Summary

In this chapter, we have seen the following:

- N-ary trees generalize the idea of binary trees by providing a list of children instead of just two fixed fields.
- In general, processing N-ary trees is best accomplished by mutual recursion—one function to process a tree node, and one to process a collection of children.
- Leaf nodes are easily detected as nodes where the number of children in the collection is zero.
- MATLAB algorithms can be written for all the normal operations except building, insertion, and deletion because there are few circumstances in which a tree can be dynamically manipulated.
- It should be possible to generalize N-ary trees by permitting cycles and multiple parents in their construction. However, since MATLAB cannot support references to objects, we must resort to the indirect adjacency matrix method described in Chapter 17.



Programming Projects

1. Write a recursive version of the N-ary tree depth-first string generator, and check that the same answers are achieved.
2. Write an iterative version of the N-ary tree `count(...)` method.
3. We want to determine who in the royal family tree was youngest when they ascended to the throne.
 - Extend the `Person` class used in the royal family tree to include the year at which this individual ascended to the throne. If the individual did not ascend to the throne, use `-1` for the year of ascension.
 - Change the comparison method in the `Person` class to compare ages at ascension to the throne, keeping in mind the `-1` value for those who did not reign.
 - You can determine the year of ascension for a monarch from the year of death of their predecessor on the throne. Add these values to the data set.

If you have done this correctly, you should be able to run the `largest(...)` method unchanged to find the monarch who was the youngest to ascend the throne.

The Cost of Computing

Introduction

A book about computing would be incomplete without summarizing what computer programs can and cannot do. As the chapters progressed, we discussed a number of data collections and a number of operations on those data collections. In Chapter 16 we also discussed Big O, the basic technique for evaluating the computing cost of an algorithm.

In this chapter, first we will summarize the computing cost of the classic operation types on the several data collections. Then we will discuss in general the subject of computability, and will conclude with a discussion of a very special class of problem—the **N-P Complete** problem.

- 22.1** Algorithms Operating on Collections
- 22.2** Algorithm Performance Categories
 - 22.2.1 Polynomial Algorithms
 - 22.2.2 $O(2^N)$
 - 22.2.3 $O(N!)$
- 22.3** Unreasonable Algorithms
- 22.4** Tractability of Problems
 - 22.4.1 Definitions
 - 22.4.2 Closed versus Open Problems
 - 22.4.3 More Definitions
- 22.5** N-P Complete Problems
 - 22.5.1 Yet More Definitions
 - 22.5.2 Illustrations
 - 22.5.3 The Opportunity

22.1 Algorithms Operating on Collections

Table 22.1 illustrates the Big O performance of the most common operations we have discussed on the common data collections. In summary, any traversal, mapping, folding, or filtering is $O(N)$ because all the data items have to be touched to perform these operations. Inserting (the critical component of building) into unstructured collections is usually $O(1)$ because it does not matter where in the collection the new data are placed. Searching is $O(N)$ unless the structure facilitates a binary search, and sorting is $O(N \log N)$ unless the structure cannot support random access, in which case insertion sort must be used with $O(N^2)$. The X symbols indicate that the operation is not defined for this collection.

Table 22.1 Performance of operations on collections

Operation	Array	Sorted Array	Structure	Linked List	Binary Tree	BST	N-Ary Tree	Graph
Traverse	N	N	X	N	N	N	N	X
Insert	1	N	1	1	1	Log N	X	X
Map	N	N	N	N	N	N	N	X
Filter	N	N	N	N	X	N	X	X
Fold	N	N	N	N	N	N	N	X
Search	N	Log N	X	N	N	Log N	N	N
Sort	N Log N	N Log N	X	N^2	X	X	X	X

22.2 Algorithm Performance Categories

As we discussed in Chapter 16 while presenting the Big O discussion, the performance of algorithms really does not matter for small amounts of data. However, in the information age in which we now live, we are confronted daily with enormous amounts of data, and computer scientists are busy polishing algorithms to process these massive amounts of data efficiently. For example, it is only in recent years that the telephone company has made national telephone directories searchable. Before, you had to know the area code before you could ask for a person's phone number because the national listing was too cumbersome to search.

Even with the best algorithms available, however, there are cases where exact solutions are not computable in reasonable amounts of time.

22.2.1 Polynomial Algorithms

We tend to think of algorithms where the Big O has N to a constant power as being "OK." However, consider some algorithm that is $O(N^5)$. With 256 items of data and a processor, the computer performs one operation in a

microsecond (10^{-6} sec); this algorithm therefore would take about 1.1 million seconds or 12.7 days, to complete. Probably something to avoid, but it could be worse.

22.2.2 $O(2^N)$

You probably heard the story of the peasant who did a good deed for the king. In return, the king, not being very bright, offered to do anything the peasant wanted. The peasant, being a bit too greedy for his own good, suggested that they take a chessboard and put a grain of rice on the first square (see Figure 22.1). Each day for the next 63 days, they would put on the next square double the number of grains as on the previous square. If one were to chart the number of grains of rice involved, it would look like the one shown in Table 22.2.

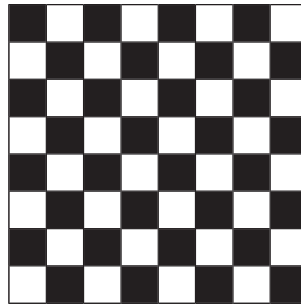


Figure 22.1 *The king and the peasant*

Table 22.2 2^N performance

Day	Grains
1	1
2	2
3	4
4	8
5	16
6	32
.	.
.	.
63	9,223,000,000,000,000
64	18,450,000,000,000,000

If you could produce a grain of rice every nanosecond (10^{-9} sec), it would take 585 years just to produce the grain for the last square.

(Of course, the king cut off the peasant's head when he realized...)

The Towers of Hanoi problem is also $O(2^N)$. Consider the child's puzzle toy shown in Figure 22.2. The objective is to move all the rings from peg A to peg C. However, they can only be moved one ring at a time, and you cannot put a larger ring on top of a smaller ring. Figure 22.3 illustrates the solution with three rings. When you realize what is happening, you notice that for each ring you add, you have to move all the smaller rings twice to get the new ring from A to B and then B to C— $O(2^N)$. So with 64 rings and moving one ring every microsecond, it would take 584,000 years to finish the puzzle.

But it could be worse...

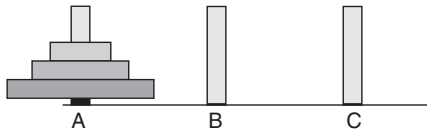


Figure 22.2 The Towers of Hanoi

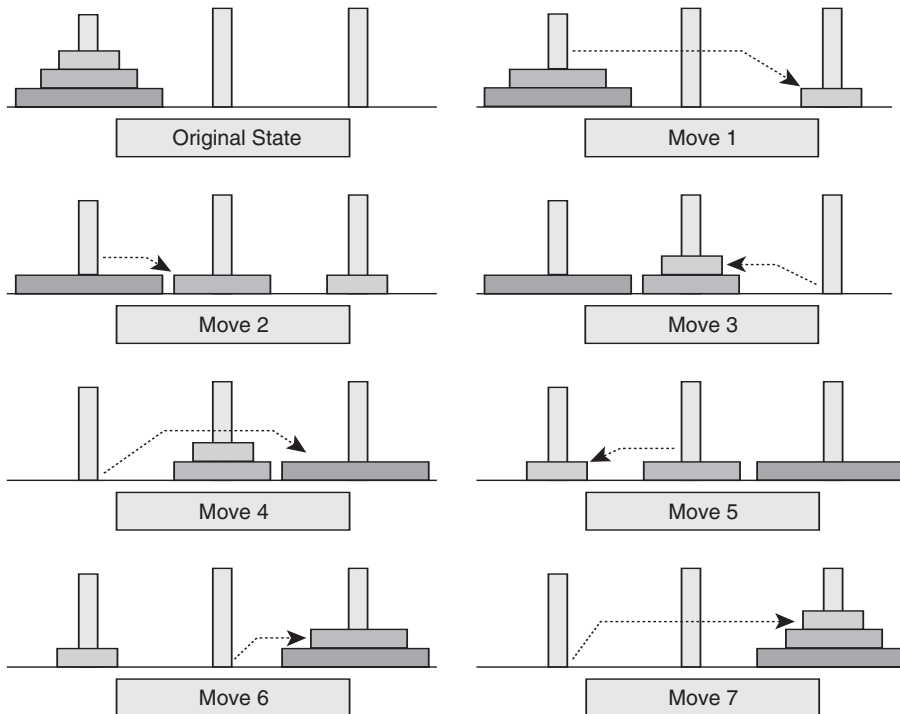


Figure 22.3 Towers of Hanoi solution

22.2.3 $O(N!)$

This is the sad story of our bathroom floor. Our interior decorator came across this new kind of tile with random patterns, as shown in Figure 22.4. The general idea is that you want to match the patterns at the tile edges. So we measured the bathroom floor and discovered we would need only 25 tiles to do the job. We bought a box of tiles and expected that we would have the bathroom retiled by dinner time. Fortunately, we decided not to spread the adhesive until we had the pattern figured out.

We picked the first tile at random, placed it in one corner, and went through the other 24 tiles looking for a match to the first edge. This repeated with the second, third, and fourth tiles down to the last one, and wouldn't you know it, the last one didn't fit! I had to start again with a new configuration. How many combinations are there to try? Yes, $N!$, and $25!$ works out to be 15,500,000,000,000,000,000,000,000 (roughly). So if I could try one combination every microsecond, I would be working at tiling the floor for 470 billion years!

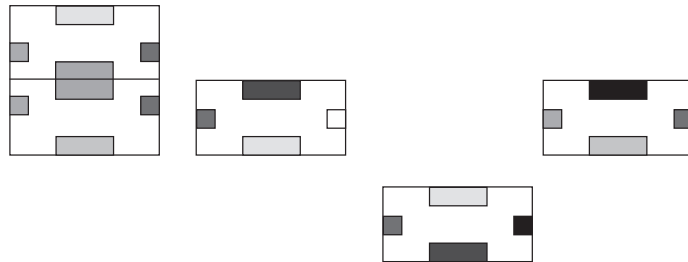


Figure 22.4 Random tile patterns



22.3 Unreasonable Algorithms

Although the computation time of polynomial algorithms (where the Big O has N raised to a constant power) can get quite nasty, we class them as **reasonable** algorithms. However, where N gets into the exponent in some form, we refer to the algorithms as exponential algorithms and classify them as **unreasonable**. While we don't really care about the runtime performance for very small amounts of data, the worst of the exponential algorithms take an enormous amount of time even on a really fast computer, as shown in Figure 22.5.

Would a faster computer help, or a large cluster of computers, or both? All these approaches merely improve the performance by linear amounts; if you really need to solve the kind of problem where these algorithms are invoked, you have to use sophisticated approximation techniques.

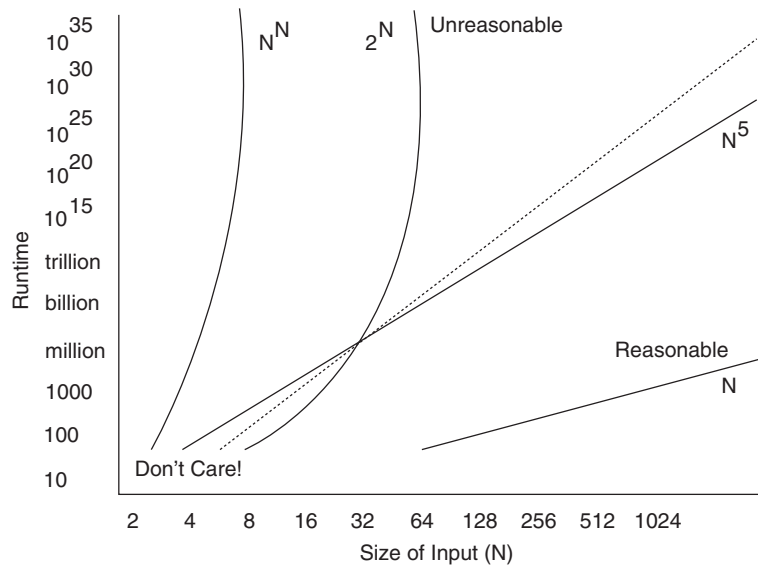


Figure 22.5 Algorithm categories

22.4 Tractability of Problems

Since algorithms are really only used to solve problems, we also need some way to assess the advisability of even starting to solve a problem whose solution might end up being unreasonable. We turn from the performance of algorithms to assessing the tractability of problems.

22.4.1 Definitions

First we consider two definitions:

- The **upper bound** of a problem is the Big O of the best algorithm that has ever been found to solve the problem completely
- The **lower bound** of a problem is the Big O of the best proof mathematicians can develop that the solution to this class of problem must be at least this complexity

For example, a trivial lower bound would be $O(1)$ because if you were smart enough, you could always write down the right answer immediately.

22.4.2 Closed versus Open Problems

There are some classes of problem where the upper and lower bounds have the same Big O value. For example, mathematicians are satisfied that the best sorting algorithm will be $O(N \log N)$. Since we already have sorting algorithms of $O(N \log N)$, this indicates that sorting is a closed problem—there is no room

for improving the state of the art. This does not mean that there will never be better sorting algorithms, because there will always be linear improvements, such as the progress from Merge Sort to Quick Sort. But the improvement will not affect the Big O.

There are other problems where the upper and lower bounds have not yet been met—they are still open for better algorithms or better proofs.

22.4.3 More Definitions

This leads us to the following definitions:

- A problem is said to be **tractable** if its upper and lower bounds are at worst polynomial
- A problem is said to be **intractable** if its upper and lower bounds are both exponential



22.5 N-P Complete Problems

These definitions leave an obvious hole. What do we do with problems whose upper bound is exponential (there is not yet a polynomial algorithm that solves the problem completely) yet whose lower bound is polynomial (the mathematicians cannot prove that the solution has to be exponential). This is in fact an enormous class of problems—perhaps larger than any other class of problems—referred to as N-P Complete problems.

22.5.1 Yet More Definitions

To understand this description of problems, we need a few more definitions:

- A solution is said to be **deterministic** if at each step of the way, there exists some logic indicating which choice to make without looking forward to the rest of the solution.
- A solution is said to be **non-deterministic** if no logic exists indicating which choice to make without looking forward to the rest of the solution.
- Perhaps the most remarkable attribute of this class of problem is that all these problems can be expressed in the same notation—statements in propositional calculus. This means that the problems are deemed to be **complete**, that is, transformable from one to another.

The phrase “N-P Complete” therefore describes that set of problems that is Non-Deterministic, should have a Polynomial solution, and is a **Complete** set.

22.5.2 Illustrations

The world of science and engineering is overflowing with N-P Complete problems that are being solved approximately every day:

- **Traveling salesman problem**—compute the shortest route for a traveling salesman to visit all of the cities in his territory
- **Map coloring problem**—find the algorithm that will color a given map with arbitrary boundaries using only four different colors with no two adjacent regions the same color¹
- **2-D arrangement problem**—cut out a random selection of parts from a sheet of metal with minimal waste
- **Bin packing problem**—arrange odd-shaped items as densely as possible in a given space
- **Planning problem**—schedule classes for the most efficient use of classroom space and teacher hours
- **Scheduling problem**—determine schedules for airline service or delivery trucks, minimizing the distance traveled to accommodate a specific passenger or cargo demand
- **Clique problem**—find the largest set of vertices in an undirected graph in which there is an edge between every pair of vertices

All of these problems and many more of their cousins are approximated every day of the week thousands of times. The inefficiency of the solutions to these problems is a permanent drain on the national economy.

22.5.3 The Opportunity

If you can lower the upper bound of one of the previous problems, or develop a proof that raises its lower bound, you will be famous overnight. They will name buildings after you.



Chapter Summary

In this chapter we have seen that there is a significant number of practical, everyday problems that are physically impossible to solve exactly; particularly, the class of problem described as N-P Complete. Algorithms beyond the scope of this text are used to develop and refine approximate solutions to these problems.

¹ This is easily done if you restrict the number of regions that can intersect at a point to four.

MATLAB Special Characters, Reserved Words, and Functions

Special Characters	Description	Discussed in This Section
%	Indicates a comment in an m-file	2.4.2
{ }	Defines a cell array	7.2.1
[]	The empty vector	3.3.4
[...]	Concatenates data, vectors, and arrays	3.3.1
()	Used to override operator precedence	3.3.5
()	Used to identify the formal and actual parameters of a function	5.3.2, 5.3.4
()	Used to index an array	3.3.3
(variable)	Used to allow a variable to be used as a structure field	7.3.1
'abc'	Encloses a literal character string	2.2.3, 6.2
'	Transposes an array	3.5.1
;	Separates rows in an array definition	3.5.2
;	Suppresses output when used in commands	2.4.5
:	Specifies a vector as from:incr:to	3.3.1
:	Used in slicing vectors and arrays	3.3.5
.	Used to access fields of a structure	7.3.1
...	Used to continue a MATLAB command to the next line	4.6.3
Mathematical Operators	Description	Discussed in This Section
=	Assignment operator—assigns a value to a memory location; not the same as an equality test	2.2.2
+	Scalar and array addition	3.3.5, 3.5.5
-	Scalar and array subtraction	3.3.5, 3.5.5
-	Unary negation	3.3.5, 3.5.5
*	Matrix multiplication	12.2.1
.*	Array multiplication	3.3.5, 3.5.5
/	Matrix division	12.2.2
./	Array division	3.3.5, 3.5.5
^	Matrix exponentiation	12.2.3
.^	Array exponentiation	3.3.5, 3.5.5

Logical Operators	Description	Discussed in This Section
<	Less than	3.3.5, 3.5.5
<=	Less than or equal to	3.3.5, 3.5.5
>	Greater than	3.3.5, 3.5.5
>=	Greater than or equal to	3.3.5, 3.5.5
==	Equal to	3.3.5, 3.5.5
~=	Not equal to	3.3.5, 3.5.5
&	Element-wise logical AND (vectors)	3.3.5, 3.5.5
&&	Short-circuit logical AND (scalar)	3.3.5, 3.5.5
	Element-wise logical OR (vectors)	3.3.5, 3.5.5
	Short-circuit logical OR (scalar)	3.3.5, 3.5.5
~	Unary not	3.3.5, 3.5.5
Logical Functions	Description	Discussed in This Section
all(a)	True if all the values in a (a logical vector) are true	4.3.3
and(a, b)	True if both a and b are true	4.3.3
any(a)	True if any of the values in a (a logical vector) are true	4.3.3, 17.2.4
not(a)	True if a is false; false if a is true	4.3.3
or(a, b)	True if either a or b is true	4.3.3
File Input and Output	Description	Discussed in This Section
csvread(file)	Reads comma-separated text files	8.3
csvwrite (file, data)	Writes comma-separated text files	8.3
dlmread (file, dlm)	Reads text files separated by the given delimiting character(s)	8.3
dlmwrite(file, data, dlm)	Writes text files separated by the given delimiting character(s)	8.3
fclose(fh)	Closes a text file	8.4.1
fgetl(fh)	Reads a line omitting the new-line character	8.4.2
fgets(fh)	Reads a line including the new-line character	8.4.2
fh = fopen(file, fl)	Opens a text file for reading or writing	8.4.1
fprintf(...)	Writes to the console, or to plain text files	8.3, 8.4.2
imread(file)	Reads an image file	13.3
imwrite(data, file, format)	Writes an image file	13.3
load	Loads the workspace from a file	8.2
save	Saves variables in a file	8.2
[tk rest] = strtok(str, dlm)	Extracts a token from a string and returns the remainder of the string	8.4.2

continued on next page

<code>ca = textscan(fh, format)</code>	Acquires and scans a line of text according to a specific format	8.3, 8.4.2
<code>[data Fs nb] = wavread(file)</code>	Reads a sound file in .wav format	14.3
<code>wavwrite(data, Fs, nb, file)</code>	Writes a sound file in .wav format	14.3
<code>[nums, txt, raw] = xlsread(file)</code>	Reads an Excel spreadsheet	8.3.2
<code>xlswrite(file, data, sheet, range)</code>	Writes an Excel spreadsheet in a specific row/column range	8.3.2

Format Control Characters	Description	Discussed in This Section
<code>%e</code>	Exponential notation	6.3.1
<code>%f</code>	Fixed point or decimal notation	6.3.1
<code>%g</code>	Fixed point or exponential notation	6.3.1
<code>%q</code>	A quoted string delimited by double quotes	6.3.1
<code>%s</code>	Character string	6.3.1
<code>\b</code>	Backspace	6.3.1
<code>\n</code>	Linefeed	6.3.1
<code>\t</code>	Tab	6.3.1

Display Formatting Commands	Description	Discussed in This Section
<code>format compact</code>	Sets format to compact form	help format
<code>format long</code>	Sets format to 14 decimal places	help format
<code>format long e</code>	Sets format to 14 exponential places	help format
<code>format loose</code>	Sets format back to default, non-compact form	help format
<code>format short</code>	Sets format back to default, 4 decimal places	help format
<code>format short e</code>	Sets format to 4 exponential places	help format

Workspace Management	Description	Discussed in This Section
<code>ans</code>	Default variable name for results of MATLAB calculations	2.2.2
<code>clc</code>	Clears the command screen	2.3.2
<code>clear</code>	Clears the workspace	2.4.2
<code>clf</code>	Clears the current figure	11.1.1
<code>close all</code>	Closes all graphics windows	11.1.1
<code>exit</code>	Terminates MATLAB	help exit
<code>help</code>	Invokes help utility	3.1
<code>load</code>	Loads the workspace from a file	8.2
<code>quit</code>	Terminates MATLAB	help quit
<code>save</code>	Saves variables in a file	8.2
<code>who</code>	Lists variables in the workspace	2.3.3
<code>whos</code>	Lists variables and their sizes	2.3.3

Special Constants	Description	Discussed in This Section
<code>eps</code>	Smallest possible difference between two floating point numbers	help eps
<code>false</code>	Logical false	3.2.3, 4.2
<code>inf</code>	Infinity	help inf
<code>NaN</code>	Not a number	8.3.2, 15.1.1
<code>pi</code>	Built-in mathematical constant	2.3.1
<code>true</code>	Logical true	3.2.3, 4.2

Basic Mathematical Functions	Description	Discussed in This Section
------------------------------	-------------	---------------------------

This is a partial list of the most commonly used functions. See MATLAB help for the complete list.

<code>abs(x)</code>	Computes the absolute value	help abs
<code>ceil(x)</code>	Rounds x to the nearest integer toward positive infinity	3.3.5
<code>cross(a, b)</code>	Vector cross product	3.4
<code>exp(x)</code>	Computes the value of e^x	help exp
<code>fix(x)</code>	Rounds x to the nearest integer toward zero	3.3.5
<code>floor(x)</code>	Rounds x to the nearest integer toward minus infinity	3.3.5
<code>log(x)</code>	Computes the natural log of x	help log
<code>log10(x)</code>	Computes the log base 10 of x	help log10
<code>mod(x, a)</code>	Computes the remainder when x is divided by a	help mod
<code>rem(x, a)</code>	Computes the remainder when x is divided by a	help rem
<code>round(x)</code>	Rounds x to the nearest integer	3.3.5
<code>sqrt(x)</code>	Calculates the square root of x	2.4.2

Trigonometry	Description	Discussed in This Section
--------------	-------------	---------------------------

This is a partial list of the most commonly used functions. See MATLAB help for the complete list.

<code>acos(x)</code>	Computes the inverse cosine (arcsine) of x	help acos
<code>asin(x)</code>	Computes the inverse sine (arcsine) of x	help asin
<code>atan(x)</code>	Computes the inverse tangent (arctan) of x	help atan
<code>atan2(y, x)</code>	Computes the inverse tangent given the x and y values (4 quadrant resolution)	help atan2
<code>cos(x)</code>	Computes the cosine of x	help cos
<code>sin(x)</code>	Computes the sine of x	help sin
<code>tan(x)</code>	Computes the tangent of x	help tan

Vector, Array, and Matrix Operations	Description	Discussed in This Section
<code>cumsum(v)</code>	Computes a cumulative sum of the values in v	15.3.2
<code>deal(...)</code>	Distributes cell array results among variables	7.2.2

continued on next page

<code>det(a)</code>	Computes the determinant of a matrix	help det
<code>diag(a)</code>	Extracts the diagonal from a matrix or (if provided with a vector) constructs a matrix with the given diagonal	3.5.2
<code>eye(n)</code>	Generates the identity matrix	12.2.1
<code>find(<logical a>)</code>	Computes a linear list of the locations of the true values in a logical array	3.2.5, 3.4.5
<code>fliplr(a)</code>	Flips a matrix from left to right	help fliplr
<code>inv(a)</code>	Computes the inverse of a matrix	12.2.3
<code>length(a)</code>	Determines the largest dimension of an array	3.3.2, 3.5.1
<code>linspace(fr,to,n)</code>	Defines a linearly spaced vector	3.3.1
<code>magic(n)</code>	Generates a magic square	3.5.2
<code>[v,in] = max(a)</code>	Finds the maximum value and its position in a	3.3.5
<code>mean(a)</code>	Computes the average of the elements in a	3.3.5
<code>meshgrid(x, y)</code>	Maps each of two vectors into separate two-dimensional arrays	11.4.1
<code>[v,in] = min(a)</code>	Finds the minimum value and its position in a	3.3.5
<code>ones(r, c)</code>	Generates an array filled with the value 1	3.3.1, 3.5.1
<code>prod(x)</code>	Product of all the items in x	help prod
<code>rand(r, c)</code>	Calculates an $r \times c$ array of evenly distributed random numbers in the range 0...1	3.3.1
<code>randn(r, c)</code>	Calculates an $r \times c$ array of normally distributed random numbers in the range 0...1	3.3.1
<code>size(a)</code>	Determines the dimensions of an array	3.3.2, 3.5.1
<code>sparse</code>	Defines a sparse matrix	17.2.3
<code>[v,in] = sort(v)</code>	Sorts the vector v (is a vector or a cell array of strings)	16.4.1, 16.5
<code>sum(a)</code>	Finds the sum of an array	3.3.5
<code>zeros(r, c)</code>	Builds an array filled with the value 0	3.3.1

2-D Plotting	Description	Discussed in This Section
<code>bar</code>	Generates a bar graph	11.2.4
<code>barh</code>	Generates a horizontal bar graph	11.2.4
<code>contour</code>	Generates a contour plot	11.4.5
<code>hist</code>	Generates a histogram	11.2.4
<code>loglog</code>	Generates an x-y plot, with both axes scaled logarithmically	11.2.4
<code>pie</code>	Generates a pie chart	11.2.4
<code>plot</code>	Creates an x-y plot	11.1.2, 11.2.1
<code>polar</code>	Creates a polar plot	11.2.4
<code>semilogx</code>	Generates an x-y plot, with the x-axis scaled logarithmically	11.2.4
<code>semilogy</code>	Generates an x-y plot, with the y-axis scaled logarithmically	11.2.4

3-D Plotting	Description	Discussed in This Section
<code>bar3</code>	Generates a three-dimensional bar graph	11.3.3
<code>barh3</code>	Generates a horizontal three-dimensional bar graph	11.3.3
<code>gplot</code>	Plot a graph	17.2.8
<code>mesh</code>	Generates a mesh plot of a surface	11.4.1
<code>meshc</code>	Generates a mesh plot of a surface with contours	11.4.5
<code>meshz</code>	Generates a mesh plot of a surface with a skirt	11.4.5
<code>meshgrid(r, c)</code>	Creates a plaid for 3-D plots	11.4.1
<code>peaks</code>	Creates a sample matrix used to demonstrate graphing functions	11.4.2
<code>pie3</code>	Generates a three-dimensional pie chart	11.3.3
<code>plot3</code>	Generates a three-dimensional line plot	11.3.1
<code>sphere</code>	Example function used to demonstrate graphing	11.4.5
<code>surf</code>	Generates a surface plot	11.4.1
<code>surfc</code>	Generates a combination surface and contour plot	11.4.5
<code>waterfall</code>	Generates a mesh plot of a surface with one skirt edge	11.4.1

Plot Appearance Control Character	Line Type	Discussed in This Section
<code>-</code>	Solid	11.2.1
<code>:</code>	Dotted	11.2.1
<code>-.</code>	Dash-dot	11.2.1
<code>—</code>	Dashed	11.2.1
<code>.</code>	Point	11.2.1
<code>o</code>	Circle	11.2.1
<code>x</code>	x-mark	11.2.1
<code>+</code>	Plus	11.2.1
<code>*</code>	Star	11.2.1
<code>s</code>	Square	11.2.1
<code>d</code>	Diamond	11.2.1
<code>v</code>	Triangle down	11.2.1
<code>^</code>	Triangle up	11.2.1
<code><</code>	Triangle left	11.2.1
<code>></code>	Triangle right	11.2.1
<code>p</code>	Pentagram	11.2.1
<code>h</code>	Hexagram	11.2.1

Control Character	Color	Discussed in This Section
<code>b</code>	Blue	11.2.2
<code>c</code>	Cyan	11.2.2

continued on next page

<code>g</code>	Green	11.2.2
<code>k</code>	Black	11.2.2
<code>m</code>	Magenta	11.2.2
<code>r</code>	Red	11.2.2
<code>y</code>	Yellow	11.2.2

Figure Control	Description	Discussed in This Section
<code>axis</code>	Freezes the current axis scaling for subsequent plots or specifies the axis dimensions	11.1.2
<code>figure</code>	Opens a new figure window	11.1.1
<code>grid off/on</code>	Turns the grid off or on	11.1.2
<code>hold off/on</code>	Instructs MATLAB to erase figure contents before adding new information or not	11.1.2
<code>legend(ca)</code>	Adds a legend to a graph	11.1.2
<code>shading <value></code>	Shades a surface plot with one color per grid section	11.1.2
<code>subplot(plts, n)</code>	Divides the graphics window up into sections available for plotting	11.1.3
<code>text(x,y,str)</code>	Adds a textbox to a graph	11.1.2
<code>title(str)</code>	Adds a title to a plot	11.1.2
<code>xlabel(str)</code>	Adds a label to the x-axis	11.1.2
<code>ylabel(str)</code>	Adds a label to the y-axis	11.1.2
<code>zlabel(str)</code>	Adds a label to the z-axis	11.1.2

Color Map Values	Description	Discussed in This Section
<code>autumn</code>	Colormap for surface plots and images	11.4.1
<code>bone</code>	Colormap for surface plots and images	11.4.1
<code>colorcube</code>	Colormap for surface plots and images	11.4.1
<code>cool</code>	Colormap for surface plots and images	11.4.1
<code>copper</code>	Colormap for surface plots and images	11.4.1
<code>flag</code>	Colormap for surface plots and images	11.4.1
<code>hot</code>	Colormap for surface plots and images	11.4.1
<code>hsv</code>	Colormap for surface plots and images	11.4.1
<code>jet</code>	Default Colormap	11.4.1
<code>pink</code>	Colormap for surface plots and images	11.4.1
<code>prism</code>	Colormap for surface plots and images	11.4.1
<code>spring</code>	Colormap for surface plots and images	11.4.1
<code>summer</code>	Colormap for surface plots and images	11.4.1
<code>white</code>	Colormap for surface plots and images	11.4.1
<code>winter</code>	Colormap for surface plots and images	11.4.1

String Operations	Description	Discussed in This Section
<code>disp(...)</code>	Displays matrix or text	3.3.5, 6.4.1
<code>fprintf(...)</code>	Prints formatted information	6.4.2
<code>input(...)</code>	Prompts the user to enter a value	6.3.2
<code>int2str(a)</code>	Converts an integer to its numerical representation	6.3.1
<code>num2str(a,n)</code>	Converts a number to its numerical representation with <code>n</code> decimal places	6.3.1
<code>sprintf(...)</code>	Formats a string result	6.3.1
<code>sscanf(...)</code>	Formatted input conversion	6.3.2
<code>strcmp(s1, s2)</code>	Compares two strings—returns <code>true</code> if equal	6.4.3
<code>strcmpi(s1, s2)</code>	Compares two strings without regard to case—returns <code>true</code> if equal	6.4.3
<code>textscan</code>	Scans a text string	8.3.1, 8.4.2
Time-Related Functions	Description	Discussed in This Section
<code>clock</code>	Determines the current time on the CPU clock	help clock
<code>etime</code>	Finds elapsed time	help etime
<code>pause</code>	Pauses the execution of a program, either until any key is hit or for a specified number of seconds	13.4.5
<code>tic</code>	Starts a timing sequence	help tic
<code>toc</code>	Stops a timing sequence	help toc
Numerical Methods	Description	Discussed in This Section
<code>diff(v)</code>	Computes the differences between adjacent values in a vector	15.4.2
<code>interp1</code>	Computes linear and cubic interpolation	15.1.1
<code>interp2</code>	Computes linear and cubic interpolation	15.1.1
<code>interp3</code>	Computes linear and cubic interpolation	15.1.1
<code>polyfit(x, y, n)</code>	Computes a least-squares polynomial	15.2.2
<code>polyval(c, x)</code>	Evaluates a polynomial	15.2.2
<code>spline(x, y)</code>	Spline interpolation	15.1.2
Program Control	Description	Discussed in This Section
<code>break</code>	A command within a loop module that forces control to the statement following the innermost loop	4.6.4, 4.7.4
<code>case</code>	A specific value within a <code>switch</code> statement	4.1, 4.4.1
<code>catch</code>	End of a suspect code block where the exception is trapped	9.4.3
<code>continue</code>	Skips to the end of the innermost loop, but remains inside it	4.6.4, 4.7.4

continued on next page

<code>else</code>	Within an <code>if</code> statement, begins the code block executed when the condition is false	4.2, 4.3.2
<code>elseif</code>	Within an <code>if</code> statement, begins a second test when the first condition is false	4.2, 4.3.2
<code>end</code>	Terminates an <code>if</code> , <code>switch</code> , <code>for</code> , <code>while</code> , or <code>catch</code> block.	4.1, 4.3.2, 4.4.1, 4.6
<code>end</code>	Last element in a vector	3.3.5, 3.5.5
<code>for var = v</code>	A code module repeated as many times as there are elements in the vector <code>v</code>	4.1, 4.6
<code>function</code>	Identifies an m-file as a function	5.3.2
<code>error(str)</code>	Announces an error with the string provided	9.4.3
<code>global var</code>	Defines a variable as globally accessible	5.3.8, 9.10
<code>if <exp></code>	Begins a conditional module—the following code block is executed if the logical expression <code><exp></code> is true	4.1, 4.3.2
<code>lasterror</code>	Provides a structure describing the environment from which an exception was thrown	9.4.3
<code>nargin</code>	Determines the number of input parameters actually supplied by a function's caller	5.3.4
<code>nargout</code>	Determines the number of output parameters actually requested by a function's caller	5.3.4
<code>otherwise</code>	Catch-all code block at the end of a <code>switch</code> statement	4.1, 4.4.1
<code>switch variable</code>	Begins a code module selecting specific values of the <code>variable</code> (must be countable)	4.1, 4.4.1
<code>try</code>	Begins a block of suspect code from which an exception might be thrown	9.4.3
<code>while <exp></code>	A code module repeated as long as the logical expression <code><exp></code> is true	4.1, 4.5

Data Class Operations	Description	Discussed in This Section
<code>char(...)</code>	Casts to a character type	6.2, 6.5
<code>class(obj)</code>	Determines the data type of an object	7.2.4, 17.1.3
<code>double(a)</code>	Casts to type <code>double</code>	6.2
<code>int8/16/32/64(a)</code>	Casts to integer type with the specified number of bits	help
<code>uint8/16/32/64(a)</code>	Casts to unsigned integer type with the specified number of bits	help
<code>isa(obj, str)</code>	Tests for a given data type	7.2.4, 17.1.3
<code>ischar(ch)</code>	Determines whether the given object is of type <code>char</code>	6.2.3, 7.2.4
<code>iscell(...)</code>	Determines whether the given object is a cell	7.2.4
<code>isempty(a)</code>	Tests for the empty vector <code>[]</code>	17.1.2
<code>islogical(...)</code>	Determines whether the given object is of type <code>logical</code>	7.2.4
<code>isnumeric(...)</code>	Determines whether the given object is of type <code>double</code>	7.2.4

continued on next page

<code>isspace(a)</code>	Tests for the space character	6.2.3
<code>isstruct(...)</code>	Determines whether the given object is a structure	7.2.4

Structure Operations	Description	Discussed in This Section
<code>fieldnames(str)</code>	Returns a cell array containing strings that are the names of the fields	7.3.1
<code>getfield(str,<fld>)</code>	Extracts the value of the field <fld>	7.4.3
<code>isfield(str, <fld>)</code>	Returns true if the string is a field in this structure	7.4.3
<code>str = rmfield(str, <fld>)</code>	Returns a copy of the given structure with a field removed	7.3.1
<code>str = setfield (str,<fld>, <value>)</code>	Constructs a structure in which the value of field <fld> has been changed	7.4.3
<code>struct(...)</code>	Constructs a structure from <fieldname> <value> pairs of parameters	7.3.2

The ASCII Character Set

Originally, the American Standard Code for Information Interchange (ASCII) defined a mapping whereby a specific set of characters was assigned the numerical values 0–127. This was sufficient to represent the number symbols, the lowercase and uppercase alphabet, and all the common punctuation marks. However, as the need arose to represent more international characters, this numerical range was inadequate, and the next 128 values were assigned to meet this need. There is no universal agreement on this second mapping. The following table shows the values used by MATLAB.

Then, more international issues arose with other complete alphabets, and the standard international character set now uses a 16-bit representation, of which the original ASCII set are the first 256.

	0	1	2	3	4	5	6	7	8	9
0										
10										
20										
30				!	"	#	\$	%	&	'
40	(	)	*	+	,	-	.	/	0	1
50	2	3	4	5	6	7	8	9	:	;
60	<	=	>	?	@	A	B	C	D	E
70	F	G	H	I	J	K	L	M	N	O
80	P	Q	R	S	T	U	V	W	X	Y
90	Z	[	\	]	^	_	`	a	b	c
100	d	e	f	g	h	i	j	k	l	m
110	n	o	p	q	r	s	t	u	v	w
120	x	y	z	{		}	~			
130										
140										
150										
160		¡	¢	£	¤	¥	¦	§	¨	©
170	*	«	¬	®	¯	°	±	²	³	
180	´	µ	¶	·	¸	¹	º	»	¼	½
190	¼	¿	À	Á	Â	Ã	Ä	Å	Æ	Ç
200	È	É	Ê	Ë	Ì	Í	Î	Ï	Ð	Ñ
210	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û
220	Ü	Ý	Þ	ß	à	á	â	ã	ä	å
230	æ	ç	è	é	ê	ë	ì	í	î	ï
240	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù
250	ú	û	ü	ý	þ	ÿ				

Internal Number Representation

There are two different techniques whereby most computers today store the values of numbers: integer and floating-point. Integer storage has the nice property that it represents the exact value of the number stored; floating-point storage only guarantees a certain number of digits of precision. There is an upper limit to the values that can be stored in both integer and floating-point form. However, significantly larger numbers can be stored in floating-point storage than in integer storage.

By default, MATLAB sets the storage of numbers to double-precision floating-point representation. However, operations like reading images into MATLAB present the large volume of data in the more compact unsigned integer form.

Integers

Integers are represented in computer memory by blocks of data bits of various sizes. Memory is allocated in 8-bit increments, usually referred to as bytes; therefore, it is not surprising that integer storage comes in the same size increments. For a given size, the values of the data bits are represented in two different ways—signed or unsigned. Normally, of course, we expect a number to have both positive and negative values, and when the number of bits is large, this does not seem to have much impact.

However, when a small number of bits is used to store a value, one of those bits must be used to show that the number is positive or negative. The range of numbers that can be stored is therefore reduced by one bit, a factor of 2. The following figure illustrates the internal storage of 8-bit unsigned and signed values.

Unsigned 8-Bit Integer																
Power of 2:	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0								
Value of bit is 1:	128	64	32	16	8	4	2	1								
Example:	<table border="1"><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr></table>								0	0	1	0	1	1	0	1
0	0	1	0	1	1	0	1									
			32		+8	+4		+1 = 45								
Signed 8-Bit Integer																
Power of 2:	sign	2^6	2^5	2^4	2^3	2^2	2^1	2^0								
Value of bit is 1:	-128	64	32	16	8	4	2	1								
Example:	<table border="1"><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr></table>								1	1	1	0	1	1	0	1
1	1	1	0	1	1	0	1									
	-128 + 64 + 32				+8	+4		+1 = -9								

Clearly, for 8 bits, the maximum value is 127 signed, or 255 unsigned. If this is not sufficient storage, numbers can be stored in 16-, 32-, or 64-bit words, with the corresponding increase in the maximum stored size.

Floating-Point Numbers

Floating-point numbers are stored in single precision (32 bits) or double precision (64 bits) using the IEEE 754 standard. As the name suggests, the storage format includes a mantissa and an exponent, each expressed internally in a manner similar to integer storage. The fixed size of the mantissa leads to the fixed amount of precision of each storage type. The `float` data type gives 7 significant decimal digits; the `double` data type gives 15 significant decimal digits.¹

For details of these storage types, search the Web for “IEEE 754 standard.” At the time of writing, there was a good explanation at:

http://www.geocities.com/SiliconValley/Pines/6639/docs/fp_summary.html

Parameters of Each Storage Type

The following table describes the most commonly used storage types available in MATLAB, their minimum and maximum values, and their equivalent names in C.

MATLAB Name	Size (Bytes)	Minimum Value	Maximum Value	C Name
<code>uint8</code>	1	0	255	<code>unsigned char</code>
<code>int8</code>	1	-128	127	<code>char</code>
<code>uint16</code>	2	0	65,535	<code>unsigned short</code>
<code>int16</code>	2	-32,768	32,767	<code>short</code>
<code>uint32</code>	4	0	4,294,967,295	<code>unsigned int</code>
<code>int32</code>	4	-2,147,483,648	2,147,483,647	<code>int</code>
<code>uint64</code>	8	0	18,446,744,073,709,551,615	<code>unsigned long int</code>
<code>int64</code>	8	-9,223,372,036,854,775,808	9,223,372,036,854,775,807	<code>long int</code>
<code>float</code>	4	$\sim -3.4\text{E}+38$	$\sim 3.4\text{E}+38$	<code>float</code>
<code>double</code>	8	$\sim -1.7\text{E}+308$	$\sim 1.7\text{E}+308$	<code>double</code>

¹ Note that although this seems to be a large amount of precision, you must always design your programs to preserve that precision. If, for example, you were to subtract two numbers almost equal in value, the precision of the result would be significantly worse than that of the original numbers.

Web Reference Materials

At the time of writing, the following Web sites were available for additional programming tutorials and examples.

MathWorks

http://www.mathworks.com/academia/student_center/tutorials/launchpad.html
http://www.mathworks.com/academia/student_center/tutorials/index.html#
http://www.mathworks.com/academia/student_center/homework/
http://www.mathworks.com/support/tech-notes/list_all.html

Georgia Tech College of Computing

http://www.cc.gatech.edu/classes/AY2004/cs1371_summer/
http://www.cc.gatech.edu/classes/AY2004/cs1371_spring/
http://www.cc.gatech.edu/classes/AY2002/cs7635_spring/tutorial/

Georgia Tech Electrical and Computer Engineering

<http://users.ece.gatech.edu/~bonnie/book/TUTORIAL/tutorial.html>
<http://users.ece.gatech.edu/~bonnie/book/TUTORIAL/tutorial.pdf>
<http://users.ece.gatech.edu/~bonnie/book/applets.html>
http://www.eedsp.gatech.edu/Information/MATLAB_User_Guide/index.shtml

Georgia Tech Aerospace Engineering

<http://www.ae.gatech.edu/~msmith/academic/ae2220/MATLAB.pdf>

Answers to True or False and Fill in the Blanks

Chapter 1

Answers to True or False

1. True.
2. False. Although Charles Babbage is usually credited with the design of the first computer, one could argue that the counting boards in use in 500 BC from which the abacus was derived would qualify as a computer design.
3. False. Operating systems arrived on the scene quite late in the development of computers. Before then, the computer ran one application that did all the work, and this is still possible today.
4. False. The driver is just a pluggable operating system component.
5. True. For a computer to be useful there has to be hardware to carry data to and from the processor.
6. True.
7. False. A solution solves the whole problem by assembling solutions to manageable subproblems. An algorithm is a series of steps to solve a small subproblem.

Answers to Fill in the Blanks

1. theoretical; practical
2. the Von Neumann architecture
3. virtual memory
4. utilities
5. application programs
6. logic
7. solutions to subproblems
8. states; states

Chapter 2

Answers to True or False

1. True.
2. False. Written correctly, algorithms can be generalized to solve a range of subproblems.
3. True. Both functional and object-oriented programs require procedural components to function on a processor.
4. True.
5. False. This is merely the assignment of the sum of x and y to z ; you cannot make any inference about the value of y from this expression.

6. False. Untyped languages merely leave the programmer free of needing to define the type of data. The CPU has to have information about the nature of each data item in order to process it correctly.
7. True.
8. False. In general, especially in MATLAB, the class of an item refers to its data type. The more restrictive definition combining the data type with the operations performed on it is an OOP restriction.
9. False. You must use scripts for permanent command storage.
10. True.
11. True. Clicking the icon to the left brings up the Document window.
12. False. Double-clicking a file name opens the file in the editor.
13. True.
14. True.
15. False. The asterisk indicates that the file has been changed since it was saved.
16. False. Comments appear only in the text of the script for human understanding of the logic.
17. False. Only the **(F5)** hot key saves before executing.

Answers to Fill in the Blanks

1. Abstraction
2. An algorithm
3. side effects
4. a numeric character
5. name; type
6. class; object; class
7. perform calculations; perform calculations
8. double-click; repeat the execution
9. name; current value; data type
10. double-click
11. double-click; variable name
12. automatically; MATLAB command
13. percent sign (%)
14. ignore; the end of the current line

Chapter 3

Answers to True or False

1. False. Homogeneous collections must consist of data of the same type. This could be `double`, `logical`, `char`, or any of the types you saw in this text.
2. True.
3. False. Because a column vector has more columns than rows, it returns the number of columns.
4. True. Regrettably, you can. This is the array linearization. Should you use this? No.
5. False. Either array can be a scalar quantity (a 1×1 array).
6. True, as long as the indices in the index vector do not exceed the dimensions of **A**.

7. False. The position of the values in the logical index vector correspond to the position of values in the vector being indexed. Longer logical index vectors are reaching beyond the end of the original vector.
8. True.

Answers to Fill in the Blanks

1. numerical value; position in the vector
2. starting value; increment; ending value
3. elements; true values
4. parentheses
5. have the same dimensions; a scalar
6. the empty vector, []; complete rows or columns
7. bad; logical difficulties; indexing; copy the rows and columns you want to keep

Chapter 4

Answers to True or False

1. False. Comments are colored green; keywords that control execution are colored blue.
2. False. The MATLAB editor inserts indentation only to clarify for the reader the flow of control in a script.
3. True. If the `if` statement has no `else` clause, or the `switch` statement has no `otherwise` clause and the data provided matches none of the specified cases.
4. True.
5. False. The result that invalidates all other `&&` expressions is `false`.
6. True. But you can still use `break` to exit the loop early.
7. False. But it ought to be. This is really bad programming practice.
8. False. The expression specifies the reason to stay in the loop.

Answers to Fill in the Blanks

1. key command words
2. `all(...)`
3. `otherwise`
4. `true`
5. variable; values of data
6. `for` or `while`; innermost containing

Chapter 5

Answers to True or False

1. False. Functions have access to all the system data and functions, and can also reach global data directly.
2. False. Although this ought to be True. MATLAB calls user-defined functions by the name of the m-file, and ignores the name specified there.
3. True.
4. False. Functions can be defined with no parameters required.
5. False. Any result for which a variable is not provided by the caller is ignored.
6. False. This is merely a convention suggested to clarify the source of their definition.

Answers to Fill in the Blanks

1. Procedural abstraction
2. actual; formal
3. vector-like container of variable names
4. Local Scope
5. first; first function; other auxiliary functions in the same file

Chapter 6**Answers to True or False**

1. False. Casting changes the way the computer views a piece of data without changing it.
2. True.
3. False. It can be represented within a string by inserting two successive quote marks: (' ').
4. False. MATLAB will automatically cast the string to its ASCII values first.
5. True. But they have to be explicitly converted to characters and concatenated into one string.
6. False. Unequal length strings are reported as not being equal.
7. True.

Answers to Fill in the Blanks

1. a special internal representation
2. characters; numbers; punctuation marks; 0–127
3. `uint8(...)`; each letter
4. format control string; value parameters
5. `if`; cannot
6. `char(...)`; pads them with blanks

Chapter 7**Answers to True or False**

1. False. None of the collective operations defined for numerical arrays can be applied to cell arrays or structures.
2. True.
3. True.
4. True.
5. False. It returns a new structure with the field and value removed.
6. True.
7. False. If `stra` is a structure array with the field `data`, the expression `{stra.data}` will extract all the values into a cell array.

Answers to Fill in the Blanks

1. extracted one at a time; replaced
2. arrays; containers
3. cell containing 42
4. `str.(field) = 42`
5. `struct(...)`; field name as a string; cell array of field contents

Chapter 8

Answers to True or False

1. True. Although the actual storage technique on a hard drive may have blocks of characters distributed randomly on its surface; the software that reads and writes the disk serializes the characters.
2. False. You save the variable names and their current values, not the programs that generated the data.
3. False. While some applications permit delimited strings to be embedded in comma- or tab-delimited files, MATLAB's readers read only numerical data into arrays.
4. False. You can use the qualifier 'a' to indicate that you will append to the end of an existing file.
5. True. If you read past the end of a file, a numerical -1 is returned.

Answers to Fill in the Blanks

1. values; organization
2. numerical values; double array; text data; cell array; both string and numerical data; cell array
3. numerical array; filled with zero
4. system dependent (for example, in the standard version on a PC, this is indicated by returning a file handle of -1)

Chapter 9

Answers to True or False

1. True.
2. False. If that function or any function it calls throws an exception, all the frames down to the function containing a `try ... catch` block are popped off the stack.
3. False. Tail recursive functions perform the math "on the way in."
4. True. But this is a bad practice.
5. False. MATLAB actually does not care what the name of the file is. When a function is called, MATLAB finds the function by file name and starts the first function in that file whatever it is called. Local functions in the file must be called from that first function.

Answers to Fill in the Blanks

1. an alternative technique
2. wrapper function; check for erroneous data
3. runtime errors; programming errors; bad data
4. any tests or setup; as a helper to the main function call
5. result; stand-alone recursive call

Chapter 10

Answers to True or False

1. True.
2. False. All the elements might fail a test you apply to determine whether to change them or not.

Appendix E Answers to True or False and Fill in the Blanks

3. False. Filtering might remove elements from the collection, but those that remain are not changed.
4. False. It really is folding because the two results are different attributes of the same element of the collection.
5. True. The `break` statement allows you to exit a `for` loop early; the code is a little obscure if written this way.
6. False. Sorting requires some criterion for deciding that one element must precede another—alphabetical order is a good example.

Answers to Fill in the Blanks

1. character of the data; basic operation(s) we are asked to perform
2. beginning with an empty collection; inserting elements one at a time
3. two or more collections
4. filter
5. folding
6. finding what you seek; failing to find it
7. traverse; writing

Chapter 11

Answers to True or False

1. True. If the `x` vector is omitted, `1:N` is assumed for the independent parameter, and if the `str` is omitted, a solid blue line is used.
2. False. To apply to a specific data plot, the enhancement functions must follow the plotting function.
3. False. Any area not provided with a plot remains blank.
4. False. The default line colors rotate through the active color map.
5. True.
6. False. Bodies of rotation, for example, use one of the axis directions as an independent parameter.
7. False. It is the `x` and `z` axes (those axes not the axis of rotation).
8. False. The curve does not need to be continuous.
9. False. You can rotate the data to align an arbitrary axis with the `x`-axis, perform the body of rotation there, and invert the rotation.

Answers to Fill in the Blanks

1. new figure; the next higher
2. `clf`; `close all`
3. dependent; separate; independent
4. `z`; an `x-y` plaid
5. plaid; angles
6. linear curve; specified axis

Chapter 12

Answers to True or False

1. True. Even the most primitive data members encapsulate their data and control the operations that can be performed on the data.
2. False. It also works if one or both of the matrices are scalar.

3. True, only if one is a scalar; otherwise, False. If **A** and **B** are not square, one will fail; even if they are square, they will have different answers.
4. True.
5. False. It is equivalent to back dividing: $\mathbf{A} \setminus \mathbf{B}$.
6. True.

Answers to Fill in the Blanks

1. data; procedural
2. column; row
3. translate the object; perform the rotation; translate the rotated object back to **P**
4. **N** independent; **N** unknown; $\mathbf{N} * (\mathbf{N} + 1)$ constant

Chapter 13

Answers to True or False

1. False. 0 is the absence of light, which will give a black screen.
2. True. `imread(...)` can be adapted to read any supported image file, returning different results for different image styles.
3. True.
4. False. For two reasons—it only works for 2-D arrays, and the rotation is counter-clockwise.
5. True. Consider Figure 13.18. The original number of $1600 \times 1200 \times 3$ pixels has been reduced to a smattering of pixels of interest.

Answers to Fill in the Blanks

1. three; `uint8`; 0–255
2. pixel; `uint8`
3. decoded; true color; three color layers.
4. figure window; `image(...)`
5. Cropping; shrinking; stretching
6. transpose; major diagonal

Chapter 14

Answers to True or False

1. True.
2. False. Removing samples raises the frequency.
3. False. The number of bits in the recording have no significant effect on an untrained ear.
4. True.
5. True.

Answers to Fill in the Blanks

1. amplitude; frequency
2. **A/D**; **D/A**; pressure
3. 7; 5; 2; 12
4. sound energy; frequency band

Chapter 15

Answers to True or False

1. True. The provision is that while the linear interpolation does not give an error, it returns NaN for data points that are out of range.
2. True. There is a unique cubic parametric curve between each pair of points. The curve is parametric rather than a function of the independent variable in order to permit the curve to “double back” if necessary for smoothness.
3. False. All curve fitting does is provide the coefficients of a polynomial. You can insert any value of the independent variable.
4. True.
5. True. Simpson’s rule better captures fluctuations in the function being integrated.
6. False. The `diff(...)` function shortens the vector by one element.
7. False. The numerical technique presented here will find the zeros of any data set, regardless of its origin.

Answers to Fill in the Blanks

1. Interpolation
2. coefficients of the equation; sum of the squared distances
3. central difference approximation; slope
4. `cumsum(...)`; dot product; the data vector; a vector of time differences
5. critical points; differentiating; finding the zero crossings

Chapter 16

Answers to True or False

1. True. First we perform all the algebra to reduce the compound expression to a sequential series of $O(\dots)$ values, and then we add them and remove any terms that increase more slowly with N than other terms.
2. False. Linear search algorithms are $O(N)$, but binary search is $O(\log N)$.
3. True. Mathematicians have proven that one cannot sort with a faster Big O. However, better algorithms can provide a constant multiplier improvement.
4. False. Most of them have some kind of optimization that reduces the length of the minor passes.
5. True. Since Big O should reflect the worst case performance, and quick sort on a sorted collection is $O(N^2)$.

Answers to Fill in the Blanks

1. Big O; work; data
2. N
3. Merge; quick
4. sorted vector; set of indices; numerical value

Index

Symbols

`./` (array division) 56, 77, A-1
`.^` (array exponentiation) 56, 77, 297, A-1
`.*` (array multiplication) 54, 56, 77, A-1
`=` (assignment operator) 22, A-1
`\` (backslash) 132, 297
`{ }` (braces) 149, 167, A-1
`:` (colon operator) 58, 77, A-1
`.` (dot) 160, 167, A-1
`&` (element-wise AND) 55, 56, 78, A-2
`|` (element-wise OR) 55, 56, 78, A-2
`...` (ellipses) 37, 142, A-1
`[ ]` (empty vector) 49, A-1
`==` (equal to) 56, 77, A-2
`>` (greater than) 56, 77, A-2
`>=` (greater than or equal to) 56, 77, A-2
`<` (less than) 56, 77, A-2
`<=` (less than or equal to) 54, 77, A-2
`\` (matrix back division) 314
`/` (matrix division), *see* matrix division
`^` (matrix exponentiation) 297, 314, A-1
`*` (matrix multiplication), *see* matrix multiplication
`-` (minus sign) 54, 77, A-1
`~=` (not equal to) 56, 77
`( )` (parentheses) 77, 121, A-1
`%` (percent sign) 35, 132, A-1, A-3
`+` (plus sign) 77, A-1
`'` (quote mark) 23, 63, 77, A-1
`;` (semicolon) 30, 32, 37, 64, 77, A-1
`&&` (short-circuit AND) 55, 56, 78, 91, A-2

`||` (short-circuit OR) 55, 56, 78, 91, A-2
`~` (unary not) 56, 78, A-2
`_` (underscore character) 22
Numbers
2-D plots/plotting 252–258
 enhancing 253–255
 parametric plots 256–257
 rotated 258
 simple plots 252–253
2-D rotation 300–304
3-D plots/plotting 258–261, *see also* surface plots
3-D rotation 304–307
4GLs 11
A
 `abs()` function A-4
 abstraction 19, 48, 112, 294
 accessing
 arrays 65–66
 structures 153–155, 157–160
 `acos()` function, A-4
 `acosd()` function 199
 activation stack 192–193
 actual parameters 115
 A/D (analog-to-digital) devices 348–349
 algorithms 19–20, *see also* sorting
 analyzing complex 414–415
 measuring cost of 412–415
 performance analysis of 425–426
 `all()` function 91, 105, A-2
 `alpha()` function 276, 288
 ALU (Arithmetic and Logic Unit) 5
 American Standard Code for Information Interchange (ASCII) 128, B-1–B-2
 ampersand (`&`) 55, 56

amplitude 348
analog sound, digital vs. 348
analog-to-digital (A/D) devices 348–349
AND
 element-wise (`&`) 55, 56, 78, A-2
 short-circuit (`&&`) 55, 56, 78, 91, A-2
`and()` function 105, A-2
`ans` variable 27, A-3
Ansari X Prize 39
`any()` function 91, 105, A-2
API (Application Programmer Interface) 114
apostrophe (`'`) 63, 77
Application Programmer Interface (API) 114
arbitrary axis, rotations about an 276
Arithmetic and Logic Unit (ALU) 5
arithmetic operations
 with arrays 66–67
 with strings 130
 with vectors 54–55
array division (`./`) 56, 77, A-1
array exponentiation (`.^`) 56, 77, A-1
array multiplication (`.*`) 54, 56, 77, A-1
arrays 62–76
 accessing elements of 65–66
 arithmetic operations with 66–67
 cell 148–152
 concatenation of 69
 creating 64–65
 example using 74–76
 inserting data into 222
 library functions with 68–69
 linear 49
 linearized 70–72
 logical operations with 67–68

- arrays (*cont.*)
 - matrices vs. 63
 - operating on 66–74
 - properties of 63–64
 - removing elements of 66
 - reshaping of 70
 - slicing of 69
 - of strings 137
 - structure 156–157
- ASCII, *see* American Standard Code for Information Interchange
- `asin()` function A-4
- assemblers 13
- assembly language 11
- assignment operator (=) 22, A-1
- assignment statements 22
- `atan()` function A-4
- `atan2()` function A-4
- AU (file extension) 176
- .au files 349
- audio files 176
- `auread()` function 176, 349, 374
- `auwrite()` function 176, 374
- auxiliary (local) functions 117
- AVI (file extension) 176
- `aviread()` function 176
- `axis()` function 248, 287, A-7
- B**
 - \b (format control character) A-3
- Babbage, Charles 3
- back, inserting at the 222
- back dividing 297, 299, 314
- backslash (\) 132
- backups 10
- backward difference approximation 395–396
- `bar()` function 257, 287, A-5
- `bar3()` function 261, 288, A-6
- `barh()` function 257, 287, A-5
- `barh3()` function 261, 288, A-6
- Basic Input/Output System (BIOS) 7
- `before()` function 223
- behavioral abstraction 294
- Big O algebra 412–415
 - $O(1)$ (independent of N) 413
 - $O(2^N)$ (exponential growth) 414
 - $O(\log N)$ (binary search) 413–414
 - $O(N)$ (linear with N) 413
 - $O(N^2)$ (proportional to N^2) 414
- binary files 173
- binary search ($O(\log N)$) 413–414, 425–426
- binary strings 11
- BIOS (Basic Input/Output System) 7
- bits 5–6
- black box view (of a function) 112–113
- bodies of rotation 272–276
 - arbitrary axis, rotation about 276
 - continuous functions, rotating 272–274
 - discrete functions, rotating 274–276
- `break` command 100, 104, A-8
- break points 38
- breaking
 - for loops 98
 - while loops 101
- brush tool 279
- bubble sort 417–419, 429
- bucket sort 424–425, 429
- bugs 12
- building (operation) 221, 226–229
- building a collection 226–229
- built-in functions 48
- bytes 6
- C**
 - C (programming language) 12
- cache 7
- CAD (computer-aided design) 10
- calling (functions) 115–116
- case keyword 92, 93, 104, A-8
- casting 128–130
- catch keyword 198, 199, 215, A-8
- CDA (Common Data Format) 176
- CDF (file extension) 176
- `cdfread()` function 176
- `cdfwrite()` function 176
- `ceil()` function 57, 78, A-4
- cell arrays 148–152
 - accessing 149–151
 - creating 148–149
 - extracting/sorting 427
 - inserting items into 222
 - processing 151–152
 - using 151
- central difference approximation 396
- Central Processing Unit (CPU) 4, 5, 7, 192
- ceramic composition (example) 311–312
- `char()` function 130, 137, 142, A-9
- character generators 127
- character mapping 128
- character strings 127–142
 - arithmetic operations on 130
 - arrays of 137
 - and casting 128–130
 - comparison of 135–137
 - concatenation of 130
 - conversion from numbers to 131–132
 - conversion of, to numbers 132–134
 - data output from 134–135
 - and delimiters 129
 - example using 138–142
 - format control strings 131–132
 - functions for analysis of 131
 - logical operations on 130
 - and mapping 128
 - as numerical values 128
 - slicing of 130
 - and tokens 129
- `class()` function 152, 167, A-9
- classes 25
- `clc` command 27, 28, A-3
- `clear` command 36, A-3
- `clf` command 287, A-3
- `clock()` function, A-8
- `close all` command 287, A-3
- closing files 180
- code blocks 85, 86
- `coef()` function 387
- collages, creating 330–333
- collections of data, *see* data collections
- colon (:) operator 58, 77, A-1
- color indicators, A-6–A-7
- color map values, A-7
- color mapped images 321–322
- color masking 325–330
- `colormap()` function 248, 287
- Colossus computer 3–4
- column vectors 64
- Command History window 27–28
- Command mode 26
- Command window 26–27, 37

comments 35, 38
 Common Data Format (CDA) 176
 comparison (of character strings) 135–137
 comparison operators 56
 compile errors (compile-time errors) 12
 compilers 12
 compound surfaces, assembling 277–279
 computer architectures, history of 3–5
 computer languages 2–3, 11–12
 computer-aided design (CAD) 10
 concatenation
 of arrays 69
 of sounds 350–353
 of strings 130
 of vectors 57–58
 of words 22
 conditional execution 86–87
 continue statement 100, 104, A-8
 continuous functions, rotating 272–274
 contour() function 276, 288, A-5
 Control Unit 5
 conversion
 from numbers to strings 131–132
 from strings to numbers 132–134
 cos() function A-4
 CPU, *see* Central Processing Unit
 critical points 209–210
 cross() function 78, A-4
 CSV (file extension) 176
 csvread() function 176, 186, A-2
 csvwrite() function 176, 186, A-2
 cube
 drawing a 262–264
 rotating a 305–307
 cubic spline interpolation 382–384
 cumsum() function 393, 404, A-4
 Current Directory window 31–32, 36
 curve fitting 309–311, 384–390
 example of 389–390
 with linear regression 385–386
 with polynomial regression 387–388
 cylinder, constructing a 269–270
 cylinder() function 114–117, 276, 288

D

D/A (digital-to-analog) devices 349
 data
 linking plots to 280–282
 spreadsheet 184–186
 data abstraction 19, 48, 294
 data buses 6
 data collections 48
 building 226–229
 filtering 230–232
 mapping 229–230
 searching 234–235
 sorting 235
 summarizing 232–234
 traversing 223–226
 data typing 23–25
 database management 10
 deal() function 149, 151, 167, A-4
 debugging 12, 38
 defragmentation 10
 delimited text files 175, 178–179
 delimiters 129, 174
 derivative 394
 design templates 87–88
 for functions 113–114
 for if statements 88
 for switch statements 92
 desktop publishing 10
 det() function A-5
 diag() function 64, 78, A-5
 diagonal arrays 64
 diff() function 396, 404, A-8
 difference engine 3, 4
 differentiation 394–397
 digital sound, analog vs. 348
 digital-to-analog (D/A) devices 349
 discrete functions, rotating 274–276
 disp() function 60, 78, 134–135, 143, A-8
 display formatting commands, A-3
 dissimilar objects, collecting 148
 division
 array ./ 56, 77, A-1
 matrix 297, 299–300, 307–311
 DLM (file extension) 176
 dlmread() function 176, 179, 186, A-2
 dlmwrite() function 176, 179, 186, A-2
 <documentation> section 113
 dot (.) notation 160, 167

dot operator 67

double() function 130, 142, A-9
 double storage type, C-2
 drivers 8

E

%e (format control character) A-3
 edges, detecting 339–342
 edit command 34
 Edit mode 26
 editing (of plots) 250
 Editor window 33, 34
 electrical circuit analysis 313–314
 elements
 of arrays 65–66
 of vectors 49
 element-wise AND (&) 55, 56, 78, A-2
 element-wise OR (|) 55, 56, 78, A-2
 ellipses (...) 37, A-1
 ellipsoid() function 276, 288
 else keyword 86, 87, 104, A-9
 elseif keyword 86, 88–89, 104, A-9
 embedded computers 8
 empty vector ([]) 49, A-1
 encapsulation 112, 117–118
 encryption (example) 138–142
 end() function 78, 104
 end keyword 86, 198, A-9
 endless recursion 193
 engineering applications
 ceramic composition 311–312
 edge detection 339–342
 electrical circuit analysis 313–314
 encryption 138–142
 forces and moments 61–62
 geographic data, visualizing 283–286
 geopolitical data, processing 238–243
 images, cleaning up 402–403
 liquid levels, computation of 101–103
 oil rig structural integrity 371–374
 robot arm motion 211–215
 rocket data, analyzing 397–402
 soil volume, computation of 74–76
 solid object, measurement of 119–120
 sorting 430–432

- engineering applications (*cont.*)
 - spacecraft launch 39–41
 - spreadsheet data 184–186
 - structure, assembling a 162–168
- eps command A-4
- equal to (=) 56, 77, A-2
- erroneous commands, correcting 26
- error() function 198, 215, A-9
- etime() function A-8
- even() function 205
- Excel spreadsheets 177–178
- exceptions 196–200
 - generic implementation for 197–198
 - historical approach to 197
 - MATLAB implementation for 198–200
- execution 12
 - conditional 86–87
 - errors in 12
- exit command A-3
- exp() function A-4
- exponential growth ($O(2^N)$) 414
- exponentiation
 - array 56, 77
 - matrix 297
- Extended Markup Language (XML) 176
- extensions, file 176
- extrapolation 384
- eye() function 296, 314, A-5
- F**
 - %f (format control character) A-3
 - fact() function 201
 - false values 52, 78, 86–87, 90, A-4
 - Fast Fourier Transform (FFT) 359–363
 - about 359–360
 - simple spectral analysis using 361–363
 - fclose() function 180, 187, A-2
 - FFT, *see* Fast Fourier Transform
 - fft() function 361, 374
 - fgetl() function 180–181, 187, A-2
 - fgets() function 180, 187, A-2
 - fh (file handle) 180, A-2
 - Fibonacci series 207–208
 - fieldnames() function 153, 159, 160, A-10
 - figure() function 248, 287, A-7
 - file extensions 176
 - files, *see also* Input/Output (I/O)
 - binary 173
 - delimited text 175, 178–179
 - opening/closing 180
 - reading text files 180–183
 - writing text files 183
 - fill() function 257, 287
 - filtering (operation) 221, 230–232
 - find() function 55, 70, 78, A-5
 - FITS (file extension) 176
 - fitsread() function 176
 - fix() function 57, 78, A-4
 - flash cards 7
 - fliplr() function A-5
 - float storage type C-2
 - floating-point numbers, internal representation of C-2
 - floor() function 57, 78, A-4
 - flowcharts 87
 - folding (operation) 221, 232–234
 - fopen() function 180, 187, A-2
 - for loops 95–98
 - breaking 98
 - general template for 95
 - indexing implementation using 97–98
 - MATLAB implementation using 96–97
 - while loops vs. 94
 - for var A-9
 - forces, using vectors to calculate 61–62
 - formal parameters 115
 - format command 30, A-3
 - format compact A-3
 - format control strings 131–132
 - format long A-3
 - format long e A-3
 - format loose A-3
 - format short A-3
 - format short e A-3
 - forward difference approximation 395–396
 - Fourier transform 359–360, *see also* Fast Fourier Transform (FFT)
 - fprintf() function 135, 143, 176, 187, A-2, A-8
 - frequency 348, 354–359
 - front, inserting into the 222
 - function keyword 121, A-9
 - <function name> section 113
 - functional programming 20
 - functions (in general) 112–113
 - black box view of 112–113
 - defined 112
 - derivative of 394
 - instances of 193
 - integral of 390–391
 - zeroes of 208–211
 - functions (in MATLAB) 48, 113–120
 - auxiliary (local) 117
 - calling 115–116
 - constructor 155–156
 - definition, function 113–114
 - encapsulation in 117–118
 - example using 119–120
 - and global variables 118–119
 - returning multiple results from 116–117
 - storing and using 115
- G**
 - %g (format control character) A-3
 - Gaussian Elimination 297
 - general-purpose computers 8
 - generations (of computer languages) 11
 - generative recursion 205
 - geographic data, visualizing 283–286
 - geopolitical data, processing 238–243
 - getfield() function 160, 167, A-10
 - global keyword 118
 - Global Scope 117–118
 - global var keyword 121, 215, A-9
 - global variables 118–119
 - goto statement 100
 - gplot() function A-6
 - Graphics window 33
 - graphs, creating 33–34
 - gray scale images 321
 - greater than (>) 56, 77, A-2
 - greater than or equal to (>=) 56, 77, A-2
 - grid off 249, 287, A-7
 - grid on 249, 287, A-7
 - gt() function 418, 420
- H**
 - half note steps 354
 - hardware, computer 5–6

- hardwiring 6
- HDF (file extension) 176
- hdfread() function 176
- heap 8
- help command 48, 78, 114, 121, A-3
- heterogeneous collections 148
- high-level I/O functions 175–179
 - with delimited text files 178–179
 - with Excel spreadsheets 177–178
- high-level languages 11
- hist() function 257, 287, A-5
- hold off 249, 287, A-7
- hold on 249, 287, A-7
- homogeneous collections 48
- I**
- identity matrix 296
- if statements 87–92, 94, 136, A-9
 - general template for 88
 - in logical expressions 90–91
 - MATLAB implementation using 88–90
 - and short-circuit evaluation 91–92
- ifft() function 361, 374
- image() function 323, 343
- images 319–342
 - about 320
 - cleaning up 402–403
 - color mapped 321–322
 - color masking with 325–330
 - creating a collage from 330–333
 - creating a kaleidoscope from 333–336
 - displaying 323
 - examples using 339–342, 389–390, 402–403
 - gray scale 321
 - JPEG 322
 - operating on 323–339
 - reading 323
 - resolution of 320
 - stretching/shrinking 324–325
 - on a surface 336–339
 - true color 321
 - writing 323
- imread() function 176, 323, 343, A-2
- imshow() function 323, 343
- imwrite() function 176, 323, 343, A-2
- indexing (of vectors) 50–52
- indexing a vector (term) 51
- inf A-4
- in-line coding 201
- inner dimensions 295
- input 6
- input() function 93, 104, 133–134, 143, 200, A-8
- Input/Output (I/O) 174–186
 - example 184–186
 - high-level 175–179
 - lower-level 180–183
 - and MATLAB workspace 174–175
- Input/Output (I/O) devices 4–6
- inserting data (into a collection) 221–223
- insertion sort 415–417, 429
- instances, function 193
- instrument sounds, analyzing 364–366
- int2str() function 131, 142, A-8
- int8/16/32/64 storage type A-9, C-2
- integers, internal representation of, C-1
- integral 390–391
- integration 390–394
- interactive environment 13
- interp1() function 381, 384, 404, A-8
- interp2() function 382, 404, A-8
- interp3() function 382, 404, A-8
- interpolation 380–384
 - cubic spline 382–384
 - and extrapolation 384
 - linear 380–382
- intersecting lines 308–309
- inv() function 297, 299, 314, A-5
- I/O, *see* Input/Output
- I/O devices, *see* Input/Output devices
- isa() function 152, 167, A-9
- iscell() function 152, 167, A-9
- ischar() function 131, 142, 152, 167, A-9
- isempty() function A-10
- isfield() function 160, 167, A-10
- islogical() function 152, 167, A-9
- isnumeric() function 152, 167, A-9
- isspace() function 131, 142, A-10
- isstruct() function 152, 167, A-10
- iteration
 - about 94–95
 - example using 101–103
 - with for loops 95–98
 - with while loops 98–101
- J**
- JPEG files/images 322, 325, 330
- K**
- kaleidoscope, creating a 333–336
- Klein bottle 278–279
- L**
- lasterror() function 199, 215, A-9
- Launch Pad window 25
- least squares technique 386
- legend() function 249, 287, A-7
- length() function 50, 60, 63, 78, A-5
- less than (<) 56, 77, A-2
- less than or equal to (<=) 54, 77, A-2
- library functions
 - with arrays 68–69
 - with vectors 57
- light() function 337, 343
- lightangle() function 276, 288
- line(s)
 - intersecting 308–309
 - rotating a 301–302
- linear arrays 49
- linear equations, simultaneous 307–311
- linear interpolation 380–382
- linear matrices 49
- linear regression 385–386
- linearized arrays 70–72
- linkers 12
- linspace() function 49, 78, 324, 343, A-5
- liquid levels, computation of 101–103
- load() function 186, A-2, A-3
- loaders 12
- local (auxiliary) functions 117
- Local Scope 118
- log() function A-4
- log10() function A-4
- logic errors (logical errors) 12, 24
- logical expressions 90–91
- logical indexing 52

- logical operations
 - with arrays 67–68
 - with strings 130
 - with vectors 55–56
- loglog() function 257, 287, A-5
- loop-and-a-half iteration style 100–101
- lower-level I/O functions 180–183
 - opening/closing files 180
 - reading text files 180–183
 - writing text files 183
- lt() function 416, 420
- M**
 - .m extension 35
 - machine languages 11
 - magic() function 64, 78, A-5
 - mainframes 8
 - mapping, character 128
 - mapping (operation) 221, 229–230
 - material keyword 337, 343
 - mathematical computation
 - programs 11
 - MATLAB
 - about 13, 17–18
 - starting/stopping 21
 - user interface with 25–34
 - matrix-(ces) 293–314
 - arrays vs. 63
 - examples using 311–314
 - identity 296
 - inverses of 297
 - linear 49
 - operations on 294–300
 - matrix back division (\) 297, 299, 314
 - matrix division (/) 299–300, A-1
 - about 297
 - for solving simultaneous linear equations 307–311
 - matrix exponentiation (^) 297, 314, A-1
 - matrix multiplication (*) 54, 297–298, A-1
 - for 2-D rotation 300–304
 - for 3-D rotation 304–307
 - about 294–296
 - scalar vs. 296
 - max() function 57, 68, 78, A-5
 - mean() function 57, 68, 78, A-5
 - mechanical memory 6–7
 - memory, computer 6–8
 - Mercator projection 336
 - merge sort 422–424, 429
 - mesh() function 262, 288, A-6
 - meshc() function 276, 288, A-6
 - meshgrid() function 262, 264, 276, 288, A-5, A-6
 - meshz() function 276, 288, A-6
 - m-files 35
 - min() function 57, 68, 78, A-5
 - minus, unary (–) 54, 77, A-1
 - mod() function A-4
 - modes 26
 - moment (of a force) 61–62
 - movie files 176
 - multiplication
 - array 54, 56, 77
 - matrix, *see* matrix multiplication
 - musical sounds 353–359
 - about 353–354
 - changing frequency of 354–359
 - mutual recursion 205
 - N**
 - \n (format control character) A-3
 - NaN keyword 178, 186, 279–280, 404, A-4
 - nargin() function 116, 121, A-9
 - nargout() function 116, 121, A-9
 - natural languages 12
 - Newman, Max 4
 - Newton’s method 210
 - not equal to (~ =) 56, 77
 - not() function 105, A-2
 - Notepad 176
 - notes, musical 354
 - num2str() function 131, 143, A-8
 - numbers
 - conversion from, to strings 131–132
 - conversion from strings to 132–134
 - numerical indexing 51–52
 - numerical methods 379–403
 - curve fitting 384–390
 - differentiation 394–397
 - examples using 397–403
 - integration 390–394
 - interpolation 380–384
 - O**
 - object code 12
 - object-oriented programming (OOP) 21
 - objects 25
 - octaves 354
 - odd() function 205
 - Ohm’s Law 313
 - oil rigs, structural integrity of 371–374
 - ones() function 49, 64, 78, 333, A-5
 - OOP (object-oriented programming) 21
 - opening files 180
 - operating systems (OS) 7–10
 - operations
 - on arrays 66–74
 - frequency domain 364–371
 - on images 323–339
 - on strings 130, 134–137
 - on vectors 53–61
 - operators, precedence of 56
 - OR
 - element-wise (|) 55, 56, 78, A-2
 - short-circuit (||) 55, 56, 78, 91, A-2
 - or() function 105, A-2
 - order, inserting in 222
 - OS, *see* operating systems
 - otherwise keyword 86, 92, 93, 104, A-9
 - output 6
 - P**
 - page buffer 7
 - pages 7
 - palindromes, detecting 206
 - parabolic dish, simple 264–266
 - paradigms, programming 20–21
 - parameters
 - formal vs. actual 115
 - value 131
 - <parameters> section 113
 - parametric plots
 - 2-D 256–257
 - 3-D 260–261
 - parentheses (()) 77, 121, A-1
 - “passing by value” 115
 - pause() function A-8
 - PCs (personal computers) 8

- peaks() function 288, A-6
- percent sign (%) 35, 132, A-1, A-3
- personal computers (PCs) 8
- physics
 - of sound 348
 - vectors used in 61–62
- pi A-4
- pie() function 257, 287, A-5
- pie3() function 261, 276, 288, A-6
- pixels 320
- plaid surfaces 262
- playback 348–349
- plot appearance indicators A-6
- plot() function 248, 252, 258, 287, A-5
- plot3() function 288, A-6
- plots (plotting) 247–286
 - 2-D 252–258
 - 3-D 258–261
 - example using 283–286
 - figures as containers for 248
 - functions for enhancing 248–249
 - interactions with plotted data 279–282
 - linking, to source data 280–282
 - manual editing of 250
 - style tips for 250, 256
 - sub- 249–251
 - surface plots 261–279
- polar() function 258, 287, A-5
- polyfit() function 387, 404, 432, A-8
- polynomial regression 387–388
- polyval() function 387, 404, A-8
- problem solving 13–14, 219–243
 - assembling solution steps for 220
 - basic plan for 220, 221
 - building a collection 226–229
 - example 238–243
 - filtering a collection 230–232
 - inserting into a collection 221–223
 - larger problems 236–237
 - mapping a collection 229–230
 - searching a collection 234–235
 - sorting a collection 235
 - summarizing a collection 232–234
 - traversing a collection 223–226
- procedural abstraction 19, 112, 294
- procedural programming 20
- prod() function A-5
- program bugs 12
- programming 219
- programming languages 11–12, 19–21
- pseudo-random values 138
- Q
- %q (format control character) A-3
- quick sort 419–421, 429
- quick sort in place 421–422
- quit command A-3
- quote mark (') 23, A-1
- R
- RAM (Random-Access Memory) 7
- rand() function 49, 64, 78, 138, A-5
- randn() function 49, 78, A-5
- Random-Access Memory (RAM) 7
- reading (text files) 180–183
- Read-Only Memory (ROM) 7
- recording 348–349
- recording rate 349
- recursion 191–215
 - about 193–194
 - and activation stack 192–193
 - endless 193
 - examples of 205–215
 - generative 205
 - in MATLAB 194–196
 - mutual 205
 - requirements for 194
 - tail 203–204
- rem() function A-4
- removing elements (of arrays) 66
- reshape() function 70, 305, 314
- reshaping (of arrays) 70
- resolution
 - image 320
 - of recorded data 349
- <return info> section 113
- RGB (red, green, blue) 320, 322
- rmfield() function 154, 161, A-10
- robot arm motion 211–215
- rocket data, analyzing 397–402
- ROM (Read-Only Memory) 7
- rot90() function 334, 343
- rotations, *see also* bodies of rotation
 - 2-D 300–304
 - 3-D 304–307
- round() function 57, 78, 324, A-4
- row vectors 64
- rows() function 68
- runtime errors 12
- Rutan, Burt 39
- S
- %s (format control character) A-3
- sanity checks 62
- save() function 186, A-2, A-3
- saving scripts 35, 36
- scalar multiplication, matrix vs. 296
- scalar vectors 53
- scale, playing a musical 354–357
- scripts 13, 35–41
 - creating 35–36
 - debugging 38
 - example using 39–41
 - punctuating 37
 - running 37
 - saving 35, 36
- searching (operation) 221, 234–235
- security issues 27
- semicolon (;) 30, 32, 37, 64, 77, A-1
- semilogx() function 258, 287, A-5
- semilogy() function 258, 287, A-5
- setfield() function 160, 167, A-10
- shading() function 249, 287, A-7
- short-circuit AND (&&) 55, 56, 78, 91, A-2
- short-circuit evaluation 91–92
- short-circuit OR (∥) 55, 56, 78, 91, A-2
- shortening (of vectors) 52–53
- shrinking images 324–325
- Simpson's rule 391, 392
- simultaneous linear equations, solving 307–311
- sin() function A-4
- size() function 50, 60, 63, 78, A-5
- slicing
 - of arrays 69
 - of sounds 350–353
 - of strings 130
 - of vectors 58–59
- software, computer 5, 8–12
- soil volume, computation of 74–76
- solid object, measurement of 119–120
- solid-state memory 6–7
- sort() function 427, 432, A-5

- sorting 221, 235, 412–432
 - applications 426–429
 - bubble sort 417–419, 429
 - bucket sort 424–425, 429
 - example using 430–432
 - insertion sort 415–417, 429
 - MATLAB's internal sort algorithm 427–428
 - and measuring algorithm cost 412–415
 - merge sort 422–424, 429
 - quick sort 419–421, 429
 - quick sort in place 421–422
- sound(s) 347–374
 - adding, to spectrum 366–368
 - analog vs. digital 348
 - analyzing instrument 364–366
 - engineering example 371–374
 - file formats for 349–350
 - frequency domain operations 364–371
 - musical 353–359
 - physics of 348
 - recording/playback of 348–349
 - slicing/concatenating 350–353
 - time domain operations on 350–359
 - using Fourier transform with 359–363
- sound() function 349–350, 374
- source code 12
- source data, linking plots to 280–282
- Space Ship One 39
- spacecraft launch (example) 39–41
- special-purpose processors 8
- spectrum(--a)
 - adding sounds to 366–368
 - analysis of 361–363
 - discontinuities in 370
 - manipulation of 368–371
- sphere, constructing a 271
- sphere() function 277, 288, A-6
- spline() function 383, 384, 404, A-8
- spreadsheet data (example) 184–186
- spreadsheets 10
- spreadsheets, Excel 177–178
- sprintf() function 131, 143, A-8
- sqrt() function 36, A-4
- square arrays 64
- sscanf() function 133, 134, 143, A-8
- stack 8, 192–193
- stack frame 193
- Start button 25, 26, 34
- starting MATLAB 21
- states 13
- stopping MATLAB 21
- storage types C-2
- strcmp() function 143, A-8
- strcmpi() function 143, A-8
- stretching images 324–325
- strings, *see* character strings
- strong typing 24
- strtok() function 181, 187, A-2
- struct() function 155, 157, 167, A-10
- structure(s) 153–156
 - assembling a (example) 162–168
 - constructing/accessing 153–155, 157–160
 - functions of 155–156
 - manipulation of 160–162
- structure arrays 156–157, 222
- subplot() function 249, 287, A-7
- sum() function 57, 68, 78, 161, 386, A-5
- summarizing (operation) 221, 232–234
- supercomputers 8
- surf() function 262, 288, 336, A-6
- surface, images on a 336–339
- surface plots 261–279
 - basic functions for creating 262
 - bodies of rotation 272–276
 - compound surfaces, assembly of 277–279
 - cube 262–264
 - manipulation of 266–268
 - parabolic dish 264–266
 - parametric surfaces 3-D 268–271
 - special-purpose plots 276–277
- surfsc() function 277, 288, A-6
- surfz() function 277
- switch keyword 86, 104
- switch statements 92–94, A-9
 - general template for 92
 - MATLAB implementation using 93–94
- syntax 11
- T
- \t (format control character) A-3
- TAB (file extension) 176
- tail recursion 203–204
- tan() function A-4
- technology, advances in 2
- templates, design, *see* design templates
- text files
 - delimited 175, 178–179
 - reading 180–183
 - writing 183
- text() function 249, 287, A-7
- textscan() function 176, 181, 187, 228–229, A-3, A-8
- tic() function A-8
- title() function 249, 287, A-7
- toc() function A-8
- tokens 129
- trapezoidal rule 391–392
- traversing (operation) 221, 223–226
- tril() function 334, 343
- true color images 321
- true values 52, 78, 86–87, 90, A-4
- try keyword 198, 199, 215, A-9
- tune, playing a simple 355–359
- twinkling stars, simulating 302–304
- type, data 25
- typed languages 24
- typographical errors 24
- U
- uint8/16/32/64 storage type 130, 142, 320, 331, 343, A-9, C-2
- unary minus (-) 54, 77, A-1
- unary not (~) 56, 78, A-2
- unary operators 56
- underscore character (_) 22
- untyped languages 23
- user interface 25–34
 - Command History window 27–28
 - Command window 26–27
 - Current Directory window 31–32
 - Editor window 33, 34
 - Graphics window 33
 - Variable Editor window 32–33
 - Workspace window 28–31
- utilities 9
- V
- value(s) 25
 - assigning, to variables 22–23
 - pseudo-random 138

- value parameters 131
 - variable(s)
 - assigning values to 22–23
 - global 118–119
 - unnamed 32
 - (variable) 167, A-1
 - Variable Editor window 32–33
 - variable scoping 117
 - vector(s) 48–62
 - arithmetic operations with 54–55
 - column 64
 - concatenation of 57–58
 - creating 29, 49–50
 - elements of 49
 - empty 49
 - example using 61–62
 - extracting/sorting 427
 - indexing of 50–52
 - of indices 58
 - inserting data into 221
 - library functions with 57
 - logical operations with 55–56
 - operating on 53–61
 - row 64
 - scalar 53
 - shortening of 52–53
 - size of 50
 - slicing of 58–59
 - view() function 249, 287
 - virtual memory 7
 - von Neumann, John 4
 - von Neumann architecture 4–5
- ## W
- waterfall() function 277, 288, A-6
 - .wav files 349–350
 - wavread() function 176, 349, 374, A-3
 - wavwrite() function 176, 374, A-3
 - weak typing 24
 - while loops 98–101, A-9
 - breaking 101
 - general template for 98–99
 - for loops vs. 94
 - MATLAB implementations using 99–101
 - who command 30, 31, A-3
 - whole note steps 354
 - whos command 31, A-3
 - WK1(file extension) 176
 - wklread() function 176
 - wklwrite() function 176
 - WMV (file extension) 176
 - word processors 10
 - workspace, saving your 174–175
 - Workspace window 28–31
 - wrapper functions 191, 200–203
 - writing (text files) 183
- ## X
- xlabel() function 249, 287, A-7
 - XLS (file extension) 176
 - xlsread() function 176–178, 184, 187, A-3
 - xlswrite() function 176, 178, 187, A-3
 - XML (file extension) 176
- ## Y
- ylabel() function 249, 287, A-7
- ## Z
- zeroes of a function 208–211
 - zeros() function 49, 64, 78, A-5
 - zlabel() function 249, 287, A-7